

Unifideck

Operational plan for integrating the new 5-layer architecture

Version: 1.3 Definitive Edition — May 2026
Author: HardCPP (src893)
Reference: unifideck_architecture_technical_documentation.pdf

Plan metrics

Metric	Value
Total fiches	459 fiches across 4 phases
Backend (Python)	350 files 62 798 lines
Frontend (TS/TSX)	82 files 5 456 lines
Base subpackage (config + entry)	8 files 525 lines
Core modules (Backend)	22 files 3 583 lines
Store connectors (Backend)	106 files 24 299 lines
Services (Backend)	75 files 7 794 lines
Refactored support modules	156 files 27 122 lines
CI/CD workflows	8 files 529 lines
Scripting helpers	9 files 1 523 lines
Total volume delivered	73 620 lines

Revision history

Version	Date	Description	Author
1.3	May 2026	Add new frontend ported from the production version.	HardCPP
1.2	April 2026	Cherry Picking with all backend modules, classes, methods... pre-implemented	HardCPP
1.1	April 2026	Complete refactoring of the store plugins + new automatic translation + updated configuration file.	HardCPP
1.0	April 2026	Full document regenerated from scratch: 4 phases (package contracts, backend, frontend + CI/CD), stubs completed.	HardCPP

Table of contents

	— base subpackage	
OP-01	↳ __init__.py	p. 13
OP-01a	↳ requirements.txt	p. 14
OP-01b	↳ requirements-dev.txt	p. 15
OP-01c	↳ pyproject.toml	p. 16
OP-01d	↳ tsconfig.json	p. 20
OP-01e	↳ vitest.config.ts	p. 22
OP-01f	↳ package.json	p. 24
OP-01g	↳ main.py	p. 27
	— defaults subpackage	
OP-02a	↳ config.json	p. 29
	— bin subpackage	
OP-03a	↳ unifideck-launcher	p. 35

Phase 1 — Backend

OP-04	— core subpackage	p. 38
OP-05	— types subpackage	p. 39
OP-05a	↳ events.py	p. 40
OP-05b	↳ results.py	p. 42
OP-05c	↳ domain.py	p. 45
OP-06	— io subpackage	p. 46
OP-06a	↳ async_file_ops.py	p. 47
OP-06b	↳ safe_file_op.py	p. 51
OP-07	— binaries subpackage	p. 52
OP-07a	↳ binary_resolver.py	p. 53
OP-07b	↳ binary_signatures.py	p. 55
OP-07c	↳ cli_timeouts.py	p. 57
OP-08	— net subpackage	p. 58
OP-08a	↳ ssl_helpers.py	p. 59
OP-04a	↳ cache_manager.py	p. 60
OP-04b	↳ metrics_collector.py	p. 63
OP-04c	↳ exe_finder.py	p. 65
OP-04d	↳ sync_service.py	p. 67
OP-04d2	↳ cross_store_dedup.py	p. 72
OP-04e	↳ manifest.py	p. 75
OP-04f	↳ paths.py	p. 79
OP-09	— eventbus subpackage	p. 80
OP-10	— supervision subpackage	p. 81
OP-10a	↳ watchdog_handler.py	p. 82
OP-10b	↳ metrics_handler.py	p. 85
OP-09a	↳ event_bus.py	p. 87
OP-09b	↳ event_priority.py	p. 89
OP-09c	↳ priority_dispatcher.py	p. 91
OP-09d	↳ event_replay.py	p. 96
OP-09e	↳ event_bus_extensions.py	p. 98
OP-09f	↳ event_bus_reliability.py	p. 102

OP-09g	↳ event_bus_scaling.py	p. 103
OP-09h	↳ event_bus_devex.py	p. 104
OP-09i	↳ bus_pipeline.py	p. 108
OP-11	— config subpackage	p. 109
OP-11a	↳ config_manager.py	p. 110
OP-11b	↳ config_persistence.py	p. 114
OP-11c	↳ i18n_schema.py	p. 115
OP-11e	↳ validator.py	p. 117
OP-11f	↳ key_presence.py	p. 122
OP-11g	↳ startup.py	p. 125
OP-11h	↳ user_config_path.py	p. 127
OP-12	— services subpackage	p. 128
OP-13	— bootstrap subpackage	p. 129
OP-13a	↳ paths.py	p. 130
OP-13b	↳ container.py	p. 132
OP-13c	↳ service_defs.py	p. 133
OP-13d	↳ constructor.py	p. 136
OP-13e	↳ startup.py	p. 138
OP-13f	↳ teardown.py	p. 139
OP-13g	↳ store_injector.py	p. 140
OP-14	— shortcut subpackage	p. 141
OP-14a	↳ service.py	p. 142
OP-14b	↳ events.py	p. 144
OP-14c	↳ games_map.py	p. 145
OP-14d	↳ shortcut.py	p. 146
OP-14e	↳ games_map_mixin.py	p. 148
OP-14f	↳ vdf_shortcuts.py	p. 151
OP-14g	↳ persistence.py	p. 152
OP-15	— download subpackage	p. 153
OP-15a	↳ service.py	p. 154
OP-15b	↳ worker.py	p. 156
OP-15c	↳ models.py	p. 158
OP-15d	↳ validators.py	p. 159
OP-15e	↳ persistence.py	p. 160
OP-16	— artwork subpackage	p. 161
OP-16a	↳ service.py	p. 162
OP-16b	↳ event_handlers.py	p. 164
OP-16c	↳ fetcher.py	p. 166
OP-17	— cloud_save subpackage	p. 168
OP-17a	↳ service.py	p. 169
OP-17b	↳ sync.py	p. 171
OP-17c	↳ manifest.py	p. 173
OP-17d	↳ fs_ops.py	p. 174
OP-17e	↳ paths.py	p. 176
OP-17f	↳ constants.py	p. 177
OP-18	— playtime subpackage	p. 178
OP-18a	↳ service.py	p. 179
OP-18b	↳ db.py	p. 182
OP-19	— security subpackage	p. 184
OP-19a	↳ service.py	p. 185
OP-19b	↳ audit_log.py	p. 188

OP-19c	↳ bruteforce.py	p. 189
OP-19d	↳ config_readers.py	p. 191
OP-19e	↳ bus_emitter.py	p. 192
OP-19f	↳ device_reset.py	p. 193
OP-19g	↳ tokens.py	p. 195
OP-19h	↳ permissions.py	p. 196
OP-19i	↳ auth.py	p. 197
OP-19j	↳ config.py	p. 198
OP-20	— launcher subpackage	p. 199
OP-20a	↳ service.py	p. 200
OP-20b	↳ circuit_breaker.py	p. 205
OP-20c	↳ error_toasts.py	p. 207
OP-20d	↳ orchestrator.py	p. 209
OP-20e	↳ helpers.py	p. 211
OP-20f	↳ builder.py	p. 215
OP-21	— launch_history subpackage	p. 216
OP-21a	↳ service.py	p. 217
OP-21b	↳ failures.py	p. 219
OP-21c	↳ bypass.py	p. 221
OP-21d	↳ config_readers.py	p. 222
OP-21e	↳ persistence.py	p. 223
OP-21f	↳ constants.py	p. 224
OP-22	— microsoft_subscription subpackage	p. 225
OP-22a	↳ service.py	p. 226
OP-22b	↳ cache_mixin.py	p. 228
OP-22c	↳ probe_emission.py	p. 230
OP-22d	↳ event_handlers.py	p. 232
OP-22e	↳ cache.py	p. 233
OP-22f	↳ time_utils.py	p. 234
OP-22g	↳ constants.py	p. 235
OP-12a	↳ metadata_service.py	p. 236
OP-12b	↳ proton_service.py	p. 238
OP-12c	↳ account_service.py	p. 240
OP-12d	↳ feature_flag_service.py	p. 242
OP-12e	↳ probe_reaction_service.py	p. 244
OP-23	— security subpackage	p. 246
OP-23a	↳ device_identity.py	p. 247
OP-23b	↳ secure_token_store.py	p. 249
OP-23c	↳ audit_emitter.py	p. 254
OP-23d	↳ device_fingerprint.py	p. 257
OP-23e	↳ audit_decorators.py	p. 260
OP-24	— rpc subpackage	p. 262
OP-25	— handlers subpackage	p. 263
OP-25a	↳ base.py	p. 264
OP-25b	↳ action.py	p. 265
OP-25c	↳ download.py	p. 267
OP-25d	↳ launch.py	p. 268
OP-25e	↳ observability.py	p. 270
OP-25f	↳ security.py	p. 273
OP-25g	↳ store.py	p. 274
OP-25h	↳ ui.py	p. 276

OP-26	— mixins subpackage	p. 278
OP-26a	↳ observability.py	p. 279
OP-26b	↳ security.py	p. 282
OP-26c	↳ download.py	p. 283
OP-26d	↳ launch.py	p. 284
OP-26e	↳ store.py	p. 286
OP-26f	↳ sync.py	p. 287
OP-26g	↳ ui.py	p. 288
OP-26h	↳ cloud_failure.py	p. 289
OP-26i	↳ config_validation.py	p. 290
OP-26j	↳ playtime.py	p. 291
OP-26k	↳ action.py	p. 292
OP-24a	↳ errors.py	p. 293
OP-24b	↳ wrapper.py	p. 294
OP-24c	↳ auto_wire.py	p. 296
OP-24d	↳ composer.py	p. 297
OP-27	— actions subpackage	p. 298
OP-27a	↳ unifideck_uri.py	p. 299
OP-27b	↳ dispatch.py	p. 301
OP-28	— auth subpackage	p. 303
OP-29	— edge_browser subpackage	p. 304
OP-29a	↳ edge.py	p. 305
OP-29b	↳ env.py	p. 308
OP-29c	↳ launch.py	p. 311
OP-29d	↳ detection.py	p. 313
OP-29e	↳ installer.py	p. 315
OP-29f	↳ cdp_client.py	p. 320
OP-29g	↳ profile.py	p. 324
OP-29h	↳ process_ops.py	p. 328
OP-28a	↳ browser.py	p. 330
OP-28b	↳ orchestrator.py	p. 334
OP-30	— cdp subpackage	p. 339
OP-30a	↳ cdp_client.py	p. 340
OP-30b	↳ cdp_inject.py	p. 344
OP-30c	↳ xcloud_browser_shims.py	p. 346
OP-30d	↳ page_inject.py	p. 353
OP-31	— metadata subpackage	p. 357
OP-31a	↳ metacritic.py	p. 358
OP-31b	↳ unificdb.py	p. 362
OP-32	— steam subpackage	p. 365
OP-32a	↳ steamgriddb.py	p. 366
OP-32b	↳ library.py	p. 370
OP-32c	↳ shortcuts.py	p. 373
OP-32d	↳ owned_games.py	p. 376
OP-33	— utils subpackage	p. 378
OP-33a	↳ paths.py	p. 379
OP-33b	↳ locale.py	p. 381
OP-33c	↳ config_helpers.py	p. 384
OP-34	— compatibility subpackage	p. 385
OP-34a	↳ proton_helpers.py	p. 386
OP-34b	↳ library.py	p. 392

OP-35	— launcher subpackage	p. 397
OP-36	— types subpackage	p. 398
OP-36a	↳ exit_codes.py	p. 399
OP-36b	↳ errors.py	p. 400
OP-36c	↳ context.py	p. 402
OP-36d	↳ options.py	p. 404
OP-37	— flows subpackage	p. 406
OP-37a	↳ xcloud.py	p. 407
OP-37b	↳ native.py	p. 409
OP-37c	↳ auth.py	p. 412
OP-38	— cdp subpackage	p. 415
OP-38a	↳ cdp_primitives.py	p. 416
OP-38b	↳ xcloud_cdp.py	p. 418
OP-38c	↳ steam_controller_popup.py	p. 421
OP-38d	↳ steam_controller_popup_targets.py	p. 425
OP-38e	↳ steam_controller_popup_fiber.py	p. 427
OP-39	— cloud subpackage	p. 433
OP-39a	↳ cloud_failure.py	p. 434
OP-39b	↳ disk_space.py	p. 438
OP-39c	↳ save_size_cache.py	p. 440
OP-40	— diagnostics subpackage	p. 443
OP-40a	↳ correlation.py	p. 444
OP-40b	↳ telemetry.py	p. 445
OP-40c	↳ save_folder_inspector.py	p. 446
OP-40d	↳ log_archive.py	p. 448
OP-41	— proton subpackage	p. 451
OP-42	— infrastructure subpackage	p. 452
OP-42a	↳ core.py	p. 453
OP-42b	↳ umu_runtime.py	p. 456
OP-42c	↳ selector.py	p. 458
OP-42d	↳ prefix_layout.py	p. 461
OP-43	— handlers subpackage	p. 462
OP-43a	↳ ubisoft.py	p. 463
OP-43b	↳ epic.py	p. 467
OP-43c	↳ generic.py	p. 470
OP-43d	↳ gog_linux_dosbox.py	p. 473
OP-44	— fixes subpackage	p. 476
OP-44a	↳ game_fixes.py	p. 477
OP-44b	↳ galaxy_stub.py	p. 480
OP-44c	↳ epic_prefix_fix.py	p. 482
OP-44d	↳ epic_registry.py	p. 486
OP-44e	↳ auth_args_stripper.py	p. 491
OP-45	— language_setup subpackage	p. 492
OP-45a	↳ resolver.py	p. 493
OP-45b	↳ matchers.py	p. 495
OP-45c	↳ registry_io.py	p. 496
OP-45d	↳ amazon.py	p. 498
OP-45e	↳ gog.py	p. 499
OP-45f	↳ ubisoft.py	p. 501
OP-35a	↳ signals.py	p. 503
OP-35b	↳ rpc.py	p. 505

OP-35c	↳ dispatcher.py	p. 507
OP-35d	↳ packaging.py	p. 511
OP-35e	↳ bootstrap.py	p. 513
OP-46	— stores subpackage	p. 515
OP-47	— shared subpackage	p. 516
OP-47a	↳ store_registry.py	p. 517
OP-47b	↳ store_base.py	p. 523
OP-47c	↳ dlc.py	p. 526
OP-47d	↳ cli_install_helpers.py	p. 527
OP-48	— EpicStore subpackage	p. 528
OP-48a	↳ store.py	p. 529
OP-48b	↳ auth.py	p. 534
OP-48c	↳ library.py	p. 538
OP-48d	↳ install.py	p. 541
OP-48e	↳ updates.py	p. 545
OP-48f	↳ filter.py	p. 548
OP-48g	↳ exe_resolver.py	p. 550
OP-48h	↳ legendary.py	p. 552
OP-49	— AmazonStore subpackage	p. 553
OP-49a	↳ amazon_store.py	p. 554
OP-49b	↳ amazon_auth.py	p. 559
OP-49c	↳ amazon_library.py	p. 563
OP-49d	↳ amazon_install.py	p. 566
OP-49e	↳ amazon_updates.py	p. 571
OP-49f	↳ amazon_fuel.py	p. 573
OP-50	— GOGStore subpackage	p. 575
OP-51	— install subpackage	p. 576
OP-51a	↳ installer.py	p. 577
OP-51b	↳ primitives.py	p. 584
OP-51c	↳ languages.py	p. 586
OP-51d	↳ planner.py	p. 587
OP-51e	↳ uninstall_pipeline.py	p. 593
OP-51f	↳ progress.py	p. 595
OP-51g	↳ marker.py	p. 601
OP-51h	↳ helpers.py	p. 605
OP-52	— tokens subpackage	p. 608
OP-52a	↳ manager.py	p. 609
OP-52b	↳ storage.py	p. 612
OP-52c	↳ oauth.py	p. 616
OP-52d	↳ gogdl_credentials.py	p. 618
OP-52e	↳ user_info.py	p. 620
OP-50a	↳ store.py	p. 621
OP-50b	↳ config.py	p. 627
OP-50c	↳ library.py	p. 630
OP-50d	↳ library_migration.py	p. 636
OP-50e	↳ exe_resolver.py	p. 639
OP-50f	↳ dlc.py	p. 645
OP-50g	↳ updates.py	p. 652
OP-50h	↳ auth.py	p. 657
OP-50i	↳ http.py	p. 659
OP-53	— MicrosoftStore subpackage	p. 660

OP-54	— tokens subpackage	p. 661
OP-54a	↳ manager.py	p. 662
OP-54b	↳ persistence.py	p. 663
OP-54c	↳ oauth.py	p. 667
OP-54d	↳ xbl_chain.py	p. 669
OP-53a	↳ microsoft_store.py	p. 672
OP-53b	↳ microsoft_catalog.py	p. 678
OP-53c	↳ microsoft_browser_auth.py	p. 682
OP-53d	↳ microsoft_config.py	p. 684
OP-53e	↳ microsoft_auth.py	p. 687
OP-53f	↳ microsoft_subscription.py	p. 691
OP-55	— UbisoftStore subpackage	p. 694
OP-56	— installer subpackage	p. 695
OP-56a	↳ installer.py	p. 696
OP-56b	↳ cache.py	p. 701
OP-56c	↳ launch_env.py	p. 703
OP-56d	↳ launcher.py	p. 704
OP-56e	↳ manual_ui.py	p. 707
OP-56f	↳ registry.py	p. 713
OP-56g	↳ uninstall.py	p. 718
OP-56h	↳ update_op.py	p. 724
OP-57	— library subpackage	p. 726
OP-57a	↳ facade.py	p. 727
OP-57b	↳ fetch.py	p. 729
OP-57c	↳ data_loader.py	p. 731
OP-57d	↳ game_builder.py	p. 733
OP-57e	↳ manifest.py	p. 737
OP-57f	↳ detection.py	p. 743
OP-57g	↳ detection_cascade.py	p. 748
OP-57h	↳ detection_helpers.py	p. 754
OP-57i	↳ wine_path.py	p. 759
OP-58	— auth subpackage	p. 760
OP-58a	↳ facade.py	p. 761
OP-58b	↳ context.py	p. 764
OP-58c	↳ shortcut.py	p. 767
OP-58d	↳ session_monitor.py	p. 774
OP-58e	↳ direct_signin.py	p. 776
OP-58f	↳ shortcut_ops.py	p. 779
OP-59	— prefix subpackage	p. 781
OP-59a	↳ manager.py	p. 782
OP-59b	↳ helpers.py	p. 785
OP-59c	↳ template_builder.py	p. 791
OP-59d	↳ auth_builder.py	p. 794
OP-60	— session subpackage	p. 798
OP-60a	↳ facade.py	p. 799
OP-60b	↳ payload.py	p. 802
OP-60c	↳ reader.py	p. 807
OP-60d	↳ propagator.py	p. 810
OP-55a	↳ store.py	p. 813
OP-55b	↳ config.py	p. 817
OP-55c	↳ paths.py	p. 823

OP-55d	↳ binaries.py	p. 825
OP-55e	↳ parser.py	p. 831
OP-55f	↳ parser_binary.py	p. 837
OP-55g	↳ id_map.py	p. 839
OP-55h	↳ id_map_sources.py	p. 843
OP-55i	↳ steam_filter.py	p. 849
OP-55j	↳ specialists.py	p. 851
OP-61	— bootstrap subpackage	p. 854
OP-61a	↳ cache_registry.py	p. 855
OP-61b	↳ teardown.py	p. 856
OP-61c	↳ pipeline_factory.py	p. 857
OP-61d	↳ boot.py	p. 858

Phase 2 — Frontend

OP-62	— types subpackage	p. 861
OP-62a	↳ api.ts	p. 862
OP-62b	↳ events.ts	p. 865
OP-62c	↳ store.ts	p. 867
OP-62d	↳ steam.ts	p. 868
OP-62e	↳ downloads.ts	p. 872
OP-62f	↳ playtime.ts	p. 874
OP-62g	↳ syncProgress.ts	p. 876
OP-63	— steam-bridge subpackage	p. 877
OP-63a	↳ SteamBridge.ts	p. 878
OP-63b	↳ react-tree.ts	p. 881
OP-63c	↳ app-details-classes.ts	p. 883
OP-63d	↳ router-patch.ts	p. 885
OP-63e	↳ shortcut-types.ts	p. 886
OP-64	— api subpackage	p. 888
OP-64a	↳ rpc-routes.ts	p. 889
OP-64b	↳ useRPC.ts	p. 891
OP-64c	↳ rpc-errors.ts	p. 894
OP-64d	↳ event-bus-client.ts	p. 896
OP-65	— contexts subpackage	p. 900
OP-65a	↳ StoreContext.tsx	p. 901
OP-65b	↳ SyncContext.tsx	p. 903
OP-65c	↳ AuthContext.tsx	p. 906
OP-65d	↳ DownloadContext.tsx	p. 909
OP-65e	↳ LocaleContext.tsx	p. 912
OP-65f	↳ RootProvider.tsx	p. 914
OP-66	— hooks subpackage	p. 915
OP-66a	↳ useStoreAuth.ts	p. 916
OP-66b	↳ useGameActions.ts	p. 918
OP-66c	↳ useGameInfo.ts	p. 920
OP-66d	↳ useViewMode.ts	p. 923
OP-66e	↳ usePlaySection.ts	p. 925
OP-66f	↳ useHidePlaySection.ts	p. 927
OP-66g	↳ useDownloadProgress.ts	p. 929
OP-66h	↳ useToast.ts	p. 931
OP-66i	↳ useSteamLibrary.ts	p. 933
OP-67	— components subpackage	p. 936
OP-68	— play subpackage	p. 937
OP-68a	↳ PlaySectionWrapper.tsx	p. 938
OP-68b	↳ NotInstalledButtons.tsx	p. 940
OP-68c	↳ DownloadingButtons.tsx	p. 942
OP-68d	↳ InstalledButtons.tsx	p. 944
OP-69	— info subpackage	p. 946
OP-69a	↳ GameInfoPanel.tsx	p. 947
OP-69b	↳ GameInfoHeader.tsx	p. 949
OP-69c	↳ GameInfoMetadata.tsx	p. 951
OP-69d	↳ GameInfoScores.tsx	p. 953
OP-70	— settings subpackage	p. 955

OP-70a	↳ StorageSettings.tsx	p. 956
OP-70b	↳ StoragePathPicker.tsx	p. 958
OP-70c	↳ StoreConnections.tsx	p. 960
OP-70d	↳ LibrarySync.tsx	p. 962
OP-70e	↳ LanguageSelector.tsx	p. 964
OP-71	— modals subpackage	p. 965
OP-71a	↳ AccountSwitchModal.tsx	p. 966
OP-71b	↳ SteamRestartModal.tsx	p. 968
OP-71c	↳ UninstallConfirmModal.tsx	p. 969
OP-71d	↳ CloudSaveConflictModal.tsx	p. 970
OP-71e	↳ ToastEventListener.tsx	p. 972
OP-72	— downloads subpackage	p. 974
OP-72a	↳ DownloadsTab.tsx	p. 975
OP-72b	↳ DownloadItemRow.tsx	p. 977
OP-73	— shared subpackage	p. 979
OP-73a	↳ StoreIcon.tsx	p. 980
OP-73b	↳ GameGrid.tsx	p. 982
OP-74	— services subpackage	p. 984
OP-75	— auth subpackage	p. 985
OP-75a	↳ AuthDispatcher.ts	p. 986
OP-76	— views subpackage	p. 989
OP-76a	↳ QuickAccessPanel.tsx	p. 990
OP-76b	↳ AppDetailsPatch.tsx	p. 992
OP-77	↳ bootstrap-tasks.ts	p. 994
OP-78	↳ teardown.ts	p. 997
OP-79	↳ index.tsx	p. 998
OP-80	— i18n subpackage	p. 1000
OP-80a	↳ translations.ts	p. 1001
OP-80b	↳ en-US.json	p. 1004
OP-81	— utils subpackage	p. 1006
OP-81a	↳ format.ts	p. 1007
OP-81b	↳ controllerConfig.ts	p. 1008
OP-81c	↳ authShortcutLaunch.ts	p. 1010
OP-81d	↳ ubisoftShortcutLaunch.ts	p. 1019

Phase 3 — CI/CD

OP-82	↳ build-plugin.yml	p. 1025
OP-83	↳ translations.yml	p. 1027
OP-84	↳ complexity.yml	p. 1029
OP-85	↳ .github/workflows/quality.yml	p. 1031
OP-86	↳ .github/workflows/security.yml	p. 1033
OP-87	↳ .github/workflows/tests.yml	p. 1034

Phase 4 — Scripting

OP-90	↳ locale_config.py	p. 1036
OP-91	↳ gen_locale_imports.py	p. 1041
OP-92	↳ lint_rtl.py	p. 1044
OP-93	↳ translate_at_build.py	p. 1047
OP-94	↳ validate_event_schemas.py	p. 1052
OP-95	↳ ensure_executable_bits.py	p. 1054
OP-97	↳ check_config_keys.py	p. 1055
OP-98	↳ volumetry_check.py	p. 1058

OP-01

__init__.py

Fichier : py_modules/unifideck/__init__.py | Lignes : 2 | Depend de : (rien)

```
from __future__ import annotations  
  
__version__ = "0.7.0"
```

OP-01a

requirements.txt

Fichier : requirements.txt | Lignes : 4 | Depend de : (rien)

```
aiohttp>=3.9,<4.0
websockets>=12.0,<14.0
cryptography>=42.0,<47.0
jsonschema>=4.0,<5.0
```

OP-01b

requirements-dev.txt

Fichier : requirements-dev.txt | Lignes : 9 | Depend de : OP-01a

```
-r requirements.txt
ruff>=0.5,<1.0
mypy>=1.10,<2.0
import-linter>=2.0,<3.0
pip-audit>=2.7,<3.0
pytest>=8.0,<9.0
pytest-asyncio>=0.23,<1.0
pytest-cov>=5.0,<7.0
PyYAML>=6.0,<7.0
```

OP-01c

pyproject.toml

Fichier : pyproject.toml | Lignes : 176 | Depend de : OP-01a

```
[project]
name = "unifideck"
version = "0.7.0"
description = "Unified Steam Deck game library aggregator – Decky Loader plugin"
requires-python = ">=3.11"
authors = [{ name = "HardCPP (src893)" }]

[tool.ruff]
line-length = 100
target-version = "py311"
extend-exclude = [
    "fiche_sources",
    "build",
    "dist",
    "node_modules",
    "__pycache__",
]

[tool.ruff.lint]
select = [
    "E", "W",
    "F",
    "I",
    "B",
    "C4",
    "UP",
    "SIM",
    "RET",
    "BLE",
    "G",
    "PLC", "PLE",
    "PLR0911",
    "PLR0912",
    "PLR0913",
    "PLR0915",
    "PLR0917",
    "N",
    "ASYNC",
    "S",
    "PTH",
    "TID",
    "PGH",
    "D",
]
ignore = [
    "E501",
    "S101",
    "S603",
    "S607",
    "PLC0415",
    "D105",
    "D203",
    "D213",
    "D401",
    "D406", "D407",
```



```

    "D413",
    "D205",
    "D208",
    "D301",
    "D400",
    "D415",
    "D417",
    "N805",
    "TID252",
    "RET503",
    "SIM108",
    "SIM105",
    "PTH",
    "ASYNC240",
    "ASYNC109",
]

[tool.ruff.lint.per-file-ignores]
"__init__.py" = ["F401"]
"tests/**/*.py" = ["S101", "ANN", "PLR2004"]
"main.py" = [
    "PLR0915",
    "E402",
]
"py_modules/unifideck/core/types/**" = ["N805", "A"]

[tool.ruff.lint.isort]
known-first-party = ["unifideck"]
combine-as-imports = true

[tool.ruff.lint.flake8-bugbear]
extend-immutable-calls = ["Events"]

[tool.ruff.lint.pylint]
max-args = 7
max-positional-args = 5

[tool.ruff.format]
quote-style = "double"
indent-style = "space"

[tool.mypy]
python_version = "3.11"
strict_optional = true
warn_redundant_casts = true
warn_unused_ignores = true
warn_return_any = true
check_untyped_defs = true
no_implicit_optional = true
explicit_package_bases = true
mypy_path = "py_modules"
exclude = "(fiche_sources|build|dist|node_modules)/"

[[tool.mypy.overrides]]
module = [
    "unifideck.core.*",
    "unifideck.event_bus.*",
]

[[tool.mypy.overrides]]
module = [

```

```

    "decky",
    "decky_plugin",
    "vdf",
    "websockets.*",
    "aiohttp",
    "aiohttp.*",
    "jsonschema",
    "jsonschema.*",
    "locale_config",
    "yaml",
]
ignore_missing_imports = true

[[tool.mypy.overrides]]
module = [
    "unifideck.accounts.account_manager",
    "unifideck.cloud.cloud_save",
    "unifideck.download.manager",
    "unifideck.playtime.database",
    "unifideck.playtime.models",
    "unifideck.registry.games_registry",
    "unifideck.steam.steam_utils",
    "unifideck.cdp.cdp_utils",
    "unifideck.stores.ubisoft_parser",
    "unifideck.stores.ubisoft.ubisoft_binary_resolver",
]
ignore_missing_imports = true

[tool.pytest.ini_options]
minversion = "8.0"
testpaths = ["tests"]
python_files = ["test_*.py"]
python_classes = ["Test*"]
python_functions = ["test_*"]
asyncio_mode = "auto"
addopts = [
    "-ra",
    "--strict-markers",
    "--strict-config",
    "-q",
]
markers = [
    "unit: fast isolated tests (<3s)",
    "integration: component interactions with mocks (<30s)",
    "e2e: full flow on Steam Deck (minutes)",
    "slow: deselect with '-m \"not slow\"'",
]

[tool.coverage.run]
source = ["py_modules/unifideck", "main"]
branch = true
omit = [
    "*/__init__.py",
    "*/tests/*",
]

[tool.coverage.report]
fail_under = 40
show_missing = true
skip_covered = false
exclude_lines = [

```

```
    "pragma: no cover",
    "raise NotImplementedError",
    "if TYPE_CHECKING:",
    "if __name__ == '__main__.':",
]

[tool.bandit]
exclude_dirs = ["fiche_sources", "tests", "build", "dist"]
skips = [
    "B101",
    "B404",
    "B603",
    "B607",
]
```

OP-01d

tsconfig.json

Fichier : tsconfig.json | Lignes : 60 | Depend de : (rien)

```
/* TypeScript compiler config for the Unifideck frontend.
 *
 * Consumed by :
 * - rollup via @rollup/plugin-typescript (compile to ESM)
 * - eslint via @typescript-eslint/parser
 * - vitest via its esbuild loader (tests + coverage)
 * - `tsc --noEmit` from the typecheck npm script
 *
 * `strict` is enforced because the new architecture lifts
 * untyped Steam internals into the SteamBridge facade
 * (OP-F02a) – strict checking guarantees the bridge is the
 * only place where `unknown` boundary casts happen.
 *
 * `paths` aliases the `src/*` tree under a stable `@/*`
 * prefix so deep imports (e.g. components/play/...) stay
 * readable. The alias is mirrored in vitest.config.ts and
 * in the rollup TypeScript plugin via the `paths` option.
 */
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "jsx": "react-jsx",
    "outDir": "./dist",
    "rootDir": ".",
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    },
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": false,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "forceConsistentCasingInFileNames": true,
    "isolatedModules": true,
    "resolveJsonModule": true,
    "skipLibCheck": true,
    "declaration": false,
    "sourceMap": true,
    "removeComments": false
  },
  "include": [
    "src/**/*",
    "src/**/*.json"
  ],
}
```

```
"exclude": [  
  "node_modules",  
  "dist",  
  "build",  
  "**/*.test.ts",  
  "**/*.test.tsx"  
]  
}
```

OP-01e

vitest.config.ts

Fichier : vitest.config.ts | Lignes : 72 | Depend de : OP-01d (tsconfig paths), OP-01f (package.json)

```
/**
 * vitest test runner configuration.
 *
 * Backs the npm scripts declared in package.json (OP-01f) :
 *   test          - single-shot run
 *   test:watch     - watcher mode
 *   test:coverage  - runs with v8 coverage
 *
 * Test environment is jsdom so React Testing Library can
 * mount components against a virtual DOM. The setup file
 * registers @testing-library/jest-dom matchers globally.
 *
 * Coverage thresholds match the Phase 3 scope statement
 * (70 % minimum line coverage). They are advisory only at
 * the start of Phase 3 and become blocking once the suite
 * reaches the target on `main`. The thresholds key can be
 * commented out during the ramp-up if PRs bisect uncovered
 * legacy code that was imported as-is (il8n, format).
 *
 * The `@/*` alias mirrors tsconfig.json (OP-01d) so test
 * files import via the same path style as the runtime code.
 */
import { defineConfig } from "vitest/config";
import path from "node:path";

export default defineConfig({
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
  },
  test: {
    environment: "jsdom",
    globals: true,
    setupFiles: ["./.vitest.setup.ts"],
    include: ["src/**/*.test.ts", "src/**/*.spec.ts", "src/**/*.tsx"],
    exclude: ["node_modules", "dist", "build"],
    coverage: {
      provider: "v8",
      reporter: ["text", "html", "lcov"],
      reportsDirectory: "./coverage",
      include: ["src/**/*.ts", "src/**/*.tsx"],
      exclude: [
        "src/**/*.test.ts", "src/**/*.spec.ts", "src/**/*.tsx",
        "src/**/*.index.ts",
        "src/types/**",
        "src/il8n/locales/**",
        "src/il8n/locales.generated.ts",
      ],
    },
    thresholds: {
      lines: 70,
      functions: 70,
      branches: 65,
      statements: 70,
    },
  },
});
```

```
    },
    // Steam internals (window.SteamClient, window.appStore)
    // are not present in jsdom – every test that touches
    // them MUST mock the SteamBridge facade. The pool option
    // below isolates each spec file in its own worker so
    // mock leakage across suites is impossible.
    pool: "threads",
    poolOptions: {
      threads: {
        isolate: true,
      },
    },
    // Deterministic test order helps when chasing a flake.
    sequence: {
      shuffle: false,
    },
  },
});
```

OP-01f

package.json

Fichier : package.json | Lignes : 129 | Depend de : (rien)

```
{
  "name": "unifideck-decky",
  "version": "0.7.0",
  "description": "Unified game library for Steam Deck – browse, install and launch games from Epic Games, GOG, Amazon, Ubisoft and Microsoft xCloud directly inside Steam's interface. Built on a 5-layer architecture with EventBus, service DI, and 90%+ test coverage.",
  "type": "module",
  "scripts": {
    "build": "rollup -c rollup.config.mjs",
    "watch": "rollup -c -w",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:coverage": "vitest run --coverage",
    "test:backend": "python3 -m pytest py_modules/ --cov=py_modules/unifideck --cov-report=term-missing -q",
    "test:all": "npm run test:coverage && npm run test:backend",
    "lint": "eslint src/ --ext .ts,.tsx",
    "lint:fix": "eslint src/ --ext .ts,.tsx --fix",
    "check:prettier": "prettier --config .prettierrc --check .",
    "fix:prettier": "prettier --config .prettierrc --write .",
    "typecheck": "tsc --noEmit"
  },
  "lint-staged": {
    "*.{js,ts,jsx,tsx}": [
      "eslint --fix",
      "prettier --write"
    ],
    "*.{json,md,yml}": "prettier --write"
  },
  "keywords": [
    "decky",
    "plugin",
    "steamdeck",
    "epic",
    "gog",
    "amazon",
    "ubisoft",
    "microsoft",
    "xcloud",
    "library",
    "multi-store"
  ],
  "author": "HardCPP",
  "license": "GPL-3.0-or-later",
  "devDependencies": {
    "@decky/rollup": "^1.0.2",
    "@rollup/plugin-commonjs": "^29.0.0",
    "@rollup/plugin-json": "^6.1.0",
    "@rollup/plugin-node-resolve": "^16.0.3",
    "@rollup/plugin-replace": "^6.0.3",
    "@rollup/plugin-typescript": "^12.3.0",
    "@testing-library/react": "^16.1.0",
    "@testing-library/jest-dom": "^6.6.3",
    "@types/react": "^19.1.1",
  }
}
```



```

"@types/react-dom": "^19.1.1",
"@types/webpack": "^5.28.5",
"@typescript-eslint/eslint-plugin": "^8.0.0",
"@typescript-eslint/parser": "^8.0.0",
"eslint": "^8.57.0",
"eslint-config-prettier": "^8.5.0",
"eslint-plugin-jsx-ally": "^6.10.0",
"eslint-plugin-prettier": "^4.0.0",
"eslint-plugin-react": "^7.37.0",
"eslint-plugin-react-hooks": "^5.0.0",
"jsdom": "^25.0.0",
"lint-staged": "^16.2.7",
"prettier": "^2.8.8",
"rollup": "^4.53.3",
"typescript": "^5.6.2",
"vitest": "^2.1.0",
"@vitest/coverage-v8": "^2.1.0"
},
"dependencies": {
  "@decky/api": "^1.1.3",
  "@decky/ui": "^4.11.1",
  "il8next": "^25.7.4",
  "react-il8next": "^16.5.3",
  "react-icons": "^5.3.0",
  "tslib": "^2.7.0"
},
"pnpm": {
  "peerDependencyRules": {
    "ignoreMissing": [
      "react",
      "react-dom"
    ]
  }
},
"supportedArchitectures": {
  "os": [
    "current",
    "linux"
  ],
  "cpu": [
    "current",
    "x64"
  ],
  "libc": [
    "current",
    "musl",
    "glibc"
  ]
}
},
"remote_binary_bundling": true,
"remote_binary": [
  {
    "sha256hash": "5663b221d46ffd98ed4ca1bbaa803af9ac22b371fa1af8261b11758cb9b20893",
    "url": "https://github.com/Heroic-Games-Launcher/legendary/releases/download/0.20.38/legendary_linux_x86_64",
    "name": "legendary"
  },
  {
    "sha256hash": "01d7e42821d5310cdc59f09d1b573af7a1e2e35f56741a1076dab9da05fd0cd8",
    "url": "https://github.com/Heroic-Games-Launcher/heroic-gogdl/releases/download/v1.1.2/gogdl_linux_x86_64",

```

```
    "name": "gogdl"
  },
  {
    "sha256hash": "431f82fc74000e6c864409f1d8fb495d696c03928808e3e8acffc45179312a7b",
    "url": "https://raw.githubusercontent.com/Winetricks/winetricks/20260125/src/winetricks",
    "name": "winetricks"
  },
  {
    "sha256hash": "3a8c080c864a5952a01d7661693c60727b34a355ae21e9eab2047096b606c1df",
    "url": "https://github.com/imLinguin/nile/releases/download/v1.1.2/nile_linux_x86_64",
    "name": "nile"
  },
  {
    "sha256hash": "2d6694d544fd3155d90d540e70bc1be767a6b9fdda130275f2b79616ff14e843",
    "url": "https://github.com/imLinguin/comet/releases/download/v0.3.2/comet-x86_64-unknown-linux-gnu",
    "name": "comet"
  }
]
}
```

OP-01g

main.py

Fichier : main.py | Lignes : 73 | Depend de : OP-05, OP-09a, OP-09i, OP-11a, OP-47b, OP-04a, OP-04c, OP-13, OP-05c, OP-24, C

```

from __future__ import annotations

import logging
import os
import sys
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from unifideck.event_bus.bus_pipeline import BusPipeline

DECKY_PLUGIN_DIR = os.environ.get(
    "DECKY_PLUGIN_DIR", os.path.dirname(__file__),
)

sys.path.insert(0, os.path.join(DECKY_PLUGIN_DIR, "py_modules"))

from unifideck.config.user_config_path import resolve_user_config_path
from unifideck.rpc import auto_wrap_rpc_methods
from unifideck.rpc.mixins.action import ActionRPCMixin
from unifideck.rpc.mixins.cloud_failure import CloudFailureRPCMixin
from unifideck.rpc.mixins.config_validation import ConfigValidationRPCMixin
from unifideck.rpc.mixins.download import DownloadRPCMixin
from unifideck.rpc.mixins.launch import LaunchRPCMixin
from unifideck.rpc.mixins.observability import ObservabilityRPCMixin
from unifideck.rpc.mixins.playtime import PlaytimeRPCMixin
from unifideck.rpc.mixins.security import SecurityRPCMixin
from unifideck.rpc.mixins.store import StoreRPCMixin
from unifideck.rpc.mixins.sync import SyncRPCMixin
from unifideck.rpc.mixins.ui import UIRPCMixin

logger = logging.getLogger(__name__)

@auto_wrap_rpc_methods
class Plugin(
    ObservabilityRPCMixin,
    SecurityRPCMixin,
    DownloadRPCMixin,
    LaunchRPCMixin,
    StoreRPCMixin,
    SyncRPCMixin,
    UIRPCMixin,
    CloudFailureRPCMixin,
    ConfigValidationRPCMixin,
    PlaytimeRPCMixin,
    ActionRPCMixin,
):
    async def _main(self) -> None:
        from unifideck.bootstrap.boot import boot_plugin
        await boot_plugin(
            self,
            decky_plugin_dir=DECKY_PLUGIN_DIR,
            user_config_path_resolver=resolve_user_config_path,
        )

```

```
async def _validate_config(self) -> None:
    from unifideck.config.startup import validate_config_at_startup
    defaults_path = os.path.join(
        DECKY_PLUGIN_DIR, "defaults", "config.json",
    )
    (
        self._config_validation_result,
        self._config_degraded,
    ) = await validate_config_at_startup(
        bus=self.bus,
        config=self.config,
        defaults_path=defaults_path,
        user_config_path=self._user_config_path,
    )

async def _build_eventbus_pipeline(self) -> BusPipeline:
    from unifideck.bootstrap.pipeline_factory import (
        build_eventbus_pipeline,
    )
    return await build_eventbus_pipeline(self)

async def _unload(self) -> None:
    from unifideck.bootstrap.teardown import unload_plugin
    await unload_plugin(self)

def _register_caches(self) -> None:
    from unifideck.bootstrap.cache_registry import (
        register_default_caches,
    )
    register_default_caches(self.cache)
```

OP-02a

config.json

Fichier : defaults/config.json | Lignes : 289 | Depend de : (rien)

```
{
  "paths": {
    "data_dir": "~/.local/share/unifideck",
    "cache_dir": "~/.cache/unifideck",
    "steam_candidates": [
      "~/.steam/steam",
      "~/.local/share/Steam",
      "~/.var/app/com.valvesoftware.Steam/.steam/steam"
    ],
    "games_map": "~/.local/share/unifideck/games.map",
    "sd_card_root": "/run/media"
  },
  "cache_ttl": {
    "steam_metadata": 604800,
    "compat": 604800,
    "metacritic_metadata": 2592000
  },
  "sync": {
    "artwork_concurrency": 4
  },
  "dedup": {
    "tracked_stores": [
      "epic",
      "gog",
      "amazon",
      "ubisoft"
    ]
  },
  "download": {
    "custom_path": ""
  },
  "cli_timeouts": {
    "auth_check": 10,
    "version_check": 2,
    "library_fetch": 30,
    "install_poll": 60,
    "uninstall": 120
  },
  "stores": {
    "gog": {
      "client_id": "46899977096215655",
      "client_secret": "9d85c43b1482497dbbce61f6e4aa173a433796eeae2ca8c5f6129f2dc4de46d9",
      "auth_url": "https://auth.gog.com/auth",
      "token_url": "https://auth.gog.com/token",
      "redirect_uri": "https://embed.gog.com/on_login_success?origin=client",
      "allowed_redirect_uris": [
        "https://embed.gog.com/on_login_success"
      ],
      "base_url": "https://embed.gog.com",
      "api_gog_url": "https://api.gog.com",
      "token_file": "~/.config/unifideck/gog_token.json",
      "gogdl_config_dir": "~/.config/unifideck/gogdl",
      "download_dir": "~/.GOG Games",
      "token_refresh_threshold_seconds": 2400,
      "supported_languages": ["en", "de", "fr", "pl", "ru", "pt", "es", "it", "zh", "ko",
```

```

        "ja"],
        "user_agent": "Unifideck/1.0"
    },
    "epic": {
        "cli_binary_search_paths": [
            "bin/legendary"
        ],
        "cli_version_flag": "--version",
        "user_file": "~/.config/legendary/user.json",
        "default_install_root": "~/Games/Epic",
        "auth_redirect_urls": [
            "https://legendary.epicgames.com/callback",
            "https://www.epicgames.com/id/api/redirect"
        ],
        "auth_url_markers": [
            "epicgames.com"
        ],
        "info_timeout_seconds": 30,
        "list_updates_timeout_seconds": 60,
        "installed_cache_ttl_seconds": 30,
        "size_cache_ttl_seconds": 300
    },
    "amazon": {
        "cli_binary_search_paths": [
            "bin/nile"
        ],
        "cli_version_flag": "--version",
        "nile_config_dir": "~/.config/nile",
        "user_file": "~/.config/nile/user.json",
        "default_install_root": "~/Games/Amazon",
        "auth_redirect_urls": [
            "https://www.amazon.com/ap/maplanding",
            "https://amazon.com/ap/maplanding"
        ],
        "nile_register_success_markers": [
            "Successfully registered",
            "Succesfully registered"
        ],
        "list_updates_timeout_seconds": 60,
        "get_size_timeout_seconds": 60
    },
    "ubisoft": {
        "data_dir": "~/.local/share/unifideck",
        "id_map_file": "~/.local/share/unifideck/ubisoft_id_map.json",
        "visible_games_file": "~/.local/share/unifideck/ubisoft_visible_games.json",
        "prefixes_dir": "~/.local/share/unifideck/prefixes/ubisoft-games",
        "installer_cache_dir": "~/.cache/unifideck/ubisoft",
        "upc_session_file": "~/.local/share/unifideck/ubisoft_session_marker",
        "game_id_db_file": "~/.cache/unifideck/ubisoft/game_id_db.txt",
        "default_install_base": "~/Games/Ubisoft",
        "sdcard_install_base": "/run/media/mmcblk0p1/Games/Ubisoft",
        "template_prefix_name": ".template",
        "auth_prefix_name": ".upc-auth",
        "auth_shortcut_store_id": "ubisoft:.upc-auth",
        "auth_shortcut_launch_wait_ms": 2000,
        "installer_url": "https://ubistatic3-a.akamaihd.net/orbit/launcher_installer/UbisoftCo
nnectInstaller.exe",
        "installer_filename": "UbisoftConnectInstaller.exe",
        "bootstrap_marker": ".unifideck_bootstrapped",
        "game_id_db_url": "https://raw.githubusercontent.com/iArtorias/ubisoft_game_ids/main/g

```

```

    ame_ids.txt",
    "game_id_db_max_age_seconds": 604800,
    "upc_relative_path": "drive_c/Program Files (x86)/Ubisoft/Ubisoft Game
    Launcher/upc.exe",
    "upc_connect_relative_path": "drive_c/Program Files (x86)/Ubisoft/Ubisoft Game
    Launcher/UbisoftConnect.exe",
    "configurations_relative_path": "users/{user}/Local Settings/Application Data/Ubisoft
    Game Launcher/cache/configuration/confi
    "ownership_relative_path": "users/{user}/Local Settings/Application Data/Ubisoft Game
    Launcher/cache/ownership",
    "upc_credential_files": [
        "ConnectSecureStorage.dat",
        "OldSecureStorage.dat"
    ],
    "wine_system_users": [
        "steamuser",
        "deck",
        "Public"
    ],
    "filter_steam_linked": true,
    "steam_library_cross_ref": false
},
"microsoft": {
    "client_id": "000000004C12AE6F",
    "token_file": "~/.local/share/unifideck/microsoft_tokens.json",
    "scope": "service::user.auth.xboxlive.com::MBI_SSL",
    "auth_url": "https://login.live.com/oauth20_authorize.srf",
    "token_url": "https://login.live.com/oauth20_token.srf",
    "redirect_uri": "https://login.live.com/oauth20_desktop.srf",
    "allowed_redirect_uris": [
        "https://login.live.com/oauth20_desktop.srf",
        "https://login.microsoftonline.com/common/oauth2/nativeclient"
    ],
    "xbl_auth_url": "https://user.auth.xboxlive.com/user/authenticate",
    "xsts_url": "https://xsts.auth.xboxlive.com/xsts/authorize",
    "xcloud_catalog_url": "https://catalog.gamepass.com/sigls/v2?id=fdd9e2a7-0fee-49f6-ad6
    9-4354098401ff",

    "xcloud_titles_url": "https://catalog.gamepass.com/v3/products?market=US&language=en-U
    S",
    "xcloud_launch_url": "https://www.xbox.com/play/launch",
    "gssv_relying_party": "http://gssv.xboxlive.com/",
    "subscription_check_url": "https://xgpuweb.gssv-play-prod.xboxlive.com/v2/login/user"
}
},
"metadata": {
    "metacritic": {
        "composer_url": "https://backend.metacritic.com/composer/metacritic/pages/games/{slug}
        /web",
        "fetch_timeout_seconds": 15
    },
    "unifidb": {
        "cdn_base": "https://cdn.jsdelivr.net/gh/mubaraknumann/unifiDB@main",
        "match_threshold": 0.65,
        "fetch_timeout_seconds": 15
    },
    "steam_store": {
        "search_url": "https://store.steampowered.com/api/storesearch",
        "search_timeout_seconds": 15
    }
}
},

```

```
"artwork": {
  "steamgriddb_api_key": "",
  "steamgriddb_api_base": "https://www.steamgriddb.com/api/v2",
  "download_timeout_seconds": 30,
  "failure_cooldown_seconds": 3600
},
"cdp": {
  "host": "127.0.0.1",
  "port": 8080,
  "eval_timeout_seconds": 30,
  "response_timeout_seconds": 10
},
"accounts": {
  "poll_interval_seconds": 5
},
"cloud": {
  "enabled": true,
  "root": "",
  "tolerance_seconds": 2,
  "sync_wait_timeout_seconds": 5,
  "failure_behavior": {
    "default": "toast",
    "epic": "toast",
    "gog": "toast",
    "amazon": "toast",
    "ubisoft": "toast"
  }
},
"logs": {
  "archive_path": "~/.local/share/unifideck/launches",
  "export_path": "~/Downloads",
  "retention_days": 7
},
"circuit_breaker": {
  "failures_threshold": 3,
  "window_seconds": 600,
  "fast_boot_seconds": 10
},
"disk_space": {
  "min_free_bytes": 1073741824,
  "size_multiplier": 1.5,
  "size_cache_path": "~/.cache/unifideck/cloud_save_sizes.json",
  "size_cache_ttl_seconds": 2592000
},
"compat": {
  "protondb_timeout_seconds": 30,
  "deck_verified_timeout_seconds": 10
},
"auth": {
  "browser_poll_interval_seconds": 0.5,
  "browser_oauth_timeout_seconds": 300
},
"discovery": {
  "manifest_filename": ".unifideck_manifest.json"
},
"ui": {
  "locale": "en-US"
},
"probes": {
  "probe_to_features": {
    "steam_client_input": [
```



```

        "controller_hints",
        "custom_binding_ui",

        "steam_input_overlay"
    ],
    "router_hook_patch": [
        "library_view_patch",
        "quickaccess_menu_icon"
    ],
    "steam_client_apps": [
        "circuit_breaker_ui"
    ]
},
"probe_to_handlers": {
    "steam_client_apps": [
        "ArtworkService._on_shortcut_created",
        "ShortcutService._on_download_complete",
        "ShortcutService._on_sync_complete",
        "ShortcutService._on_game_uninstalled"
    ],
    "steam_client_downloads": [
        "ShortcutService._on_download_complete"
    ]
}
},
"proton": {
    "settings_path": "~/.local/share/unifideck/proton_settings.json",
    "shortcuts_registry_path": "~/.local/share/unifideck/shortcuts_registry.json"
},
"binary_resolver": {
    "version_check_timeout_seconds": 10
},
"i18n": {
    "locales": [
        {"tag": "en-US", "deepl_code": null, "name": "English", "rtl": false},
        {"tag": "fr-FR", "deepl_code": "FR", "name": "Français", "rtl": false},
        {"tag": "de-DE", "deepl_code": "DE", "name": "Deutsch", "rtl": false},
        {"tag": "es-ES", "deepl_code": "ES", "name": "Español", "rtl": false},
        {"tag": "it-IT", "deepl_code": "IT", "name": "Italiano", "rtl": false},
        {"tag": "pt-BR", "deepl_code": "PT-BR", "name": "Português (Brasil)", "rtl": false},
        {"tag": "nl-NL", "deepl_code": "NL", "name": "Nederlands", "rtl": false},
        {"tag": "pl-PL", "deepl_code": "PL", "name": "Polski", "rtl": false},
        {"tag": "ru-RU", "deepl_code": "RU", "name": "Русский", "rtl": false},
        {"tag": "uk-UA", "deepl_code": "UK", "name": "Українська", "rtl": false},
        {"tag": "tr-TR", "deepl_code": "TR", "name": "Türkçe", "rtl": false},
        {"tag": "ja-JP", "deepl_code": "JA", "name": "Japanese", "rtl": false},
        {"tag": "ko-KR", "deepl_code": "KO", "name": "Korean", "rtl": false},
        {"tag": "zh-CN", "deepl_code": "ZH-HANS", "name": "Simplified Chinese", "rtl": false},
        {"tag": "zh-TW", "deepl_code": "ZH-HANT", "name": "Traditional Chinese", "rtl": false},
        {"tag": "ar-SA", "deepl_code": "AR", "name": "Arabic", "rtl": true}
    ]
},
"security": {
    "audit_log_capacity": 500,
    "bruteforce_window_seconds": 60.0,
    "bruteforce_warning_threshold": 5,
    "bruteforce_escalation_threshold": 20,
    "fingerprint_path": "~/.config/unifideck/device_fingerprint.json",
    "token_files_to_wipe_on_reset": [
        "~/.local/share/unifideck/microsoft_tokens.json",

```

```
    "~/config/unifideck/gog_token.json"  
  ]  
}
```

OP-03a

unifideck-launcher

Fichier : bin/unifideck-launcher | Lignes : 29 | Depend de : (rien)

```
from __future__ import annotations
import os
os.environ.pop("LD_LIBRARY_PATH", None)
os.environ.pop("LD_PRELOAD", None)
import sys
from pathlib import Path
def _bootstrap_path() -> None:
    plugin_dir = Path(__file__).resolve().parent.parent
    py_modules = plugin_dir / "py_modules"
    if py_modules.is_dir():
        sys.path.insert(0, str(py_modules))
def main() -> int:
    _bootstrap_path()
    try:
        from unifideck.launcher.dispatcher import main as dispatcher_main
    except ImportError as exc:
        print(
            f"[unifideck-launcher] failed to import dispatcher: {exc}",
            file=sys.stderr,
        )
        print(
            f"[unifideck-launcher] plugin_dir="
            f"{Path(__file__).resolve().parent.parent}",
            file=sys.stderr,
        )
        return 2
    return dispatcher_main(sys.argv)
if __name__ == "__main__":
    sys.exit(main())
```

Phase 1 — Backend

Refactor of `py_modules/unifideck/` into a 5-layer architecture

Scope

The Backend phase delivers the new Python runtime: a 5-layer architecture (`core / event_bus / config / service / launcher`) that replaces the legacy plugin instance with an EventBus-driven service container. Each subpackage is documented as a fiche-set listing the exhaustive file inventory, dependencies, and code listings.

Coverage spans 350 backend fiches: types, IO primitives, EventBus with priority dispatch and replay, supervision (watchdog + metrics), config validation, RPC handlers + mixins, action dispatcher, auth edge browser, CDP shims, store connectors (Epic / Amazon / GOG / Microsoft / Ubisoft), launcher orchestration, Proton infrastructure, language setup, and the bootstrap pipeline.

OP-01

__init__.py

Fichier : py_modules/unifideck/__init__.py | Lignes : 2 | Depend de : (rien)

```
from __future__ import annotations
__version__ = "0.7.0"
```

OP-04

__init__.py

Fichier : py_modules/unifideck/core/__init__.py | Lignes : 14 | Depend de : (rien)

```
from .cache_manager import CacheManager
from .types import (
    AuthResult,
    CLITool,
    DownloadResult,
    Events,
    Game,
    InstallResult,
    Result,
    StoreError,
    StoreInfo,
    StoreStatus,
    SyncResult,
)
```

OP-05

`__init__.py`

Fichier : `py_modules/unifideck/core/types/__init__.py` | Lignes : 41 | Depend de : (rien)

```
from __future__ import annotations
from .domain import (
    CLITool,
    Game,
    StoreInfo,
)
from .events import (
    ErrorCode,
    Events,
    GameTag,
    OwnershipType,
    StoreEnum,
    StoreStatus,
    SubscriptionTier,
)
from .results import (
    AccountResult,
    ArtworkResult,
    AuthResult,
    CloudSaveResult,
    DownloadResult,
    InstallResult,
    MetadataResult,
    PlaytimeResult,
    Result,
    StoreAuthError,
    StoreDownloadError,
    StoreError,
    StoreSyncError,
    SyncResult,
)
__all__ = [
    "ErrorCode", "Events", "GameTag", "OwnershipType",
    "StoreEnum", "StoreStatus", "SubscriptionTier",
    "AccountResult", "ArtworkResult", "AuthResult",
    "CloudSaveResult", "DownloadResult", "InstallResult",
    "MetadataResult", "PlaytimeResult", "Result",
    "StoreAuthError", "StoreDownloadError", "StoreError",
    "StoreSyncError", "SyncResult",
    "CLITool", "Game", "StoreInfo",
]
```

OP-05a

events.py

Fichier : py_modules/unifideck/core/types/events.py | Lignes : 103 | Depend de : (rien)

```
from __future__ import annotations
from enum import StrEnum
class Events(StrEnum):
    """Events."""
    PLUGIN_LOADED = "plugin_loaded"
    PLUGIN_UNLOADING = "plugin_unloading"
    SYNC_STARTED = "sync_started"
    SYNC_PROGRESS = "sync_progress"
    SYNC_COMPLETE = "sync_complete"
    SYNC_FAILED = "sync_failed"
    SYNC_CANCELLED = "sync_cancelled"
    STORE_AUTH_STARTED = "store_auth_started"
    STORE_AUTH_COMPLETE = "store_auth_complete"
    STORE_AUTH_FAILED = "store_auth_failed"
    STORE_LOGOUT = "store_logout"
    STORE_REGISTERED = "store_registered"
    GAME_INSTALLED = "game_installed"
    GAME_UNINSTALLED = "game_uninstalled"
    GAME_UPDATE_AVAILABLE = "game_update_available"
    GAME_LAUNCHED = "game_launched"
    GAME_STOPPED = "game_stopped"
    LAUNCHER_STAGE = "launcher_stage"
    CIRCUIT_STATE_CHANGED = "circuit_state_changed"
    DOWNLOAD_QUEUED = "download_queued"
    DOWNLOAD_STARTED = "download_started"
    DOWNLOAD_PROGRESS = "download_progress"
    DOWNLOAD_COMPLETE = "download_complete"
    DOWNLOAD_FAILED = "download_failed"
    DOWNLOAD_CANCELLED = "download_cancelled"
    STORE_ERROR = "store_error"
    RUNTIME_PROBES_REPORTED = "runtime_probes_reported"
    SECURITY_TOKEN_ENCRYPTED = "security_token_encrypted"
    SECURITY_TOKEN_DECRYPTED = "security_token_decrypted"
    SECURITY_DECRYPT_FAILED = "security_decrypt_failed"
    SECURITY_TOKEN_FILE_MIGRATED = "security_token_file_migrated"
    SECURITY_LEGACY_PLAINTEXT_DETECTED = "security_legacy_plaintext_detected"
    SECURITY_AUTH_FLOW_STARTED = "security_auth_flow_started"
    SECURITY_AUTH_FLOW_COMPLETED = "security_auth_flow_completed"
    SECURITY_AUTH_FLOW_FAILED = "security_auth_flow_failed"
    SECURITY_PERMISSIONS_CHECK = "security_permissions_check"
    SECURITY_PERMISSIONS_REPAIRED = "security_permissions_repaired"
    SECURITY_BRUTEFORCE_SUSPECTED = "security_bruteforce_suspected"
    SECURITY_DEVICE_RESET_DETECTED = "security_device_reset_detected"
    SECURITY_FINGERPRINT_INITIALIZED = "security_fingerprint_initialized"
    SECURITY_EXTERNAL_AUTH_CHECK_FAILED = "security_external_auth_check_failed"
    CONFIG_VALIDATION_COMPLETED = "config_validation_completed"
    CONFIG_VALIDATION_FAILED = "config_validation_failed"
    SUBSCRIPTION_DETECTED = "subscription_detected"
    SUBSCRIPTION_EXPIRED = "subscription_expired"
    SUBSCRIPTION_CHECK_FAILED = "subscription_check_failed"
    SYNC_SKIPPED = "sync_skipped"
    SYNC_DEDUP = "sync_dedup"
    ACCOUNT_SWITCHED = "account_switched"

    SHORTCUT_CREATED = "shortcut_created"
```



```
ARTWORK_REQUEST = "artwork_request"
class StoreStatus(StrEnum):
    """Store status."""
    UNAVAILABLE = "unavailable"
    NOT_AUTHENTICATED = "not_authenticated"
    AVAILABLE = "available"
    ERROR = "error"
class StoreEnum(StrEnum):
    """Store enum."""
    EPIC = "epic"
    GOG = "gog"
    AMAZON = "amazon"
    MICROSOFT = "microsoft"
    UBISOFT = "ubisoft"
class OwnershipType(StrEnum):
    """Ownership type."""
    OWNED = "owned"
    SUBSCRIBED = "subscribed"
    TRIAL = "trial"
    UNKNOWN = "unknown"
class GameTag(StrEnum):
    """Game tag."""
    NATIVE = "native"
    PROTON = "proton"
    CLOUD = "cloud"
    XCLOUD = "xcloud"
    DLC = "dlc"
    BETA = "beta"
    DEMO = "demo"
    HIDDEN = "hidden"
class ErrorCode(StrEnum):
    """Error code."""
    NOT_AUTHENTICATED = "not_authenticated"
    TOKEN_EXPIRED = "token_expired"
    NETWORK_ERROR = "network_error"
    NOT_FOUND = "not_found"
    PERMISSION_DENIED = "permission_denied"
    QUOTA_EXCEEDED = "quota_exceeded"
    INSUFFICIENT_SPACE = "insufficient_space"
    BINARY_MISSING = "binary_missing"
    TIMEOUT = "timeout"
    UNKNOWN = "unknown"
class SubscriptionTier(StrEnum):
    """Subscription tier."""
    NONE = "none"
    ESSENTIAL = "essential"
    PREMIUM = "premium"
    ULTIMATE = "ultimate"
    ACTIVE_UNKNOWN = "active_unknown"
```

OP-05b

results.py

Fichier : py_modules/unifideck/core/types/results.py | Lignes : 138 | Depend de : (rien)

```

"""core/types/results.py – Result dataclasses and StoreError exceptions.
The split keeps `events.py` free of runtime imports (no dataclass
module needed) and lets future result additions touch only this
file without rebuilding the enums.
Design: every result subclass adds its own fields but inherits
`success: bool`, `error: Optional[str]`, and `store: Optional[str]`
from `Result`. This lets generic code (logging, telemetry) treat
all results uniformly via isinstance checks on `Result`.
Reference: Technical Document v1.0 – Section 3.4.5 (Result types).
"""

from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any
@dataclass
class Result:
    """Base dataclass for every store operation return value.
    Fields:
        success: True if the operation completed without error.
            Callers should NEVER parse `error` to decide success –
            always check this flag first.
        error: Optional error string. Human-readable message, free-
            form. Populated on failure; None on success. Must NOT be
            parsed by callers for control-flow decisions – use
            `error_code` for that. The string is intended for logs
            and user-visible messages only.
        error_code: Optional machine-readable error identifier.
            When populated, it's the authoritative classification of
            the failure (a `LauncherErrorCode` enum value or a stable
            string like "exit_<rc>" for subprocess exit propagation).
            The dispatcher's exit code mapping dispatches on this
            field exclusively – no more fragile `not_implemented`
            in err_str` string-matching. None on success or for
            results that don't need machine classification (historical
            store results that pre-date this field).
        store: Optional store ID that produced the result, for
            logging and frontend routing. None for global operations.
        metadata: Free-form store-specific payload for fields that
            don't belong on the canonical `Result` surface. Mirrors
            `Game.metadata`: keeps the base class slim while letting
            individual stores carry their own signaling flags (e.g.
            `pending`, `needs_2fa`, `auth_type`). Frontend should
            treat these as optional hints, never as contract.
    """
    success: bool = True
    error: str | None = None
    error_code: str | None = None
    store: str | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
@dataclass
class AuthResult(Result):
    """Returned by `store.auth_action('start'|'complete'|'status')`."""
    action: str | None = None
    next_step: str | None = None
    url: str | None = None
    tokens_cached: bool = False

```

```

@dataclass
class InstallResult(Result):
    """Returned by `store.install_game()` and `uninstall_game()`."""
    game_id: str | None = None
    install_path: str | None = None
    size_bytes: int = 0

@dataclass
class SyncResult(Result):
    """Returned by `SyncService.sync()`."""
    games: list[Any] = field(default_factory=list)
    count: int = 0
    duration_ms: int = 0

@dataclass
class DownloadResult(Result):
    """Returned by DownloadService operations."""
    download_id: str | None = None
    bytes_downloaded: int = 0
    bytes_total: int = 0
    progress_pct: float = 0.0

@dataclass
class PlaytimeResult(Result):
    """Returned by PlaytimeService queries."""
    game_id: str | None = None
    total_seconds: int = 0
    last_played: str | None = None # ISO 8601
    session_count: int = 0

@dataclass
class MetadataResult(Result):
    """Returned by MetadataService.get()."""
    game_id: str | None = None
    title: str | None = None
    description: str | None = None
    release_date: str | None = None
    genres: list[str] = field(default_factory=list)
    metacritic_score: int | None = None
    raw: dict[str, Any] = field(default_factory=dict)

@dataclass
class ArtworkResult(Result):
    """Returned by ArtworkService.fetch()."""
    game_id: str | None = None
    hero_path: str | None = None
    logo_path: str | None = None
    grid_path: str | None = None
    icon_path: str | None = None
    source: str | None = None

@dataclass
class CloudSaveResult(Result):
    """Returned by CloudSaveService.upload()/download()."""
    game_id: str | None = None
    direction: str | None = None # "upload" or "download"
    files_processed: int = 0
    bytes_transferred: int = 0

@dataclass
class AccountResult(Result):
    """Returned by AccountService.get_steam_user()."""
    steam_id64: str | None = None
    persona_name: str | None = None
    account_name: str | None = None
    most_recent: bool = False

```

```
# — Exception hierarchy —————
class StoreError(Exception):
    """Base class for all store-originated errors.
    Store connectors should raise subclasses of StoreError rather
    than bare Exception so callers can catch `StoreError` to match
    any store failure without catching unrelated programmer errors.
    """
    def __init__( # noqa: D107 – class docstring documents the constructor's contract
        self,
        message: str,
        *,
        store: str | None = None,
        code: str | None = None,
    ) -> None:
        """Initialize the instance."""
        super().__init__(message)
        self.store = store
        self.code = code
class StoreAuthError(StoreError):
    """Raised on any auth-related failure."""
class StoreSyncError(StoreError):
    """Raised when library fetching fails unrecoverably."""
class StoreDownloadError(StoreError):
    """Raised when an install/uninstall/update fails."""
```

OP-05c

domain.py

Fichier : py_modules/unifideck/core/types/domain.py | Lignes : 36 | Depend de : (rien)

```
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any
@dataclass
class Game:
    """Game."""
    app_id: int
    store: str
    store_game_id: str
    title: str
    installed: bool = False
    install_path: str | None = None
    exe_path: str | None = None
    size_bytes: int = 0
    tags: list[str] = field(default_factory=list)
    icon_url: str | None = None
    hero_url: str | None = None
    logo_url: str | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
@dataclass
class StoreInfo:
    """Store info."""
    name: str
    display_name: str
    auth_method: str
    icon_asset: str
    uses_wine: bool = False
    supports_install: bool = True
    supports_cloud_saves: bool = False
@dataclass
class CLITool:
    """Clitool."""
    name: str
    search_paths: list[str] = field(default_factory=list)
    version_flag: str = "--version"
    min_version: str | None = None
```

OP-06

`__init__.py`

Fichier : `py_modules/unifideck/core/io/__init__.py` | Lignes : 6 | Depend de : (rien)

```
from . import async_file_ops
from .safe_file_op import safe_file_op
__all__ = [
    "async_file_ops",
    "safe_file_op",
]
```

OP-06a

async_file_ops.py

Fichier : py_modules/unifideck/core/io/async_file_ops.py | Lignes : 196 | Depend de : (rien)

```
import asyncio
import json
import logging
import os
import shutil
from pathlib import Path
from typing import Any, cast
logger = logging.getLogger(__name__)
PathLike = str | os.PathLike
async def exists(path: PathLike) -> bool:
    """Exists."""
    return await asyncio.to_thread(
        lambda: Path(path).exists(),
    )
async def is_file(path: PathLike) -> bool:
    """Check whether file."""
    return await asyncio.to_thread(
        lambda: Path(path).is_file(),
    )
async def is_dir(path: PathLike) -> bool:
    """Check whether dir."""
    return await asyncio.to_thread(
        lambda: Path(path).is_dir(),
    )
async def listdir(path: PathLike) -> list[str]:
    """Listdir."""
    try:
        return await asyncio.to_thread(
            lambda: [p.name for p in Path(path).iterdir()],
        )
    except OSError as e:
        logger.warning(
            "[AsyncFileOps] listdir(%s) failed: %s", path, e,
        )
        return []
async def stat(path: PathLike) -> os.stat_result | None:
    """Stat."""
    try:
        return await asyncio.to_thread(
            lambda: Path(path).stat(),
        )
    except OSError:
        return None
async def makedirs(path: PathLike, mode: int = 0o755,
                  exist_ok: bool = True) -> bool:
    """Makedirs."""
    try:
        await asyncio.to_thread(
            lambda: Path(path).mkdir(
                mode=mode, parents=True, exist_ok=exist_ok,
            ),
        )
        return True
    except OSError as e:
        logger.error(
```

```

        "[AsyncFileOps] makedirs(%s) failed: %s", path, e,
    )
    return False
async def ensure_dir(path: PathLike) -> bool:
    """Ensure dir."""
    return await makedirs(path, exist_ok=True)
async def copy(src: PathLike, dst: PathLike) -> bool:
    """Copy."""
    try:
        await asyncio.to_thread(shutil.copy2, src, dst)
        return True
    except (OSError, shutil.Error) as e:
        logger.error(
            "[AsyncFileOps] copy(%s -> %s) failed: %s",
            src, dst, e,
        )
        return False

async def move(src: PathLike, dst: PathLike) -> bool:
    """Move."""
    try:
        await asyncio.to_thread(shutil.move, str(src), str(dst))
        return True
    except (OSError, shutil.Error) as e:
        logger.error(
            "[AsyncFileOps] move(%s -> %s) failed: %s",
            src, dst, e,
        )
        return False
async def remove(path: PathLike) -> bool:
    """Remove."""
    try:
        await asyncio.to_thread(
            lambda: Path(path).unlink(missing_ok=True),
        )
        return True
    except OSError as e:
        logger.error(
            "[AsyncFileOps] remove(%s) failed: %s", path, e,
        )
        return False
async def read_text(path: PathLike, encoding: str = "utf-8") -> str | None:
    """Read text."""
    try:
        return await asyncio.to_thread(
            lambda: Path(path).read_text(encoding=encoding)
        )
    except (OSError, UnicodeDecodeError) as e:
        logger.warning("[AsyncFileOps] read_text(%s) failed: %s", path, e)
        return None
async def write_text(path: PathLike, content: str,
                    encoding: str = "utf-8", mode: int = 0o644) -> bool:
    """Write text."""
    return await asyncio.to_thread(_write_text_sync, path, content, encoding, mode)
def _write_text_sync(path: PathLike, content: str,
                    encoding: str, mode: int) -> bool:
    """Write text sync."""
    p = Path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    tmp = Path(str(p) + ".tmp")

```



```

try:
    tmp.write_text(content, encoding=encoding)
    tmp.replace(p)
    if mode != 0o644:
        p.chmod(mode)
    return True
except OSError as e:
    logger.error(
        "[AsyncFileOps] write_text(%s) failed: %s", path, e,
    )
    if tmp.exists():
        try:
            tmp.unlink()
        except OSError:
            pass
    return False
async def write_bytes(path: PathLike, data: bytes) -> bool:
    """Write bytes."""
    return await asyncio.to_thread(_write_bytes_sync, path, data)
def _write_bytes_sync(path: PathLike, data: bytes) -> bool:
    """Write bytes sync."""
    p = Path(path)
    tmp = p.with_suffix(p.suffix + ".tmp")
    try:
        tmp.parent.mkdir(parents=True, exist_ok=True)
        tmp.write_bytes(data)
        tmp.replace(p)
        return True
    except OSError:
        try:
            if tmp.exists():
                tmp.unlink()
        except OSError:
            pass
        return False
async def read_json(path: PathLike) -> dict[str, Any]:
    """Read JSON."""
    return await asyncio.to_thread(_read_json_sync, path)
def _read_json_sync(path: PathLike) -> dict[str, Any]:
    """Read JSON sync."""
    p = Path(path)
    if not p.exists():
        return {}
    try:
        with p.open(encoding="utf-8") as f:
            return cast("dict[str, Any]", json.load(f))
    except (
        json.JSONDecodeError, UnicodeDecodeError, OSError,
    ) as e:
        logger.warning(
            "[AsyncFileOps] failed to read JSON %s: %s", path, e,
        )
        return {}
async def write_json(path: PathLike, data: Any, indent: int = 2,
                    mode: int = 0o644) -> bool:
    """Write JSON."""
    return await asyncio.to_thread(_write_json_sync, path, data, indent, mode)
def _write_json_sync(path: PathLike, data: Any, indent: int,
                    mode: int = 0o644) -> bool:

```

```
"""Write JSON sync."""
p = Path(path)
p.parent.mkdir(parents=True, exist_ok=True)
tmp = Path(str(p) + ".tmp")
try:
    content = json.dumps(
        data, ensure_ascii=False, indent=indent,
    )
    tmp.write_text(content, encoding="utf-8")
    tmp.replace(p)
    if mode != 0o644:
        p.chmod(mode)
    return True
except (OSError, TypeError, ValueError) as e:
    logger.error(
        "[AsyncFileOps] write_json(%s) failed: %s", path, e,
    )
    if tmp.exists():
        try:
            tmp.unlink()
        except OSError:
            pass
    return False
```

OP-06b

safe_file_op.py

Fichier : py_modules/unifideck/core/io/safe_file_op.py | Lignes : 48 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import functools
import logging
from collections.abc import Awaitable, Callable
from typing import Any, TypeVar
logger = logging.getLogger(__name__)
T = TypeVar("T")
_Callable = Callable[..., T | Awaitable[T]]
def safe_file_op(
    default: Any = None,
    *,
    log_level: int = logging.WARNING,
) -> Callable[[Callable], Callable]:
    """Safe file op."""
    def decorator(fn: Callable) -> Callable:
        """Decorator."""
        fname = getattr(fn, "__name__", repr(fn))
        if asyncio.iscoroutinefunction(fn):
            @functools.wraps(fn)
            async def async_wrapper(*args: Any, **kwargs: Any) -> Any:
                """Async wrapper."""
                try:
                    return await fn(*args, **kwargs)
                except OSError as e:
                    path_hint = args[0] if args else kwargs.get("path", "?")
                    logger.log(
                        log_level,
                        "[safe_file_op] %s(%r) failed: %s: %s",
                        fname, path_hint, type(e).__name__, e,
                    )
                    return default
            return async_wrapper
        @functools.wraps(fn)
        def sync_wrapper(*args: Any, **kwargs: Any) -> Any:
            """Sync wrapper."""
            try:
                return fn(*args, **kwargs)
            except OSError as e:
                path_hint = args[0] if args else kwargs.get("path", "?")
                logger.log(
                    log_level,
                    "[safe_file_op] %s(%r) failed: %s: %s",
                    fname, path_hint, type(e).__name__, e,
                )
                return default
            return sync_wrapper
        return decorator

```

OP-07

`__init__.py`

Fichier : `py_modules/unifideck/core/bin/__init__.py` | Lignes : 16 | Depend de : (rien)

```
from .binary_resolver import (
    BinaryResolver,
    binary_resolver,
)
from .binary_signatures import (
    compute_sha256,
    verify_bundled_binary,
)
from .cli_timeouts import read_cli_timeouts
__all__ = [
    "BinaryResolver",
    "binary_resolver",
    "compute_sha256",
    "verify_bundled_binary",
    "read_cli_timeouts",
]
```

OP-07a

binary_resolver.py

Fichier : py_modules/unifideck/core/bin/binary_resolver.py | Lignes : 94 | Depend de : (rien)

```
import logging
import shutil
import stat
import subprocess
from pathlib import Path
from ..types.domain import CLITool
logger = logging.getLogger(__name__)
def _is_executable(path: str) -> bool:
    """Is executable."""
    try:
        st = Path(path).stat()
    except OSError:
        return False
    if not stat.S_ISREG(st.st_mode):
        return False
    return bool(st.st_mode & stat.S_IXUSR)
class BinaryResolver:
    """Binary resolver."""
    def __init__(self, config=None) -> None:
        """Initialize the instance."""
        self._version_timeout = 10
        if config is not None:
            try:
                self._version_timeout = int(config.get(
                    "binary_resolver.version_check_timeout_seconds"))
            except (TypeError, ValueError):
                pass
    def resolve(self, tool: CLITool) -> str | None:
        """Resolve."""
        for candidate in tool.search_paths:
            expanded = str(Path(candidate).expanduser())
            if (
                Path(expanded).is_absolute()
                and _is_executable(expanded)
            ):
                logger.debug(
                    "[BinaryResolver] %s found in search_paths: "
                    "%s",
                    tool.name, expanded,
                )
                return expanded
        which = shutil.which(tool.name)
        if which and _is_executable(which):
            logger.debug(
                "[BinaryResolver] %s found in PATH: %s",
                tool.name, which,
            )
            return which
        local = Path.home() / ".local" / "bin" / tool.name
        if _is_executable(str(local)):
            logger.debug(
                "[BinaryResolver] %s found in ~/.local/bin",
                tool.name,
            )
            return str(local)
```

```
        logger.info(
            "[BinaryResolver] %s not found in any tier",
            tool.name,
        )
        return None

def check_version(
    self, tool: CLITool, binary_path: str,
) -> str | None:

    """Check version."""
    try:
        result = subprocess.run(
            [binary_path, tool.version_flag],
            capture_output=True,
            text=True,
            timeout=self._version_timeout,
            check=False,
        )
    except (
        subprocess.TimeoutExpired,
        FileNotFoundError,
        OSError,
    ) as e:
        logger.warning(
            "[BinaryResolver] version check failed for "
            "%s: %s",
            tool.name, e,
        )
        return None
    version = (
        result.stdout.strip() or result.stderr.strip()
    ).splitlines()
    if version:
        v = version[0].strip()
        logger.debug(
            "[BinaryResolver] %s version: %s", tool.name, v,
        )
        return v
    return None
binary_resolver = BinaryResolver()
```

OP-07b

binary_signatures.py

Fichier : py_modules/unifideck/core/bin/binary_signatures.py | Lignes : 59 | Depend de : (rien)

```
from __future__ import annotations
import hashlib
import logging
from pathlib import Path
logger = logging.getLogger(__name__)
_KNOWN_HASHES: dict[str, str] = {
    "legendary": "",
    "nile": "",
    "gogdl": "",
}
def compute_sha256(path: str, chunk_size: int = 65536) -> str | None:
    """Compute sha256."""
    try:
        h = hashlib.sha256()
        with Path(path).open("rb") as f:
            while True:
                chunk = f.read(chunk_size)
                if not chunk:
                    break
                h.update(chunk)
        return h.hexdigest()
    except OSError as e:
        logger.warning(
            "[binary_signatures] Cannot read %s: %s", path, e,
        )
        return None
def verify_bundled_binary(
    tool_name: str,
    path: str,
) -> bool | None:
    """Verify bundled binary."""
    expected = _KNOWN_HASHES.get(tool_name, "")
    if not expected:
        logger.debug(
            "[binary_signatures] No reference hash for %s – "
            "returning None (caller decides policy)", tool_name,
        )
        return None
    if not Path(path).is_file():
        logger.warning(
            "[binary_signatures] %s not found at %s",
            tool_name, path,
        )
        return None
    actual = compute_sha256(path)
    if actual is None:
        return None
    if actual != expected:
        logger.error(
            "[binary_signatures] SECURITY: %s hash mismatch at %s. "
            "Expected %s, got %s. Refusing to trust this binary.",
            tool_name, path, expected, actual,
        )
        return False
    logger.info(
```

```
        "[binary_signatures] %s at %s verified (sha256=%s)",  
        tool_name, path, actual,  
    )  
    return True
```


OP-07c

cli_timeouts.py

Fichier : py_modules/unifideck/core/bin/cli_timeouts.py | Lignes : 24 | Depend de : (rien)

```
from __future__ import annotations
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ...config import ConfigManager
DEFAULT_TIMEOUTS: dict[str, int] = {
    "auth_check": 10,
    "version_check": 2,
    "library_fetch": 30,
    "install_poll": 60,
    "uninstall": 120,
}
def read_cli_timeouts(config: ConfigManager | None) -> dict[str, int]:
    """Read cli timeouts."""
    if config is None:
        return dict(DEFAULT_TIMEOUTS)
    out: dict[str, int] = {}
    for key, default in DEFAULT_TIMEOUTS.items():
        try:
            out[key] = int(
                config.get(f"cli_timeouts.{key}", default),
            )
        except (TypeError, ValueError):
            out[key] = default
    return out
```

OP-08

`__init__.py`

Fichier : `py_modules/unifideck/core/net/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from .ssl_helpers import ssl_ctx_permissive, ssl_ctx_strict
__all__ = ["ssl_ctx_permissive", "ssl_ctx_strict"]
```

OP-08a

ssl_helpers.py

Fichier : py_modules/unifideck/core/net/ssl_helpers.py | Lignes : 36 | Depend de : (rien)

```
from __future__ import annotations
import logging
import ssl
from threading import Lock
logger = logging.getLogger(__name__)
_strict_lock = Lock()
_strict_ctx: ssl.SSLContext | None = None
_permissive_lock = Lock()
_permissive_ctx: ssl.SSLContext | None = None
_permissive_warned = False
def ssl_ctx_strict() -> ssl.SSLContext:
    """Ssl ctx strict."""
    global _strict_ctx
    if _strict_ctx is None:
        with _strict_lock:
            if _strict_ctx is None:
                _strict_ctx = ssl.create_default_context()
    return _strict_ctx
def ssl_ctx_permissive(reason: str) -> ssl.SSLContext:
    """Ssl ctx permissive."""
    global _permissive_ctx, _permissive_warned
    if _permissive_ctx is None:
        with _permissive_lock:
            if _permissive_ctx is None:
                ctx = ssl.create_default_context()
                ctx.check_hostname = False
                ctx.verify_mode = ssl.CERT_NONE
                _permissive_ctx = ctx
    if not _permissive_warned:
        logger.warning(
            "[ssl_helpers] permissive SSL context requested - "
            "hostname + cert chain validation DISABLED. Reason: %s",
            reason,
        )
    _permissive_warned = True
    return _permissive_ctx
```

OP-04a

cache_manager.py

Fichier : py_modules/unifideck/core/cache_manager.py | Lignes : 171 | Depend de : (rien)

```
import json
import logging
import time
from pathlib import Path
from typing import Any
logger = logging.getLogger(__name__)
class CacheStore:
    """Cache store."""
    def __init__(self, name: str, path: Path, ttl_seconds: int = 0) -> None:
        """Initialize the instance."""
        self.name = name
        self.path = path
        self.ttl = ttl_seconds
        self._data: dict[str, Any] = {}
        self._ts: dict[str, float] = {}
        self._load()
    def get(self, key: str) -> Any | None:
        """Get."""
        if key not in self._data:
            return None
        if self.ttl > 0:
            ts = self._ts.get(key, 0)
            if time.time() > (ts + self.ttl):
                self._data.pop(key, None)
                self._ts.pop(key, None)
                return None
        return self._data[key]
    def set(self, key: str, value: Any) -> None:
        """Set."""
        self._data[key] = value
        self._ts[key] = time.time()
        self._save()
    def delete(self, key: str) -> None:
        """Delete."""
        self._data.pop(key, None)
        self._ts.pop(key, None)
        self._save()
    def clear(self) -> None:
        """Clear."""
        self._data.clear()
        self._ts.clear()
        self._save()
    def size(self) -> int:
        """Size."""
        return len(self._data)

    def _load(self) -> None:
        """Load."""
        if not self.path.exists():
            return
        try:
            raw = json.loads(
                self.path.read_text(encoding="utf-8"),
            )
```

```

        self._data = dict(raw.get("data", {}))
        self._ts = {
            k: float(v)
            for k, v in raw.get("_ts", {}).items()
        }
        return
    except (json.JSONDecodeError, OSError, ValueError) as e:
        logger.warning(
            "[CacheManager] %s corrupted (%s), trying backup",
            self.name, type(e).__name__,
        )
    bak = self.path.with_suffix(self.path.suffix + ".bak")
    if bak.exists():
        try:
            raw = json.loads(
                bak.read_text(encoding="utf-8"),
            )
            self._data = dict(raw.get("data", {}))
            self._ts = {
                k: float(v)
                for k, v in raw.get("_ts", {}).items()
            }
            logger.info(
                "[CacheManager] %s restored from backup",
                self.name,
            )
            self._save()
            return
        except (json.JSONDecodeError, OSError, ValueError):
            logger.error(
                "[CacheManager] %s backup also corrupt",
                self.name,
            )
    self._data = {}
    self._ts = {}

def _save(self) -> None:
    """Save."""
    self.path.parent.mkdir(parents=True, exist_ok=True)
    payload = {"data": self._data, "_ts": self._ts}
    if self.path.exists():
        bak = self.path.with_suffix(
            self.path.suffix + ".bak",
        )
        try:
            bak.write_bytes(self.path.read_bytes())
        except OSError as e:
            logger.warning(
                "[CacheManager] backup failed for %s: %s",
                self.name, e,
            )
    tmp = self.path.with_suffix(
        self.path.suffix + ".tmp",
    )
    try:
        tmp.write_text(
            json.dumps(
                payload, ensure_ascii=False, indent=2,
            ),
            encoding="utf-8",

```

```

        )
        tmp.replace(self.path)
        try:
            self.path.chmod(0o600)
        except OSError as e:
            logger.debug(
                "[CacheManager] chmod %s failed: %s",
                self.path, e,
            )
        except OSError as e:
            logger.error(
                "[CacheManager] write failed for %s: %s",
                self.name, e,
            )
        if tmp.exists():
            try:
                tmp.unlink()
            except OSError:
                pass

class CacheManager:
    """Cache manager."""
    def __init__(self, base_path: str) -> None:
        """Initialize the instance."""
        self.base_path = Path(base_path)
        self.base_path.mkdir(parents=True, exist_ok=True)
        self._stores: dict[str, CacheStore] = {}
    def register(self, name: str, ttl_seconds: int = 0) -> None:
        """Register."""
        if name in self._stores:
            return
        path = self.base_path / f"{name}_cache.json"
        self._stores[name] = CacheStore(name, path, ttl_seconds)
    def _get_store(self, name: str) -> CacheStore:
        """Get store."""
        if name not in self._stores:
            raise ValueError(f"Cache {name!r} not registered")
        return self._stores[name]
    def get(self, cache: str, key: str) -> Any | None:
        """Get."""
        return self._get_store(cache).get(key)
    def set(self, cache: str, key: str, value: Any) -> None:
        """Set."""
        self._get_store(cache).set(key, value)
    def delete(self, cache: str, key: str) -> None:
        """Delete."""
        self._get_store(cache).delete(key)
    def clear(self, cache: str) -> None:
        """Clear."""
        self._get_store(cache).clear()
    def clear_all(self) -> None:
        """Clear all."""
        for store in self._stores.values():
            store.clear()
    def cache_size(self, cache: str) -> int:
        """Cache size."""
        return self._get_store(cache).size()
    def registered_names(self) -> list[str]:
        """Registered names."""
        return list(self._stores.keys())

```

OP-04b

metrics_collector.py

Fichier : py_modules/unifideck/core/metrics_collector.py | Lignes : 92 | Depend de : (rien)

```
from __future__ import annotations
import logging
import time
from typing import Any
from ..core.types import Events
from ..event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
class MetricsCollector:
    """Metrics collector."""
    def __init__(self, bus: EventBus) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._counters: dict[str, int] = {}
        self._gauges: dict[str, float] = {}
        self._pending_timers: dict[str, float] = {}
        self._timers: dict[str, float] = {}
        self._started_at = time.time()
        self._subscribe_all()
    def _subscribe_all(self) -> None:
        """Subscribe all."""
        counter_events = [
            (Events.STORE_AUTH_STARTED, "auth_attempts"),
            (Events.STORE_AUTH_COMPLETE, "auth_successes"),
            (Events.STORE_AUTH_FAILED, "auth_failures"),
            (Events.SYNC_FAILED, "sync_failures"),
            (Events.DOWNLOAD_QUEUED, "download_queued"),
            (Events.DOWNLOAD_COMPLETE, "download_completed"),
            (Events.DOWNLOAD_FAILED, "download_failed"),
        ]
        for event, name in counter_events:
            self._bus.on(
                event,
                lambda n=name, **kw: self._inc_counter(n)
                from unifideck.event_bus.event_bus_devex import auto_wire
                auto_wire(self, self._bus)
                logger.info("[MetricsCollector] wired (%d counter + decorated handlers)",
                    len(counter_events))
            )
    async def stop(self) -> None:
        """Stop."""
        pass
    def get_plugin_metrics(self) -> dict[str, Any]:
        """Get plugin metrics."""
        return {
            "counters": dict(self._counters),
            "timers_ms": dict(self._timers),
            "gauges": dict(self._gauges),
            "uptime_s": int(time.time() - self._started_at),
        }
    def reset(self) -> None:
        """Reset."""
        self._counters.clear()
        self._gauges.clear()
        self._pending_timers.clear()
        self._timers.clear()
    def _inc_counter(self, name: str) -> None:
```

```

        """Inc counter."""
        self._counters[name] = self._counters.get(name, 0) + 1
def _on_auth_start(self, store: str = "", **kwargs) -> None:
    """On auth start."""
    self._pending_timers[f"auth:{store}"] = time.monotonic()
def _on_auth_complete(self, store: str = "", **kwargs) -> None:
    """On auth complete."""
    self._complete_timer(f"auth:{store}", "auth_duration_ms")

def _on_sync_start(self, **kwargs) -> None:

    """On sync start."""
    self._pending_timers["sync"] = time.monotonic()
def _on_sync_complete(self, **kwargs) -> None:
    """On sync complete."""
    self._complete_timer("sync", "sync_duration_ms")
def _on_download_start(self, store: str = "",
game_id: str = "", **kwargs) -> None:
    """On download start."""
    self._pending_timers[f"dl:{store}:{game_id}"] = time.monotonic()
def _on_download_complete(self, store: str = "",
game_id: str = "", **kwargs) -> None:
    """On download complete."""
    self._complete_timer(
        f"dl:{store}:{game_id}", "download_duration_ms",
    )
def _on_sync_gauge(self, games=None, stores_synced=None, **kw):
    """On sync gauge."""
    if games is not None:
        self._gauges["sync_games_total"] = float(len(games))
        if stores_synced is not None:
            self._gauges["sync_stores_count"] = float(len(stores_synced))
def _complete_timer(self, key: str, metric_name: str) -> None:
    """Complete timer."""
    started = self._pending_timers.pop(key, None)
    if started is None:
        return
    duration_ms = (time.monotonic() - started) * 1000
    self._timers[metric_name] = duration_ms

```


OP-04c

exe_finder.py

Fichier : py_modules/unifideck/core/exe_finder.py | Lignes : 107 | Depend de : (rien)

```
import logging
import os
from pathlib import Path
from typing import cast
logger = logging.getLogger(__name__)
WRAPPER_EXES = {
    "unitycrashhandler64.exe",
    "unitycrashhandler32.exe",
    "crashreportclient.exe",
    "crashpad_handler.exe",
    "bugreport.exe",
    "ue4prereqsetup_x64.exe",
    "dxwebsetup.exe",
    "vcredist_x64.exe",
    "vcredist_x86.exe",
    "dotnetfx35setup.exe",
    "ndp48-x86-x64-allos-enu.exe",
    "dxsetup.exe",
    "unins000.exe",
    "unins001.exe",
    "uninstall.exe",
    "installer.exe",
    "setup.exe",
    "updater.exe",
    "patcher.exe",
    "launcher.exe",
    "unrealcefsubprocess.exe",
}
class ExeFinder:
    """Exe finder."""
    def find(
        self,
        install_path: str,
        hints: list[str] | None = None,
    ) -> str | None:
        """Find."""
        if not install_path or not Path(install_path).is_dir():
            return None
        hint_lower = {h.lower() for h in hints} if hints else set()
        candidates = [
            (
                self._score_candidate(
                    path, depth, filename, hint_lower,
                ),
                path,
            )
            for path, depth, filename in (
                self._walk_exe_candidates(install_path)
            )
        ]
        return self._rank_candidates(candidates, install_path)

    def _walk_exe_candidates(
        self, install_path: str,
    ):
```

```

    """Walk exe candidates."""
    for root, dirs, files in os.walk(install_path):
        rel = os.path.relpath(root, install_path)
        depth = 0 if rel == "." else rel.count(os.sep) + 1
        if depth > 3:
            dirs.clear()
            continue
        for filename in files:
            lower = filename.lower()
            if not lower.endswith(".exe"):
                continue
            if lower in WRAPPER_EXES:
                continue
            yield str(Path(root) / filename), depth, filename
    @staticmethod
    def _score_candidate(
        full_path: str,
        depth: int,
        filename: str,
        hint_lower: set,
    ) -> int:
        """Score candidate."""
        score = 0
        if filename.lower() in hint_lower:
            score += 1000
        score += (4 - depth) * 100
        try:
            size_mb = (
                Path(full_path).stat().st_size // (1024 * 1024)
            )
            score += min(size_mb, 500)
        except OSError:
            pass
        return score
    @staticmethod
    def _rank_candidates(
        candidates: list[tuple],
        install_path: str,
    ) -> str | None:
        """Rank candidates."""
        if not candidates:
            logger.debug(
                "[ExeFinder] No .exe found in %s", install_path,
            )
            return None
        candidates.sort(key=lambda x: x[0], reverse=True)
        best_score, best_path = candidates[0]
        logger.info(
            "[ExeFinder] Best candidate (score=%d): %s",
            best_score, best_path,
        )
        return cast("str | None", best_path)
    exe_finder = ExeFinder()

```

OP-04d

sync_service.py

Fichier : py_modules/unifideck/core/sync_service.py | Lignes : 282 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import time
from typing import TYPE_CHECKING, Any
from ..event_bus import EventBus
from ..stores import StoreRegistry
from ..cross_store_dedup import deduplicate_libraries
from ..steam.owned_games import get_owned_titles as _steam_owned_titles
from .types import Events, Game, SyncResult
if TYPE_CHECKING:
    from ..config import ConfigManager
    from ..stores.shared.store_base import StoreBase
logger = logging.getLogger(__name__)
class SyncService:
    """Sync service."""
    def __init__(
        self,
        registry: StoreRegistry,
        bus: EventBus,
        config: ConfigManager | None = None,
    ) -> None:
        """Initialize the instance."""
        self._registry = registry
        self._bus = bus
        self._config = config
        self._lock = asyncio.Lock()
        self._cancel_event = asyncio.Event()
        self._all_games: dict[str, list[Game]] = {}
        self._last_sync_time: float | None = None
        self._current_store: str | None = None
    async def sync_all(self, *, force: bool = False) -> SyncResult:
        """Sync all."""
        if self._lock.locked() and not force:
            logger.warning(
                "[SyncService] sync_all() called while "
                "another sync is running – rejected",
            )
            return SyncResult(
                success=False,
                error="sync_already_running",
                games=[],
            )
        async with self._lock:
            return await self._run_sync()
    async def _run_sync(self) -> SyncResult:
        """Run sync."""
        self._cancel_event.clear()
        started = time.monotonic()
        available_stores = self._registry.available()
        total = len(available_stores)
        store_names = [s.store_name for s in available_stores]
        await self._bus.emit(
            Events.SYNC_STARTED,
            stores=store_names,

```

```

        scope="all",
    )
    logger.info("[SyncService] sync starting (%d stores)", total)
    libraries: dict[str, list[Game]] = {}
    errors: dict[str, str] = {}
    if total == 0:
        logger.warning(
            "[SyncService] no available stores – nothing to sync",
        )
        await self._bus.emit(
            Events.SYNC_COMPLETE, games=[], stores_synced=[],
        )
        return SyncResult(
            success=True, games=[], count=0, duration_ms=0,
        )
    for idx, store in enumerate(available_stores):
        if self._cancel_event.is_set():
            logger.info(
                "[SyncService] sync cancelled at store %d/%d",
                idx, total,
            )
            await self._bus.emit(Events.SYNC_CANCELLED)
            return SyncResult(
                success=False,
                error="cancelled",
                games=self._flatten(libraries),
            )
        self._current_store = store.store_name
        await self._emit_progress(store.store_name, idx, total)
        games, err = await self._sync_one_store(store)
        libraries[store.store_name] = games
        if err is not None:
            errors[store.store_name] = err
    self._current_store = None
    duration_ms = int((time.monotonic() - started) * 1000)
    libraries = await self._apply_dedup_and_emit(libraries)
    self._all_games = libraries
    self._last_sync_time = time.time()
    result = self._aggregate_results(
        libraries, errors, duration_ms, total,
    )
    await self._bus.emit(
        Events.SYNC_COMPLETE,
        games=result.games,
        stores_synced=list(libraries.keys()),
        errors=errors,
        duration_ms=duration_ms,
    )
    return result
async def _apply_dedup_and_emit(
    self, libraries: dict[str, list[Game]],
) -> dict[str, list[Game]]:
    """Apply dedup and emit."""
    tracked = self._tracked_stores()
    steam_owned = _steam_owned_titles(self._config)
    deduped, dropped_per_store = deduplicate_libraries(
        libraries,
        tracked_stores=tracked,
        steam_owned_titles=steam_owned,
    )

```

```

total_dropped = sum(dropped_per_store.values())
if total_dropped:
    await self._bus.emit(
        Events.SYNC_DEDUP,
        total_removed=total_dropped,
        per_store=dict(dropped_per_store),
    )
return deduped

def _tracked_stores(self) -> tuple[str, ...]:

    """Tracked stores."""
    default = ("epic", "gog", "amazon", "ubisoft")
    if self._config is None:
        return default
    try:
        value = self._config.get("dedup.tracked_stores", list(default))
    except Exception:
        return default
    if not isinstance(value, (list, tuple)):
        logger.warning(
            "[SyncService] dedup.tracked_stores has wrong type "
            "(%s); falling back to defaults",
            type(value).__name__,
        )
        return default
    return tuple(value)

async def sync_single_store(
    self, store_name: str,
) -> tuple[bool, str | None]:
    """Sync single store."""
    store = self._registry.get_store(store_name)
    if store is None:
        logger.warning(
            "[SyncService] refresh-library: unknown store %r",
            store_name,
        )
        return False, "unknown_store"
    await self._bus.emit(
        Events.SYNC_STARTED,
        stores=[store_name],
        scope="single",
    )
    await self._emit_progress(store_name, 0, 1)
    games, err = await self._sync_one_store(store)
    if self._all_games is None:
        self._all_games = {}
    self._all_games[store_name] = games
    self._all_games = await self._apply_dedup_and_emit(self._all_games)
    self._last_sync_time = time.time()
    await self._bus.emit(
        Events.SYNC_COMPLETE,
        games=self._flatten(self._all_games),
        stores_synced=[store_name],
        errors={store_name: err} if err else {},
        duration_ms=0,
    )
    return err is None, err

async def _sync_one_store(
    self, store: StoreBase,

```

```

) -> tuple[list[Game], str | None]:

    """Sync one store."""
    try:
        games = await store.get_library()
        if games is None:
            games = []
        logger.info(
            "[SyncService] %s: %d games",
            store.store_name, len(games),
        )
        return games, None
    except Exception as e:
        logger.exception(
            "[SyncService] %s sync failed: %s",
            store.store_name, e,
        )
        await self._bus.emit(
            Events.SYNC_FAILED,
            store=store.store_name,
            error=str(e),
        )
        await self._bus.emit(
            Events.LAUNCHER_STAGE,
            severity="warning",
            i18n_key="toasts.library.syncStoreFailed",
            i18n_params={
                "store": store.store_name,
                "error": str(e)[:120],
            },
            duration_ms=8000,
            action={
                "i18n_label_key": "toasts.actions.retryLibrarySync",
                "target_url": (
                    f"unifideck://refresh-library/{store.store_name}"
                ),
            },
            store=store.store_name,
        )
        return [], str(e)
    async def _emit_progress(
        self, store_name: str, idx: int, total: int,
    ) -> None:
        """Emit progress."""
        await self._bus.emit(
            Events.SYNC_PROGRESS,
            store=store_name,
            progress=int((idx / total) * 100) if total else 0,
            current=idx + 1,
            total=total,
        )
    def _aggregate_results(
        self,
        libraries: dict[str, list[Game]],
        errors: dict[str, str],
        duration_ms: int,
        total_stores: int,
    ) -> SyncResult:
        """Aggregate results."""
        merged = self._flatten(libraries)
        logger.info(

```

```

        "[SyncService] sync complete – %d games across %d stores "
        "in %dms (%d errors)",
        len(merged), len(libraries), duration_ms, len(errors),
    )
    return SyncResult(
        success=len(errors) < total_stores,
        games=merged,
        count=len(merged),
        duration_ms=duration_ms,
        error=None if not errors else f"{len(errors)}_stores_failed",
    )
async def cancel(self) -> bool:
    """Check whether ncel."""
    if not self._lock.locked():
        return False
    self._cancel_event.set()
    logger.info("[SyncService] cancel requested")
    return True

def get_status(self) -> dict[str, Any]:
    """Get status."""
    return {
        "syncing": self._lock.locked(),
        "current_store": self._current_store,
        "last_sync_time": self._last_sync_time,
        "total_games": sum(
            len(g) for g in self._all_games.values()
        ),
    }
def get_all_games(self) -> list[Game]:
    """Get all games."""
    return self._flatten(self._all_games)
def get_games_by_store(self, store: str) -> list[Game]:
    """Get games by store."""
    return list(self._all_games.get(store, []))
def get_game_info(self, app_id: int) -> dict[str, Any] | None:
    """Get game info."""
    for games in self._all_games.values():
        for game in games:
            if game.app_id == app_id:
                from dataclasses import asdict
                return asdict(game)
    return None
@staticmethod
def _flatten(libraries: dict[str, list[Game]]) -> list[Game]:
    """Flatten."""
    merged: list[Game] = []
    for games in libraries.values():
        merged.extend(games)
    return merged

```

OP-04d2

cross_store_dedup.py

Fichier : py_modules/unifideck/core/cross_store_dedup.py | Lignes : 128 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from ..metadata.unifidb import normalize_title_for_matching
if TYPE_CHECKING:
    from .types import Game
logger = logging.getLogger(__name__)
def deduplicate_libraries(
    libraries: dict[str, list[Game]],
    *,
    tracked_stores: tuple[str, ...] | list[str],
    steam_owned_titles: frozenset[str] | None = None,
) -> tuple[dict[str, list[Game]], dict[str, int]]:
    """Deduplicate libraries."""
    if not libraries:
        return {}, {}
    tracked_set: frozenset[str] = frozenset(tracked_stores)
    deduped: dict[str, list[Game]] = {}
    dropped: dict[str, int] = {}
    if steam_owned_titles:
        libraries = _filter_against_steam(
            libraries, steam_owned_titles, tracked_set, dropped,
        )
    for store_name, games in libraries.items():
        if store_name not in tracked_set:
            deduped[store_name] = list(games)
    claimed: dict[str, _ClaimedEntry] = {}
    for store_name in _ordered_store_names(libraries, tracked_stores):
        kept, dropped_count = _dedup_store_games(
            store_name, libraries[store_name],
            deduped, claimed, dropped,
        )
        deduped[store_name] = kept
        if dropped_count:
            dropped[store_name] = dropped.get(store_name, 0) + dropped_count
    total = sum(dropped.values())
    if total:
        logger.info(
            "[cross_store_dedup] removed %d duplicate(s) across "
            "%d store(s)",
            total, len(dropped),
        )
    return deduped, dropped
class _ClaimedEntry:
    """Claimed entry."""
    __slots__ = ("store", "installed")
    def __init__(self, *, store: str, installed: bool) -> None:
        """Initialize the instance."""
        self.store = store
        self.installed = installed

def _filter_against_steam(
    libraries: dict[str, list[Game]],
    steam_owned: frozenset[str],
    tracked: frozenset[str],

```



```

        dropped: dict[str, int],
    ) -> dict[str, list[Game]]:

        """Filter against steam."""
        filtered: dict[str, list[Game]] = {}
        for store_name, games in libraries.items():
            if store_name not in tracked:
                filtered[store_name] = list(games)
                continue
            kept: list[Game] = []
            dropped_here = 0
            for game in games:
                key = normalize_title_for_matching(game.title)
                if key and key in steam_owned:
                    dropped_here += 1
                    continue
                kept.append(game)
            filtered[store_name] = kept
            if dropped_here:
                dropped[store_name] = dropped.get(store_name, 0) + dropped_here
        return filtered
def _dedup_store_games(
    store_name: str,
    games: list[Game],
    deduped: dict[str, list[Game]],
    claimed: dict[str, _ClaimedEntry],
    dropped: dict[str, int],
) -> tuple[list[Game], int]:
    """Dedup store games."""
    kept: list[Game] = []
    dropped_here = 0
    for game in games:
        key = normalize_title_for_matching(game.title)
        if not key:
            kept.append(game)
            continue
        existing = claimed.get(key)
        if existing is None:
            claimed[key] = _ClaimedEntry(
                store=store_name, installed=game.installed,
            )
            kept.append(game)
            continue
        if _current_wins(game, existing):
            _evict_from_store(deduped, existing.store, key)
            dropped[existing.store] = dropped.get(existing.store, 0) + 1
            claimed[key] = _ClaimedEntry(
                store=store_name, installed=True,
            )
            kept.append(game)
        else:
            dropped_here += 1
    return kept, dropped_here
def _current_wins(game: Game, existing: _ClaimedEntry) -> bool:
    """Current wins."""
    return game.installed and not existing.installed
def _ordered_store_names(
    libraries: dict[str, list[Game]],
    tracked_stores: tuple[str, ...] | list[str],
) -> list[str]:
    """Ordered store names."""

```

```
        return [name for name in tracked_stores if name in libraries]
def _evict_from_store(
    deduped: dict[str, list[Game]],
    store_name: str,
    title_key: str,
) -> None:
    """Evict from store."""
    games = deduped.get(store_name)
    if not games:
        return
    for idx, game in enumerate(games):
        if normalize_title_for_matching(game.title) == title_key:
            del games[idx]
    return
```

OP-04e

manifest.py

Fichier : py_modules/unifideck/core/manifest.py | Lignes : 213 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
from dataclasses import dataclass, field
from datetime import UTC, datetime
from pathlib import Path
from typing import TYPE_CHECKING, Any, cast
from ..core.types import Events
from ..event_bus.event_bus import EventBus
from ..utils.config_helpers import get_cfg
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
DEFAULT_MANIFEST_FILENAME = ".unifideck_manifest.json"
@dataclass
class GameManifest:
    """Game manifest."""
    unifideck_version: str
    store: str
    store_id: str
    title: str
    executable_relative: str
    installed_at: str
    platform: str = "windows"
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        return {
            "unifideck_version": self.unifideck_version,
            "store": self.store,
            "store_id": self.store_id,
            "title": self.title,
            "executable_relative": self.executable_relative,
            "installed_at": self.installed_at,
            "platform": self.platform,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> GameManifest | None:
        """From dict."""
        try:
            return cls(
                unifideck_version=data["unifideck_version"],
                store=data["store"],
                store_id=data["store_id"],
                title=data.get("title", ""),
                executable_relative=data.get("executable_relative", ""),
                installed_at=data.get("installed_at", ""),
                platform=data.get("platform", "windows"),
            )
        except (KeyError, TypeError):
            return None
@dataclass
class DiscoveryResult:
    """Discovery result."""
    scanned_directories: int = 0

```

```

manifests_found: int = 0
games_registered: int = 0
errors: list[str] = field(default_factory=list)
def to_dict(self) -> dict[str, Any]:
    """To dict."""
    return {
        "scanned_directories": self.scanned_directories,
        "manifests_found": self.manifests_found,
        "games_registered": self.games_registered,
        "errors": list(self.errors),
    }

def build_manifest(
    store: str,
    store_id: str,
    title: str,
    executable_relative: str,
    platform: str = "windows",
    unifideck_version: str = "1.0",
) -> GameManifest:
    """Build manifest."""
    return GameManifest(
        unifideck_version=unifideck_version,
        store=store,
        store_id=store_id,
        title=title,
        executable_relative=executable_relative,
        installed_at=datetime.now(UTC).isoformat(),
        platform=platform,
    )

def _cfg(config: ConfigManager | None, key: str, default: Any) -> Any:
    """Cfg."""
    return get_cfg(config, key, default)

async def write_manifest(
    install_dir: str,
    store: str,
    store_id: str,
    title: str,
    executable_relative: str,
    platform: str = "windows",
    config: ConfigManager | None = None,
) -> bool:
    """Write manifest."""
    manifest = build_manifest(
        store, store_id, title, executable_relative, platform,
    )
    filename = get_cfg(
        config, "discovery.manifest_filename",
        DEFAULT_MANIFEST_FILENAME,
    )
    path = Path(install_dir) / filename
    def _write_sync() -> None:
        """Write sync."""
        with path.open("w", encoding="utf-8") as f:
            json.dump(manifest.to_dict(), f, indent=2)
    try:
        await asyncio.to_thread(_write_sync)
        logger.info(
            "[discovery] wrote manifest %s:%s → %s",
            store, store_id, path,

```

```

        )
        return True
    except OSError as e:
        logger.error(
            "[discovery] write_manifest %s:%s failed: %s",
            store, store_id, e,
        )
        return False
    async def read_manifest(
        game_dir: str, config: ConfigManager | None = None,
    ) -> GameManifest | None:
        """Read manifest."""
        filename = get_cfg(
            config, "discovery.manifest_filename",
            DEFAULT_MANIFEST_FILENAME,
        )
        path = Path(game_dir) / filename
        def _read_sync() -> dict[str, Any] | None:
            """Read sync."""
            if not path.is_file():
                return None
            try:
                with path.open(encoding="utf-8") as f:
                    return cast("dict[str, Any] | None", json.load(f))
            except (OSError, json.JSONDecodeError) as e:
                logger.debug("[discovery] read %s failed: %s", path, e)
                return None
        raw = await asyncio.to_thread(_read_sync)
        if raw is None:
            return None
        return GameManifest.from_dict(raw)

    async def discover_all(
        bus: EventBus | None = None,
        config: ConfigManager | None = None,
    ) -> DiscoveryResult:
        """Discover all."""
        from ..utils.paths import get_all_game_directories
        result = DiscoveryResult()
        roots = await asyncio.to_thread(get_all_game_directories, config)
        result.scanned_directories = len(roots)
        logger.info("[discovery] scanning %d roots", len(roots))
        for root in roots:
            try:
                await _scan_one_root(root, bus, result, config)
            except OSError as e:
                result.errors.append(f"{root}: {e}")
        logger.info(
            "[discovery] done – %d manifests, %d games registered, %d errors",
            result.manifests_found, result.games_registered,
            len(result.errors),
        )
        return result
    async def _scan_one_root(
        root: str,
        bus: EventBus | None,
        result: DiscoveryResult,
        config: ConfigManager | None,
    ) -> None:
        """Scan one root."""

```

```

def _list(p: str) -> list[str]:
    """List."""
    root_path = Path(p)
    return [
        str(entry)
        for entry in root_path.iterdir()
        if entry.is_dir()
    ]
try:
    subdirs = await asyncio.to_thread(_list, root)
except OSError:
    return
for game_dir in subdirs:
    manifest = await read_manifest(game_dir, config)
    if manifest is None:
        continue
    result.manifests_found += 1
    if bus is not None:
        try:
            await bus.emit(
                Events.GAME_INSTALLED,
                store=manifest.store,
                game_id=manifest.store_id,
                title=manifest.title,
                install_path=game_dir,
                executable=manifest.executable_relative,
            )
            result.games_registered += 1
        except (RuntimeError, asyncio.CancelledError,
                AttributeError, OSError) as e:
            result.errors.append(
                f"{manifest.store}:{manifest.store_id}: {e}",
            )
async def discover_installed_games(registry=None, bus=None, config=None):
    """Discover installed games."""
    result = await discover_all(bus=bus, config=config)
    return result.to_dict() if hasattr(result, "to_dict") else result
async def discover_and_log(bus=None, config=None):
    """Discover and log."""
    return await discover_all(bus=bus, config=config)

```

OP-04f

paths.py

Fichier : py_modules/unifideck/core/paths.py | Lignes : 37 | Depend de : (rien)

```
from __future__ import annotations
import logging
import os
from pathlib import Path
logger = logging.getLogger(__name__)
_DECKY_DEFAULT_PATH = Path.home() / "homebrew" / "plugins" / "unifideck"
def resolve_plugin_dir(start: Path | None = None) -> Path:
    """Resolve plugin dir."""
    explicit = os.environ.get("UNIFIDECK_PLUGIN_DIR")
    if explicit:
        p = Path(explicit).expanduser()
        if p.is_dir():
            return p
        logger.warning(
            "[paths] UNIFIDECK_PLUGIN_DIR=%s is not a "
            "directory, ignoring",
            explicit,
        )
    decky = os.environ.get("DECKY_PLUGIN_DIR")
    if decky:
        p = Path(decky).expanduser()
        if p.is_dir():
            return p
    if _DECKY_DEFAULT_PATH.is_dir():
        return _DECKY_DEFAULT_PATH
    here = (start or Path(__file__)).resolve()
    for parent in here.parents:
        if (parent / "plugin.json").is_file():
            return parent
    logger.warning(
        "[paths] plugin directory could not be resolved, "
        "falling back to %s", _DECKY_DEFAULT_PATH,
    )
    return _DECKY_DEFAULT_PATH
def resolve_py_modules_dir() -> Path:
    """Resolve py modules dir."""
    return resolve_plugin_dir() / "py_modules"
```

OP-09

`__init__.py`

Fichier : `py_modules/unifideck/event_bus/__init__.py` | Lignes : 52 | Depend de : (rien)

```
from .event_bus import EventBus
from .event_bus_devex import (
    SchemaExtractor,
    auto_wire,
    subscribe,
)
from .event_bus_extensions import (
    DeadLetterQueue,
    DebugSnapshot,
    EventSchema,
    PredicateFilter,
    TypedEventRegistry,
)
from .event_bus_reliability import CircuitBreaker
from .event_bus_scaling import BatchDispatcher
from .event_priority import (
    EventPriority,
    get_coalesce_key,
    get_priority,
)
from .event_replay import EventReplayBuffer
from .priority_dispatcher import PriorityDispatcher
from .supervision.metrics_handler import (
    HandlerLatencyCollector,
    HandlerLatencyStats,
)
from .supervision.watchdog_handler import (
    HandlerQuarantinedError,
    HandlerWatchdog,
)
__all__ = [
    "EventBus",
    "EventPriority",
    "get_priority",
    "get_coalesce_key",
    "PriorityDispatcher",
    "HandlerWatchdog",
    "HandlerQuarantinedError",
    "HandlerLatencyCollector",
    "HandlerLatencyStats",
    "EventReplayBuffer",
    "DeadLetterQueue",
    "PredicateFilter",
    "TypedEventRegistry",
    "EventSchema",
    "DebugSnapshot",
    "CircuitBreaker",
    "BatchDispatcher",
    "subscribe",
    "auto_wire",
    "SchemaExtractor",
]
```


OP-10

__init__.py

Fichier : py_modules/unifideck/event_bus/supervision/__init__.py | Lignes : 3 | Depend de : (rien)

```
from __future__ import annotations
from .metrics_handler import HandlerLatencyCollector
from .watchdog_handler import HandlerWatchdog
```

OP-10a

watchdog_handler.py

Fichier : py_modules/unifideck/event_bus/supervision/watchdog_handler.py | Lignes : 137 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from collections.abc import Callable
from dataclasses import dataclass
from typing import Any
logger = logging.getLogger(__name__)
DEFAULT_HANDLER_TIMEOUT_SEC = 5.0
DEFAULT_QUARANTINE_THRESHOLD = 10
@dataclass
class HandlerTimeoutMetrics:
    """Handler timeout metrics."""
    name: str
    invocations: int = 0
    timeouts: int = 0
    consecutive_timeouts: int = 0
    quarantined: bool = False
    last_error: str | None = None
class HandlerWatchdog:
    """Handler watchdog."""
    def __init__(
        self,
        *,
        default_timeout: float = DEFAULT_HANDLER_TIMEOUT_SEC,
        quarantine_threshold: int = DEFAULT_QUARANTINE_THRESHOLD,
    ) -> None:
        """Initialize the instance."""
        self._default_timeout = default_timeout
        self._quarantine_threshold = quarantine_threshold
        self._metrics: dict[str, HandlerTimeoutMetrics] = {}
        self._timeouts: dict[str, float] = {}
    def register(
        self,
        handler_name: str,
        timeout: float | None = None,
    ) -> None:
        """Register."""
        if timeout is not None:
            self._timeouts[handler_name] = timeout
        if handler_name not in self._metrics:
            self._metrics[handler_name] = HandlerTimeoutMetrics(
                name=handler_name,
            )
    def unregister(self, handler_name: str) -> None:
        """Unregister."""
        self._timeouts.pop(handler_name, None)
        m = self._metrics.get(handler_name)
        if m is not None:
            m.quarantined = False
            m.consecutive_timeouts = 0
    async def invoke(
        self,
        handler_name: str,
        handler: Callable[..., Any],

```

```

    *args: Any,
    **kwargs: Any,
) -> Any:

    """Invoke."""
    metrics = self._metrics.setdefault(
        handler_name, HandlerTimeoutMetrics(name=handler_name),
    )
    if metrics.quarantined:
        raise HandlerQuarantinedError(handler_name)
    timeout = self._timeouts.get(handler_name, self._default_timeout)
    metrics.invocations += 1
    try:
        result = await asyncio.wait_for(
            handler(*args, **kwargs),
            timeout=timeout,
        )
        metrics.consecutive_timeouts = 0
        metrics.last_error = None
        return result
    except TimeoutError:
        self._record_timeout(metrics, timeout)
        raise

def release_quarantine(self, handler_name: str) -> bool:
    """Release quarantine."""
    m = self._metrics.get(handler_name)
    if m is None or not m.quarantined:
        return False
    m.quarantined = False
    m.consecutive_timeouts = 0
    logger.info(
        "[HandlerWatchdog] released %s from quarantine",
        handler_name,
    )
    return True

def quarantine_preemptive(
    self, handler_name: str, reason: str = "preemptive",
) -> bool:
    """Quarantine preemptive."""
    metrics = self._metrics.setdefault(
        handler_name, HandlerTimeoutMetrics(name=handler_name),
    )
    if metrics.quarantined:
        return False
    metrics.quarantined = True
    metrics.last_error = f"quarantined preemptively: {reason}"
    logger.warning(
        "[HandlerWatchdog] %s quarantined preemptively (%s)",
        handler_name, reason,
    )
    return True

def get_metrics(self) -> dict[str, HandlerTimeoutMetrics]:
    """Get metrics."""
    return dict(self._metrics)

def _record_timeout(
    self,
    metrics: HandlerTimeoutMetrics,
    timeout: float,
) -> None:
    """Record timeout."""
    metrics.timeouts += 1

```

```
metrics.consecutive_timeouts += 1
metrics.last_error = f"timeout after {timeout:.1f}s"
logger.warning(
    "[HandlerWatchdog] %s timed out (%d/%d consecutive)",
    metrics.name,
    metrics.consecutive_timeouts,
    self._quarantine_threshold,
)
if metrics.consecutive_timeouts >= self._quarantine_threshold:
    metrics.quarantined = True
    logger.error(
        "[HandlerWatchdog] QUARANTINED %s after %d "
        "consecutive timeouts – will be skipped until "
        "release_quarantine() is called",
        metrics.name,
        metrics.consecutive_timeouts,
    )
```

```
class HandlerQuarantinedError(Exception):
```

```
    """Handler quarantined error."""
    def __init__(self, handler_name: str) -> None:
        """Initialize the instance."""
        super().__init__(f"handler {handler_name} is quarantined")
        self.handler_name = handler_name
```

OP-10b

metrics_handler.py

Fichier : py_modules/unifideck/event_bus/supervision/metrics_handler.py | Lignes : 82 | Depend de : (rien)

```

from __future__ import annotations
import statistics
from collections import deque
from dataclasses import dataclass, field
from typing import Any
ROLLING_WINDOW_SIZE = 100
@dataclass
class HandlerLatencyStats:
    """Handler latency stats."""
    name: str
    invocations: int = 0
    total_ms: float = 0.0
    max_ms: float = 0.0
    p50_ms: float = 0.0
    p95_ms: float = 0.0
    _window: deque[float] = field(
        default_factory=lambda: deque(maxlen=ROLLING_WINDOW_SIZE),
    )
    def record(self, duration_ms: float) -> None:
        """Record."""
        self.invocations += 1
        self.total_ms += duration_ms
        if duration_ms > self.max_ms:
            self.max_ms = duration_ms
        self._window.append(duration_ms)
        self._recompute_percentiles()
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        avg = self.total_ms / self.invocations if self.invocations else 0.0
        return {
            "name": self.name,
            "invocations": self.invocations,
            "total_ms": round(self.total_ms, 2),
            "avg_ms": round(avg, 2),
            "max_ms": round(self.max_ms, 2),
            "p50_ms": round(self.p50_ms, 2),
            "p95_ms": round(self.p95_ms, 2),
        }
    def _recompute_percentiles(self) -> None:
        """Recompute percentiles."""
        n = len(self._window)
        if n == 0:
            return
        if n == 1:
            self.p50_ms = self.p95_ms = self._window[0]
            return
        qs = statistics.quantiles(self._window, n=20)
        self.p50_ms = qs[9]
        self.p95_ms = qs[18]
class HandlerLatencyCollector:
    """Handler latency collector."""
    def __init__(self) -> None:
        """Initialize the instance."""
        self._stats: dict[str, HandlerLatencyStats] = {}
    def record(self, handler_name: str, duration_ms: float) -> None:

```

```
    """Record."""
    stats = self._stats.get(handler_name)
    if stats is None:
        stats = HandlerLatencyStats(name=handler_name)
        self._stats[handler_name] = stats
    stats.record(duration_ms)
def get_snapshot(self) -> dict[str, dict[str, float]]:
    """Get snapshot."""
    return {
        name: stats.to_dict() for name, stats in self._stats.items()
    }

def get_top_n(self, n: int = 10) -> dict[str, dict[str, float]]:

    """Get top n."""
    sorted_stats = sorted(
        self._stats.values(),
        key=lambda s: s.p95_ms,
        reverse=True,
    )
    return {s.name: s.to_dict() for s in sorted_stats[:n]}
def reset(self, handler_name: str) -> bool:
    """Reset."""
    if handler_name in self._stats:
        self._stats[handler_name] = HandlerLatencyStats(
            name=handler_name,
        )
    return True
return False
```

OP-09a

event_bus.py

Fichier : py_modules/unifideck/event_bus/event_bus.py | Lignes : 107 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import inspect
import logging
import time
from collections.abc import Awaitable, Callable
from typing import Any
from ..core.types import Events

logger = logging.getLogger(__name__)
Handler = Callable[..., Awaitable[Any]] | Callable[..., Any]

class EventBus:
    """Event bus."""
    def __init__(self) -> None:
        """Initialize the instance."""
        self._handlers: dict[str, list[Handler]] = {}
        self._once: dict[str, list[Handler]] = {}
    def on(self, event: Events | str, handler: Handler) -> None:
        """On."""
        key = self._key(event)
        self._handlers.setdefault(key, []).append(handler)
        logger.debug("[EventBus] on(%s) -> %s handlers",
            key, len(self._handlers[key]))
    def once(self, event: Events | str, handler: Handler) -> None:
        """Once."""
        key = self._key(event)
        self._handlers.setdefault(key, []).append(handler)
        self._once.setdefault(key, []).append(handler)
        logger.debug("[EventBus] once(%s)", key)
    def off(self, event: Events | str, handler: Handler) -> bool:
        """Off."""
        key = self._key(event)
        if key not in self._handlers:
            return False
        try:
            self._handlers[key].remove(handler)
        except ValueError:
            return False
        if key in self._once and handler in self._once[key]:
            self._once[key].remove(handler)
        return True
    def clear(self, event: Events | None = None) -> None:
        """Clear."""
        if event is None:
            self._handlers.clear()
            self._once.clear()
            logger.debug("[EventBus] cleared all handlers")
        else:
            key = self._key(event)
            self._handlers.pop(key, None)
            self._once.pop(key, None)
            logger.debug("[EventBus] cleared %s", key)
    def handler_count(self, event: Events | str) -> int:
        """Handler count."""
        return len(self._handlers.get(self._key(event), []))

```

```

async def emit(self, event: Events | str, **payload: Any) -> list[Any]:
    """Emit."""
    key = self._key(event)
    handlers = list(self._handlers.get(key, []))
    if not handlers:
        logger.debug("[DIAG] event=%s handlers=0", key)
        return []
    started = time.monotonic()
    logger.debug(
        "[DIAG] event=%s handlers=%d payload_keys=%s",
        key, len(handlers), list(payload.keys()),
    )
    tasks = [self._invoke(h, payload) for h in handlers]
    results = await asyncio.gather(
        *tasks, return_exceptions=True,
    )
    once_list = self._once.get(key, [])
    if once_list:
        remaining = [
            h for h in self._handlers[key]
            if h not in once_list
        ]
        self._handlers[key] = remaining
        self._once[key] = []
    dt_total = (time.monotonic() - started) * 1000
    ok = sum(
        1 for r in results
        if not isinstance(r, Exception)
    )
    for i, r in enumerate(results):
        if isinstance(r, Exception):
            logger.error(
                "[EventBus] handler #%d for %s failed: "
                "%s: %s",
                i, key, type(r).__name__, r,
            )
    logger.debug(
        "[DIAG] event=%s total=%.2fms success=%d/%d",
        key, dt_total, ok, len(results),
    )
    return results
async def _invoke(self, handler: Handler,
    payload: dict[str, Any]) -> Any:
    """Invoke."""
    if inspect.iscoroutinefunction(handler):
        return await handler(**payload)
    return await asyncio.to_thread(handler, **payload)
@staticmethod
def _key(event: Events | str) -> str:
    """Key."""
    if isinstance(event, Events):
        return event.value
    return str(event)

```


OP-09b

event_priority.py

Fichier : py_modules/unifideck/event_bus/event_priority.py | Lignes : 59 | Depend de : (rien)

```
from __future__ import annotations
from enum import IntEnum
from ..core.types import Events
class EventPriority(IntEnum):
    """Event priority."""
    CRITICAL = 0
    NORMAL = 1
    BACKGROUND = 2
_DEFAULT_PRIORITY: dict[Events, EventPriority] = {
    Events.PLUGIN_LOADED: EventPriority.CRITICAL,
    Events.PLUGIN_UNLOADING: EventPriority.CRITICAL,
    Events.GAME_LAUNCHED: EventPriority.CRITICAL,
    Events.GAME_STOPPED: EventPriority.CRITICAL,
    Events.GAME_INSTALLED: EventPriority.NORMAL,
    Events.GAME_UNINSTALLED: EventPriority.NORMAL,
    Events.GAME_UPDATE_AVAILABLE: EventPriority.BACKGROUND,
    Events.SYNC_STARTED: EventPriority.NORMAL,
    Events.SYNC_COMPLETE: EventPriority.NORMAL,
    Events.SYNC_CANCELLED: EventPriority.NORMAL,
    Events.SYNC_FAILED: EventPriority.NORMAL,
    Events.SYNC_PROGRESS: EventPriority.BACKGROUND,
    Events.STORE_AUTH_STARTED: EventPriority.NORMAL,
    Events.STORE_AUTH_COMPLETE: EventPriority.NORMAL,
    Events.STORE_AUTH_FAILED: EventPriority.NORMAL,
    Events.STORE_LOGOUT: EventPriority.NORMAL,
    Events.DOWNLOAD_QUEUED: EventPriority.NORMAL,
    Events.DOWNLOAD_STARTED: EventPriority.NORMAL,
    Events.DOWNLOAD_COMPLETE: EventPriority.NORMAL,
    Events.DOWNLOAD_CANCELLED: EventPriority.NORMAL,
    Events.DOWNLOAD_FAILED: EventPriority.NORMAL,
    Events.DOWNLOAD_PROGRESS: EventPriority.BACKGROUND,
    Events.STORE_ERROR: EventPriority.BACKGROUND,
}
COALESCE_KEY: dict[Events, str] = {
    Events.SYNC_PROGRESS: "store",
    Events.DOWNLOAD_PROGRESS: "download_id",
}
def get_priority(
    event: Events | str,
) -> EventPriority:
    """Get priority."""
    if isinstance(event, Events):
        return _DEFAULT_PRIORITY.get(event, EventPriority.NORMAL)
    try:
        resolved = Events(event)
        return _DEFAULT_PRIORITY.get(resolved, EventPriority.NORMAL)
    except ValueError:
        return EventPriority.NORMAL
def get_coalesce_key(
    event: Events | str,
) -> str:
    """Get coalesce key."""
    if isinstance(event, Events):
        return COALESCE_KEY.get(event, "")
    try:
```

```
        resolved = Events(event)
        return COALESCE_KEY.get(resolved, "")
    except ValueError:
        return ""
```

OP-09c

priority_dispatcher.py

Fichier : py_modules/unifideck/event_bus/priority_dispatcher.py | Lignes : 242 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import time
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
from ..core.types import Events
from .event_priority import (
    EventPriority,
    get_coalesce_key,
    get_priority,
)
if TYPE_CHECKING:
    from .event_bus import EventBus
    from .event_bus_scaling import BatchDispatcher
    from .event_replay import EventReplayBuffer
    from .supervision.watchdog_handler import HandlerWatchdog
logger = logging.getLogger(__name__)
DEFAULT_BACKGROUND_CAP = 500
DROP_WARNING_INTERVAL_SEC = 60.0
@dataclass(order=True)
class _QueueItem:
    """Queue item."""
    priority: int
    seq: int
    event: Events | str | None = field(compare=False)
    kwargs: dict[str, Any] = field(compare=False)
    dropped: bool = field(default=False, compare=False)
@dataclass
class DispatcherMetrics:
    """Dispatcher metrics."""
    emitted_total: int = 0
    dispatched_total: int = 0
    coalesced_total: int = 0
    dropped_background_total: int = 0
    pending_by_priority: dict[str, int] = field(
        default_factory=lambda: {"CRITICAL": 0, "NORMAL": 0, "BACKGROUND": 0},
    )
class PriorityDispatcher:

    """Priority dispatcher."""
    def __init__(
        self,
        bus: EventBus,
        *,
        background_cap: int = DEFAULT_BACKGROUND_CAP,
        watchdog: HandlerWatchdog | None = None,
        latency_collector: Any = None,
        replay_buffer: EventReplayBuffer | None = None,
        batch_dispatcher: BatchDispatcher | None = None,
    ) -> None:

        """Initialize the instance."""
        self._bus = bus
        self._background_cap = background_cap

```

```

self._watchdog = watchdog
self._latency = latency_collector
self._replay = replay_buffer
self._batcher = batch_dispatcher
self._queue: asyncio.PriorityQueue[_QueueItem] = (
    asyncio.PriorityQueue()
)
self._coalesce_map: dict[tuple[str, str], _QueueItem] = {}
self._seq = 0
self._metrics = DispatcherMetrics()
self._last_drop_warn: float = 0.0
self._worker_task: asyncio.Task | None = None
self._stopping = False
async def start(self) -> None:
    """Start."""
    if self._worker_task is not None and not self._worker_task.done():
        return
    self._stopping = False
    self._worker_task = asyncio.create_task(
        self._worker(),
        name="priority-dispatcher",
    )
async def stop(self) -> None:
    """Stop."""
    self._stopping = True
    if self._worker_task is None:
        return
    await self._queue.put(
        _QueueItem(
            priority=int(EventPriority.CRITICAL),
            seq=-1,
            event=None,
            kwargs={},
        ),
    )
    try:
        await asyncio.wait_for(self._worker_task, timeout=5.0)
    except TimeoutError:
        logger.warning(
            "[PriorityDispatcher] worker did not stop in 5s – "
            "cancelling",
        )
        self._worker_task.cancel()
    self._worker_task = None
def enqueue(
    self,
    event: Events | str,
    *,
    priority: EventPriority | None = None,
    **kwargs: Any,
) -> bool:
    """Enqueue."""
    self._metrics.emitted_total += 1
    prio = priority if priority is not None else get_priority(event)
    if self._is_saturated(prio):
        self._record_drop()
        return False
    if self._coalesce_if_possible(event, prio, kwargs):
        return True
    self._push(event, prio, kwargs)
    return True

```

```

def get_metrics(self) -> DispatcherMetrics:

    """Get metrics."""
    pending = {"CRITICAL": 0, "NORMAL": 0, "BACKGROUND": 0}
    for item in list(self._queue._queue):
        if item.dropped:
            continue
        name = EventPriority(item.priority).name
        pending[name] = pending.get(name, 0) + 1
    self._metrics.pending_by_priority = pending
    return self._metrics

def _is_saturated(self, prio: EventPriority) -> bool:
    """Is saturated."""
    if prio != EventPriority.BACKGROUND:
        return False
    pending_bg = sum(
        not item.dropped
        and item.priority == int(EventPriority.BACKGROUND)
        for item in list(self._queue._queue)
    )
    return pending_bg >= self._background_cap

def _record_drop(self) -> None:
    """Record drop."""
    self._metrics.dropped_background_total += 1
    now = time.monotonic()
    if now - self._last_drop_warn >= DROP_WARNING_INTERVAL_SEC:
        self._last_drop_warn = now
        logger.warning(
            "[PriorityDispatcher] BACKGROUND queue saturated – "
            "dropped %d events total (cap=%d)",
            self._metrics.dropped_background_total,
            self._background_cap,
        )
    logger.debug(
        "[PriorityDispatcher] drop #%d",
        self._metrics.dropped_background_total,
    )

def _coalesce_if_possible(
    self,
    event: Events | str,
    prio: EventPriority,
    kwargs: dict[str, Any],
) -> bool:
    """Coalesce if possible."""
    key_name = get_coalesce_key(event)
    if not key_name or key_name not in kwargs:
        return False
    event_str = event.value if isinstance(event, Events) else str(event)
    coalesce_map_key = (event_str, str(kwargs[key_name]))
    existing = self._coalesce_map.get(coalesce_map_key)
    if existing is None or existing.dropped:
        return False
    existing.dropped = True
    self._metrics.coalesced_total += 1
    self._push(event, prio, kwargs, coalesce_map_key)
    return True

def _push(
    self,
    event: Events | str,
    prio: EventPriority,

```

```

        kwargs: dict[str, Any],
        coalesce_map_key: tuple[str, str] | None = None,
    ) -> None:
        """Push."""
        self._seq += 1
        item = _QueueItem(
            priority=int(prio),
            seq=self._seq,
            event=event,
            kwargs=kwargs,
        )
        self._queue.put_nowait(item)
        if coalesce_map_key is None:
            key_name = get_coalesce_key(event)
            if key_name and key_name in kwargs:
                event_str = (
                    event.value if isinstance(event, Events) else str(event)
                )
                coalesce_map_key = (event_str, str(kwargs[key_name]))
        if coalesce_map_key is not None:
            self._coalesce_map[coalesce_map_key] = item

    async def _worker(self) -> None:
        """Worker."""
        while not self._stopping:
            item = await self._queue.get()
            try:
                if self._stopping and item.seq == -1:
                    return
                if item.dropped:
                    continue
                await self._dispatch_one(item)
            except Exception as e:
                self._handle_dispatch_error(item, e)
            finally:
                self._queue.task_done()

    async def _dispatch_one(self, item: _QueueItem) -> None:
        """Dispatch one."""
        import time
        if item.event is None:
            return
        event_str = (
            item.event.value
            if isinstance(item.event, Events)
            else str(item.event)
        )
        t0 = time.monotonic()
        if self._batcher is not None and get_coalesce_key(item.event):
            should_flush = self._batcher.add(event_str, item.kwargs)
            if should_flush:
                batch = self._batcher.drain(event_str)
                await self._bus.emit(
                    f"{event_str}_batch", batch=batch,
                )
        else:
            await self._bus.emit(item.event, **item.kwargs)
        duration_ms = (time.monotonic() - t0) * 1000
        self._metrics.dispatched_total += 1
        if self._latency is not None:
            self._latency.record(event_str, duration_ms)

```

```
        if self._replay is not None:
            self._replay.record(item.event, item.kwargs)
    def _handle_dispatch_error(
        self, item: _QueueItem, err: Exception,
    ) -> None:
        """Handle dispatch error."""
        logger.exception(
            "[PriorityDispatcher] handler error on %s: %s",
            item.event, err,
        )
```

OP-09d

event_replay.py

Fichier : py_modules/unifideck/event_bus/event_replay.py | Lignes : 108 | Depend de : (rien)

```

from __future__ import annotations
import time
from collections import deque
from collections.abc import Iterable
from dataclasses import dataclass
from typing import Any
from ..core.types import Events
MAX_SNAPSHOT_ENTRIES = 500
_DEFAULT_CAPS: dict[Events, int] = {
    Events.SYNC_PROGRESS: 50,
    Events.DOWNLOAD_PROGRESS: 50,
    Events.GAME_INSTALLED: 20,
    Events.GAME_UNINSTALLED: 20,
    Events.STORE_AUTH_COMPLETE: 10,
    Events.STORE_LOGOUT: 10,
}
_FALLBACK_CAP = 20
@dataclass
class _RecordedEvent:
    """Recorded event."""
    event: str
    kwargs: dict[str, Any]
    timestamp: float
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        return {
            "event": self.event,
            "kwargs": self.kwargs,
            "timestamp": round(self.timestamp, 3),
        }
class EventReplayBuffer:
    """Event replay buffer."""
    def __init__(
        self,
        *,
        fallback_cap: int = _FALLBACK_CAP,
        caps: dict[Events, int] | None = None,
    ) -> None:
        """Initialize the instance."""
        self._fallback_cap = fallback_cap
        self._caps = dict(_DEFAULT_CAPS)
        if caps:
            self._caps.update(caps)
        self._buffers: dict[str, deque[_RecordedEvent]] = {}
    def record(
        self,
        event: Events | str,
        kwargs: dict[str, Any],
    ) -> None:
        """Record."""
        event_str = event.value if isinstance(event, Events) else str(event)
        buf = self._buffers.get(event_str)
        if buf is None:
            cap = self._resolve_cap(event)
            buf = deque(maxlen=cap)

```



```

        self._buffers[event_str] = buf
    buf.append(
        _RecordedEvent(
            event=event_str,
            kwargs=kwargs,
            timestamp=time.monotonic(),
        ),
    )

def snapshot(
    self,
    events: Iterable[Events | str] | None = None,
    limit: int = MAX_SNAPSHOT_ENTRIES,
) -> list[dict[str, Any]]:

    """Snapshot."""
    limit = min(limit, MAX_SNAPSHOT_ENTRIES)
    wanted = self._resolve_wanted_set(events)
    gathered: list[_RecordedEvent] = []
    for event_str, buf in self._buffers.items():
        if wanted is not None and event_str not in wanted:
            continue
        gathered.extend(buf)
    gathered.sort(key=lambda r: r.timestamp, reverse=True)
    return [r.to_dict() for r in gathered[:limit]]

def size(self, event: Events | str | None = None) -> int:
    """Size."""
    if event is None:
        return sum(len(b) for b in self._buffers.values())
    event_str = event.value if isinstance(event, Events) else str(event)
    buf = self._buffers.get(event_str)
    return len(buf) if buf is not None else 0

def clear(self) -> None:
    """Clear."""
    self._buffers.clear()

def _resolve_cap(self, event: Events | str) -> int:
    """Resolve cap."""
    if isinstance(event, Events):
        return self._caps.get(event, self._fallback_cap)
    try:
        resolved = Events(event)
        return self._caps.get(resolved, self._fallback_cap)
    except ValueError:
        return self._fallback_cap

@staticmethod
def _resolve_wanted_set(
    events: Iterable[Events | str] | None,
) -> set | None:
    """Resolve wanted set."""
    if events is None:
        return None
    out: set = set()
    for e in events:
        out.add(e.value if isinstance(e, Events) else str(e))
    return out

```

OP-09e

event_bus_extensions.py

Fichier : py_modules/unifideck/event_bus/event_bus_extensions.py | Lignes : 196 | Depend de : (rien)

```

"""Four small capabilities grouped in one module to minimize file
proliferation. Each is a standalone class, so SRP is preserved at
the class level even though they share a file:
- TypedEventRegistry (P7.4): runtime validation of event kwargs
  against a declared schema, with mypy-friendly Protocol stubs.
- DeadLetterQueue (P7.5): capture events whose handlers all
  failed, for later replay. Persisted to a JSONL file via the
Security note: the DLQ file is created with mode 0o600 (owner-only
read/write) because it may incidentally contain sensitive kwargs
like store names or game IDs. OAuth tokens must never be passed
as event kwargs; this is a callsite responsibility the DLQ cannot
enforce.
"""

from __future__ import annotations
import logging
from collections.abc import Callable
from dataclasses import dataclass, field
from typing import (
    TYPE_CHECKING,
    Any,
    Protocol,
)
from ..core.types import Events
if TYPE_CHECKING:
    from .event_bus import EventBus
    from .event_replay import EventReplayBuffer
    from .priority_dispatcher import PriorityDispatcher
    from .supervision.metrics_handler import HandlerLatencyCollector
    from .supervision.watchdog_handler import HandlerWatchdog
logger = logging.getLogger(__name__)
# — P7.4 — Typed event schemas —————
class EventPayload(Protocol):
    """Marker Protocol for typed event payloads.
    Concrete payloads inherit from this via Protocol subclassing:
    class SyncCompletePayload(EventPayload, Protocol):
        games: list
        stores_synced: list
        duration_ms: int
    """

@dataclass
class EventSchema:
    """Declarative kwargs contract for one event type."""
    required: set[str] = field(default_factory=set)
    optional: set[str] = field(default_factory=set)
    def validate(self, kwargs: dict[str, Any]) -> str | None:
        """Return None on success, or an error message string."""
        missing = self.required - set(kwargs.keys())
        if missing:
            return f"missing required kwargs: {sorted(missing)}"
        allowed = self.required | self.optional
        extra = set(kwargs.keys()) - allowed
        if extra:
            return f"unexpected kwargs: {sorted(extra)}"
        return None

```

```

class TypedEventRegistry:
    """Holds per-event schemas and validates at emit time."""
    def __init__(self) -> None: # noqa: D107 – class docstring documents the constructor's
        contract
        self._schemas: dict[str, EventSchema] = {}
    def declare(self, event: Events | str, schema: EventSchema) -> None: # noqa: D102 –
        documentation pending (Sprint D)
        key = event.value if isinstance(event, Events) else str(event)
        self._schemas[key] = schema
    def validate( # noqa: D102 – documentation pending (Sprint D)
        self, event: Events | str, kwargs: dict[str, Any],
    ) -> str | None:
        key = event.value if isinstance(event, Events) else str(event)
        schema = self._schemas.get(key)
        if schema is None:
            return None # unregistered events pass through
        return schema.validate(kwargs)
# — P7.6 – Predicate filter on subscriptions —————
# A predicate is any callable taking kwargs and returning bool.
Predicate = Callable[..., bool]
class PredicateFilter:
    """Wraps a handler with an arbitrary pre-invocation filter.
    Usage at subscription time:
        filter = PredicateFilter(handler, lambda store, **_: store == "epic")
        bus.on(Events.GAME_LAUNCHED, filter)
    """
    def __init__(self, handler: Callable[..., Any], predicate: Predicate) -> None: # noqa:
        D107 – class docstring documents the co
        self._handler = handler
        self._predicate = predicate
        # Preserve the inner handler name for watchdog/metrics
        self.__name__ = getattr(handler, "__name__", "filtered_handler")
        self.__qualname__ = getattr(handler, "__qualname__", self.__name__)
    async def __call__(self, *args: Any, **kwargs: Any) -> Any: # noqa: D102 –
        documentation pending (Sprint D)
        try:
            matches = self._predicate(*args, **kwargs)
        except Exception:
            # A broken predicate should not silently eat events –
            # log and pass through so the handler still runs.
            logger.exception(
                "[PredicateFilter] predicate raised; passing through",
            )
            matches = True
        if not matches:
            return None
        return await self._handler(*args, **kwargs)
# — P7.7 – Debug snapshot —————
class DebugSnapshot:
    """Collects the full state of bus + dispatcher for debugging.
    Called by a dev-only RPC `debug_snapshot()` on the Plugin class.
    Returns a JSON-serializable dict that operators can paste into
    bug tickets to reproduce issues. Never called in production hot
    paths – cost is measured once per call, not per event.
    """

    @staticmethod
    def collect(
        bus: EventBus,
        dispatcher: PriorityDispatcher | None = None,
        watchdog: HandlerWatchdog | None = None,
    )

```

```

    metrics: HandlerLatencyCollector | None = None,
    replay: EventReplayBuffer | None = None,
    dlq: DeadLetterQueue | None = None,
) -> dict[str, Any]:
    """Gather every observable slice of state into one dict."""
    snapshot: dict[str, Any] = {
        "bus": {
            "handler_counts": DebugSnapshot._safe_call(
                getattr(bus, "health", None),
            ),
        },
    }
    if dispatcher is not None:
        m = dispatcher.get_metrics()
        snapshot["dispatcher"] = {
            "emitted_total": m.emitted_total,
            "dispatched_total": m.dispatched_total,
            "coalesced_total": m.coalesced_total,
            "dropped_background_total": m.dropped_background_total,
            "pending_by_priority": m.pending_by_priority,
        }
    if watchdog is not None:
        snapshot["watchdog"] = {
            name: {
                "invocations": ms.invocations,
                "timeouts": ms.timeouts,
                "consecutive_timeouts": ms.consecutive_timeouts,
                "quarantined": ms.quarantined,
            }
            for name, ms in watchdog.get_metrics().items()
        }
    if metrics is not None:
        snapshot["handler_metrics"] = metrics.get_top_n(n=10)
    if replay is not None:
        snapshot["replay_sizes"] = {
            "total": replay.size(),
        }
    if dlq is not None:
        snapshot["dlq_entries"] = len(dlq)
    return snapshot
@staticmethod
def _safe_call(fn: Callable | None) -> Any:
    if fn is None:
        return None
    try:
        return fn()
    except Exception as e: # noqa: BLE001 – deliberate: isolate user callback
        return {"error": str(e)}

```

— P7.5 – Dead letter queue

```

class DeadLetterQueue:
    """Capture events whose handlers all failed.
    When an event is emitted and ZERO of its registered handlers
    complete successfully (every one raises or times out), the
    event is appended to the dead letter queue so operators can
    inspect it later rather than losing the payload silently.
    Kept deliberately minimal: a bounded ring buffer of (event,
    payload, reason) tuples. Production callers either drain
    the queue for a diagnostics RPC, or log-and-forget on
    shutdown. There is no retry mechanism – DLQ is for audit,
    not for recovery.
    """

```

```
def __init__(self, max_size: int = 256) -> None: # noqa: D107 – class docstring
    documents the constructor's contract
    self._max_size = int(max_size)
    self._entries: list[dict[str, Any]] = []

def record(
    self,
    event: str,
    payload: dict[str, Any] | None,
    reason: str,
) -> None:
    """Append one failed event. Oldest entries are dropped."""
    self._entries.append({
        "event": event,
        "payload": payload or {},
        "reason": reason,
    })
    if len(self._entries) > self._max_size:
        self._entries = self._entries[-self._max_size:]
def snapshot(self) -> list[dict[str, Any]]:
    """Return a copy of the current buffer (newest last)."""
    return list(self._entries)
def clear(self) -> None:
    """Drop every recorded event."""
    self._entries.clear()
def __len__(self) -> int:
    return len(self._entries)
```

OP-09f

event_bus_reliability.py

Fichier : py_modules/unifideck/event_bus/event_bus_reliability.py | Lignes : 55 | Depend de : (rien)

```

from __future__ import annotations
import logging
import time
from collections import deque
from dataclasses import dataclass, field
logger = logging.getLogger(__name__)
CB_WINDOW_SIZE = 20
CB_OPEN_THRESHOLD = 0.5
CB_RESET_TIMEOUT_SEC = 30.0
@dataclass
class _CBState:
    """Cbstate."""
    window: deque[bool] = field(
        default_factory=lambda: deque(maxlen=CB_WINDOW_SIZE),
    )
    open_until: float = 0.0
class CircuitBreaker:
    """Circuit breaker."""
    def __init__(
        self,
        *,
        open_threshold: float = CB_OPEN_THRESHOLD,
        reset_timeout: float = CB_RESET_TIMEOUT_SEC,
    ) -> None:
        """Initialize the instance."""
        self._open_threshold = open_threshold
        self._reset_timeout = reset_timeout
        self._state: dict[str, _CBState] = {}
    def allow(self, handler_name: str) -> bool:
        """Allow."""
        s = self._state.get(handler_name)
        if s is None or s.open_until == 0.0:
            return True
        if time.monotonic() >= s.open_until:
            s.open_until = 0.0
            return True
        return False
    def record(self, handler_name: str, success: bool) -> None:
        """Record."""
        s = self._state.setdefault(handler_name, _CBState())
        s.window.append(success)
        if len(s.window) < (s.window.maxlen or 0):
            return
        failures = s.window.count(False)
        rate = failures / len(s.window)
        if rate >= self._open_threshold and s.open_until == 0.0:
            s.open_until = time.monotonic() + self._reset_timeout
            logger.warning(
                "[CircuitBreaker] %s opened (failure rate=%.0f%%)",
                handler_name, rate * 100,
            )
    def is_open(self, handler_name: str) -> bool:
        """Check whether open."""
        s = self._state.get(handler_name)
        return s is not None and s.open_until > time.monotonic()

```

OP-09g

event_bus_scaling.py

Fichier : py_modules/unifideck/event_bus/event_bus_scaling.py | Lignes : 48 | Depend de : (rien)

```
from __future__ import annotations
import time
from collections.abc import Callable
from typing import Any
DEFAULT_BATCH_WINDOW_MS = 50
DEFAULT_BATCH_MAX_SIZE = 100
class BatchDispatcher:
    """Batch dispatcher."""
    def __init__(
        self,
        *,
        window_ms: int = DEFAULT_BATCH_WINDOW_MS,
        max_size: int = DEFAULT_BATCH_MAX_SIZE,
    ) -> None:
        """Initialize the instance."""
        self._window_ms = window_ms
        self._max_size = max_size
        self._buffers: dict[str, list[Any]] = {}
        self._last_flush_ms: dict[str, float] = {}
    def add(self, key: str, item: Any) -> bool:
        """Add."""
        buf = self._buffers.setdefault(key, [])
        buf.append(item)
        if len(buf) >= self._max_size:
            return True
        last = self._last_flush_ms.get(key)
        now_ms = time.monotonic() * 1000
        if last is None:
            self._last_flush_ms[key] = now_ms
            return False
        return (now_ms - last) >= self._window_ms
    def drain(self, key: str) -> list[Any]:
        """Drain."""
        items = self._buffers.pop(key, [])
        self._last_flush_ms[key] = time.monotonic() * 1000
        return items
    def flush_all(self) -> dict[str, list[Any]]:
        """Flush all."""
        out = {k: v for k, v in self._buffers.items() if v}
        self._buffers.clear()
        return out
    @staticmethod
    def handler_supports_batch(handler: Callable) -> bool:
        """Handler supports batch."""
        return (
            getattr(handler, "supports_batch", False) is True
            and callable(getattr(handler, "on_batch", None))
        )
```

OP-09h

event_bus_devex.py

Fichier : py_modules/unifideck/event_bus/event_bus_devex.py | Lignes : 177 | Depend de : (rien)

```

from __future__ import annotations
import ast
import logging
from collections.abc import Callable
from dataclasses import dataclass
from typing import (
    TYPE_CHECKING,
    Any,
)
if TYPE_CHECKING:
    from ..core.types import Events
    from .event_bus import EventBus
    from .supervision.watchdog_handler import HandlerWatchdog
logger = logging.getLogger(__name__)
@dataclass
class _Subscription:
    """Subscription."""
    event: str
    handler: Callable
    priority: int | None = None
    timeout: float | None = None
    scope: str | None = None
class SubscriptionRegistry:
    """Subscription registry."""
    def __init__(self) -> None:
        """Initialize the instance."""
        self._subs: list[_Subscription] = []
    def add(self, sub: _Subscription) -> None:
        """Add."""
        self._subs.append(sub)
    def all(self) -> list[_Subscription]:
        """All."""
        return list(self._subs)
    def apply(self, bus: EventBus) -> int:
        """Apply."""
        count = 0
        for s in self._subs:
            if hasattr(bus, "on"):
                bus.on(s.event, s.handler)
                count += 1
        return count
    def clear(self) -> None:
        """Clear."""
        self._subs.clear()

default_registry = SubscriptionRegistry()
def subscribe(
    event: str | Events,
    *,
    priority: int | None = None,
    timeout: float | None = None,
    scope: str | None = None,
    registry: SubscriptionRegistry | None = None,
):
    """Subscribe."""

```



```

def decorator(fn: Callable) -> Callable:
    """Decorator."""
    event_key = getattr(event, "value", event)
    meta = _Subscription(
        event=str(event_key),
        handler=fn,
        priority=priority,
        timeout=timeout,
        scope=scope,
    )
    setattr(fn, "__subscribe_meta__", meta)
    if _looks_like_instance_method(fn):
        return fn
    reg = registry or default_registry
    reg.add(getattr(fn, "__subscribe_meta__"))
    return fn
return decorator

def _looks_like_instance_method(fn: Callable) -> bool:
    """Looks like instance method."""
    try:
        import inspect
        sig = inspect.signature(fn)
        params = list(sig.parameters.keys())
        return bool(params) and params[0] in ("self", "cls")
    except (TypeError, ValueError):
        return False

def auto_wire(
    instance: Any,
    bus: EventBus,
    *,
    registry: SubscriptionRegistry | None = None,
    watchdog: HandlerWatchdog | None = None,
) -> int:
    """Auto wire."""
    count = 0
    for attr_name in dir(instance):
        if attr_name.startswith("__"):
            continue
        attr, meta = _resolve_subscribe_target(instance, attr_name)
        if meta is None:
            continue
        if not hasattr(bus, "on"):
            continue
        bus.on(meta.event, attr)
        count += 1
        if watchdog is not None:
            _register_with_watchdog(instance, attr_name, watchdog)
        if registry is not None:
            registry.add(
                _Subscription(
                    event=meta.event,
                    handler=attr,
                    priority=meta.priority,
                    timeout=meta.timeout,
                    scope=meta.scope,
                ),
            )
    return count

def _resolve_subscribe_target(instance: Any, attr_name: str):
    """Resolve subscribe target."""
    try:

```

```

        attr = getattr(instance, attr_name)
    except AttributeError:
        return None, None
    meta = getattr(attr, "__subscribe_meta__", None)
    if meta is None:
        func = getattr(attr, "__func__", None)
        if func is not None:
            meta = getattr(func, "__subscribe_meta__", None)
    return (attr if meta is not None else None), meta

def _register_with_watchdog(
    instance: Any, attr_name: str, watchdog: HandlerWatchdog,
) -> None:
    """Register with watchdog."""
    qualname = f"{type(instance).__name__}.{attr_name}"
    try:
        watchdog.register(qualname)
    except (AttributeError, RuntimeError) as e:
        logger.debug(
            "[event_bus_devex] watchdog register failed for %s: %s",
            qualname, e,
        )

class SchemaExtractor:
    """Schema extractor."""
    @staticmethod
    def extract_from_source(source: str) -> dict[str, set[str]]:
        """Extract from source."""
        out: dict[str, set[str]] = {}
        try:
            tree = ast.parse(source)
        except SyntaxError:
            return out
        for node in ast.walk(tree):
            if not isinstance(node, ast.Call):
                continue
            if not SchemaExtractor._is_emit_call(node):
                continue
            event_name = SchemaExtractor._extract_event_name(node)
            if event_name is None:
                continue
            kwarg_names = {
                kw.arg for kw in node.keywords if kw.arg is not None
            }
            out.setdefault(event_name, set()).update(kwarg_names)
        return out
    @staticmethod
    def _is_emit_call(node: ast.Call) -> bool:
        """Is emit call."""
        func = node.func
        if isinstance(func, ast.Attribute) and func.attr == "emit":
            return True
        return bool(isinstance(func, ast.Attribute) and func.attr == "enqueue")
    @staticmethod
    def _extract_event_name(node: ast.Call) -> str | None:
        """Extract event name."""
        if not node.args:
            return None
        first = node.args[0]
        if isinstance(first, ast.Attribute):
            return first.attr

```

```
if isinstance(first, ast.Constant) and isinstance(first.value, str):  
    return first.value  
return None
```

OP-09i

bus_pipeline.py

Fichier : py_modules/unifideck/event_bus/bus_pipeline.py | Lignes : 15 | Depend de : (rien)

```
from __future__ import annotations
from typing import TYPE_CHECKING, NamedTuple
if TYPE_CHECKING:
    from .event_bus_scaling import BatchDispatcher
    from .event_replay import EventReplayBuffer
    from .priority_dispatcher import PriorityDispatcher
    from .supervision.metrics_handler import HandlerLatencyCollector
    from .supervision.watchdog_handler import HandlerWatchdog
class BusPipeline(NamedTuple):
    """Bus pipeline."""
    watchdog: HandlerWatchdog
    latency: HandlerLatencyCollector
    replay: EventReplayBuffer
    batcher: BatchDispatcher
    dispatcher: PriorityDispatcher
```

OP-11

__init__.py

Fichier : py_modules/unifideck/config/__init__.py | Lignes : 24 | Depend de : (rien)

```
from .config_manager import ConfigManager
from .config_persistence import (
    atomic_write_json,
    load_json_layer,
)
from .il8n_schema import (
    ConfigSchemaError,
    validate_il8n_schema,
)
from .validator import (
    ConfigValidator,
    ValidationError,
    ValidationResult,
)
__all__ = [
    "ConfigManager",
    "load_json_layer",
    "atomic_write_json",
    "validate_il8n_schema",
    "ConfigSchemaError",
    "ConfigValidator",
    "ValidationResult",
    "ValidationError",
]
```

OP-11a

config_manager.py

Fichier : py_modules/unifideck/config/config_manager.py | Lignes : 231 | Depend de : (rien)

```

import json
import logging
from pathlib import Path
from typing import Any
logger = logging.getLogger(__name__)
_FALLBACK: dict[str, Any] = {
    "data_dir": "~/.local/share/unifideck",
    "ui": {
        "language": "en-US",
    },
    "sync": {
        "interval_seconds": 300,
        "cache_ttl_seconds": 3600,
    },
    "download": {
        "timeout_seconds": 30,
        "max_retries": 3,
        "artwork_concurrent": 4,
    },
    "stores": {
        "epic": {},
        "gog": {"client_secret": ""},
        "amazon": {},
        "ubisoft": {},
        "microsoft": {},
    },
}
def _deep_merge(base: dict[str, Any],
                override: dict[str, Any]) -> dict[str, Any]:
    """Deep merge."""
    result = dict(base)
    for k, v in override.items():
        if k in result and isinstance(result[k], dict) and isinstance(v, dict):
            result[k] = _deep_merge(result[k], v)
        else:
            result[k] = v
    return result
def _get_by_path(d: dict[str, Any], dotted: str) -> Any:
    """Get by path."""
    node: Any = d
    for part in dotted.split("."):
        if not isinstance(node, dict) or part not in node:
            raise KeyError(dotted)
        node = node[part]
    return node
def _set_by_path(d: dict[str, Any], dotted: str, value: Any) -> None:
    """Set by path."""
    parts = dotted.split(".")
    node = d
    for part in parts[:-1]:
        if part not in node or not isinstance(node[part], dict):
            node[part] = {}
        node = node[part]
    node[parts[-1]] = value
class ConfigManager:

```

```

"""Config manager."""
def __init__(
    self,
    defaults_path: str | None = None,
    user_path: str | None = None,
) -> None:
    """Initialize the instance."""
    self._defaults_path = Path(defaults_path) if defaults_path else None
    self._user_path = Path(user_path) if user_path else None
    self._merged: dict[str, Any] = {}
    self.reload()

def reload(self) -> None:

    """Reload."""
    merged = dict(_FALLBACK)
    if self._defaults_path and self._defaults_path.exists():
        try:
            data = json.loads(
                self._defaults_path.read_text(
                    encoding="utf-8",
                ),
            )
            data = {
                k: v for k, v in data.items()
                if not k.startswith("_")
            }
            merged = _deep_merge(merged, data)
        except (json.JSONDecodeError, OSError) as e:
            logger.warning(
                "[ConfigManager] defaults unreadable "
                "({s}): {s}", type(e).__name__, e,
            )
    if self._user_path and self._user_path.exists():
        try:
            data = json.loads(
                self._user_path.read_text(
                    encoding="utf-8",
                ),
            )
            merged = _deep_merge(merged, data)
        except (json.JSONDecodeError, OSError) as e:
            logger.warning(
                "[ConfigManager] user config unreadable "
                "({s}): {s}", type(e).__name__, e,
            )
    self._merged = merged
    self._validate_i18n_schema()
def _validate_i18n_schema(self) -> None:
    """Validate I18N schema."""
    if "i18n" not in self._merged:
        return
    if not self._defaults_path:
        return
    scripts_dir = self._defaults_path.parent.parent / "scripts"
    if not (scripts_dir / "locale_config.py").is_file():
        logger.debug(
            "[ConfigManager] locale_config.py not found at {s} - "
            "skipping i18n schema validation",
            scripts_dir,
        )

```

```

        return
import sys
scripts_str = str(scripts_dir)
added = False
if scripts_str not in sys.path:
    sys.path.insert(0, scripts_str)
    added = True
try:
    from locale_config import (
        LocaleConfigError,
        load_from_dict,
    )
    try:
        load_from_dict(self._merged)
    except LocaleConfigError as e:
        logger.error(
            "[ConfigManager] i18n schema validation failed: %s", e,
        )
        raise
finally:
    if added:
        sys.path.remove(scripts_str)
def get(self, key: str, default: Any = None) -> Any:
    """Get."""
    try:
        return _get_by_path(self._merged, key)
    except KeyError:
        return default
def get_str(self, key: str, default: str = "") -> str:
    """Get str."""
    v = self.get(key, default)
    return str(v) if v is not None else default

def get_int(self, key: str, default: int = 0) -> int:
    """Get int."""
    v = self.get(key, default)
    try:
        return int(v)
    except (TypeError, ValueError):
        return default
def get_bool(self, key: str, default: bool = False) -> bool:
    """Get bool."""
    v = self.get(key, default)
    if isinstance(v, bool):
        return v
    if isinstance(v, str):
        return v.strip().lower() in (
            "true", "1", "yes", "on",
        )
    return bool(v)
def set(self, key: str, value: Any) -> None:
    """Set."""
    _set_by_path(self._merged, key, value)
    if not self._user_path:
        return
    user_data: dict[str, Any] = {}
    if self._user_path.exists():
        try:
            user_data = json.loads(
                self._user_path.read_text(encoding="utf-8"),

```



```

        )
    except (json.JSONDecodeError, OSError):
        user_data = {}
    _set_by_path(user_data, key, value)
    self._user_path.parent.mkdir(
        parents=True, exist_ok=True,
    )
    tmp = self._user_path.with_suffix(
        self._user_path.suffix + ".tmp",
    )
    try:
        tmp.write_text(
            json.dumps(
                user_data,
                ensure_ascii=False,
                indent=2,
            ),
            encoding="utf-8",
        )
        tmp.replace(self._user_path)
    except OSError as e:
        logger.error(
            "[ConfigManager] failed to persist %s: %s",
            key, e,
        )
    if tmp.exists():
        try:
            tmp.unlink()
        except OSError:
            pass

@property
def data_dir(self) -> str:
    """Data dir."""
    raw = self.get("paths.data_dir") or self.get_str(
        "data_dir", "~/.local/share/unifideck",
    )
    return str(Path(raw).expanduser())

@property
def cache_dir(self) -> str:
    """Cache dir."""
    raw = self.get("paths.cache_dir")
    if raw:
        return str(Path(raw).expanduser())
    return str(Path(self.data_dir) / "cache")

def path(self, *parts: str) -> str:
    """Path."""
    if len(parts) == 1 and parts[0] == "cache":
        return self.cache_dir
    return str(Path(self.data_dir).joinpath(*parts))

def __getitem__(self, key: str) -> Any:
    """Getitem."""
    val = self.get(key)
    if val is None:
        raise KeyError(key)
    return val

def __contains__(self, key: str) -> bool:
    """Contains."""
    return self.get(key) is not None

```

OP-11b

config_persistence.py

Fichier : py_modules/unifideck/config/config_persistence.py | Lignes : 55 | Depend de : (rien)

```

from __future__ import annotations
import json
import logging
import os
import tempfile
from pathlib import Path
from typing import Any
logger = logging.getLogger(__name__)
def load_json_layer(path: Path) -> dict[str, Any]:
    """Load JSON layer."""
    if not path or not path.exists():
        return {}
    try:
        data = json.loads(path.read_text(encoding="utf-8"))
    except (json.JSONDecodeError, OSError) as e:
        logger.warning(
            "[config_persistence] %s unreadable (%s): %s",
            path, type(e).__name__, e,
        )
        return {}
    if not isinstance(data, dict):
        logger.warning(
            "[config_persistence] %s is not a JSON object (got %s) - "
            "ignoring",
            path, type(data).__name__,
        )
        return {}
    return {k: v for k, v in data.items() if not k.startswith("_")}
def atomic_write_json(
    path: Path,
    data: dict[str, Any],
    mode: int = 0o600,
) -> None:
    """Atomic write JSON."""
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp_path_init = tempfile.mkstemp(
        prefix=f"{path.name}.",
        suffix=".tmp",
        dir=str(path.parent),
    )
    tmp_path: str | None = tmp_path_init
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=2, sort_keys=True)
            f.flush()
            os.fsync(f.fileno())
        Path(tmp_path_init).chmod(mode)
        Path(tmp_path_init).replace(path)
        tmp_path = None
    finally:
        if tmp_path is not None:
            try:
                Path(tmp_path).unlink()
            except OSError:
                pass

```

OP-11c

i18n_schema.py

Fichier : py_modules/unifideck/config/i18n_schema.py | Lignes : 108 | Depend de : (rien)

```

"""config/i18n_schema.py – i18n schema validation for ConfigManager.
Moved from core/config_schema.py to unifideck.config and
renamed to i18n_schema to dispel the name clash with config/schema.json
(the JSON Schema Draft-07 descriptor used by ConfigValidator).
This module specifically validates the i18n section of the merged
configuration against the locale_config catalogue shipped under
scripts/. It is intentionally independent from the broader JSON
Schema validator in config/validator.py: the two validate different
things (locale identifiers vs. shape of the whole config.json).
Responsibilities:
1. Locating the shared scripts/locale_config.py module
2. Injecting scripts/ into sys.path for the import
3. Calling load_from_dict and converting its errors to logs
A shim at core/config_schema.py preserves backward compatibility
for any caller still importing from the old path.
Why delegate to scripts/locale_config.py rather than duplicating
the schema here: the build-time translator at
scripts/translate_at_build.py already uses locale_config to decide
which locales to generate. A duplicate schema definition would
drift and produce cryptic runtime errors the next time someone
adds a locale without touching both copies.
Reference: Technical Document v1.0 – Section 3.4.10 (i18n pipeline).
"""

from __future__ import annotations
import logging
import sys
from pathlib import Path
from typing import Any
logger = logging.getLogger(__name__)
class ConfigSchemaError(Exception):
    """Raised when the merged config violates the declared schema.
    Distinct from ValueError/KeyError so callers can catch schema
    violations specifically and report them as startup failures
    rather than random runtime glitches.
    """

def validate_i18n_schema(
    merged: dict[str, Any],
    defaults_path: Path | None,
) -> None:
    """Validate the `i18n` section of the merged config.
    Behaviour:
    - No i18n section → return silently (legacy configs pre-date
      the i18n feature and must stay loadable)
    - scripts/locale_config.py missing → log at DEBUG and return
      (development checkouts may not ship the scripts/ folder)
    - LocaleConfigError from locale_config → log at ERROR and
      re-raise as ConfigSchemaError so the plugin fails loudly
      at startup rather than silently at runtime
    Args:
    merged: the fully-merged config dict (defaults + user overrides)
    defaults_path: path to defaults/config.json, used to locate
      scripts/ relative to it (defaults_path.parent.parent/scripts)
    Raises:
    ConfigSchemaError: if i18n is present and fails validation
    """

```

```

if "il8n" not in merged:

    return # legacy config – tolerable
if defaults_path is None:
    logger.debug(
        "[config_schema] defaults_path=None – cannot locate "
        "scripts/ for il8n validation, skipping",
    )
    return
scripts_dir = defaults_path.parent.parent / "scripts"
if not (scripts_dir / "locale_config.py").is_file():
    logger.debug(
        "[config_schema] locale_config.py not found at %s – "
        "skipping il8n schema validation",
        scripts_dir,
    )
    return
# Temporarily inject scripts/ into sys.path so we can import
# locale_config. Use a try/finally to always restore the
# original path even if the import or validation raises.
# `added` tracks whether WE are the ones who inserted – we only
# remove on our way out, never removing a path an earlier caller
# (or the user's PYTHONPATH) already put there.
scripts_str = str(scripts_dir)
added = False
if scripts_str not in sys.path:
    sys.path.insert(0, scripts_str)
    added = True
try:
    from locale_config import (
        LocaleConfigError,
        load_from_dict,
    )
    try:
        load_from_dict(merged)
    except LocaleConfigError as e:
        logger.error(
            "[config_schema] il8n schema validation failed: %s", e,
        )
        raise ConfigSchemaError(
            f"il8n schema violation: {e}",
        ) from e
finally:
    if added:
        try:
            sys.path.remove(scripts_str)
        except ValueError:
            # best-effort cleanup; file may already be gone or locked
            pass
# — Legacy compatibility alias
# The original name was ``validate_il8n``. Some callers import
# that form; keep it as an alias so the rename stays non-
# breaking until every call site is updated.
validate_il8n = validate_il8n_schema

```

OP-11e

validator.py

Fichier : py_modules/unifideck/config/validator.py | Lignes : 251 | Depend de : (rien)

```

from __future__ import annotations
import json
import logging
from dataclasses import dataclass, field
from pathlib import Path
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from ..event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
_SCHEMA_PATH = str(Path(__file__).resolve().parent / "schema.json")
_SOURCE_DEFAULTS = "defaults"
_SOURCE_USER = "user_overrides"
_SOURCE_MERGED = "merged"
@dataclass(frozen=True)
class ValidationError:
    """Validation error."""
    source: str
    path: str
    message: str
@dataclass
class ValidationResult:
    """Validation result."""
    success: bool = False
    errors: list[ValidationError] = field(default_factory=list)
    defaults_validated: bool = False
    user_overrides_present: bool = False
class ConfigValidator:
    """Config validator."""
    def __init__(self, bus: EventBus | None = None) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._schema: dict | None = None
    async def validate_config(
        self,
        defaults_path: str,
        user_path: str | None = None,
    ) -> ValidationResult:
        """Validate config."""
        result = ValidationResult()
        schema = self._load_schema()
        if schema is None:
            result.errors.append(ValidationError(
                source=_SOURCE_DEFAULTS, path="",
                message="Cannot load validator schema.json",
            ))
            self._emit_result(result)
            return result
        defaults = self._validate_defaults(
            defaults_path, schema, result,
        )
        if defaults is None:
            self._emit_result(result)
            return result
        merged = self._validate_user_overrides(
            defaults, user_path, schema, result,

```

```

    )
    self._validate_merged(merged, schema, result)
    if len(result.errors) > 50:
        result.errors = result.errors[:50]
    result.success = (
        result.defaults_validated and len(result.errors) == 0
    )
    self._emit_result(result)
    return result

def _validate_defaults(
    self, defaults_path: str, schema: dict,
    result: ValidationResult,
) -> dict | None:

    """Validate defaults."""
    defaults = self._read_json(defaults_path)
    if defaults is None:
        result.errors.append(ValidationError(
            source=_SOURCE_DEFAULTS, path="",
            message=f"Cannot read defaults file at {defaults_path}",
        ))
        return None
    errors = self._validate_against_schema(
        defaults, schema, _SOURCE_DEFAULTS,
    )
    result.errors.extend(errors)
    result.defaults_validated = len(errors) == 0
    return defaults

def _validate_user_overrides(
    self,
    defaults: dict,
    user_path: str | None,
    schema: dict,
    result: ValidationResult,
) -> dict:

    """Validate user overrides."""
    if user_path is None:
        return defaults
    expanded = str(Path(user_path).expanduser())
    if not Path(expanded).is_file():
        return defaults
    result.user_overrides_present = True
    user_data = self._read_json(expanded)
    if user_data is None:
        result.errors.append(ValidationError(
            source=_SOURCE_USER, path="",
            message=(
                f"Cannot read or parse user overrides "
                f"at {expanded}"
            ),
        ))
        return defaults
    relaxed_schema = self._strip_required(schema)
    user_errors = self._validate_against_schema(
        user_data, relaxed_schema, _SOURCE_USER,
    )
    result.errors.extend(user_errors)
    if user_errors:
        return defaults
    return self._deep_merge(defaults, user_data)

```

```

@staticmethod
def _strip_required(schema: Any) -> Any:
    """Strip required."""
    if isinstance(schema, dict):
        out = {}
        for key, value in schema.items():
            if key == "required":
                continue
            out[key] = ConfigValidator._strip_required(value)
        return out
    if isinstance(schema, list):
        return [
            ConfigValidator._strip_required(item) for item in schema
        ]
    return schema
def _validate_merged(
    self, merged: dict, schema: dict,
    result: ValidationResult,
) -> None:
    """Validate merged."""
    if not result.defaults_validated:
        return
    if any(e.source == _SOURCE_USER for e in result.errors):
        return
    merged_errors = self._validate_against_schema(
        merged, schema, _SOURCE_MERGED,
    )
    result.errors.extend(merged_errors)

def _load_schema(self) -> dict | None:
    """Load schema."""
    if self._schema is not None:
        return self._schema
    try:
        with Path(_SCHEMA_PATH).open(encoding="utf-8") as f:
            self._schema = json.load(f)
        return self._schema
    except (OSError, json.JSONDecodeError) as e:
        logger.error(
            "[ConfigValidator] cannot load schema at %s: %s",
            _SCHEMA_PATH, e,
        )
        return None
@staticmethod
def _read_json(path: str) -> dict | None:
    """Read JSON."""
    expanded = str(Path(path).expanduser())
    try:
        with Path(expanded).open(encoding="utf-8") as f:
            data = json.load(f)
    except (OSError, json.JSONDecodeError) as e:
        logger.warning(
            "[ConfigValidator] cannot read %s: %s", expanded, e,
        )
        return None
    if not isinstance(data, dict):
        logger.warning(
            "[ConfigValidator] %s is not a JSON object", expanded,
        )
        return None

```

```

        return data
    @staticmethod
    def _validate_against_schema(
        data: dict, schema: dict, source: str,
    ) -> list[ValidationError]:
        """Validate against schema."""
        try:
            import jsonschema
        except ImportError:
            logger.error(
                "[ConfigValidator] jsonschema not installed - "
                "validation skipped",
            )
            return [ValidationError(
                source=source,
                path="",
                message="jsonschema library not installed",
            )]
        errors: list[ValidationError] = []
        validator = jsonschema.Draft7Validator(schema)
        for err in validator.iter_errors(data):
            path = ".".join(str(p) for p in err.absolute_path)
            errors.append(ValidationError(
                source=source,
                path=path,
                message=err.message[:256],
            ))
        return errors
    @staticmethod
    def _deep_merge(base: dict, overrides: dict) -> dict:
        """Deep merge."""
        result = dict(base)
        for key, value in overrides.items():
            if (
                key in result
                and isinstance(result[key], dict)
                and isinstance(value, dict)
            ):
                result[key] = ConfigValidator._deep_merge(
                    result[key], value,
                )
            else:
                result[key] = value
        return result

    def _emit_result(self, result: ValidationResult) -> None:
        """Emit result."""
        if self._bus is None:
            return
        try:
            import asyncio
            from ..core.types.events import Events
        except ImportError:
            return
        if result.success:
            loop = asyncio.get_running_loop()
            loop.create_task(self._bus.emit(
                Events.CONFIG_VALIDATION_COMPLETED,
                defaults_validated=result.defaults_validated,
            ))

```



```
        user_overrides_present=result.user_overrides_present,
    ))
else:
    loop.create_task(self._bus.emit(
        Events.CONFIG_VALIDATION_FAILED,
        error_count=len(result.errors),
        defaults_validated=result.defaults_validated,
        user_overrides_present=result.user_overrides_present,
        first_error_source=(
            result.errors[0].source if result.errors else ""
        ),
        first_error_path=(
            result.errors[0].path if result.errors else ""
        ),
    ))
except (RuntimeError, asyncio.CancelledError) as e:
    logger.debug(
        "[ConfigValidator] event emit failed: %s", e,
    )
```

OP-11f

key_presence.py

Fichier : py_modules/unifideck/config/key_presence.py | Lignes : 166 | Depend de : (rien)

```
"""config/key_presence.py – Verify every code-referenced config key resolves.
Complements ``ConfigValidator`` (which checks shape + types against the
JSON Schema). This module answers a different question: after the
3-layer merge (``FALLBACK`` → ``defaults/config.json`` → user overrides),
does every key actually read by the backend code resolve to a
non-``None`` value?
```

Why this matters

The ConfigValidator is permissive about optional keys – only 9 out of 105 schema nodes are ``required``. Without this extra check, a key that is:

- * read by code at runtime (e.g. ``config.get("cloud.enabled")``)
- * declared in the schema but not marked required
- * accidentally omitted from ``defaults/config.json``

...would silently return ``None`` and crash the feature that relied on it. This check makes that class of bug fail at boot with a clear error rather than surface as an obscure ``AttributeError`` on ``None`` later.

Design

The list of required-at-runtime keys is **declared in code** (the ``RUNTIME\_REQUIRED\_KEYS`` tuple below) rather than scraped with a regex from call sites. Scraping is fragile – renaming a key in code wouldn't update the scraped list, and the presence check would still pass against the old name. An explicit list forces the developer to update it alongside the call site, which is exactly what we want: a machine-readable contract that code and config stay in sync.

Updating RUNTIME_REQUIRED_KEYS

When adding a new ``config.get("some.key", default)`` or ``\_cfg(config, "some.key", default)`` call anywhere in ``py\_modules/``:

1. Add ``"some.key"`` to ``RUNTIME\_REQUIRED\_KEYS`` below.
2. Add the key to ``defaults/config.json`` with a sensible value.
3. Add the key to ``py\_modules/unifideck/config/schema.json`` under the right section so typos in user overrides are caught.
4. Remove the hardcoded ``default`` arg from the call site – it is now redundant and merely obscures a missing declaration.

The pre-commit hook (``scripts/check\_config\_keys.py``) enforces that every key string literal passed to ``config.get`` or ``\_cfg`` appears in this tuple, so step 1 cannot be forgotten in practice.

```
"""
from __future__ import annotations
import logging
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from .config_manager import ConfigManager
logger = logging.getLogger(__name__)
# Every key the backend code reads via ``config.get`` or ``_cfg``.
# Sorted alphabetically for grep-friendliness; duplicates removed.
# Organised by top-level section so the reader can spot gaps per domain.
RUNTIME_REQUIRED_KEYS: tuple[str, ...] = (
    # accounts
    "accounts.poll_interval_seconds",
    # artwork
    "artwork.download_timeout_seconds",
    "artwork.failure_cooldown_seconds",
```

```
"artwork.steamgriddb_api_base",
"artwork.steamgriddb_api_key",
# auth
"auth.browser_oauth_timeout_seconds",

"auth.browser_poll_interval_seconds",
# binary_resolver
"binary_resolver.version_check_timeout_seconds",
# cache_ttl
"cache_ttl.compat",
"cache_ttl.metacritic_metadata",
"cache_ttl.steam_metadata",
# cdp
"cdp.eval_timeout_seconds",
"cdp.host",
"cdp.port",
"cdp.response_timeout_seconds",
# cloud
"cloud.enabled",
"cloud.root",
"cloud.sync_wait_timeout_seconds",
"cloud.tolerance_seconds",
# compat (ProtonDB + Deck Verified API timeouts)
"compat.deck_verified_timeout_seconds",
"compat.protondb_timeout_seconds",
# dedup – Microsoft Store is intentionally absent so xCloud /
# Game Pass entries are never filtered against Steam-native or
# cross-store duplicates.
"dedup.tracked_stores",
# discovery
"discovery.manifest_filename",
# download
"download.custom_path",
# il8n – whole sub-dict is consumed as a unit by utils/locale
"il8n",
# metadata
"metadata.metacritic.composer_url",
"metadata.metacritic.fetch_timeout_seconds",
"metadata.steam_store.search_timeout_seconds",
"metadata.steam_store.search_url",
"metadata.unifidb.cdn_base",
"metadata.unifidb.fetch_timeout_seconds",
"metadata.unifidb.match_threshold",
# paths
"paths.data_dir",
"paths.games_map",
"paths.sd_card_root",
"paths.steam_candidates",
# probes
"probes.probe_to_features",
"probes.probe_to_handlers",
# stores – per-store sub-dicts consumed whole by the store ctor
"stores.amazon",
"stores.amazon.user_file",
"stores.epic",
"stores.epic.user_file",
"stores.gog.client_secret",
"stores.microsoft.subscription_check_url",
# sync
"sync.artwork_concurrency",
# ui
```

```

    "ui.locale",
)
class KeyPresenceError(RuntimeError):
    """Raised when one or more runtime-required keys resolve to None."""

def assert_all_keys_resolve(config: ConfigManager) -> None:
    """Verify every key in ``RUNTIME_REQUIRED_KEYS`` returns non-None.
    Raises:
        KeyPresenceError: If any required key is missing. The error
            message lists every missing path at once so operators can
            fix their ``defaults/config.json`` in a single edit rather
            than iterating through N reboots.
    This runs once at plugin boot, right after
    ``ConfigValidator.validate_config`` succeeds but before any service
    instantiates. A failure here is fatal – it means the plugin code
    depends on a value that isn't declared anywhere, which is a
    developer error (either RUNTIME_REQUIRED_KEYS or defaults/config.json
    is out of sync with the call sites).
    """
    missing: list[str] = []
    for key in RUNTIME_REQUIRED_KEYS:
        # Pass a sentinel rather than None so keys legitimately set to
        # None in config are distinguished from absent keys. Anything
        # other than the sentinel is considered "present".
        sentinel = object()
        value = config.get(key, sentinel)
        if value is sentinel:
            missing.append(key)
    if missing:
        msg = (
            f"[config] {len(missing)} required key(s) missing from "
            f"the merged config. Expected in defaults/config.json:\n "
            + "\n ".join(missing)
        )
        logger.error(msg)
        raise KeyPresenceError(msg)
    logger.info(
        "[config] key presence check passed: %d keys resolve",
        len(RUNTIME_REQUIRED_KEYS),
    )

def collect_missing_keys(config: ConfigManager) -> list[str]:
    """Return the list of missing keys without raising.
    Useful for diagnostics paths where the caller wants to report what
    went wrong rather than abort. The main boot path uses the stricter
    ``assert_all_keys_resolve`` variant.
    """
    missing: list[str] = []
    sentinel = object()
    for key in RUNTIME_REQUIRED_KEYS:
        if config.get(key, sentinel) is sentinel:
            missing.append(key)
    return missing

```

OP-11g

startup.py

Fichier : py_modules/unifideck/config/startup.py | Lignes : 67 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from unifideck.config import ConfigValidator, ValidationResult
if TYPE_CHECKING:
    from unifideck.config import ConfigManager
    from unifideck.event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
async def validate_config_at_startup(
    *,
    bus: EventBus,
    config: ConfigManager,
    defaults_path: str,
    user_config_path: str,
) -> tuple[ValidationResult, bool]:
    """Validate config at startup."""
    validator = ConfigValidator(bus=bus)
    result: ValidationResult = await validator.validate_config(
        defaults_path=defaults_path,
        user_path=user_config_path,
    )
    if not result.success:
        _log_schema_failure(result)
        return result, True
    logger.info(
        "[Unifideck] config validation OK "
        "(%d section(s) validated)",
        19,
    )
    from unifideck.config.key_presence import collect_missing_keys
    missing = collect_missing_keys(config)
    if missing:
        _log_missing_keys(missing)
        return result, True
    logger.info("[Unifideck] runtime key-presence check OK")
    return result, False
def _log_schema_failure(result: ValidationResult) -> None:
    """Log schema failure."""
    first = result.errors[0] if result.errors else None
    first_path = first.path if first else "<unknown>"
    first_msg = first.message if first else "<unknown>"
    logger.warning(
        "[Unifideck] config validation FAILED – starting in "
        "degraded mode. %d error(s). First: %s: %s",
        len(result.errors),
        first_path,
        first_msg,
    )
def _log_missing_keys(missing: list[str]) -> None:
    """Log missing keys."""
    sample = ", ".join(missing[:10])
    overflow = (
        f" (+{len(missing) - 10} more)"

```

```
        if len(missing) > 10
            else ""
    )
    logger.warning(
        "[Unifideck] %d runtime-required config key(s) "
        "missing from defaults/config.json: %s%s. "
        "Affected features will run with None values. "
        "Add each key to defaults/config.json or remove "
        "it from RUNTIME_REQUIRED_KEYS if the code no "
        "longer reads it.",
        len(missing),
        sample,
        overflow,
    )
```

OP-11h

user_config_path.py

Fichier : py_modules/unifideck/config/user_config_path.py | Lignes : 11 | Depend de : (rien)

```
from __future__ import annotations
import os
def resolve_user_config_path() -> str:
    """Resolve user config path."""
    env = os.environ.get("UNIFIDECK_USER_CONFIG")
    if env:
        return os.path.expanduser(env)
    xdg = os.environ.get("XDG_CONFIG_HOME")
    if xdg:
        return os.path.join(xdg, "unifideck", "config.json")
    return os.path.expanduser("~/config/unifideck/config.json")
```

OP-12

__init__.py

Fichier : py_modules/unifideck/services/__init__.py | Lignes : 0 | Depend de : (rien)

OP-13

__init__.py

Fichier : py_modules/unifideck/services/bootstrap/__init__.py | Lignes : 16 | Depend de : (rien)

```
from __future__ import annotations
from .constructor import bootstrap_services, build_service_subset
from .container import ServiceContainer
from .paths import ServicePaths
from .startup import start_async_services
from .store_injector import inject_store_dependencies
from .teardown import stop_all_services

__all__ = [
    "ServiceContainer",
    "ServicePaths",
    "bootstrap_services",
    "build_service_subset",
    "inject_store_dependencies",
    "start_async_services",
    "stop_all_services",
]
```

OP-13a

paths.py

Fichier : py_modules/unifideck/services/bootstrap/paths.py | Lignes : 63 | Depend de : (rien)

```
from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ...config import ConfigManager
@dataclass
class ServicePaths:
    """Service paths."""
    data_dir: str
    steam_root: str
    shortcuts_path: str
    games_map_path: str
    config_vdf_path: str
    loginusers_path: str
    grid_dir: str
    queue_file: str
    playtime_db: str
    local_save_root: str
    cloud_root: str | None
    @classmethod
    def from_config(cls, config: ConfigManager) -> ServicePaths:
        """From config."""
        from unifideck.steam.library import find_steam_path
        data_dir = str(
            Path(
                config.get("paths.data_dir"),
            ).expanduser(),
        )
        Path(data_dir).mkdir(parents=True, exist_ok=True)
        steam_root = (
            find_steam_path(config)
            or str(Path("~/steam/steam").expanduser())
        )
        steam_root_path = Path(steam_root)
        data_dir_path = Path(data_dir)
        return cls(
            data_dir=data_dir,
            steam_root=steam_root,
            shortcuts_path=str(
                steam_root_path
                / "userdata" / "0" / "config" / "shortcuts.vdf",
            ),
            games_map_path=str(
                Path(
                    config.get("paths.games_map"),
                ).expanduser(),
            ),
            config_vdf_path=str(
                steam_root_path / "config" / "config.vdf",
            ),
            loginusers_path=str(
                steam_root_path / "config" / "loginusers.vdf",
            ),
            grid_dir=str(
```

```
        steam_root_path
        / "userdata" / "0" / "config" / "grid",
    ),
    queue_file=str(data_dir_path / "download_queue.json"),
    playtime_db=str(data_dir_path / "playtime.db"),
    local_save_root=str(data_dir_path / "saves"),
    cloud_root=config.get("cloud.root") or None,
)
```

OP-13b

container.py

Fichier : py_modules/unifideck/services/bootstrap/container.py | Lignes : 39 | Depend de : (rien)

```
from __future__ import annotations
from dataclasses import dataclass
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ...cdp.cdp_client import CDPClient
    from ...core.metrics_collector import MetricsCollector
    from ..account_service import AccountService
    from ..artwork import ArtworkService
    from ..cloud_save import CloudSaveService
    from ..download import DownloadService
    from ..feature_flag_service import FeatureFlagService
    from ..launch_history import LaunchHistoryService
    from ..metadata_service import MetadataService
    from ..microsoft_subscription import (
        MicrosoftSubscriptionService,
    )
    from ..playtime import PlaytimeService
    from ..probe_reaction_service import ProbeReactionService
    from ..proton_service import ProtonService
    from ..security import SecurityService
    from ..shortcut import ShortcutService
@dataclass
class ServiceContainer:
    """Service container."""
    shortcut: ShortcutService | None = None
    download: DownloadService | None = None
    metadata: MetadataService | None = None
    artwork: ArtworkService | None = None
    proton: ProtonService | None = None
    cdp: CDPClient | None = None
    cloudsave: CloudSaveService | None = None
    metrics: MetricsCollector | None = None
    account: AccountService | None = None
    playtime: PlaytimeService | None = None
    feature_flags: FeatureFlagService | None = None
    probe_reaction: ProbeReactionService | None = None
    security: SecurityService | None = None
    launch_history: LaunchHistoryService | None = None
    microsoft_subscription: MicrosoftSubscriptionService | None = None
```

OP-13c

service_defs.py

Fichier : py_modules/unifideck/services/bootstrap/service_defs.py | Lignes : 122 | Depend de : (rien)

```

from __future__ import annotations
from importlib import import_module
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...core.cache_manager import CacheManager
    from ...event_bus.bus_pipeline import BusPipeline
    from ...event_bus.event_bus import EventBus
    from ...stores import StoreRegistry
    from .paths import ServicePaths
_SERVICE_DEFS: tuple[tuple, ...] = (
    (
        "shortcut", "unifideck.services.shortcut",
        "ShortcutService",
        lambda b, r, c, cfg, p, pl: (b, p.shortcuts_path, p.games_map_path),
        lambda b, r, c, cfg, p, pl: {},
    ),
    (
        "download", "unifideck.services.download",
        "DownloadService",
        lambda b, r, c, cfg, p, pl: (b, r, p.queue_file),
        lambda b, r, c, cfg, p, pl: {},
    ),
    (
        "metadata", "unifideck.services.metadata_service",
        "MetadataService",
        lambda b, r, c, cfg, p, pl: (b, c),
        lambda b, r, c, cfg, p, pl: {"config": cfg},
    ),
    (
        "artwork", "unifideck.services.artwork",
        "ArtworkService",
        lambda b, r, c, cfg, p, pl: (b, c, p.grid_dir),
        lambda b, r, c, cfg, p, pl: {"config": cfg},
    ),
    (
        "proton", "unifideck.services.proton_service",
        "ProtonService",
        lambda b, r, c, cfg, p, pl: (b, p.config_vdf_path),
        lambda b, r, c, cfg, p, pl: {},
    ),
    (
        "cdp", "unifideck.cdp.cdp_client",
        "CDPClient",
        lambda b, r, c, cfg, p, pl: (),
        lambda b, r, c, cfg, p, pl: {"config": cfg},
    ),
    (
        "cloudsave", "unifideck.services.cloud_save",
        "CloudSaveService",
        lambda b, r, c, cfg, p, pl: (b, p.local_save_root),
        lambda b, r, c, cfg, p, pl: {"cloud_root": p.cloud_root},
    ),
    (
        "metrics", "unifideck.core.metrics_collector",

```

```

        "MetricsCollector",
        lambda b, r, c, cfg, p, pl: (b,),
        lambda b, r, c, cfg, p, pl: {},
    ),
    (
        "account", "unifideck.services.account_service",
        "AccountService",
        lambda b, r, c, cfg, p, pl: (b, p.loginusers_path),
        lambda b, r, c, cfg, p, pl: {},
    ),
    (
        "playtime", "unifideck.services.playtime",
        "PlaytimeService",
        lambda b, r, c, cfg, p, pl: (b, p.playtime_db),

        lambda b, r, c, cfg, p, pl: {},
    ),
    (
        "feature_flags", "unifideck.services.feature_flag_service",
        "FeatureFlagService",
        lambda b, r, c, cfg, p, pl: (b,),
        lambda b, r, c, cfg, p, pl: {"config": cfg},
    ),
    (
        "probe_reaction", "unifideck.services.probe_reaction_service",
        "ProbeReactionService",
        lambda b, r, c, cfg, p, pl: (b, pl.watchdog),
        lambda b, r, c, cfg, p, pl: {"config": cfg},
    ),
    (
        "security", "unifideck.services.security",
        "SecurityService",
        lambda b, r, c, cfg, p, pl: (b,),
        lambda b, r, c, cfg, p, pl: {
            "config": cfg,
            "replay": pl.replay if pl else None,
        },
    ),
    (
        "launch_history", "unifideck.services.launch_history",
        "LaunchHistoryService",
        lambda b, r, c, cfg, p, pl: (cfg,),
        lambda b, r, c, cfg, p, pl: {"bus": b},
    ),
    (
        "microsoft_subscription",
        "unifideck.services.microsoft_subscription",
        "MicrosoftSubscriptionService",
        lambda b, r, c, cfg, p, pl: (b, c),
        lambda b, r, c, cfg, p, pl: {"config": cfg},
    ),
)
def _instantiate_service(
    def_entry: tuple,
    bus: EventBus,
    registry: StoreRegistry | None,
    cache: CacheManager | None,
    config: ConfigManager,
    paths: ServicePaths,
    pipeline: BusPipeline | None = None,
) -> Any:

```

```
"""Instantiate service."""
_attr, module_path, class_name, build_args, build_kw = def_entry
module = import_module(module_path)
cls = getattr(module, class_name)
args = build_args(bus, registry, cache, config, paths, pipeline)
kwargs = build_kw(bus, registry, cache, config, paths, pipeline)
return cls(*args, **kwargs)
```

OP-13d

constructor.py

Fichier : py_modules/unifideck/services/bootstrap/constructor.py | Lignes : 70 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
from .container import ServiceContainer
from .paths import ServicePaths
from .service_defs import _SERVICE_DEFS, _instantiate_service
if TYPE_CHECKING:
    from collections.abc import Iterable
    from ...config import ConfigManager
    from ...core.cache_manager import CacheManager
    from ...event_bus.bus_pipeline import BusPipeline
    from ...event_bus.event_bus import EventBus
    from ...stores import StoreRegistry
logger = logging.getLogger(__name__)
def bootstrap_services(
    bus: EventBus,
    registry: StoreRegistry,
    cache: CacheManager,
    config: ConfigManager,
    pipeline: BusPipeline,
) -> ServiceContainer:
    """Bootstrap services."""
    paths = ServicePaths.from_config(config)
    container = ServiceContainer()
    for def_entry in _SERVICE_DEFS:
        attr = def_entry[0]
        try:
            instance = _instantiate_service(
                def_entry, bus, registry, cache, config, paths,
                pipeline,
            )
            setattr(container, attr, instance)
            logger.debug(
                "[bootstrap] %s wired: %s.%s",
                attr, def_entry[1], def_entry[2],
            )
        except Exception as e:
            logger.warning(
                "[bootstrap] failed to wire %s (%s.%s): %s",
                attr, def_entry[1], def_entry[2], e,
            )
    return container

def build_service_subset(
    bus: EventBus,
    config: ConfigManager,
    paths: ServicePaths,
    attrs: Iterable[str],
) -> dict[str, Any]:
    """Build service subset."""
    selected = set(attrs)
    services: dict[str, Any] = {}
    for def_entry in _SERVICE_DEFS:
        attr = def_entry[0]

```



```
if attr not in selected:
    continue
try:
    services[attr] = _instantiate_service(
        def_entry, bus, None, None, config, paths, None,
    )
    logger.debug(
        "[subset] %s wired: %s.%s",
        attr, def_entry[1], def_entry[2],
    )
except Exception as e:
    logger.warning(
        "[subset] failed to wire %s (%s.%s): %s",
        attr, def_entry[1], def_entry[2], e,
    )
    services[attr] = None
return services
```

OP-13e

startup.py

Fichier : py_modules/unifideck/services/bootstrap/startup.py | Lignes : 46 | Depend de : (rien)

```
from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from .container import ServiceContainer
logger = logging.getLogger(__name__)
async def start_async_services(container: ServiceContainer) -> None:
    """Start async services."""
    for attr in (
        "download",
        "account",
        "playtime",
        "security",
        "launch_history",
    ):
        svc = getattr(container, attr, None)
        if svc is None:
            continue
        start = getattr(svc, "start", None)
        if start is None:
            continue
        try:
            await start()
        except Exception as e:
            logger.warning(
                "[bootstrap] %s.start failed: %s", attr, e,
            )
    _self_heal_executable_bits()
def _self_heal_executable_bits() -> None:
    """Self heal executable bits."""
    try:
        plugin_dir_path = Path(__file__).resolve().parents[4]
        from ...launcher.packaging import ensure_executable_files
        fixed = ensure_executable_files(plugin_dir_path)
        if fixed > 0:
            logger.info(
                "[bootstrap] executable bit self-heal: "
                "%d file(s) fixed",
                fixed,
            )
    except Exception as e:
        logger.warning(
            "[bootstrap] executable bit self-heal failed: %s",
            e,
        )
```

OP-13f

teardown.py

Fichier : py_modules/unifideck/services/bootstrap/teardown.py | Lignes : 32 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from .container import ServiceContainer
logger = logging.getLogger(__name__)
async def stop_all_services(container: ServiceContainer) -> None:
    """Stop all services."""
    teardown_order = [
        "cloud_prompt",
        "security", "probe_reaction", "feature_flags",
        "playtime", "account", "metrics", "cloudsave",
        "proton", "artwork", "metadata", "download",
        "shortcut", "cdp",
    ]
    for attr in teardown_order:
        svc = getattr(container, attr, None)
        if svc is None:
            continue
        stop_fn = (
            getattr(svc, "stop", None)
            or getattr(svc, "disconnect", None)
        )
        if stop_fn is None:
            continue
        try:
            await stop_fn()
        except Exception as e:
            logger.warning(
                "[bootstrap] %s.%s raised: %s",
                attr, stop_fn.__name__, e,
            )
```

OP-13g

store_injector.py

Fichier : py_modules/unifideck/services/bootstrap/store_injector.py | Lignes : 53 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ...stores import StoreRegistry
    from .container import ServiceContainer
logger = logging.getLogger(__name__)
_STORE_INJECTIONS: dict[str, tuple[tuple[str, str], ...]] = {
    "microsoft": (
        ("_subscription_service", "microsoft_subscription"),
    ),
}
def inject_store_dependencies(
    registry: StoreRegistry | None,
    container: ServiceContainer,
) -> None:
    """Inject store dependencies."""
    if registry is None:
        return
    for store_id, mappings in _STORE_INJECTIONS.items():
        try:
            store = registry.get(store_id)
        except Exception:
            logger.debug(
                "[bootstrap] registry.get(%r) raised, skipping "
                "injections",
                store_id,
            )
            continue
        if store is None:
            continue
        for store_attr, container_attr in mappings:
            svc = getattr(container, container_attr, None)
            if svc is None:
                logger.info(
                    "[bootstrap] %s.%s not injected "
                    "(container.%s is None)",
                    store_id, store_attr, container_attr,
                )
                continue
            try:
                setattr(store, store_attr, svc)
                logger.info(
                    "[bootstrap] injected %s.%s ← container.%s",
                    store_id,
                    store_attr,
                    container_attr,
                )
            except Exception as e:
                logger.warning(
                    "[bootstrap] failed to inject %s.%s: %s",
                    store_id, store_attr, e,
                )
```

OP-14

`__init__.py`

Fichier : `py_modules/unifideck/services/shortcut/__init__.py` | Lignes : 9 | Depend de : (rien)

```
from __future__ import annotations
from .games_map import GameMapEntry, generate_app_id
from .service import UNIFIDECK_TAG, ShortcutService
__all__ = [
    "UNIFIDECK_TAG",
    "GameMapEntry",
    "ShortcutService",
    "generate_app_id",
]
```

OP-14a

service.py

Fichier : py_modules/unifideck/services/shortcut/service.py | Lignes : 73 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
from ...core.types import Events
from ...event_bus.event_bus_devex import auto_wire
from . import persistence
from .events import EventsMixin
from .games_map import GameMapEntry, generate_app_id
from .games_map_mixin import UNIFIDECK_TAG, _GamesMapMixin
from .vdf_shortcuts import _VdfShortcutsMixin
if TYPE_CHECKING:
    from ...event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
__all__ = ["ShortcutService", "UNIFIDECK_TAG"]
class ShortcutService(
    _GamesMapMixin, _VdfShortcutsMixin, EventsMixin,
):
    """Shortcut service."""
    def __init__(
        self,
        bus: EventBus,
        shortcuts_path: str,
        games_map_path: str,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._shortcuts_path = shortcuts_path
        self._games_map_path = games_map_path
        self._shortcuts: list[dict[str, Any]] = []
        self._games_map: dict[str, GameMapEntry] = {}
        self._shortcuts_loaded: bool = False
        self._games_map_loaded: bool = False
        auto_wire(self, self._bus)
        logger.info("[ShortcutService] wired (3 subscriptions)")
    async def stop(self) -> None:
        """Stop."""
        self._bus.off(
            Events.DOWNLOAD_COMPLETE, self._on_download_complete,
        )
        self._bus.off(
            Events.GAME_UNINSTALLED, self._on_game_uninstalled,
        )
        self._bus.off(
            Events.SYNC_COMPLETE, self._on_sync_complete,
        )
    @staticmethod
    def generate_app_id(exe: str, title: str) -> int:
        """Generate app ID."""
        return generate_app_id(exe, title)
    async def _load_shortcuts(self) -> None:
        """Load shortcuts."""
        if self._shortcuts_loaded:
            return
        self._shortcuts_loaded = True
        self._shortcuts = await persistence.load_shortcuts(

```

```
        self._shortcuts_path,
    )
    async def _load_games_map(self) -> None:
        """Load games map."""
        if self._games_map_loaded:
            return
        self._games_map_loaded = True
        self._games_map = await persistence.load_games_map(
            self._games_map_path,
        )

    async def _save_all(self) -> None:
        """Save all."""
        await persistence.save_all(
            self._shortcuts_path,
            self._shortcuts,
            self._games_map_path,
            self._games_map,
        )
```

OP-14b

events.py

Fichier : py_modules/unifideck/services/shortcut/events.py | Lignes : 26 | Depend de : (rien)

```
from __future__ import annotations
from typing import TYPE_CHECKING, Any
from ...core.types import Events, Game
from ...event_bus.event_bus_devex import subscribe
if TYPE_CHECKING:
    pass
class EventsMixin:
    """Events mixin."""
    @subscribe(Events.DOWNLOAD_COMPLETE)
    async def _on_download_complete(self, **kwargs: Any) -> None:
        """On download complete."""
        game = kwargs.get("game")
        if isinstance(game, Game):
            await self.add_game(game)
    @subscribe(Events.GAME_UNINSTALLED)
    async def _on_game_uninstalled(self, **kwargs: Any) -> None:
        """On game uninstalled."""
        app_id = kwargs.get("app_id")
        if isinstance(app_id, int):
            await self.remove_game(app_id)
    @subscribe(Events.SYNC_COMPLETE)
    async def _on_sync_complete(self, **kwargs: Any) -> None:
        """On sync complete."""
        games = kwargs.get("games", [])
        if games:
            await self.reconcile(games)
```


OP-14c

games_map.py

Fichier : py_modules/unifideck/services/shortcut/games_map.py | Lignes : 43 | Depend de : (rien)

```
from __future__ import annotations
import zlib
from pathlib import Path
from typing import NamedTuple
class GameMapEntry(NamedTuple):
    """Game map entry."""
    exe: str
    work_dir: str
def generate_app_id(exe: str, title: str) -> int:
    """Generate app ID."""
    key = (exe + title).encode("utf-8")
    crc = zlib.crc32(key) | 0x80000000
    return crc - 0x100000000 if crc >= 0x80000000 else crc
def parse_games_map(content: str) -> dict[str, GameMapEntry]:
    """Parse games map."""
    result: dict[str, GameMapEntry] = {}
    for raw_line in content.splitlines():
        line = raw_line.strip()
        if not line or line.startswith("#"):
            continue
        if "=" not in line:
            continue
        key, _, value = line.partition("=")
        key = key.strip()
        value = value.strip()
        if not key or not value:
            continue
        if "\t" in value:
            exe, _, work_dir = value.partition("\t")
            exe = exe.strip()
            work_dir = work_dir.strip()
        else:
            exe = value
            work_dir = str(Path(exe).parent) or exe
        result[key] = GameMapEntry(exe=exe, work_dir=work_dir)
    return result
def format_games_map(mapping: dict[str, GameMapEntry]) -> str:
    """Format games map."""
    lines = [
        f"{k}={entry.exe}\t{entry.work_dir}"
        for k, entry in sorted(mapping.items())
    ]
    return "\n".join(lines) + "\n"
```

OP-14d

shortcut.py

Fichier : py_modules/unifideck/services/shortcut/auth_shortcut.py | Lignes : 112 | Depend de : (rien)

```

from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING
from ...core.types import Events, Result
from .games_map import generate_app_id
if TYPE_CHECKING:
    from .service import ShortcutService
logger = logging.getLogger(__name__)
_UNIFIDECK_TAG = "Unifideck"
async def build_auth_shortcut(
    service: ShortcutService,
    store: str,
    launcher_path: str,
    title: str,
) -> Result:
    """Build auth shortcut."""
    if not launcher_path:
        return Result(success=False, error="no_launcher_path")
    if not title:
        return Result(success=False, error="no_title")
    app_id = generate_app_id(launcher_path, title)
    unsigned_id = app_id if app_id >= 0 else app_id + 2**32
    launch_options = (
        f"{store}:{store}-auth "
        f"UNIFIDECK_{store.upper()}_ACTION=auth"
    )
    await service._load_shortcuts()
    correct_idx, dirty = _prune_malformed_duplicates(
        service, store, app_id, title,
    )
    if correct_idx is None:
        entry = _build_auth_entry(
            app_id, title, launcher_path, launch_options, store,
        )
        service._shortcuts.append(entry)
        dirty = True
        logger.info(
            "[ShortcutService] created auth shortcut for %s "
            "(app_id=%d, unsigned=%d)",
            store, app_id, unsigned_id,
        )
    if dirty:
        await service._save_all()
    await service._bus.emit(
        Events.SHORTCUT_CREATED,
        store=store,
        app_id=app_id,
        unsigned_id=unsigned_id,
        title=title,
        is_auth=True,
    )
    return Result(success=True, error=str(unsigned_id))

def _prune_malformed_duplicates(

```

```

    service: ShortcutService,
    store: str,
    app_id: int,
    title: str,
) -> tuple[int | None, bool]:

    """Prune malformed duplicates."""
    auth_prefix = f"{store}:{store}-auth"
    matching = [
        (i, s) for i, s in enumerate(service._shortcuts)
        if auth_prefix in s.get("LaunchOptions", "")
    ]
    correct_idx: int | None = None
    for i, entry in matching:
        if (
            entry.get("appid") == app_id
            and entry.get("AppName") == title
            and "UNIFIDECK_" in entry.get("LaunchOptions", "")
        ):
            correct_idx = i
            break
    dirty = False
    if matching:
        malformed_indices = sorted(
            [i for i, _ in matching if i != correct_idx],
            reverse=True,
        )
        for idx in malformed_indices:
            logger.warning(
                "[ShortcutService] removing malformed auth "
                "shortcut for %s at idx=%d",
                store, idx,
            )
            del service._shortcuts[idx]
            dirty = True
    return correct_idx, dirty

def _build_auth_entry(
    app_id: int,
    title: str,
    launcher_path: str,
    launch_options: str,
    store: str,
) -> dict:
    """Build auth entry."""
    return {
        "appid": app_id,
        "AppName": title,
        "Exe": f'"{launcher_path}"',
        "StartDir": '"' + str(Path(launcher_path).parent) + '"',
        "LaunchOptions": launch_options,
        "IsHidden": 1,
        "AllowDesktopConfig": 1,
        "OpenVR": 0,
        "icon": "",
        "tags": {
            "0": _UNIFIDECK_TAG,
            "1": store.capitalize(),
        },
    }

```

OP-14e

games_map_mixin.py

Fichier : py_modules/unifideck/services/shortcut/games_map_mixin.py | Lignes : 138 | Depend de : (rien)

```

from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING, Any
from .games_map import GameMapEntry, generate_app_id
if TYPE_CHECKING:
    from ...core.types import Game
    from ...event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
UNIFIDECK_TAG = "Unifideck"
class _GamesMapMixin:
    """Games map mixin."""
    _bus: EventBus
    _shortcuts: list[dict[str, Any]]
    _games_map: dict[str, GameMapEntry]
    async def add_game(self, game: Game) -> Any:
        """Add game."""
        from ...core.types import Events, Result
        if not game.exe_path:
            return Result(success=False, error="no_executable")
        app_id = generate_app_id(game.exe_path, game.title)
        await self._load_shortcuts()
        await self._load_games_map()
        entry = self._build_shortcut_entry(game, app_id)
        existing_idx = next(
            (
                i for i, s in enumerate(self._shortcuts)
                if s.get("appid") == app_id
            ),
            None,
        )
        if existing_idx is not None:
            self._shortcuts[existing_idx] = entry
        else:
            self._shortcuts.append(entry)
        work_dir = (
            game.install_path or str(Path(game.exe_path).parent)
        )
        key = f"{game.store}:{game.store_game_id}"
        self._games_map[key] = GameMapEntry(
            exe=game.exe_path, work_dir=work_dir,
        )
        await self._save_all()
        await self._bus.emit(
            Events.GAME_INSTALLED,
            store=game.store,
            game_id=game.store_game_id,
            app_id=app_id,
        )
        return Result(success=True)
    async def get_exe_for_game_key(
        self, store: str, game_id: str,
    ) -> str | None:
        """Get exe for game key."""
        entry = await self.get_entry_for_game_key(store, game_id)

```

```

        return entry.exe if entry else None
    async def get_entry_for_game_key(
        self, store: str, game_id: str,
    ) -> GameMapEntry | None:
        """Get entry for game key."""
        await self._load_games_map()
        return self._games_map.get(f"{store}:{game_id}")

    async def remove_game(self, app_id: int) -> Any:

        """Remove game."""
        from ...core.types import Result
        await self._load_shortcuts()
        before = len(self._shortcuts)
        self._shortcuts = [
            s for s in self._shortcuts
            if s.get("appid") != app_id
        ]
        if len(self._shortcuts) == before:
            return Result(success=False, error="not_found")
        await self._save_all()
        return Result(success=True)

    async def reconcile(self, games: list[Game]) -> Any:
        """Reconcile."""
        from ...core.types import Result
        await self._load_shortcuts()
        await self._load_games_map()
        target_ids = {
            generate_app_id(g.exe_path, g.title): g
            for g in games
            if g.exe_path and g.installed
        }
        kept = [
            s for s in self._shortcuts
            if (UNIFIDECK_TAG not in s.get("tags", {}).values())
            or (s.get("appid") in target_ids)
        ]
        existing_ids = {s.get("appid") for s in kept}
        for app_id, game in target_ids.items():
            if app_id not in existing_ids:
                kept.append(
                    self._build_shortcut_entry(game, app_id),
                )
        self._shortcuts = kept
        target_keys = {
            f"{g.store}:{g.store_game_id}": g
            for g in games
            if g.exe_path and g.installed
        }
        for stale in list(self._games_map.keys()):
            if stale not in target_keys:
                del self._games_map[stale]
        for key, g in target_keys.items():
            exe_path = g.exe_path
            assert exe_path is not None
            work_dir = (
                g.install_path or str(Path(exe_path).parent)
            )
            self._games_map[key] = GameMapEntry(
                exe=exe_path, work_dir=work_dir,
            )

```

```
        await self._save_all()
        logger.info(
            "[ShortcutService] reconciled %d shortcuts",
            len(kept),
        )
        return Result(success=True)
def _build_shortcut_entry(
    self, game: Game, app_id: int,
) -> dict[str, Any]:
    """Build shortcut entry."""
    return {
        "appid": app_id,
        "AppName": game.title,
        "Exe": f'"{game.exe_path}"',
        "StartDir": f'"{game.install_path or ""}"',
        "LaunchOptions": (
            f"{game.store}:{game.store_game_id}"
        ),
        "icon": game.icon_url or "",
        "tags": {
            "0": UNIFIDECK_TAG,
            "1": game.store.capitalize(),
        },
    }
```

OP-14f

vdf_shortcuts.py

Fichier : py_modules/unifideck/services/shortcut/vdf_shortcuts.py | Lignes : 33 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import Any
from . import auth_shortcut as _auth
logger = logging.getLogger(__name__)
class _VdfShortcutsMixin:
    """Vdf shortcuts mixin."""
    _shortcuts: list[dict[str, Any]]
    async def read_shortcuts(self) -> dict[str, Any]:
        """Read shortcuts."""
        await self._load_shortcuts()
        return {
            "shortcuts": {
                str(i): entry
                for i, entry in enumerate(self._shortcuts)
            },
        }
    async def write_shortcuts(self, data: dict[str, Any]) -> None:
        """Write shortcuts."""
        shortcuts_map = data.get("shortcuts", {})
        self._shortcuts = list(shortcuts_map.values())
        await self._save_all()
    async def add_auth_shortcut(
        self,
        store: str,
        launcher_path: str,
        title: str,
    ) -> Any:
        """Add auth shortcut."""
        return await _auth.build_auth_shortcut(
            self,
            store, launcher_path, title,
        )
```

OP-14g

persistence.py

Fichier : py_modules/unifideck/services/shortcut/persistence.py | Lignes : 55 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os as _os
from typing import Any
from ...core.io import async_file_ops as aio
from ...steam.shortcuts import read_shortcuts as _vdf_read, write_shortcuts as _vdf_write
from .games_map import GameMapEntry, format_games_map, parse_games_map
logger = logging.getLogger(__name__)
_GAMES_MAP_READ_ATTEMPTS = 3
_GAMES_MAP_RETRY_DELAY_S = 0.1
async def load_shortcuts(shortcuts_path: str) -> list[dict[str, Any]]:
    """Load shortcuts."""
    if not await aio.is_file(shortcuts_path):
        return []
    return await asyncio.to_thread(_vdf_read, shortcuts_path)
async def load_games_map(
    games_map_path: str,
) -> dict[str, GameMapEntry]:
    """Load games map."""
    if not await aio.is_file(games_map_path):
        return {}
    last_err: Exception | None = None
    for attempt in range(_GAMES_MAP_READ_ATTEMPTS):
        try:
            content = await aio.read_text(games_map_path)
            if content is None:
                last_err = OSError("read_text returned None")
                continue
            return parse_games_map(content)
        except Exception as err:
            last_err = err
            logger.warning(
                "[ShortcutService] games.map read attempt %d/%d failed: %s",
                attempt + 1, _GAMES_MAP_READ_ATTEMPTS, err,
            )
            await asyncio.sleep(_GAMES_MAP_RETRY_DELAY_S)
    logger.error(
        "[ShortcutService] games.map read gave up after %d attempts "
        "(last error: %s), starting empty",
        _GAMES_MAP_READ_ATTEMPTS, last_err,
    )
    return {}
async def save_all(
    shortcuts_path: str,
    shortcuts: list[dict[str, Any]],
    games_map_path: str,
    games_map: dict[str, GameMapEntry],
) -> None:
    """Save all."""
    await asyncio.to_thread(_vdf_write, shortcuts_path, shortcuts)
    content = format_games_map(games_map)
    tmp_path = f"{games_map_path}.tmp"
    await aio.write_text(tmp_path, content)
    await asyncio.to_thread(_os.replace, tmp_path, games_map_path)

```


OP-15

__init__.py

Fichier : py_modules/unifideck/services/download/__init__.py | Lignes : 8 | Depend de : (rien)

```
from __future__ import annotations
from .models import DownloadItem, classify_download_error
from .service import DownloadService
__all__ = [
    "DownloadItem",
    "DownloadService",
    "classify_download_error",
]
```

OP-15a

service.py

Fichier : py_modules/unifideck/services/download/service.py | Lignes : 102 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import Any
from ...core.types import Events, Result
from ...event_bus.event_bus import EventBus
from ...stores import StoreRegistry
from . import persistence
from .models import DownloadItem
from .validators import item_key, validate_path
from .worker import _WorkerMixin
logger = logging.getLogger(__name__)
class DownloadService(_WorkerMixin):
    """Download service."""
    def __init__(
        self,
        bus: EventBus,
        registry: StoreRegistry,
        queue_file: str,
        max_concurrent: int = 1,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._registry = registry
        self._queue_file = queue_file
        self._max_concurrent = max_concurrent
        self._queue: list[DownloadItem] = []
        self._running: dict[str, DownloadItem] = {}
        self._lock = asyncio.Lock()
        self._worker_task: asyncio.Task | None = None
    async def start(self) -> None:
        """Start."""
        await self._load_queue()
        self._worker_task = asyncio.create_task(self._worker_loop())
        logger.info(
            "[DownloadService] started with %d pending",
            len(self._queue),
        )
    async def stop(self) -> None:
        """Stop."""
        if self._worker_task:
            self._worker_task.cancel()
            try:
                await self._worker_task
            except asyncio.CancelledError:
                pass
    async def add(
        self, store: str, game_id: str,
        install_path: str, title: str = "",
    ) -> Result:
        """Add."""
        key = f"{store}:{game_id}"
        if key in self._running:

```

```

        return Result(success=False, error="already_running")
    if any(item_key(i) == key for i in self._queue):
        return Result(success=False, error="already_queued")
    validation = validate_path(install_path)
    if not validation.success:
        return validation
    item = DownloadItem(
        store=store, game_id=game_id,
        install_path=install_path, title=title,
    )
    async with self._lock:
        self._queue.append(item)
        await self._save_queue()
    await self._bus.emit(
        Events.DOWNLOAD_QUEUED,
        store=store, game_id=game_id,
    )
    return Result(success=True)
async def cancel(self, store: str, game_id: str) -> Result:
    """Check whether ncel."""
    key = f"{store}:{game_id}"
    async with self._lock:
        before = len(self._queue)
        self._queue = [
            i for i in self._queue
            if item_key(i) != key
        ]
        removed = len(self._queue) < before
        if removed:
            await self._save_queue()
    if removed:
        await self._bus.emit(
            Events.DOWNLOAD_CANCELLED,
            store=store, game_id=game_id,
        )
        return Result(success=True)
    return Result(success=False, error="not_in_queue")
def get_queue(self) -> dict[str, Any]:
    """Get queue."""
    return {
        "queued": [i.to_dict() for i in self._queue],
        "running": [i.to_dict() for i in self._running.values()],
    }
async def _load_queue(self) -> None:
    """Load queue."""
    self._queue = await persistence.load_queue(self._queue_file)
async def _save_queue(self) -> None:
    """Save queue."""
    await persistence.save_queue(self._queue_file, self._queue)

```

OP-15b

worker.py

Fichier : py_modules/unifideck/services/download/worker.py | Lignes : 100 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
from typing import TYPE_CHECKING
from ...core.types import Events, InstallResult
from .models import DownloadItem, classify_download_error
from .validators import item_key
if TYPE_CHECKING:
    from ...event_bus.event_bus import EventBus
    from ...stores import StoreRegistry
logger = logging.getLogger(__name__)
class _WorkerMixin:
    """Worker mixin."""
    _bus: EventBus
    _registry: StoreRegistry
    _lock: asyncio.Lock
    _max_concurrent: int
    _queue: list[DownloadItem]
    _running: dict[str, DownloadItem]
    async def _worker_loop(self) -> None:
        """Worker loop."""
        while True:
            if (
                len(self._running) >= self._max_concurrent
                or not self._queue
            ):
                await asyncio.sleep(0.5)
                continue
            async with self._lock:
                if not self._queue:
                    continue
                item = self._queue.pop(0)
                key = item_key(item)
                self._running[key] = item
                await self._save_queue()
                asyncio.create_task(self._run_install(item))

    async def _run_install(self, item: DownloadItem) -> None:
        """Run install."""
        store = self._registry.get(item.store)
        if store is None:
            logger.error(
                "[DownloadService] unknown store: %s",
                item.store,
            )
            item.status = "failed"
            item.error = "unknown_store"
            self._cleanup_running(item)
            return
        item.status = "running"
        await self._bus.emit(
            Events.DOWNLOAD_STARTED,
            store=item.store, game_id=item.game_id,
        )
```

```
try:
    result: InstallResult = await store.install_game(
        item.game_id,
        base_path=item.install_path,
        install_path=item.install_path,
        progress_cb=lambda p: self._update_progress(item, p),
    )
except Exception as e:
    logger.exception("[DownloadService] install error")
    item.status = "failed"
    item.error = classify_download_error(e)
    await self._bus.emit(
        Events.DOWNLOAD_FAILED,
        store=item.store,
        game_id=item.game_id,
        error=item.error,
    )
    self._cleanup_running(item)
    return
if result.success:
    item.status = "complete"
    item.progress = 100.0
    await self._bus.emit(
        Events.DOWNLOAD_COMPLETE,
        store=item.store, game_id=item.game_id,
        install_path=result.install_path,
        executable=result.metadata.get("executable"),
    )
else:
    item.status = "failed"
    item.error = result.error or "unknown"
    await self._bus.emit(
        Events.DOWNLOAD_FAILED,
        store=item.store,
        game_id=item.game_id,
        error=item.error,
    )
    self._cleanup_running(item)
def _cleanup_running(self, item: DownloadItem) -> None:
    """Cleanup running."""
    key = item_key(item)
    self._running.pop(key, None)
def _update_progress(
    self, item: DownloadItem, progress: float,
) -> None:
    """Update progress."""
    item.progress = progress
```

OP-15c

models.py

Fichier : py_modules/unifideck/services/download/models.py | Lignes : 45 | Depend de : (rien)

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Any
@dataclass
class DownloadItem:
    """Download item."""
    store: str
    game_id: str
    install_path: str
    title: str = ""
    progress: float = 0.0
    status: str = "queued"
    error: str = ""
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        return {
            "store": self.store,
            "game_id": self.game_id,
            "install_path": self.install_path,
            "title": self.title,
            "progress": self.progress,
            "status": self.status,
            "error": self.error,
        }
    @classmethod
    def from_dict(cls, d: dict[str, Any]) -> DownloadItem:
        """From dict."""
        return cls(**{
            k: v for k, v in d.items()
            if k in cls.__dataclass_fields__
        })
def classify_download_error(exc: Exception) -> str:
    """Classify download error."""
    msg = str(exc).lower()
    if "permission" in msg or "denied" in msg:
        return "permission_denied"
    if "no space" in msg or "disk full" in msg:
        return "disk_full"
    if "timeout" in msg:
        return "timeout"
    if "network" in msg or "connection" in msg:
        return "network_error"
    if "not found" in msg or "404" in msg:
        return "not_found"
    return "unknown_error"
```

OP-15d

validators.py

Fichier : py_modules/unifideck/services/download/validators.py | Lignes : 36 | Depend de : (rien)

```
from __future__ import annotations
import os
from pathlib import Path
from ...core.types import Result
from .models import DownloadItem

def item_key(item: DownloadItem) -> str:
    """Item key."""
    return f"{item.store}:{item.game_id}"

def validate_path(path: str) -> Result:
    """Validate path."""
    if not path:
        return Result(success=False, error="empty_path")
    p = Path(path)
    if not p.is_dir():
        try:
            p.mkdir(parents=True, exist_ok=True)
        except OSError as e:
            return Result(
                success=False,
                error=f"mkdir_failed: {e}",
            )
    if not os.access(path, os.W_OK):
        return Result(success=False, error="not_writable")
    try:
        stat = os.statvfs(path)
        free_gb = (
            stat.f_bavail * stat.f_frsize
        ) / (1024 ** 3)
        if free_gb < 1.0:
            return Result(
                success=False,
                error=f"low_space:{free_gb:.1f}GB",
            )
    except OSError:
        pass
    return Result(success=True)
```

OP-15e

persistence.py

Fichier : py_modules/unifideck/services/download/persistence.py | Lignes : 33 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import Any, cast
from ...core.io import async_file_ops as aio
from .models import DownloadItem
logger = logging.getLogger(__name__)
async def load_queue(queue_file: str) -> list[DownloadItem]:
    """Load queue."""
    if not await aio.is_file(queue_file):
        return []
    try:
        raw_data = await aio.read_json(queue_file)
        data: list[dict[str, Any]] = (
            cast("list[dict[str, Any]]", raw_data)
            if isinstance(raw_data, list) else []
        )
    except Exception as e:
        logger.warning(
            "[DownloadService] queue load failed: %s", e,
        )
    return []
    return [DownloadItem.from_dict(raw) for raw in data]
async def save_queue(
    queue_file: str, queue: list[DownloadItem],
) -> None:
    """Save queue."""
    data = [i.to_dict() for i in queue]
    try:
        await aio.write_json(queue_file, data)
    except Exception as e:
        logger.warning(
            "[DownloadService] queue save failed: %s", e,
        )
```


OP-16

__init__.py

Fichier : py_modules/unifideck/services/artwork/__init__.py | Lignes : 3 | Depend de : (rien)

```
from __future__ import annotations
from .service import ArtworkService
__all__ = ["ArtworkService"]
```

OP-16a

service.py

Fichier : py_modules/unifideck/services/artwork/service.py | Lignes : 99 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import TYPE_CHECKING
from ...core.cache_manager import CacheManager
from ...core.types import Events
from ...event_bus.event_bus import EventBus
from ...utils.config_helpers import get_cfg
from .event_handlers import _EventHandlersMixin
from .fetcher import download_and_save, find_artwork_url
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
DEFAULT_MAX_CONCURRENT = 4
DEFAULT_FAILURE_COOLDOWN = 3600
DEFAULT_DOWNLOAD_TIMEOUT = 30
CACHE_NAMESPACE = "artwork_attempts"
class ArtworkService(_EventHandlersMixin):
    """Artwork service."""
    def __init__(
        self,
        bus: EventBus,
        cache: CacheManager,
        grid_dir: str,
        api_key: str | None = None,
        config: ConfigManager | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._cache = cache
        self._grid_dir = grid_dir
        self._config = config
        self._api_key = api_key or get_cfg(
            config, "artwork.steamgriddb_api_key", ""
        )
        max_conc = int(get_cfg(
            config, "sync.artwork_concurrency",
            DEFAULT_MAX_CONCURRENT,
        ))
        self._semaphore = asyncio.Semaphore(max_conc)
        self._failure_cooldown = int(get_cfg(
            config, "artwork.failure_cooldown_seconds",
            DEFAULT_FAILURE_COOLDOWN,
        ))
        self._download_timeout = int(get_cfg(
            config, "artwork.download_timeout_seconds",
            DEFAULT_DOWNLOAD_TIMEOUT,
        ))
        self._cache.register(CACHE_NAMESPACE)
        from ...event_bus.event_bus_devex import auto_wire
        auto_wire(self, self._bus)
        logger.info(
            "[ArtworkService] wired (4 subscriptions, grid_dir=%s)",
            self._grid_dir,
        )

```

```
async def stop(self) -> None:
    """Stop."""
    self._bus.off(Events.GAME_INSTALLED, self._on_game_installed)
    self._bus.off(Events.SYNC_COMPLETE, self._on_sync_complete)
    self._bus.off(Events.ARTWORK_REQUEST, self._on_artwork_request)
    self._bus.off(Events.SHORTCUT_CREATED, self._on_shortcut_created)

async def fetch_artwork(self, app_id: int, store: str,
game_id: str, title: str) -> bool:

    """Fetch artwork."""
    async with self._semaphore:
        attempts = (
            self._cache.get(CACHE_NAMESPACE, str(app_id))
            or {}
        )
        if attempts.get("failed_at"):
            import time as _t
            if (
                _t.time() - attempts["failed_at"]
                < self._failure_cooldown
            ):
                return False
        any_saved = False
        for kind in ("grid", "hero", "logo", "icon"):
            url = await find_artwork_url(
                title, kind, self._api_key, self._config,
            )
            if not url:
                continue
            saved = await download_and_save(
                self._grid_dir, app_id, kind, url,
                self._download_timeout,
            )
            if saved:
                any_saved = True
        if any_saved:
            attempts["last_success"] = True
            attempts.pop("failed_at", None)
        else:
            import time as _t
            attempts["failed_at"] = _t.time()
        self._cache.set(
            CACHE_NAMESPACE, str(app_id), attempts,
        )
        return any_saved
```

OP-16b

event_handlers.py

Fichier : py_modules/unifideck/services/artwork/event_handlers.py | Lignes : 80 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import TYPE_CHECKING, Any
from ...core.types import Events
from ...event_bus.event_bus_devex import subscribe
from .fetcher import has_artwork
if TYPE_CHECKING:
    from ...core.types import Game
logger = logging.getLogger(__name__)
class _EventHandlersMixin:
    """Event handlers mixin."""
    _grid_dir: str
    @subscribe(Events.GAME_INSTALLED)
    async def _on_game_installed(self, **kwargs: Any) -> None:
        """On game installed."""
        app_id = kwargs.get("app_id")
        store = kwargs.get("store")
        game_id = kwargs.get("game_id")
        title = kwargs.get("title", "")
        if app_id and store and game_id:
            await self.fetch_artwork(app_id, store, game_id, title)
    @subscribe(Events.ARTWORK_REQUEST)
    async def _on_artwork_request(self, **kwargs: Any) -> None:
        """On artwork request."""
        app_id = kwargs.get("app_id")
        title = kwargs.get("title", "")
        if not app_id or not title:
            logger.debug(
                "[ArtworkService] ARTWORK_REQUEST ignored: "
                "missing app_id or title",
            )
            return
        force = bool(kwargs.get("force", False))
        if not force and await has_artwork(self._grid_dir, app_id):
            logger.debug(
                "[ArtworkService] artwork already present "
                "for app_id=%d, skipping",
                app_id,
            )
            return
        store = kwargs.get("store", "")
        game_id = kwargs.get("game_id", "")
        await self.fetch_artwork(app_id, store, game_id, title)
    @subscribe(Events.SHORTCUT_CREATED)
    async def _on_shortcut_created(self, **kwargs: Any) -> None:
        """On shortcut created."""
        if not kwargs.get("is_auth"):
            return
        unsigned_id = kwargs.get("unsigned_id")
        store = kwargs.get("store", "")
        if not unsigned_id or not store:
            return
        title_for_lookup = {
            "amazon": "Amazon Games",

```

```
        "epic": "Epic Games",
        "gog": "GOG Galaxy",
        "microsoft": "Xbox",
        "ubisoft": "Ubisoft Connect",
    }.get(store, store.capitalize())
    if not await has_artwork(self._grid_dir, unsigned_id):
        await self.fetch_artwork(
            unsigned_id, store,
            f"{store}-auth", title_for_lookup,
        )

@subscribe(Events.SYNC_COMPLETE)
async def _on_sync_complete(self, **kwargs: Any) -> None:
    """On sync complete."""
    games: list[Game] = kwargs.get("games", [])
    tasks = []
    for game in games:
        if not game.app_id or not game.title:
            continue
        if await has_artwork(self._grid_dir, game.app_id):
            continue
        tasks.append(self.fetch_artwork(
            game.app_id, game.store, game.store_game_id, game.title,
        ))
    if tasks:
        await asyncio.gather(*tasks, return_exceptions=True)
```

OP-16c

fetcher.py

Fichier : py_modules/unifideck/services/artwork/fetcher.py | Lignes : 74 | Depend de : (rien)

```

from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
_KIND_SUFFIX = {
    "grid": "p.jpg",
    "hero": "_hero.jpg",
    "logo": "_logo.png",
    "icon": "_icon.jpg",
}
}
async def has_artwork(grid_dir: str, app_id: int) -> bool:
    """Check whether artwork."""
    from ...core.io import async_file_ops as aio
    grid_path = Path(grid_dir)
    grid_file = str(grid_path / f"{app_id}p.jpg")
    hero_file = str(grid_path / f"{app_id}_hero.jpg")
    return (
        await aio.is_file(grid_file)
        and await aio.is_file(hero_file)
    )
async def find_artwork_url(
    title: str,
    kind: str,
    api_key: str,
    config: ConfigManager | None,
) -> str | None:
    """Find artwork URL."""
    try:
        from ...steam import steamgriddb
        return await steamgriddb.search_artwork(
            title, kind, api_key, config=config,
        )
    except Exception as e:
        logger.debug(
            "[ArtworkService] search failed (%s/%s): %s",
            title, kind, e,
        )
    return None

async def download_and_save(
    grid_dir: str,
    app_id: int,
    kind: str,
    url: str,
    timeout: int,
) -> bool:
    """Download and save."""
    suffix = _KIND_SUFFIX.get(kind, ".jpg")
    target = str(Path(grid_dir) / f"{app_id}{suffix}")
    try:
        import aiohttp

```

```
    async with (
        aiohttp.ClientSession() as session,
        session.get(url, timeout=timeout) as resp,
    ):
        if resp.status != 200:
            return False
        data = await resp.read()
except Exception as e:
    logger.debug(
        "[ArtworkService] download failed: %s", e,
    )
    return False
from ...core.io import async_file_ops as aio
try:
    await aio.write_bytes(target, data)
    return True
except Exception as e:
    logger.warning(
        "[ArtworkService] save failed: %s", e,
    )
    return False
```

OP-17

__init__.py

Fichier : py_modules/unifideck/services/cloud_save/__init__.py | Lignes : 3 | Depend de : (rien)

```
from __future__ import annotations
from .service import CloudSaveService
__all__ = ["CloudSaveService"]
```


OP-17a

service.py

Fichier : py_modules/unifideck/services/cloud_save/service.py | Lignes : 60 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import TYPE_CHECKING, Any
from ...core.types import Events
from ...event_bus.event_bus import EventBus
from ...event_bus.event_bus_devex import auto_wire, subscribe
from ...utils.config_helpers import get_cfg
from .paths import local_save_dir
from .sync import _SyncMixin
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
class CloudSaveService(_SyncMixin):
    """Cloud save service."""
    def __init__(
        self,
        bus: EventBus,
        local_save_root: str,
        cloud_root: str | None,
        config: ConfigManager | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._local_root = local_save_root
        self._cloud_root = cloud_root
        self._syncing: dict[str, asyncio.Event] = {}
        self._enabled = bool(get_cfg(config, "cloud.enabled", True))
        self._tolerance = float(
            get_cfg(config, "cloud.tolerance_seconds", 2),
        )
        self._sync_wait_timeout = float(
            get_cfg(config, "cloud.sync_wait_timeout_seconds", 5),
        )
        auto_wire(self, self._bus)
        logger.info(
            "[CloudSaveService] wired (2 subscriptions, cloud=%s)",
            self._cloud_root or "DISABLED",
        )
    async def stop(self) -> None:
        """Stop."""
        self._bus.off(Events.GAME_LAUNCHED, self._on_game_launched)
        self._bus.off(Events.GAME_STOPPED, self._on_game_stopped)
    @subscribe(Events.GAME_LAUNCHED)
    async def _on_game_launched(self, **kwargs: Any) -> None:
        """On game launched."""
        store = kwargs.get("store")
        game_id = kwargs.get("game_id")
        if store and game_id and self._cloud_root:
            await self._sync_down(store, game_id)
    @subscribe(Events.GAME_STOPPED)
    async def _on_game_stopped(self, **kwargs: Any) -> None:
        """On game stopped."""
        store = kwargs.get("store")
        game_id = kwargs.get("game_id")

```

```
    if store and game_id and self._cloud_root:
        await self.sync_up(store, game_id)
def get_local_save_dir(self, store: str, game_id: str) -> str:
    """Get local save dir."""
    return local_save_dir(self._local_root, store, game_id)
```

OP-17b

sync.py

Fichier : py_modules/unifideck/services/cloud_save/sync.py | Lignes : 109 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
from typing import TYPE_CHECKING
from .fs_ops import copy_tree
from .manifest import build_manifest, read_manifest, write_manifest
from .paths import local_save_dir, remote_save_dir
if TYPE_CHECKING:
    from ...core.types import Result
    from ...event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
class _SyncMixin:
    """Sync mixin."""
    _bus: EventBus
    _cloud_root: str | None
    _local_root: str
    _syncing: dict[str, asyncio.Event]
    _tolerance: float
    _sync_wait_timeout: float
    async def sync_down(self, store: str, game_id: str) -> Result:
        """Sync down."""
        from ...core.types import Result
        if not self._cloud_root:
            return Result(success=True)
        key = f"{store}:{game_id}"
        if key in self._syncing:
            return Result(success=False, error="already_syncing")
        self._syncing[key] = asyncio.Event()
        try:
            local = local_save_dir(self._local_root, store, game_id)
            remote = remote_save_dir(self._cloud_root, store, game_id)
            remote_manifest = await read_manifest(remote)
            if not remote_manifest:
                return Result(success=True)
            local_manifest = await build_manifest(local)
            remote_ts = max(remote_manifest.values(), default=0)
            local_ts = max(local_manifest.values(), default=0)
            if local_ts > remote_ts + self._tolerance:
                await self._bus.emit(
                    "sync_conflict",
                    store=store, game_id=game_id,
                    local_ts=local_ts, remote_ts=remote_ts,
                )
                return Result(success=False, error="conflict")
            if remote_ts > local_ts:
                await asyncio.to_thread(
                    copy_tree, remote, local,
                    skip_manifest=True,
                )
                logger.info(
                    "[CloudSaveService] sync_down %s/%s: %d files copied",
                    store, game_id, len(remote_manifest),
                )
            return Result(success=True)

```

```

    finally:
        self._syncing[key].set()
        self._syncing.pop(key, None)

async def sync_up(self, store: str, game_id: str) -> Result:

    """Sync up."""
    from ...core.types import Result
    if not self._cloud_root:
        return Result(success=True)
    key = f"{store}:{game_id}"
    if key in self._syncing:
        try:
            await asyncio.wait_for(
                self._syncing[key].wait(),
                timeout=self._sync_wait_timeout,
            )
        except TimeoutError:
            return Result(
                success=False,
                error="down_sync_in_progress",
            )
    local = local_save_dir(self._local_root, store, game_id)
    remote = remote_save_dir(self._cloud_root, store, game_id)
    if not await asyncio.to_thread(os.path.isdir, local):
        return Result(success=True)
    await asyncio.to_thread(os.makedirs, remote, exist_ok=True)
    await asyncio.to_thread(
        copy_tree, local, remote,
        skip_manifest=True,
    )
    manifest = await build_manifest(local)
    await write_manifest(remote, manifest)
    logger.info(
        "[CloudSaveService] sync_up %s/%s: %d files uploaded",
        store, game_id, len(manifest),
    )
    return Result(success=True)

async def resolve_conflict(
    self, store: str, game_id: str, choice: str,
) -> Result:
    """Resolve conflict."""
    from ...core.types import Result
    if not self._cloud_root:
        return Result(success=True)
    if choice == "local":
        return await self.sync_up(store, game_id)
    if choice == "remote":
        local = local_save_dir(self._local_root, store, game_id)
        remote = remote_save_dir(self._cloud_root, store, game_id)
        await asyncio.to_thread(
            copy_tree, remote, local,
            skip_manifest=True,
        )
        return Result(success=True)
    return Result(success=False, error="invalid_choice")

```

OP-17c

manifest.py

Fichier : py_modules/unifideck/services/cloud_save/manifest.py | Lignes : 45 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import json
import logging
from pathlib import Path
from typing import cast
from .constants import MANIFEST_FILE
from .fs_ops import read_text, walk_mtimes, write_text
logger = logging.getLogger(__name__)
async def read_manifest(directory: str) -> dict[str, float]:
    """Read manifest."""
    path = str(Path(directory) / MANIFEST_FILE)
    if not await asyncio.to_thread(
        lambda: Path(path).is_file(),
    ):
        return {}
    try:
        raw = await asyncio.to_thread(read_text, path)
        return cast("dict[str, float]", json.loads(raw))
    except (OSError, json.JSONDecodeError):
        return {}
async def write_manifest(
    directory: str, manifest: dict[str, float],
) -> None:
    """Write manifest."""
    path = str(Path(directory) / MANIFEST_FILE)
    tmp = f"{path}.tmp"
    try:
        await asyncio.to_thread(
            write_text, tmp, json.dumps(manifest),
        )
        await asyncio.to_thread(
            lambda: Path(tmp).replace(path),
        )
    except OSError as e:
        logger.warning(
            "[CloudSaveService] manifest write failed: %s", e,
        )
async def build_manifest(directory: str) -> dict[str, float]:
    """Build manifest."""
    if not await asyncio.to_thread(
        lambda: Path(directory).is_dir(),
    ):
        return {}
    return await asyncio.to_thread(walk_mtimes, directory)
```

OP-17d

fs_ops.py

Fichier : py_modules/unifideck/services/cloud_save/fs_ops.py | Lignes : 58 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
import shutil
from pathlib import Path
from .constants import MANIFEST_FILE
logger = logging.getLogger(__name__)
def walk_mtimes(root: str) -> dict[str, float]:
    """Walk mtimes."""
    result: dict[str, float] = {}
    root_path = Path(root)
    for dirpath, _, files in os.walk(root):
        for name in files:
            if name.startswith(".") or name == MANIFEST_FILE:
                continue
            full = Path(dirpath) / name
            rel = str(full.relative_to(root_path))
            try:
                result[rel] = full.stat().st_mtime
            except OSError:
                continue
    return result
def copy_tree(
    src: str, dst: str, skip_manifest: bool = False,
) -> None:
    """Copy tree."""
    src_path = Path(src)
    dst_path = Path(dst)
    if not src_path.is_dir():
        return
    dst_path.mkdir(parents=True, exist_ok=True)
    for dirpath, _dirs, files in os.walk(src):
        dirpath_p = Path(dirpath)
        rel = dirpath_p.relative_to(src_path)
        target_dir = (
            dst_path / rel if str(rel) != "." else dst_path
        )
        target_dir.mkdir(parents=True, exist_ok=True)
        for name in files:
            if skip_manifest and name == MANIFEST_FILE:
                continue
            if name.startswith("."):
                continue
            src_file = dirpath_p / name
            dst_file = target_dir / name
            try:
                shutil.copy2(src_file, dst_file)
            except OSError as e:
                logger.debug(
                    "[CloudSaveService] copy %s failed: %s",
                    src_file, e,
                )
def read_text(path: str) -> str:
    """Read text."""
    return Path(path).read_text(encoding="utf-8")

```

```
def write_text(path: str, content: str) -> None:
    """Write text."""
    Path(path).write_text(content, encoding="utf-8")
```

OP-17e

paths.py

Fichier : py_modules/unifideck/services/cloud_save/paths.py | Lignes : 8 | Depend de : (rien)

```
from __future__ import annotations
from pathlib import Path
def local_save_dir(local_root: str, store: str, game_id: str) -> str:
    """Local save dir."""
    return str(Path(local_root) / store / game_id)
def remote_save_dir(cloud_root: str, store: str, game_id: str) -> str:
    """Remote save dir."""
    return str(Path(cloud_root) / store / game_id)
```


OP-17f

constants.py

Fichier : py_modules/unifideck/services/cloud_save/constants.py | Lignes : 2 | Depend de : (rien)

```
from __future__ import annotations
MANIFEST_FILE = ".unifideck_sync.json"
```

OP-18

`__init__.py`

Fichier : `py_modules/unifideck/services/playtime/__init__.py` | Lignes : 3 | Depend de : (rien)

```
from __future__ import annotations
from .service import PlaytimeService
__all__ = ["PlaytimeService"]
```

OP-18a

service.py

Fichier : py_modules/unifideck/services/playtime/service.py | Lignes : 115 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import time
from typing import Any
from ...core.types import Events
from ...event_bus.event_bus import EventBus
from ...event_bus.event_bus_devex import auto_wire, subscribe
from .db import PlaytimeDB
logger = logging.getLogger(__name__)
_MIN_SESSION_SECONDS = 5
class PlaytimeService:
    """Playtime service."""
    def __init__(self, bus: EventBus, db_path: str) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._db_path = db_path
        self._active: dict[str, float] = {}
        self._db: PlaytimeDB | None = None
        n = auto_wire(self, self._bus)
        logger.info("[PlaytimeService] wired (%d subscriptions)", n)
    async def start(self) -> None:
        """Start."""
        self._db = await asyncio.to_thread(PlaytimeDB, self._db_path)
        logger.info("[PlaytimeService] database ready")
    async def stop(self) -> None:
        """Stop."""
        self._bus.off(Events.GAME_LAUNCHED, self._on_game_launched)
        self._bus.off(Events.GAME_STOPPED, self._on_game_stopped)
        for key in list(self._active.keys()):
            store, game_id = key.split(":", 1)
            await self._end_session(store, game_id)
        if self._db is not None:
            await asyncio.to_thread(self._db.close)
    @subscribe(Events.GAME_LAUNCHED)
    async def _on_game_launched(self, **kwargs) -> None:
        """On game launched."""
        store = kwargs.get("store")
        game_id = kwargs.get("game_id")
        if store and game_id:
            self._active[f"{store}:{game_id}"] = time.time()
    @subscribe(Events.GAME_STOPPED)
    async def _on_game_stopped(self, **kwargs) -> None:
        """On game stopped."""
        store = kwargs.get("store")
        game_id = kwargs.get("game_id")
        if store and game_id:
            await self._end_session(store, game_id)
    async def get_playtime(
        self, store: str, game_id: str,
    ) -> dict[str, Any]:
        """Get playtime."""
        assert self._db is not None
        row = await asyncio.to_thread(
            self._db.fetch_total, store, game_id,

```

```

    )
    if row is None:
        return {
            "total_seconds": 0,
            "session_count": 0,
            "last_played": None,
        }
    return {
        "total_seconds": row[0],
        "session_count": row[1],
        "last_played": row[2],
    }

async def get_all_playtimes(self) -> list[dict[str, Any]]:

    """Get all playtimes."""
    assert self._db is not None
    rows = await asyncio.to_thread(self._db.fetch_all_totals)
    return [
        {
            "store": r[0],
            "game_id": r[1],
            "total_seconds": r[2],
            "session_count": r[3],
            "last_played": r[4],
        }
        for r in rows
    ]

async def get_session_history(
    self, store: str, game_id: str, limit: int = 20,
) -> list[dict]:
    """Get session history."""
    assert self._db is not None
    rows = await asyncio.to_thread(
        self._db.fetch_sessions, store, game_id, limit,
    )
    return [
        {"start": r[0], "end": r[1], "duration": r[2]}
        for r in rows
    ]

async def _end_session(
    self, store: str, game_id: str,
) -> None:
    """End session."""
    key = f"{store}:{game_id}"
    start_ts = self._active.pop(key, None)
    if start_ts is None:
        return
    end_ts = time.time()
    duration = int(end_ts - start_ts)
    if duration < _MIN_SESSION_SECONDS:
        return
    assert self._db is not None
    await asyncio.to_thread(
        self._db.insert_session,
        store, game_id, start_ts, end_ts, duration,
    )
    await self._bus.emit(
        "playtime_updated",
        store=store, game_id=game_id,
        duration_seconds=duration,
    )

```

)

OP-18b

db.py

Fichier : py_modules/unifideck/services/playtime/db.py | Lignes : 92 | Depend de : (rien)

```

from __future__ import annotations
import sqlite3
from typing import Any, cast
class PlaytimeDB:
    """Playtime db."""
    def __init__(self, db_path: str) -> None:
        """Initialize the instance."""
        self._conn = sqlite3.connect(db_path)
        self._init_schema()
    def _init_schema(self) -> None:
        """Init schema."""
        self._conn.execute("""
            CREATE TABLE IF NOT EXISTS sessions (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                store TEXT NOT NULL,
                game_id TEXT NOT NULL,
                start_ts REAL NOT NULL,
                end_ts REAL NOT NULL,
                duration_s INTEGER NOT NULL
            )
        """)
        self._conn.execute("""
            CREATE TABLE IF NOT EXISTS totals (
                store TEXT NOT NULL,
                game_id TEXT NOT NULL,
                total_s INTEGER NOT NULL DEFAULT 0,
                session_count INTEGER NOT NULL DEFAULT 0,
                last_played REAL,
                PRIMARY KEY (store, game_id)
            )
        """)
        self._conn.execute("""
            CREATE INDEX IF NOT EXISTS idx_sessions_game
            ON sessions(store, game_id)
        """)
        self._conn.commit()
    def close(self) -> None:
        """Close."""
        self._conn.close()
    def insert_session(
        self,
        store: str,
        game_id: str,
        start_ts: float,
        end_ts: float,
        duration: int,
    ) -> None:
        """Insert session."""
        with self._conn:
            self._conn.execute(
                "INSERT INTO sessions(store, game_id, start_ts, "
                "end_ts, duration_s) VALUES (?, ?, ?, ?, ?)",
                (store, game_id, start_ts, end_ts, duration),
            )
            self._conn.execute(

```

```

        "INSERT INTO totals(store, game_id, total_s, "
        "session_count, last_played) "
        "VALUES (?, ?, ?, 1, ?) "
        "ON CONFLICT(store, game_id) DO UPDATE SET "
        "total_s = total_s + excluded.total_s, "
        "session_count = session_count + 1, "
        "last_played = excluded.last_played",
        (store, game_id, duration, end_ts),
    )

def fetch_total(
    self, store: str, game_id: str,
) -> tuple | None:
    """Fetch total."""
    cur = self._conn.execute(
        "SELECT total_s, session_count, last_played "
        "FROM totals WHERE store=? AND game_id=?",
        (store, game_id),
    )
    return cast("tuple[Any, ...] | None", cur.fetchone())
def fetch_all_totals(self) -> list[tuple]:
    """Fetch all totals."""
    cur = self._conn.execute(
        "SELECT store, game_id, total_s, session_count, "
        "last_played FROM totals ORDER BY last_played DESC",
    )
    return cur.fetchall()
def fetch_sessions(
    self, store: str, game_id: str, limit: int,
) -> list[tuple]:
    """Fetch sessions."""
    cur = self._conn.execute(
        "SELECT start_ts, end_ts, duration_s FROM sessions "
        "WHERE store=? AND game_id=? ORDER BY start_ts DESC "
        "LIMIT ?",
        (store, game_id, limit),
    )
    return cur.fetchall()

```

OP-19

`__init__.py`

Fichier : `py_modules/unifideck/services/security/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from __future__ import annotations
from .service import SecurityService
```


OP-19a

service.py

Fichier : py_modules/unifideck/services/security/service.py | Lignes : 134 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from ...core.types.events import Events
from ...event_bus.event_bus_devex import auto_wire
from ...security import DeviceFingerprint
from . import device_reset
from .audit_log import AuditLog
from .auth import AuthAuditMixin
from .bruteforce import BruteForceDetector
from .bus_emitter import emit_security_event
from .config import ConfigAuditMixin
from .config_readers import read_float, read_int, read_str
from .permissions import PermissionsMixin
from .tokens import TokenAuditMixin
if TYPE_CHECKING:
    from typing import Any
    from ...config import ConfigManager
    from ...event_bus.event_bus import EventBus
    from ...event_bus.event_replay import EventReplayBuffer
logger = logging.getLogger(__name__)
_DEFAULT_AUDIT_CAPACITY = 500
_DEFAULT_BRUTEFORCE_WINDOW_S = 60.0
_DEFAULT_BRUTEFORCE_WARNING = 5
_DEFAULT_BRUTEFORCE_ESCALATION = 20
_DEFAULT_FINGERPRINT_PATH = "~/config/unifideck/device_fingerprint.json"
class SecurityService(
    TokenAuditMixin,
    PermissionsMixin,
    AuthAuditMixin,
    ConfigAuditMixin,
):
    """Security service."""
    def __init__(
        self,
        bus: EventBus,
        config: ConfigManager | None = None,
        fingerprint: DeviceFingerprint | None = None,
        replay: EventReplayBuffer | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._config = config
        self._replay = replay
        capacity = read_int(
            config, "security.audit_log_capacity",
            _DEFAULT_AUDIT_CAPACITY,
        )
        window_s = read_float(
            config, "security.bruteforce_window_seconds",
            _DEFAULT_BRUTEFORCE_WINDOW_S,
        )
        warning = read_int(

```

```

        config, "security.bruteforce_warning_threshold",
        _DEFAULT_BRUTEFORCE_WARNING,
    )
    escalation = read_int(
        config, "security.bruteforce_escalation_threshold",
        _DEFAULT_BRUTEFORCE_ESCALATION,
    )
    self._audit = AuditLog(capacity=capacity)
    self._bf = BruteForceDetector(
        window_seconds=window_s,
        warning_threshold=warning,
        escalation_threshold=escalation,
        on_threshold_crossed=self._emit_bruteforce,
    )
    self.fingerprint = fingerprint or self._build_fingerprint()
    auto_wire(self, self._bus)
    logger.info(
        "[SecurityService] initialized (audit=%d, bf=%d/%d/%gs)",
        capacity, warning, escalation, window_s,
    )
)
async def start(self) -> None:
    """Start."""
    self._drain_config_validation_replay()
    await device_reset.check_device_fingerprint(self)
def _drain_config_validation_replay(self) -> None:
    """Drain config validation replay."""
    if self._replay is None:
        return
    try:
        missed = self._replay.snapshot(
            events=[Events.CONFIG_VALIDATION_FAILED],
        )
        for entry in missed:
            self._audit.record(
                "CONFIG_VALIDATION_FAILED",
                entry.get("kwargs", {}),
            )
        if missed:
            logger.info(
                "[SecurityService] replayed %d missed "
                "CONFIG_VALIDATION_FAILED event(s)",
                len(missed),
            )
    except Exception:
        logger.exception(
            "[SecurityService] replay drain failed (non-fatal)",
        )
def _build_fingerprint(self) -> DeviceFingerprint:
    """Build fingerprint."""
    path = read_str(
        self._config, "security.fingerprint_path",
        _DEFAULT_FINGERPRINT_PATH,
    )
    return DeviceFingerprint(path=path)
def _emit_bruteforce(
    self, *, level: str, recent_failures: int,
) -> None:
    """Emit bruteforce."""
    emit_security_event(
        self._bus, "SECURITY_BRUTEFORCE_SUSPECTED",
        level=level, recent_failures=recent_failures,
    )

```

```
)

def get_audit_log(
    self, limit: int | None = None,
) -> list[dict[str, Any]]:

    """Get audit log."""
    return self._audit.snapshot(limit=limit)
def get_counters(self) -> dict[str, int]:
    """Get counters."""
    return self._audit.counters()
def get_bruteforce_status(self) -> dict[str, Any]:
    """Get bruteforce status."""
    return self._bf.status()
def clear_audit_log(self) -> None:
    """Clear audit log."""
    self._audit.clear()
    logger.info("[SecurityService] audit log and counters cleared")
def reset_bruteforce_state(self) -> None:
    """Reset bruteforce state."""
    self._bf.reset()
    logger.info("[SecurityService] brute-force state reset")
```

OP-19b

audit_log.py

Fichier : py_modules/unifideck/services/security/audit_log.py | Lignes : 43 | Depend de : (rien)

```
from __future__ import annotations
import logging
import time
from collections import deque
from typing import Any
logger = logging.getLogger(__name__)
class AuditLog:
    """Audit log."""
    def __init__(self, capacity: int) -> None:
        """Initialize the instance."""
        self._entries: deque[dict[str, Any]] = deque(maxlen=capacity)
        self._counters: dict[str, int] = {}
    def record(self, event_name: str, payload: dict[str, Any]) -> None:
        """Record."""
        try:
            entry = {
                "event": event_name,
                "timestamp": time.time(),
                "payload": dict(payload),
            }
            self._entries.append(entry)
            self._counters[event_name] = (
                self._counters.get(event_name, 0) + 1
            )
        except Exception as e:
            logger.debug(
                "[AuditLog] record failed: %s", e,
            )
    def snapshot(
        self, limit: int | None = None,
    ) -> list[dict[str, Any]]:
        """Snapshot."""
        entries = list(reversed(self._entries))
        if limit is not None and limit > 0:
            entries = entries[:limit]
        return entries
    def counters(self) -> dict[str, int]:
        """Counters."""
        return dict(self._counters)
    def clear(self) -> None:
        """Clear."""
        self._entries.clear()
        self._counters.clear()
```

OP-19c

bruteforce.py

Fichier : py_modules/unifideck/services/security/bruteforce.py | Lignes : 62 | Depend de : (rien)

```

from __future__ import annotations
import logging
import time
from collections import deque
from collections.abc import Callable
from typing import Any
logger = logging.getLogger(__name__)
class BruteForceDetector:
    """Brute force detector."""
    def __init__(
        self,
        window_seconds: float,
        warning_threshold: int,
        escalation_threshold: int,
        on_threshold_crossed: Callable[..., None],
    ) -> None:
        """Initialize the instance."""
        self._window = window_seconds
        self._warning = warning_threshold
        self._escalation = escalation_threshold
        self._failures: deque[float] = deque(maxlen=escalation_threshold * 2)
        self._escalated = False
        self._on_crossed = on_threshold_crossed
    def check(self) -> None:
        """Check."""
        now = time.monotonic()
        self._failures.append(now)
        recent = sum(
            1 for ts in self._failures
            if now - ts <= self._window
        )
        if recent >= self._escalation and not self._escalated:
            self._escalated = True
            logger.error(
                "[BruteForceDetector] ESCALATION: %d failures "
                "in %.0fs", recent, self._window,
            )
            self._on_crossed(level="escalation", recent_failures=recent)
        elif recent >= self._warning:
            logger.warning(
                "[BruteForceDetector] warning: %d failures "
                "in %.0fs", recent, self._window,
            )
            self._on_crossed(level="warning", recent_failures=recent)
    def status(self) -> dict[str, Any]:
        """Status."""
        now = time.monotonic()
        recent = sum(
            1 for ts in self._failures
            if now - ts <= self._window
        )
        return {
            "recent_failures": recent,
            "window_seconds": self._window,
            "warning_threshold": self._warning,

```

```
        "escalation_threshold": self._escalation,  
        "escalated": self._escalated,  
    }  
    def reset(self) -> None:  
        """Reset."""  
        self._failures.clear()  
        self._escalated = False
```

OP-19d

config_readers.py

Fichier : py_modules/unifideck/services/security/config_readers.py | Lignes : 44 | Depend de : (rien)

```
from __future__ import annotations
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ...config import ConfigManager
def read_int(
    config: ConfigManager | None, key: str, default: int,
) -> int:
    """Read int."""
    if config is None or not hasattr(config, "get"):
        return default
    try:
        val = config.get(key, default)
        return int(val) if val else default
    except (TypeError, ValueError):
        return default
def read_float(
    config: ConfigManager | None, key: str, default: float,
) -> float:
    """Read float."""
    if config is None or not hasattr(config, "get"):
        return default
    try:
        val = config.get(key, default)
        return float(val) if val else default
    except (TypeError, ValueError):
        return default
def read_str(
    config: ConfigManager | None, key: str, default: str,
) -> str:
    """Read str."""
    if config is None or not hasattr(config, "get"):
        return default
    val = config.get(key, default)
    return str(val) if val else default
def read_list(
    config: ConfigManager | None, key: str,
) -> list[str]:
    """Read list."""
    if config is None or not hasattr(config, "get"):
        return []
    val = config.get(key, None)
    if not isinstance(val, list):
        return []
    return [str(x) for x in val if isinstance(x, str) and x]
```

OP-19e

bus_emitter.py

Fichier : py_modules/unifideck/services/security/bus_emitter.py | Lignes : 25 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
from ...core.types.events import Events
from ...event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
def emit_security_event(
    bus: EventBus, event_name: str, **kwargs: object,
) -> None:
    """Emit security event."""
    try:
        event = getattr(Events, event_name)
        try:
            loop = asyncio.get_running_loop()
        except RuntimeError:
            return
        loop.create_task(
            bus.emit(event, **kwargs),
            name=f"security-emit-{event_name}",
        )
    except Exception as e:
        logger.debug(
            "[bus_emitter] emit %s failed: %s",
            event_name, e,
        )
```


OP-19f

device_reset.py

Fichier : py_modules/unifideck/services/security/device_reset.py | Lignes : 80 | Depend de : (rien)

```
from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING
from ...security import DeviceIdentityError, FingerprintState
from .bus_emitter import emit_security_event
from .config_readers import read_list
if TYPE_CHECKING:
    from .service import SecurityService
logger = logging.getLogger(__name__)
async def check_device_fingerprint(
    service: SecurityService,
) -> FingerprintState:
    """Check device fingerprint."""
    try:
        state = service._fingerprint.verify_or_initialize()
    except DeviceIdentityError as e:
        logger.error(
            "[SecurityService] fingerprint check failed: %s", e,
        )
        return FingerprintState(
            machine_id_hash="",
            first_seen=0.0,
            last_verified=0.0,
            is_new=False,
            mismatch=False,
        )
    if state.is_new:
        emit_security_event(
            service._bus, "SECURITY_FINGERPRINT_INITIALIZED",
        )
        service._audit.record(
            "SECURITY_FINGERPRINT_INITIALIZED", {},
        )
    elif state.mismatch:
        await handle_device_reset(service, state)
    return state

async def handle_device_reset(
    service: SecurityService, state: FingerprintState,
) -> None:
    """Handle device reset."""
    logger.error(
        "[SecurityService] DEVICE RESET DETECTED – "
        "machine-id no longer matches stored fingerprint",
    )
    token_files = read_list(
        service._config, "security.token_files_to_wipe_on_reset",
    )
    wiped: list[str] = []
    for rel_path in token_files:
        full_path = Path(rel_path).expanduser()
        full = str(full_path)
        if not full_path.is_file():
            return
```

```
        continue
    try:
        full_path.unlink()
        wiped.append(full)
        logger.warning(
            "[SecurityService] wiped stale token: %s", full,
        )
    except OSError as e:
        logger.warning(
            "[SecurityService] failed to wipe %s: %s",
            full, e,
        )
    emit_security_event(
        service._bus, "SECURITY_DEVICE_RESET_DETECTED",
        wiped_files=wiped,
        wiped_count=len(wiped),
    )
    service._audit.record(
        "SECURITY_DEVICE_RESET_DETECTED",
        {"wiped_count": len(wiped)},
    )
    try:
        service._fingerprint.reinitialize()
    except DeviceIdentityError as e:
        logger.error(
            "[SecurityService] reinit failed: %s", e,
        )
```

OP-19g

tokens.py

Fichier : py_modules/unifideck/services/security/tokens.py | Lignes : 38 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
from ...core.types.events import Events
from ...event_bus.event_bus_devex import subscribe
if TYPE_CHECKING:
    from .audit_log import AuditLog
    from .bruteforce import BruteForceDetector
logger = logging.getLogger(__name__)
class TokenAuditMixin:
    """Token audit mixin."""
    _audit: AuditLog
    _bf: BruteForceDetector
    @subscribe(Events.SECURITY_TOKEN_ENCRYPTED)
    async def _on_token_encrypted(self, **kwargs: Any) -> None:
        """On token encrypted."""
        self._audit.record("SECURITY_TOKEN_ENCRYPTED", kwargs)
    @subscribe(Events.SECURITY_TOKEN_DECRYPTED)
    async def _on_token_decrypted(self, **kwargs: Any) -> None:
        """On token decrypted."""
        self._audit.record("SECURITY_TOKEN_DECRYPTED", kwargs)
    @subscribe(Events.SECURITY_DECRYPT_FAILED)
    async def _on_decrypt_failed(self, **kwargs: Any) -> None:
        """On decrypt failed."""
        self._audit.record("SECURITY_DECRYPT_FAILED", kwargs)
        reason = kwargs.get("reason", "unknown")
        logger.warning(
            "[SecurityService] decrypt failure: %s", reason,
        )
        self._bf.check()
    @subscribe(Events.SECURITY_TOKEN_FILE_MIGRATED)
    async def _on_token_file_migrated(self, **kwargs: Any) -> None:
        """On token file migrated."""
        self._audit.record("SECURITY_TOKEN_FILE_MIGRATED", kwargs)
    @subscribe(Events.SECURITY_LEGACY_PLAINTEXT_DETECTED)
    async def _on_legacy_plaintext(self, **kwargs: Any) -> None:
        """On legacy plaintext."""
        self._audit.record("SECURITY_LEGACY_PLAINTEXT_DETECTED", kwargs)
```

OP-19h

permissions.py

Fichier : py_modules/unifideck/services/security/permissions.py | Lignes : 45 | Depend de : (rien)

```
from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING, Any
from ...core.types.events import Events
from ...event_bus.event_bus_devex import subscribe
from .bus_emitter import emit_security_event
if TYPE_CHECKING:
    from ...event_bus.event_bus import EventBus
    from .audit_log import AuditLog
logger = logging.getLogger(__name__)
class PermissionsMixin:
    """Permissions mixin."""
    _audit: AuditLog
    _bus: EventBus
    @subscribe(Events.SECURITY_PERMISSIONS_CHECK)
    async def _on_permissions_check(self, **kwargs: Any) -> None:
        """On permissions check."""
        self._audit.record("SECURITY_PERMISSIONS_CHECK", kwargs)
        path = kwargs.get("path")
        mode = kwargs.get("mode")
        if not path or mode is None:
            return
        if mode == 0o600:
            return
        try:
            Path(path).chmod(0o600)
        except OSError as e:
            logger.warning(
                "[SecurityService] chmod 0o600 failed on %s: %s",
                path, e,
            )
            return
        logger.warning(
            "[SecurityService] repaired permissions on %s "
            "(was %o, now 0o600)", path, mode,
        )
        emit_security_event(
            self._bus, "SECURITY_PERMISSIONS_REPAIRED",
            path=path, previous_mode=mode,
        )
        self._audit.record(
            "SECURITY_PERMISSIONS_REPAIRED",
            {"path": path, "previous_mode": mode},
        )
```

OP-19i

auth.py**Fichier :** py_modules/unifideck/services/security/auth.py | **Lignes :** 41 | **Depend de :** (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
from ...core.types.events import Events
from ...event_bus.event_bus_devex import subscribe
if TYPE_CHECKING:
    from .audit_log import AuditLog
logger = logging.getLogger(__name__)
class AuthAuditMixin:
    """Auth audit mixin."""
    _audit: AuditLog
    @subscribe(Events.SECURITY_AUTH_FLOW_STARTED)
    async def _on_auth_started(self, **kwargs: Any) -> None:
        """On auth started."""
        self._audit.record("SECURITY_AUTH_FLOW_STARTED", kwargs)
    @subscribe(Events.SECURITY_AUTH_FLOW_COMPLETED)
    async def _on_auth_completed(self, **kwargs: Any) -> None:
        """On auth completed."""
        self._audit.record("SECURITY_AUTH_FLOW_COMPLETED", kwargs)
    @subscribe(Events.SECURITY_AUTH_FLOW_FAILED)
    async def _on_auth_failed(self, **kwargs: Any) -> None:
        """On auth failed."""
        self._audit.record("SECURITY_AUTH_FLOW_FAILED", kwargs)
        reason = kwargs.get("reason", "unknown")
        logger.warning(
            "[SecurityService] auth flow failed: %s", reason,
        )
    @subscribe(Events.SECURITY_EXTERNAL_AUTH_CHECK_FAILED)
    async def _on_external_auth_check_failed(
        self, **kwargs: Any,
    ) -> None:
        """On external auth check failed."""
        self._audit.record(
            "SECURITY_EXTERNAL_AUTH_CHECK_FAILED", kwargs,
        )
        store = kwargs.get("store", "unknown")
        reason = kwargs.get("reason", "unknown")
        logger.warning(
            "[SecurityService] external auth check failed: "
            "%s / %s", store, reason,
        )
    )
```

OP-19j

config.py**Fichier :** py_modules/unifideck/services/security/config.py | Lignes : 36 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
from ...core.types.events import Events
from ...event_bus.event_bus_devex import subscribe
if TYPE_CHECKING:
    from .audit_log import AuditLog
logger = logging.getLogger(__name__)
class ConfigAuditMixin:
    """Config audit mixin."""
    _audit: AuditLog
    @subscribe(Events.CONFIG_VALIDATION_COMPLETED)
    async def _on_config_validation_completed(
        self, **kwargs: Any,
    ) -> None:
        """On config validation completed."""
        self._audit.record("CONFIG_VALIDATION_COMPLETED", kwargs)
        logger.info(
            "[SecurityService] config validation completed "
            "(defaults_validated=%s, user_overrides=%s)",
            kwargs.get("defaults_validated", False),
            kwargs.get("user_overrides_present", False),
        )
    @subscribe(Events.CONFIG_VALIDATION_FAILED)
    async def _on_config_validation_failed(
        self, **kwargs: Any,
    ) -> None:
        """On config validation failed."""
        self._audit.record("CONFIG_VALIDATION_FAILED", kwargs)
        logger.warning(
            "[SecurityService] config validation failed: "
            "%d error(s), first at %s (source=%s)",
            kwargs.get("error_count", 0),
            kwargs.get("first_error_path", "<unknown>"),
            kwargs.get("first_error_source", "<unknown>"),
        )
```

OP-20

`__init__.py`

Fichier : `py_modules/unifideck/services/launcher/__init__.py` | Lignes : 3 | Depend de : (rien)

```
from __future__ import annotations
from .builder import build_standalone
from .service import LauncherService
```

OP-20a

service.py

Fichier : py_modules/unifideck/services/launcher/service.py | Lignes : 252 | Depend de : (rien)

```

"""services.launcher.service – LauncherService DI facade.
LauncherService is the single entry point used by main.py and the
dispatcher CLI. It holds references to the existing backend
services (ShortcutService for games_map access, ProtonService for
compat-tool resolution, CloudSaveService for pre/post sync, and
EdgeBrowser for xCloud + OAuth kiosk modes) and orchestrates a
single launch end-to-end.
**No duplication of existing code.** Every non-trivial piece of
logic is delegated to a backend service that already implements
it. The only code that lives in this subpackage is:
- The dispatch logic (which store handler to call for a given
  LaunchContext) – in ``launch`` below
- The signal-handler wiring (signals.py)
- The launch stage event sequence (launched → stopped)
- Wrapping subprocess.run calls for store-specific CLI tools
  (legendary, gogdl, nile – these aren't a service)
Module layout
-----
This class used to live as a single 821 LOC module. During the
2026-04-18 volumetry refactor it was split into five sibling
modules that this class delegates to:
- ``circuit_breaker`` : pre-launch failure-protection
- ``error_toasts`` : post-failure user reporting
- ``orchestrator`` : per-platform launch entry points
- ``helpers`` : technical primitives for launch flows
- ``builder`` : standalone-CLI factory (separate file)
Dependencies (all injected – never instantiated here):
- bus: EventBus for emit_game_launched / _stopped
- shortcut_svc: ShortcutService for games_map read/write
- proton_svc: ProtonService for compat tool selection
- cloud_svc: CloudSaveService for sync_down / sync_up
- edge_browser: EdgeBrowser for auth flows and xCloud kiosk mode
The service_bootstrap already wires up the four services above.
LauncherService joins the bootstrap order after all of them so
its dependencies are guaranteed to be ready at construction time.
"""

from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
from ...core.types import Result
from ...launcher.rpc import (
    emit_game_launched,
    emit_game_stopped,
    emit_stage,
)
from ...launcher.signals import (
    GameProcessRegistry,
    SignalState,
    install_signal_handlers,
)
from ...launcher.types.context import LaunchContext, RuntimeState
from ...launcher.types.errors import LauncherError
from . import circuit_breaker, error_toasts, helpers, orchestrator
if TYPE_CHECKING:
    from ...auth.edge_browser import EdgeBrowser

```



```

from ...event_bus import EventBus
from ...launcher.proton.infrastructure.core import ProtonLaunchPlan
from ..cloud_save import CloudSaveService
from ..proton_service import ProtonService
from ..shortcut import ShortcutService
logger = logging.getLogger(__name__)

class LauncherService:
    """Facade that orchestrates a single game launch.
    Injected by ServiceBootstrap with references to every backend
    service it needs. Does not instantiate anything itself. Does
    not duplicate logic that lives elsewhere – the bash modules
    that used to reimplement Proton selection, cloud save sync,
    and Edge kiosk launch are replaced by delegation to the
    existing services.
    """
    def __init__( # noqa: D107 – class docstring documents the constructor's contract
        self,
        bus: EventBus,
        shortcut_svc: ShortcutService,
        proton_svc: ProtonService,
        cloud_svc: CloudSaveService,
        edge_browser: EdgeBrowser,
        config: Any | None = None,
        launch_history: Any | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._shortcut_svc = shortcut_svc
        self._proton_svc = proton_svc
        self._cloud_svc = cloud_svc
        self._edge_browser = edge_browser
        self._config = config
        self._launch_history = launch_history
        self._signal_state = SignalState()
        self._registry = GameProcessRegistry(self._signal_state)
    async def start(self) -> None:
        """Install signal handlers. Called by ServiceBootstrap.
        Idempotent: re-installing handlers during hot-reload is
        safe. The state is reused so any tracked PIDs from the
        previous instance are still honoured.
        """
        install_signal_handlers(self._registry)
        logger.info("[LauncherService] signal handlers installed")
    async def stop(self) -> None:
        """Bootstrap teardown hook. No-op for now.
        Signal handlers are process-global and don't need to be
        removed – they'll be replaced if another instance starts.
        """

# =====
# Public API
# =====

    async def launch(self, ctx: LaunchContext) -> Result:
        """Launch a game described by the immutable LaunchContext.
        Dispatches on the context flags to the right handler:
        Windows games via Proton, xCloud via the streaming helper,
        native Linux, or OAuth auth actions.
        Control flow:
        1. Check circuit breaker – short-circuit if open
        2. Emit stage toast "launching ${game_title}"

```

```

        3. If ctx.is_launch_action is False: delegate to auth
        4. If ctx.is_xcloud: delegate to xcloud handler
        5. If ctx.is_windows_game: delegate to Windows launcher
        6. Otherwise: delegate to native Linux launcher
        7. On LauncherError: record failure + emit toast
    """
    logger.info(
        "[LauncherService] launch request: %s", ctx.to_log_dict,
    )
    state = RuntimeState()
    refusal = await self._check_circuit_breaker(ctx)
    if refusal is not None:
        return refusal
    # Record subprocess start time for fast-boot detection in
    # the finally block. monotonic to be immune to NTP jumps.
    import time as _time
    self._launch_started_at = _time.monotonic()
    try:
        await emit_stage(
            self._bus,
            il8n_key="toasts.launcher.launchingGame",
            game_title=ctx.game_key,
            priority="low",
        )
        if not ctx.is_launch_action:
            # OAuth shortcut path – delegate to auth.py
            from ...launcher.flows.auth import handle_store_auth
            return await handle_store_auth(ctx, self._edge_browser)
        if ctx.is_xcloud:
            return await self._launch_xcloud(ctx)
        if ctx.is_windows_game:
            return await self._launch_windows(ctx, state)
        # Native Linux game – delegate to native.py
        return await self._launch_native(ctx, state)
    except LauncherError as err:
        return await self._handle_launcher_error(ctx, err)
    finally:
        # No ``return`` in this ``finally`` (B012): real Result
        # is emitted from the try/except branches above. Circuit
        # breaker post-flight classification is handled in
        # LaunchHistoryService._on_game_stopped via @subscribe.
        logger.info(
            "[LauncherService] launch finished: state=%s signal=%s",
            state.to_log_dict,
            self._signal_state.terminated_by_signal,
        )

async def _launch_xcloud(self, ctx: LaunchContext) -> Result:
    """xCloud streaming path.
    Delegates to the ``launch_xcloud`` helper. We fire
    ``GAME_LAUNCHED`` before streaming so Playtime tracking
    starts immediately, and ``GAME_STOPPED`` in the finally
    so the counterpart fires even if the browser throws.
    """ # noqa: D403 – "xCloud" is the product name
    from ...launcher.flows.xcloud import launch_xcloud
    await emit_game_launched(
        self._bus, store=ctx.store, game_id=ctx.game_id,
    )
    try:
        return await launch_xcloud(ctx, self._edge_browser)
    finally:

```

```

        # No ``return`` here (B012) – real Result bubbled.
        await emit_game_stopped(
            self._bus,
            store=ctx.store,
            game_id=ctx.game_id,
            exit_code=0,
            elapsed_seconds=0.0,
            terminated_by_signal=False,
        )
    # =====
    # Thin delegators – extracted to sibling modules for cohesion
    # =====
    async def _get_launch_id_or_none(self) -> str | None:
        """Get launch ID or none."""
        return await circuit_breaker.get_launch_id_or_none(self)
    async def _emit_circuit_open_toast(
        self, ctx: LaunchContext, failure_count: int,
    ) -> None:
        """Emit circuit open toast."""
        await circuit_breaker.emit_circuit_open_toast(
            self, ctx, failure_count,
        )
    async def _check_circuit_breaker(
        self, ctx: LaunchContext,
    ) -> Result | None:
        """Check circuit breaker."""
        return await circuit_breaker.check_circuit_breaker(self, ctx)
    async def _emit_launcher_error_toast(
        self, ctx: LaunchContext, err_code: str,
    ) -> None:
        """Emit launcher error toast."""
        await error_toasts.emit_launcher_error_toast(
            self, ctx, err_code,
        )
    async def _handle_launcher_error(
        self, ctx: LaunchContext, err: LauncherError,
    ) -> Result:
        """Handle launcher error."""
        return await error_toasts.handle_launcher_error(self, ctx, err)
    async def _launch_windows(
        self, ctx: LaunchContext, state: RuntimeState,
    ) -> Result:
        """Launch windows."""
        return await orchestrator.launch_windows(self, ctx, state)
    async def _launch_native(
        self, ctx: LaunchContext, state: RuntimeState,
    ) -> Result:
        """Launch native."""
        return await orchestrator.launch_native(self, ctx, state)
    async def _prepare_windows_plan(
        self, ctx: LaunchContext, state: RuntimeState,
    ) -> tuple[ProtonLaunchPlan, object]:
        """Prepare windows plan."""
        return await helpers.prepare_windows_plan(self, ctx, state)
    async def _cloud_sync_phase(
        self, ctx: LaunchContext, direction: str,
    ) -> None:
        """Cloud sync phase."""
        await helpers.cloud_sync_phase(self, ctx, direction)
    async def _run_game_subprocess(
        self,

```

```
        plan: ProtonLaunchPlan,
        ctx: LaunchContext,
        state: RuntimeState,
    ) -> int:
        """Run game subprocess."""
        return await helpers.run_game_subprocess(self, plan, ctx, state)

    async def _sync_saves_and_track_size(
        self, ctx: LaunchContext, phase: str,
    ) -> None:
        """Sync saves and track size."""
        await helpers.sync_saves_and_track_size(self, ctx, phase)
    def _resolve_exit_code(self, state: RuntimeState) -> int:
        """Resolve exit code."""
        return helpers.resolve_exit_code(self, state)
    def _elapsed_since_launch(self) -> float:
        """Elapsed since launch."""
        return helpers.elapsed_since_launch(self)
```

OP-20b

circuit_breaker.py

Fichier : py_modules/unifideck/services/launcher/circuit_breaker.py | Lignes : 74 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from ...core.types import Result
from ...core.types.events import Events
if TYPE_CHECKING:
    from ...launcher.types.context import LaunchContext
    from .service import LauncherService
logger = logging.getLogger(__name__)
async def get_launch_id_or_none(svc: LauncherService) -> str | None:
    """Get launch ID or none."""
    try:
        from ...launcher.diagnostics.correlation import get_launch_id
        lid = get_launch_id()
        return None if lid == "-" else lid
    except Exception:
        return None
async def emit_circuit_open_toast(
    svc: LauncherService, ctx: LaunchContext, failure_count: int,
) -> None:
    """Emit circuit open toast."""
    lid = await get_launch_id_or_none(svc)
    toast_payload: dict[str, object] = {
        "severity": "error",
        "i18n_key": "toasts.launcher.errorCircuitBreakerOpen",
        "i18n_params": {
            "game_key": ctx.game_key,
            "count": failure_count,
        },
        "duration_ms": 10000,
        "store": ctx.store,
        "game_id": ctx.game_id,
    }
    if lid:
        toast_payload["action"] = {
            "i18n_label_key": "toasts.actions.showLogs",
            "target_url": f"unifideck://show-logs/{lid}",
        }
    try:
        await svc._bus.emit(
            Events.LAUNCHER_STAGE, **toast_payload,
        )
    except Exception:
        logger.exception(
            "[LauncherService] circuit_open toast emit failed",
        )

async def check_circuit_breaker(
    svc: LauncherService, ctx: LaunchContext,
) -> Result | None:
    """Check circuit breaker."""
    if (
        not ctx.is_launch_action
        or svc._launch_history is None
    )

```

```
        or not svc._launch_history.is_circuit_open(ctx.game_key)
    ):
        return None
    recent = svc._launch_history.get_recent_failures(ctx.game_key)
    logger.warning(
        "[LauncherService] circuit OPEN for %s "
        "(%d recent failures), refusing launch",
        ctx.game_key, len(recent),
    )
    await emit_circuit_open_toast(svc, ctx, len(recent))
    window_s = int(svc._launch_history.window_seconds())
    return Result(
        success=False,
        error=(
            f"Launch refused: {len(recent)} recent failures "
            f"in the last {window_s}s. "
            f"Use 'Force launch' to bypass."
        ),
        error_code="circuit_open",
        store=ctx.store,
    )
```

OP-20c

error_toasts.py

Fichier : py_modules/unifideck/services/launcher/error_toasts.py | Lignes : 69 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from ...core.types import Result
from ...core.types.events import Events
from .circuit_breaker import get_launch_id_or_none
if TYPE_CHECKING:
    from ...launcher.types.context import LaunchContext
    from ...launcher.types.errors import LauncherError
    from .service import LauncherService
logger = logging.getLogger(__name__)
async def emit_launcher_error_toast(
    svc: LauncherService, ctx: LaunchContext, err_code: str,
) -> None:
    """Emit launcher error toast."""
    lid = await get_launch_id_or_none(svc)
    toast: dict[str, object] = {
        "severity": "error",
        "i18n_key": "toasts.launcher.launcherError",
        "i18n_params": {
            "game_key": ctx.game_key,
            "error_code": err_code,
            "error_i18n_key": f"launcher.error.{err_code}",
        },
        "duration_ms": 10000,
        "store": ctx.store,
        "game_id": ctx.game_id,
    }
    if lid:
        toast["action"] = {
            "i18n_label_key": "toasts.actions.showLogs",
            "target_url": f"unifideck://show-logs/{lid}",
        }
    try:
        await svc._bus.emit(Events.LAUNCHER_STAGE, **toast)
    except Exception:
        logger.exception(
            "[LauncherService] launcher_error toast emit failed",
        )

async def handle_launcher_error(
    svc: LauncherService, ctx: LaunchContext, err: LauncherError,
) -> Result:
    """Handle launcher error."""
    logger.error(
        "[LauncherService] launch failed: %s", err.to_log_dict,
    )
    err_code = getattr(err, "code", None) or type(err).__name__
    is_cancel = (
        "cancel" in err_code.lower()
        or "cancel" in type(err).__name__.lower()
    )
    if (
        ctx.is_launch_action

```

```
        and svc._launch_history is not None
        and not is_cancel
    ):
        from ..launch_history import (
            FAILURE_KIND_LAUNCHER_ERROR,
        )
        svc._launch_history.record_failure(
            ctx.game_key, FAILURE_KIND_LAUNCHER_ERROR,
        )
        await emit_launcher_error_toast(svc, ctx, err_code)
    return Result(
        success=False,
        error=str(err),
        error_code=err_code,
        store=ctx.store,
    )
```


OP-20d

orchestrator.py

Fichier : py_modules/unifideck/services/launcher/orchestrator.py | Lignes : 73 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from ...core.types import Result
from ...launcher.rpc import emit_game_launched, emit_game_stopped
from ...launcher.types.errors import LauncherError
from .helpers import (
    cloud_sync_phase,
    elapsed_since_launch,
    prepare_windows_plan,
    resolve_exit_code,
    run_game_subprocess,
    sync_saves_and_track_size,
)
if TYPE_CHECKING:
    from ...launcher.types.context import LaunchContext, RuntimeState
    from .service import LauncherService
logger = logging.getLogger(__name__)
async def launch_windows(
    svc: LauncherService,
    ctx: LaunchContext,
    state: RuntimeState,
) -> Result:
    """Launch windows."""
    plan, _parsed = await prepare_windows_plan(svc, ctx, state)
    await cloud_sync_phase(svc, ctx, direction="down")
    await emit_game_launched(
        svc._bus, store=ctx.store, game_id=ctx.game_id,
    )
    try:
        rc = await run_game_subprocess(svc, plan, ctx, state)
        await cloud_sync_phase(svc, ctx, direction="up")
        return Result(
            success=(rc == 0),
            error=None if rc == 0 else f"game exited with code {rc}",
            error_code=None if rc == 0 else f"exit_{rc}",
            store=ctx.store,
        )
    except LauncherError:
        raise

async def launch_native(
    svc: LauncherService,
    ctx: LaunchContext,
    state: RuntimeState,
) -> Result:
    """Launch native."""
    from ...launcher.flows.native import native_launch
    from ...launcher.types.options import parse_launch_options
    parsed = parse_launch_options(ctx.raw_options)
    state.wrappers = list(parsed.wrappers)
    state.game_args = list(parsed.game_args)
    state.lsfg_requested = parsed.lsfg_requested
    await sync_saves_and_track_size(svc, ctx, phase="sync_down")

```

```
await emit_game_launched(
    svc._bus, store=ctx.store, game_id=ctx.game_id,
)
try:
    result = await native_launch(ctx, state)
finally:
    exit_code = resolve_exit_code(svc, state)
    elapsed = elapsed_since_launch(svc)
    await emit_game_stopped(
        svc._bus,
        store=ctx.store,
        game_id=ctx.game_id,
        exit_code=exit_code,
        elapsed_seconds=elapsed,
        terminated_by_signal=(
            svc._signal_state.terminated_by_signal
        ),
    )
await sync_saves_and_track_size(svc, ctx, phase="sync_up")
return result
```

OP-20e

helpers.py

Fichier : py_modules/unifideck/services/launcher/helpers.py | Lignes : 212 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
from collections.abc import Callable
from typing import TYPE_CHECKING, cast
from ...launcher.rpc import emit_game_stopped
from ...launcher.types.context import RuntimeState
from ...launcher.types.errors import LauncherError
if TYPE_CHECKING:
    from ...launcher.proton.infrastructure.core import ProtonLaunchPlan
    from ...launcher.types.context import LaunchContext
    from .service import LauncherService
logger = logging.getLogger(__name__)

async def prepare_windows_plan(
    svc: LauncherService,
    ctx: LaunchContext,
    state: RuntimeState,
) -> tuple[ProtonLaunchPlan, object]:

    """Prepare windows plan."""
    from ...launcher.diagnostics.telemetry import PhaseTimer
    from ...launcher.proton.infrastructure.core import proton_prepare
    from ...launcher.proton.infrastructure.selector import (
        find_python_3_10_plus,
        select_proton_version,
    )
    from ...launcher.proton.infrastructure.umu_runtime import ensure_umu_runtime_ready
    from ...launcher.types.options import apply_lsfg_env, parse_launch_options
    parsed = parse_launch_options(ctx.raw_options)
    state.wrappers = list(parsed.wrappers)
    state.game_args = list(parsed.game_args)
    state.lsfg_requested = parsed.lsfg_requested
    async with PhaseTimer(
        svc._bus, "resolve_runtime",
        extra={"store": ctx.store},
    ):
        python_bin = find_python_3_10_plus()
        steam_app_id = os.environ.get(
            "UNIFIDECK_SHORTCUT_APPID",
        )
        proton_path, tool_id = select_proton_version(
            steam_app_id=steam_app_id,
        )
    async with PhaseTimer(
        svc._bus, "umu_runtime_ready",
        extra={"store": ctx.store},
    ):
        ensure_umu_runtime_ready()
    lsfg_overlay = apply_lsfg_env(parsed)
    async with PhaseTimer(
        svc._bus, "proton_prepare",
        extra={"store": ctx.store},
    ):
        plan = proton_prepare(

```

```

        ctx,
        state,
        python_bin=python_bin,
        proton_path=proton_path,
        proton_tool_id=tool_id,
        on_process_start=cast(
            "Callable[[object], None]",
            svc._registry.track,
        ),
    )
    plan.env.update(lsfg_overlay)
    plan.env.update(parsed.env_overrides)
    return plan, parsed

async def cloud_sync_phase(
    svc: LauncherService,
    ctx: LaunchContext,
    direction: str,
) -> None:

    """Cloud sync phase."""
    from pathlib import (
        Path as _P,
    )
    from ...launcher.cloud.disk_space import assert_enough_space
    from ...launcher.cloud.save_size_cache import (
        measure_directory_size,
        record_observed_size,
    )
    from ...launcher.diagnostics.telemetry import PhaseTimer
    async with PhaseTimer(
        svc._bus, f"cloud_sync_{direction}",
        extra={"store": ctx.store},
    ):
        try:
            assert_enough_space(
                _P(ctx.plugin_dir).expanduser(), svc._config,
                store=ctx.store, game_id=ctx.game_id,
            )
            if direction == "down":
                await svc._cloud_svc.sync_down(
                    ctx.store, ctx.game_id,
                )
            else:
                await svc._cloud_svc.sync_up(
                    ctx.store, ctx.game_id,
                )
            _local_dir = svc._cloud_svc.get_local_save_dir(
                ctx.store, ctx.game_id,
            )
            _size = measure_directory_size(_local_dir)
            if _size > 0:
                record_observed_size(
                    svc._config, ctx.store, ctx.game_id, _size,
                )
        except Exception as err:
            from ...launcher.cloud.cloud_failure import (
                handle_cloud_sync_failure,
            )
            await handle_cloud_sync_failure(
                svc._bus, svc._config,

```

```

        phase=f"sync_{direction}",
        store=ctx.store, game_id=ctx.game_id,
        error=err,
    )

async def run_game_subprocess(
    svc: LauncherService,
    plan: ProtonLaunchPlan,
    ctx: LaunchContext,
    state: RuntimeState,
) -> int:

    """Run game subprocess."""
    from ...launcher import proton as proton_pkg
    from ...launcher.diagnostics.telemetry import PhaseTimer
    async with PhaseTimer(
        svc._bus, "game_run",
        extra={"store": ctx.store},
    ):
        try:
            rc = await proton_pkg.dispatch(plan)
        except LauncherError:
            raise
        finally:
            exit_code = state.game_exit_code
            if exit_code is None:
                exit_code = 1
            if svc._signal_state.terminated_by_signal:
                exit_code = 143
                state.terminated_by_signal = True
            import time as _t
            _elapsed = (
                _t.monotonic() - svc._launch_started_at
                if hasattr(svc, "_launch_started_at")
                else 0.0
            )
            await emit_game_stopped(
                svc._bus,
                store=ctx.store,
                game_id=ctx.game_id,
                exit_code=exit_code,
                elapsed_seconds=_elapsed,
                terminated_by_signal=(
                    svc._signal_state.terminated_by_signal
                ),
            )
        return rc

async def sync_saves_and_track_size(
    svc: LauncherService,
    ctx: LaunchContext,
    phase: str,
) -> None:
    """Sync saves and track size."""
    from pathlib import (
        Path as _P,
    )
    from ...launcher.cloud.cloud_failure import handle_cloud_sync_failure
    from ...launcher.cloud.disk_space import assert_enough_space
    from ...launcher.cloud.save_size_cache import (
        measure_directory_size,
        record_observed_size,

```

```

)
try:
    assert_enough_space(
        _P(ctx.plugin_dir).expanduser(), svc._config,
        store=ctx.store, game_id=ctx.game_id,
    )
    if phase == "sync_down":
        await svc._cloud_svc.sync_down(ctx.store, ctx.game_id)
    else:
        await svc._cloud_svc.sync_up(ctx.store, ctx.game_id)
    local_dir = svc._cloud_svc.get_local_save_dir(
        ctx.store, ctx.game_id,
    )
    size = measure_directory_size(local_dir)
    if size > 0:
        record_observed_size(
            svc._config, ctx.store, ctx.game_id, size,
        )
except Exception as err:
    await handle_cloud_sync_failure(
        svc._bus, svc._config,
        phase=phase,
        store=ctx.store, game_id=ctx.game_id,
        error=err,
    )

def resolve_exit_code(
    svc: LauncherService, state: RuntimeState,
) -> int:
    """Resolve exit code."""
    if state.game_exit_code is not None:
        return state.game_exit_code
    if svc._signal_state.terminated_by_signal:
        state.terminated_by_signal = True
        return 143
    return 1

def elapsed_since_launch(svc: LauncherService) -> float:
    """Elapsed since launch."""
    if not hasattr(svc, "_launch_started_at"):
        return 0.0
    import time as _t
    return _t.monotonic() - svc._launch_started_at

```

OP-20f

builder.py

Fichier : py_modules/unifideck/services/launcher/builder.py | Lignes : 52 | Depend de : (rien)

```
from __future__ import annotations
from .service import LauncherService
def _pick_first_shortcuts_vdf(userdata_root):
    """Pick first shortcuts VDF."""
    from pathlib import Path as _Path
    root = _Path(userdata_root)
    if not root.is_dir():
        return None
    for user_dir in root.iterdir():
        candidate = user_dir / "config" / "shortcuts.vdf"
        if candidate.is_file():
            return candidate
    return None
def build_standalone() -> LauncherService:
    """Build standalone."""
    from pathlib import Path as _Path
    from ...auth.edge_browser import EdgeBrowser
    from ...event_bus import EventBus
    from ..cloud_save import CloudSaveService
    from ..proton_service import ProtonService
    from ..shortcut import ShortcutService
    bus = EventBus()
    data_dir = _Path("~/local/share/unifideck").expanduser()
    data_dir.mkdir(parents=True, exist_ok=True)
    userdata_root = _Path("~/steam/root/userdata").expanduser()
    shortcuts_file = _pick_first_shortcuts_vdf(userdata_root)
    games_map_path = data_dir / "games.map"
    shortcut_svc = ShortcutService(
        bus=bus,
        shortcuts_path=str(shortcuts_file) if shortcuts_file else "",
        games_map_path=str(games_map_path),
    )
    proton_svc = ProtonService(
        bus=bus,
        config_vdf_path=str(userdata_root / "config" / "config.vdf"),
    )
    cloud_svc = CloudSaveService(
        bus=bus,
        local_save_root=str(data_dir / "saves"),
        cloud_root=None,
    )
    edge_browser = EdgeBrowser(
        cdp_port=9222,
        locale_fn=lambda: "en-US",
    )
    return LauncherService(
        bus=bus,
        shortcut_svc=shortcut_svc,
        proton_svc=proton_svc,
        cloud_svc=cloud_svc,
        edge_browser=edge_browser,
    )
```

OP-21

`__init__.py`

Fichier : `py_modules/unifideck/services/launch_history/__init__.py` | Lignes : 8 | Depend de : (rien)

```
from __future__ import annotations
from .constants import FAILURE_KIND_FAST_BOOT, FAILURE_KIND_LAUNCHER_ERROR
from .service import LaunchHistoryService
__all__ = [
    "FAILURE_KIND_FAST_BOOT",
    "FAILURE_KIND_LAUNCHER_ERROR",
    "LaunchHistoryService",
]
```


OP-21a

service.py

Fichier : py_modules/unifideck/services/launch_history/service.py | Lignes : 98 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from pathlib import Path
from typing import Any
from ...core.types.events import Events
from ...event_bus.event_bus_devex import subscribe
from .bypass import _BypassMixin
from .config_readers import (
    DEFAULT_FAST_BOOT_SECONDS,
    DEFAULT_THRESHOLD,
    DEFAULT_WINDOW_SECONDS,
    read_fast_boot_seconds,
    read_threshold,
    read_window_seconds,
)
from .constants import FAILURE_KIND_FAST_BOOT
from .failures import _FailuresMixin
logger = logging.getLogger(__name__)
class LaunchHistoryService(_FailuresMixin, _BypassMixin):
    """Launch history service."""
    DEFAULT_THRESHOLD = DEFAULT_THRESHOLD
    DEFAULT_WINDOW_SECONDS = DEFAULT_WINDOW_SECONDS
    DEFAULT_FAST_BOOT_SECONDS = DEFAULT_FAST_BOOT_SECONDS
    def __init__(
        self,
        config: Any | None = None,
        storage_path: Path | None = None,
        bus: Any | None = None,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        if storage_path is None:
            storage_path = (
                Path.home() / ".local" / "share" / "unifideck"
                / "launch_history.json"
            )
        self._path = Path(storage_path)
        self._bus = bus
    def threshold(self) -> int:
        """Threshold."""
        return read_threshold(self._config)
    def window_seconds(self) -> float:
        """Window seconds."""
        return read_window_seconds(self._config)
    def fast_boot_seconds(self) -> float:
        """Fast boot seconds."""
        return read_fast_boot_seconds(self._config)
    def _emit_state(
        self, game_key: str, trigger: str,
    ) -> None:
        """Emit state."""
        if self._bus is None:

```

```

        return
    try:
        recent = self.get_recent_failures(game_key)
        payload = {
            "game_key": game_key,
            "state": "open" if self.is_circuit_open(game_key) else "closed",
            "recent_count": len(recent),
            "failure_kinds": [
                f.get("kind", "unknown") for f in recent
            ],
            "trigger": trigger,
        }
    try:
        loop = asyncio.get_running_loop()
        loop.create_task(
            self._bus.emit(
                Events.CIRCUIT_STATE_CHANGED, **payload,
            ),
            name=f"circuit-event-{game_key}",
        )
    except RuntimeError:
        pass
    except Exception:
        logger.warning(
            "[LaunchHistory] _emit_state failed for %s/%s",
            game_key, trigger,
        )
    @subscribe(Events.GAME_STOPPED)
    async def _on_game_stopped(self, **kwargs) -> None:
        """On game stopped."""
        store = kwargs.get("store", "")
        game_id = kwargs.get("game_id", "")
        exit_code = kwargs.get("exit_code", 0)
        elapsed = kwargs.get("elapsed_seconds", 0.0)
        terminated_by_signal = kwargs.get("terminated_by_signal", False)
        if not store or not game_id:
            return
        game_key = f"{store}:{game_id}"
        if exit_code == 0:
            self.record_success(game_key)
            return
        if terminated_by_signal:
            return
        if exit_code != 0 and elapsed < self.fast_boot_seconds():
            self.record_failure(game_key, FAILURE_KIND_FAST_BOOT)

```

OP-21b

failures.py

Fichier : py_modules/unifideck/services/launch_history/failures.py | Lignes : 71 | Depend de : (rien)

```

from __future__ import annotations
import logging
import time
from pathlib import Path
from typing import Any
from .constants import _VALID_KINDS
from .persistence import load_history, save_history
logger = logging.getLogger(__name__)
class _FailuresMixin:
    """Failures mixin."""
    _path: Path
    def get_recent_failures(
        self, game_key: str,
    ) -> list[dict[str, Any]]:
        """Get recent failures."""
        all_failures = load_history(self._path).get(game_key, {}).get(
            "failures", [],
        )
        cutoff = time.time() - self.window_seconds()
        return [
            f for f in all_failures
            if f.get("ts", 0) >= cutoff
        ]
    def is_circuit_open(self, game_key: str) -> bool:
        """Check whether circuit open."""
        threshold: int = self.threshold()
        return len(self.get_recent_failures(game_key)) >= threshold
    def record_failure(self, game_key: str, kind: str) -> None:
        """Record failure."""
        if kind not in _VALID_KINDS:
            logger.warning(
                "[LaunchHistory] ignoring unknown failure kind: %s",
                kind,
            )
            return
        data = load_history(self._path)
        cutoff = time.time() - self.window_seconds()
        for gk in list(data.keys()):
            entry = data.get(gk, {})
            failures = entry.get("failures", [])
            kept = [f for f in failures if f.get("ts", 0) >= cutoff]
            if kept:
                data[gk] = {"failures": kept}
            else:
                del data[gk]
        data.setdefault(game_key, {"failures": []})
        data[game_key]["failures"].append({
            "ts": time.time(),
            "kind": kind,
        })
        save_history(self._path, data)
        logger.info(
            "[LaunchHistory] recorded %s for %s (total in window: %d)",
            kind, game_key, len(data[game_key]["failures"]),
        )

```

```
def clear_failures(self, game_key: str) -> None:
    """Clear failures."""
    data = load_history(self._path)
    if game_key in data:
        del data[game_key]
        save_history(self._path, data)
        logger.info(
            "[LaunchHistory] cleared failures for %s", game_key,
        )
        self._emit_state(game_key, "clear_failures")

def record_success(self, game_key: str) -> None:
    """Record success."""
    if load_history(self._path).get(game_key):
        self.clear_failures(game_key)
    else:
        self._emit_state(game_key, "record_success")
```

OP-21c

bypass.py

Fichier : py_modules/unifideck/services/launch_history/bypass.py | Lignes : 54 | Depend de : (rien)

```

from __future__ import annotations
import logging
import time
from pathlib import Path
from .persistence import load_history, save_history
logger = logging.getLogger(__name__)
_BYPASS_VALIDITY_SECONDS = 300
class _BypassMixin:
    """Bypass mixin."""
    _path: Path
    def arm_bypass(self, game_key: str) -> None:
        """Arm bypass."""
        try:
            data = load_history(self._path)
            entry = data.setdefault(game_key, {"failures": []})
            entry["bypass_armed"] = time.time()
            save_history(self._path, data)
            logger.info(
                "[LaunchHistory] bypass armed for %s (5-minute window)",
                game_key,
            )
            self._emit_state(game_key, "arm_bypass")
        except Exception:
            logger.exception(
                "[LaunchHistory] arm_bypass failed for %s", game_key,
            )
    def consume_bypass(self, game_key: str) -> bool:
        """Consume bypass."""
        try:
            data = load_history(self._path)
            entry = data.get(game_key)
            if not entry or "bypass_armed" not in entry:
                return False
            armed_at = entry["bypass_armed"]
            elapsed = time.time() - armed_at
            del entry["bypass_armed"]
            save_history(self._path, data)
            if elapsed > _BYPASS_VALIDITY_SECONDS:
                logger.info(
                    "[LaunchHistory] bypass for %s expired "
                    "(%.0fs old), ignored", game_key, elapsed,
                )
                return False
            logger.info(
                "[LaunchHistory] bypass consumed for %s (%.0fs old)",
                game_key, elapsed,
            )
            self._emit_state(game_key, "consume_bypass")
            return True
        except Exception:
            logger.exception(
                "[LaunchHistory] consume_bypass failed for %s", game_key,
            )
            return False

```

OP-21d

config_readers.py

Fichier : py_modules/unifideck/services/launch_history/config_readers.py | Lignes : 29 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
DEFAULT_THRESHOLD = 3
DEFAULT_WINDOW_SECONDS = 600.0
DEFAULT_FAST_BOOT_SECONDS = 10.0
def read_threshold(config: Any | None) -> int:
    """Read threshold."""
    if config is None:
        return DEFAULT_THRESHOLD
    return int(config.get_int(
        "circuit_breaker.failures_threshold",
        DEFAULT_THRESHOLD,
    ))
def read_window_seconds(config: Any | None) -> float:
    """Read window seconds."""
    if config is None:
        return DEFAULT_WINDOW_SECONDS
    return float(config.get_int(
        "circuit_breaker.window_seconds",
        int(DEFAULT_WINDOW_SECONDS),
    ))
def read_fast_boot_seconds(config: Any | None) -> float:
    """Read fast boot seconds."""
    if config is None:
        return DEFAULT_FAST_BOOT_SECONDS
    return float(config.get_int(
        "circuit_breaker.fast_boot_seconds",
        int(DEFAULT_FAST_BOOT_SECONDS),
    ))
```

OP-21e

persistence.py

Fichier : py_modules/unifideck/services/launch_history/persistence.py | Lignes : 37 | Depend de : (rien)

```
from __future__ import annotations
import json
import logging
from pathlib import Path
from typing import Any, cast
logger = logging.getLogger(__name__)
def load_history(path: Path) -> dict[str, Any]:
    """Load history."""
    if not path.exists():
        return {}
    try:
        return cast(
            "dict[str, Any]",
            json.loads(path.read_text() or "{}"),
        )
    except (json.JSONDecodeError, OSError) as err:
        logger.warning(
            "[LaunchHistory] could not read %s: %s – starting fresh",
            path, err,
        )
    return {}
def save_history(path: Path, data: dict[str, Any]) -> None:
    """Save history."""
    path.parent.mkdir(parents=True, exist_ok=True)
    tmp = path.with_suffix(path.suffix + ".tmp")
    try:
        tmp.write_text(json.dumps(data, indent=2))
        tmp.replace(path)
    except OSError as err:
        logger.error(
            "[LaunchHistory] save failed for %s: %s",
            path, err,
        )
    try:
        tmp.unlink(missing_ok=True)
    except OSError:
        pass
```

OP-21f

constants.py

Fichier : py_modules/unifideck/services/launch_history/constants.py | Lignes : 7 | Depend de : (rien)

```
from __future__ import annotations
FAILURE_KIND_FAST_BOOT = "fast_boot"
FAILURE_KIND_LAUNCHER_ERROR = "launcher_error"
_VALID_KINDS = frozenset({
    FAILURE_KIND_FAST_BOOT,
    FAILURE_KIND_LAUNCHER_ERROR,
})
```


OP-22

`__init__.py`

Fichier : `py_modules/unifideck/services/microsoft_subscription/__init__.py` | Lignes : 3 | Depend de : (rien)

```
from __future__ import annotations
from .service import MicrosoftSubscriptionService
__all__ = ["MicrosoftSubscriptionService"]
```

OP-22a

service.py**Fichier :** py_modules/unifideck/services/microsoft_subscription/service.py | Lignes : 110 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import time
from typing import TYPE_CHECKING
from ...core.types import SubscriptionTier
from ...core.types.events import Events
from ...event_bus.event_bus import EventBus
from ...event_bus.event_bus_devex import auto_wire
from .cache_mixin import _CacheMixin
from .constants import _CACHE_STORE_NAME
from .event_handlers import _EventHandlersMixin
from .probe_emission import _ProbeEmissionMixin
from .time_utils import _fmt_ts
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...core.cache_manager import CacheManager
    from ...stores.microsoft.tokens import (
        MicrosoftTokenManager,
        XBLTokenChain,
    )
logger = logging.getLogger(__name__)
class MicrosoftSubscriptionService(
    _CacheMixin, _ProbeEmissionMixin, _EventHandlersMixin,
):
    """Microsoft subscription service."""
    def __init__(
        self,
        bus: EventBus,
        cache: CacheManager,
        config: ConfigManager | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._cache = cache
        self._config = config
        try:
            self._cache.register(_CACHE_STORE_NAME, ttl_seconds=0)
        except Exception:
            logger.exception(
                "[MSSubSvc] could not register cache store %s",
                _CACHE_STORE_NAME,
            )
        self._lock = asyncio.Lock()
        self._last_emitted: dict[str, SubscriptionTier] = {}
        self._last_standard_chain: XBLTokenChain | None = None
        auto_wire(self, self._bus)
        logger.info(
            "[MSSubSvc] initialized (endpoint=%s)",
            self._probe_url(),
        )

    async def get_tier(
        self,
        token_manager: MicrosoftTokenManager,

```

```

) -> SubscriptionTier:

    """Get tier."""
    cache_key = await self._resolve_cache_key(token_manager)
    async with self._lock:
        cached = self._read_cache(cache_key)
        if cached is not None and cached.is_fresh():
            logger.debug(
                "[MSSubSvc] cache hit for %s: tier=%s "
                "(expires in %ds)",
                cache_key,
                cached.tier.value,
                int(cached.expires_at - time.time()),
            )
            return cached.tier
        probe_result = await self._run_probe(token_manager)
        if probe_result.ok:
            await self._store_tier_result(
                cache_key, probe_result.tier,
            )
            result_tier: SubscriptionTier = probe_result.tier
            return result_tier
        if cached is not None:
            logger.warning(
                "[MSSubSvc] probe failed (%s), using stale "
                "cache tier=%s from %s",
                probe_result.error,
                cached.tier.value,
                _fmt_ts(cached.detected_at),
            )
            return cached.tier
        await self._bus.emit(
            Events.SUBSCRIPTION_CHECK_FAILED,
            store="microsoft",
            reason=probe_result.error or "unknown",
        )
        logger.warning(
            "[MSSubSvc] probe failed (%s) and no cache "
            "- returning NONE",
            probe_result.error,
        )
        return SubscriptionTier.NONE
    async def has_active_subscription(
        self,
        token_manager: MicrosoftTokenManager,
    ) -> bool:
        """Check whether active subscription."""
        tier = await self.get_tier(token_manager)
        return tier != SubscriptionTier.NONE
    async def invalidate(self) -> None:
        """Invalidate."""
        try:
            self._cache.clear(_CACHE_STORE_NAME)
        except Exception:
            logger.exception("[MSSubSvc] cache clear failed")
        self._last_emitted.clear()
        logger.info("[MSSubSvc] cache invalidated")

```

OP-22b

cache_mixin.py

Fichier : py_modules/unifideck/services/microsoft_subscription/cache_mixin.py | Lignes : 65 | Depend de : (rien)

```

from __future__ import annotations
import logging
import time
from typing import TYPE_CHECKING
from .cache import _CachedEntry
from .constants import _CACHE_KEY_PREFIX, _CACHE_STORE_NAME
from .time_utils import _end_of_month_utc
if TYPE_CHECKING:
    from ...core.cache_manager import CacheManager
    from ...core.types import SubscriptionTier
    from ...stores.microsoft.tokens import (
        MicrosoftTokenManager,
        XBLTokenChain,
    )
logger = logging.getLogger(__name__)
class _CacheMixin:
    """Cache mixin."""
    _cache: CacheManager
    _last_standard_chain: XBLTokenChain | None
    async def _resolve_cache_key(
        self,
        token_manager: MicrosoftTokenManager,
    ) -> str:
        """Resolve cache key."""
        xuid: str | None = None
        try:
            chain = await token_manager.build_chain()
            if chain is not None:
                xuid = chain.xuid
                self._last_standard_chain = chain
        except Exception:
            logger.debug(
                "[MSSubSvc] could not build chain for key resolution",
                exc_info=True,
            )
        return f"{_CACHE_KEY_PREFIX}{xuid or 'default'}"
    def _read_cache(self, key: str) -> _CachedEntry | None:
        """Read cache."""
        try:
            raw = self._cache.get(_CACHE_STORE_NAME, key)
        except Exception:
            logger.exception("[MSSubSvc] cache read failed")
            return None
        if raw is None:
            return None
        if isinstance(raw, dict):
            return _CachedEntry.from_dict(raw)
        return None
    def _write_cache(self, key: str, entry: _CachedEntry) -> None:
        """Write cache."""
        try:
            self._cache.set(_CACHE_STORE_NAME, key, entry.to_dict())
        except Exception:
            logger.exception("[MSSubSvc] cache write failed")
    async def _store_tier_result(

```

```
        self, cache_key: str, tier: SubscriptionTier,
    ) -> None:
        """Store tier result."""
        entry = _CachedEntry(
            tier=tier,
            expires_at=_end_of_month_utc(),
            detected_at=time.time(),
        )
        self._write_cache(cache_key, entry)
        await self._emit_state_change(cache_key, tier)
```

OP-22c

probe_emission.py

Fichier : py_modules/unifideck/services/microsoft_subscription/probe_emission.py | Lignes : 80 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from .constants import _DEFAULT_PROBE_URL
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...core.types import SubscriptionTier
    from ...event_bus.event_bus import EventBus
    from ...stores.microsoft.microsoft_subscription import SubscriptionProbeResult
    from ...stores.microsoft.tokens import (
        MicrosoftTokenManager,
        XBLTokenChain,
    )
logger = logging.getLogger(__name__)
class _ProbeEmissionMixin:
    """Probe emission mixin."""
    _bus: EventBus
    _config: ConfigManager | None
    _last_emitted: dict[str, SubscriptionTier]
    _last_standard_chain: XBLTokenChain | None
    async def _run_probe(
        self,
        token_manager: MicrosoftTokenManager,
    ) -> SubscriptionProbeResult:
        """Run probe."""
        from ...core.types import SubscriptionTier
        from ...stores.microsoft.microsoft_subscription import (
            SubscriptionProbeResult,
            probe_subscription,
        )
        xbl_token = None
        if self._last_standard_chain is not None:
            xbl_token = self._last_standard_chain.xbl_token
        gssv_chain = await token_manager.build_gssv_chain(
            xbl_token=xbl_token,
        )
        if gssv_chain is None:
            return SubscriptionProbeResult(
                tier=SubscriptionTier.NONE,
                ok=False,
                error="gssv_chain_failed",
            )
        return await probe_subscription(
            user_hash=gssv_chain.user_hash,
            gssv_xsts_token=gssv_chain.xsts_token,
            endpoint_url=self._probe_url(),
        )
    def _probe_url(self) -> str:
        """Probe URL."""
        if self._config is None:
            return _DEFAULT_PROBE_URL
        try:
            raw = self._config.get(
                "stores.microsoft.subscription_check_url",
            )

```

```
        return str(raw) if raw else _DEFAULT_PROBE_URL
    except Exception:
        return _DEFAULT_PROBE_URL

async def _emit_state_change(
    self,
    cache_key: str,
    tier: SubscriptionTier,
) -> None:

    """Emit state change."""
    from ...core.types import Events, SubscriptionTier
    last = self._last_emitted.get(cache_key)
    if last == tier:
        return
    self._last_emitted[cache_key] = tier
    if tier == SubscriptionTier.NONE:
        await self._bus.emit(
            Events.SUBSCRIPTION_EXPIRED,
            store="microsoft",
        )
    else:
        await self._bus.emit(
            Events.SUBSCRIPTION_DETECTED,
            store="microsoft",
            tier=tier.value,
        )
```

OP-22d

event_handlers.py

Fichier : py_modules/unifideck/services/microsoft_subscription/event_handlers.py | Lignes : 24 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import Any
from ...core.types import Events
from ...event_bus.event_bus_devex import subscribe
logger = logging.getLogger(__name__)
class _EventHandlersMixin:
    """Event handlers mixin."""
    @subscribe(Events.STORE_LOGOUT)
    async def _on_logout(self, **kwargs: Any) -> None:
        """On logout."""
        if kwargs.get("store") != "microsoft":
            return
        await self.invalidate()
    @subscribe(Events.STORE_AUTH_COMPLETE)
    async def _on_auth_complete(self, **kwargs: Any) -> None:
        """On auth complete."""
        if kwargs.get("store") != "microsoft":
            return
        await self.invalidate()
    @subscribe(Events.ACCOUNT_SWITCHED)
    async def _on_account_switched(self, **kwargs: Any) -> None:
        """On account switched."""
        await self.invalidate()
```


OP-22e

cache.py

Fichier : py_modules/unifideck/services/microsoft_subscription/cache.py | Lignes : 34 | Depend de : (rien)

```
from __future__ import annotations
import time
from dataclasses import dataclass
from typing import Any
from ...core.types import SubscriptionTier
@dataclass(frozen=True)
class _CachedEntry:
    """Cached entry."""
    tier: SubscriptionTier
    expires_at: float
    detected_at: float
    def is_fresh(self, now: float | None = None) -> bool:
        """Check whether fresh."""
        return (now if now is not None else time.time()) < self.expires_at
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        return {
            "tier": self.tier.value,
            "expires_at": self.expires_at,
            "detected_at": self.detected_at,
        }
    @classmethod
    def from_dict(
        cls, raw: dict[str, Any],
    ) -> _CachedEntry | None:
        """From dict."""
        try:
            return cls(
                tier=SubscriptionTier(raw["tier"]),
                expires_at=float(raw["expires_at"]),
                detected_at=float(raw["detected_at"]),
            )
        except (KeyError, ValueError, TypeError):
            return None
```

OP-22f

time_utils.py

Fichier : py_modules/unifideck/services/microsoft_subscription/time_utils.py | Lignes : 13 | Depend de : (rien)

```
from __future__ import annotations
from datetime import UTC, datetime
def _end_of_month_utc(now: datetime | None = None) -> float:
    """End of month utc."""
    now = now if now is not None else datetime.now(UTC)
    if now.month == 12:
        nxt = datetime(now.year + 1, 1, 1, tzinfo=UTC)
    else:
        nxt = datetime(now.year, now.month + 1, 1, tzinfo=UTC)
    return nxt.timestamp()
def _fmt_ts(ts: float) -> str:
    """Fmt ts."""
    return datetime.fromtimestamp(ts, tz=UTC).isoformat()
```

OP-22g

constants.py

Fichier : py_modules/unifideck/services/microsoft_subscription/constants.py | Lignes : 6 | Depend de : (rien)

```
from __future__ import annotations
_CACHE_STORE_NAME = "microsoft_subscription"
_CACHE_KEY_PREFIX = "ms_sub_"
_DEFAULT_PROBE_URL = (
    "https://xgpuweb.gssv-play-prod.xboxlive.com/v2/login/user"
)
```

OP-12a

metadata_service.py

Fichier : py_modules/unifideck/services/metadata_service.py | Lignes : 105 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any, cast
from ..core.cache_manager import CacheManager
from ..core.types import Events, Game
from ..event_bus.event_bus import EventBus
from ..event_bus.event_bus_devex import subscribe
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
CACHE_NAMESPACE = "metadata"
DEFAULT_CACHE_TTL = 7 * 24 * 3600
class MetadataService:
    """Metadata service."""
    def __init__(
        self,
        bus: EventBus,
        cache: CacheManager,
        config: ConfigManager | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._cache = cache
        self._config = config
        ttl = DEFAULT_CACHE_TTL
        if config is not None:
            try:
                ttl = int(config.get(
                    "cache_ttl.steam_metadata"))
            except (TypeError, ValueError):
                pass
        self._cache.register(CACHE_NAMESPACE, ttl_seconds=ttl)
        mc_ttl = ttl
        if config is not None:
            try:
                mc_ttl = int(config.get(
                    "cache_ttl.metacritic_metadata"))
            except (TypeError, ValueError):
                pass
        self._cache.register("metacritic", ttl_seconds=mc_ttl)
        from ..event_bus.event_bus_devex import auto_wire
        auto_wire(self, self._bus)
        logger.info("[MetadataService] wired (1 subscription)")
    async def stop(self) -> None:
        """Stop."""
        self._bus.off(Events.SYNC_COMPLETE, self._on_sync_complete)
    @subscribe(Events.SYNC_COMPLETE)
    async def _on_sync_complete(self, **kwargs) -> None:
        """On sync complete."""
        games: list[Game] = kwargs.get("games", [])
        for game in games:
            await self.enrich(game)

    async def enrich(self, game: Game) -> dict[str, Any]:

```

```

    """Enrich."""
    cache_key = f"{game.store}:{game.store_game_id}"
    cached = self._cache.get(CACHE_NAMESPACE, cache_key)
    if cached is not None:
        return cast("dict[str, Any]", cached)
    metadata: dict[str, Any] = {}
    steam_data = await self._fetch_steam_store(game.title)
    if steam_data:
        metadata["steam"] = steam_data
        metadata["steam_app_id"] = steam_data.get("app_id")
    unifidb_data = await self._fetch_unifidb(game)
    if unifidb_data:
        metadata["unifidb"] = unifidb_data
    metacritic_data = await self._fetch_metacritic(game.title)
    if metacritic_data:
        metadata["metacritic"] = metacritic_data
    self._cache.set(CACHE_NAMESPACE, cache_key, metadata)
    return metadata

async def _fetch_steam_store(
    self, title: str,
) -> dict[str, Any] | None:
    """Fetch steam store."""
    try:
        from ..steam.library import search_store
        return await search_store(title, config=self._config)
    except Exception as e:
        logger.debug("[MetadataService] steam fetch: %s", e)
        return None

async def _fetch_unifidb(
    self, game: Game,
) -> dict[str, Any] | None:
    """Fetch unifidb."""
    try:
        from ..metadata.unifidb import lookup
        return await lookup(
            game.store, game.store_game_id, game.title,
            config=self._config,
        )
    except Exception as e:
        logger.debug("[MetadataService] unifidb fetch: %s", e)
        return None

async def _fetch_metacritic(
    self, title: str,
) -> dict[str, Any] | None:
    """Fetch metacritic."""
    try:
        from ..metadata.metacritic import fetch_score
        result = await fetch_score(title, config=self._config)
        return result.to_dict() if result else None
    except Exception as e:
        logger.debug("[MetadataService] metacritic fetch: %s", e)
        return None

```

OP-12b

proton_service.py

Fichier : py_modules/unifideck/services/proton_service.py | Lignes : 99 | Depend de : (rien)

```

from __future__ import annotations
import logging
import re
from ..core.types import Events, Result
from ..event_bus.event_bus import EventBus
from ..event_bus.event_bus_devex import subscribe
logger = logging.getLogger(__name__)
DEFAULT_TOOLS: dict[str, str] = {
    "epic": "proton_experimental",
    "gog": "proton_experimental",
    "amazon": "proton_experimental",
    "ubisoft": "proton_experimental",
    "microsoft": "",
}
}
class ProtonService:
    """Proton service."""
    def __init__(self, bus: EventBus, config_vdf_path: str,
        overrides: dict[str, str] | None = None) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._config_vdf = config_vdf_path
        self._tools: dict[str, str] = {**DEFAULT_TOOLS,
            **(overrides or {})}
        from ..event_bus.event_bus_devex import auto_wire
        auto_wire(self, self._bus)
        logger.info("[ProtonService] wired (1 subscription)")
    async def stop(self) -> None:
        """Stop."""
        self._bus.off(Events.GAME_INSTALLED, self._on_game_installed)
    @subscribe(Events.GAME_INSTALLED)
    async def _on_game_installed(self, **kwargs) -> None:
        """On game installed."""
        app_id = kwargs.get("app_id")
        store = kwargs.get("store", "")
        if not app_id:
            return
        tool = self._tools.get(store, "")
        if not tool:
            return
        await self.set_compat_tool(app_id, tool)
    async def set_compat_tool(self, app_id: int, tool: str) -> Result:
        """Set compat tool."""
        from ..core.io import async_file_ops as aio
        if not await aio.is_file(self._config_vdf):
            return Result(
                success=False, error="config_vdf_missing",
            )
        content = await aio.read_text(self._config_vdf)
        if content is None:
            return Result(
                success=False, error="config_vdf_read_failed",
            )
        new_content = self._inject_compat_tool(
            content, app_id, tool,
        )

```

```

    if new_content == content:
        return Result(success=True)
    try:
        await aio.write_text(
            self._config_vdf, new_content,
        )
    except Exception as e:
        return Result(success=False, error=str(e))
    logger.info(
        "[ProtonService] app %d → %s", app_id, tool,
    )
    return Result(success=True)

@staticmethod
def _inject_compat_tool(content: str, app_id: int,
    tool: str) -> str:
    """Inject compat tool."""
    block_re = re.compile(
        rf'"{app_id}"\s*\{{\^[^}]*"name"\s*\^[^}]*\}}\}',
        re.DOTALL,
    )
    new_block = (
        f'"{app_id}"\n {{\n'
        f'  "name" "{tool}"\n'
        f'  "config" ""\n'
        f'  "priority" "250"\n'
        f' }}'
    )
    if block_re.search(content):
        return block_re.sub(new_block, content)
    section_re = re.compile(
        r'"CompatToolMapping"\s*\{',
    )
    m = section_re.search(content)
    if m:
        insert_at = m.end()
        return (
            content[:insert_at]
            + "\n " + new_block
            + content[insert_at:]
        )
    return content.rstrip() + (
        '\n "CompatToolMapping"\n {\n '
        + new_block + '\n }\n'
    )

```

OP-12c

account_service.py

Fichier : py_modules/unifideck/services/account_service.py | Lignes : 102 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import re
from typing import TYPE_CHECKING
from ..core.types import Events
from ..event_bus.event_bus import EventBus
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
DEFAULT_POLL_INTERVAL = 5
class AccountService:
    """Account service."""
    def __init__(
        self,
        bus: EventBus,
        loginusers_path: str,
        config: ConfigManager | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._loginusers_path = loginusers_path
        self._current_user: str | None = None
        self._poll_task: asyncio.Task | None = None
        self._poll_interval = DEFAULT_POLL_INTERVAL
        if config is not None:
            try:
                self._poll_interval = int(config.get(
                    "accounts.poll_interval_seconds"))
            except (TypeError, ValueError):
                pass
    async def start(self) -> None:
        """Start."""
        self._current_user = await self._read_active_user()
        logger.info("[AccountService] initial user=%s",
            self._current_user)
        self._poll_task = asyncio.create_task(self._poll_loop())
    async def stop(self) -> None:
        """Stop."""
        if self._poll_task:
            self._poll_task.cancel()
            try:
                await self._poll_task
            except asyncio.CancelledError:
                pass
    def get_current_user(self) -> str | None:
        """Get current user."""
        return self._current_user
    async def force_check(self) -> bool:
        """Force check."""
        return await self._check_once()
    async def _poll_loop(self) -> None:
        """Poll loop."""
        try:
            while True:
```



```

        try:
            await self._check_once()
        except Exception as e:
            logger.warning("[AccountService] poll error: %s", e)
            await asyncio.sleep(self._poll_interval)
    except asyncio.CancelledError:
        raise

async def _check_once(self) -> bool:

    """Check once."""
    new_user = await self._read_active_user()
    if new_user is None:
        return False
    if new_user != self._current_user:
        previous = self._current_user
        self._current_user = new_user
        logger.info(
            "[AccountService] account switch: %s → %s",
            previous, new_user,
        )
        await self._bus.emit(
            Events.ACCOUNT_SWITCHED,
            previous_user=previous,
            current_user=new_user,
        )
        return True
    return False

async def _read_active_user(self) -> str | None:
    """Read active user."""
    from ..core.io import async_file_ops as aio
    try:
        if not await aio.is_file(self._loginusers_path):
            return None
        content = await aio.read_text(self._loginusers_path)
    except Exception:
        return None
    if content is None:
        return None
    return self._extract_most_recent(content)

@staticmethod
def _extract_most_recent(vdf_text: str) -> str | None:
    """Extract most recent."""
    pattern = re.compile(
        r'(\d{17})\s*\{[^}]*"MostRecent"\s*"1"',
        re.DOTALL,
    )
    m = pattern.search(vdf_text)
    return m.group(1) if m else None

```

OP-12d

feature_flag_service.py

Fichier : py_modules/unifideck/services/feature_flag_service.py | Lignes : 110 | Depend de : (rien)

```

from __future__ import annotations
import logging
from ..core.types import Events
from ..event_bus.event_bus import EventBus
from ..event_bus.event_bus_devex import auto_wire, subscribe
logger = logging.getLogger(__name__)
PROBE_TO_FEATURES: dict[str, list[str]] = {
    "steam_client_apps": [
        "shortcut_creation",
        "artwork_injection",
        "play_button_override",
    ],
    "steam_client_downloads": [
        "download_progress_polling",
        "download_queue_ui",
    ],
    "steam_client_input": [
        "controller_hints",
    ],
    "router_hook_patch": [
        "library_view_patch",
    ],
    "rpc_roundtrip": [
        "diagnostics_panel_polling",
    ],
}
ALL_FEATURES: list[str] = sorted({
    f for features in PROBE_TO_FEATURES.values() for f in features
})
class FeatureFlagService:
    """Feature flag service."""
    def __init__(self, bus: EventBus, config: object | None = None) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._mapping = self._load_mapping(config)
        self._all_features = sorted({
            f for fs in self._mapping.values() for f in fs
        })
        self._flags: dict[str, bool] = dict.fromkeys(self._all_features, True)
        self._last_report: dict | None = None
        auto_wire(self, self._bus)
        logger.info(
            "[FeatureFlagService] initialized with %d features "
            "from %d probe mappings",
            len(self._flags), len(self._mapping),
        )
    @staticmethod
    def _load_mapping(
        config: object | None,
    ) -> dict[str, list[str]]:
        """Load mapping."""
        if config is None or not hasattr(config, "get"):
            return dict(PROBE_TO_FEATURES)
        raw = config.get("probes.probe_to_features")
        if not isinstance(raw, dict):

```

```

        return dict(PROBE_TO_FEATURES)
merged = dict(PROBE_TO_FEATURES)
for k, v in raw.items():
    if isinstance(v, list) and all(
        isinstance(x, str) for x in v
    ):
        merged[k] = v
return merged

def get_flags(self) -> dict[str, bool]:

    """Get flags."""
    return dict(self._flags)
def is_enabled(self, feature: str) -> bool:
    """Check whether enabled."""
    return self._flags.get(feature, True)
@subscribe(Events.RUNTIME_PROBES_REPORTED)
async def _on_probes_reported(self, **kwargs) -> None:
    """On probes reported."""
    probes = kwargs.get("probes")
    if not isinstance(probes, list):
        logger.warning(
            "[FeatureFlagService] probes kwarg missing or bad type",
        )
    return
    self._last_report = {"probes": probes}
    changed = self._apply_probes_to_flags(probes)
    if changed:
        logger.info(
            "[FeatureFlagService] updated %d features: %s",
            len(changed), sorted(changed),
        )
def _apply_probes_to_flags(self, probes: list) -> list[str]:
    """Apply probes to flags."""
    changed: list[str] = []
    for probe in probes:
        if not isinstance(probe, dict):
            continue
        probe_id = probe.get("id") or probe.get("name")
        if not isinstance(probe_id, str):
            continue
        is_fail = (
            probe.get("verdict") == "fail"
            or probe.get("severity") == "error"
        )
        is_ok = (
            probe.get("verdict") == "ok"
            or probe.get("severity") == "info"
        )
        if not (is_fail or is_ok):
            continue
        new_state = not is_fail
        for feature in self._mapping.get(probe_id, []):
            if self._flags.get(feature) != new_state:
                self._flags[feature] = new_state
                changed.append(feature)
    return changed

```

OP-12e

probe_reaction_service.py

Fichier : py_modules/unifideck/services/probe_reaction_service.py | Lignes : 114 | Depend de : (rien)

```

from __future__ import annotations
import logging
import time
from collections import deque
from typing import Any
from ..core.types import Events
from ..event_bus.event_bus import EventBus
from ..event_bus.event_bus_devex import auto_wire, subscribe
logger = logging.getLogger(__name__)
PROBE_TO_HANDLERS: dict[str, list[str]] = {
    "steam_client_apps": [
        "ArtworkService._on_shortcut_created",
        "ShortcutService._on_download_complete",
        "ShortcutService._on_sync_complete",
    ],
    "steam_client_downloads": [
        "ShortcutService._on_download_complete",
    ],
}
}
HISTORY_MAX_ENTRIES = 50
class ProbeReactionService:
    """Probe reaction service."""
    def __init__(
        self,
        bus: EventBus,
        watchdog: Any,
        config: object | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._watchdog = watchdog
        self._mapping = self._load_mapping(config)
        self._history: deque[dict] = deque(maxlen=HISTORY_MAX_ENTRIES)
        auto_wire(self, self._bus, watchdog=watchdog)
        logger.info(
            "[ProbeReactionService] initialized with %d probe mappings",
            len(self._mapping),
        )
    @staticmethod
    def _load_mapping(
        config: object | None,
    ) -> dict[str, list[str]]:
        """Load mapping."""
        if config is None or not hasattr(config, "get"):
            return dict(PROBE_TO_HANDLERS)
        raw = config.get("probes.probe_to_handlers")
        if not isinstance(raw, dict):
            return dict(PROBE_TO_HANDLERS)
        merged = dict(PROBE_TO_HANDLERS)
        for k, v in raw.items():
            if isinstance(v, list) and all(
                isinstance(x, str) for x in v
            ):
                merged[k] = v
        return merged

```

```

def get_history(self) -> list[dict]:
    """Get history."""
    return list(self._history)
@subscribe(Events.RUNTIME_PROBES_REPORTED)
async def _on_probes_reported(self, **kwargs) -> None:
    """On probes reported."""
    probes = kwargs.get("probes")
    if not isinstance(probes, list):
        return
    self._record_in_history(probes)
    self._quarantine_affected_handlers(probes)

def _record_in_history(self, probes: list) -> None:
    """Record in history."""
    entry = {
        "timestamp": time.monotonic(),
        "probes": [
            {
                "id": p.get("id") or p.get("name", "?"),
                "verdict": (
                    p.get("verdict")
                    or ("fail" if p.get("severity") == "error"
                       else "ok")
                ),
            }
            for p in probes
            if isinstance(p, dict)
        ],
    }
    self._history.append(entry)
def _quarantine_affected_handlers(self, probes: list) -> None:
    """Quarantine affected handlers."""
    if self._watchdog is None:
        return
    affected: dict[str, str] = {}
    for probe in probes:
        if not isinstance(probe, dict):
            continue
        is_fail = (
            probe.get("verdict") == "fail"
            or probe.get("severity") == "error"
        )
        if not is_fail:
            continue
        probe_id = probe.get("id") or probe.get("name")
        if not isinstance(probe_id, str):
            continue
        for handler_name in self._mapping.get(probe_id, []):
            affected.setdefault(handler_name, probe_id)
    quarantined: list[str] = []
    for handler_name, probe_id in affected.items():
        if self._watchdog.quarantine_preemptive(
            handler_name, reason=f"probe:{probe_id}",
        ):
            quarantined.append(handler_name)
    if quarantined:
        logger.warning(
            "[ProbeReactionService] quarantined %d handlers: %s",
            len(quarantined), quarantined,
        )

```

OP-23

`__init__.py`

Fichier : `py_modules/unifideck/security/__init__.py` | Lignes : 40 | Depend de : (rien)

```
from .audit_emitter import (
    audit_auth_flow,
    emit_auth_completed,
    emit_auth_failed,
    emit_auth_started,
    emit_external_auth_check_failed,
    emit_legacy_plaintext_detected,
    emit_permissions_check,
    emit_token_file_migrated,
)
from .device_fingerprint import (
    DeviceFingerprint,
    FingerprintState,
)
from .device_identity import (
    DeviceIdentity,
    DeviceIdentityError,
    FakeDeviceIdentity,
)
from .secure_token_store import (
    SecureTokenStore,
    SecureTokenStoreError,
)
__all__ = [
    "DeviceIdentity",
    "DeviceIdentityError",
    "FakeDeviceIdentity",
    "DeviceFingerprint",
    "FingerprintState",
    "SecureTokenStore",
    "SecureTokenStoreError",
    "audit_auth_flow",
    "emit_auth_started",
    "emit_auth_completed",
    "emit_auth_failed",
    "emit_token_file_migrated",
    "emit_legacy_plaintext_detected",
    "emit_permissions_check",
    "emit_external_auth_check_failed",
]
```

OP-23a

device_identity.py

Fichier : py_modules/unifideck/security/device_identity.py | Lignes : 105 | Depend de : (rien)

```

"""security/device_identity.py – Stable hardware identifier reader.
Reads /etc/machine-id, the systemd-managed unique identifier
generated once at SteamOS install time. This file is:
- Stable across reboots and SteamOS updates
- Different on every Deck (32 hex chars from 128 random bits)
- Reset only on a complete SteamOS reinstall (which is the
  expected behaviour: tokens become unrecoverable, forcing a
  fresh OAuth flow on the new system)
- Readable without privileges (mode 0o444 typically)
This identifier is NOT a secret – any process running on the
device can read it. It is used here purely as a *device binding*
factor: combined with a static salt, it produces a key that is
unique to a specific Deck and cannot be reused on a different
device. It does NOT protect against a local attacker who has
read access to the user's home directory.
The read is wrapped in a class to make injection trivial in
tests – production code calls DeviceIdentity().read(), tests call
FakeDeviceIdentity("aabbccdd...").read().
"""

from __future__ import annotations
import logging
logger = logging.getLogger(__name__)
# Path to the systemd machine-id file. Same on every modern
# Linux distribution including SteamOS. The fallback location
# /var/lib/dbus/machine-id is a symlink to /etc/machine-id on
# all systems we target, so we don't bother reading it.
_MACHINE_ID_PATH = "/etc/machine-id"
class DeviceIdentityError(RuntimeError):
    """Raised when the machine-id cannot be read or is malformed.
    We treat this as fatal for the secure storage layer: there
    is no safe fallback that doesn't compromise the entire
    threat model. Better to crash loudly than to silently
    use a constant value (which would make the encryption
    pointless device-wide).
    """

class DeviceIdentity:
    """Reads and caches the device's machine-id.
    The value is read once on first access and cached for the
    lifetime of the instance. /etc/machine-id never changes at
    runtime (only on a reboot after a deliberate reset), so
    re-reading it on every encryption operation would be wasted
    syscalls.
    """
    def __init__(self, path: str = _MACHINE_ID_PATH) -> None:
        """Create a reader for the given machine-id path.
        The path argument exists purely for tests – production
        always uses _MACHINE_ID_PATH. Tests can point at a
        fixture file with a known value.
        """
        self._path = path
        self._cached: str | None = None

    def read(self) -> str:
        """Return the machine-id as a 32-char lowercase hex string.
        Raises DeviceIdentityError if the file is missing, unreadable,

```

```

or contains a value that doesn't look like a machine-id.
Result is cached on first call.
"""
if self._cached is not None:
    return self._cached
try:
    with open(self._path, encoding="utf-8") as f:
        raw = f.read().strip()
except OSError as e:
    raise DeviceIdentityError(
        f"cannot read {self._path}: {e}",
    ) from e
if not self._looks_valid(raw):
    raise DeviceIdentityError(
        f"machine-id at {self._path} is malformed: "
        f"expected 32 hex chars, got {len(raw)} chars",
    )
self._cached = raw.lower()
logger.debug("[DeviceIdentity] read 32-char id from %s", self._path)
return self._cached

@staticmethod
def _looks_valid(value: str) -> bool:
    """Return True if value matches the machine-id format.
    systemd machine-ids are always exactly 32 lowercase hex
    characters (128 bits encoded). We reject anything else
    loudly rather than trying to be clever.
    """
    if len(value) != 32:
        return False
    try:
        int(value, 16)
    except ValueError:
        return False
    return True

class FakeDeviceIdentity:
    """Test double that returns a fixed value.
    Useful for unit tests that need a deterministic encryption
    key without touching the real /etc/machine-id. Implements
    the same .read() interface as DeviceIdentity.
    """
    def __init__(self, value: str) -> None:
        """Create a fake reader returning `value` on every call."""
        if not DeviceIdentity._looks_valid(value):
            raise ValueError(
                f"FakeDeviceIdentity requires a valid 32-char hex string, "
                f"got {len(value)} chars",
            )
        self._value = value.lower()
    def read(self) -> str:
        """Return the fixed value passed to the constructor."""
        return self._value

```


OP-23b

secure_token_store.py

Fichier : py_modules/unifideck/security/secure_token_store.py | Lignes : 247 | Depend de : (rien)

```

"""core/secure_token_store.py – Encrypted on-disk token storage.
Provides authenticated symmetric encryption for OAuth tokens
persisted by store modules (currently GOG and Microsoft). Built
on cryptography's AESGCM primitive with a key derived via scrypt
from the device's machine-id and a static domain-separation salt.
Threat model addressed
-----

```

This layer protects against ONE specific scenario: accidental exfiltration of the encrypted token file from the device. A user who shares a bug report archive, syncs ~/.config to a cloud backup, or uploads a file to a forum will not leak usable tokens because the recipient cannot decrypt the file without the machine-id of the original Steam Deck.

Threat model NOT addressed

This layer does NOT protect against:

- A local attacker with read access to the user's home (they can read /etc/machine-id and reconstruct the key)
- A malicious process running under the same uid as Unifideck
- Memory dumps while tokens are in use (plaintext in memory)
- Compromise of the OAuth token at the network level

These are out of scope for a Decky plugin. A real defence-in-depth solution would require a TPM-backed key, kernel-enforced process isolation, or a hardware security module – none of which are accessible from a userspace Python plugin on SteamOS.

Format

Encrypted file layout (binary):

- [4 bytes] magic header "UFD1"
- [12 bytes] AES-GCM nonce (random per encryption)
- [N bytes] ciphertext + 16-byte GCM auth tag

Total overhead: 32 bytes per file. The magic header lets us detect non-encrypted files from a previous Unifideck version and trigger a one-shot migration without crashing.

KDF parameters

scrypt(N=2**14, r=8, p=1) – the same parameters as the original scrypt paper for "interactive" use. ~50 ms on a Steam Deck APU which is acceptable on the auth path (called once per save) but slow enough to make brute-forcing impractical.

The salt is a fixed 16-byte constant baked into this module. It is NOT a secret – its only purpose is domain separation, so that a key derived for Unifideck cannot be reused on another piece of software that happens to derive keys from the same machine-id.

"""

```

from __future__ import annotations
import json
import logging
import os
from typing import Any
from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
from .device_identity import DeviceIdentity, DeviceIdentityError
logger = logging.getLogger(__name__)

```

```

# Magic header that identifies a Unifideck-encrypted file.
# "UFD" + format version 1. If we ever change the format

# (different KDF, different cipher), bump the digit and add a
# branch in _decrypt to handle legacy versions.
_MAGIC = b"UFD1"
_NONCE_SIZE = 12 # AES-GCM standard nonce size
_KEY_SIZE = 32 # AES-256
# Static salt for the KDF. Different from any other software
# that might derive keys from the same machine-id. NOT secret.
# 16 bytes is the cryptography library's recommended size.
_KDF_SALT = b"unifideck-secure-token-store-v1!" # 32 bytes
assert len(_KDF_SALT) == 32, "salt must be exactly 32 bytes"
# scrypt parameters – interactive use, ~50ms on Steam Deck APU.
_SCRYPT_N = 2 ** 14
_SCRYPT_R = 8
_SCRYPT_P = 1
class SecureTokenStoreError(RuntimeError):
    """Raised when encryption or decryption fails irrecoverably.
    Distinct from DeviceIdentityError so callers can decide whether
    to wipe the file (corrupt ciphertext) or escalate (no
    machine-id available – system misconfigured).
    """
class SecureTokenStore:
    """Encrypts and decrypts JSON token payloads on disk.
    Stateless apart from the lazily-derived key, which is cached
    after first use. Safe to instantiate once per store and reuse
    for every load/save cycle. Not thread-safe – callers should
    serialize access if multiple coroutines write concurrently.
    """
    def __init__(
        self,
        device_identity: DeviceIdentity | None = None,
        bus: Any = None,
    ) -> None:
        """Create a store using the given DeviceIdentity reader.
        device_identity: injected for tests (FakeDeviceIdentity).
        Defaults to a real DeviceIdentity() reading /etc/machine-id.
        bus: optional EventBus for emitting SECURITY_* audit events.
        When None (default), the store works exactly as before but
        emits nothing – keeping unit tests minimal. Production code
        in main.py passes the real bus so SecurityService can
        observe every encryption operation.
        """
        self._device_identity = device_identity or DeviceIdentity()
        self._bus = bus
        self._key: bytes | None = None
# — Public API —
def encrypt_payload(self, payload: dict[str, Any]) -> bytes:
    """Serialise `payload` to JSON, encrypt, return raw bytes.
    The returned bytes are ready to write to disk as a binary
    file. Callers should still set 0o600 permissions on the
    file itself – encryption protects the contents, not the
    metadata (which would still leak which keys exist).
    """
    plaintext = json.dumps(payload, separators=(",", ":")).encode("utf-8")
    return self._encrypt(plaintext)

def decrypt_payload(self, blob: bytes) -> dict[str, Any]:
    """Decrypt and parse `blob` back into a payload dict.
    Raises SecureTokenStoreError on any failure (corrupted

```

```

file, wrong machine-id, tampered ciphertext, malformed
JSON inside). Callers should handle this by treating
the file as unusable and forcing a re-auth.
"""
plaintext = self._decrypt(blob)
try:
    data = json.loads(plaintext.decode("utf-8"))
except (ValueError, UnicodeDecodeError) as e:
    raise SecureTokenStoreError(
        f"decrypted payload is not valid JSON: {e}",
    ) from e
if not isinstance(data, dict):
    raise SecureTokenStoreError(
        f"decrypted payload is not a JSON object "
        f"(got {type(data).__name__})",
    )
return data
def is_encrypted(self, blob: bytes) -> bool:
    """Return True if `blob` looks like an encrypted file.
    Used by migration logic: a legacy unencrypted JSON file
    starts with `{` (0x7b), an encrypted file starts with
    the magic header `UFD1`. The check is cheap (4 byte
    comparison) so callers can do it on every load to
    transparently handle both formats during the migration
    window.
    """
    return blob[:4] == _MAGIC
# — Private helpers —
def _get_key(self) -> bytes:
    """Derive the AES-256 key from the machine-id, cached.
    Raises SecureTokenStoreError if the machine-id cannot
    be read. The KDF call takes ~50ms on a Steam Deck so
    we cache the result for the instance lifetime.
    """
    if self._key is not None:
        return self._key
    try:
        mid = self._device_identity.read()
    except DeviceIdentityError as e:
        raise SecureTokenStoreError(
            f"cannot derive key without machine-id: {e}",
        ) from e
    kdf = Scrypt(
        salt=_KDF_SALT,
        length=_KEY_SIZE,
        n=_SCRYPT_N,
        r=_SCRYPT_R,
        p=_SCRYPT_P,
    )
    self._key = kdf.derive(mid.encode("utf-8"))
    logger.debug("[SecureTokenStore] derived AES-256 key from machine-id")
    return self._key
def _encrypt(self, plaintext: bytes) -> bytes:
    """Encrypt `plaintext` and return magic + nonce + ciphertext.
    Generates a fresh random nonce on every call. AES-GCM
    catastrophically fails (key recovery) if a nonce is
    reused with the same key, so we never derive nonces
    from the plaintext or use a counter.
    emits SECURITY_TOKEN_ENCRYPTED on the bus
    if one was injected. Failures during the crypto call
    itself bubble up unchanged (they would be a hard bug,

```

```

        not a runtime condition).
        """
        key = self._get_key()
        nonce = os.urandom(_NONCE_SIZE)
        aesgcm = AESGCM(key)
        ciphertext = aesgcm.encrypt(nonce, plaintext, associated_data=None)
        self._emit_security_event(
            "SECURITY_TOKEN_ENCRYPTED",
            byte_count=len(plaintext),
        )
        return _MAGIC + nonce + ciphertext

def _decrypt(self, blob: bytes) -> bytes:
    """Verify magic, extract nonce, authenticate + decrypt.
    Raises SecureTokenStoreError on any anomaly: short blob,
    wrong magic, GCM auth failure (means tampered or wrong
    key, indistinguishable by design).
    emits SECURITY_TOKEN_DECRYPTED on success
    or SECURITY_DECRYPT_FAILED with a reason string on any
    failure path. The reason never includes ciphertext bytes
    or partial plaintext, only the failure category.
    """
    if len(blob) < len(_MAGIC) + _NONCE_SIZE + 16:
        self._emit_security_event(
            "SECURITY_DECRYPT_FAILED", reason="blob_too_short",
        )
        raise SecureTokenStoreError(
            f"encrypted blob too short: {len(blob)} bytes",
        )
    if not self.is_encrypted(blob):
        self._emit_security_event(
            "SECURITY_DECRYPT_FAILED", reason="bad_magic",
        )
        raise SecureTokenStoreError(
            "blob does not start with UFD1 magic header",
        )
    nonce = blob[len(_MAGIC):len(_MAGIC) + _NONCE_SIZE]
    ciphertext = blob[len(_MAGIC) + _NONCE_SIZE:]
    key = self._get_key()
    aesgcm = AESGCM(key)
    try:
        plaintext = aesgcm.decrypt(nonce, ciphertext, associated_data=None)
    except InvalidTag as e:
        self._emit_security_event(
            "SECURITY_DECRYPT_FAILED", reason="gcm_auth_failed",
        )
        raise SecureTokenStoreError(
            "AES-GCM authentication failed – file is corrupt, "
            "tampered, or was encrypted on a different device",
        ) from e
    self._emit_security_event(
        "SECURITY_TOKEN_DECRYPTED", byte_count=len(plaintext),
    )
    return plaintext

def _emit_security_event(self, event_name: str, **kwargs) -> None:
    """Emit a SECURITY_* event on the bus if one is configured.
    The event enum is imported lazily to avoid a circular
    import between core.types.events and the security package.
    Failures to emit (bus down, handler raised) are caught and
    logged at debug level – security audit must NEVER block
    the actual crypto operation.

```

```
"""
if self._bus is None:
    return
try:
    from ..core.types.events import Events
    event = getattr(Events, event_name)
    self._bus.emit(event, **kwargs)
except Exception as e: # noqa: BLE001 – intentional: defensive catch logged below
    logger.debug(
        "[SecureTokenStore] failed to emit %s: %s",
        event_name, e,
    )
```

OP-23c

audit_emitter.py

Fichier : py_modules/unifideck/security/audit_emitter.py | Lignes : 151 | Depend de : (rien)

```

from __future__ import annotations
import functools
import logging
import time
from collections.abc import Callable
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from ..event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
def _safe_emit(bus: EventBus, event_name: str, **kwargs: Any) -> None:
    """Safe emit."""
    if bus is None:
        return
    try:
        import asyncio
        from ..core.types.events import Events
        event = getattr(Events, event_name)
        try:
            loop = asyncio.get_running_loop()
        except RuntimeError:
            return
        loop.create_task(
            bus.emit(event, **kwargs),
            name=f"audit-emit-{event_name}",
        )
    except Exception as e:
        logger.debug(
            "[audit_emitter] failed to emit %s: %s",
            event_name, e,
        )
def emit_auth_started(
    bus: EventBus, store: str, method: str | None = None,
) -> None:
    """Emit auth started."""
    kwargs = {"store": store}
    if method is not None:
        kwargs["method"] = method
    _safe_emit(bus, "SECURITY_AUTH_FLOW_STARTED", **kwargs)
def emit_auth_completed(
    bus: EventBus,
    store: str,
    duration_seconds: float | None = None,
    method: str | None = None,
) -> None:
    """Emit auth completed."""
    kwargs: dict[str, Any] = {"store": store}
    if duration_seconds is not None:
        kwargs["duration_seconds"] = round(duration_seconds, 3)
    if method is not None:
        kwargs["method"] = method
    _safe_emit(bus, "SECURITY_AUTH_FLOW_COMPLETED", **kwargs)
def emit_auth_failed(
    bus: EventBus,
    store: str,
    reason: str,

```

```

        duration_seconds: float | None = None,
        method: str | None = None,
    ) -> None:
        """Emit auth failed."""
        kwargs: dict[str, Any] = {"store": store, "reason": reason}
        if duration_seconds is not None:
            kwargs["duration_seconds"] = round(duration_seconds, 3)
        if method is not None:
            kwargs["method"] = method
        _safe_emit(bus, "SECURITY_AUTH_FLOW_FAILED", **kwargs)

def emit_token_file_migrated(
    bus: EventBus, store: str, from_path: str, to_path: str,
) -> None:
    """Emit token file migrated."""
    _safe_emit(
        bus, "SECURITY_TOKEN_FILE_MIGRATED",
        store=store, from_path=from_path, to_path=to_path,
    )

def emit_legacy_plaintext_detected(
    bus: EventBus, store: str, path: str,
) -> None:
    """Emit legacy plaintext detected."""
    _safe_emit(
        bus, "SECURITY_LEGACY_PLAINTEXT_DETECTED",
        store=store, path=path,
    )

def emit_permissions_check(
    bus: EventBus, store: str, path: str, mode: int,
) -> None:
    """Emit permissions check."""
    _safe_emit(
        bus, "SECURITY_PERMISSIONS_CHECK",
        store=store, path=path, mode=mode,
    )

def emit_external_auth_check_failed(
    bus: EventBus, store: str, reason: str, detail: str = "",
) -> None:
    """Emit external auth check failed."""
    kwargs = {"store": store, "reason": reason}
    if detail:
        kwargs["detail"] = str(detail)[:64]
    _safe_emit(
        bus, "SECURITY_EXTERNAL_AUTH_CHECK_FAILED", **kwargs,
    )

def audit_auth_flow(store: str, method: str = "oauth") -> Callable:
    """Audit auth flow."""
    def decorator(target: Callable) -> Callable:
        """Decorator."""
        @functools.wraps(target)
        async def wrapper(self: Any, *args: Any, **kwargs: Any) -> Any:
            """Wrapper."""
            bus = getattr(self, "_bus", None)
            if bus is None:
                return await target(self, *args, **kwargs)
            emit_auth_started(bus, store, method=method)
            t0 = time.monotonic()
            try:
                result = await target(self, *args, **kwargs)
            except Exception as e:

```

```

        _emit_flow_outcome(
            bus, store, method, False,
            time.monotonic() - t0,
            type(e).__name__,
        )
        raise
    _emit_flow_outcome(
        bus, store, method,
        bool(getattr(result, "success", False)),
        time.monotonic() - t0,
        _extract_failure_reason(result),
    )
    return result
return wrapper
return decorator
def _emit_flow_outcome(
    bus: EventBus,
    store: str,
    method: str,
    success: bool,
    duration: float,
    failure_reason: str,
) -> None:
    """Emit flow outcome."""
    if success:
        emit_auth_completed(bus, store, duration, method=method)
    else:
        emit_auth_failed(
            bus, store, failure_reason, duration, method=method,
        )
def _extract_failure_reason(result: Any) -> str:
    """Extract failure reason."""
    for attr in ("error", "error_code", "reason", "message"):
        val = getattr(result, attr, None)
        if val:
            return str(val)[:64]
    return "unknown"

```


OP-23d

device_fingerprint.py

Fichier : py_modules/unifideck/security/device_fingerprint.py | Lignes : 119 | Depend de : (rien)

```
from __future__ import annotations
import hashlib
import json
import logging
import os
import time
from dataclasses import dataclass
from .device_identity import DeviceIdentity
logger = logging.getLogger(__name__)
_FORMAT_VERSION = 1
@dataclass(frozen=True)
class FingerprintState:
    """Fingerprint state."""
    machine_id_hash: str
    first_seen: float
    last_verified: float
    is_new: bool = False
    mismatch: bool = False
class DeviceFingerprint:
    """Device fingerprint."""
    def __init__(
        self,
        path: str,
        device_identity: DeviceIdentity | None = None,
    ) -> None:
        """Initialize the instance."""
        self._path = os.path.expanduser(path)
        self._device_identity = device_identity or DeviceIdentity()
    def verify_or_initialize(self) -> FingerprintState:
        """Verify or initialize."""
        current_hash = self._compute_current_hash()
        stored = self._load()
        if stored is None:
            return self._initialize(current_hash)
        if stored.get("machine_id_hash") != current_hash:
            return FingerprintState(
                machine_id_hash=current_hash,
                first_seen=float(stored.get("first_seen", 0.0)),
                last_verified=float(stored.get("last_verified", 0.0)),
                is_new=False,
                mismatch=True,
            )
        now = time.time()
        self._save({
            "machine_id_hash": current_hash,
            "first_seen": float(stored.get("first_seen", now)),
            "last_verified": now,
            "version": _FORMAT_VERSION,
        })
        return FingerprintState(
            machine_id_hash=current_hash,
            first_seen=float(stored.get("first_seen", now)),
            last_verified=now,
            is_new=False,
            mismatch=False,
```

```

    )
def reinitialize(self) -> FingerprintState:
    """Reinitialize."""
    current_hash = self._compute_current_hash()
    return self._initialize(current_hash)
def _compute_current_hash(self) -> str:
    """Compute current hash."""
    mid = self._device_identity.read()
    digest = hashlib.sha256(mid.encode("utf-8")).hexdigest()
    return f"sha256:{digest}"

def _initialize(self, current_hash: str) -> FingerprintState:
    """Initialize."""
    now = time.time()
    payload = {
        "machine_id_hash": current_hash,
        "first_seen": now,
        "last_verified": now,
        "version": _FORMAT_VERSION,
    }
    self._save(payload)
    logger.info(
        "[DeviceFingerprint] initialized at %s", self._path,
    )
    return FingerprintState(
        machine_id_hash=current_hash,
        first_seen=now,
        last_verified=now,
        is_new=True,
        mismatch=False,
    )
def _load(self) -> dict | None:
    """Load."""
    if not os.path.isfile(self._path):
        return None
    try:
        with open(self._path, encoding="utf-8") as f:
            data = json.load(f)
    except (OSError, json.JSONDecodeError) as e:
        logger.warning(
            "[DeviceFingerprint] load failed: %s", e,
        )
        return None
    if not isinstance(data, dict):
        return None
    return data
def _save(self, payload: dict) -> None:
    """Save."""
    try:
        parent = os.path.dirname(self._path)
        if parent:
            os.makedirs(parent, exist_ok=True)
        tmp = self._path + ".tmp"
        fd = os.open(
            tmp,
            os.O_WRONLY | os.O_CREAT | os.O_TRUNC,
            0o600,
        )
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            json.dump(payload, f, indent=2)

```

```
        os.replace(tmp, self._path)
    except OSError as e:
        logger.warning(
            "[DeviceFingerprint] save failed: %s", e,
        )
```

OP-23e

audit_decorators.py

Fichier : py_modules/unifideck/security/audit_decorators.py | Lignes : 90 | Depend de : (rien)

```

from __future__ import annotations
import functools
import logging
import time
from collections.abc import Callable
from typing import Any
logger = logging.getLogger(__name__)
def audit_auth_flow(
    store: str,
    method: str = "oauth",
) -> Callable:
    """Audit auth flow."""
    def decorator(func: Callable) -> Callable:
        """Decorator."""
        @functools.wraps(func)
        async def wrapper(self: Any, *args: Any, **kwargs: Any) -> Any:
            """Wrapper."""
            bus = getattr(self, "_bus", None)
            started_at = time.time()
            _emit_audit(
                bus, "SECURITY_AUTH_FLOW_STARTED",
                store=store, method=method,
            )
            try:
                result = await func(self, *args, **kwargs)
            except Exception as exc:
                duration_ms = int((time.time() - started_at) * 1000)
                _emit_audit(
                    bus, "SECURITY_AUTH_FLOW_FAILED",
                    store=store, method=method,
                    reason=type(exc).__name__,
                    duration_ms=duration_ms,
                )
                raise
            duration_ms = int((time.time() - started_at) * 1000)
            _emit_audit(
                bus, "SECURITY_AUTH_FLOW_COMPLETED",
                store=store, method=method,
                duration_ms=duration_ms,
            )
            return result
        return wrapper
    return decorator
def audit_token_op(
    operation: str,
    store: str,
) -> Callable:
    """Audit token op."""
    def decorator(func: Callable) -> Callable:
        """Decorator."""
        @functools.wraps(func)
        async def wrapper(self: Any, *args: Any, **kwargs: Any) -> Any:
            """Wrapper."""
            bus = getattr(self, "_bus", None)
            result = await func(self, *args, **kwargs)

```

```
        if operation == "migrate" and isinstance(result, str):
            _maybe_emit_migration(bus, self, store, result)
        return result
    return wrapper
return decorator

def _emit_audit(bus: Any, event_name: str, **kwargs: Any) -> None:
    """Emit audit."""
    if bus is None:
        return
    try:
        from ..core.types.events import Events
        event = getattr(Events, event_name)
        bus.emit(event, **kwargs)
    except Exception as e:
        logger.debug(
            "[audit_decorators] failed to emit %s: %s",
            event_name, e,
        )

def _maybe_emit_migration(
    bus: Any, instance: Any, store: str, result_path: str,
) -> None:
    """Maybe emit migration."""
    if bus is None:
        return
    flag = getattr(instance, "_migration_occurred", False)
    if not flag:
        return
    _emit_audit(
        bus, "SECURITY_TOKEN_FILE_MIGRATED",
        store=store, new_path=result_path,
    )
    try:
        instance._migration_occurred = False
    except AttributeError:
        pass
```

OP-24

`__init__.py`

Fichier : `py_modules/unifideck/rpc/__init__.py` | Lignes : 8 | Depend de : (rien)

```
from .auto_wire import auto_wrap_rpc_methods
from .errors import RpcError
from .wrapper import rpc_wrapper
__all__ = [
    "RpcError",
    "rpc_wrapper",
    "auto_wrap_rpc_methods",
]
```

OP-25

`__init__.py`

Fichier : `py_modules/unifideck/rpc/handlers/__init__.py` | Lignes : 21 | Depend de : (rien)

```
from __future__ import annotations
from unifideck.rpc.handlers.action_handlers import ActionHandlers
from unifideck.rpc.handlers.base import RpcHandlerBase
from unifideck.rpc.handlers.download_handlers import DownloadHandlers
from unifideck.rpc.handlers.launch_handlers import LaunchHandlers
from unifideck.rpc.handlers.observability_handlers import (
    ObservabilityHandlers,
)
from unifideck.rpc.handlers.security_handlers import SecurityHandlers
from unifideck.rpc.handlers.store_handlers import StoreHandlers
from unifideck.rpc.handlers.ui_handlers import UIHandlers
__all__ = [
    "ActionHandlers",
    "DownloadHandlers",
    "LaunchHandlers",
    "ObservabilityHandlers",
    "RpcHandlerBase",
    "SecurityHandlers",
    "StoreHandlers",
    "UIHandlers",
]
```

OP-25a

base.py**Fichier :** py_modules/unifideck/rpc/handlers/base.py | **Lignes :** 43 | **Depend de :** (rien)

```
from __future__ import annotations
from typing import TYPE_CHECKING, TypeVar
from unifideck.rpc.wrapper import RpcError
if TYPE_CHECKING:
    from unifideck.config.config_manager import ConfigManager
    from unifideck.core.cache_manager import CacheManager
    from unifideck.core.sync_service import SyncService
    from unifideck.event_bus.event_bus import EventBus
    from unifideck.services.bootstrap import ServiceContainer
    from unifideck.stores.shared.store_registry import StoreRegistry
T = TypeVar("T")
class RpcHandlerBase:
    """Rpc handler base."""
    def __init__(
        self,
        bus: EventBus,
        registry: StoreRegistry,
        cache: CacheManager,
        config: ConfigManager,
        sync_service: SyncService,
        services: ServiceContainer,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._registry = registry
        self._cache = cache
        self._config = config
        self._sync = sync_service
        self._services = services
    @staticmethod
    def _require(svc: T | None, name: str) -> T:
        """Require."""
        if svc is None:
            raise RpcError("service_unavailable", service=name)
        return svc
    def handler_methods(self) -> list[str]:
        """Handler methods."""
        return [
            name for name in dir(self)
            if not name.startswith("_")
            and callable(getattr(self, name))
            and name != "handler_methods"
        ]
```


OP-25b

action.py

Fichier : py_modules/unifideck/rpc/handlers/action_handlers.py | Lignes : 86 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
from typing import Any, cast
from unifideck.actions.unifideck_uri import (
    SCOPE_FRONTEND,
    ParsedAction,
    parse_unifideck_uri,
)
from unifideck.rpc.handlers.base import RpcHandlerBase
from unifideck.rpc.wrapper import RpcError
class ActionHandlers(RpcHandlerBase):
    """Action handlers."""
    async def dispatch_unifideck_action(self, uri: str) -> Any:
        """Dispatch unifideck action."""
        action = parse_unifideck_uri(uri)
        if not action.valid:
            raise RpcError("invalid_uri", reason=action.error, uri=uri)
        if action.scope == SCOPE_FRONTEND:
            raise RpcError(
                "frontend_scope_verb",
                verb=action.verb,
                hint="frontend should handle settings/* locally",
            )
        handler = self._resolve_verb_handler(action.verb)
        return cast(dict, await handler(action))
    def _resolve_verb_handler(self, verb: str):
        """Resolve verb handler."""
        handlers = {
            "auth": self._handle_auth,
            "retry-sync": self._handle_retry_sync,
            "refresh-library": self._handle_refresh_library,
            "refresh-all-libraries": self._handle_refresh_all_libraries,
        }
        if verb not in handlers:
            raise RpcError(
                "unhandled_backend_verb",
                verb=verb,
                hint="add a handler in ActionHandlers",
            )
        return handlers[verb]
    async def _handle_auth(self, action: ParsedAction) -> Any:
        """Handle auth."""
        store = action.args[0]
        return cast(dict, await self._registry.auth_action(store, "start"))
    async def _handle_retry_sync(self, action: ParsedAction) -> Any:
        """Handle retry sync."""
        store, game_id, phase = action.args
        svc = self._services.cloudsave
        if svc is None:
            raise RpcError("service_unavailable", service="cloudsave")
        if phase == "sync_down":
            result = await svc.sync_down(store, game_id)
        elif phase == "sync_up":
            result = await svc.sync_up(store, game_id)
        else:

```

```
        raise RpcError(
            "invalid_phase",
            phase=phase,
            supported=["sync_down", "sync_up"],
        )
    return {
        "success": result.success,
        "error": result.error,
        "store": store,
        "game_id": game_id,
        "phase": phase,
    }

async def _handle_refresh_library(
    self, action: ParsedAction,
) -> Any:
    """Handle refresh library."""
    store = action.args[0]
    asyncio.create_task(
        self._sync.sync_single_store(store),
        name=f"refresh-library-{store}",
    )
    return {"success": True, "store": store, "status": "scheduled"}
async def _handle_refresh_all_libraries(
    self, _action: ParsedAction,
) -> Any:
    """Handle refresh all libraries."""
    asyncio.create_task(
        self._sync.sync_all(),
        name="refresh-all-libraries",
    )
    return {"success": True, "status": "scheduled"}
```

OP-25c

download.py

Fichier : py_modules/unifideck/rpc/handlers/download_handlers.py | Lignes : 18 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any, cast
from unifideck.rpc.handlers.base import RpcHandlerBase
class DownloadHandlers(RpcHandlerBase):
    """Download handlers."""
    async def cancel_download(
        self, store: str, game_id: str,
    ) -> Any:
        """Check whether download."""
        svc = self._require(self._services.download, "download")
        return cast(dict, await svc.cancel(store, game_id))
    async def get_download_queue(self) -> Any:
        """Get download queue."""
        svc = self._require(self._services.download, "download")
        return cast(dict, svc.get_queue())
    async def get_storage_locations(self) -> Any:
        """Get storage locations."""
        return []
```

OP-25d

launch.py

Fichier : py_modules/unifideck/rpc/handlers/launch_handlers.py | Lignes : 84 | Depend de : (rien)

```

from __future__ import annotations
from typing import Any, cast
from unifideck.rpc.handlers.base import RpcHandlerBase
from unifideck.rpc.wrapper import RpcError
class LaunchHandlers(RpcHandlerBase):
    """Launch handlers."""
    def _ensure_launch_history(self):
        """Ensure launch history."""
        svc = self._services.launch_history
        if svc is None:
            raise RpcError(
                "service_unavailable", service="launch_history",
            )
        return svc
    async def get_launch_failures(self, game_key: str) -> Any:
        """Get launch failures."""
        svc = self._ensure_launch_history()
        return {
            "failures": svc.get_recent_failures(game_key),
            "threshold": svc.threshold(),
            "window_seconds": svc.window_seconds(),
            "is_circuit_open": svc.is_circuit_open(game_key),
        }
    async def clear_launch_failures(self, game_key: str) -> Any:
        """Clear launch failures."""
        svc = self._ensure_launch_history()
        svc.clear_failures(game_key)
        return {"success": True, "game_key": game_key}
    async def arm_circuit_bypass(self, game_key: str) -> Any:
        """Arm circuit bypass."""
        svc = self._ensure_launch_history()
        svc.arm_bypass(game_key)
        return {"success": True, "game_key": game_key}
    async def get_launch_logs(
        self, launch_id: str, max_lines: int = 500,
    ) -> Any:
        """Get launch logs."""
        svc = self._ensure_launch_history()
        return cast(dict, svc.read_launch_log(launch_id, max_lines=max_lines))
    async def export_launch_logs(
        self, launch_id: str, dest_path: str = "",
    ) -> Any:
        """Export launch logs."""
        import time
        from unifideck.launcher.diagnostics.log_archive import export_launch_logs
        if not dest_path:
            export_root = self._config.get_str(
                "logs.export_path", "~/Downloads",
            )
            ts = time.strftime("%Y%m%d-%H%M%S")
            dest_path = (
                f"{export_root}/unifideck-launch-{launch_id}-{ts}.log"
            )
        return export_launch_logs(launch_id, dest_path, self._config)

```

```
async def list_save_folder(
    self,
    store: str,
    game_id: str,
    max_depth: int = 2,
    filter_substring: str = "",
) -> Any:

    """List save folder."""
    from unifideck.launcher.diagnostics.save_folder_inspector import (
        inspect_save_folder,
    )
    svc = self._services.cloudsave
    if svc is None:
        raise RpcError("service_unavailable", service="cloudsave")
    root = svc.get_local_save_dir(store, game_id)
    result = inspect_save_folder(
        root,
        max_depth=max_depth,
        filter_substring=filter_substring,
        max_files=500,
    )
    return {"store": store, "game_id": game_id, **result}
async def get_playtime(self, store: str, game_id: str) -> Any:
    """Get playtime."""
    svc = self._require(self._services.playtime, "playtime")
    return cast(dict, await svc.get_playtime(store, game_id))
async def get_all_playtimes(self) -> Any:
    """Get all playtimes."""
    svc = self._require(self._services.playtime, "playtime")
    return cast(dict, await svc.get_all_playtimes())
```

OP-25e

observability.py

Fichier : py_modules/unifideck/rpc/handlers/observability_handlers.py | Lignes : 137 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any, cast
from unifideck.rpc.handlers.base import RpcHandlerBase
from unifideck.rpc.wrapper import RpcError
if TYPE_CHECKING:
    from unifideck.event_bus.priority_dispatcher import PriorityDispatcher
    from unifideck.event_bus.supervision.watchdog_handler import HandlerWatchdog
logger = logging.getLogger(__name__)
class ObservabilityHandlers(RpcHandlerBase):
    """Observability handlers."""
    dispatcher: PriorityDispatcher | None = None
    watchdog: HandlerWatchdog | None = None
    latency: Any = None
    replay: Any = None
    runtime_probes: list[dict] | None = None
    def set_bus_collaborators(
        self,
        *,
        dispatcher: PriorityDispatcher | None = None,
        watchdog: HandlerWatchdog | None = None,
        latency: Any = None,
        replay: Any = None,
    ) -> None:
        """Set bus collaborators."""
        self.dispatcher = dispatcher
        self.watchdog = watchdog
        self.latency = latency
        self.replay = replay
    async def get_plugin_metrics(self) -> Any:
        """Get plugin metrics."""
        metrics = self._require(self._services.metrics, "metrics")
        return cast(dict, metrics.get_plugin_metrics())
    async def get_bus_health(self) -> Any:
        """Get bus health."""
        health: dict[str, Any] = {}
        health_fn = getattr(self._bus, "health", None)
        if callable(health_fn):
            health = health_fn()
        self._collect_dispatcher_metrics(health)
        self._collect_watchdog_metrics(health)
        self._collect_latency_metrics(health)
        self._collect_misc_metrics(health)
        return cast(dict, health)
    def _collect_dispatcher_metrics(self, health: dict) -> None:
        """Collect dispatcher metrics."""
        if self.dispatcher is None:
            return
        m = self.dispatcher.get_metrics()
        health["dispatcher"] = {
            "emitted_total": m.emitted_total,
            "dispatched_total": m.dispatched_total,
            "coalesced_total": m.coalesced_total,
            "dropped_background_total": m.dropped_background_total,
            "pending_by_priority": m.pending_by_priority,

```

```

    }

def _collect_watchdog_metrics(self, health: dict) -> None:
    """Collect watchdog metrics."""
    if self.watchdog is None:
        return
    health["watchdog"] = {
        name: {
            "invocations": s.invocations,
            "timeouts": s.timeouts,
            "consecutive_timeouts": s.consecutive_timeouts,
            "quarantined": s.quarantined,
        }
        for name, s in self.watchdog.get_metrics().items()
    }

def _collect_latency_metrics(self, health: dict) -> None:
    """Collect latency metrics."""
    if self.latency is not None:
        health["latency_top10"] = self.latency.get_top_n(n=10)

def _collect_misc_metrics(self, health: dict) -> None:
    """Collect misc metrics."""
    if self.replay is not None:
        health["replay_size"] = self.replay.size()
    if self.runtime_probes is not None:
        health["runtime_probes"] = self.runtime_probes

async def subscribe_replay(self, events: list) -> Any:
    """Subscribe replay."""
    if self.replay is None:
        return []
    return cast(list, self.replay.snapshot(events=events))

async def release_quarantine(self, handler_name: str) -> Any:
    """Release quarantine."""
    if self.watchdog is None:
        return {"success": False, "error": "watchdog not wired"}
    released = self.watchdog.release_quarantine(handler_name)
    return {"success": released}

async def get_feature_flags(self) -> Any:
    """Get feature flags."""
    if self._services.feature_flags is None:
        raise RpcError(
            "service_unavailable", service="feature_flags",
        )
    return cast(dict, self._services.feature_flags.get_flags())

async def get_probe_history(self) -> Any:
    """Get probe history."""
    if self._services.probe_reaction is None:
        raise RpcError(
            "service_unavailable", service="probe_reaction",
        )
    return cast(list, self._services.probe_reaction.get_history())

async def report_runtime_probes(self, probes: list) -> Any:
    """Report runtime probes."""
    if not isinstance(probes, list):
        return {"ok": False, "error": "probes must be a list"}
    self.runtime_probes = probes
    self._log_probe_severities(probes)
    await self._emit_probes_event(probes)
    has_errors = any(p.get("severity") == "error" for p in probes)
    return {
        "ok": True, "count": len(probes), "has_errors": has_errors,
    }

```

```
}
def _log_probe_severities(self, probes: list) -> None:
    """Log probe severities."""
    for p in probes:
        name = p.get("name", "?")
        severity = p.get("severity", "?")
        message = p.get("message", "")
        line = f"[runtime-probe] {name}: {severity} - {message}"
        if severity == "error":
            logger.error(line)
        elif severity == "warning":
            logger.warning(line)
        else:
            logger.info(line)
    async def _emit_probes_event(self, probes: list) -> None:
        """Emit probes event."""
        try:
            from unifideck.core.types import Events
            await self._bus.emit(
                Events.RUNTIME_PROBES_REPORTED, probes=probes,
            )
        except Exception:
            logger.debug("[runtime-probe] bus emit failed")
```


OP-25f

security.py

Fichier : py_modules/unifideck/rpc/handlers/security_handlers.py | Lignes : 29 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any, cast
from unifideck.rpc.handlers.base import RpcHandlerBase
class SecurityHandlers(RpcHandlerBase):
    """Security handlers."""
    async def get_security_audit_log(
        self, limit: int = 100,
    ) -> Any:
        """Get security audit log."""
        svc = self._require(self._services.security, "security")
        return cast(list, svc.get_audit_log(limit=limit))
    async def get_security_counters(self) -> Any:
        """Get security counters."""
        svc = self._require(self._services.security, "security")
        return cast(dict, svc.get_counters())
    async def get_security_bruteforce_status(self) -> Any:
        """Get security bruteforce status."""
        svc = self._require(self._services.security, "security")
        return cast(dict, svc.get_bruteforce_status())
    async def clear_security_audit_log(self) -> Any:
        """Clear security audit log."""
        svc = self._require(self._services.security, "security")
        svc.clear_audit_log()
        return {"success": True}
    async def reset_security_bruteforce(self) -> Any:
        """Reset security bruteforce."""
        svc = self._require(self._services.security, "security")
        svc.reset_bruteforce_state()
        return {"success": True}
```

OP-25g

store.py

Fichier : py_modules/unifideck/rpc/handlers/store_handlers.py | Lignes : 63 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any, cast
from unifideck.rpc.handlers.base import RpcHandlerBase
from unifideck.rpc.wrapper import RpcError
class StoreHandlers(RpcHandlerBase):
    """Store handlers."""
    async def store_auth(
        self, store: str, action: str, **kw: Any,
    ) -> Any:
        """Store auth."""
        return cast(dict, await self._registry.auth_action(store, action, **kw))
    async def check_store_status(self) -> Any:
        """Check store status."""
        return cast(list, await self._registry.check_all_status())
    async def get_store_infos(self) -> Any:
        """Get store infos."""
        return cast(list, self._registry.get_store_infos())
    async def clear_store_auths(self) -> Any:
        """Clear store auths."""
        return cast(dict, await self._registry.logout_all())
    async def sync_libraries(self, **kw: Any) -> Any:
        """Sync libraries."""
        return cast(dict, await self._sync.sync_all(**kw))
    async def force_sync_libraries(self, **kw: Any) -> Any:
        """Force sync libraries."""
        return cast(dict, await self._sync.sync_all(force=True, **kw))
    async def get_sync_status(self) -> Any:
        """Get sync status."""
        return cast(dict, self._sync.get_status())
    async def get_sync_progress(self) -> Any:
        """Get sync progress."""
        return cast(dict, self._sync.get_status())
    async def cancel_sync(self) -> Any:
        """Check whether sync."""
        return cast(dict, await self._sync.cancel())
    async def get_all_unifideck_games(self) -> Any:
        """Get all unifideck games."""
        return cast(list, self._sync.get_all_games())
    async def get_game_info(self, app_id: int) -> Any:
        """Get game info."""
        return cast(dict, self._sync.get_game_info(app_id))
    async def install_game(
        self, store: str, game_id: str, **kw: Any,
    ) -> Any:
        """Install game."""
        s = self._registry.get_store(store)
        if s is None:
            raise RpcError("store_not_found", store=store)
        return cast(dict, await s.install_game(game_id, **kw))
    async def uninstall_game(self, store: str, game_id: str) -> Any:
        """Uninstall game."""
        s = self._registry.get_store(store)
        if s is None:
            raise RpcError("store_not_found", store=store)
        return cast(dict, await s.uninstall_game(game_id))
```

```
async def check_game_update(
    self, store: str, game_id: str,
) -> Any:
    """Check game update."""
    s = self._registry.get_store(store)
    if s is None:
        raise RpcError("store_not_found", store=store)
    return cast(dict, await s.check_game_update(game_id))
```

OP-25h

ui.py

Fichier : py_modules/unifideck/rpc/handlers/ui_handlers.py | Lignes : 110 | Depend de : (rien)

```

from __future__ import annotations
from typing import Any, ClassVar, cast
from unifideck.core.types import Events
from unifideck.rpc.handlers.base import RpcHandlerBase
from unifideck.rpc.wrapper import RpcError
class UIHandlers(RpcHandlerBase):
    """Uihandlers."""
    _CLOUD_FAILURE_STORES: ClassVar[tuple[str, ...]] = (
        "default", "epic", "gog", "amazon", "ubisoft",
    )
    _CLOUD_FAILURE_MODES: ClassVar[tuple[str, ...]] = ("silent", "toast")
    config_validation_result: Any = None
    async def notify_game_launched(
        self, store: str, game_id: str, **kw: Any,
    ) -> Any:
        """Notify game launched."""
        await self._bus.emit(
            Events.GAME_LAUNCHED, store=store, game_id=game_id, **kw,
        )
        return {"success": True}
    async def notify_game_stopped(
        self, store: str, game_id: str, exit_code: int = 0,
    ) -> Any:
        """Notify game stopped."""
        await self._bus.emit(
            Events.GAME_STOPPED,
            store=store, game_id=game_id, exit_code=exit_code,
        )
        return {"success": True}
    async def hide_play_section(self, app_id: int) -> Any:
        """Hide play section."""
        from unifideck.cdp.cdp_inject import get_cdp_client
        injector = await get_cdp_client()
        return {"ok": await injector.hide_play_section(app_id)}
    async def unhide_play_section(self, app_id: int) -> Any:
        """Unhide play section."""
        from unifideck.cdp.cdp_inject import get_cdp_client
        injector = await get_cdp_client()
        return {"ok": await injector.show_play_section(app_id)}
    async def inject_hide_css(self, app_id: int, css: str) -> Any:
        """Inject hide CSS."""
        from unifideck.cdp.cdp_inject import get_cdp_client
        injector = await get_cdp_client()
        marker = f"hide-{app_id}"
        return {"ok": await injector.inject_css(css, marker)}
    async def get_game_metadata(
        self, store: str, game_id: str,
    ) -> Any:
        """Get game metadata."""
        metadata = self._require(self._services.metadata, "metadata")
        for game in self._sync.get_games_by_store(store):
            if game.store_game_id == game_id:
                return cast(dict, await metadata.enrich(game))
        return {}
    async def get_language_preference(self) -> Any:

```

```

    """Get language preference."""
    return {"locale": self._config.get("ui.locale")}
async def set_language_preference(self, locale: str) -> Any:
    """Set language preference."""
    self._config.set("ui.locale", locale)
    return {"success": True, "locale": locale}
async def get_cloud_failure_behaviors(self) -> Any:
    """Get cloud failure behaviors."""
    return {
        store: self._config.get_str(
            f"cloud.failure_behavior.{store}", "toast",
        )
        for store in self._CLOUD_FAILURE_STORES
    }

async def set_cloud_failure_behavior(
    self, store: str, value: str,
) -> Any:

    """Set cloud failure behavior."""
    if store not in self._CLOUD_FAILURE_STORES:
        raise RpcError(
            "unsupported_store",
            store=store,
            supported=list(self._CLOUD_FAILURE_STORES),
        )
    if value not in self._CLOUD_FAILURE_MODES:
        raise RpcError(
            "invalid_behavior",
            value=value,
            supported=list(self._CLOUD_FAILURE_MODES),
        )
    self._config.set(
        f"cloud.failure_behavior.{store}", value,
    )
    return {"success": True, "store": store, "value": value}
async def get_config_validation_status(self) -> Any:
    """Get config validation status."""
    result = self.config_validation_result
    if result is None:
        return {
            "degraded": False,
            "defaults_validated": True,
            "user_overrides_present": False,
            "error_count": 0,
            "errors": [],
        }
    return {
        "degraded": not result.success,
        "defaults_validated": result.defaults_validated,
        "user_overrides_present": result.user_overrides_present,
        "error_count": len(result.errors),
        "errors": [
            {"source": e.source, "path": e.path, "message": e.message}
            for e in result.errors[:20]
        ],
    }
}

```

OP-26

`__init__.py`

Fichier : `py_modules/unifideck/rpc/mixins/__init__.py` | Lignes : 25 | Depend de : (rien)

```
from __future__ import annotations
from .action import ActionRPCMixin
from .cloud_failure import CloudFailureRPCMixin
from .config_validation import ConfigValidationRPCMixin
from .download import DownloadRPCMixin
from .launch import LaunchRPCMixin
from .observability import ObservabilityRPCMixin
from .playtime import PlaytimeRPCMixin
from .security import SecurityRPCMixin
from .store import StoreRPCMixin
from .sync import SyncRPCMixin
from .ui import UIRPCMixin
__all__ = [
    "ActionRPCMixin",
    "CloudFailureRPCMixin",
    "ConfigValidationRPCMixin",
    "DownloadRPCMixin",
    "LaunchRPCMixin",
    "ObservabilityRPCMixin",
    "PlaytimeRPCMixin",
    "SecurityRPCMixin",
    "StoreRPCMixin",
    "SyncRPCMixin",
    "UIRPCMixin",
]
```

OP-26a

observability.py

Fichier : py_modules/unifideck/rpc/mixins/observability.py | Lignes : 120 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import Any
from unifideck.core.types import Events
from unifideck.rpc import RpcError
logger = logging.getLogger(__name__)
class ObservabilityRPCMixin:
    """Observability rpcmixin."""
    bus: Any
    services: Any
    dispatcher: Any
    watchdog: Any
    latency: Any
    replay: Any
    runtime_probes: Any
    async def get_plugin_metrics(self) -> Any:
        """Get plugin metrics."""
        return self.services.metrics.snapshot()
    async def get_bus_health(self) -> Any:
        """Get bus health."""
        health = self.bus.health()
        self._collect_dispatcher_metrics(health)
        self._collect_watchdog_metrics(health)
        self._collect_latency_metrics(health)
        self._collect_misc_metrics(health)
        return health
    async def subscribe_replay(self, events: list) -> Any:
        """Subscribe replay."""
        if not hasattr(self, "replay") or self.replay is None:
            return []
        return self.replay.snapshot(events=events)
    async def release_quarantine(self, handler_name: str) -> Any:
        """Release quarantine."""
        if not hasattr(self, "watchdog") or self.watchdog is None:
            return {"success": False, "error": "watchdog not wired"}
        released = self.watchdog.release_quarantine(handler_name)
        return {"success": released}
    async def get_feature_flags(self) -> Any:
        """Get feature flags."""
        if self.services.feature_flags is None:
            raise RpcError(
                "service_unavailable", service="feature_flags",
            )
        return self.services.feature_flags.get_flags()
    async def get_probe_history(self) -> Any:
        """Get probe history."""
        if self.services.probe_reaction is None:
            raise RpcError(
                "service_unavailable", service="probe_reaction",
            )
        return self.services.probe_reaction.get_history()

    async def report_runtime_probes(self, probes: list) -> Any:
        """Report runtime probes."""
```

```

    if not isinstance(probes, list):
        return {"ok": False, "error": "probes must be a list"}
    self.runtime_probes = probes
    for p in probes:
        _log_probe(p)
    try:
        await self.bus.emit(
            Events.RUNTIME_PROBES_REPORTED,
            probes=probes,
        )
    except Exception:
        logger.debug("[runtime-probe] bus emit failed")
    has_errors = any(
        p.get("severity") == "error" for p in probes
    )
    return {
        "ok": True,
        "count": len(probes),
        "has_errors": has_errors,
    }
def _collect_dispatcher_metrics(self, health: dict) -> None:
    """Collect dispatcher metrics."""
    if not hasattr(self, "dispatcher") or self.dispatcher is None:
        return
    m = self.dispatcher.get_metrics()
    health["dispatcher"] = {
        "emitted_total": m.emitted_total,
        "dispatched_total": m.dispatched_total,
        "coalesced_total": m.coalesced_total,
        "dropped_background_total": m.dropped_background_total,
        "pending_by_priority": m.pending_by_priority,
    }
def _collect_watchdog_metrics(self, health: dict) -> None:
    """Collect watchdog metrics."""
    if not hasattr(self, "watchdog") or self.watchdog is None:
        return
    health["watchdog"] = {
        name: {
            "invocations": s.invocations,
            "timeouts": s.timeouts,
            "consecutive_timeouts": s.consecutive_timeouts,
            "quarantined": s.quarantined,
        }
        for name, s in self.watchdog.get_metrics().items()
    }
def _collect_latency_metrics(self, health: dict) -> None:
    """Collect latency metrics."""
    if hasattr(self, "latency") and self.latency is not None:
        health["latency_top10"] = self.latency.get_top_n(n=10)
def _collect_misc_metrics(self, health: dict) -> None:
    """Collect misc metrics."""
    if hasattr(self, "replay") and self.replay is not None:
        health["replay_size"] = self.replay.size()
    if hasattr(self, "runtime_probes"):
        health["runtime_probes"] = self.runtime_probes
def _log_probe(p: dict) -> None:
    """Log probe."""
    name = p.get("name", "?")
    severity = p.get("severity", "?")
    message = p.get("message", "")
    line = f"[runtime-probe] {name}: {severity} - {message}"

```



```
if severity == "error":  
    logger.error(line)  
elif severity == "warning":  
    logger.warning(line)  
else:  
    logger.info(line)
```

OP-26b

security.py

Fichier : py_modules/unifideck/rpc/mixins/security.py | Lignes : 37 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
from unifideck.rpc import RpcError
class SecurityRPCMixin:
    """Security rpcmixin."""
    services: Any
    async def get_security_audit_log(
        self, limit: int = 100,
    ) -> Any:
        """Get security audit log."""
        security = self._require_security()
        return security.get_audit_log(limit=limit)
    async def get_security_counters(self) -> Any:
        """Get security counters."""
        security = self._require_security()
        return security.get_counters()
    async def get_security_bruteforce_status(self) -> Any:
        """Get security bruteforce status."""
        security = self._require_security()
        return security.get_bruteforce_status()
    async def clear_security_audit_log(self) -> Any:
        """Clear security audit log."""
        security = self._require_security()
        security.clear_audit_log()
        return {"success": True}
    async def reset_security_bruteforce(self) -> Any:
        """Reset security bruteforce."""
        security = self._require_security()
        security.reset_bruteforce_state()
        return {"success": True}
    def _require_security(self) -> Any:
        """Require security."""
        if self.services.security is None:
            raise RpcError(
                "service_unavailable", service="security",
            )
        return self.services.security
```

OP-26c

download.py

Fichier : py_modules/unifideck/rpc/mixins/download.py | Lignes : 46 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
from unifideck.rpc import RpcError
class DownloadRPCMixin:
    """Download rpcmixin."""
    registry: Any
    services: Any
    async def install_game(
        self, store: str, game_id: str, **kw: Any,
    ) -> Any:
        """Install game."""
        s = self._require_store(store)
        return await s.install_game(game_id, **kw)
    async def uninstall_game(self, store: str, game_id: str) -> Any:
        """Uninstall game."""
        s = self._require_store(store)
        return await s.uninstall_game(game_id)
    async def check_game_update(self, store: str, game_id: str) -> Any:
        """Check game update."""
        s = self._require_store(store)
        return await s.check_game_update(game_id)
    async def cancel_download(self, download_id: str) -> Any:
        """Check whether download."""
        download = self._require_download()
        return await download.cancel(download_id)
    async def get_download_queue(self) -> Any:
        """Get download queue."""
        download = self._require_download()
        return download.get_queue()
    async def get_storage_locations(self) -> Any:
        """Get storage locations."""
        download = self._require_download()
        return download.get_storage_locations()
    def _require_store(self, store: str) -> Any:
        """Require store."""
        s = self.registry.get(store)
        if s is None:
            raise RpcError("store_not_found", store=store)
        return s
    def _require_download(self) -> Any:
        """Require download."""
        if self.services.download is None:
            raise RpcError(
                "service_unavailable", service="download",
            )
        return self.services.download
```

OP-26d

launch.py

Fichier : py_modules/unifideck/rpc/mixins/launch.py | Lignes : 99 | Depend de : (rien)

```

from __future__ import annotations
import time as _time
from typing import Any
from unifideck.core.types import Events
from unifideck.rpc import RpcError
class LaunchRPCMixin:
    """Launch rpcmixin."""
    bus: Any
    config: Any
    services: Any
    async def notify_game_launched(
        self, store: str, game_id: str, **kw: Any,
    ) -> Any:
        """Notify game launched."""
        await self.bus.emit(
            Events.GAME_LAUNCHED, store=store, game_id=game_id, **kw,
        )
        return {"success": True}
    async def notify_game_stopped(
        self, store: str, game_id: str, exit_code: int = 0,
    ) -> Any:
        """Notify game stopped."""
        await self.bus.emit(
            Events.GAME_STOPPED,
            store=store, game_id=game_id, exit_code=exit_code,
        )
        return {"success": True}
    async def get_launch_failures(self, game_key: str) -> Any:
        """Get launch failures."""
        svc = self._require_launch_history()
        return {
            "failures": svc.get_recent_failures(game_key),
            "threshold": svc.threshold(),
            "window_seconds": svc.window_seconds(),
            "is_circuit_open": svc.is_circuit_open(game_key),
        }
    async def clear_launch_failures(self, game_key: str) -> Any:
        """Clear launch failures."""
        svc = self._require_launch_history()
        svc.clear_failures(game_key)
        return {"success": True, "game_key": game_key}
    async def arm_circuit_bypass(self, game_key: str) -> Any:
        """Arm circuit bypass."""
        svc = self._require_launch_history()
        svc.arm_bypass(game_key)
        return {"success": True, "game_key": game_key}
    async def get_launch_logs(
        self, launch_id: str, max_lines: int = 500,
    ) -> Any:
        """Get launch logs."""
        from unifideck.launcher.diagnostics.log_archive import read_launch_logs
        return read_launch_logs(
            launch_id, self.config, max_lines=max_lines,
        )
    async def export_launch_logs(

```

```

        self, launch_id: str, dest_path: str = "",
    ) -> Any:
        """Export launch logs."""
        from unifideck.launcher.diagnostics.log_archive import export_launch_logs
        if not dest_path:
            export_root = (
                self.config.get_str("logs.export_path", "~/Downloads")
                if self.config else "~/Downloads"
            )
            ts = _time.strftime("%Y%m%d-%H%M%S")
            dest_path = (
                f"{export_root}/unifideck-launch-{launch_id}-{ts}.log"
            )
        return export_launch_logs(launch_id, dest_path, self.config)

    async def list_save_folder(
        self, store: str, game_id: str,
        max_depth: int = 2,
        filter_substring: str = "",
    ) -> Any:
        """List save folder."""
        svc = self.services.cloudsave
        if svc is None:
            raise RpcError(
                "service_unavailable", service="cloudsave",
            )
        root = svc.get_local_save_dir(store, game_id)
        from unifideck.launcher.diagnostics.save_folder_inspector import (
            inspect_save_folder,
        )
        result = inspect_save_folder(
            root,
            max_depth=max_depth,
            filter_substring=filter_substring,
            max_files=500,
        )
        return {"store": store, "game_id": game_id, **result}

    def _require_launch_history(self) -> Any:
        """Require launch history."""
        svc = self.services.launch_history
        if svc is None:
            raise RpcError(
                "service_unavailable", service="launch_history",
            )
        return svc

```

OP-26e

store.py

Fichier : py_modules/unifideck/rpc/mixins/store.py | Lignes : 19 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
class StoreRPCMixin:
    """Store rpcmixin."""
    registry: Any
    async def store_auth(
        self, store: str, action: str, **kw: Any,
    ) -> Any:
        """Store auth."""
        return await self.registry.auth_action(store, action, **kw)
    async def check_store_status(self) -> Any:
        """Check store status."""
        return await self.registry.check_all_status()
    async def get_store_infos(self) -> Any:
        """Get store infos."""
        return self.registry.get_store_infos()
    async def clear_store_auths(self) -> Any:
        """Clear store auths."""
        return await self.registry.logout_all()
```

OP-26f

sync.py**Fichier :** py_modules/unifideck/rpc/mixins/sync.py | Lignes : 26 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
class SyncRPCMixin:
    """Sync rpcmixin."""
    sync_service: Any
    async def sync_libraries(self, **kw: Any) -> Any:
        """Sync libraries."""
        return await self.sync_service.sync(**kw)
    async def force_sync_libraries(self, **kw: Any) -> Any:
        """Force sync libraries."""
        return await self.sync_service.sync(force=True, **kw)
    async def get_sync_status(self) -> Any:
        """Get sync status."""
        return self.sync_service.get_status()
    async def get_sync_progress(self) -> Any:
        """Get sync progress."""
        return self.sync_service.get_progress()
    async def cancel_sync(self) -> Any:
        """Check whether sync."""
        return await self.sync_service.cancel()
    async def get_all_unifideck_games(self) -> Any:
        """Get all unifideck games."""
        return await self.sync_service.get_all_games()
    async def get_game_info(self, app_id: int) -> Any:
        """Get game info."""
        return await self.sync_service.get_game_info(app_id)
```

OP-26g

ui.py

Fichier : py_modules/unifideck/rpc/mixins/ui.py | Lignes : 25 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
class UIRPCMixin:
    """Uirpcmixin."""
    config: Any
    services: Any
    async def get_game_metadata(self, store: str, game_id: str) -> Any:
        """Get game metadata."""
        return await self.services.metadata.get(store, game_id)
    async def hide_play_section(self, app_id: int) -> Any:
        """Hide play section."""
        return await self.services.cdp.hide_play_section(app_id)
    async def unhide_play_section(self, app_id: int) -> Any:
        """Unhide play section."""
        return await self.services.cdp.unhide_play_section(app_id)
    async def inject_hide_css(self, app_id: int, css: str) -> Any:
        """Inject hide CSS."""
        return await self.services.cdp.inject_css(app_id, css)
    async def get_language_preference(self) -> Any:
        """Get language preference."""
        return {"locale": self.config.get("ui.locale", "en-US")}
    async def set_language_preference(self, locale: str) -> Any:
        """Set language preference."""
        self.config.set("ui.locale", locale)
        return {"success": True, "locale": locale}
```


OP-26h

cloud_failure.py

Fichier : py_modules/unifideck/rpc/mixins/cloud_failure.py | Lignes : 38 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
from unifideck.rpc import RpcError
class CloudFailureRPCMixin:
    """Cloud failure rpcmixin."""
    config: Any
    _CLOUD_FAILURE_STORES: tuple[str, ...] = (
        "default", "epic", "gog", "amazon", "ubisoft",
    )
    _CLOUD_FAILURE_MODES: tuple[str, ...] = ("silent", "toast")
    async def get_cloud_failure_behaviors(self) -> Any:
        """Get cloud failure behaviors."""
        result = {}
        for store in self._CLOUD_FAILURE_STORES:
            result[store] = self.config.get_str(
                f"cloud.failure_behavior.{store}", "toast",
            )
        return result
    async def set_cloud_failure_behavior(
        self, store: str, value: str,
    ) -> Any:
        """Set cloud failure behavior."""
        if store not in self._CLOUD_FAILURE_STORES:
            raise RpcError(
                "unsupported_store",
                store=store,
                supported=list(self._CLOUD_FAILURE_STORES),
            )
        if value not in self._CLOUD_FAILURE_MODES:
            raise RpcError(
                "invalid_behavior",
                value=value,
                supported=list(self._CLOUD_FAILURE_MODES),
            )
        self.config.set(
            f"cloud.failure_behavior.{store}", value,
        )
        return {"success": True, "store": store, "value": value}
```

OP-26i

config_validation.py

Fichier : py_modules/unifideck/rpc/mixins/config_validation.py | Lignes : 29 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
class ConfigValidationRPCMixin:
    """Config validation rpcmixin."""
    async def get_config_validation_status(self) -> Any:
        """Get config validation status."""
        result = getattr(self, "_config_validation_result", None)
        if result is None:
            return {
                "degraded": False,
                "defaults_validated": True,
                "user_overrides_present": False,
                "error_count": 0,
                "errors": [],
            }
        return {
            "degraded": not result.success,
            "defaults_validated": result.defaults_validated,
            "user_overrides_present": result.user_overrides_present,
            "error_count": len(result.errors),
            "errors": [
                {
                    "source": e.source,
                    "path": e.path,
                    "message": e.message,
                }
                for e in result.errors[:20]
            ],
        }
```

OP-26j

playtime.py

Fichier : py_modules/unifideck/rpc/mixins/playtime.py | Lignes : 11 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
class PlaytimeRPCMixin:
    """Playtime rpcmixin."""
    services: Any
    async def get_playtime(self, store: str, game_id: str) -> Any:
        """Get playtime."""
        return await self.services.playtime.get(store, game_id)
    async def get_all_playtimes(self) -> Any:
        """Get all playtimes."""
        return await self.services.playtime.get_all()
```

OP-26k

action.py

Fichier : py_modules/unifideck/rpc/mixins/action.py | Lignes : 15 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
class ActionRPCMixin:
    """Action rpcmixin."""
    registry: Any
    services: Any
    async def dispatch_unifideck_action(self, uri: str) -> Any:
        """Dispatch unifideck action."""
        from unifideck.actions.dispatch import dispatch_backend_action
        return await dispatch_backend_action(
            uri=uri,
            registry=self.registry,
            cloudsave=self.services.cloudsave,
            sync_service=getattr(self, "sync_service", None),
        )
```

OP-24a

errors.py**Fichier :** py_modules/unifideck/rpc/errors.py | Lignes : 11 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
class RpcError(Exception):
    """Rpc error."""
    def __init__(
        self, code: str, message: str = "", **context: Any,
    ) -> None:
        """Initialize the instance."""
        super().__init__(message or code)
        self.code = code
        self.context: dict[str, Any] = dict(context)
```

OP-24b

wrapper.py

Fichier : py_modules/unifideck/rpc/wrapper.py | Lignes : 73 | Depend de : (rien)

```

from __future__ import annotations
import functools
import inspect
import logging
from collections.abc import Awaitable, Callable
from dataclasses import asdict, is_dataclass
from typing import Any, TypeVar
from .errors import RpcError

logger = logging.getLogger(__name__)
F = TypeVar("F", bound=Callable[..., Awaitable[Any]])

def _serialize(value: Any) -> Any:
    """Serialize."""
    if is_dataclass(value) and not isinstance(value, type):
        return asdict(value)
    if isinstance(value, list):
        return [_serialize(v) for v in value]
    if isinstance(value, tuple):
        return [_serialize(v) for v in value]
    if isinstance(value, dict):
        return {k: _serialize(v) for k, v in value.items()}
    return value

def _to_envelope(value: Any) -> dict[str, Any]:
    """To envelope."""
    if isinstance(value, dict) and "success" in value:
        success = bool(value["success"])
        error = value.get("error")
        if "data" in value:
            data = value["data"]
        else:
            data = {
                k: v for k, v in value.items()
                if k not in ("success", "error")
            }
        if not data:
            data = None
    return {
        "success": success,
        "error": error,
        "data": _serialize(data),
    }

def rpc_wrapper(func: F) -> F:
    """Rpc wrapper."""
    if getattr(func, "__rpc_wrapped__", False):
        return func
    if not inspect.iscoroutinefunction(func):
        raise TypeError(
            f"@rpc_wrapper requires an async function, "
            f"{func.__qualname__} is not a coroutine"
        )
    @functools.wraps(func)
    async def wrapper(*args: Any, **kwargs: Any) -> dict[str, Any]:
        """Wrapper."""

```

```
try:
    result = await func(*args, **kwargs)
    return _to_envelope(result)
except RpcError as err:
    return {
        "success": False,
        "error": err.code,
        "data": err.context or None,
    }
except Exception as err:
    logger.exception(
        "[rpc] uncaught exception in %s", func.__qualname__,
    )
    return {
        "success": False,
        "error": "internal_error",
        "data": {"detail": repr(err)},
    }
wrapper.__rpc_wrapped__ = True
return wrapper
```

OP-24c

auto_wire.py

Fichier : py_modules/unifideck/rpc/auto_wire.py | Lignes : 17 | Depend de : (rien)

```
from __future__ import annotations
import inspect
from .wrapper import rpc_wrapper
def auto_wrap_rpc_methods(cls: type) -> type:
    """Auto wrap RPC methods."""
    already_wrapped: set[str] = set()
    for klass in cls.__mro__:
        if klass is object:
            continue
        for name, attr in list(klass.__dict__.items()):
            if name.startswith("_") or name in already_wrapped:
                continue
            if not inspect.iscoroutinefunction(attr):
                continue
            setattr(cls, name, rpc_wrapper(attr))
            already_wrapped.add(name)
    return cls
```


OP-24d

composer.py

Fichier : py_modules/unifideck/rpc/composer.py | Lignes : 25 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from unifideck.rpc.handlers.base import RpcHandlerBase
logger = logging.getLogger(__name__)
def bind_handlers(plugin: Any, *groups: RpcHandlerBase) -> list[str]:
    """Bind handlers."""
    seen: dict[str, str] = {}
    for group in groups:
        group_name = type(group).__name__
        for name in group.handler_methods():
            if name in seen:
                raise ValueError(
                    f"RPC name collision: '{name}' defined by both "
                    f"{seen[name]} and {group_name}",
                )
            seen[name] = group_name
            setattr(plugin, name, getattr(group, name))
    bound = sorted(seen.keys())
    logger.info(
        "[rpc.composer] bound %d RPC methods from %d groups",
        len(bound), len(groups),
    )
    return bound
```

OP-27

__init__.py

Fichier : py_modules/unifideck/actions/__init__.py | Lignes : 0 | Depend de : (rien)

OP-27a

unifideck_uri.py

Fichier : py_modules/unifideck/actions/unifideck_uri.py | Lignes : 105 | Depend de : (rien)

```

from __future__ import annotations
import logging
from dataclasses import dataclass, field
from urllib.parse import parse_qs, urlparse
logger = logging.getLogger(__name__)
SCOPE_BACKEND = "backend"
SCOPE_FRONTEND = "frontend"
@dataclass(frozen=True)
class ParsedAction:
    """Parsed action."""
    valid: bool
    verb: str = ""
    scope: str = ""
    args: tuple[str, ...] = ()
    query: dict[str, str] = field(default_factory=dict)
    error: str = ""

_VERB_REGISTRY: dict[str, tuple[str, int, int, str]] = {
    "auth": (
        SCOPE_BACKEND, 1, 1,
        "Start OAuth flow for a store. Args: <store>.",
    ),
    "retry-sync": (
        SCOPE_BACKEND, 3, 3,
        "Retry a cloud save sync. Args: <store> <game_id> <phase>. "
        "Phase is 'sync_down' or 'sync_up'.",
    ),
    "refresh-library": (
        SCOPE_BACKEND, 1, 1,
        "Refresh the game library for a single store. Args: "
        "<store>. Fire-and-forget: the RPC returns immediately "
        "and the sync runs in the background; the UI Library "
        "view picks up the result through its own SYNC_PROGRESS "
        "event subscription.",
    ),
    "refresh-all-libraries": (
        SCOPE_BACKEND, 0, 0,
        "Refresh every registered store's library. Args: none. "
        "Fire-and-forget same as refresh-library, but drives "
        "SyncService.sync_all() across all stores in parallel. "
        "Wired to the 'Refresh all libraries' button in the "
        "UnifideckSettingsPanel.",
    ),
    "open-save-folder": (
        SCOPE_FRONTEND, 2, 2,
        "Open the SaveFolderModal for a game. Args: <store> "
        "<game_id>. Frontend-only: the listener component "
        "opens the modal via Decky's showModal helper; the "
        "modal itself calls the backend list_save_folder RPC "
        "to populate its data.",
    ),
    "show-logs": (
        SCOPE_FRONTEND, 1, 1,
        "Open the LaunchLogsModal for a past launch. Args: "
        "<launch_id>. Frontend-only: the listener component "

```

```

        "opens the modal, which fetches logs via the backend "
        "get_launch_logs RPC. Used by launcher-failure toast "
        "actions on errorCircuitBreakerOpen and generic "
        "LAUNCHER_ERROR codes.",
    ),
    "settings": (
        SCOPE_FRONTEND, 1, 2,
        "Navigate to a settings section. Args: <section> "
        "[<focus_target>]. Frontend-only.",
    ),
}

def list_supported_verbs() -> list[str]:
    """List supported verbs."""
    return sorted(_VERB_REGISTRY.keys())

def parse_unifideck_uri(uri: str) -> ParsedAction:
    """Parse unifideck URI."""
    if not uri:
        return ParsedAction(valid=False, error="empty_uri")
    try:
        parsed = urlparse(uri)
    except Exception as err:
        return ParsedAction(valid=False, error=f"parse_error:{err}")
    if parsed.scheme != "unifideck":
        return ParsedAction(
            valid=False,
            error=f"wrong_scheme:{parsed.scheme}",
        )
    verb = parsed.netloc
    if not verb:
        return ParsedAction(valid=False, error="missing_verb")
    if verb not in _VERB_REGISTRY:
        return ParsedAction(
            valid=False, verb=verb,
            error=f"unknown_verb:{verb}",
        )
    scope, min_args, max_args, _doc = _VERB_REGISTRY[verb]
    raw_path = parsed.path.lstrip("/")
    args = tuple(p for p in raw_path.split("/") if p) if raw_path else ()
    if not (min_args <= len(args) <= max_args):
        return ParsedAction(
            valid=False, verb=verb, scope=scope,
            error=(
                f"wrong_arg_count:got_{len(args)}_"
                f"expected_{min_args}_to_{max_args}"
            ),
        )
    raw_query = parse_qs(parsed.query) if parsed.query else {}
    query = {k: v[0] for k, v in raw_query.items() if v}
    return ParsedAction(
        valid=True, verb=verb, scope=scope, args=args, query=query,
    )

```

OP-27b

dispatch.py

Fichier : py_modules/unifideck/actions/dispatch.py | Lignes : 97 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import TYPE_CHECKING, Any
from unifideck.actions.unifideck_uri import (
    SCOPE_FRONTEND,
    parse_unifideck_uri,
)
from unifideck.rpc import RpcError
if TYPE_CHECKING:
    from unifideck.core.sync_service import SyncService
    from unifideck.services.cloud_save import CloudSaveService
    from unifideck.stores import StoreRegistry
logger = logging.getLogger(__name__)
async def dispatch_backend_action(
    *,
    uri: str,
    registry: StoreRegistry,
    cloudsave: CloudSaveService | None,
    sync_service: SyncService | None,
) -> Any:
    """Dispatch backend action."""
    action = parse_unifideck_uri(uri)
    if not action.valid:
        raise RpcError("invalid_uri", reason=action.error, uri=uri)
    if action.scope == SCOPE_FRONTEND:
        raise RpcError(
            "frontend_scope_verb",
            verb=action.verb,
            hint="frontend should handle settings/* locally",
        )
    if action.verb == "auth":
        return await _dispatch_auth(action, registry)
    if action.verb == "retry-sync":
        return await _dispatch_retry_sync(action, cloudsave)
    if action.verb == "refresh-library":
        return _dispatch_refresh_library(action, sync_service)
    if action.verb == "refresh-all-libraries":
        return _dispatch_refresh_all(sync_service)
    raise RpcError(
        "unhandled_backend_verb",
        verb=action.verb,
        hint="add a handler in dispatch_backend_action",
    )
async def _dispatch_auth(action: Any, registry: StoreRegistry) -> Any:
    """Dispatch auth."""
    store = action.args[0]
    return await registry.auth_action(store, "start")
async def _dispatch_retry_sync(
    action: Any,
    cloudsave: CloudSaveService | None,
) -> dict[str, Any]:
    """Dispatch retry sync."""

```

```
if cloudsaver is None:
    raise RpcError("service_unavailable", service="cloudsave")
store, game_id, phase = action.args
if phase == "sync_down":
    result = await cloudsaver.sync_down(store, game_id)
elif phase == "sync_up":
    result = await cloudsaver.sync_up(store, game_id)
else:
    raise RpcError(
        "invalid_phase",
        phase=phase,
        supported=["sync_down", "sync_up"],
    )
return {
    "success": result.success,
    "error": result.error,
    "store": store,
    "game_id": game_id,
    "phase": phase,
}

def _dispatch_refresh_library(
    action: Any,
    sync_service: SyncService | None,
) -> dict[str, Any]:
    """Dispatch refresh library."""
    if sync_service is None:
        raise RpcError("service_unavailable", service="sync_service")
    store = action.args[0]
    asyncio.create_task(
        sync_service.sync_single_store(store),
        name=f"refresh-library-{store}",
    )
    return {"success": True, "store": store, "status": "scheduled"}

def _dispatch_refresh_all(
    action: Any,
    sync_service: SyncService | None,
) -> dict[str, Any]:
    """Dispatch refresh all."""
    if sync_service is None:
        raise RpcError("service_unavailable", service="sync_service")
    asyncio.create_task(
        sync_service.sync_all(),
        name="refresh-all-libraries",
    )
    return {"success": True, "status": "scheduled"}
```

OP-28

`__init__.py`

Fichier : `py_modules/unifideck/auth/__init__.py` | Lignes : 1 | Depend de : (rien)

```
from .browser import CDPOAuthMonitor, OAuthBrowserMonitor
```

OP-29

`__init__.py`

Fichier : `py_modules/unifideck/auth/edge_browser/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from __future__ import annotations
from .edge import EdgeBrowser
```


OP-29a

edge.py

Fichier : py_modules/unifideck/auth/edge_browser/edge.py | Lignes : 169 | Depend de : (rien)

```
from __future__ import annotations
import logging
import subprocess
from collections.abc import Callable
from pathlib import Path
from typing import Any
from . import launch as _launch, process_ops
from .cdp_client import EdgeCDPClient
from .env import clean_env
from .installer import EdgeInstaller
from .profile import EdgeProfileManager
logger = logging.getLogger(__name__)
PROFILE_DIR = str(Path(
    "~/local/share/unifideck/edge-auth",
).expanduser())
LOG_FILE = str(Path(
    "~/local/share/unifideck/edge-auth.log",
).expanduser())
_LEGACY_PROFILE_DIR = str(Path(
    "~/local/share/unifideck/chromium-auth",
).expanduser())
_LEGACY_LOG_FILE = str(Path(
    "~/local/share/unifideck/chromium-auth.log",
).expanduser())
_MS_COOKIE_DOMAINS = (
    "%xbox.com%", "%microsoft.com%", "%live.com%", "%microsoftonline.com%",
)
_BASE_FLAGS = [
    "--no-first-run",
    "--disable-translate",
    "--disable-infobars",
    "--disable-session-crashed-bubble",
    "--disable-features=TranslateUI",
    "--password-store=basic",
    "--disable-dev-shm-usage",
    "--disable-background-networking",
]
def _make_profile_manager() -> EdgeProfileManager:
    """Make profile manager."""
    return EdgeProfileManager(
        profile_dir=PROFILE_DIR,
        log_file=LOG_FILE,
        legacy_profile_dir=_LEGACY_PROFILE_DIR,
        legacy_log_file=_LEGACY_LOG_FILE,
        cookie_domain_patterns=_MS_COOKIE_DOMAINS,
    )
class EdgeBrowser:
    """Edge browser."""
    def __init__(
        self,
        cdp_port: int = 9222,
        locale_fn: Callable[[], str] | None = None,
    ):
        """Initialize the instance."""
        self.cdp_port = cdp_port
```

```

self.locale_fn = locale_fn or (lambda: "en-US")
self.process: subprocess.Popen | None = None
self._installer = EdgeInstaller(clean_env_fn=clean_env)
self._cdp = EdgeCDPClient(cdp_port=cdp_port)
self._profile = _make_profile_manager()
self._migrate_legacy_profile()

def _migrate_legacy_profile(self) -> None:

    """Migrate legacy profile."""
    self._profile.migrate_legacy_profile()
def is_running(self) -> bool:
    """Check whether running."""
    if self.process is not None and self.process.poll() is None:
        return True
    return self._get_browser_ws_url() is not None
def _singleton_paths(self) -> list[str]:
    """Singleton paths."""
    return self._profile._singleton_paths()
def _has_stale_singleton_socket(self) -> bool:
    """Has stale singleton socket."""
    return self._profile._has_stale_singleton_socket()
def cleanup_stale_profile_state(self) -> None:
    """Cleanup stale profile state."""
    self._profile.cleanup_stale_state()
def _get_browser_ws_url(self) -> str | None:
    """Get browser ws URL."""
    return self._cdp.get_browser_ws_url()
def _list_cdp_targets(self) -> list[dict[str, Any]]:
    """List CDP targets."""
    return self._cdp.list_targets()
async def navigate_tab(
    self,
    url: str,
    timeout: float = 15.0,
) -> bool:
    """Navigate tab."""
    return await self._cdp.navigate_tab(url, timeout=timeout)
async def _close_all_cdp_targets(
    self, *, log_prefix: str,
) -> bool:
    """Close all CDP targets."""
    return await self._cdp.close_all_targets(log_prefix=log_prefix)
async def prepare_auth_launch(self) -> None:
    """Prepare auth launch."""
    await self._close_all_cdp_targets(log_prefix="lingering auth")
    self.cleanup_stale_profile_state()
async def close_auth_browser(self) -> bool:
    """Close auth browser."""
    closed = await self._close_all_cdp_targets(log_prefix="auth")
    if closed:
        self.cleanup_stale_profile_state()
    return closed
@staticmethod
def ensure_controller_permissions() -> bool:
    """Ensure controller permissions."""
    return EdgeInstaller(
        clean_env_fn=clean_env,
    ).ensure_controller_permissions()
def _flatpak_remote_names(self, scope: str) -> set[str]:
    """Flatpak remote names."""

```

```

        return self._installer._flatpak_remote_names(scope)
    async def _ensure_user_flathub_remote(self) -> bool:
        """Ensure user flathub remote."""
        return await self._installer._ensure_user_flathub_remote()
    def find_cmd(self) -> list[str] | None:
        """Find cmd."""
        return self._installer.find_cmd()
    @property
    def is_installed(self) -> bool:
        """Check whether installed."""
        return self._installer.is_installed
    @staticmethod
    def _get_default_browser() -> str | None:
        """Get default browser."""
        return EdgeInstaller(clean_env_fn=clean_env)._get_default_browser()
    @staticmethod
    def _restore_default_browser(original: str | None) -> None:
        """Restore default browser."""
        EdgeInstaller(
            clean_env_fn=clean_env,
        )._restore_default_browser(original)
    async def install(self) -> dict[str, Any]:
        """Install."""
        return await self._installer.install()
    def launch_auth(self, auth_url: str) -> bool:
        """Launch auth."""
        return _launch.launch_auth(self, auth_url)
    def launch_xcloud(self, xcloud_url: str) -> bool:
        """Launch xcloud."""
        return _launch.launch_xcloud(self, xcloud_url)
    def kill(self) -> None:
        """Kill."""
        process_ops.graceful_kill(self.process)
        self.process = None
        self.cleanup_stale_profile_state()

    @staticmethod
    def has_xbox_session() -> bool:
        """Check whether xbox session."""
        return _make_profile_manager().has_xbox_session()
    @staticmethod
    def clear_cookies() -> None:
        """Clear cookies."""
        _make_profile_manager().clear_cookies()
    @staticmethod
    def clear_profile_data() -> None:
        """Clear profile data."""
        _make_profile_manager().clear_profile_data()
    async def wait_and_check_crash(self) -> bool:
        """Wait and check crash."""
        result = await process_ops.wait_and_check_crash(
            self.process, self._cdp.probe_cdp, LOG_FILE,
        )
        if not result:
            self.process = None
        return result

```

OP-29b

env.py**Fichier :** py_modules/unifideck/auth/edge_browser/env.py | Lignes : 173 | Depend de : (rien)

```
from __future__ import annotations
import logging
import os
import subprocess
from pathlib import Path
logger = logging.getLogger(__name__)
_SESSION_ENV_KEYS = (
    "DISPLAY",
    "WAYLAND_DISPLAY",
    "GAMESCOPE_WAYLAND_DISPLAY",
    "DBUS_SESSION_BUS_ADDRESS",
    "DESKTOP_SESSION",
    "GTK_IM_MODULE",
    "QT_IM_MODULE",
    "XAUTHORITY",
    "XDG_RUNTIME_DIR",
    "XMODIFIERS",
    "XDG_SESSION_TYPE",
    "XDG_CURRENT_DESKTOP",
)
def _seed_from_own_env(result: dict[str, str]) -> None:
    """Seed from own env."""
    for key in _SESSION_ENV_KEYS:
        value = os.environ.get(key)
        if value:
            result[key] = value
def _read_gamescope_env_file(
    runtime_dir: str, result: dict[str, str],
) -> None:
    """Read gamescope env file."""
    gamescope_env = Path(runtime_dir) / "gamescope-environment"
    if not gamescope_env.exists():
        return
    try:
        with gamescope_env.open(
            encoding="utf-8", errors="replace",
        ) as f:
            for raw_line in f:
                line = raw_line.strip()
                if not line or "=" not in line:
                    continue
                key, value = line.split("=", 1)
                if (
                    key in _SESSION_ENV_KEYS
                    and key not in result
                    and value
                ):
                    result[key] = value
    except OSError:
        pass
def _parse_proc_environ(
    pid: str, result: dict[str, str],
) -> bool:
```

```

"""Parse proc environ."""
try:
    with Path(f"/proc/{pid}/environ").open("rb") as f:
        env_bytes = f.read()
except (PermissionError, FileNotFoundError, OSError):
    return False
for entry in env_bytes.split(b"\x00"):
    decoded = entry.decode("utf-8", errors="replace")
    if "=" not in decoded:
        continue
    key, value = decoded.split("=", 1)
    if (
        key in _SESSION_ENV_KEYS
        and key not in result
        and value
    ):
        result[key] = value
return bool(
    result.get("DISPLAY") or result.get("WAYLAND_DISPLAY"),
)
def _scan_steam_process_env(
    uid: int, result: dict[str, str],
) -> None:
    """Scan steam process env."""
    try:
        for proc_name in (
            "steam", "gamescope-session", "gamescope",
        ):
            pids = subprocess.run(
                ["pgrep", "-u", str(uid), "-x", proc_name],
                capture_output=True,
                text=True,
                timeout=5,
            ).stdout.strip().split("\n")
            for raw_pid in pids:
                pid = raw_pid.strip()
                if not pid:
                    continue
                if _parse_proc_environ(pid, result):
                    logger.info(
                        "[Edge] Session env detected from "
                        "PID %s (%s): DISPLAY=%s "
                        "WAYLAND_DISPLAY=%s",
                        pid, proc_name,
                        result.get('DISPLAY'),
                        result.get('WAYLAND_DISPLAY'),
                    )
            return
    except Exception as e:
        logger.debug(
            "[Edge] Session env detection error: %s", e,
        )

def _apply_fallbacks(
    uid: int, home: str, runtime_dir: str, result: dict[str, str],
) -> None:
    """Apply fallbacks."""
    if (
        not result.get("DISPLAY")
        and not result.get("WAYLAND_DISPLAY")
    )

```

```

):
    result["DISPLAY"] = ":0"
if not result.get("XDG_RUNTIME_DIR"):
    result["XDG_RUNTIME_DIR"] = runtime_dir
if (
    "DBUS_SESSION_BUS_ADDRESS" not in result
    and Path(f"{runtime_dir}/bus").exists()
):
    result["DBUS_SESSION_BUS_ADDRESS"] = (
        f"unix:path={runtime_dir}/bus"
    )
if "XAUTHORITY" not in result:
    xauth_files = [
        str(p) for p in Path(runtime_dir).glob("xauth_*")
    ]
    if xauth_files:
        result["XAUTHORITY"] = xauth_files[0]
    elif (Path(home) / ".Xauthority").exists():
        result["XAUTHORITY"] = str(Path(home) / ".Xauthority")
if (
    not result.get("WAYLAND_DISPLAY")
    and result.get("GAMESCOPE_WAYLAND_DISPLAY")
    and result.get("XDG_RUNTIME_DIR")
):
    gamescope_socket = (
        Path(result["XDG_RUNTIME_DIR"])
        / result["GAMESCOPE_WAYLAND_DISPLAY"]
    )
    if gamescope_socket.exists():
        result["WAYLAND_DISPLAY"] = (
            result["GAMESCOPE_WAYLAND_DISPLAY"]
        )
if (
    result.get("GTK_IM_MODULE") == "Steam"
    and not result.get("XMODIFIERS")
):
    result["XMODIFIERS"] = "@im=Steam"
def _detect_session_env(uid: int, home: str) -> dict[str, str]:
    """Detect session env."""
    result: dict[str, str] = {}
    runtime_dir = f"/run/user/{uid}"
    _seed_from_own_env(result)
    _read_gamescope_env_file(runtime_dir, result)
    _scan_steam_process_env(uid, result)
    _apply_fallbacks(uid, home, runtime_dir, result)
    return result
def clean_env() -> dict:
    """Clean env."""
    home = str(Path.home())
    uid = Path(home).stat().st_uid
    env = {
        k: v for k, v in os.environ.items()
        if k not in ("LD_LIBRARY_PATH", "LD_PRELOAD")
    }
    env.update(_detect_session_env(uid, home))
    env.setdefault("XDG_RUNTIME_DIR", f"/run/user/{uid}")
    env.setdefault("SteamGameId", "0")
    env.setdefault("STEAM_COMPAT_APP_ID", "0")
    env.setdefault("SteamAppId", "0")
    env["GTK_MODULES"] = ""
    return env

```

OP-29c

launch.py

Fichier : py_modules/unifideck/auth/edge_browser/launch.py | Lignes : 110 | Depend de : (rien)

```
from __future__ import annotations
import logging
import os
import subprocess
from pathlib import Path
from typing import TYPE_CHECKING
from .env import clean_env
if TYPE_CHECKING:
    from .edge import EdgeBrowser
logger = logging.getLogger(__name__)
def _prepare_for_launch(browser: EdgeBrowser) -> list[str] | None:
    """Prepare for launch."""
    browser.kill()
    browser.cleanup_stale_profile_state()
    cmd = browser.find_cmd()
    if not cmd:
        return None
    from .edge import PROFILE_DIR
    Path(PROFILE_DIR).mkdir(parents=True, exist_ok=True)
    return cmd

def _spawn_edge_process(
    browser: EdgeBrowser,
    args: list[str],
    log_mode: str,
    label: str,
) -> bool:

    """Spawn edge process."""
    from .edge import LOG_FILE
    env = clean_env()
    logger.info(
        "[Edge] Launching %s browser: %s...",
        label.lower(), ' '.join(args[:7]),
    )
    logger.info(
        "[Edge] %s browser env DISPLAY=%s "
        "WAYLAND_DISPLAY=%s SteamAppId=%s",
        label, env.get('DISPLAY'), env.get('WAYLAND_DISPLAY'),
        env.get('SteamAppId'),
    )
    stderr_fh = None
    try:
        stderr_fh = Path(LOG_FILE).open(log_mode)
    except Exception:
        pass
    try:
        browser.process = subprocess.Popen(
            args,
            stdout=subprocess.DEVNULL,
            stderr=(
                stderr_fh if stderr_fh else subprocess.DEVNULL
            ),
            env=env,
            preexec_fn=os.setpgroup,
        )
```

```

    )
    logger.info(
        "[Edge] %s browser PID: %s", label, browser.process.pid,
    )
    return True
except Exception as e:
    logger.exception(
        "[Edge] Failed to launch %s browser: %s", label.lower(), e,
    )
    return False
finally:
    if stderr_fh is not None:
        stderr_fh.close()
def launch_auth(browser: EdgeBrowser, auth_url: str) -> bool:
    """Launch auth."""
    cmd = _prepare_for_launch(browser)
    if not cmd:
        logger.warning("[Edge] No compatible browser found for auth")
        return False
    from .edge import _BASE_FLAGS, PROFILE_DIR
    args = cmd + [
        f"--app={auth_url}",
        "--class=unifideck-auth",
        f"--remote-debugging-port={browser.cdp_port}",
        f"--user-data-dir={PROFILE_DIR}",
    ] + _BASE_FLAGS + [
        "--start-fullscreen",
        "--enable-touch-events",
        "--window-size=1280,800",
        f"--lang={browser.locale_fn().split('-')[0]}",
    ]
    return _spawn_edge_process(browser, args, log_mode="w", label="Auth")

def launch_xcloud(browser: EdgeBrowser, xcloud_url: str) -> bool:
    """Launch xcloud."""
    cmd = _prepare_for_launch(browser)
    if not cmd:
        logger.warning("[Edge] No compatible browser found for xCloud")
        return False
    from .edge import _BASE_FLAGS, PROFILE_DIR
    xcloud_cdp_port = browser.cdp_port + 1
    args = cmd + [
        "--kiosk",
        "--class=unifideck-xcloud",
        f"--remote-debugging-port={xcloud_cdp_port}",
        f"--user-data-dir={PROFILE_DIR}",
    ] + _BASE_FLAGS + [
        "--autoplay-policy=no-user-gesture-required",
        "--window-size=1024,720",
        "--force-device-scale-factor=1.25",
        "--device-scale-factor=1.25",
        f"--lang={browser.locale_fn().split('-')[0]}",
        xcloud_url,
    ]
    logger.info(
        "[Edge] Launching xCloud kiosk: %s", xcloud_url[:80],
    )
    return _spawn_edge_process(browser, args, log_mode="a", label="xCloud")

```


OP-29d

detection.py

Fichier : py_modules/unifideck/auth/edge_browser/detection.py | Lignes : 68 | Depend de : (rien)

```
from __future__ import annotations
import logging
import shutil
import subprocess
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from collections.abc import Callable
logger = logging.getLogger(__name__)
_FLATPAK_APPS = ("com.microsoft.Edge",)
_NATIVE_BINS = ("microsoft-edge", "microsoft-edge-stable")
def flatpak_remote_names(
    clean_env_fn: Callable[[], dict], scope: str,
) -> set[str]:
    """Flatpak remote names."""
    if scope not in ("--user", "--system"):
        return set()
    try:
        result = subprocess.run(
            ["flatpak", "remotes", scope, "--columns=name"],
            capture_output=True,
            text=True,
            timeout=5,
            env=clean_env_fn(),
            check=False,
        )
    except Exception:
        return set()
    if result.returncode != 0:
        return set()
    remotes: set[str] = set()
    for raw_line in result.stdout.splitlines():
        line = raw_line.strip()
        if not line or line.lower() == "name":
            continue
        remotes.add(line)
    return remotes
def find_edge_cmd(
    clean_env_fn: Callable[[], dict],
) -> list[str] | None:
    """Find edge cmd."""
    if shutil.which("flatpak"):
        for app_id in _FLATPAK_APPS:
            cmd = _try_flatpak_app(app_id, clean_env_fn)
            if cmd is not None:
                return cmd
        for binary in _NATIVE_BINS:
            if shutil.which(binary):
                return [binary]
    return None
def _try_flatpak_app(
    app_id: str, clean_env_fn: Callable[[], dict],
) -> list[str] | None:
    """Try flatpak app."""
    try:
        for flag in ("--user", "--system"):
```

```
        result = subprocess.run(
            ["flatpak", "info", flag, app_id],
            capture_output=True, timeout=5,
            env=clean_env_fn(),
        )
        if result.returncode == 0:
            return ["flatpak", "run", app_id]
    except Exception:
        pass
    return None

def is_edge_installed(clean_env_fn: Callable[[[]], dict]) -> bool:
    """Check whether edge installed."""
    return find_edge_cmd(clean_env_fn) is not None
```

OP-29e

installer.py

Fichier : py_modules/unifideck/auth/edge_browser/installer.py | Lignes : 251 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import shutil
import subprocess
from pathlib import Path
from typing import TYPE_CHECKING, Any
from .detection import find_edge_cmd, flatpak_remote_names, is_edge_installed
if TYPE_CHECKING:
    from collections.abc import Callable
logger = logging.getLogger(__name__)
_FLATPAK_APPS = ("com.microsoft.Edge",)
_EDGE_FLATPAK_APP = "com.microsoft.Edge"
_FLATHUB_REMOTE = "flathub"
_FLATHUB_REMOTE_URL = "https://dl.flathub.org/repo/flathub.flatpakrepo"
_NATIVE_BINS = ("microsoft-edge", "microsoft-edge-stable")
class EdgeInstaller:
    """Edge installer."""
    def __init__(self, clean_env_fn: Callable[[], dict]) -> None:
        """Initialize the instance."""
        self._clean_env = clean_env_fn

    def ensure_controller_permissions(self) -> bool:
        """Ensure controller permissions."""
        if not shutil.which("flatpak"):
            return False
        overrides_path = Path(
            f"~/.local/share/flatpak/overrides/{_EDGE_FLATPAK_APP}",
        ).expanduser()
        try:
            if overrides_path.is_file():
                with overrides_path.open() as fh:
                    if "/run/udev" in fh.read():
                        logger.debug(
                            "[Edge] Edge udev override already present",
                        )
                        return True
        except OSError:
            pass
        logger.info(
            "[Edge] Applying flatpak /run/udev:ro override for "
            "controller support",
        )
        try:
            proc = subprocess.run(
                [
                    "flatpak", "--user", "override",
                    "--filesystem=/run/udev:ro",
                    _EDGE_FLATPAK_APP,
                ],
                capture_output=True,
                timeout=30,
                check=False,
            )
```

```

        if proc.returncode == 0:
            logger.info(
                "[Edge] Edge udev override applied successfully",
            )
            return True
        stderr = proc.stderr.decode(
            "utf-8", errors="replace",
        )[:200]
        logger.warning(
            "[Edge] Edge udev override failed: %s", stderr,
        )
        return False
    except Exception as exc:
        logger.warning(
            "[Edge] Edge udev override error: %s", exc,
        )
        return False
def _flatpak_remote_names(self, scope: str) -> set[str]:
    """Flatpak remote names."""
    return flatpak_remote_names(self._clean_env, scope)
def find_cmd(self) -> list[str] | None:
    """Find cmd."""
    return find_edge_cmd(self._clean_env)
@property
def is_installed(self) -> bool:
    """Check whether installed."""
    return is_edge_installed(self._clean_env)

async def _ensure_user_flathub_remote(self) -> bool:
    """Ensure user flathub remote."""
    if _FLATHUB_REMOTE in flatpak_remote_names(
        self._clean_env, "--user",
    ):
        return True
    logger.info(
        "[Edge] Adding user flathub remote for "
        "browser installation",
    )
    try:
        proc = await asyncio.get_event_loop().run_in_executor(
            None,
            lambda: subprocess.run(
                [
                    "flatpak",
                    "remote-add",
                    "--if-not-exists",
                    "--user",
                    _FLATHUB_REMOTE,
                    _FLATHUB_REMOTE_URL,
                ],
                capture_output=True,
                timeout=60,
                env=self._clean_env(),
                check=False,
            ),
        )
    except Exception as e:
        logger.warning(
            "[Edge] Could not add user flathub remote: %s", e,
        )

```

```

        return False
    if proc.returncode != 0:
        stderr = proc.stderr.decode(
            "utf-8", errors="replace",
        )[:200]
        logger.warning(
            "[Edge] Adding user flathub remote failed: %s",
            stderr,
        )
        return False
    return (
        _FLATHUB_REMOTE
        in flatpak_remote_names(self._clean_env, "--user")
    )
def _get_default_browser(self) -> str | None:
    """Get default browser."""
    try:
        result = subprocess.run(
            ["xdg-settings", "get", "default-web-browser"],
            capture_output=True, text=True, timeout=5,
            env=self._clean_env(),
        )
        if result.returncode == 0 and result.stdout.strip():
            return result.stdout.strip()
    except Exception:
        pass
    return None

def _restore_default_browser(self, original: str | None) -> None:
    """Restore default browser."""
    if not original:
        return
    try:
        current = subprocess.run(
            ["xdg-settings", "get", "default-web-browser"],
            capture_output=True, text=True, timeout=5,
            env=self._clean_env(),
            check=False,
        ).stdout.strip()
        if current != original:
            subprocess.run(
                [
                    "xdg-settings", "set",
                    "default-web-browser", original,
                ],
                capture_output=True, timeout=5,
                env=self._clean_env(),
                check=False,
            )
            logger.info(
                "[Edge] Restored default browser to %s",
                original,
            )
    except Exception as e:
        logger.debug(
            "[Edge] Could not restore default browser: %s", e,
        )

async def install(self) -> dict[str, Any]:

```

```

"""Install."""
if not shutil.which("flatpak"):
    return {"success": False, "error": "microsoft.flatpakNotFound"}
if self.is_installed:
    return {
        "success": True,
        "message": "microsoft.browserAlreadyInstalled",
    }
if not await self._ensure_user_flathub_remote():
    return {
        "success": False,
        "error": "microsoft.browserInstallFailed",
    }
original_browser = self._get_default_browser()
logger.info(
    "[Edge] Attempting to install Microsoft Edge via flatpak...",
)
try:
    proc = await self._run_flatpak_install()
    if proc.returncode == 0:
        logger.info(
            "[Edge] Microsoft Edge installed successfully",
        )
        self._restore_default_browser(original_browser)
        await self._wait_for_edge_ready()
        self.ensure_controller_permissions()
        return {
            "success": True,
            "message": "microsoft.browserInstalled",
        }
    stderr = proc.stderr.decode(
        "utf-8", errors="replace",
    )[:200]
    logger.warning(
        "[Edge] Microsoft Edge install failed: %s", stderr,
    )
    return {
        "success": False,
        "error": "microsoft.browserInstallFailed",
    }
except subprocess.TimeoutExpired:
    logger.warning("[Edge] Microsoft Edge install timed out")
    return {
        "success": False,
        "error": "microsoft.edgeInstallTimeout",
    }
except Exception as e:
    logger.warning(
        "[Edge] Microsoft Edge install error: %s", e,
    )
    return {
        "success": False,
        "error": "microsoft.browserInstallFailed",
    }
}

async def _run_flatpak_install(
    self,
) -> subprocess.CompletedProcess[bytes]:
    """Run flatpak install."""
    return await asyncio.get_event_loop().run_in_executor(
        None,
        lambda: subprocess.run(

```

```
        [
            "flatpak",
            "install",
            "--user",
            "--noninteractive",
            "-y",
            _FLATHUB_REMOTE,
            _EDGE_FLATPAK_APP,
        ],
        capture_output=True,
        timeout=300,
        env=self._clean_env(),
        check=False,
    ),
)
)
async def _wait_for_edge_ready(self) -> None:
    """Wait for edge ready."""
    for _ in range(10):
        if self.find_cmd() is not None:
            return
        await asyncio.sleep(1)
```

OP-29f

cdp_client.py

Fichier : py_modules/unifideck/auth/edge_browser/cdp_client.py | Lignes : 183 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import json
import logging
from typing import Any
logger = logging.getLogger(__name__)
class EdgeCDPClient:
    """Edge cdpclient."""
    def __init__(self, cdp_port: int) -> None:
        """Initialize the instance."""
        self.cdp_port = cdp_port
    def get_browser_ws_url(self) -> str | None:
        """Get browser ws URL."""
        import urllib.request as _req
        try:
            with _req.urlopen(
                f"http://127.0.0.1:{self.cdp_port}/json/version",
                timeout=1,
            ) as r:
                data = json.loads(r.read().decode())
                ws_url = data.get("websocketDebuggerUrl")
                return ws_url if ws_url else None
        except Exception:
            return None
    def probe_cdp(self) -> bool:
        """Probe CDP."""
        import urllib.request
        try:
            with urllib.request.urlopen(
                f"http://127.0.0.1:{self.cdp_port}/json/version",
                timeout=1,
            ):
                return True
        except Exception:
            return False
    def list_targets(self) -> list[dict[str, Any]]:
        """List targets."""
        import urllib.request as _req
        try:
            with _req.urlopen(
                f"http://127.0.0.1:{self.cdp_port}/json/list",
                timeout=1,
            ) as r:
                data = json.loads(r.read().decode())
                return data if isinstance(data, list) else []
        except Exception:
            return []

    async def navigate_tab(
        self,
        url: str,
        timeout: float = 15.0,
    ) -> bool:
        """Navigate tab."""
```



```

targets = self.list_targets()
page_target = next(
    (t for t in targets if t.get("type") == "page"), None,
)
if not page_target:
    logger.warning("[Edge] navigate_tab: no page target found")
    return False
ws_url = page_target.get("websocketDebuggerUrl")
if not ws_url:
    logger.warning(
        "[Edge] navigate_tab: no websocketDebuggerUrl",
    )
    return False
try:
    import websockets
except ImportError:
    logger.warning(
        "[Edge] navigate_tab: websockets not available",
    )
    return False
try:
    async with websockets.connect(ws_url, close_timeout=3) as ws:
        await ws.send(json.dumps({
            "id": 1,
            "method": "Page.enable",
            "params": {},
        }))
        try:
            await asyncio.wait_for(ws.recv(), timeout=3)
        except TimeoutError:
            pass
        await ws.send(json.dumps({
            "id": 2,
            "method": "Page.navigate",
            "params": {"url": url},
        }))
        deadline = asyncio.get_event_loop().time() + timeout
        return await _await_navigation_result(
            ws, deadline, url,
        )
except Exception as exc:
    logger.warning("[Edge] navigate_tab failed: %s", exc)
    return False

async def close_all_targets(self, *, log_prefix: str) -> bool:
    """Close all targets."""
    targets = self.list_targets()
    if not targets:
        return False
    import urllib.error as _err
    import urllib.request as _req
    def _close_target(target_id: str) -> None:
        """Close target."""
        with _req.urlopen(
            f"http://127.0.0.1:{self.cdp_port}"
            f"/json/close/{target_id}",
            timeout=2,
        ) as r:
            r.read()
        closed_any = False

```

```

for target in targets:
    target_id = target.get("id")
    if not target_id:
        continue
    try:
        await asyncio.to_thread(_close_target, target_id)
        closed_any = True
    except _err.HTTPError as e:
        if e.code != 404:
            logger.warning(
                "[Edge] Could not close %s target %s: %s",
                log_prefix, target_id, e,
            )
    except Exception as e:
        logger.warning(
            "[Edge] Could not close %s target %s: %s",
            log_prefix, target_id, e,
        )
    if closed_any:
        for _ in range(20):
            await asyncio.sleep(0.25)
            if not self.get_browser_ws_url():
                break
        logger.info(
            "[Edge] Closed %s browser targets via "
            "DevTools HTTP",
            log_prefix,
        )
    return closed_any

async def _await_navigation_result(
    ws: Any, deadline: float, url: str,
) -> bool:

    """Await navigation result."""
    got_navigate_ok = False
    while asyncio.get_event_loop().time() < deadline:
        remaining = deadline - asyncio.get_event_loop().time()
        if remaining <= 0:
            break
        try:
            raw = await asyncio.wait_for(
                ws.recv(), timeout=remaining,
            )
        except TimeoutError:
            break
        msg = json.loads(raw)
        if msg.get("id") == 2:
            if "error" in msg:
                logger.warning(
                    "[Edge] navigate_tab error: %s",
                    msg["error"],
                )
            return False
        got_navigate_ok = True
    if (
        msg.get("method") in (
            "Page.frameStoppedLoading",
            "Page.loadEventFired",
        )
        and got_navigate_ok
    )

```

```
    ):
        logger.info(
            "[Edge] navigate_tab: loaded %s", url,
        )
        return True
if got_navigate_ok:
    logger.info(
        "[Edge] navigate_tab: navigation sent, "
        "load timed out for %s", url,
    )
    return True
return False
```

OP-29g

profile.py

Fichier : py_modules/unifideck/auth/edge_browser/profile.py | Lignes : 175 | Depend de : (rien)

```
from __future__ import annotations
import logging
import shutil
import sqlite3
import tempfile
from pathlib import Path
from typing import cast

logger = logging.getLogger(__name__)
class EdgeProfileManager:
    """Edge profile manager."""
    def __init__(
        self,
        *,
        profile_dir: str,
        log_file: str,
        legacy_profile_dir: str,
        legacy_log_file: str,
        cookie_domain_patterns: tuple[str, ...],
    ) -> None:
        """Initialize the instance."""
        self.profile_dir = profile_dir
        self.log_file = log_file
        self.legacy_profile_dir = legacy_profile_dir
        self.legacy_log_file = legacy_log_file
        self.cookie_domain_patterns = cookie_domain_patterns
    def migrate_legacy_profile(self) -> None:
        """Migrate legacy profile."""
        legacy_exists = Path(self.legacy_profile_dir).is_dir()
        new_exists = Path(self.profile_dir).is_dir()
        if not legacy_exists:
            return
        if new_exists:
            logger.debug(
                "[EdgeBrowser] both %s and %s exist; skipping "
                "migration",
                self.legacy_profile_dir, self.profile_dir,
            )
            return
        try:
            shutil.move(self.legacy_profile_dir, self.profile_dir)
            logger.info(
                "[EdgeBrowser] migrated legacy profile %s → %s",
                self.legacy_profile_dir, self.profile_dir,
            )
        except OSError as e:
            logger.warning(
                "[EdgeBrowser] legacy profile migration failed "
                "(%s → %s): %s – users may need to re-auth",
                self.legacy_profile_dir, self.profile_dir, e,
            )
        if (
            Path(self.legacy_log_file).is_file()
            and not Path(self.log_file).is_file()
        ):
            try:
```

```

        shutil.move(self.legacy_log_file, self.log_file)
    except OSError:
        pass
def _singleton_paths(self) -> list[str]:
    """Singleton paths."""
    profile = Path(self.profile_dir)
    return [
        str(profile / "SingletonLock"),
        str(profile / "SingletonCookie"),
        str(profile / "SingletonSocket"),
    ]

def _has_stale_singleton_socket(self) -> bool:
    """Has stale singleton socket."""
    socket_path = Path(self.profile_dir) / "SingletonSocket"
    if not socket_path.is_symlink():
        return False
    try:
        target = socket_path.readlink()
    except OSError:
        return False
    return not Path(target).exists()
def cleanup_stale_state(self) -> None:
    """Cleanup stale state."""
    if not self._has_stale_singleton_socket():
        return
    removed: list[str] = []
    for path in self._singleton_paths():
        try:
            Path(path).unlink()
            removed.append(Path(path).name)
        except FileNotFoundError:
            continue
        except OSError as e:
            logger.warning(
                "[Edge] Failed to remove stale profile "
                "artifact %s: %s", path, e,
            )
    if removed:
        logger.info(
            "[Edge] Removed stale browser profile artifacts: %s",
            ", ".join(sorted(removed)),
        )
def has_xbox_session(self) -> bool:
    """Check whether xbox session."""
    cookie_db = Path(self.profile_dir) / "Default" / "Cookies"
    if not cookie_db.exists():
        return True
    tmp_path = None
    try:
        with tempfile.NamedTemporaryFile(
            suffix=".db", delete=False,
        ) as tmp:
            tmp_path = tmp.name
        shutil.copy2(str(cookie_db), tmp_path)
        conn = sqlite3.connect(tmp_path, timeout=5)
        try:
            cursor = conn.execute(
                "SELECT COUNT(*) FROM cookies "
                "WHERE host_key LIKE '%xbox.com%'",
            )

```

```

        )
        count = cursor.fetchone()[0]
        return cast("bool", count > 0)
    finally:
        conn.close()
except Exception as e:
    logger.debug("[Edge] Could not read cookie DB: %s", e)
    return True
finally:
    if tmp_path and Path(tmp_path).exists():
        Path(tmp_path).unlink()

def clear_cookies(self) -> None:
    """Clear cookies."""
    cookie_db = Path(self.profile_dir) / "Default" / "Cookies"
    if not cookie_db.exists():
        return
    try:
        conn = sqlite3.connect(str(cookie_db), timeout=5)
        try:
            for pattern in self.cookie_domain_patterns:
                conn.execute(
                    "DELETE FROM cookies WHERE host_key LIKE ?",
                    (pattern,),
                )
            conn.commit()
            logger.info(
                "[Edge] Cleared Xbox/MS cookies from shared "
                "browser profile",
            )
        except Exception:
            conn.rollback()
            raise
        finally:
            conn.close()
    except Exception as e:
        logger.debug(
            "[Edge] Could not clear shared browser cookies: %s", e,
        )

def clear_profile_data(self) -> None:
    """Clear profile data."""
    removed: list[str] = []
    for path in (self.profile_dir, self.log_file):
        path_obj = Path(path)
        if not path_obj.exists():
            continue
        try:
            if path_obj.is_dir() and not path_obj.is_symlink():
                shutil.rmtree(path)
            else:
                path_obj.unlink()
            removed.append(path_obj.name)
        except Exception as e:
            logger.warning(
                "[Edge] Could not clear auth profile path %s: %s",
                path, e,
            )
    if removed:
        logger.info(
            "[Edge] Cleared auth state: %s",

```

```
    ", ".join(sorted(removed)),  
  )
```

OP-29h

process_ops.py

Fichier : py_modules/unifideck/auth/edge_browser/process_ops.py | Lignes : 88 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import os
import signal
import subprocess
from pathlib import Path
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from collections.abc import Callable
logger = logging.getLogger(__name__)
_TERM_TIMEOUT_S = 10
_KILL_TIMEOUT_S = 3
_COOKIE_FLUSH_DELAY_S = 1
_STARTUP_POLL_STEPS = 20
_STARTUP_POLL_INTERVAL_S = 0.5
_CRASH_LOG_TAIL_CHARS = 300
def graceful_kill(proc: subprocess.Popen[bytes] | None) -> None:
    """Graceful kill."""
    if proc is None:
        return
    try:
        import time
        time.sleep(_COOKIE_FLUSH_DELAY_S)
        _signal_group_or_single(proc, signal.SIGTERM)
        proc.wait(timeout=_TERM_TIMEOUT_S)
        logger.info("[Edge] Auth browser closed (cookies flushed)")
    except subprocess.TimeoutExpired:
        _force_kill(proc)
    except Exception as e:
        logger.debug("[Edge] Auth browser kill error (non-fatal): %s", e)
def _signal_group_or_single(
    proc: subprocess.Popen[bytes], sig: int,
) -> None:
    """Signal group or single."""
    pgid = _safe_getpgid(proc.pid)
    if pgid is not None and pgid != os.getpgrp():
        os.killpg(pgid, sig)
    elif sig == signal.SIGTERM:
        proc.terminate()
    else:
        proc.kill()
def _safe_getpgid(pid: int) -> int | None:
    """Safe getpgid."""
    try:
        return os.getpgid(pid)
    except Exception:
        return None
def _force_kill(proc: subprocess.Popen[bytes]) -> None:
    """Force kill."""
    logger.debug("[Edge] Auth browser didn't exit -- sending SIGKILL")
    try:
        _signal_group_or_single(proc, signal.SIGKILL)
        proc.wait(timeout=_KILL_TIMEOUT_S)
    except Exception:
```



```
        pass

    async def wait_and_check_crash(
        proc: subprocess.Popen[bytes] | None,
        probe_cdp: Callable[[], bool],
        log_file: str,
    ) -> bool:

        """Wait and check crash."""
        if proc is None:
            return False
        for _ in range(_STARTUP_POLL_STEPS):
            await asyncio.sleep(_STARTUP_POLL_INTERVAL_S)
            if proc.poll() is not None:
                _log_crash_tail(log_file)
                return False
            if await asyncio.to_thread(probe_cdp):
                return True
        logger.warning(
            "[Edge] Auth browser started but CDP port not "
            "responding after %ds",
            int(_STARTUP_POLL_STEPS * _STARTUP_POLL_INTERVAL_S),
        )
        return True

def _log_crash_tail(log_file: str) -> None:
    """Log crash tail."""
    err = ""
    try:
        with Path(log_file).open() as f:
            err = f.read()[:_CRASH_LOG_TAIL_CHARS]
    except Exception:
        pass
    logger.error(
        "[Edge] Auth browser crashed before CDP. stderr: %s", err,
    )
```

OP-28a

browser.py

Fichier : py_modules/unifideck/auth/browser.py | Lignes : 202 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import re
import time
from collections.abc import Iterable
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
from urllib.parse import parse_qs, urlparse
from ..utils.config_helpers import get_cfg
if TYPE_CHECKING:
    from ..cdp.cdp_client import CDPClient
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
DEFAULT_POLL_INTERVAL = 0.5
DEFAULT_OAUTH_TIMEOUT = 300
@dataclass
class AuthCaptureResult:
    """Auth capture result."""
    success: bool
    redirect_url: str | None = None
    params: dict[str, str] = field(default_factory=dict)
    elapsed_seconds: float = 0.0
    error: str | None = None
    @property
    def code(self) -> str | None:
        """Code."""
        return self.params.get("code")
    @property
    def state(self) -> str | None:
        """State."""
        return self.params.get("state")
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        return {
            "success": self.success,
            "redirect_url": self.redirect_url,
            "params": dict(self.params),
            "elapsed_seconds": self.elapsed_seconds,
            "error": self.error,
        }
def extract_oauth_params(url: str) -> dict[str, str]:
    """Extract oauth params."""
    if not url:
        return {}
    parsed = urlparse(url)
    out: dict[str, str] = {}
    for key, values in parse_qs(parsed.query).items():
        if values:
            out[key] = values[0]
    if parsed.fragment:
        for key, values in parse_qs(parsed.fragment).items():
            if values and key not in out:
                out[key] = values[0]
    return out
```

```

def match_redirect(
    url: str, allowed_uris: Iterable[str],
) -> bool:

    """Match redirect."""
    if not url:
        return False
    parsed = urlparse(url)
    if parsed.scheme != "https" and not (
        parsed.scheme == "http"
        and parsed.hostname in (
            "localhost", "127.0.0.1", "::1",
        )
    ):
        return False
    base = (
        f"{parsed.scheme}://{parsed.netloc}{parsed.path}"
    )
    for prefix in allowed_uris:
        if not prefix:
            continue
        prefix_parsed = urlparse(prefix)
        prefix_base = (
            f"{prefix_parsed.scheme}://"
            f"{prefix_parsed.netloc}{prefix_parsed.path}"
        )
        if base.startswith(prefix_base):
            return True
    return False

def _cfg(config: ConfigManager | None, key: str, default: Any) -> Any:
    """Cfg."""
    return get_cfg(config, key, default)

class OAuthBrowserMonitor:
    """OAuth browser monitor."""
    def __init__(
        self,
        cdp_client: CDPClient,
        config: ConfigManager | None = None,
    ) -> None:
        """Initialize the instance."""
        self._cdp = cdp_client
        self._config = config
        self._poll_interval = float(get_cfg(
            config, "auth.browser_poll_interval_seconds",
            DEFAULT_POLL_INTERVAL,
        ))
        self._default_timeout = float(get_cfg(
            config, "auth.browser_oauth_timeout_seconds",
            DEFAULT_OAUTH_TIMEOUT,
        ))

    async def wait_for_redirect(
        self,
        allowed_uris: list[str],
        timeout: float | None = None,
    ) -> AuthCaptureResult:

        """Wait for redirect."""
        deadline = time.monotonic() + (
            timeout if timeout is not None

```

```

        else self._default_timeout
    )
    start = time.monotonic()
    while time.monotonic() < deadline:
        try:
            targets = await self._list_targets()
        except Exception as e:
            logger.debug(
                "[auth/browser] target list: %s", e,
            )
            await asyncio.sleep(self._poll_interval)
            continue
        for target in targets:
            url = target.get("url", "")
            if match_redirect(url, allowed_uris):
                elapsed = time.monotonic() - start
                params = extract_oauth_params(url)
                logger.info(
                    "[auth/browser] captured redirect "
                    "after %.1fs", elapsed,
                )
                return AuthCaptureResult(
                    success=True,
                    redirect_url=url,
                    params=params,
                    elapsed_seconds=elapsed,
                )
            await asyncio.sleep(self._poll_interval)
    return AuthCaptureResult(
        success=False,
        error="timeout",
        elapsed_seconds=time.monotonic() - start,
    )

async def close_oauth_tab(
    self, url_substring: str,
) -> bool:
    """Close oauth tab."""
    try:
        targets = await self._list_targets()
    except Exception:
        return False
    for target in targets:
        if url_substring in target.get("url", ""):
            target_id = target.get("id")
            if not target_id:
                continue
            try:
                await self._cdp.close_target(target_id)
                return True
            except Exception as e:
                logger.debug(
                    "[auth/browser] close failed: %s",
                    e,
                )
            return False
    return False

async def clear_store_cookies(self, domain: str) -> bool:
    """Clear store cookies."""
    if not re.match(

```

```
        r"^[a-zA-Z0-9][a-zA-Z0-9.\-]*$", domain,
    ):
        logger.warning(
            "[auth/browser] rejected invalid cookie "
            "domain: %r", domain,
        )
        return False
    try:
        await self._cdp.eval_js(
            "document.cookie.split(';').forEach(c => "
            "document.cookie = c.replace(/^ +/, '')"
            ".replace(/=.*/,"
            f" '=;expires=Thu, 01 Jan 1970 00:00:00 GMT;"
            f"path=/;domain={domain}')");",
        )
        return True
    except Exception as e:
        logger.debug(
            "[auth/browser] cookie clear failed: %s", e,
        )
        return False
async def _list_targets(self) -> list[dict[str, Any]]:
    """List targets."""
    try:
        return await self._cdp.list_targets()
    except Exception as e:
        logger.debug(
            "[auth/browser] list_targets failed: %s", e,
        )
        return []
CDPOAuthMonitor = OAuthBrowserMonitor
```

OP-28b

orchestrator.py

Fichier : py_modules/unifideck/auth/orchestrator.py | Lignes : 281 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from collections.abc import Awaitable, Callable
from dataclasses import dataclass
from pathlib import Path
from typing import TYPE_CHECKING
from ..core.types import AuthResult, Events
if TYPE_CHECKING:
    from ..event_bus.event_bus import EventBus
    from .browser import OAuthBrowserMonitor
    GetUrlCallback = Callable[[], Awaitable[str]]
    ExchangeCodeCallback = Callable[[str], Awaitable[AuthResult]]
logger = logging.getLogger(__name__)
@dataclass
class OrchestratorConfig:
    """Orchestrator config."""
    timeout: float = 300.0
    browser_launch_grace: float = 1.5
class AuthOrchestrator:
    """Auth orchestrator."""
    def __init__(
        self,
        bus: EventBus,
        browser_monitor: OAuthBrowserMonitor,
        store_name: str,
        config: OrchestratorConfig | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._monitor = browser_monitor
        self._store = store_name
        self._cfg = config or OrchestratorConfig()
        self._bg_task: asyncio.Task | None = None

    async def run_flow(
        self,
        get_url: GetUrlCallback,
        allowed_uris: list[str],
        exchange_code: ExchangeCodeCallback,
        *,
        timeout: float | None = None,
        write_url_file: str | None = None,
        background: bool = False,
    ) -> AuthResult:
        """Run flow."""
        deadline = timeout if timeout is not None else self._cfg.timeout
        await self._emit_started()
        try:
            url = await get_url()
        except Exception as e:
            logger.error(
                "[AuthOrchestrator/%s] get_url failed: %s",
                self._store, e,

```

```

        )
        return await self._emit_failed(
            "get_url_failed", str(e),
        )
    if not url:
        return await self._emit_failed(
            "no_url", "get_url returned empty string",
        )
    if write_url_file:
        write_ok = await self._write_url_atomically(
            write_url_file, url,
        )
        if not write_ok:
            return await self._emit_failed(
                "url_write_failed",
                f"could not write URL to {write_url_file}",
                url=url,
            )
    if background:
        return self._spawn_background_task(
            url=url,
            allowed_uris=allowed_uris,
            exchange_code=exchange_code,
            deadline=deadline,
        )
    return await self._await_redirect_and_exchange(
        url=url,
        allowed_uris=allowed_uris,
        exchange_code=exchange_code,
        deadline=deadline,
    )
def cancel_background(self) -> bool:
    """Check whether background."""
    task = self._bg_task
    if task is None or task.done():
        return False
    task.cancel()
    self._bg_task = None
    return True

async def _await_redirect_and_exchange(
    self,
    url: str,
    allowed_uris: list[str],
    exchange_code: ExchangeCodeCallback,
    deadline: float,
) -> AuthResult:
    """Await redirect and exchange."""
    logger.info(
        "[AuthOrchestrator/%s] waiting for redirect to %s "
        "(timeout=%s)",
        self._store, allowed_uris, deadline,
    )
    await asyncio.sleep(self._cfg.browser_launch_grace)
    try:
        capture = await self._monitor.wait_for_redirect(
            allowed_uris=allowed_uris,
            timeout=deadline,
        )
    except asyncio.CancelledError:

```

```

        logger.info(
            "[AuthOrchestrator/%s] flow cancelled", self._store,
        )
        raise
    except Exception as e:
        logger.error(
            "[AuthOrchestrator/%s] monitor crashed: %s",
            self._store, e,
        )
        return await self._emit_failed("monitor_crashed", str(e))
    if not capture.success:
        return await self._emit_failed(
            capture.error or "capture_failed",
            f"browser capture failed after "
            f"{capture.elapsed_seconds:.1f}s",
            url=url,
        )
    code = capture.code
    if not code:
        return await self._emit_failed(
            "no_code",
            f"redirect to {capture.redirect_url} carried no "
            f"`code` parameter",
            url=url,
        )
    await self._close_tab_safely(capture.redirect_url)
    return await self._finalize_auth_exchange(
        code, url, exchange_code,
    )

async def _finalize_auth_exchange(
    self,
    code: str,
    url: str,
    exchange_code: ExchangeCodeCallback,
) -> AuthResult:
    """Finalize auth exchange."""
    try:
        result = await exchange_code(code)
    except Exception as e:
        logger.error(
            "[AuthOrchestrator/%s] exchange_code failed: %s",
            self._store, e,
        )
        return await self._emit_failed(
            "exchange_failed", str(e), url=url,
        )
    result.store = self._store
    if result.success:
        await self._bus.emit(
            Events.STORE_AUTH_COMPLETE, store=self._store,
        )
        logger.info(
            "[AuthOrchestrator/%s] auth complete", self._store,
        )
    else:
        await self._bus.emit(
            Events.STORE_AUTH_FAILED,
            store=self._store,
            error=result.error or "exchange_returned_failure",

```



```

        )
        logger.warning(
            "[AuthOrchestrator/%s] exchange failed: %s",
            self._store, result.error,
        )
    )
    return result
def _spawn_background_task(
    self,
    url: str,
    allowed_uris: list[str],
    exchange_code: ExchangeCodeCallback,
    deadline: float,
) -> AuthResult:
    """Spawn background task."""
    self.cancel_background()
    async def _background_runner() -> None:
        """Background runner."""
        try:
            await self._await_redirect_and_exchange(
                url=url,
                allowed_uris=allowed_uris,
                exchange_code=exchange_code,
                deadline=deadline,
            )
        except asyncio.CancelledError:
            pass
        finally:
            if self._bg_task is not None and self._bg_task.done():
                self._bg_task = None
    self._bg_task = asyncio.create_task(
        _background_runner(),
        name=f"auth_flow_{self._store}",
    )
    logger.info(
        "[AuthOrchestrator/%s] background flow started",
        self._store,
    )
    return AuthResult(
        success=True,
        store=self._store,
        url=url,
        metadata={"pending": True},
    )
async def _emit_started(self) -> None:
    """Emit started."""
    await self._bus.emit(Events.STORE_AUTH_STARTED, store=self._store)

async def _emit_failed(
    self,
    error_code: str,
    detail: str,
    url: str | None = None,
) -> AuthResult:
    """Emit failed."""
    logger.warning(
        "[AuthOrchestrator/%s] %s: %s",
        self._store, error_code, detail,
    )
    await self._bus.emit(
        Events.STORE_AUTH_FAILED,

```

```

        store=self._store,
        error=error_code,
    )
    return AuthResult(
        success=False,
        error=error_code,
        store=self._store,
        url=url,
    )
async def _close_tab_safely(self, url_substring: str | None) -> None:
    """Close tab safely."""
    try:
        if url_substring is None:
            return
        domain = url_substring
        if "://" in domain:
            domain = domain.split("://", 1)[1]
        if "/" in domain:
            domain = domain.split("/", 1)[0]
        await self._monitor.close_oauth_tab(domain)
    except Exception as e:
        logger.debug(
            "[AuthOrchestrator/%s] close_oauth_tab failed "
            "(ignored): %s",
            self._store, e,
        )
    )
@staticmethod
async def _write_url_atomically(path: str, url: str) -> bool:
    """Write URL atomically."""
    def _write_sync() -> str:
        """Write sync."""
        expanded = Path(path).expanduser()
        parent = expanded.parent
        parent.mkdir(parents=True, exist_ok=True)
        tmp = expanded.with_name(expanded.name + ".tmp")
        with tmp.open("w", encoding="utf-8") as f:
            f.write(url)
        tmp.replace(expanded)
        return str(expanded)
    try:
        expanded = await asyncio.to_thread(_write_sync)
        logger.debug(
            "[AuthOrchestrator] wrote auth URL to %s", expanded,
        )
        return True
    except OSError as e:
        logger.error(
            "[AuthOrchestrator] failed to write %s: %s",
            expanded, e,
        )
    return False

```

OP-30

`__init__.py`

Fichier : `py_modules/unifideck/cdp/__init__.py` | Lignes : 9 | Depend de : (rien)

```
from .cdp_inject import (
    SteamCSSInjector,
    get_cdp_client,
    shutdown_cdp_client,
)
try:
    from .cdp_utils import create_cef_debugging_flag
except ImportError:
    pass
```

OP-30a

cdp_client.py

Fichier : py_modules/unifideck/cdp/cdp_client.py | Lignes : 206 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
from typing import TYPE_CHECKING, Any, cast
from unifideck.utils.config_helpers import get_cfg
logger = logging.getLogger(__name__)
if TYPE_CHECKING:
    from ..config import ConfigManager
class CDPClient:
    """Cdpclient."""
    def __init__(
        self,
        host: str | None = None,
        port: int | None = None,
        config: ConfigManager | None = None,
    ) -> None:
        """Initialize the instance."""
        self._host = host or get_cfg(config, "cdp.host", "127.0.0.1")
        self._port = port or int(get_cfg(config, "cdp.port", 8080))
        self._eval_timeout = float(
            get_cfg(config, "cdp.eval_timeout_seconds", 30)
        )
        self._response_timeout = float(
            get_cfg(config, "cdp.response_timeout_seconds", 10)
        )
        self._ws = None
        self._request_id = 0
        self._pending: dict[int, asyncio.Future] = {}
        self._recv_task: asyncio.Task | None = None
    async def connect(self, target_url_substring: str = "") -> bool:
        """Connect."""
        targets = await self._list_targets()
        target = self._pick_target(targets, target_url_substring)
        if not target or "websocketDebuggerUrl" not in target:
            return False
        try:
            import websockets
            self._ws = await websockets.connect(
                target["websocketDebuggerUrl"],
                max_size=None,
            )
        except Exception as e:
            logger.error("[CDPClient] ws connect failed: %s", e)
            return False
        self._recv_task = asyncio.create_task(self._recv_loop())
        return True
    async def disconnect(self) -> None:
        """Disconnect."""
        if self._recv_task:
            self._recv_task.cancel()
            try:
                await self._recv_task
            except asyncio.CancelledError:
                pass

```

```

    if self._ws:
        await self._ws.close()
        self._ws = None

async def inject_css(self, css: str) -> bool:

    """Inject CSS."""
    expression = (
        "(() => {"
        "const s = document.createElement('style');"
        f"s.textContent = {json.dumps(css)};"
        "document.head.appendChild(s);"
        "return true; })()"
    )
    result = await self.evaluate(expression)
    return bool(result and result.get("result", {}).get("value"))
async def evaluate(self, expression: str) -> dict | None:
    """Evaluate."""
    return await self._send("Runtime.evaluate", {
        "expression": expression,
        "returnByValue": True,
    })
async def eval_js(self, expression: str) -> Any:
    """Eval js."""
    result = await self.evaluate(expression)
    if not result:
        return None
    return result.get("result", {}).get("value")
async def list_targets(self) -> list[dict[str, Any]]:
    """List targets."""
    return await self._list_targets()
async def close_target(self, target_id: str) -> bool:
    """Close target."""
    if not target_id:
        return False
    import aiohttp
    url = (
        f"http://{self._host}:{self._port}"
        f"/json/close/{target_id}"
    )
    try:
        async with (
            aiohttp.ClientSession() as session,
            session.get(url, timeout=5) as resp,
        ):
            return cast("bool", resp.status == 200)
    except Exception as e:
        logger.debug(
            "[CDPClient] close_target failed: %s", e,
        )
        return False
async def navigate(self, url: str) -> bool:
    """Navigate."""
    result = await self._send("Page.navigate", {"url": url})
    return result is not None
async def wait_for_url(self, substring: str,
                      timeout: float | None = None) -> str | None:
    """Wait for URL."""
    deadline = asyncio.get_event_loop().time() + (
        timeout
        if timeout is not None
    )

```

```

        else self._eval_timeout
    )
    while asyncio.get_event_loop().time() < deadline:
        result = await self.evaluate(
            "window.location.href",
        )
        if result:
            url = (
                result.get("result", {}).get("value")
                or ""
            )
            if substring in url:
                return url
            await asyncio.sleep(0.5)
    return None

async def _list_targets(self) -> list[dict[str, Any]]:
    """List targets."""
    import aiohttp
    url = f"http://{self._host}:{self._port}/json"
    try:
        async with (
            aiohttp.ClientSession() as session,
            session.get(url, timeout=5) as resp,
        ):
            if resp.status != 200:
                return []
            return cast("list[dict[str, Any]]", await resp.json())
    except Exception as e:
        logger.debug(
            "[CDPClient] list targets failed: %s", e,
        )
        return []

@staticmethod
def _pick_target(targets: list[dict[str, Any]],
                 substring: str) -> dict[str, Any] | None:
    """Pick target."""
    for t in targets:
        if t.get("type") != "page":
            continue
        if not substring or substring in t.get("url", ""):
            return t
    return None

async def _send(self, method: str,
               params: dict[str, Any]) -> dict | None:
    """Send."""
    if not self._ws:
        return None
    self._request_id += 1
    req_id = self._request_id
    message = {
        "id": req_id,
        "method": method,
        "params": params,
    }
    future: asyncio.Future = (
        asyncio.get_event_loop().create_future()
    )
    self._pending[req_id] = future
    try:

```

```
        await self._ws.send(json.dumps(message))
        return await asyncio.wait_for(
            future, timeout=self._response_timeout,
        )
    except TimeoutError:
        logger.warning(
            "[CDPClient] timeout on %s", method,
        )
        return None
    finally:
        self._pending.pop(req_id, None)
async def _recv_loop(self) -> None:
    """Recv loop."""
    if self._ws is None:
        logger.warning(
            "[CDPClient] _recv_loop started before connect()",
        )
        return
    ws = self._ws
    try:
        async for raw in ws:
            try:
                msg = json.loads(raw)
            except json.JSONDecodeError:
                continue
            req_id = msg.get("id")
            if req_id and req_id in self._pending:
                self._pending[req_id].set_result(
                    msg.get("result"))
    except asyncio.CancelledError:
        raise
    except Exception as e:
        logger.warning("[CDPClient] recv loop error: %s", e)
```

OP-30b

cdp_inject.py

Fichier : py_modules/unifideck/cdp/cdp_inject.py | Lignes : 106 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from .cdp_client import CDPClient
logger = logging.getLogger(__name__)
STEAM_TAB_URL_MARKER = "steamloopback.host"
STYLE_ID_PREFIX = "unifideck-style-"
HIDE_PLAY_CSS = """
div[class*="appactionbutton_PlayButton"][data-app-id="__APP_ID__"],
div[class*="library_AppActionButton"][data-app-id="__APP_ID__"]
button[class*="play_PlayBtn"] {
    display: none !important;
}
""".strip()
def is_steam_ui_tab(page: dict[str, Any]) -> bool:
    """Check whether steam ui tab."""
    if not isinstance(page, dict):
        return False
    url = page.get("url", "")
    return STEAM_TAB_URL_MARKER in url
def escape_css_for_template_literal(css: str) -> str:
    """Escape CSS for template literal."""
    return (
        css.replace("\\", "\\")
        .replace("`", "\\`")
        .replace("${", "\\${")
    )
def build_marker_id(name: str) -> str:
    """Build marker ID."""
    safe = "".join(c if c.isalnum() or c in "-_" else "_" for c in name)
    return f"{STYLE_ID_PREFIX}{safe}"
class SteamCSSInjector:
    """Steam cssinjector."""
    def __init__(self, cdp_client: CDPClient) -> None:
        """Initialize the instance."""
        self._cdp = cdp_client
    async def connect_to_steam(self) -> bool:
        """Connect to steam."""
        try:
            return await self._cdp.connect(STEAM_TAB_URL_MARKER)
        except Exception as e:
            logger.warning("[cdp_inject] connect failed: %s", e)
            return False

    async def inject_css(self, css: str, marker: str) -> bool:
        """Inject CSS."""
        marker_id = build_marker_id(marker)
        escaped = escape_css_for_template_literal(css)
        js = f"""
        (() => {{
            const id = "{marker_id}";
            let el = document.getElementById(id);
            if (!el) {{

```



```

        el = document.createElement("style");
        el.id = id;
        document.head.appendChild(el);
    }}
    el.textContent = `{escaped}`;
    return true;
    }}()
    """
    try:
        result = await self._cdp.eval_js(js)
        return bool(result)
    except Exception as e:
        logger.warning(
            "[cdp_inject] eval failed for %s: %s", marker, e,
        )
        return False
async def remove_css(self, marker: str) -> bool:
    """Remove CSS."""
    marker_id = build_marker_id(marker)
    js = f"""
    (() => {{
    const el = document.getElementById("{marker_id}");
    if (el) {{ el.remove(); return true; }}
    return false;
    }})()
    """
    try:
        return bool(await self._cdp.eval_js(js))
    except Exception as e:
        logger.debug(
            "[cdp_inject] remove failed for %s: %s", marker, e,
        )
        return False
async def hide_play_section(self, app_id: int) -> bool:
    """Hide play section."""
    css = HIDE_PLAY_CSS.replace("__APP_ID__", str(app_id))
    return await self.inject_css(css, f"app-{app_id}")
async def show_play_section(self, app_id: int) -> bool:
    """Show play section."""
    return await self.remove_css(f"app-{app_id}")
_singleton_injector: SteamCSSInjector | None = None
async def get_cdp_client() -> SteamCSSInjector:
    """Get CDP client."""
    global _singleton_injector
    if _singleton_injector is None:
        from .cdp_client import CDPClient
        client = CDPClient()
        _singleton_injector = SteamCSSInjector(client)
    return _singleton_injector
async def shutdown_cdp_client() -> None:
    """Shutdown CDP client."""
    global _singleton_injector
    _singleton_injector = None

```

OP-30c

xcloud_browser_shims.py

Fichier : py_modules/unifideck/cdp/xcloud_browser_shims.py | Lignes : 400 | Depend de : (rien)

```
import json
_XCLOUD_BROWSER_SHIMS_JS = r"""
(function() {
  'use strict';
  if (window.__unifideck_xcloud_helper) return;
  var state = {
    injectedAt: Date.now(),
    reconnects: 0,
    listenerRegistrations: 0,
    lastReason: 'init',
  };
  window.__unifideck_xcloud_helper = state;
  var XBOX_GAMEPAD_ID = 'Xbox 360 Controller (XInput STANDARD GAMEPAD)';
  var defaultChromiumVersion = '120.0.0.0';
  var proxyCache = typeof WeakMap === 'function' ? new WeakMap() : null;
  var originalGetGamepads =
    typeof navigator.getGamepads === 'function'
    ? navigator.getGamepads.bind(navigator)
    : null;
  var originalWebkitGetGamepads =
    typeof navigator.webkitGetGamepads === 'function'
    ? navigator.webkitGetGamepads.bind(navigator)
    : null;
  function getChromiumVersion() {
    try {
      var match = String(navigator.userAgent || '').match(/s(?:Chrome|Edg)\v([\d.]+)/);
      if (match && match[1]) {
        return match[1];
      }
    } catch (e) {}
    return defaultChromiumVersion;
  }
  function spoofBrowserIdentity() {
    var chromiumVersion = getChromiumVersion();
    var edgeUserAgent =
      'Mozilla/5.0 (Windows NT 10.0; Win64; x64) ' +
      'AppleWebKit/537.36 (KHTML, like Gecko) ' +
      'Chrome/' + chromiumVersion + ' Safari/537.36 Edg/' + chromiumVersion;
    state.originalUserAgent = String(navigator.userAgent || '');
    state.spoofedUserAgent = edgeUserAgent;
    try {
      if (!('orgUserAgent' in navigator)) {
        Object.defineProperty(navigator, 'orgUserAgent', {
          configurable: true,
          value: navigator.userAgent,
        });
      }
    } catch (e) {}
    try {
      Object.defineProperty(navigator, 'userAgent', {
        configurable: true,
        get: function() {
          return edgeUserAgent;
        },
      });
    }
  }
})();
```

```
} catch (e) {}
try {
Object.defineProperty(navigator, 'platform', {
configurable: true,
get: function() {
return 'Win32';
},
});
} catch (e) {}
try {
if ('userAgentData' in navigator && !('orgUserAgentData' in navigator)) {
Object.defineProperty(navigator, 'orgUserAgentData', {
configurable: true,
value: navigator.userAgentData,
});
}
if ('userAgentData' in navigator) {
Object.defineProperty(navigator, 'userAgentData', {
configurable: true,
get: function() {
return undefined;
},
});
}
} catch (e) {}
}
function safeFocus() {
try {
if (typeof window.focus === 'function') window.focus();
} catch (e) {}
}
var pointerLockElement = null;
try {
Object.defineProperty(document, 'fullscreenElement', {
configurable: true,
get: function() {
return document.documentElement;
},
});
} catch (e) {}
try {
if (typeof HTMLElement.prototype.requestFullscreen !== 'function') {
HTMLElement.prototype.requestFullscreen = function() {
return Promise.resolve();
};
}
} catch (e) {}
try {
Object.defineProperty(document, 'pointerLockElement', {
configurable: true,
get: function() {
return pointerLockElement;
},
});
} catch (e) {}
try {
HTMLElement.prototype.requestPointerLock = function() {
pointerLockElement = document.documentElement;
document.dispatchEvent(new Event('pointerlockchange'));
};
}
```

```

} catch (e) {}
try {
document.exitPointerLock = function() {
pointerLockElement = null;
document.dispatchEvent(new Event('pointerlockchange'));
};
} catch (e) {}
function copyButtons(buttons) {
return Array.prototype.map.call(buttons || [], function(button) {
if (!button) return button;
return {
pressed: !!button.pressed,
touched: !!button.touched,
value: typeof button.value === 'number' ? button.value : 0,
};
});
}
function serializeGamepad(gamepad) {
return {
axes: Array.prototype.slice.call(gamepad.axes || []),
buttons: copyButtons(gamepad.buttons),
connected: !!gamepad.connected,
id: shouldSpoofGamepad(gamepad)
? XBOX_GAMEPAD_ID
: String(gamepad.id || ''),
index: typeof gamepad.index === 'number' ? gamepad.index : 0,
mapping: shouldSpoofGamepad(gamepad) ? 'standard' : (gamepad.mapping || ''),
timestamp:
typeof gamepad.timestamp === 'number' ? gamepad.timestamp : Date.now(),
};
}
function shouldSpoofGamepad(gamepad) {
if (!gamepad || !gamepad.connected) {
return false;
}
var id = String(gamepad.id || '');
return !(id.indexOf('Xbox') !== -1 && gamepad.mapping === 'standard');
}

function normalizeGamepad(gamepad) {
if (!shouldSpoofGamepad(gamepad)) {
return gamepad;
}
if (proxyCache && proxyCache.has(gamepad)) {
return proxyCache.get(gamepad);
}
var wrapped = null;
try {
wrapped = new Proxy(gamepad, {
get: function(target, prop, receiver) {
if (prop === 'id') return XBOX_GAMEPAD_ID;
if (prop === 'mapping') return 'standard';
if (prop === 'toJSON') {
return function() {
return serializeGamepad(target);
};
};
}
return Reflect.get(target, prop, receiver);
},
ownKeys: function(target) {
var keys = Reflect.ownKeys(target);

```

```
if (keys.indexOf('id') === -1) keys.push('id');
if (keys.indexOf('mapping') === -1) keys.push('mapping');
return keys;
},
getOwnPropertyDescriptor: function(target, prop) {
  if (prop === 'id') {
    return {
      configurable: true,
      enumerable: true,
      value: XBOX_GAMEPAD_ID,
    };
  }
  if (prop === 'mapping') {
    return {
      configurable: true,
      enumerable: true,
      value: 'standard',
    };
  }
  return (
    Object.getOwnPropertyDescriptor(target, prop) || {
      configurable: true,
      enumerable: true,
      value: Reflect.get(target, prop),
    }
  );
},
});
} catch (e) {
  wrapped = serializeGamepad(gamepad);
}
if (proxyCache) {
  proxyCache.set(gamepad, wrapped);
}
return wrapped;
}
function remapGamepadArray(gamepads) {
  var result = [];
  for (var i = 0; i < gamepads.length; i += 1) {
    result[i] = gamepads[i] ? normalizeGamepad(gamepads[i]) : null;
  }
  return result;
}
function overrideNavigatorMethod(name, implementation) {
  if (!name || typeof implementation !== 'function') {
    return false;
  }
  try {
    Object.defineProperty(navigator, name, {
      configurable: true,
      writable: true,
      value: implementation,
    });
    return true;
  } catch (e) {}
  try {
    var proto = Object.getPrototypeOf(navigator);
    if (proto) {
      Object.defineProperty(proto, name, {
        configurable: true,
        writable: true,
```

```
value: implementation,
});
return true;
}
} catch (e) {}
try {
navigator[name] = implementation;
return navigator[name] === implementation;
} catch (e) {}
return false;
}
function installGamepadOverride() {
if (originalGetGamepads) {
overrideNavigatorMethod('getGamepads', function() {
return remapGamepadArray(originalGetGamepads() || []);
});
}
if (originalWebkitGetGamepads) {
overrideNavigatorMethod('webkitGetGamepads', function() {
return remapGamepadArray(originalWebkitGetGamepads() || []);
});
}
}
function getConnectedGamepads() {
if (typeof navigator.getGamepads !== 'function') {
return [];
}
var pads = navigator.getGamepads() || [];
var result = [];
for (var i = 0; i < pads.length; i += 1) {
var pad = pads[i];
if (pad && pad.connected) {
result.push(pad);
}
}
return result;
}
function getPadSignature(gamepad) {
if (!gamepad) return '';
return [
gamepad.index,
gamepad.id,
gamepad.mapping,
gamepad.connected ? '1' : '0',
].join(':');
}
function getPadsSignature(gamepads) {
return (gamepads || []).map(getPadSignature).join('|');
}
function makeGamepadEvent(type, gamepad) {
try {
return new GamepadEvent(type, { gamepad: gamepad });
} catch (e) {}
try {
var evt = new Event(type);
evt.gamepad = gamepad;
return evt;
} catch (inner) {}
return null;
}
```

```

}
}
function dispatchGamepadEvent(type, gamepad) {
  try {
    var windowEvt = makeGamepadEvent(type, gamepad);
    windowEvt && window.dispatchEvent(windowEvt);
  } catch (e) {}
  try {
    var documentEvt = makeGamepadEvent(type, gamepad);
    documentEvt && document.dispatchEvent(documentEvt);
  } catch (e) {}
}
function dispatchReconnect(gamepad, reason) {
  if (!gamepad) return;
  state.reconnects += 1;
  state.lastReason = reason;
  var normalizedPad = normalizeGamepad(gamepad);
  dispatchGamepadEvent('gamepaddisconnected', normalizedPad);
  dispatchGamepadEvent('gamepadconnected', normalizedPad);
}
function resyncGamepads(reason) {
  safeFocus();
  var pads = getConnectedGamepads();

  state.lastResyncAt = Date.now();
  state.lastCount = pads.length;
  state.lastPadIds = pads.map(function(pad) { return pad.id; });
  state.lastSignature = getPadsSignature(pads);
  for (var i = 0; i < pads.length; i += 1) {
    dispatchReconnect(pads[i], reason);
  }
}
function periodicScan() {
  var pads = getConnectedGamepads();
  var signature = getPadsSignature(pads);
  state.lastCount = pads.length;
  state.lastPadIds = pads.map(function(pad) { return pad.id; });
  if (signature && signature !== state.lastSignature) {
    resyncGamepads('periodic-change');
    return;
  }
  if (
    pads.length &&
    (!state.lastResyncAt || Date.now() - state.lastResyncAt >= 5000)
  ) {
    resyncGamepads('periodic-refresh');
  }
}
function patchListenerRegistration(target) {
  if (!target || typeof target.addEventListener !== 'function') {
    return;
  }
  var originalAddEventListener = target.addEventListener.bind(target);
  target.addEventListener = function(type, listener, options) {
    var result = originalAddEventListener(type, listener, options);
    if (type === 'gamepadconnected' || type === 'gamepaddisconnected') {
      state.listenerRegistrations += 1;
      window.setTimeout(function() {
        resyncGamepads('listener-' + type);
      }, 0);
    }
  }
}

```

```

return result;
};
}
patchListenerRegistration(window);
patchListenerRegistration(document);
spoofBrowserIdentity();
window.addEventListener('focus', function() {
window.setTimeout(function() { resyncGamepads('focus'); }, 50);
});
document.addEventListener('visibilitychange', function() {
if (!document.hidden) {
window.setTimeout(function() { resyncGamepads('visibility'); }, 50);
}
});
installGamepadOverride();
window.setTimeout(function() { resyncGamepads('startup-1s'); }, 1000);
window.setTimeout(function() { resyncGamepads('startup-3s'); }, 3000);
window.setInterval(periodicScan, 1000);
})();
"""
def get_xcloud_browser_shims_js() -> str:
    """Get xcloud browser shims js."""
    return _XCLOUD_BROWSER_SHIMS_JS
def get_xcloud_navigation_js(target_url: str) -> str:
    """Get xcloud navigation js."""
    if not target_url:
        return ""
    encoded_target = json.dumps(target_url)
    return f"""
    (function() {{
    'use strict';
    var targetUrl = {encoded_target};
    if (!targetUrl) return;
    window.__unifideck_xcloud_target_url = targetUrl;
    if (window.location.href === targetUrl) return;
    window.setTimeout(function() {{
    try {{
    if (window.location.href !== targetUrl) {{
    window.location.assign(targetUrl);
    }}
    }} catch (e) {{}}
    }}, 250);
    }}})();
    """

```


OP-30d

page_inject.py

Fichier : py_modules/unifideck/cdp/page_inject.py | Lignes : 183 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import contextlib
import json
import logging
from typing import Any
import aiohttp
logger = logging.getLogger(__name__)
async def list_page_targets(
    port: int,
    *,
    timeout: float = 3.0,
) -> list[dict[str, Any]]:
    """List page targets."""
    url = f"http://127.0.0.1:{port}/json"
    async with aiohttp.ClientSession() as session, session.get(
        url,
        timeout=aiohttp.ClientTimeout(total=timeout),
    ) as response:
        response.raise_for_status()
        payload = await response.json(content_type=None)
    if not isinstance(payload, list):
        return []
    return [item for item in payload if isinstance(item, dict)]
def _target_url_matches(
    target: dict[str, Any], patterns: list[str],
) -> bool:
    """Target URL matches."""
    url = str(target.get("url", ""))
    return any(pattern and pattern in url for pattern in patterns)
_CLOSE_MSG_TYPES = (
    aiohttp.WSMsgType.CLOSED,
    aiohttp.WSMsgType.CLOSING,
    aiohttp.WSMsgType.ERROR,
)
async def _drain_until_reply(
    websocket: aiohttp.ClientWebSocketResponse,
    msg_id: int,
    ws_timeout: float,
    logger_prefix: str,
) -> bool:
    """Drain until reply."""
    while True:
        message = await websocket.receive(timeout=ws_timeout)
        if message.type in _CLOSE_MSG_TYPES:
            logger.debug(
                "[%s] websocket closed during inject",
                logger_prefix,
            )
            return False
        if message.type != aiohttp.WSMsgType.TEXT:
            continue
        payload = json.loads(message.data)
        if payload.get("id") != msg_id:
            continue
```

```

        if "error" in payload:
            logger.debug(
                "[%s] Runtime.evaluate error: %s",
                logger_prefix, payload["error"],
            )
            return False
        return True

async def _inject_into_target(
    target: dict[str, Any],
    sources: list[str],
    *,
    ws_timeout: float,
    logger_prefix: str,
) -> bool:

    """Inject into target."""
    ws_url = target.get("websocketDebuggerUrl")
    if not isinstance(ws_url, str) or not ws_url:
        return False
    msg_id = 0
    try:
        async with aiohttp.ClientSession() as session, session.ws_connect(
            ws_url,
            heartbeat=10,
            autoping=True,
            timeout=aiohttp.ClientTimeout(total=ws_timeout),
        ) as websocket:
            for source in sources:
                if not source:
                    continue
                msg_id += 1
                await websocket.send_json({
                    "id": msg_id,
                    "method": "Runtime.evaluate",
                    "params": {
                        "expression": source,
                        "awaitPromise": True,
                        "returnByValue": True,
                        "userGesture": True,
                    },
                })
                if not await _drain_until_reply(
                    websocket, msg_id, ws_timeout, logger_prefix,
                ):
                    return False
            return True
    except (TimeoutError, aiohttp.ClientError, OSError) as exc:
        logger.debug(
            "[%s] inject into %s failed: %s",
            logger_prefix, target.get("id"), exc,
        )
        return False

async def inject_scripts(
    port: int,
    sources: list[str],
    *,
    url_patterns: list[str],
    timeout: float = 45.0,
    logger_prefix: str = "cdp-inject",

```

```

    poll_delay: float = 0.5,
) -> bool:

    """Inject scripts."""
    if not sources or not url_patterns:
        return False
    deadline = asyncio.get_running_loop().time() + timeout
    injected_once = False
    while asyncio.get_running_loop().time() < deadline:
        try:
            targets = await list_page_targets(port, timeout=3.0)
        except (TimeoutError, aiohttp.ClientError, OSError) as exc:
            logger.debug(
                "[%s] list_page_targets failed: %s",
                logger_prefix, exc,
            )
            await asyncio.sleep(poll_delay)
            continue
        page_targets = [
            t for t in targets
            if t.get("type") == "page"
            and _target_url_matches(t, url_patterns)
        ]
        if not page_targets:
            await asyncio.sleep(poll_delay)
            continue
        all_ok, had_success = await _inject_into_matching_targets(
            page_targets, sources, timeout, logger_prefix,
        )
        if had_success:
            injected_once = True
            if injected_once and all_ok:
                return True
            await asyncio.sleep(poll_delay)
        if injected_once:
            return True
        logger.warning(
            "[%s] timed out waiting for matching page (patterns=%r)",
            logger_prefix, url_patterns,
        )
    return False

async def _inject_into_matching_targets(
    page_targets: list[dict[str, Any]],
    sources: list[str],
    timeout: float,
    logger_prefix: str,
) -> tuple[bool, bool]:
    """Inject into matching targets."""
    all_ok = True
    had_success = False
    for target in page_targets:
        ok = await _inject_into_target(
            target,
            sources,
            ws_timeout=min(15.0, timeout),
            logger_prefix=logger_prefix,
        )
        if ok:
            had_success = True
            logger.info(
                "[%s] injected %d script(s) into %s",

```

```
        logger_prefix, len(sources),
        target.get("url", "?"),
    )
    else:
        all_ok = False
    return all_ok, had_success
@contextlib.asynccontextmanager
async def _session_timeout(total: float):
    """Session timeout."""
    yield aiohttp.ClientTimeout(total=total)
```

OP-31

__init__.py

Fichier : py_modules/unifideck/metadata/__init__.py | Lignes : 0 | Depend de : (rien)

OP-31a

metacritic.py

Fichier : py_modules/unifideck/metadata/metacritic.py | Lignes : 201 | Depend de : (rien)

```

from __future__ import annotations
import logging
import re
import unicodedata
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any, cast
from ..utils.config_helpers import get_cfg
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
METACRITIC_COMPOSER_URL = (
    "https://backend.metacritic.com/composer/metacritic/pages/"
    "games-critic-reviews/{slug}/platform/pc/web"
)
DEFAULT_FETCH_TIMEOUT = 10
_EDITION_SUFFIXES = [
    r"?\s*Director's Cut",
    r"?\s*Game of the Year Edition",
    r"?\s*GOTY Edition",
    r"?\s*Remastered",
    r"?\s*Definitive Edition",
    r"?\s*Bonus Edition",
    r"?\s*Deluxe Edition",
    r"?\s*Special Edition",
    r"?\s*Anniversary Edition",
    r"?\s*Complete Edition",
    r"?\s*Ultimate Edition",
    r"?\s*Gold Edition",
    r"?\s*Enhanced Edition",
]
_ROMAN_MAP = {"1": "I", "2": "II", "3": "III", "4": "IV", "5": "V"}
_ARABIC_MAP = {v: k for k, v in _ROMAN_MAP.items()}
def slugify_game_name(name: str) -> str:
    """Slugify game name."""
    name = (
        unicodedata.normalize("NFKD", name)
        .encode("ascii", "ignore")
        .decode("utf-8")
        .lower()
    )
    name = name.replace("+", "-plus-")
    name = re.sub(r"^[a-z0-9 \-]+", "", name)
    name = re.sub(r"\s+", "-", name)
    name = re.sub(r"-+", "-", name)
    return name.strip("-")
def clean_title(title: str) -> str:
    """Clean title."""
    return re.sub(r"[\u2122\u00AE]", "", title).strip()
def strip_suffixes(title: str) -> str:
    """Strip suffixes."""
    cleaned = title
    for suffix in _EDITION_SUFFIXES:
        cleaned = re.sub(suffix, "", cleaned, flags=re.IGNORECASE)
    return cleaned.strip()
def to_roman(num_str: str) -> str | None:

```

```

    """To roman."""
    return _ROMAN_MAP.get(num_str)
def to_arabic(roman_str: str) -> str | None:
    """To arabic."""
    return _ARABIC_MAP.get(roman_str)

def get_numeral_variants(title: str) -> list[str]:

    """Get numeral variants."""
    candidates: list[str] = []
    m = re.search(r"\b(I|II|III|IV|V)$", title)
    if m:
        arabic = to_arabic(m.group(1))
        if arabic:
            candidates.append(title[:m.start()] + arabic)
    m = re.search(r"\b([1-5])$", title)
    if m:
        roman = to_roman(m.group(1))
        if roman:
            candidates.append(title[:m.start()] + roman)
    def _arabic_to_roman(match: re.Match) -> str:
        """Arabic to roman."""
        return to_roman(match.group(1)) or match.group(0)
    def _roman_to_arabic(match: re.Match) -> str:
        """Roman to arabic."""
        return to_arabic(match.group(1)) or match.group(0)
    subbed = re.sub(r"\b([1-5])\b", _arabic_to_roman, title)
    if subbed != title:
        candidates.append(subbed)
    subbed = re.sub(r"\b(I|II|III|IV|V)\b", _roman_to_arabic, title)
    if subbed != title:
        candidates.append(subbed)
    return list(dict.fromkeys(candidates))
def sanitize_description(text: str, max_length: int = 1000) -> str:
    """Sanitize description."""
    text = re.sub(r"\s+", " ", text).strip()
    if len(text) > max_length:
        return text[: max_length - 1].rstrip() + "..."
    return text
@dataclass
class MetacriticScore:
    """Metacritic score."""
    title: str
    slug: str
    metascore: int | None
    user_score: float | None
    description: str
    url: str
    def to_dict(self) -> dict:
        """To dict."""
        return {
            "title": self.title,
            "slug": self.slug,
            "metascore": self.metascore,
            "user_score": self.user_score,
            "description": self.description,
            "url": self.url,
        }
    async def fetch_score(
        title: str, config: ConfigManager | None = None,
    ) -> MetacriticScore | None:

```

```

    """Fetch score."""
    composer_url = get_cfg(
        config, "metadata.metacritic.composer_url",
        "https://backend.metacritic.com/composer/metacritic/"
        "pages/games/{slug}/web",
    )
    timeout = get_cfg(
        config, "metadata.metacritic.fetch_timeout_seconds", 15,
    )
    candidates = _slug_candidates(title)
    logger.debug("[metacritic] %d slug candidates for %r",
        len(candidates), title)
    for slug in candidates:
        data = await _fetch_composer(slug, composer_url, timeout)
        if data is None:
            continue
        score = _parse_composer_response(title, slug, data)
        if score is not None:
            return score
    return None
def _cfg(config: ConfigManager | None, key: str, default: Any) -> Any:
    """Cfg."""
    return get_cfg(config, key, default)

def _slug_candidates(title: str) -> list[str]:
    """Slug candidates."""
    variants = {title}
    cleaned = clean_title(title)
    variants.add(cleaned)
    variants.add(strip_suffixes(cleaned))
    for v in list(variants):
        for alt in get_numeral_variants(v):
            variants.add(alt)
    return [slugify_game_name(v) for v in variants if v.strip()]
async def _fetch_composer(
    slug: str, url_template: str, timeout: int,
) -> dict | None:
    """Fetch composer."""
    import aiohttp
    url = url_template.format(slug=slug)
    try:
        async with (
            aiohttp.ClientSession() as session,
            session.get(url, timeout=timeout) as resp,
        ):
            if resp.status != 200:
                return None
            return cast("dict[Any, Any] | None", await resp.json())
    except Exception as e:
        logger.debug(
            "[metacritic] fetch(%s) failed: %s", slug, e,
        )
        return None
def _parse_composer_response(
    title: str, slug: str, data: dict,
) -> MetacriticScore | None:
    """Parse composer response."""
    try:
        components = data.get("components", [])
        game_info = next(

```



```
        (
            c for c in components
            if c.get("type") == "gameInfo"
        ),
        None,
    )
    if not game_info:
        return None
    payload = game_info.get("data", {}).get("item", {})
    if not payload:
        return None
    return MetacriticScore(
        title=title,
        slug=slug,
        metascore=payload.get(
            "criticScoreSummary", {}
        ).get("score"),
        user_score=payload.get(
            "userScoreSummary", {}
        ).get("score"),
        description=sanitize_description(
            payload.get("description", "")
        ),
        url=f"https://www.metacritic.com/game/{slug}/",
    )
except (AttributeError, TypeError, KeyError) as e:
    logger.debug("[metacritic] parse error: %s", e)
    return None
```

OP-31b

unifidb.py

Fichier : py_modules/unifideck/metadata/unifidb.py | Lignes : 173 | Depend de : (rien)

```

from __future__ import annotations
import logging
import re
import string
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any
from ..utils.config_helpers import get_cfg
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
UNIFIDB_CDN_BASE = (
    "https://cdn.jsdelivr.net/gh/mubaraknumann/unifiDB@main"
)
MATCH_THRESHOLD = 0.65
def normalize_title_for_matching(title: str) -> str:
    """Normalize title for matching."""
    title = title.lower()
    title = re.sub(r"[\u2122\u00AE]", "", title)
    title = title.translate(
        str.maketrans("", "", string.punctuation),
    )
    title = re.sub(r"\s+", " ", title)
    return title.strip()
def get_first_char_for_bucket(title: str) -> str:
    """Get first char for bucket."""
    normalized = normalize_title_for_matching(title)
    for article in ("the ", "a ", "an "):
        if normalized.startswith(article):
            normalized = normalized[len(article):]
            break
    if not normalized:
        return "0_9"
    first = normalized[0]
    if not first.isalpha():
        return "0_9"
    second = (
        normalized[1]
        if len(normalized) > 1 and normalized[1].isalnum()
        else first
    )
    return f"{first}{second}"
def score_title_match(search: str, candidate: str) -> float:
    """Score title match."""
    a = normalize_title_for_matching(search)
    b = normalize_title_for_matching(candidate)
    if a == b:
        return 1.0
    if not a or not b:
        return 0.0
    if a in b or b in a:
        shorter = min(len(a), len(b))
        longer = max(len(a), len(b))
        if longer <= 2 * shorter:
            return 0.85
    tokens_a = set(a.split())

```

```

tokens_b = set(b.split())
if not tokens_a or not tokens_b:
    return 0.0
intersect = tokens_a & tokens_b
union = tokens_a | tokens_b
return 0.8 * (len(intersect) / len(union))

def extract_store_id(
    game: dict[str, Any], store: str,
) -> str | None:

    """Extract store ID."""
    external = game.get("external_ids") or {}
    if not isinstance(external, dict):
        return None
    val = external.get(store)
    return str(val) if val is not None else None

def get_best_match(
    search_title: str,
    candidates: list[dict[str, Any]],
    threshold: float = MATCH_THRESHOLD,
) -> dict[str, Any] | None:
    """Get best match."""
    if not candidates:
        return None
    scored: list[tuple[float, dict[str, Any]]] = []
    for c in candidates:
        name = c.get("title") or c.get("name") or ""
        score = score_title_match(search_title, name)
        if score >= threshold:
            scored.append((score, c))
    if not scored:
        return None
    scored.sort(key=lambda x: x[0], reverse=True)
    return scored[0][1]

def game_to_cache_format(
    game: dict[str, Any],
) -> dict[str, Any]:
    """Game to cache format."""
    return {
        "title": game.get("title") or game.get("name") or "",
        "description": game.get("description", ""),
        "release_date": game.get("release_date", ""),
        "publisher": game.get("publisher", ""),
        "developers": game.get("developers", []),
        "genres": game.get("genres", []),
        "platforms": game.get("platforms", []),
        "external_ids": game.get("external_ids", {}),
    }

@dataclass
class UnifiDBResult:
    """Unifi dbresult."""
    title: str
    description: str
    release_date: str
    publisher: str
    developers: list[str]
    genres: list[str]
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        return {

```

```

        "title": self.title,
        "description": self.description,
        "release_date": self.release_date,
        "publisher": self.publisher,
        "developers": self.developers,
        "genres": self.genres,
    }

async def lookup(
    store: str, game_id: str, title: str,
    config: ConfigManager | None = None,
) -> dict[str, Any] | None:
    """Lookup."""
    cdn_base = get_cfg(
        config, "metadata.unifidb.cdn_base", UNIFIDB_CDN_BASE,
    )
    threshold = get_cfg(
        config, "metadata.unifidb.match_threshold", MATCH_THRESHOLD,
    )
    timeout = get_cfg(
        config, "metadata.unifidb.fetch_timeout_seconds", 15,
    )
    bucket = get_first_char_for_bucket(title)
    games = await _fetch_bucket(bucket, cdn_base, timeout)
    if not games:
        return None
    for game in games:
        if extract_store_id(game, store) == game_id:
            logger.debug(
                "[unifidb] id match: %s:%s", store, game_id,
            )
            return game_to_cache_format(game)
    best = get_best_match(title, games, threshold)
    if best:
        logger.debug("[unifidb] title match: %r", title)
        return game_to_cache_format(best)
    return None

async def _fetch_bucket(
    bucket: str, cdn_base: str, timeout: int,
) -> list[dict[str, Any]]:
    """Fetch bucket."""
    import aiohttp
    first_char = bucket[0] if bucket else "0_9"
    url = f"{cdn_base}/games/{first_char}/{bucket}.json"
    try:
        async with (
            aiohttp.ClientSession() as session,
            session.get(url, timeout=timeout) as resp,
        ):
            if resp.status != 200:
                return []
            data = await resp.json()
            if isinstance(data, list):
                return data
            return []
    except Exception as e:
        logger.debug(
            "[unifidb] fetch(%s) failed: %s", url, e,
        )
    return []

```

OP-32

`__init__.py`

Fichier : `py_modules/unifideck/steam/__init__.py` | Lignes : 13 | Depend de : (rien)

```
from .library import find_steam_path, search_store
from .steamgriddb import (
    SteamGridDBClient,
    fetch_all_kinds,
    search_artwork,
)
try:
    from .steam_utils import (
        get_logged_in_steam_user,
        migrate_user0_to_logged_in_user,
    )
except ImportError:
    pass
```

OP-32a

steamgriddb.py

Fichier : py_modules/unifideck/steam/steamgriddb.py | Lignes : 180 | Depend de : (rien)

```
from __future__ import annotations
import logging
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any
from unifideck.utils.config_helpers import get_cfg
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
SGDB_API_BASE = "https://www.steamgriddb.com/api/v2"
ARTWORK_KINDS = {
    "grid": ("grids", ["600x900"]),
    "hero": ("heroes", ["1920x620", "3840x1240"]),
    "logo": ("logos", None),
    "icon": ("icons", None),
}
@dataclass
class ArtworkAsset:
    """Artwork asset."""
    url: str
    width: int
    height: int
    style: str
    mime: str
    game_id: int
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        return {
            "url": self.url,
            "width": self.width,
            "height": self.height,
            "style": self.style,
            "mime": self.mime,
            "game_id": self.game_id,
        }
async def search_artwork(
    title: str,
    kind: str,
    api_key: str | None = None,
    config: ConfigManager | None = None,
) -> str | None:
    """Search artwork."""
    if kind not in ARTWORK_KINDS:
        raise ValueError(f"unknown artwork kind: {kind}")
    if not api_key:
        return None
    base = get_cfg(
        config, "artwork.steamgriddb_api_base",
        "https://www.steamgriddb.com/api/v2",
    )
    timeout = get_cfg(config, "artwork.download_timeout_seconds", 30)
    game = await _search_game(title, api_key, base, timeout)
    if game is None:
        return None
    endpoint, _dimensions = ARTWORK_KINDS[kind]
    assets = await _fetch_assets(
```

```

        game["id"], endpoint, api_key, base, timeout,
    )
    best = _pick_best_asset(assets)
    return best.url if best else None

async def fetch_all_kinds(
    title: str,
    api_key: str | None,
    config: ConfigManager | None = None,
) -> dict[str, str | None]:

    """Fetch all kinds."""
    if not api_key:
        return dict.fromkeys(ARTWORK_KINDS)
    base = get_cfg(
        config, "artwork.steamgriddb_api_base",
        "https://www.steamgriddb.com/api/v2",
    )
    timeout = get_cfg(config, "artwork.download_timeout_seconds", 30)
    game = await _search_game(title, api_key, base, timeout)
    if game is None:
        return dict.fromkeys(ARTWORK_KINDS)
    game_id = game["id"]
    results: dict[str, str | None] = {}
    for kind, (endpoint, _dims) in ARTWORK_KINDS.items():
        assets = await _fetch_assets(
            game_id, endpoint, api_key, base, timeout,
        )
        best = _pick_best_asset(assets)
        results[kind] = best.url if best else None
    return results

def _cfg(config: ConfigManager | None, key: str, default: Any) -> Any:
    """Cfg."""
    return get_cfg(config, key, default)

async def _search_game(
    title: str, api_key: str, base: str, timeout: int,
) -> dict[str, Any] | None:
    """Search game."""
    import aiohttp
    url = f"{base}/search/autocomplete/{title}"
    headers = {"Authorization": f"Bearer {api_key}"}
    try:
        async with (
            aiohttp.ClientSession() as session,
            session.get(
                url, headers=headers, timeout=timeout,
            ) as resp,
        ):
            if resp.status != 200:
                logger.debug(
                    "[sgdb] search(%s) → HTTP %d",
                    title, resp.status,
                )
                return None
            payload = await resp.json()
    except Exception as e:
        logger.debug(
            "[sgdb] search(%s) failed: %s", title, e,
        )
        return None
    if not payload.get("success"):

```

```

        return None
    data = payload.get("data") or []
    return data[0] if data else None

async def _fetch_assets(
    game_id: int, endpoint: str, api_key: str,
    base: str, timeout: int,
) -> list[ArtworkAsset]:

    """Fetch assets."""
    import aiohttp
    url = f"{base}/{endpoint}/game/{game_id}"
    headers = {"Authorization": f"Bearer {api_key}"}
    try:
        async with (
            aiohttp.ClientSession() as session,
            session.get(
                url, headers=headers, timeout=timeout,
            ) as resp,
        ):
            if resp.status != 200:
                return []
            payload = await resp.json()
    except Exception as e:
        logger.debug(
            "[sgdb] fetch(%d, %s) failed: %s",
            game_id, endpoint, e,
        )
        return []
    if not payload.get("success"):
        return []
    results: list[ArtworkAsset] = []
    for item in payload.get("data", []):
        try:
            results.append(ArtworkAsset(
                url=item["url"],
                width=item.get("width", 0),
                height=item.get("height", 0),
                style=item.get("style", ""),
                mime=item.get("mime", "image/png"),
                game_id=game_id,
            ))
        except KeyError:
            continue
    return results

def _pick_best_asset(
    assets: list[ArtworkAsset],
) -> ArtworkAsset | None:
    """Pick best asset."""
    if not assets:
        return None
    def rank(asset: ArtworkAsset) -> tuple:
        """Rank."""
        style_rank = 1 if asset.style == "alternate" else 0
        res = asset.width * asset.height
        return (style_rank, res)
    return max(assets, key=rank)

class SteamGridDBClient:
    """Steam grid dbclient."""
    def __init__(self, api_key=None):
        """Initialize the instance."""

```



```
        self.api_key = api_key
    async def search_artwork(self, title, kind, **kwargs):
        """Search artwork."""
        return await search_artwork(title, kind, self.api_key)
    async def fetch_all_kinds(self, title, **kwargs):
        """Fetch all kinds."""
        return await fetch_all_kinds(title, self.api_key)
```

OP-32b

library.py

Fichier : py_modules/unifideck/steam/library.py | Lignes : 151 | Depend de : (rien)

```

from __future__ import annotations
import logging
import re
from dataclasses import dataclass
from pathlib import Path
from typing import TYPE_CHECKING, Any
from ..utils.config_helpers import get_cfg
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
STEAM_PATH_CANDIDATES = (
    "~/steam/steam",
    "~/local/share/Steam",
    "~/var/app/com.valvesoftware.Steam/steam/steam",
)
STEAM_STORE_SEARCH_URL = (
    "https://store.steampowered.com/api/storesearch"
)
def find_steam_path(config: ConfigManager | None = None) -> str | None:
    """Find steam path."""
    candidates = get_cfg(
        config, "paths.steam_candidates", list(STEAM_PATH_CANDIDATES),
    )
    for candidate in candidates:
        expanded = Path(candidate).expanduser()
        if (expanded / "steamapps").is_dir():
            return str(expanded)
    return None
def find_grid_path(
    steam_path: str | None = None,
    config: ConfigManager | None = None,
) -> str | None:
    """Find grid path."""
    steam = steam_path or find_steam_path(config)
    if not steam:
        return None
    user_id = _find_most_recent_user(steam)
    if not user_id:
        return None
    grid_dir = (
        Path(steam) / "userdata" / user_id / "config" / "grid"
    )
    try:
        grid_dir.mkdir(parents=True, exist_ok=True)
    except OSError as e:
        logger.warning(
            "[steam/library] grid mkdir failed: %s", e,
        )
        return None
    return str(grid_dir)
def find_shortcuts_vdf(
    steam_path: str | None = None,
    config: ConfigManager | None = None,
) -> str | None:
    """Find shortcuts VDF."""

```

```

steam = steam_path or find_steam_path(config)
if not steam:
    return None
user_id = _find_most_recent_user(steam)
if not user_id:
    return None
return str(
    Path(steam) / "userdata" / user_id
    / "config" / "shortcuts.vdf",
)

def _cfg(config: ConfigManager | None, key: str, default: Any) -> Any:
    """Cfg."""
    return get_cfg(config, key, default)
def _find_most_recent_user(steam_path: str) -> str | None:
    """Find most recent user."""
    loginusers = (
        Path(steam_path) / "config" / "loginusers.vdf"
    )
    if not loginusers.is_file():
        return None
    try:
        text = loginusers.read_text(encoding="utf-8")
    except OSError:
        return None
    pattern = re.compile(
        r'(\d{17})\s*\{[^}]*"MostRecent"\s*"1"',
        re.DOTALL,
    )
    m = pattern.search(text)
    return m.group(1) if m else None
@dataclass
class SteamStoreResult:
    """Steam store result."""
    app_id: int
    name: str
    header_image: str
    price: str
    release_date: str
    def to_dict(self) -> dict:
        """To dict."""
        return {
            "app_id": self.app_id,
            "name": self.name,
            "header_image": self.header_image,
            "price": self.price,
            "release_date": self.release_date,
        }
async def search_store(
    title: str, config: ConfigManager | None = None,
) -> dict | None:
    """Search store."""
    import aiohttp
    url = get_cfg(
        config, "metadata.steam_store.search_url",
        STEAM_STORE_SEARCH_URL,
    )
    timeout = get_cfg(
        config, "metadata.steam_store.search_timeout_seconds", 15,
    )

```

```
params = {"term": title, "l": "english", "cc": "US"}
try:
    async with (
        aiohttp.ClientSession() as session,
        session.get(
            url, params=params, timeout=timeout,
        ) as resp,
    ):
        if resp.status != 200:
            return None
        data = await resp.json()
except Exception as e:
    logger.debug(
        "[steam/library] search(%s) failed: %s",
        title, e,
    )
    return None
items = data.get("items") or []
if not items:
    return None
item = items[0]
price = ""
if isinstance(item.get("price"), dict):
    price = item["price"].get("final", "")
return SteamStoreResult(
    app_id=int(item.get("id", 0)),
    name=item.get("name", ""),
    header_image=item.get("tiny_image", ""),
    price=str(price),
    release_date="",
).to_dict()
```

```
async def batch_search_store(titles: list[str]) -> dict:
```

```
    """Batch search store."""
    results = {}
    for title in titles:
        results[title] = await search_store(title)
    return results
```

OP-32c

shortcuts.py

Fichier : py_modules/unifideck/steam/shortcuts.py | Lignes : 150 | Depend de : (rien)

```

"""steam/shortcuts.py – Steam shortcuts.vdf I/O utilities.
Moved from shortcuts/vdf.py and renamed. The old
location was a solo-file subpackage whose identity was unclear
(was it "all shortcut management" or "just VDF codec"?). The
new home makes the answer explicit: this is **Steam's**
shortcuts file format, handled alongside the other Steam-client
interactions (library scan, SteamGridDB artwork). The former
filename `vdf.py` was also misleading – this module is NOT a
generic VDF codec; all 4 public functions are hardcoded to the
shortcuts.vdf layout (load_shortcuts_vdf, read_shortcuts,
save_shortcuts_vdf, write_shortcuts).
Thin wrapper around the external `vdf` library with safer
I/O semantics:
- Atomic write-then-rename so Steam never reads a half-written
  file
- Automatic `.backup` before overwriting (for rollback)
- Write validation: re-parse the file after writing and compare
  shortcut counts; restore the backup if validation fails
- Structured logging instead of `print` statements
- Functions are synchronous – callers should wrap them in
  `asyncio.to_thread()` (done by ShortcutService via AsyncFileOps)
The legacy module used `print()` for errors and didn't do atomic
writes, which created a small window where `shortcuts.vdf` could
be read by Steam mid-write and corrupt the library. This version
eliminates that race condition.
Reference: Technical Document v1.0 – Section 3.6.2 (shortcuts.vdf
format), Figure 23.
"""

from __future__ import annotations
import logging
import os
import shutil
from typing import Any, cast

try:
    import vdf
except ImportError:
    vdf = None
    logger = logging.getLogger(__name__)

class VDFError(Exception):
    """Raised when VDF parsing or writing fails."""

def load_shortcuts_vdf(path: str) -> dict[str, Any]:
    """Load and parse a shortcuts.vdf file.
    Returns an empty `{"shortcuts": {}}` structure if the file does
    not exist (Steam's behavior on first plugin run). Raises
    VDFError if the file exists but cannot be parsed.
    """
    if vdf is None:
        raise VDFError("vdf library not installed")
    if not os.path.isfile(path):
        return {"shortcuts": {}}
    try:
        with open(path, "rb") as f:
            return cast("dict[str, Any]", vdf.binary_loads(f.read()))
    except Exception as e:
        raise VDFError(f"failed to parse {path}: {e}") from e

```

```

def read_shortcuts(path: str) -> list[dict[str, Any]]:
    """Convenience: return the list of shortcut entries.
    The raw VDF structure is `{"shortcuts": {"0": {...}, "1": {...}}`
    where keys are string-indexed. ShortcutService prefers working
    with a flat list, so we flatten here and renumber on save.
    """
    data = load_shortcuts_vdf(path)
    raw = data.get("shortcuts", {})
    if isinstance(raw, dict):
        # Preserve insertion order using the numeric string keys
        return [raw[k] for k in sorted(raw.keys(), key=_sort_key)]
    return []

def save_shortcuts_vdf(path: str, data: dict[str, Any]) -> None:
    """Write the full shortcuts.vdf structure atomically.
    Performs a 3-step write: backup → write to ``.tmp`` → rename.
    This guarantees Steam never sees a partial file even if the
    process is killed mid-write.
    Also validates the write by re-reading the file and comparing
    the shortcut count; restores from backup if validation fails.
    Raises VDFError on any failure.
    """
    if vdf is None:
        raise VDFError("vdf library not installed")
    expected_count = len(data.get("shortcuts", {}))
    tmp_path = f"{path}.tmp"
    backup_path = f"{path}.backup"
    # Step 1: back up the current file (if any)
    if os.path.isfile(path):
        try:
            shutil.copy2(path, backup_path)
        except OSError as e:
            raise VDFError(f"backup failed: {e}") from e
    # Step 2: write to a temporary file
    try:
        with open(tmp_path, "wb") as f:
            f.write(vdf.binary_dumps(data))
            f.flush()
            os.fsync(f.fileno())
    except Exception as e:
        _cleanup_tmp(tmp_path)
        raise VDFError(f"write failed: {e}") from e
    # Step 3: atomic rename
    try:
        os.replace(tmp_path, path)
    except OSError as e:
        _cleanup_tmp(tmp_path)
        raise VDFError(f"rename failed: {e}") from e
    # Step 4: validate the write
    try:
        validation = load_shortcuts_vdf(path)
        actual_count = len(validation.get("shortcuts", {}))
    except VDFError as e:
        _restore_backup(backup_path, path)
        raise VDFError(f"validation read failed: {e}") from e
    if actual_count != expected_count:
        _restore_backup(backup_path, path)
        raise VDFError(
            f"validation count mismatch: expected "
            f"{expected_count}, got {actual_count} – backup "
            f"restored",

```

```

    )
    logger.info(
        "[vdf] wrote %d shortcuts to %s", actual_count, path,
    )
def write_shortcuts(path: str, shortcuts: list[dict[str, Any]]) -> None:
    """Convenience: serialize a flat list of shortcut entries.
    Renumbers the entries as string keys ("0", "1", "2", ...) which
    is the format Steam expects. Delegates the actual write to
    `save_shortcuts_vdf()`.
    """
    data = {"shortcuts": {str(i): entry
        for i, entry in enumerate(shortcuts)}}
    save_shortcuts_vdf(path, data)

# — Helpers —————
def _sort_key(k: str) -> int:
    """Sort key for shortcut dict keys (numeric strings)."""
    try:
        return int(k)
    except ValueError:
        return 999999 # non-numeric keys go to the end
def _cleanup_tmp(tmp_path: str) -> None:
    """Remove a leftover .tmp file, ignoring errors."""
    try:
        os.remove(tmp_path)
    except OSError:
        # best-effort cleanup; file may already be gone or locked
        pass
def _restore_backup(backup_path: str, path: str) -> None:
    """Restore the backup file over the target (best-effort)."""
    if os.path.isfile(backup_path):
        try:
            shutil.copy2(backup_path, path)
            logger.warning("[vdf] restored backup to %s", path)
        except OSError as e:
            logger.error("[vdf] backup restore failed: %s", e)

```

OP-32d

owned_games.py

Fichier : py_modules/unifideck/steam/owned_games.py | Lignes : 113 | Depend de : (rien)

```

from __future__ import annotations
import logging
import re
from pathlib import Path
from typing import TYPE_CHECKING
from ..metadata.unifideb import normalize_title_for_matching
from .library import find_steam_path
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
_ACF_NAME_PATTERN = re.compile(r'"name"\s+([^\s]*)')
_LIBFOLDER_PATH_PATTERN = re.compile(r'"path"\s+([^\s]*)')
_Fingerprint = tuple[str, float | None, tuple[tuple[str, float | None], ...]]
_cache: tuple[_Fingerprint, frozenset[str]] | None = None
def get_owned_titles(
    config: ConfigManager | None = None,
) -> frozenset[str]:
    """Get owned titles."""
    global _cache
    steam_path = find_steam_path(config)
    if steam_path is None:
        logger.debug("[owned_games] no Steam install found")
        return frozenset()
    fingerprint = _compute_fingerprint(Path(steam_path))
    if _cache is not None and _cache[0] == fingerprint:
        return _cache[1]
    titles = _scan_all_libraries(Path(steam_path))
    logger.info(
        "[owned_games] indexed %d Steam-native title(s) from %s",
        len(titles), steam_path,
    )
    _cache = (fingerprint, titles)
    return titles
def invalidate_cache() -> None:
    """Invalidate cache."""
    global _cache
    _cache = None
def _compute_fingerprint(steam_path: Path) -> _Fingerprint:
    """Compute fingerprint."""
    libfolders_vdf = steam_path / "steamapps" / "libraryfolders.vdf"
    libfolders_mtime = _stat_mtime(libfolders_vdf)
    library_roots = _list_library_roots(steam_path)
    per_library: list[tuple[str, float | None]] = []
    for root in library_roots:
        steamapps = root / "steamapps"
        per_library.append((str(root), _stat_mtime(steamapps)))
    return (str(steam_path), libfolders_mtime, tuple(per_library))
def _stat_mtime(path: Path) -> float | None:
    """Stat mtime."""
    try:
        return path.stat().st_mtime
    except OSError:
        return None
def _scan_all_libraries(steam_path: Path) -> frozenset[str]:

```



```

"""Scan all libraries."""
titles: set[str] = set()
for library_root in _list_library_roots(steam_path):
    try:
        titles.update(_titles_from_library(library_root))
    except OSError as e:
        logger.warning(
            "[owned_games] could not scan %s: %s",
            library_root, e,
        )
return frozenset(titles)
def _list_library_roots(steam_path: Path) -> list[Path]:
    """List library roots."""
    roots: list[Path] = [steam_path]
    libfolders_vdf = steam_path / "steamapps" / "libraryfolders.vdf"
    if not libfolders_vdf.is_file():
        return roots
    try:
        content = libfolders_vdf.read_text(
            encoding="utf-8", errors="replace",
        )
    except OSError as e:
        logger.warning(
            "[owned_games] cannot read %s: %s", libfolders_vdf, e,
        )
    return roots
    for match in _LIBFOLDER_PATH_PATTERN.finditer(content):
        candidate = Path(match.group(1))
        if candidate == steam_path:
            continue
        if (candidate / "steamapps").is_dir():
            roots.append(candidate)
    return roots
def _titles_from_library(library_root: Path) -> set[str]:
    """Titles from library."""
    steamapps = library_root / "steamapps"
    if not steamapps.is_dir():
        return set()
    titles: set[str] = set()
    for manifest in steamapps.glob("appmanifest_*.acf"):
        title = _extract_name_from_manifest(manifest)
        if title:
            normalized = normalize_title_for_matching(title)
            if normalized:
                titles.add(normalized)
    return titles
def _extract_name_from_manifest(manifest: Path) -> str | None:
    """Extract name from manifest."""
    try:
        content = manifest.read_text(
            encoding="utf-8", errors="replace",
        )
    except OSError as e:
        logger.warning(
            "[owned_games] cannot read %s: %s", manifest, e,
        )
    return None
    match = _ACF_NAME_PATTERN.search(content)
    return match.group(1) if match else None

```

OP-33

`__init__.py`

Fichier : `py_modules/unifideck/utils/__init__.py` | Lignes : 12 | Depend de : (rien)

```
from .paths import (
    DEFAULT_GAMES_MAP,
    DEFAULT_INSTALL_DIRS,
    DEFAULT_PATHS,
    DEFAULT_SD_ROOT,
    GAMES_MAP_PATH,
    dedupe_paths,
    ensure_games_map_dir,
    expand,
    get_all_game_directories,
    get_games_map_path,
)
```

OP-33a

paths.py

Fichier : py_modules/unifideck/utils/paths.py | Lignes : 111 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
from pathlib import Path
from typing import TYPE_CHECKING, Any
from .config_helpers import get_cfg
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
DEFAULT_INSTALL_DIRS = {
    "epic": "~/Games/Epic",
    "gog": "~/GOG Games",
    "amazon": "~/Games/Amazon",
    "microsoft": "~/Games/Microsoft",
    "ubisoft": "~/Games/Ubisoft",
}
DEFAULT_GAMES_MAP = "~/.local/share/unifideck/games.map"
DEFAULT_SD_ROOT = "/run/media"
GAMES_MAP_PATH = str(Path(DEFAULT_GAMES_MAP).expanduser())
DEFAULT_PATHS = {
    store: str(Path(path).expanduser())
    for store, path in DEFAULT_INSTALL_DIRS.items()
}
def expand(path: str) -> str:
    """Expand."""
    return str(Path(os.path.expandvars(path)).expanduser())
def dedupe_paths(paths: list[str]) -> list[str]:
    """Dedupe paths."""
    seen: set[str] = set()
    out: list[str] = []
    for p in paths:
        norm = os.path.normpath(p)
        if norm in seen:
            continue
        seen.add(norm)
        out.append(p)
    return out
def _cfg(config: ConfigManager | None, key: str, default: Any) -> Any:
    """Cfg."""
    return get_cfg(config, key, default)
def get_all_game_directories(config: ConfigManager | None = None) -> list[str]:
    """Get all game directories."""
    candidates: list[str] = []
    for store, default in DEFAULT_INSTALL_DIRS.items():
        path = _cfg(config, f"stores.{store}.install_dir", default)
        candidates.append(expand(path))
    custom = get_cfg(config, "download.custom_path", "")
    if custom:
        candidates.append(expand(custom))
    media_root = get_cfg(config, "paths.sd_card_root", DEFAULT_SD_ROOT)
    candidates.extend(_scan_mount_root(media_root))
    existing = [p for p in candidates if Path(p).is_dir()]
    return dedupe_paths(existing)
def _collect_game_dirs(parent_path: Path) -> list[str]:

```

```

    """Collect game dirs."""
    found: list[str] = []
    for sub in ("Games", "GOG Games"):
        p = parent_path / sub
        if p.is_dir() and not p.is_symlink():
            found.append(str(p))
    return found
def _scan_level2(level1_path: Path) -> list[str]:
    """Scan level2."""
    found: list[str] = []
    try:
        for level2_path in level1_path.iterdir():
            if (
                not level2_path.is_dir()
                or level2_path.is_symlink()
            ):
                continue
            found.extend(_collect_game_dirs(level2_path))
    except OSError:
        pass
    return found
def _scan_mount_root(root: str) -> list[str]:
    """Scan mount root."""
    root_path = Path(root)
    if not root_path.is_dir():
        return []
    found: list[str] = []
    try:
        for level1_path in root_path.iterdir():
            if (
                not level1_path.is_dir()
                or level1_path.is_symlink()
            ):
                continue
            found.extend(_collect_game_dirs(level1_path))
            found.extend(_scan_level2(level1_path))
    except OSError as e:
        logger.debug(
            "[paths] mount scan failed on %s: %s", root, e,
        )
    return found
def get_games_map_path(config: ConfigManager | None = None) -> str:
    """Get games map path."""
    raw = get_cfg(config, "paths.games_map", DEFAULT_GAMES_MAP)
    return expand(raw)
def ensure_games_map_dir(config: ConfigManager | None = None) -> str | None:
    """Ensure games map dir."""
    path = Path(get_games_map_path(config))
    parent = path.parent
    try:
        parent.mkdir(parents=True, exist_ok=True)
        return str(parent)
    except OSError as e:
        logger.warning(
            "[paths] mkdir %s failed: %s", parent, e,
        )
    return None

```

OP-33b

locale.py

Fichier : py_modules/unifideck/utils/locale.py | Lignes : 126 | Depend de : (rien)

```

from __future__ import annotations
import locale as _locale
import logging
import sys
from pathlib import Path
from typing import TYPE_CHECKING, Any
from .config_helpers import get_cfg
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
_USER_LANGUAGE_KEY = "ui.language"
_LOCALE_CONFIG_MODULE = None
def _import_locale_config():
    """Import locale config."""
    global _LOCALE_CONFIG_MODULE
    if _LOCALE_CONFIG_MODULE is not None:
        return _LOCALE_CONFIG_MODULE
    here = Path(__file__).resolve()
    candidates = [
        here.parent.parent.parent.parent / "scripts",
        here.parent.parent.parent / "scripts",
    ]
    for scripts_dir in candidates:
        target = scripts_dir / "locale_config.py"
        if target.is_file():
            scripts_str = str(scripts_dir)
            if scripts_str not in sys.path:
                sys.path.insert(0, scripts_str)
            try:
                import locale_config
                _LOCALE_CONFIG_MODULE = locale_config
                return _LOCALE_CONFIG_MODULE
            except ImportError as e:
                logger.debug(
                    "[locale] Found %s but import failed: %s",
                    target, e,
                )
    return None
logger.debug(
    "[locale] scripts/locale_config.py not found on any "
    "candidate path; locale resolution will use degraded "
    "mode",
)
return None
def get_locale_config(config: ConfigManager | None):
    """Get locale config."""
    lc_module = _import_locale_config()
    if lc_module is None:
        return None
    i18n_section = get_cfg(config, "i18n", None)
    if not isinstance(i18n_section, dict):
        return None
    try:
        return lc_module.load_from_dict(
            {"i18n": i18n_section},

```

```

    )
except Exception as e:
    logger.warning(
        "[locale] i18n schema validation failed at "
        "runtime: %s", e,
    )
    return None

def get_unifideck_locale(config: ConfigManager | None) -> str:
    """Get unifideck locale."""
    lc = get_locale_config(config)
    saved = get_cfg(config, _USER_LANGUAGE_KEY, None)
    if isinstance(saved, str) and saved:
        normalised = saved.replace("_", "-")
        if lc is not None and lc.get(normalised) is not None:
            logger.debug(
                "[locale] user preference: %s", normalised,
            )
            return normalised
        if lc is None:
            logger.debug(
                "[locale] user preference (unvalidated): %s",
                normalised,
            )
            return normalised
        logger.debug(
            "[locale] user preference '%s' not in "
            "i18n.locales, falling back to system", saved,
        )
    system = _detect_system_locale(lc)
    if system:
        logger.debug("[locale] system: %s", system)
        return system
    if lc is not None:
        source_tag = str(lc.source.tag)
        logger.debug(
            "[locale] source fallback: %s", source_tag,
        )
        return source_tag
    logger.warning(
        "[locale] no config available, using hardcoded "
        "en-US",
    )
    return "en-US"

def get_unifideck_market(config: ConfigManager | None) -> str:
    """Get unifideck market."""
    tag = get_unifideck_locale(config)
    parts = tag.split("-")
    if len(parts) >= 2 and len(parts[-1]) == 2:
        return parts[-1].upper()
    return "US"

def _detect_system_locale(lc: Any) -> str | None:
    """Detect system locale."""
    if lc is None:
        return None
    try:
        lang_tuple = _locale.getlocale()
    except (ValueError, TypeError) as e:
        logger.debug("[locale] getlocale() failed: %s", e)
        return None

```

```
if not lang_tuple or not lang_tuple[0]:
    return None
prefix = lang_tuple[0].split("_")[0].lower()
if not prefix:
    return None
for loc in lc.locales:
    if (
        loc.tag.lower().startswith(prefix + "-")
        or loc.tag.lower() == prefix
    ):
        return str(loc.tag)
return None
```

OP-33c

config_helpers.py

Fichier : py_modules/unifideck/utils/config_helpers.py | Lignes : 51 | Depend de : (rien)

```

from __future__ import annotations
import logging
import sys
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from unifideck.config import ConfigManager
logger = logging.getLogger(__name__)
_none_sites_seen: set[tuple[str, int]] = set()
def get_cfg(
    config: ConfigManager | None,
    key: str,
    default: Any,
) -> Any:
    """Get cfg."""
    if config is None:
        frame = sys._getframe(1)
        site = (frame.f_globals.get("__name__", "?"), frame.f_lineno)
        if site not in _none_sites_seen:
            _none_sites_seen.add(site)
            logger.warning(
                "[config_helpers] config=None at %s:%d key=%r "
                "- falling back to default=%r. Likely a forgotten "
                "ConfigManager injection.",
                site[0], site[1], key, default,
            )
        return default
    try:
        return config.get(key, default)
    except Exception:
        return default
_COLD_START_CONFIG_PATH = "~/.local/share/unifideck/config.json"
def read_config_int_cold_start(key: str, default: int) -> int:
    """Read config int cold start."""
    import json
    from pathlib import Path as _P
    config_path = _P(_COLD_START_CONFIG_PATH).expanduser()
    if not config_path.is_file():
        return default
    try:
        with config_path.open() as f:
            data = json.load(f)
    except (OSError, ValueError):
        return default
    node = data
    for part in key.split("."):
        if not isinstance(node, dict) or part not in node:
            return default
        node = node[part]
    if not isinstance(node, int) or node <= 0:
        return default
    return node

```


OP-34

__init__.py

Fichier : py_modules/unifideck/compatibility/__init__.py | Lignes : 24 | Depend de : (rien)

```
from .library import (
    BackgroundCompatFetcher,
    CompatLibrary,
    CompatRating,
    fetch_deck_verified,
    fetch_protondb_rating,
    get_compat_for_title,
    load_compat_cache,
    prefetch_compat,
    save_compat_cache,
    search_steam_store,
)
from .proton_helpers import (
    CompatToolResult,
    ProtonToolsManager,
    get_compat_tool_for_app,
    get_compat_tool_for_game,
    get_saved_proton_tool,
    is_linux_runtime,
    resolve_proton_path,
    restore_compat_tool,
    save_proton_setting,
    temporarily_clear_compat_tool,
)
```

OP-34a

proton_helpers.py

Fichier : py_modules/unifideck/compatibility/proton_helpers.py | Lignes : 320 | Depend de : (rien)

```
from __future__ import annotations
import json
import logging
import os
import re
from dataclasses import dataclass
from pathlib import Path
from typing import TYPE_CHECKING, Any, cast
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
LINUX_RUNTIME_PREFIXES = (
    "steamlinuxruntime",
    "scout",
    "soldier",
    "sniper",
    "medic",
)
DEFAULT_CONFIG_VDF_RELATIVE = "config/config.vdf"
DEFAULT_PROTON_SETTINGS_RELATIVE = (
    ".local/share/unifideck/proton_settings.json"
)
DEFAULT_SHORTCUTS_REGISTRY_RELATIVE = (
    ".local/share/unifideck/shortcuts_registry.json"
)
@dataclass
class CompatToolResult:
    """Compat tool result."""
    success: bool
    appid: int
    tool_name: str
    previous: str | None = None
    error: str | None = None
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        return {
            "success": self.success,
            "appid": self.appid,
            "tool_name": self.tool_name,
            "previous": self.previous,
            "error": self.error,
        }
def is_linux_runtime(tool_name: str) -> bool:
    """Check whether linux runtime."""
    if not tool_name:
        return False
    lower = tool_name.lower()
    return any(
        lower.startswith(prefix) or f"_{prefix}" in lower
        for prefix in LINUX_RUNTIME_PREFIXES
    )
def parse_compat_tool(content: str, appid: int) -> str:
    """Parse compat tool."""
```

```

    if not content:
        return ""
    appid_str = str(appid)
    if f'"{appid_str}"' not in content:
        return ""
    marker = '"CompatToolMapping"'
    marker_pos = content.find(marker)
    if marker_pos < 0:
        return ""
    pattern = re.compile(
        rf'"{appid_str}"\s*\{{{[^}]*}\}', re.DOTALL,
    )
    m = pattern.search(content, marker_pos)
    if not m:
        return ""
    name_match = re.search(r'"name"\s+("[^"]*)"', m.group(1))
    return name_match.group(1) if name_match else ""
def inject_compat_tool(
    content: str, appid: int, tool_name: str,
) -> str:
    """Inject compat tool."""
    if tool_name and not re.match(
        r"^[A-Za-z0-9._-]*$", tool_name,
    ):
        raise ValueError(
            f"invalid compat tool name: {tool_name!r} "
            f"(must match [A-Za-z0-9._-])",
        )
    if not content:
        return content
    appid_str = str(appid)
    pattern = re.compile(
        rf'("{appid_str}"\s*\{{{[^}]*}\}"name"\s+)"["]*"',
        re.DOTALL,
    )
    new_content, count = pattern.subn(
        rf'\1"{tool_name}"', content, count=1,
    )
    if count > 0:
        return new_content
    marker = '"CompatToolMapping"'
    marker_pos = content.find(marker)
    if marker_pos < 0:
        return content
    open_brace = content.find("{", marker_pos)
    if open_brace < 0:
        return content
    insertion = (
        f'\n\t\t{appid_str}\n'
        f'\t\t{\n'
        f'\t\t\t"name"\t\t\t{tool_name}\n'
        f'\t\t\t"config"\t\t\t"\n'
        f'\t\t\t"priority"\t\t\t250\n'
        f'\t\t\t}'
    )
    return (
        content[:open_brace + 1]
        + insertion
        + content[open_brace + 1:]
    )
class ProtonToolsManager:

```

```

"""Proton tools manager."""
def __init__(self, config: ConfigManager | None = None) -> None:
    """Initialize the instance."""
    self._config = config
    self._config_vdf_path = self._resolve_config_vdf()
    self._proton_settings_path = (
        self._resolve_proton_settings()
    )
    self._shortcuts_registry_path = (
        self._resolve_shortcuts_registry()
    )

def _resolve_config_vdf(self) -> Path:

    """Resolve config VDF."""
    from ..steam.library import find_steam_path
    steam = find_steam_path(self._config)
    if steam is None:
        return (
            Path.home()
            / ".local/share/Steam"
            / DEFAULT_CONFIG_VDF_RELATIVE
        )
    return Path(steam) / DEFAULT_CONFIG_VDF_RELATIVE
def _resolve_proton_settings(self) -> Path:
    """Resolve PROTON settings."""
    return Path(
        self._cfg(
            "proton.settings_path",
            "~/ " + DEFAULT_PROTON_SETTINGS_RELATIVE,
        ),
    ).expanduser()
def _resolve_shortcuts_registry(self) -> Path:
    """Resolve shortcuts registry."""
    return Path(
        self._cfg(
            "proton.shortcuts_registry_path",
            "~/ " + DEFAULT_SHORTCUTS_REGISTRY_RELATIVE,
        ),
    ).expanduser()
def _cfg(self, key: str, default: Any) -> Any:
    """Cfg."""
    if self._config is None:
        return default
    try:
        return self._config.get(key, default)
    except Exception:
        return default
def get_for_app(self, appid: int) -> CompatToolResult:
    """Get for app."""
    content = self._read_config_vdf()
    tool = parse_compat_tool(content, appid)
    return CompatToolResult(
        success=True, appid=appid, tool_name=tool,
    )
def set_for_app(
    self, appid: int, tool_name: str,
) -> CompatToolResult:
    """Set for app."""
    content = self._read_config_vdf()
    if not content:

```

```

        return CompatToolResult(
            success=False, appid=appid,
            tool_name=tool_name,
            error="config.vdf not readable",
        )
    previous = parse_compat_tool(content, appid)
    new_content = inject_compat_tool(
        content, appid, tool_name,
    )
    if not self._write_config_vdf(new_content):
        return CompatToolResult(
            success=False, appid=appid,
            tool_name=tool_name,
            error="config.vdf write failed",
        )
    return CompatToolResult(
        success=True, appid=appid, tool_name=tool_name,
        previous=previous,
    )
def clear_for_app(self, appid: int) -> CompatToolResult:
    """Clear for app."""
    return self.set_for_app(appid, "")
def list_known_tools(self) -> list[str]:
    """List known tools."""
    tools: list[str] = []
    steam_root = self._config_vdf_path.parent.parent
    for sub in ("compatibilitytools.d", "steamapps/common"):
        d = steam_root / sub
        if not d.is_dir():
            continue
        try:
            for child in sorted(d.iterdir()):
                if child.is_dir():
                    tools.append(child.name)
        except OSError:
            continue
    return tools

def _read_config_vdf(self) -> str:
    """Read config VDF."""
    try:
        return self._config_vdf_path.read_text(
            encoding="utf-8", errors="ignore",
        )
    except OSError as e:
        logger.error(
            "[proton_helpers] read %s failed: %s",
            self._config_vdf_path, e,
        )
    return ""

def _write_config_vdf(self, content: str) -> bool:
    """Write config VDF."""
    tmp = self._config_vdf_path.with_suffix(".vdf.tmp")
    try:
        tmp.parent.mkdir(parents=True, exist_ok=True)
        with tmp.open("w", encoding="utf-8") as f:
            f.write(content)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, self._config_vdf_path)

```

```

        return True
    except OSError as e:
        logger.error(
            "[proton_helpers] write failed: %s", e,
        )
        try:
            tmp.unlink()
        except OSError:
            pass
        return False
def load_proton_settings(self) -> dict[str, Any]:
    """Load PROTON settings."""
    try:
        return cast(
            "dict[str, Any]",
            json.loads(self._proton_settings_path.read_text()),
        )
    except (OSError, json.JSONDecodeError):
        return {"games": {}}
def save_proton_settings(
    self, data: dict[str, Any],
) -> bool:
    """Save PROTON settings."""
    path = self._proton_settings_path
    tmp = path.with_suffix(".json.tmp")
    try:
        path.parent.mkdir(parents=True, exist_ok=True)
        tmp.write_text(json.dumps(data, indent=2))
        os.replace(tmp, path)
        return True
    except OSError as e:
        logger.error(
            "[proton_helpers] save settings failed: %s",
            e,
        )
        return False
_singleton_pt_mgr = None
def _pt_mgr():
    """Pt mgr."""
    global _singleton_pt_mgr
    if _singleton_pt_mgr is None:
        _singleton_pt_mgr = ProtonToolsManager()
    return _singleton_pt_mgr
def get_compat_tool_for_app(appid_unsigned):
    """Get compat tool for app."""
    return (
        _pt_mgr()
        .get_for_app(int(appid_unsigned))
        .tool_name
    )
def get_compat_tool_for_game(store_game_id):
    """Get compat tool for game."""
    return {
        "tool_name": "",
        "appid": 0,
        "store_game_id": store_game_id,
    }
def temporarily_clear_compat_tool(appid_unsigned):
    """Temporarily clear compat tool."""

```

```
    result = _pt_mgr().clear_for_app(int(appid_unsigned))
    return {
        "success": result.success,
        "previous": result.previous,
    }
def restore_compat_tool(appid_unsigned, tool_name):
    """Restore compat tool."""
    result = _pt_mgr().set_for_app(
        int(appid_unsigned), tool_name,
    )
    return {"success": result.success}
def save_proton_setting(store_game_id, tool_name):
    """Save PROTON setting."""
    settings = _pt_mgr().load_proton_settings()
    settings.setdefault("games", {})[store_game_id] = tool_name
    return {
        "success": _pt_mgr().save_proton_settings(settings),
    }
def get_saved_proton_tool(store_game_id):
    """Get saved PROTON tool."""
    return (
        _pt_mgr()
        .load_proton_settings()
        .get("games", {})
        .get(store_game_id, "")
    )
def resolve_proton_path(tool_name):
    """Resolve PROTON path."""
    return tool_name
```

OP-34b

library.py

Fichier : py_modules/unifideck/compatibility/library.py | Lignes : 239 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from ..config import ConfigManager
    from ..core.cache_manager import CacheManager
from ..utils.config_helpers import get_cfg
logger = logging.getLogger(__name__)
PROTONDB_TIERS = ("platinum", "gold", "silver", "bronze", "borked")
DECK_CATEGORIES = {
    0: "unknown",
    1: "unsupported",
    2: "playable",
    3: "verified",
}
PROTONDB_URL = (
    "https://www.protondb.com/api/v1/reports/summaries/{appid}.json"
)
DECK_VERIFIED_URL = (
    "https://store.steampowered.com/saleaction/"
    "ajaxgetdeckappcompatibilityreport?nAppID={appid}"
)
DEFAULT_USER_AGENT = "Unifideck/1.0 (compat-library)"
CACHE_NAMESPACE = "compat"
@dataclass
class CompatRating:
    """Compat rating."""
    appid: int | None = None
    title: str = ""
    protondb_tier: str | None = None
    deck_status: str = "unknown"
    sources: list[str] = field(default_factory=list)
    error: str | None = None
    def to_dict(self) -> dict[str, Any]:
        """To dict."""
        return {
            "appid": self.appid,
            "title": self.title,
            "protondb_tier": self.protondb_tier,
            "deck_status": self.deck_status,
            "sources": list(self.sources),
            "error": self.error,
        }
def parse_protondb_response(payload: dict[str, Any]) -> str | None:
    """Parse protondb response."""
    if not isinstance(payload, dict):
        return None
    tier = payload.get("tier")
    if isinstance(tier, str) and tier in PROTONDB_TIERS:
        return tier
    return None
def parse_deck_verified_response(payload: dict[str, Any]) -> str:
    """Parse DECK verified response."""

```



```

if not isinstance(payload, dict):
    return "unknown"
results = payload.get("results")
if not isinstance(results, dict):
    return "unknown"
cat = results.get("resolved_category", 0)
try:
    return DECK_CATEGORIES.get(int(cat), "unknown")
except (TypeError, ValueError):
    return "unknown"

def _cfg(config: ConfigManager | None, key: str, default: Any) -> Any:
    """Cfg."""
    return get_cfg(config, key, default)
class CompatLibrary:
    """Compat library."""
    def __init__(
        self,
        cache: CacheManager | None = None,
        config: ConfigManager | None = None,
    ) -> None:
        """Initialize the instance."""
        self._cache = cache
        self._config = config
        if cache is not None:
            ttl = int(get_cfg(config, "cache_ttl.compat", 604800))
            try:
                cache.register(CACHE_NAMESPACE, ttl_seconds=ttl)
            except Exception:
                pass
    async def get_for_appid(self, appid: int) -> CompatRating:
        """Get for appid."""
        cached = self._cache_get(str(appid))
        if cached is not None:
            return CompatRating(**cached)
        result = CompatRating(appid=appid)
        result.protondb_tier = await self._fetch_protondb(appid)
        if result.protondb_tier is not None:
            result.sources.append("protondb")
        result.deck_status = await self._fetch_deck_verified(appid)
        if result.deck_status != "unknown":
            result.sources.append("deck_verified")
        self._cache_set(str(appid), result.to_dict())
        return result
    async def get_for_title(self, title: str) -> CompatRating:
        """Get for title."""
        from ..steam.library import search_store
        steam = await search_store(title, config=self._config)
        if steam is None or "app_id" not in steam:
            return CompatRating(
                title=title, error="not_found_on_steam_store",
            )
        result = await self.get_for_appid(int(steam["app_id"]))
        result.title = title
        return result
    async def bulk_fetch(
        self, titles: list[str], delay_ms: int = 50,
    ) -> dict[str, CompatRating]:
        """Bulk fetch."""
        out: dict[str, CompatRating] = {}

```

```

        for title in titles:
            out[title] = await self.get_for_title(title)
            if delay_ms > 0:
                await asyncio.sleep(delay_ms / 1000)
        return out
    async def _fetch_protodb(self, appid: int) -> str | None:
        """Fetch protodb."""
        import aiohttp
        url = PROTONDB_URL.format(appid=appid)
        timeout = int(_cfg(
            self._config, "compat.protodb_timeout_seconds", 30,
        ))
        try:
            async with (
                aiohttp.ClientSession() as session,
                session.get(
                    url,
                    headers={"User-Agent": DEFAULT_USER_AGENT},
                    timeout=timeout,
                ) as resp,
            ):
                if resp.status == 404:
                    return None
                if resp.status != 200:
                    return None
                return parse_protodb_response(
                    await resp.json(),
                )
        except Exception as e:
            logger.debug(
                "[compat] protodb(%d) failed: %s", appid, e,
            )
            return None

    async def _fetch_deck_verified(self, appid: int) -> str:
        """Fetch DECK verified."""
        import aiohttp
        url = DECK_VERIFIED_URL.format(appid=appid)
        timeout = int(_cfg(
            self._config, "compat.deck_verified_timeout_seconds", 10,
        ))
        try:
            async with (
                aiohttp.ClientSession() as session,
                session.get(
                    url,
                    headers={"User-Agent": DEFAULT_USER_AGENT},
                    timeout=timeout,
                ) as resp,
            ):
                if resp.status != 200:
                    return "unknown"
                return parse_deck_verified_response(
                    await resp.json(),
                )
        except Exception as e:
            logger.debug(
                "[compat] deck(%d) failed: %s", appid, e,
            )
            return "unknown"

```

```

def _cache_get(self, key: str) -> dict[str, Any] | None:
    """Cache get."""
    if self._cache is None:
        return None
    try:
        return self._cache.get(CACHE_NAMESPACE, key)
    except Exception:
        return None
def _cache_set(
    self, key: str, value: dict[str, Any],
) -> None:
    """Cache set."""
    if self._cache is None:
        return
    try:
        self._cache.set(CACHE_NAMESPACE, key, value)
    except Exception:
        pass
def load_compat_cache():
    """Load compat cache."""
    logger.debug("[compat] load_compat_cache called via legacy path")
    return {}
def save_compat_cache(cache):
    """Save compat cache."""
    logger.debug("[compat] save_compat_cache called via legacy path")
    return True
async def search_steam_store(session=None, title="", **kwargs):
    """Search steam store."""
    from ..steam.library import search_store
    return await search_store(title)
async def fetch_protondb_rating(session=None, appid=0, **kwargs):
    """Fetch protondb rating."""
    lib = CompatLibrary()
    return await lib._fetch_protondb(int(appid))
async def fetch_deck_verified(session=None, appid=0, **kwargs):
    """Fetch DECK verified."""
    lib = CompatLibrary()
    return await lib._fetch_deck_verified(int(appid))
async def get_compat_for_title(session=None, title="", **kwargs):
    """Get compat for title."""
    lib = CompatLibrary()
    rating = await lib.get_for_title(title)
    status = "ok" if rating.error is None else rating.error
    return (status, rating.to_dict())
async def prefetch_compat(
    titles,
    _batch_size=10,
    delay_ms=50,
):
    """Prefetch compat."""
    lib = CompatLibrary()
    return await lib.bulk_fetch(list(titles), delay_ms=delay_ms)

class BackgroundCompatFetcher:
    """Background compat fetcher."""
    def __init__(self, *args, **kwargs):
        """Initialize the instance."""
        self._lib = CompatLibrary()
    def start(self):
        """Start."""

```

```
    pass
def stop(self):
    """Stop."""
    pass
async def fetch(self, title):
    """Fetch."""
    return await self._lib.get_for_title(title)
```

OP-35

`__init__.py`

Fichier : `py_modules/unifideck/launcher/__init__.py` | Lignes : 23 | Depend de : (rien)

```
from __future__ import annotations
from .types.context import LaunchContext
from .types.errors import (
    DependencyMissingError,
    GameNotFoundError,
    LaunchAbortedError,
    LauncherError,
    PrefixCorruptedError,
    ProtonUnavailableError,
    UmuRuntimeError,
)
from .types.exit_codes import ExitCode
__all__ = [
    "LaunchContext",
    "LauncherError",
    "LaunchAbortedError",
    "DependencyMissingError",
    "GameNotFoundError",
    "PrefixCorruptedError",
    "ProtonUnavailableError",
    "UmuRuntimeError",
    "ExitCode",
]
```

OP-36

__init__.py

Fichier : py_modules/unifideck/launcher/types/__init__.py | Lignes : 0 | Depend de : (rien)

OP-36a

exit_codes.py

Fichier : py_modules/unifideck/launcher/types/exit_codes.py | Lignes : 31 | Depend de : (rien)

```
from __future__ import annotations
from enum import IntEnum
class ExitCode(IntEnum):
    """Exit code."""
    SUCCESS = 0
    GENERIC_ERROR = 1
    CONFIG_INVALID = 2
    DEPENDENCY_MISSING = 3
    NETWORK_ERROR = 4
    CANCELLED_BY_USER = 5
    TIMED_OUT = 6
    PREFIX_CORRUPTED = 7
    GAME_FAILED = 8
    CIRCUIT_BREAKER_OPEN = 9
    SIGTERM_EQUIVALENT = 143
    def user_message_key(self) -> str:
        """User message key."""
        mapping = {
            ExitCode.SUCCESS: "",
            ExitCode.GENERIC_ERROR: "toasts.launcher.errorGeneric",
            ExitCode.CONFIG_INVALID: "toasts.launcher.errorConfig",
            ExitCode.DEPENDENCY_MISSING: "toasts.launcher.errorMissingDep",
            ExitCode.NETWORK_ERROR: "toasts.launcher.errorNetwork",
            ExitCode.CANCELLED_BY_USER: "toasts.launcher.cancelled",
            ExitCode.TIMED_OUT: "toasts.launcher.errorTimeout",
            ExitCode.PREFIX_CORRUPTED: "toasts.launcher.errorPrefix",
            ExitCode.GAME_FAILED: "toasts.launcher.errorGameFailed",
            ExitCode.CIRCUIT_BREAKER_OPEN: "toasts.launcher.errorCircuitBreakerOpen",
            ExitCode.SIGTERM_EQUIVALENT: "toasts.launcher.cancelled",
        }
        return mapping.get(self, "toasts.launcher.errorGeneric")
```

OP-36b

errors.py

Fichier : py_modules/unifideck/launcher/types/errors.py | Lignes : 60 | Depend de : (rien)

```

from __future__ import annotations
from typing import Any
from .exit_codes import ExitCode
class LauncherError(Exception):
    """Launcher error."""
    exit_code: ExitCode = ExitCode.GENERIC_ERROR
    def __init__(
        self,
        message: str,
        *,
        context: dict[str, Any] | None = None,
    ) -> None:
        """Initialize the instance."""
        super().__init__(message)
        self.context: dict[str, Any] = dict(context or {})
    def with_context(self, **fields: Any) -> LauncherError:
        """With context."""
        self.context.update(fields)
        return self
    def to_log_dict(self) -> dict[str, Any]:
        """To log dict."""
        return {
            "type": type(self).__name__,
            "message": str(self),
            "exit_code": int(self.exit_code),
            "context": self.context,
        }
class LaunchAbortedError(LauncherError):
    """Launch aborted error."""
    exit_code = ExitCode.CANCELLED_BY_USER
class DependencyMissingError(LauncherError):
    """Dependency missing error."""
    exit_code = ExitCode.DEPENDENCY_MISSING
class GameNotFoundError(LauncherError):
    """Game not found error."""
    exit_code = ExitCode.CONFIG_INVALID
class PrefixCorruptedError(LauncherError):
    """Prefix corrupted error."""
    exit_code = ExitCode.PREFIX_CORRUPTED
class ProtonUnavailableError(LauncherError):
    """Proton unavailable error."""
    exit_code = ExitCode.DEPENDENCY_MISSING
class UmuRuntimeError(LauncherError):
    """Umu runtime error."""
    exit_code = ExitCode.GAME_FAILED
class GameFailedError(LauncherError):
    """Game failed error."""
    exit_code = ExitCode.GAME_FAILED
    def __init__(
        self,
        message: str,
        *,
        subprocess_rc: int,
        context: dict[str, Any] | None = None,
    ) -> None:

```



```
"""Initialize the instance."""
merged = dict(context or {})
merged.setdefault("subprocess_rc", subprocess_rc)
super().__init__(message, context=merged)
self.subprocess_rc = subprocess_rc
```

OP-36c

context.py

Fichier : py_modules/unifideck/launcher/types/context.py | Lignes : 86 | Depend de : (rien)

```

from __future__ import annotations
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any
KNOWN_STORES: tuple[str, ...] = (
    "epic",
    "gog",
    "amazon",
    "microsoft",
    "ubisoft",
)
@dataclass(frozen=True)
class LaunchContext:
    """Launch context."""
    store: str
    game_id: str
    exe_path: Path
    work_dir: Path
    plugin_dir: Path
    raw_options: str = ""
    env_overrides: dict[str, str] = field(default_factory=dict)
    is_launch_action: bool = True
    auth_store: str | None = None
    bypass_circuit_breaker: bool = False
    steam_app_id: str | None = None
    @property
    def is_xcloud(self) -> bool:
        """Check whether xcloud."""
        return str(self.exe_path) == "xcloud"
    @property
    def is_windows_game(self) -> bool:
        """Check whether windows game."""
        if self.store == "ubisoft":
            return True
        exe_str = str(self.exe_path).lower()
        return exe_str.endswith((".exe", ".cmd", ".bat"))
    @property
    def is_native_linux(self) -> bool:
        """Check whether native linux."""
        return not self.is_xcloud and not self.is_windows_game
    @property
    def game_key(self) -> str:
        """Game key."""
        return f"{self.store}:{self.game_id}"
    def to_log_dict(self) -> dict[str, Any]:
        """To log dict."""
        return {
            "store": self.store,
            "game_id": self.game_id,
            "exe_path": str(self.exe_path),
            "work_dir": str(self.work_dir),
            "is_xcloud": self.is_xcloud,
            "is_windows_game": self.is_windows_game,
            "is_launch_action": self.is_launch_action,
            "auth_store": self.auth_store,

```

```
        "bypass_circuit_breaker": self.bypass_circuit_breaker,  
    }
```

```
@dataclass
```

```
class RuntimeState:
```

```
    """Runtime state."""
```

```
    proton_path: Path | None = None
```

```
    proton_tool_id: str | None = None
```

```
    prefix_path: Path | None = None
```

```
    umu_store_code: str | None = None
```

```
    umu_id: str | None = None
```

```
    umu_wrapper: Path | None = None
```

```
    python_bin: Path | None = None
```

```
    wrappers: list[str] = field(default_factory=list)
```

```
    game_args: list[str] = field(default_factory=list)
```

```
    lsfg_requested: bool = False
```

```
    game_exit_code: int | None = None
```

```
    terminated_by_signal: bool = False
```

```
    def to_log_dict(self) -> dict[str, Any]:
```

```
        """To log dict."""
```

```
        return {
```

```
            "proton_path": str(self.proton_path) if self.proton_path else None,
```

```
            "proton_tool_id": self.proton_tool_id,
```

```
            "prefix_path": str(self.prefix_path) if self.prefix_path else None,
```

```
            "umu_store_code": self.umu_store_code,
```

```
            "umu_id": self.umu_id,
```

```
            "lsfg_requested": self.lsfg_requested,
```

```
            "game_exit_code": self.game_exit_code,
```

```
            "terminated_by_signal": self.terminated_by_signal,
```

```
            "wrappers_count": len(self.wrappers),
```

```
            "game_args_count": len(self.game_args),
```

```
        }
```

OP-36d

options.py

Fichier : py_modules/unifideck/launcher/types/options.py | Lignes : 114 | Depend de : (rien)

```

from __future__ import annotations
import os
import re
import shlex
from dataclasses import dataclass, field
from pathlib import Path
_ENV_TOKEN_RE = re.compile(r"^([A-Z_][A-Z0-9_]*)(.*)$")
_LSFG_KEY_RE = re.compile(r"^([A-Za-z_][A-Za-z0-9_]*)$")
@dataclass
class ParsedOptions:
    """Parsed options."""
    wrappers: list[str] = field(default_factory=list)
    game_args: list[str] = field(default_factory=list)
    env_overrides: dict[str, str] = field(default_factory=dict)
    lsfg_requested: bool = False
def parse_launch_options(raw: str) -> ParsedOptions:
    """Parse launch options."""
    result = ParsedOptions()
    if not raw or not raw.strip():
        return result
    try:
        tokens = shlex.split(raw)
    except ValueError:
        tokens = raw.split()
    remaining: list[str] = []
    for tok in tokens:
        m = _ENV_TOKEN_RE.match(tok)
        if m:
            result.env_overrides[m.group(1)] = m.group(2)
        else:
            remaining.append(tok)
    home = os.path.expanduser("~")
    lsfg_filtered: list[str] = []
    for tok in remaining:
        expanded = (
            tok.replace("~", home, 1)
            if tok.startswith("~") else tok
        )
        if expanded.endswith("/lsfg"):
            result.lsfg_requested = True
        else:
            lsfg_filtered.append(tok)
    if result.env_overrides.get("LSFG") == "1":
        result.lsfg_requested = True
    if result.env_overrides.get("ENABLE_LSFG") == "1":
        result.lsfg_requested = True
    _split_tokens_around_command(lsfg_filtered, result)
    return result

def _split_tokens_around_command(
    tokens: list[str], result: ParsedOptions,
) -> None:
    """Split tokens around command."""
    found_cmd = False

```

```

for tok in tokens:
    if tok == "%command%":
        found_cmd = True
        continue
    if tok == "#%command%":
        continue
    if found_cmd:
        result.game_args.append(tok)
    else:
        result.wrappers.append(tok)
if (
    not found_cmd
    and result.wrappers
    and not result.game_args
):
    result.game_args = result.wrappers
    result.wrappers = []
def apply_lsfg_env(
    opts: ParsedOptions,
    lsfg_script: Path | None = None,
) -> dict[str, str]:
    """Apply lsfg env."""
    if not opts.lsfg_requested:
        return {}
    if lsfg_script is None:
        lsfg_script = Path(os.path.expanduser("~/lsfg"))
    if not lsfg_script.is_file():
        return {}
    overlay: dict[str, str] = {"ENABLE_LSFG": "1"}
    try:
        content = lsfg_script.read_text(
            encoding="utf-8", errors="replace",
        )
    except OSError:
        return overlay
    for raw_line in content.splitlines():
        line = raw_line.strip()
        if (
            not line
            or line.startswith("#")
            or line.startswith("#!")
        ):
            continue
        if line.startswith("exec "):
            continue
        if not line.startswith("export "):
            continue
        kv = line[len("export "):]
        if "=" not in kv:
            continue
        key, _, value = kv.partition("=")
        key = key.strip()
        value = value.strip()
        if len(value) >= 2 and (
            (value[0] == "'" and value[-1] == "'")
            or (value[0] == '"' and value[-1] == '"')
        ):
            value = value[1:-1]
        if _LSFG_KEY_RE.match(key):
            overlay[key] = value
    return overlay

```

OP-37

__init__.py

Fichier : py_modules/unifideck/launcher/flows/__init__.py | Lignes : 0 | Depend de : (rien)

OP-37a

xcloud.py

Fichier : py_modules/unifideck/launcher/flows/xcloud.py | Lignes : 96 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import contextlib
import logging
from typing import TYPE_CHECKING
from ...core.types import Result
from ..cdp.xcloud_cdp import run_cdp_inject
from ..types.context import LaunchContext
from ..types.errors import DependencyMissingError
if TYPE_CHECKING:
    from ...auth.edge_browser import EdgeBrowser
logger = logging.getLogger(__name__)
_MAX_SESSION_SECONDS = 14400
_POLL_INTERVAL_SECONDS = 5.0
_XCLOUD_CDP_PORT = 9223
_CDP_INJECT_TIMEOUT = 60.0
def _read_config_int(key: str, default: int) -> int:
    """Read config int."""
    from ..utils.config_helpers import read_config_int_cold_start
    return read_config_int_cold_start(key, default)

async def launch_xcloud(
    ctx: LaunchContext,
    edge_browser: EdgeBrowser,
) -> Result:

    """Launch xcloud."""
    target_url = str(ctx.work_dir)
    logger.info(
        "[launcher.xcloud] launching: %s", target_url[:80],
    )
    if not edge_browser.is_installed:
        raise DependencyMissingError(
            "Microsoft Edge flatpak required for xCloud "
            "streaming",
            context={
                "game_id": ctx.game_id,
                "url": target_url,
            },
        )
    started = edge_browser.launch_xcloud(target_url)
    if not started:
        return Result(
            success=False,
            error="edge_launch_failed",
            store=ctx.store,
        )
    inject_task: asyncio.Task[bool] = asyncio.create_task(
        run_cdp_inject(
            port=_XCLOUD_CDP_PORT,
            launch_url=target_url,
            timeout=_CDP_INJECT_TIMEOUT,
            steam_controller_appid=int(ctx.steam_app_id or 0),
        ),
        name=f"xcloud_cdp_inject:{ctx.game_key}",
    )

```

```

)
try:
    await _wait_for_session_end(edge_browser)
finally:
    if not inject_task.done():
        inject_task.cancel()
        with contextlib.suppress(asyncio.CancelledError):
            await inject_task
logger.info(
    "[launcher.xcloud] session ended: %s", ctx.game_key,
)
return Result(success=True, store=ctx.store)
async def _wait_for_session_end(edge_browser: EdgeBrowser) -> None:
    """Wait for session end."""
    max_seconds = _read_config_int(
        "launcher.xcloud_max_seconds", _MAX_SESSION_SECONDS,
    )
    proc = edge_browser.process
    if proc is not None:
        loop = asyncio.get_event_loop()
        try:
            await asyncio.wait_for(
                loop.run_in_executor(None, proc.wait),
                timeout=max_seconds,
            )
        except TimeoutError:
            logger.warning(
                "[launcher.xcloud] session reached max "
                "duration (%ds), leaving Edge running",
                max_seconds,
            )
        return
    logger.info(
        "[launcher.xcloud] no process handle, polling "
        "fallback",
    )
    elapsed = 0.0
    while elapsed < max_seconds:
        await asyncio.sleep(_POLL_INTERVAL_SECONDS)
        elapsed += _POLL_INTERVAL_SECONDS
    logger.warning(
        "[launcher.xcloud] polling fallback reached timeout",
    )

```


OP-37b

native.py

Fichier : py_modules/unifideck/launcher/flows/native.py | Lignes : 125 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
from pathlib import Path
from ...core.types import Result
from ..types.context import LaunchContext, RuntimeState
from ..types.errors import DependencyMissingError, GameFailedError
logger = logging.getLogger(__name__)
STEAM_RUNTIME_CANDIDATES = [
    "~/steam/steam/ubuntu12_32/steam-runtime/run.sh",
    "~/local/share/Steam/ubuntu12_32/steam-runtime/run.sh",
]
def _find_steam_runtime() -> Path | None:
    """Find steam runtime."""
    for candidate in STEAM_RUNTIME_CANDIDATES:
        path = Path(candidate).expanduser()
        if path.is_file():
            return path
    return None
def _restore_steam_env(env: dict[str, str]) -> None:
    """Restore steam env."""
    steam_env = Path("~/steam/steam.env").expanduser()
    if not steam_env.is_file():
        return
    try:
        for line in steam_env.read_text(
            encoding="utf-8", errors="replace",
        ).splitlines():
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            if "=" not in line:
                continue
            key, _, value = line.partition("=")
            if key in ("STEAM_OVERLAY", "STEAM_INPUT"):
                env[key] = value
    except OSError:
        pass
def _is_gog_dosbox_wrapper(ctx: LaunchContext) -> bool:
    """Is GOG dosbox wrapper."""
    return (
        ctx.store == "gog"
        and ctx.exe_path.name == "start.sh"
    )
async def native_launch(
    ctx: LaunchContext,
    state: RuntimeState,
) -> Result:
    """Native launch."""
    exe_path = ctx.exe_path
    if not exe_path.is_file():
        raise DependencyMissingError(

```

```

        f"Native Linux executable not found: {exe_path}",
        context={"exe": str(exe_path), "store": ctx.store},
    )
    try:
        os.chmod(exe_path, 0o755)
    except OSError:
        pass
    env = _prepare_launch_env(ctx)
    argv = _build_launch_argv(ctx, state, exe_path)
    cwd = ctx.work_dir if ctx.work_dir.is_dir() else exe_path.parent
    logger.info(
        "[launcher.native] spawning: argv=%s cwd=%s",
        argv[:3],
        cwd,
    )
    proc = await asyncio.create_subprocess_exec(
        *argv,
        env=env,
        cwd=str(cwd),
        stdout=None,
        stderr=None,
        start_new_session=True,
    )
    rc = await proc.wait()
    state.game_exit_code = rc
    logger.info("[launcher.native] game exited rc=%d", rc)
    if rc != 0:
        raise GameFailedError(
            f"Native Linux game exited with code {rc}",
            subprocess_rc=rc,
            context={
                "store": ctx.store,
                "game_id": ctx.game_id,
            },
        )
    return Result(success=True, store=ctx.store)
def _prepare_launch_env(ctx: LaunchContext) -> dict[str, str]:
    """Prepare launch env."""
    env = dict(os.environ)
    env.update(ctx.env_overrides)
    _restore_steam_env(env)
    return env

def _build_launch_argv(
    ctx: LaunchContext,
    state: RuntimeState,
    exe_path: Path,
) -> list[str]:
    """Build launch argv."""
    argv: list[str] = list(state.wrappers)
    if _is_gog_dosbox_wrapper(ctx):
        logger.info(
            "[launcher.native] using GOG DOSBox wrapper module",
        )
        argv.extend([
            "python3", "-m",
            "unifideck.launcher.proton.gog_linux_dosbox",
            str(exe_path),
        ])
    else:

```

```
runtime = _find_steam_runtime()
if runtime is not None:
    logger.info(
        "[launcher.native] using Steam Runtime: %s", runtime,
    )
    argv.extend([str(runtime), str(exe_path)])
else:
    logger.info(
        "[launcher.native] no Steam Runtime, direct exec",
    )
    argv.append(str(exe_path))
argv.extend(state.game_args)
return argv
```

OP-37c

auth.py

Fichier : py_modules/unifideck/launcher/flows/auth.py | Lignes : 115 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from pathlib import Path
from typing import TYPE_CHECKING
from ...core.types import Result
from ..types.context import LaunchContext
from ..types.errors import DependencyMissingError, GameNotFoundError
if TYPE_CHECKING:
    from ...auth.edge_browser import EdgeBrowser
logger = logging.getLogger(__name__)
_AUTH_URL_FILES = {
    "epic": "epic_auth_url.txt",
    "gog": "gog_auth_url.txt",
    "amazon": "amazon_auth_url.txt",
}
_AUTH_STORE_LABELS = {
    "epic": "Epic Games",
    "gog": "GOG",
    "amazon": "Amazon Games",
}
_MAX_AUTH_SECONDS = 600
def _read_config_int(key: str, default: int) -> int:
    """Read config int."""
    from ..utils.config_helpers import read_config_int_cold_start
    return read_config_int_cold_start(key, default)
def _read_auth_url(store: str) -> str:
    """Read auth URL."""
    filename = _AUTH_URL_FILES.get(store)
    if filename is None:
        raise GameNotFoundError(
            f"Unknown auth store {store!r}",
            context={"store": store},
        )
    url_file = Path(
        f"~/.local/share/unifideck/{filename}",
    ).expanduser()
    if not url_file.is_file():
        raise GameNotFoundError(
            f"Auth URL file not found: {url_file}",
            context={"store": store, "file": str(url_file)},
        )
    try:
        url = url_file.read_text(encoding="utf-8").strip()
    except OSError as e:
        raise GameNotFoundError(
            f"Cannot read auth URL file: {e}",
            context={"store": store, "file": str(url_file)},
        ) from e
    if not url:
        raise GameNotFoundError(
            f"Auth URL file is empty: {url_file}",
            context={"store": store},
        )
    return url

```

```

async def handle_store_auth(
    ctx: LaunchContext,
    edge_browser: EdgeBrowser,
) -> Result:

    """Handle store auth."""
    store = ctx.auth_store
    if store is None:
        raise GameNotFoundError(
            "handle_store_auth called without auth_store set",
            context={"game_key": ctx.game_key},
        )
    label = _AUTH_STORE_LABELS.get(store, store.title)
    logger.info(
        "[launcher.auth] launching %s OAuth flow", label,
    )
    if not edge_browser.is_installed:
        raise DependencyMissingError(
            "Microsoft Edge flatpak required for OAuth",
            context={"store": store},
        )
    auth_url = _read_auth_url(store)
    logger.info(
        "[launcher.auth] %s auth URL resolved (%d chars)",
        label, len(auth_url),
    )
    started = edge_browser.launch_auth(auth_url)
    if not started:
        return Result(
            success=False,
            error="edge_auth_launch_failed",
            store=store,
        )
    await _wait_for_auth_end(edge_browser)
    logger.info(
        "[launcher.auth] %s auth browser closed", label,
    )
    return Result(success=True, store=store)

async def _wait_for_auth_end(edge_browser: EdgeBrowser) -> None:
    """Wait for auth end."""
    max_seconds = _read_config_int(
        "launcher.auth_max_seconds", _MAX_AUTH_SECONDS,
    )
    proc = edge_browser.process
    if proc is not None:
        loop = asyncio.get_event_loop()
        try:
            await asyncio.wait_for(
                loop.run_in_executor(None, proc.wait),
                timeout=max_seconds,
            )
        except TimeoutError:
            logger.warning(
                "[launcher.auth] auth flow reached %ds timeout",
                max_seconds,
            )
        return
    elapsed = 0.0
    while elapsed < max_seconds:
        await asyncio.sleep(5.0)

```

elapsed += 5.0

OP-38

__init__.py

Fichier : py_modules/unifideck/launcher/cdp/__init__.py | Lignes : 0 | Depend de : (rien)

OP-38a

cdp_primitives.py

Fichier : py_modules/unifideck/launcher/cdp/cdp_primitives.py | Lignes : 102 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import contextlib
import json
import logging
from typing import Any, cast
import aiohttp
from unifideck.cdp.page_inject import list_page_targets
logger = logging.getLogger(__name__)
async def wait_for_titled_target(
    cdp_port: int,
    title_substring: str,
    *,
    timeout: float = 15.0,
    poll_delay: float = 0.25,
) -> dict[str, Any] | None:
    """Wait for titled target."""
    deadline = asyncio.get_running_loop().time() + timeout
    while asyncio.get_running_loop().time() < deadline:
        try:
            targets = await list_page_targets(cdp_port, timeout=3.0)
            for target in targets:
                if title_substring in str(target.get("title", "")):
                    return dict(target)
        except asyncio.CancelledError:
            raise
        except Exception as exc:
            logger.debug("[cdp] waiting for target failed: %s", exc)
            await asyncio.sleep(poll_delay)
    return None
async def close_target(cdp_port: int, target_id: str) -> None:
    """Close target."""
    close_url = f"http://127.0.0.1:{cdp_port}/json/close/{target_id}"
    async with aiohttp.ClientSession() as session:
        with contextlib.suppress(Exception):
            async with session.get(
                close_url,
                timeout=aiohttp.ClientTimeout(total=3.0),
            ) as response:
                await response.read()
async def close_titled_targets(
    cdp_port: int, title_substring: str,
) -> None:
    """Close titled targets."""
    with contextlib.suppress(Exception):
        targets = await list_page_targets(cdp_port, timeout=3.0)
        for target in targets:
            if title_substring in str(target.get("title", "")):
                await close_target(cdp_port, str(target["id"]))
async def cdp_command(
    websocket: aiohttp.ClientWebSocketResponse,
    msg_id: int,
    method: str,
    params: dict[str, Any] | None = None,

```



```

) -> dict[str, Any]:
    """Cdp command."""
    await websocket.send_json(
        {
            "id": msg_id,
            "method": method,
            "params": params or {},
        },
    )
    while True:
        message = await websocket.receive(timeout=15)
        if message.type != aiohttp.WSMsgType.TEXT:
            if message.type in (
                aiohttp.WSMsgType.CLOSED, aiohttp.WSMsgType.CLOSING,
            ):
                raise RuntimeError("CDP websocket closed")
            if message.type == aiohttp.WSMsgType.ERROR:
                raise RuntimeError("CDP websocket error")
            continue
        payload = json.loads(message.data)
        if payload.get("id") != msg_id:
            continue
        if "error" in payload:
            raise RuntimeError(f"{method} failed: {payload['error']}")
        return cast("dict[str, Any]", payload)
    async def evaluate_in_target(
        target: dict[str, Any],
        expression: str,
        *,
        return_by_value: bool = True,
    ) -> dict[str, Any]:
        """Evaluate in target."""
        async with aiohttp.ClientSession() as session, session.ws_connect(
            target["webSocketDebuggerUrl"],
            heartbeat=10,
            autoping=True,
        ) as websocket:
            return await cdp_command(
                websocket,
                9001,
                "Runtime.evaluate",
                {
                    "expression": expression,
                    "awaitPromise": True,
                    "returnByValue": return_by_value,
                    "userGesture": True,
                },
            )

```

OP-38b

xcloud_cdp.py

Fichier : py_modules/unifideck/launcher/cdp/xcloud_cdp.py | Lignes : 156 | Depend de : (rien)

```

from __future__ import annotations
import logging
import shutil
import subprocess
import time
from urllib.parse import urlparse
from unifideck.cdp.page_inject import inject_scripts
from unifideck.cdp.xcloud_browser_shims import (
    get_xcloud_browser_shims_js,
    get_xcloud_navigation_js,
)
from .steam_controller_popup import refresh_steam_controller_layout
logger = logging.getLogger(__name__)
def _build_launch_matches(launch_url: str) -> list[str]:
    """Build launch matches."""
    if not launch_url:
        return []
    parsed = urlparse(launch_url)
    path = parsed.path.rstrip("/")
    product_id = path.split("/")[-1] if path else ""
    matches: list[str] = [launch_url]
    if path:
        matches.append(path)
    if product_id:
        matches.append(product_id)
        matches.append(f"/play/launch/{product_id}")
    deduped: list[str] = []
    for match in matches:
        if match and match not in deduped:
            deduped.append(match)
    return deduped
def _merge_matches(*match_sets: list[str]) -> list[str]:
    """Merge matches."""
    merged: list[str] = []
    for match_set in match_sets:
        for match in match_set:
            if match and match not in merged:
                merged.append(match)
    return merged
def _focus_xcloud_window() -> None:
    """Focus xcloud window."""
    if shutil.which("xdotool") is None:
        return
    search_commands = [
        ["xdotool", "search", "--onlyvisible", "--classname", "unifideck-xcloud"],
        ["xdotool", "search", "--onlyvisible", "--classname", "www.xbox.com__play"],
        [
            "xdotool", "search", "--onlyvisible", "--name",
            "Xbox Cloud Gaming|xbox.com",
        ],
    ],
    for _ in range(20):
        for command in search_commands:

```

```

        result = subprocess.run(
            command,
            capture_output=True,
            text=True,
            check=False,
            timeout=5,
        )
        window_ids = [
            line.strip() for line in result.stdout.splitlines()
            if line.strip()
        ]
        if not window_ids:
            continue
        window_id = window_ids[-1]
        for activate_cmd in (
            ["xdotool", "windowactivate", "--sync", window_id],
            ["xdotool", "windowraise", window_id],
            ["xdotool", "windowfocus", window_id],
        ):
            subprocess.run(
                activate_cmd,
                stdout=subprocess.DEVNULL,
                stderr=subprocess.DEVNULL,
                check=False,
                timeout=3,
            )
            logger.info("[xcloud_cdp] refocused xCloud window %s", window_id)
        return
    time.sleep(0.25)
async def run_cdp_inject(
    *,
    port: int,
    launch_url: str,
    timeout: float = 45.0,
    initial_matches: list[str] | None = None,
    steam_port: int = 8080,
    steam_controller_appid: int = 0,
    steam_controller_delay: float = 10.0,
    steam_controller_dwell: float = 2.5,
) -> bool:
    """Run CDP inject."""
    shims_js = get_xcloud_browser_shims_js()
    navigation_js = (
        get_xcloud_navigation_js(launch_url) if launch_url else ""
    )
    if not await _run_inject_phases(
        port, timeout,
        shims_js=shims_js,
        navigation_js=navigation_js,
        launch_url=launch_url,
        initial_matches=initial_matches,
    ):
        return False
    if steam_controller_appid > 0:
        return await refresh_steam_controller_layout(
            steam_port,
            steam_controller_appid,
            delay=steam_controller_delay,
            dwell=steam_controller_dwell,
            on_complete=_focus_xcloud_window,
        )

```

```
    return True

async def _run_inject_phases(
    port: int,
    timeout: float,
    *,
    shims_js: str,
    navigation_js: str,
    launch_url: str,
    initial_matches: list[str] | None,
) -> bool:

    """Run inject phases."""
    final_matches = _build_launch_matches(launch_url)
    base_matches = initial_matches if initial_matches else ["xbox.com"]
    first_pass_matches = _merge_matches(base_matches, final_matches)
    initial_sources = [shims_js]
    if navigation_js:
        initial_sources.append(navigation_js)
    ok = await inject_scripts(
        port,
        initial_sources,
        url_patterns=first_pass_matches,
        timeout=timeout,
        logger_prefix="xcloud-cdp",
    )
    if not ok:
        logger.warning("[xcloud_cdp] initial shim inject failed")
        return False
    if launch_url:
        targeted_sources = (
            [shims_js, navigation_js] if navigation_js else [shims_js]
        )
        ok = await inject_scripts(
            port,
            targeted_sources,
            url_patterns=final_matches,
            timeout=timeout,
            logger_prefix="xcloud-cdp-final",
        )
        if not ok:
            logger.warning("[xcloud_cdp] targeted shim inject failed")
            return False
    return True
```

OP-38c

steam_controller_popup.py

Fichier : py_modules/unifideck/launcher/cdp/steam_controller_popup.py | Lignes : 180 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from collections.abc import Awaitable, Callable
from typing import Any
import aiohttp
from .cdp_primitives import (
    close_target,
    close_titled_targets,
    wait_for_titled_target,
)
from .steam_controller_popup_fiber import (
    inspect_popup_state,
    preview_popup_config,
    resolve_popup_preview_context,
    set_active_popup_config,
)
from .steam_controller_popup_targets import (
    open_controller_popup,
    wait_for_popup_root_ready,
)
logger = logging.getLogger(__name__)
_STEAM_CONTROLLER_LAYOUT_TITLE = "Controller Layout"
_WASD_TEMPLATE_URL = "template://controller_neptune_wasd.vdf"
_JOYSTICK_TEMPLATE_URL = "template://controller_neptune_gamepad_fps.vdf"
_PostBounceHook = Callable[[], Awaitable[Any]] | Callable[[], Any] | None
async def _phase1_preview_wasd(
    websocket: aiohttp.ClientWebSocketResponse,
    h_v3_object: str,
    shortcut_appid: int,
    controller_index: int,
    dwell: float,
) -> dict[str, Any]:
    """Phase1 preview wasd."""
    await preview_popup_config(
        websocket,
        h_v3_object,
        shortcut_appid,
        controller_index,
        _WASD_TEMPLATE_URL,
        msg_id=2001,
    )
    await asyncio.sleep(dwell)
    return await inspect_popup_state(websocket, msg_id=2002)
async def _phase2_activate_joystick(
    websocket: aiohttp.ClientWebSocketResponse,
    h_v3_object: str,
    shortcut_appid: int,
    controller_index: int,
) -> None:
    """Phase2 activate joystick."""
    await set_active_popup_config(
        websocket,
        h_v3_object,
        shortcut_appid,

```

```

        controller_index,
        _JOYSTICK_TEMPLATE_URL,
        msg_id=2003,
    )
    await asyncio.sleep(0.5)

async def _phase3_confirm_joystick(
    websocket: aiohttp.ClientWebSocketResponse,
    h_v3_object: str,
    shortcut_appid: int,
    controller_index: int,
) -> dict[str, Any]:
    """Phase3 confirm joystick."""
    await preview_popup_config(
        websocket,
        h_v3_object,
        shortcut_appid,
        controller_index,
        _JOYSTICK_TEMPLATE_URL,
        msg_id=2004,
    )
    await asyncio.sleep(0.75)
    return await inspect_popup_state(websocket, msg_id=2005)

async def _run_bounce_sequence(
    websocket: aiohttp.ClientWebSocketResponse,
    shortcut_appid: int,
    dwell: float,
) -> bool:
    """Run bounce sequence."""
    if not await wait_for_popup_root_ready(websocket):
        raise RuntimeError(
            "Controller Layout popup never reached the root page",
        )
    h_v3_object, controller_index = await resolve_popup_preview_context(
        websocket,
    )
    logger.info(
        "[popup] using controller index %s for AppID %s",
        controller_index,
        shortcut_appid,
    )
    wasd_state = await _phase1_preview_wasd(
        websocket, h_v3_object, shortcut_appid, controller_index, dwell,
    )
    logger.info(
        "[popup] after-wasd title=%s url=%s",
        wasd_state.get("title"), wasd_state.get("url"),
    )
    await _phase2_activate_joystick(
        websocket, h_v3_object, shortcut_appid, controller_index,
    )
    final_state = await _phase3_confirm_joystick(
        websocket, h_v3_object, shortcut_appid, controller_index,
    )
    logger.info(
        "[popup] after-joystick title=%s url=%s",
        final_state.get("title"), final_state.get("url"),
    )
    return final_state.get("url") == _JOYSTICK_TEMPLATE_URL
async def _open_popup_and_run_bounce(

```

```

    steam_port: int,
    shortcut_appid: int,
    dwell: float,
) -> tuple[bool, str | None]:
    """Open popup and run bounce."""
    logger.info(
        "[popup] opening controller configurator for AppID %s",
        shortcut_appid,
    )
    await open_controller_popup(steam_port, shortcut_appid)
    popup_target = await wait_for_titled_target(
        steam_port, _STEAM_CONTROLLER_LAYOUT_TITLE, timeout=15.0,
    )
    if not popup_target:
        raise RuntimeError("Controller Layout popup did not open")
    popup_target_id = str(popup_target["id"])
    async with aiohttp.ClientSession() as session, session.ws_connect(
        popup_target["websocketDebuggerUrl"], heartbeat=10, autoping=True,
    ) as websocket:
        success = await _run_bounce_sequence(
            websocket, shortcut_appid, dwell,
        )
    return success, popup_target_id
async def _close_popup(steam_port: int, popup_target_id: str | None) -> None:
    """Close popup."""
    if popup_target_id:
        await close_target(steam_port, popup_target_id)
    else:
        await close_titled_targets(
            steam_port, _STEAM_CONTROLLER_LAYOUT_TITLE,
        )

async def _invoke_post_bounce_hook(on_complete: _PostBounceHook) -> None:
    """Invoke post bounce hook."""
    if on_complete is None:
        return
    try:
        result = on_complete()
        if asyncio.iscoroutine(result):
            await result
    except Exception as exc:
        logger.debug("[popup] on_complete hook failed: %s", exc)
async def refresh_steam_controller_layout(
    steam_port: int,
    shortcut_appid: int,
    *,
    delay: float,
    dwell: float,
    on_complete: _PostBounceHook = None,
) -> bool:
    """Refresh steam controller layout."""
    if shortcut_appid <= 0:
        return False
    await asyncio.sleep(delay)
    await close_titled_targets(steam_port, _STEAM_CONTROLLER_LAYOUT_TITLE)
    popup_target_id: str | None = None
    success = False
    try:
        success, popup_target_id = await _open_popup_and_run_bounce(
            steam_port, shortcut_appid, dwell,

```

```
    )
except Exception as exc:
    logger.exception("[popup] bounce failed: %s", exc)
finally:
    await _close_popup(steam_port, popup_target_id)
    await _invoke_post_bounce_hook(on_complete)
return success
```


OP-38d

steam_controller_popup_targets.py

Fichier : py_modules/unifideck/launcher/cdp/steam_controller_popup_targets.py | Lignes : 56 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import aiohttp
from .cdp_primitives import (
    cdp_command,
    evaluate_in_target,
    wait_for_titled_target,
)
logger = logging.getLogger(__name__)
_STEAM_SHARED_CONTEXT_TITLE = "SharedJSContext"
async def open_controller_popup(steam_port: int, appid: int) -> None:
    """Open controller popup."""
    shared_target = await wait_for_titled_target(
        steam_port,
        _STEAM_SHARED_CONTEXT_TITLE,
        timeout=10.0,
    )
    if not shared_target:
        raise RuntimeError("SharedJSContext target not found")
    expression = (
        f"(async () => {{ { "
        f"window.SteamClient?.Apps?.ShowControllerConfigurator?.({appid}); "
        f"return 'opened'; "
        f"}}})()"
    )
    await evaluate_in_target(shared_target, expression)
async def wait_for_popup_root_ready(
    websocket: aiohttp.ClientWebSocketResponse,
) -> bool:
    """Wait for popup root ready."""
    for attempt in range(60):
        try:
            resp = await cdp_command(
                websocket,
                3000 + attempt,
                "Runtime.evaluate",
                {
                    "expression": (
                        "Array.from("
                        "document.querySelectorAll('button,[role=\\\"link\\\"]') "
                        ").some((e) => (e.textContent || '').trim() === "
                        "'View Layout') "
                    ),
                    "returnByValue": True,
                },
            )
            value = (
                resp.get("result", {}).get("result", {}).get("value")
            )
            if value is True:
                return True
        except Exception as exc:
            logger.debug("[popup] root poll failed: %s", exc)
            await asyncio.sleep(0.25)

```

```
return False
```

OP-38e

steam_controller_popup_fiber.py

Fichier : py_modules/unifideck/launcher/cdp/steam_controller_popup_fiber.py | Lignes : 295 | Depend de : (rien)

```

from __future__ import annotations
import contextlib
import logging
from typing import Any
import aiohttp
from .cdp_primitives import cdp_command
logger = logging.getLogger(__name__)
async def _click_handler_object_id(
    websocket: aiohttp.ClientWebSocketResponse,
    msg_id: int,
) -> str:
    """Click handler object ID."""
    resp = await cdp_command(
        websocket,
        msg_id,
        "Runtime.evaluate",
        {
            "expression": r"""(() => {
                const node = Array.from(
                    document.querySelectorAll('button,[role="link"]')
                ).find((element) => (element.textContent || '').trim() === 'View Layout');
                if (!node) {
                    return null;
                }
                const fiberKey = Object.keys(node).find((key) =>
                    key.startsWith('__reactFiber'));
                let fiber = fiberKey ? node[fiberKey] : null;
                while (fiber) {
                    const props = fiber.memoizedProps || {};
                    if (
                        typeof props.onClick === 'function' &&
                        String(props.onClick).includes('ControllerConfigurator.Summary')
                    ) {
                        return props.onClick;
                    }
                    fiber = fiber.return;
                }
                return null;
            })()""",
            "awaitPromise": True,
            "returnByValue": False,
            "userGesture": True,
        },
    )
    object_id = resp.get("result", {}).get("result", {}).get("objectId")
    if not object_id:
        raise RuntimeError("Could not resolve controller popup preview context")
    return str(object_id)

async def _scopes_object_id(
    websocket: aiohttp.ClientWebSocketResponse,
    on_click_object: str,
    msg_id: int,
) -> str:

```

```

"""Scopes object ID."""
resp = await cdp_command(
    websocket,
    msg_id,
    "Runtime.getProperties",
    {
        "objectId": on_click_object,
        "ownProperties": False,
        "generatePreview": True,
    },
)
scopes_object = next(
    (
        item["value"]["objectId"]
        for item in resp.get("result", {}).get("internalProperties", [])
        if item.get("name") == "[[Scopes]]"
    ),
    None,
)
if not scopes_object:
    raise RuntimeError("Could not inspect controller popup scopes")
return str(scopes_object)
async def _scopel_object_id(
    websocket: aiohttp.ClientWebSocketResponse,
    scopes_object: str,
    msg_id: int,
) -> str:
    """Scopel object ID."""
    resp = await cdp_command(
        websocket,
        msg_id,
        "Runtime.getProperties",
        {
            "objectId": scopes_object,
            "ownProperties": True,
            "generatePreview": True,
        },
    )
    scopel_object = next(
        (
            item["value"]["objectId"]
            for item in resp.get("result", {}).get("result", [])
            if item.get("name") == "1"
        ),
        None,
    )
    if not scopel_object:
        raise RuntimeError("Could not resolve configurator module scope")
    return str(scopel_object)
async def _scopel_h_object_id(
    websocket: aiohttp.ClientWebSocketResponse,
    scopel_object: str,
    msg_id: int,
) -> str:
    """Scopel h object ID."""
    resp = await cdp_command(
        websocket,
        msg_id,
        "Runtime.getProperties",
        {
            "objectId": scopel_object,

```

```

        "ownProperties": True,
        "generatePreview": True,
    },
)
scope_lookup = {
    prop["name"]: prop.get("value", {}).get("objectId")
    for prop in resp.get("result", {}).get("result", [])
    if prop.get("value", {}).get("objectId")
}
h_object = scope_lookup.get("h")
if not h_object:
    raise RuntimeError("Could not resolve h.v3 controller helper")
return str(h_object)

async def _h_v3_object_id(
    websocket: aiohttp.ClientWebSocketResponse,
    h_object: str,
    msg_id: int,
) -> tuple[str, int]:

    """H v3 object ID."""
    resp = await cdp_command(
        websocket,
        msg_id,
        "Runtime.getProperties",
        {
            "objectId": h_object,
            "ownProperties": True,
            "generatePreview": True,
        },
    )
    v3_object = next(
        (
            prop.get("value", {}).get("objectId")
            for prop in resp.get("result", {}).get("result", [])
            if prop.get("name") == "v3"
        ),
        None,
    )
    if not v3_object:
        raise RuntimeError("Could not resolve h.v3 sub-object")
    controller_index = 0
    with contextlib.suppress(Exception):
        v3_props = await cdp_command(
            websocket,
            msg_id + 1,
            "Runtime.getProperties",
            {"objectId": v3_object, "ownProperties": True},
        )
        for prop in v3_props.get("result", {}).get("result", []):
            if prop.get("name") == "currentController":
                value = prop.get("value", {}).get("value")
                if isinstance(value, int):
                    controller_index = value
                    break
    return str(v3_object), controller_index

async def resolve_popup_preview_context(
    websocket: aiohttp.ClientWebSocketResponse,
) -> tuple[str, int]:
    """Resolve popup preview context."""
    on_click = await _click_handler_object_id(websocket, 1000)

```

```

    scopes = await _scopes_object_id(websocket, on_click, 1001)
    scopel = await _scopel_object_id(websocket, scopes, 1002)
    h_obj = await _scopel_h_object_id(websocket, scopel, 1003)
    return await _h_v3_object_id(websocket, h_obj, 1004)
async def preview_popup_config(
    websocket: aiohttp.ClientWebSocketResponse,
    h_v3_object: str,
    appid: int,
    controller_index: int,
    config_url: str,
    *,
    msg_id: int,
) -> None:
    """Preview popup config."""
    await cdp_command(
        websocket,
        msg_id,
        "Runtime.callFunctionOn",
        {
            "objectId": h_v3_object,
            "functionDeclaration": (
                "function(appid, controllerIndex, url){ "
                "this.PreviewConfigForApp(appid, controllerIndex, url); "
                "return true; "
                "}"
            ),
            "arguments": [
                {"value": appid},
                {"value": controller_index},
                {"value": config_url},
            ],
            "awaitPromise": True,
            "returnByValue": True,
        },
    )

async def set_active_popup_config(
    websocket: aiohttp.ClientWebSocketResponse,
    h_v3_object: str,
    appid: int,
    controller_index: int,
    config_url: str,
    *,
    msg_id: int,
) -> None:
    """Set active popup config."""
    await cdp_command(
        websocket,
        msg_id,
        "Runtime.callFunctionOn",
        {
            "objectId": h_v3_object,
            "functionDeclaration": (
                "function(appid, controllerIndex, url){ "
                "this.SetActiveConfigForApp(appid, controllerIndex, url, false); "
                "this.SaveEditingConfiguration(appid); "
                "if (typeof this.ClearSelectedConfigCache === 'function') { "
                "    this.ClearSelectedConfigCache(appid); "
                "}"
                "this.EnsureEditingConfiguration(appid, controllerIndex); "
            )
        }
    )

```

```

        "return true; "
        "}"
    ),
    "arguments": [
        {"value": appid},
        {"value": controller_index},
        {"value": config_url},
    ],
    "awaitPromise": True,
    "returnByValue": True,
},
)

async def inspect_popup_state(
    websocket: aiohttp.ClientWebSocketResponse,
    *,
    msg_id: int,
) -> dict[str, Any]:

    """Inspect popup state."""
    state_resp = await cdp_command(
        websocket,
        msg_id,
        "Runtime.evaluate",
        {
            "expression": r"""(() => {
                const node = Array.from(
                    document.querySelectorAll('button,[role="link"]')
                ).find((element) => (
                    (element.textContent || '').includes('Official Layout for') ||
                    (element.textContent || '').includes('Using Template:') ||
                    (element.textContent || '').includes('Gamepad With Joystick Trackpad')
                    ||
                    (element.textContent || '').includes('Keyboard (WASD) and Mouse')
                ));
                const fiberKey = node ? Object.keys(node).find(
                    (key) => key.startsWith('__reactFiber')
                ) : null;
                let fiber = fiberKey ? node[fiberKey] : null;
                let config = null;
                while (fiber) {
                    const props = fiber.memoizedProps || {};
                    if (props.config && typeof props.config === 'object') {
                        config = props.config;
                        break;
                    }
                    fiber = fiber.return;
                }
                return {
                    body: document.body
                        ? document.body.innerText.slice(0, 1200)
                        : null,
                    title: config?.Title || null,
                    url: config?.URL || null,
                    progenitor: config?.ProgenitorURL || null,
                    usesMouse: config?.bUsesMouse ?? null,
                    usesKeyboard: config?.bUsesKeyboard ?? null,
                    usesGamepad: config?.bUsesGamepad ?? null,
                };
            })()""",
            "awaitPromise": True,

```

```
        "returnByValue": True,  
        "userGesture": True,  
    },  
)  
value = state_resp.get("result", {}).get("result", {}).get("value")  
return value if isinstance(value, dict) else {}
```


OP-39

__init__.py

Fichier : py_modules/unifideck/launcher/cloud/__init__.py | Lignes : 0 | Depend de : (rien)

OP-39a

cloud_failure.py

Fichier : py_modules/unifideck/launcher/cloud/cloud_failure.py | Lignes : 179 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
from ...core.types.events import Events
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
def classify_cloud_error(err: BaseException) -> str:
    """Classify cloud error."""
    try:
        import aiohttp
    except ImportError:
        aiohttp = None
    if aiohttp is not None:
        if isinstance(err, aiohttp.ClientConnectionError):
            return "network_unreachable"
        if isinstance(err, aiohttp.ClientResponseError):
            status = getattr(err, "status", 0)
            if status == 401:
                return "auth_expired"
            if status == 403:
                return "forbidden"
            if status == 413:
                return "quota_exceeded"
            if 500 <= status < 600:
                return "server_error"
        if isinstance(err, aiohttp.ClientTimeout):
            return "timed_out"
    if isinstance(err, OSError):
        import errno
        if err.errno == errno.ENOSPC:
            return "disk_full"
        if err.errno in (errno.EACCES, errno.EPERM):
            return "permission_denied"
    try:
        from .disk_space import LowDiskSpaceError
        if isinstance(err, LowDiskSpaceError):
            return "disk_space_low"
    except ImportError:
        pass
    try:
        import asyncio
        if isinstance(err, asyncio.CancelledError):
            return "cancelled"
    except ImportError:
        pass
    return "unknown"

_DEFAULT_BEHAVIOR = "toast"
_VALID_BEHAVIORS = frozenset({"silent", "toast"})
def get_failure_behavior(config: ConfigManager | None, store: str) -> str:
    """Get failure behavior."""
    if config is None or not hasattr(config, "get_str"):
        return _DEFAULT_BEHAVIOR

```

```

key = f"cloud.failure_behavior.{store}"
raw = config.get_str(key, "")
if not raw:
    raw = config.get_str(
        "cloud.failure_behavior.default", _DEFAULT_BEHAVIOR,
    )
if raw not in _VALID_BEHAVIORS:
    logger.warning(
        "[cloud_failure] invalid behavior %r for store %s, "
        "falling back to %r", raw, store, _DEFAULT_BEHAVIOR,
    )
    return _DEFAULT_BEHAVIOR
return raw

async def handle_cloud_sync_failure(
    bus: EventBus,
    config: ConfigManager | None,
    *,
    phase: str,
    store: str,
    game_id: str,
    error: BaseException,
) -> None:
    """Handle cloud sync failure."""
    code = classify_cloud_error(error)
    behavior = get_failure_behavior(config, store)
    logger.error(
        "[cloud_failure] phase=%s store=%s game_id=%s "
        "error_code=%s behavior=%s error=%s",
        phase, store, game_id, code, behavior, error,
        exc_info=error,
    )
    if behavior == "silent":
        return
    await _emit_toast(bus, phase=phase, store=store, game_id=game_id, code=code)

async def _emit_toast(
    bus: EventBus, *,
    phase: str, store: str, game_id: str, code: str,
) -> None:
    """Emit toast."""
    if bus is None:
        return
    i18n_key = (
        "toasts.launcher.cloudSyncDownFailed"
        if phase == "sync_down"
        else "toasts.launcher.cloudSyncUpFailed"
    )
    payload: dict[str, Any] = {
        "severity": "warning",
        "i18n_key": i18n_key,
        "i18n_params": {
            "store": store,
            "error_code": code,
            "error_i18n_key": f"cloudSync.error.{code}",
        },
        "duration_ms": 6000,
        "game_id": game_id,
        "store": store,
        "phase": phase,
    }
    resolved_action = _resolve_toast_action(
        code, store=store, game_id=game_id, phase=phase,

```

```

    )
    if resolved_action is not None:
        payload["action"] = resolved_action
    try:
        await bus.emit(Events.LAUNCHER_STAGE, **payload)
    except Exception:
        logger.exception("[cloud_failure] toast emit failed")

def _resolve_toast_action(
    code: str, *, store: str, game_id: str, phase: str,
) -> dict[str, str] | None:

    """Resolve toast action."""
    action = _TOAST_ACTIONS.get(code)
    if action is None:
        return None
    ctx_vars = {"store": store, "game_id": game_id, "phase": phase}
    working: dict[str, str] = dict(action)
    for url_key in ("target_url", "fallback_url"):
        if url_key not in working:
            continue
        try:
            working[url_key] = working[url_key].format(**ctx_vars)
        except (KeyError, IndexError) as err:
            logger.warning(
                "[cloud_failure] action template error for "
                "code=%s key=%s: %s - dropping action",
                code, url_key, err,
            )
            return None
    return working

_TOAST_ACTIONS = {
    "disk_space_low": {
        "i18n_label_key": "toasts.actions.openStorageManager",
        "target_url": "steam://settings/storage",
        "fallback_url": "steam://settings",
    },
    "auth_expired": {
        "i18n_label_key": "toasts.actions.signInToStore",
        "target_url": "unifideck://auth/{store}",
    },
    "network_unreachable": {
        "i18n_label_key": "toasts.actions.retrySync",
        "target_url": "unifideck://retry-sync/{store}/{game_id}/{phase}",
    },
    "timed_out": {
        "i18n_label_key": "toasts.actions.retrySync",
        "target_url": "unifideck://retry-sync/{store}/{game_id}/{phase}",
    },
    "server_error": {
        "i18n_label_key": "toasts.actions.retrySync",
        "target_url": "unifideck://retry-sync/{store}/{game_id}/{phase}",
    },
    "unknown": {
        "i18n_label_key": "toasts.actions.openSaveFolder",
        "target_url": "unifideck://open-save-folder/{store}/{game_id}",
    },
    "permission_denied": {
        "i18n_label_key": "toasts.actions.openSaveFolder",
        "target_url": "unifideck://open-save-folder/{store}/{game_id}",
    },
}

```

```
"cancelled": {  
  "i18n_label_key": "toasts.actions.openSaveFolder",  
  "target_url": "unifideck://open-save-folder/{store}/{game_id}",  
},  
}
```

OP-39b

disk_space.py**Fichier :** py_modules/unifideck/launcher/cloud/disk_space.py | Lignes : 92 | Depend de : (rien)

```

from __future__ import annotations
import logging
import shutil
from dataclasses import dataclass
from pathlib import Path
from typing import TYPE_CHECKING, cast
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
_DEFAULT_MIN_FREE_BYTES = 1024 * 1024 * 1024
@dataclass(frozen=True)
class DiskSpaceCheck:
    """Disk space check."""
    has_space: bool
    free_bytes: int
    required_bytes: int
    path: Path
class LowDiskSpaceError(Exception):
    """Low disk space error."""
    def __init__(
        self,
        message: str,
        free_bytes: int,
        required_bytes: int,
    ) -> None:
        """Initialize the instance."""
        super().__init__(message)
        self.free_bytes = free_bytes
        self.required_bytes = required_bytes
def get_min_free_bytes(config: ConfigManager | None) -> int:
    """Get min free bytes."""
    if config is None or not hasattr(config, "get_int"):
        return _DEFAULT_MIN_FREE_BYTES
    return config.get_int(
        "disk_space.min_free_bytes", _DEFAULT_MIN_FREE_BYTES,
    )
def check_disk_space(
    path: Path, required_bytes: int,
) -> DiskSpaceCheck:
    """Check disk space."""
    probe = path
    while probe != probe.parent and not probe.exists():
        probe = probe.parent
    try:
        usage = shutil.disk_usage(str(probe))
        return DiskSpaceCheck(
            has_space=usage.free >= required_bytes,
            free_bytes=usage.free,
            required_bytes=required_bytes,
            path=probe,
        )
    except OSError as err:
        logger.warning(
            "[disk_space] disk_usage(%) failed: %s – assuming no space",
            probe, err,

```

```
)
return DiskSpaceCheck(
    has_space=False,
    free_bytes=0,
    required_bytes=required_bytes,
    path=probe,
)

def assert_enough_space(
    path: Path, config: ConfigManager | None,
    *, store: str | None = None,
    game_id: str | None = None,
) -> None:
    """Assert enough space."""
    required = get_min_free_bytes(config)
    if store and game_id:
        try:
            from .save_size_cache import get_observed_size
            cached = get_observed_size(config, store, game_id)
            if cached is not None and not cached.stale:
                multiplier = _get_multiplier(config)
                refined = int(cached.size_bytes * multiplier)
                required = max(required, refined)
        except ImportError:
            pass
    check = check_disk_space(path, required)
    if not check.has_space:
        raise LowDiskSpaceError(
            f"low disk space at {check.path}: "
            f"have {check.free_bytes} bytes, need {required}",
            free_bytes=check.free_bytes,
            required_bytes=required,
        )

def _get_multiplier(config: ConfigManager | None) -> float:
    """Get multiplier."""
    if config is None or not hasattr(config, "get_float"):
        return 1.5
    return cast("float", config.get_float("disk_space.size_multiplier", 1.5))
```

OP-39c

save_size_cache.py

Fichier : py_modules/unifideck/launcher/cloud/save_size_cache.py | Lignes : 121 | Depend de : (rien)

```

from __future__ import annotations
import json
import logging
import os
import time
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any, cast
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
_EWMA_ALPHA = 0.3
@dataclass(frozen=True)
class CachedSize:
    """Cached size."""
    size_bytes: int
    observed_at: float
    sample_count: int
    stale: bool
def _resolve_cache_path(config: ConfigManager | None) -> str:
    """Resolve cache path."""
    if config is None or not hasattr(config, "get_str"):
        raw = "~/cache/unifideck/cloud_save_sizes.json"
    else:
        raw = config.get_str(
            "disk_space.size_cache_path",
            "~/cache/unifideck/cloud_save_sizes.json",
        )
    return os.path.expanduser(raw)
def _resolve_ttl_seconds(config: ConfigManager | None) -> int:
    """Resolve TTL seconds."""
    if config is None or not hasattr(config, "get_int"):
        return 30 * 24 * 3600
    return config.get_int(
        "disk_space.size_cache_ttl_seconds", 30 * 24 * 3600,
    )
def _load(path: str) -> dict[str, Any]:
    """Load."""
    if not os.path.isfile(path):
        return {}
    try:
        with open(path, encoding="utf-8") as fh:
            return cast("dict[str, Any]", json.load(fh))
    except (OSError, json.JSONDecodeError) as err:
        logger.warning(
            "[save_size_cache] load failed for %s: %s – "
            "starting empty", path, err,
        )
    return {}
def _save(path: str, data: dict[str, Any]) -> None:
    """Save."""
    try:
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp_path = f"{path}.tmp"
        with open(tmp_path, "w", encoding="utf-8") as fh:
            json.dump(data, fh, indent=2)

```



```

        os.replace(tmp_path, path)
    except OSError as err:
        logger.warning("[save_size_cache] save failed for %s: %s", path, err)

def get_observed_size(
    config: ConfigManager | None, store: str, game_id: str,
) -> CachedSize | None:

    """Get observed size."""
    path = _resolve_cache_path(config)
    data = _load(path)
    key = f"{store}:{game_id}"
    entry = data.get(key)
    if not entry:
        return None
    ttl = _resolve_ttl_seconds(config)
    age = time.time() - entry.get("observed_at", 0)
    return CachedSize(
        size_bytes=int(entry.get("size_bytes", 0)),
        observed_at=float(entry.get("observed_at", 0)),
        sample_count=int(entry.get("sample_count", 0)),
        stale=(age > ttl),
    )

def record_observed_size(
    config: ConfigManager | None, store: str, game_id: str, size_bytes: int,
) -> None:

    """Record observed size."""
    if size_bytes < 0:
        logger.warning(
            "[save_size_cache] negative size for %s:%s ignored",
            store, game_id,
        )
        return
    path = _resolve_cache_path(config)
    data = _load(path)
    key = f"{store}:{game_id}"
    existing = data.get(key)
    if existing is None or existing.get("sample_count", 0) == 0:
        new_size = size_bytes
        new_count = 1
    else:
        old = existing["size_bytes"]
        new_size = int(_EWMA_ALPHA * size_bytes + (1 - _EWMA_ALPHA) * old)
        new_count = existing["sample_count"] + 1
    data[key] = {
        "size_bytes": new_size,
        "observed_at": time.time(),
        "sample_count": new_count,
    }
    _save(path, data)

def measure_directory_size(directory: str) -> int:

    """Measure directory size."""
    if not os.path.isdir(directory):
        return 0
    total = 0
    try:
        for root, _, files in os.walk(directory):
            for name in files:
                try:
                    total += os.path.getsize(os.path.join(root, name))
                except OSError:

```

```
                continue
except OSError as err:
    logger.warning(
        "[save_size_cache] walk failed for %s: %s", directory, err,
    )
    return 0
return total
```

OP-40

__init__.py

Fichier : py_modules/unifideck/launcher/diagnostics/__init__.py | Lignes : 0 | Depend de : (rien)

OP-40a

correlation.py

Fichier : py_modules/unifideck/launcher/diagnostics/correlation.py | Lignes : 39 | Depend de : (rien)

```
from __future__ import annotations
import contextvars
import logging
import secrets
from collections.abc import Iterator
from contextlib import contextmanager

_LAUNCH_ID: contextvars.ContextVar[str] = contextvars.ContextVar(
    "unifideck_launch_id",
    default="-",
)

def new_launch_id() -> str:
    """New launch ID."""
    return secrets.token_hex(4)

def get_launch_id() -> str:
    """Get launch ID."""
    return _LAUNCH_ID.get()

@contextmanager
def launch_id_scope(launch_id: str) -> Iterator[None]:
    """Launch ID scope."""
    token = _LAUNCH_ID.set(launch_id)
    try:
        yield
    finally:
        _LAUNCH_ID.reset(token)

class LaunchIdFilter(logging.Filter):
    """Launch ID filter."""
    def filter(self, record: logging.LogRecord) -> bool:
        """Filter."""
        record.launch_id = get_launch_id()
        return True

def install_launch_id_logging(
    root_logger: logging.Logger | None = None,
) -> None:
    """Install launch ID logging."""
    logger = root_logger or logging.getLogger()
    for existing in logger.filters:
        if isinstance(existing, LaunchIdFilter):
            return
    logger.addFilter(LaunchIdFilter())
```

OP-40b

telemetry.py

Fichier : py_modules/unifideck/launcher/diagnostics/telemetry.py | Lignes : 53 | Depend de : (rien)

```
from __future__ import annotations
import logging
import time
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from ...event_bus.event_bus import EventBus
    from .correlation import get_launch_id
logger = logging.getLogger(__name__)
LAUNCH_PHASE_TIMING_EVENT = "LAUNCH_PHASE_TIMING"
class PhaseTimer:
    """Phase timer."""
    __slots__ = ("_bus", "_phase", "_extra", "_t0")
    def __init__(
        self,
        bus: EventBus,
        phase: str,
        extra: dict | None = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._phase = phase
        self._extra = dict(extra) if extra else {}
        self._t0 = 0.0
    async def __aenter__(self) -> PhaseTimer:
        """Aenter."""
        self._t0 = time.monotonic()
        return self
    async def __aexit__(
        self, exc_type: Any, _exc_val: Any, _exc_tb: Any,
    ) -> None:
        """Aexit."""
        duration_ms = int((time.monotonic() - self._t0) * 1000)
        payload = {
            "phase": self._phase,
            "duration_ms": duration_ms,
            "launch_id": get_launch_id(),
            "success": exc_type is None,
            **self._extra,
        }
        if self._bus is not None:
            try:
                await self._bus.emit(
                    LAUNCH_PHASE_TIMING_EVENT, **payload,
                )
            except Exception:
                logger.exception(
                    "[telemetry] emit failed for phase=%s",
                    self._phase,
                )
        logger.debug(
            "[telemetry] phase=%s duration_ms=%d success=%s",
            self._phase, duration_ms, exc_type is None,
        )
    )
```

OP-40c

save_folder_inspector.py

Fichier : py_modules/unifideck/launcher/diagnostics/save_folder_inspector.py | Lignes : 78 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
from typing import Any
logger = logging.getLogger(__name__)
def inspect_save_folder(
    root: str,
    *,
    max_depth: int = 2,
    filter_substring: str = "",
    max_files: int = 500,
) -> dict[str, Any]:
    """Inspect save folder."""
    result: dict[str, Any] = {
        "path": root,
        "exists": False,
        "total_files": 0,
        "total_size": 0,
        "files": [],
        "truncated_count": 0,
        "truncated_size": 0,
    }
    if not os.path.isdir(root):
        return result
    result["exists"] = True
    substr = filter_substring.lower() if filter_substring else ""
    all_entries = _collect_file_entries(root, max_depth, substr)
    result["total_files"] = len(all_entries)
    result["total_size"] = sum(e["size"] for e in all_entries)
    all_entries.sort(key=lambda e: e["size"], reverse=True)
    _apply_file_cap(all_entries, max_files, result)
    return result

def _collect_file_entries(
    root: str, max_depth: int, substr: str,
) -> list[dict[str, Any]]:
    """Collect file entries."""
    entries: list[dict[str, Any]] = []
    try:
        for dirpath, dirnames, filenames in os.walk(root):
            rel_dir = os.path.relpath(dirpath, root)
            depth = 0 if rel_dir == "." else rel_dir.count(os.sep) + 1
            if max_depth >= 0 and depth >= max_depth:
                dirnames[:] = []
            for name in filenames:
                full = os.path.join(dirpath, name)
                rel_norm = os.path.relpath(full, root).replace(
                    os.sep, "/",
                )
                if substr and substr not in rel_norm.lower():
                    continue
                try:
                    st = os.stat(full)
                except OSError:

```

```
        continue
    entries.append({
        "rel_path": rel_norm,
        "size": st.st_size,
        "mtime": st.st_mtime,
    })
except OSError as err:
    logger.warning(
        "[save_folder_inspector] walk failed for %s: %s",
        root, err,
    )
return entries

def _apply_file_cap(
    entries: list[dict[str, Any]],
    max_files: int,
    result: dict[str, Any],
) -> None:
    """Apply file cap."""
    if len(entries) > max_files:
        result["files"] = entries[:max_files]
        dropped = entries[max_files:]
        result["truncated_count"] = len(dropped)
        result["truncated_size"] = sum(e["size"] for e in dropped)
    else:
        result["files"] = entries
```

OP-40d

log_archive.py

Fichier : py_modules/unifideck/launcher/diagnostics/log_archive.py | Lignes : 159 | Depend de : (rien)

```
from __future__ import annotations
import logging
import os
import time
from pathlib import Path
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
def _resolve_archive_dir(config: ConfigManager | None) -> Path:
    """Resolve archive dir."""
    if config is None or not hasattr(config, "get_str"):
        raw = "~/local/share/unifideck/launches"
    else:
        raw = config.get_str(
            "logs.archive_path",
            "~/local/share/unifideck/launches",
        )
    path = Path(os.path.expanduser(raw))
    try:
        path.mkdir(parents=True, exist_ok=True)
    except OSError as err:
        logger.warning(
            "[log_archive] failed to create %s: %s – archiving disabled",
            path, err,
        )
    return path
def _resolve_retention_seconds(config: ConfigManager | None) -> int:
    """Resolve retention seconds."""
    if config is None or not hasattr(config, "get_int"):
        return 7 * 24 * 3600
    return config.get_int("logs.retention_days", 7) * 24 * 3600
def prune_old_launches(config: ConfigManager | None) -> int:
    """Prune old launches."""
    archive_dir = _resolve_archive_dir(config)
    if not archive_dir.is_dir():
        return 0
    cutoff = time.time() - _resolve_retention_seconds(config)
    removed = 0
    try:
        for entry in archive_dir.iterdir():
            if not entry.is_file() or entry.suffix != ".log":
                continue
            try:
                if entry.stat().st_mtime < cutoff:
                    entry.unlink()
                    removed += 1
            except OSError as err:
                logger.warning(
                    "[log_archive] failed to prune %s: %s",
                    entry, err,
                )
    except OSError as err:
        logger.warning(
            "[log_archive] failed to scan %s: %s", archive_dir, err,
```



```

    )
    if removed > 0:
        logger.info(
            "[log_archive] pruned %d expired launch log(s)", removed,
        )
    return removed

def attach_launch_handler(
    launch_id: str, config: ConfigManager | None,
    *, min_level: int = logging.INFO,
) -> logging.Handler | None:
    """Attach launch handler."""
    archive_dir = _resolve_archive_dir(config)
    path = archive_dir / f"{launch_id}.log"
    try:
        handler = logging.FileHandler(str(path), encoding="utf-8")
        handler.setLevel(min_level)
        handler.setFormatter(logging.Formatter(
            "%(asctime)s [% (levelname)s] %(name)s: %(message)s",
        ))
        logging.getLogger("unifideck.launcher").addHandler(handler)
        return handler
    except OSError as err:
        logger.warning(
            "[log_archive] failed to attach handler for %s: %s",
            path, err,
        )
        return None

def detach_launch_handler(handler: logging.Handler | None) -> None:
    """Detach launch handler."""
    if handler is None:
        return
    try:
        logging.getLogger("unifideck.launcher").removeHandler(handler)
        handler.close()
    except Exception:
        logger.exception("[log_archive] detach handler failed")

def read_launch_logs(
    launch_id: str, config: ConfigManager | None,
    *, max_lines: int = 500,
) -> dict:
    """Read launch logs."""
    archive_dir = _resolve_archive_dir(config)
    path = archive_dir / f"{launch_id}.log"
    result = {
        "exists": False, "path": str(path), "lines": [],
        "total": 0,
    }
    if not path.is_file():
        return result
    result["exists"] = True
    try:
        with path.open("r", encoding="utf-8", errors="replace") as fh:
            raw = fh.readlines()
    except OSError as err:
        logger.warning("[log_archive] read %s failed: %s", path, err)
        return result
    result["total"] = len(raw)
    tail = raw[-max_lines:] if len(raw) > max_lines else raw
    parsed = []

```

```
for line in tail:
    level = "INFO"
    if "[ERROR]" in line or "[CRITICAL]" in line:
        level = "ERROR"
    elif "[WARNING]" in line:
        level = "WARNING"
    elif "[DEBUG]" in line:
        level = "DEBUG"
    parsed.append({"level": level, "text": line.rstrip("\n")})
result["lines"] = parsed
return result

def export_launch_logs(
    launch_id: str, dest_path: str, config: ConfigManager | None,
) -> dict:
    """Export launch logs."""
    import shutil
    archive_dir = _resolve_archive_dir(config)
    src = archive_dir / f"{launch_id}.log"
    if not src.is_file():
        return {
            "success": False,
            "error": "source_missing",
            "dest_path": None,
        }
    dst = Path(os.path.expanduser(dest_path))
    if not dst.is_absolute():
        dst = Path.home() / dst
    try:
        dst.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy2(str(src), str(dst))
    except OSError as err:
        logger.warning(
            "[log_archive] export %s → %s failed: %s",
            src, dst, err,
        )
        return {
            "success": False,
            "error": str(err),
            "dest_path": str(dst),
        }
    return {
        "success": True,
        "dest_path": str(dst),
        "error": None,
    }
```

OP-41

__init__.py

Fichier : py_modules/unifideck/launcher/proton/__init__.py | Lignes : 24 | Depend de : (rien)

```
from __future__ import annotations
from .handlers.epic import epic_launch
from .handlers.generic import generic_launch
from .handlers.ubisoft import ubisoft_launch
from .infrastructure.core import ProtonLaunchPlan, proton_prepare
from .infrastructure.selector import (
    find_python_3_10_plus,
    resolve_proton_path,
    select_proton_version,
)
from .infrastructure.umu_runtime import (
    UMU_CACHE_DIR,
    cleanup_umu_runtime_cache,
    ensure_umu_runtime_ready,
    run_umu_with_retry,
)
async def dispatch(plan: ProtonLaunchPlan) -> int:
    """Dispatch."""
    store = plan.context.store
    if store == "ubisoft":
        return await ubisoft_launch(plan)
    if store == "epic":
        return await epic_launch(plan)
    return await generic_launch(plan)
```

OP-42

__init__.py

Fichier : py_modules/unifideck/launcher/proton/infrastructure/__init__.py | Lignes : 0 | Depend de : (rien)

OP-42a

core.py

Fichier : py_modules/unifideck/launcher/proton/infrastructure/core.py | Lignes : 120 | Depend de : (rien)

```

from __future__ import annotations
import logging
import shutil
import subprocess
from collections.abc import Callable
from dataclasses import dataclass
from pathlib import Path
from ...types.context import LaunchContext, RuntimeState
from ...types.errors import DependencyMissingError
logger = logging.getLogger(__name__)
STORE_TO_UMU = {
    "epic": "egs",
    "gog": "gog",
    "amazon": "amazon",
    "ubisoft": "ubisoft",
    "microsoft": "microsoft",
}
}
@dataclass(frozen=True)
class ProtonLaunchPlan:
    """Proton launch plan."""
    context: LaunchContext
    state: RuntimeState
    python_bin: Path
    umu_wrapper: Path
    prefix_path: Path
    env: dict[str, str]
    on_process_start: Callable[[object], None] | None = None
def _ubisoft_prefix_path(ctx: LaunchContext, prefixes_dir: Path) -> Path:
    """Ubisoft prefix path."""
    import os
    ubi_name = os.environ.get("UNIFIDECK_UBISOFT_PREFIX_NAME") or ctx.game_id
    return prefixes_dir / "ubisoft" / ubi_name
def _resolve_prefix(ctx: LaunchContext) -> Path:
    """Resolve prefix."""
    prefixes_dir = Path("~/local/share/unifideck/prefixes").expanduser()
    if ctx.store == "ubisoft":
        path = _ubisoft_prefix_path(ctx, prefixes_dir)
    else:
        path = prefixes_dir / ctx.game_id
        while path.name == "pfx":
            path = path.parent
        path.mkdir(parents=True, exist_ok=True)
    return path
def _lookup_umu_id(
    ctx: LaunchContext,
    umu_store: str,
    plugin_dir: Path,
) -> str | None:
    """Lookup UMU ID."""
    helper = plugin_dir / "bin" / "umu_lookup.py"
    if not helper.is_file():
        return None
    try:
        out = subprocess.check_output(
            ["python3", str(helper), ctx.game_id, umu_store],

```

```

        stderr=subprocess.DEVNULL,
        timeout=10,
    )
    text = out.decode().strip()
    return text or None
except (subprocess.SubprocessError, OSError):
    return None

def _locate_umu_wrapper(proton_path: Path) -> Path:
    """Locate UMU wrapper."""
    bundled = proton_path.parent / "umu-run"
    if bundled.is_file():
        return bundled
    system = shutil.which("umu-run")
    if system:
        return Path(system)
    raise DependencyMissingError(
        "umu-run not found (neither bundled with proton "
        "nor in PATH)",
        context={"proton_path": str(proton_path)},
    )

def proton_prepare(
    ctx: LaunchContext,
    state: RuntimeState,
    *,
    python_bin: Path,
    proton_path: Path,
    proton_tool_id: str,
    on_process_start: Callable[[object], None] | None = None,
) -> ProtonLaunchPlan:
    """Proton prepare."""
    import os
    umu_store = STORE_TO_UMU.get(ctx.store, "none")
    prefix_path = _resolve_prefix(ctx)
    umu_id = _lookup_umu_id(ctx, umu_store, ctx.plugin_dir)
    umu_wrapper = _locate_umu_wrapper(proton_path)
    state.python_bin = python_bin
    state.proton_path = proton_path
    state.proton_tool_id = proton_tool_id
    state.prefix_path = prefix_path
    state.umu_store_code = umu_store
    state.umu_id = umu_id
    state.umu_wrapper = umu_wrapper
    env = dict(os.environ)
    env["GAMEID"] = umu_id or "umu-0"
    env["STORE"] = umu_store
    env["STEAM_COMPAT_DATA_PATH"] = str(prefix_path)
    env["STEAM_COMPAT_CLIENT_INSTALL_PATH"] = str(
        Path("~/steam/root").expanduser,
    )
    env["PROTON_VERB"] = "waitforexitandrun"
    env.update(ctx.env_overrides)
    logger.info(
        "[launcher.proton.core] plan ready: store=%s umu_store=%s "
        "umu_id=%s prefix=%s proton=%s",
        ctx.store, umu_store, umu_id, prefix_path, proton_tool_id,
    )
    return ProtonLaunchPlan(
        context=ctx,
        state=state,
    )

```

```
python_bin=python_bin,  
umu_wrapper=umu_wrapper,  
prefix_path=prefix_path,  
env=env,  
on_process_start=on_process_start,  
)
```

OP-42b

umu_runtime.py

Fichier : py_modules/unifideck/launcher/proton/infrastructure/umu_runtime.py | Lignes : 75 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
import shutil
from collections.abc import Callable
from pathlib import Path
logger = logging.getLogger(__name__)
UMU_CACHE_DIR = Path("~/local/share/umu").expanduser()
_RECOVERABLE_CODES = {2, 74}
def cleanup_umu_runtime_cache() -> None:
    """Cleanup UMU runtime cache."""
    targets = [
        UMU_CACHE_DIR / "steamrt3",
        UMU_CACHE_DIR / "compatibilitytool.vdf",
        UMU_CACHE_DIR / ".ref",
    ]
    for target in targets:
        if target.is_dir():
            shutil.rmtree(target, ignore_errors=True)
        elif target.exists():
            try:
                target.unlink()
            except OSError:
                pass
    logger.info("[launcher.umu] cache cleaned: %s", UMU_CACHE_DIR)
def ensure_umu_runtime_ready() -> None:
    """Ensure UMU runtime ready."""
    UMU_CACHE_DIR.mkdir(parents=True, exist_ok=True)
    os.environ["UMU_LOG"] = "1"
    os.environ["UMU_NO_PROTON"] = "0"
    config_dir = Path("~/config/umu").expanduser()
    config_dir.mkdir(parents=True, exist_ok=True)

async def run_umu_with_retry(
    argv: list[str],
    *,
    env: dict[str, str] | None = None,
    cwd: Path | None = None,
    max_attempts: int = 2,
    on_start: Callable[[object], None] | None = None,
) -> int:
    """Run UMU with retry."""
    last_rc = 1
    for attempt in range(1, max_attempts + 1):
        logger.info(
            "[launcher.umu] run attempt %d/%d: %s",
            attempt, max_attempts, argv[:3],
        )
        proc = await asyncio.create_subprocess_exec(
            *argv,
            env=env,
            cwd=str(cwd) if cwd else None,
            stdout=None,

```



```
        stderr=None,
        start_new_session=True,
    )
    if on_start is not None:
        try:
            on_start(proc)
        except Exception:
            logger.exception("[launcher.umu] on_start callback failed")
    rc = await proc.wait()
    last_rc = rc
    logger.info("[launcher.umu] attempt %d exit code: %d", attempt, rc)
    if rc == 0:
        return 0
    if rc in _RECOVERABLE_CODES and attempt < max_attempts:
        logger.warning(
            "[launcher.umu] recoverable rc=%d, wiping cache and retrying",
            rc,
        )
        cleanup_umu_runtime_cache()
        continue
    return rc
return last_rc
```

OP-42c

selector.py

Fichier : py_modules/unifideck/launcher/proton/infrastructure/selector.py | Lignes : 154 | Depend de : (rien)

```

from __future__ import annotations
import logging
import re
import subprocess
from pathlib import Path
from ...types.errors import DependencyMissingError, ProtonUnavailableError
logger = logging.getLogger(__name__)
PYTHON_CANDIDATES: list[str] = [
    "/usr/bin/python3.13",
    "/usr/bin/python3.12",
    "/usr/bin/python3.11",
    "/usr/bin/python3.10",
    "/usr/bin/python3",
]
ACCEPTED_VERSIONS = {"3.10", "3.11", "3.12", "3.13", "3.14"}
def find_python_3_10_plus() -> Path:
    """Find python 3 10 plus."""
    for candidate in PYTHON_CANDIDATES:
        path = Path(candidate)
        if not path.is_file():
            continue
        try:
            out = subprocess.check_output(
                [
                    candidate,
                    "-c",
                    'import sys; print(f"{sys.version_info.major}.{sys.version_info.minor}")',
                ],
                stderr=subprocess.DEVNULL,
                timeout=5,
            )
        except (subprocess.SubprocessError, OSError):
            continue
        ver = out.decode().strip()
        if ver in ACCEPTED_VERSIONS:
            logger.info("[launcher.proton] python selected: %s (%s)", candidate, ver)
            return path
    raise DependencyMissingError(
        "No Python 3.10+ interpreter found on system",
        context={"tried": PYTHON_CANDIDATES},
    )

STEAM_COMPAT_ROOTS: list[str] = [
    "~/.steam/root/compatibilitytools.d",
    "~/.local/share/Steam/compatibilitytools.d",
]
STEAM_LIBRARY_ROOTS: list[str] = [
    "~/.steam/root/steamapps/common",
    "~/.local/share/Steam/steamapps/common",
]
UNIFIDECK_COMPAT_DIR = "~/.local/share/unifideck/compat-tools"
def resolve_proton_path(tool_id: str) -> Path | None:
    """Resolve PROTON path."""
    if not tool_id:

```

```

        return None
    unifideck_path = Path(UNIFIDECK_COMPAT_DIR).expanduser() / tool_id / "proton"
    if unifideck_path.is_file():
        return unifideck_path
    for root in STEAM_COMPAT_ROOTS:
        candidate = Path(root).expanduser() / tool_id / "proton"
        if candidate.is_file():
            return candidate
    for lib in STEAM_LIBRARY_ROOTS:
        candidate = Path(lib).expanduser() / tool_id / "proton"
        if candidate.is_file():
            return candidate
    return None

def get_unifideck_proton_tool() -> str | None:
    """Get unifideck PROTON tool."""
    config_path = Path("~/local/share/unifideck/config.json").expanduser()
    if not config_path.is_file():
        return None
    try:
        import json
        with config_path.open() as f:
            cfg = json.load(f)
            tool = cfg.get("compat", {}).get("proton_tool", "")
            return tool or None
    except (OSError, ValueError):
        return None
_COMPAT_TOOL_RE = re.compile(
    r'(?P<app_id>\d+)\s*\{[^\}]*?"name"\s*(?P<name>[^\}]+)"',
    re.S,
)

def get_steam_compat_tool_override(app_id: str) -> str | None:
    """Get steam compat tool override."""
    if not app_id:
        return None
    userdata = Path("~/steam/root/userdata").expanduser()
    if not userdata.is_dir():
        return None
    for user_dir in userdata.iterdir():
        cfg = user_dir / "config" / "localconfig.vdf"
        if not cfg.is_file():
            continue
        try:
            content = cfg.read_text(encoding="utf-8", errors="replace")
        except OSError:
            continue
        for m in _COMPAT_TOOL_RE.finditer(content):
            if m.group("app_id") == app_id:
                return m.group("name")
    return None

def find_any_ge_proton() -> Path | None:
    """Find any ge PROTON."""
    candidates: list[Path] = []
    for root in STEAM_COMPAT_ROOTS:
        expanded = Path(root).expanduser()
        if not expanded.is_dir():
            continue
        for entry in expanded.iterdir():
            if entry.name.startswith("GE-Proton"):
                proton_script = entry / "proton"
                if proton_script.is_file():
                    candidates.append(proton_script)

```

```
    if not candidates:
        return None
    candidates.sort()
    return candidates[-1]

def select_proton_version(
    steam_app_id: str | None = None,
) -> tuple[Path, str]:

    """Select PROTON version."""
    tried: list[str] = []
    if steam_app_id:
        steam_tool = get_steam_compat_tool_override(steam_app_id)
        if steam_tool:
            tried.append(f"steam:{steam_tool}")
            path = resolve_proton_path(steam_tool)
            if path:
                logger.info(
                    "[launcher.proton] selected via Steam override: %s",
                    steam_tool,
                )
                return path, steam_tool
    unifideck_tool = get_unifideck_proton_tool()
    if unifideck_tool:
        tried.append(f"unifideck:{unifideck_tool}")
        path = resolve_proton_path(unifideck_tool)
        if path:
            logger.info(
                "[launcher.proton] selected via Unifideck default: %s",
                unifideck_tool,
            )
            return path, unifideck_tool
    fallback = find_any_ge_proton()
    if fallback:
        tool_id = fallback.parent.name
        tried.append(f"fallback:{tool_id}")
        logger.info("[launcher.proton] selected via GE-Proton fallback: %s", tool_id)
        return fallback, tool_id
    raise ProtonUnavailableError(
        "No usable Proton compat tool found",
        context={"tried": tried},
    )
```

OP-42d

prefix_layout.py

Fichier : py_modules/unifideck/launcher/proton/infrastructure/prefix_layout.py | Lignes : 37 | Depend de : (rien)

```
from __future__ import annotations
import logging
from pathlib import Path
logger = logging.getLogger(__name__)
PathLike = str | Path
def normalize_prefix_root(prefix_path: PathLike) -> Path:
    """Normalize prefix root."""
    p = Path(prefix_path).resolve() if isinstance(prefix_path, str) \
        else prefix_path.resolve()
    while p.name == "pfx":
        p = p.parent
    return p
def resolve_registry_prefix(prefix_root: PathLike) -> Path:
    """Resolve registry prefix."""
    root = Path(prefix_root) if isinstance(prefix_root, str) \
        else prefix_root
    direct = root / "user.reg"
    pfx = root / "pfx"
    pfx_reg = pfx / "user.reg"
    if direct.exists():
        return root
    if pfx_reg.exists():
        return pfx
    if pfx.is_dir():
        return pfx
    return root
def resolve_drive_c(prefix_root: PathLike) -> Path | None:
    """Resolve drive c."""
    root = Path(prefix_root) if isinstance(prefix_root, str) \
        else prefix_root
    modern = root / "pfx" / "drive_c"
    if modern.is_dir():
        return modern
    legacy = root / "drive_c"
    if legacy.is_dir():
        return legacy
    return None
```

OP-43

__init__.py

Fichier : py_modules/unifideck/launcher/proton/handlers/__init__.py | Lignes : 0 | Depend de : (rien)

OP-43a

ubisoft.py

Fichier : py_modules/unifideck/launcher/proton/handlers/ubisoft.py | Lignes : 181 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
from pathlib import Path
from ...types.errors import GameFailedError, UmuRuntimeError
from ..infrastructure.core import ProtonLaunchPlan
from ..infrastructure.umu_runtime import run_umu_with_retry
logger = logging.getLogger(__name__)
async def _apply_epic_wrapper_fix(plan: ProtonLaunchPlan) -> None:
    """Apply EPIC wrapper fix."""
    from ..fixes.epic_prefix_fix import apply_epic_launcher_fix
    bundled_wrapper = (
        plan.context.plugin_dir / "bin" / "EpicGamesLauncher.exe"
    )
    if not bundled_wrapper.is_file():
        logger.warning(
            "[launcher.proton.ubisoft] EpicGamesLauncher.exe "
            "wrapper missing at %s",
            bundled_wrapper,
        )
        return
    try:
        await apply_epic_launcher_fix(
            prefix_path=plan.prefix_path,
            bundled_wrapper=bundled_wrapper,
        )
        logger.info(
            "[launcher.proton.ubisoft] Epic launcher wrapper applied",
        )
    except Exception:
        logger.exception(
            "[launcher.proton.ubisoft] Epic launcher wrapper fix failed",
        )
async def _inject_registry_keys(plan: ProtonLaunchPlan) -> bool:
    """Inject registry keys."""
    from ..fixes.epic_registry import setup_registry
    legendary_config = Path("~/config/legendary").expanduser()
    try:
        result = await setup_registry(
            game_id=plan.context.game_id,
            prefix_path=plan.prefix_path,
            legendary_config=legendary_config,
        )
        return result.success
    except Exception:
        logger.exception(
            "[launcher.proton.ubisoft] registry injection crashed",
        )
        return False

def _find_upc_exe(plan: ProtonLaunchPlan) -> Path | None:
    """Find UPC exe."""
    active_prefix = os.environ.get("ACTIVE_WINEPREFIX")
    candidates: list[Path] = []

```

```

if active_prefix:
    candidates.append(
        Path(active_prefix)
        / "drive_c"
        / "Program Files (x86)"
        / "Ubisoft"
        / "Ubisoft Game Launcher"
        / "upc.exe",
    )
candidates.append(
    plan.prefix_path
    / "drive_c"
    / "Program Files (x86)"
    / "Ubisoft"
    / "Ubisoft Game Launcher"
    / "upc.exe",
)
for c in candidates:
    if c.is_file():
        return c
return None
async def ubisoft_launch(plan: ProtonLaunchPlan) -> int:
    """Ubisoft launch."""
    logger.info(
        "[launcher.proton.ubisoft] launching %s",
        plan.context.game_key,
    )
    await _apply_epic_wrapper_fix(plan)
    if not await _inject_registry_keys(plan):
        logger.warning(
            "[launcher.proton.ubisoft] registry injection "
            "failed or skipped",
        )
    _apply_language_setup(plan)
    upc_exe = _find_upc_exe(plan)
    uplay_id = os.environ.get("UPLAY_ID")
    if upc_exe and uplay_id:
        logger.info(
            "[launcher.proton.ubisoft] direct launch: "
            "uplay://launch/%s/0",
            uplay_id,
        )
        argv = [
            str(plan.python_bin),
            str(plan.umu_wrapper),
            str(upc_exe),
            f"uplay://launch/{uplay_id}/0",
        ]
        env = plan.env
    else:
        logger.warning(
            "[launcher.proton.ubisoft] upc.exe or UPLAY_ID "
            "missing, falling back to Legendary path",
        )
        argv, env = _buildLegendaryFallbackArgv(plan)
    rc = await run_umu_with_retry(
        argv, env=env, on_start=plan.on_process_start,
    )
    plan.state.game_exit_code = rc
    if rc == 0:
        return 0

```



```

    _raise_for_umu_rc(rc, plan)
    return rc

def _apply_language_setup(plan: ProtonLaunchPlan) -> None:
    """Apply language setup."""
    try:
        from ...config.config_manager import ConfigManager
        from ..language_setup import apply_ubuntu_language
        _cfg = ConfigManager(
            str(plan.context.plugin_dir / "defaults" / "config.json"),
        )
        apply_ubuntu_language(
            str(plan.prefix_path),
            space_id=plan.context.game_id,
            config=_cfg,
        )
    except Exception as err:
        logger.warning(
            "[launcher.proton.ubuntu] language setup failed: %s",
            err,
        )

def _buildLegendaryFallbackArgv(
    plan: ProtonLaunchPlan,
) -> tuple[list[str], dict[str, str]]:
    """Build LEGENDARY fallback argv."""
    env = dict(plan.env)
    env["LEGENDARY_WRAPPER_EXE"] = (
        "C:\\windows\\command\\EpicGamesLauncher.exe"
    )
    legendary_bin = os.environ.get("LEGENDARY_BIN", "legendary")
    argv = plan.state.wrappers + [
        legendary_bin,
        "launch",
        plan.context.game_id,
        "--no-wine",
        "--skip-version-check",
        "--wrapper",
        f"{plan.python_bin} {plan.umu_wrapper}",
        "--language",
        os.environ.get("EPIC_LANG", "en"),
    ]
    if plan.state.game_args:
        argv.append("--")
        argv.extend(plan.state.game_args)
    try:
        from ..fixes.auth_args_stripper import strip_epic_auth_args
        argv, _stripped = strip_epic_auth_args(argv)
    except Exception as err:
        logger.warning(
            "[launcher.proton.ubuntu] auth args strip failed: %s",
            err,
        )
    return argv, env

def _raise_for_umu_rc(rc: int, plan: ProtonLaunchPlan) -> None:
    """Raise for UMU rc."""
    if rc in {2, 74}:
        raise UmuRuntimeError(
            f"umu-run failed with unrecoverable code {rc}",
            context={"subprocess_rc": rc, "store": "ubuntu"},
        ) from None

```

```
raise GameFailedError(
    f"Ubisoft game exited with code {rc}",
    subprocess_rc=rc,
    context={
        "store": "ubisoft",
        "game_id": plan.context.game_id,
    },
) from None
```

OP-43b

epic.py

Fichier : py_modules/unifideck/launcher/proton/handlers/epic.py | Lignes : 167 | Depend de : (rien)

```
from __future__ import annotations
import json
import logging
import os
import re
from pathlib import Path
from ...types.errors import GameFailedError, UmuRuntimeError
from ..infrastructure.core import ProtonLaunchPlan
from ..infrastructure.umu_runtime import run_umu_with_retry
logger = logging.getLogger(__name__)

def _lazy_cleanup_ubisoft_artifacts(plan: ProtonLaunchPlan) -> None:

    """Lazy cleanup UBISOFT artifacts."""
    drive_cs = [plan.prefix_path / "drive_c"]
    active = os.environ.get("ACTIVE_WINEPREFIX")
    if active:
        drive_cs.append(Path(active) / "drive_c")
    targets_per_drive = [
        "windows/command/EpicGamesLauncher.exe",
        (
            "Program Files (x86)/Epic Games/Launcher/"
            "Portal/Binaries/Win32/EpicGamesLauncher.exe"
        ),
    ]
    for drive_c in drive_cs:
        if not drive_c.is_dir():
            continue
        for rel in targets_per_drive:
            target = drive_c / rel
            if target.is_file():
                try:
                    target.unlink()
                    logger.info(
                        "[launcher.proton.epic] removed stub: %s",
                        target,
                    )
                except OSError:
                    pass
    prefix_candidates = [plan.prefix_path]
    if active:
        prefix_candidates.append(Path(active))
    for prefix in prefix_candidates:
        for reg_name in ("user.reg", "system.reg"):
            reg = prefix / reg_name
            if not reg.is_file():
                continue
            try:
                content = reg.read_text(
                    encoding="utf-8", errors="replace",
                )
            except OSError:
                continue
            if "com.epicgames.launcher" not in content:
                continue
```

```

        new_content = _strip_registry_section(
            content, "com.epicgames.launcher",
        )
        if new_content != content:
            try:
                reg.write_text(
                    new_content, encoding="utf-8",
                )
                logger.info(
                    "[launcher.proton.epic] cleaned %s "
                    "from %s",
                    "com.epicgames.launcher", reg.name,
                )
            except OSError:
                pass

def _strip_registry_section(content: str, section_key: str) -> str:

    """Strip registry section."""
    lines = content.splitlines(keepends=True)
    out: list[str] = []
    skipping = False
    header_pat = re.compile(
        r"^\[.*" + re.escape(section_key) + r".*\]",
    )
    next_section_pat = re.compile(r"^\[")
    for line in lines:
        if (
            skipping
            and next_section_pat.match(line)
            and not header_pat.match(line)
        ):
            skipping = False
            out.append(line)
            continue
        if header_pat.match(line):
            skipping = True
            continue
        if skipping:
            continue
        out.append(line)
    return "".join(out)

def _resolve_exe_override(plan: ProtonLaunchPlan) -> Path | None:
    """Resolve exe override."""
    from ..fixes.game_fixes import get_exe_override
    rel = get_exe_override(plan.context.game_id)
    if not rel:
        return None
    installed = Path(
        "~/.config/legendary/installed.json",
    ).expanduser()
    if not installed.is_file():
        return None
    try:
        with installed.open() as f:
            data = json.load(f)
    except (OSError, ValueError):
        return None
    install_path = (
        data.get(plan.context.game_id, {}).get("install_path")
    )

```

```

    if not install_path:
        return None
    full = Path(install_path) / rel
    return full if full.is_file() else None

async def epic_launch(plan: ProtonLaunchPlan) -> int:
    """Epic launch."""
    logger.info(
        "[launcher.proton.epic] launching %s", plan.context.game_key,
    )
    _lazy_cleanup_ubisoft_artifacts(plan)
    env = dict(plan.env)
    env["STORE"] = "none"
    env.pop("LEGENDARY_WRAPPER_EXE", None)
    exe_override = _resolve_exe_override(plan)
    legendary_bin = os.environ.get("LEGENDARY_BIN", "legendary")
    argv: list[str] = list(plan.state.wrappers)
    argv.extend([
        legendary_bin,
        "launch",
        plan.context.game_id,
        "--no-wine",
        "--skip-version-check",
        "--wrapper",
        f"{plan.python_bin} {plan.umu_wrapper}",
        "--language",
        os.environ.get("EPIC_LANG", "en"),
    ])
    if exe_override:
        argv.extend(["--override-exe", str(exe_override)])
        logger.info(
            "[launcher.proton.epic] using EXE override: %s",
            exe_override,
        )
    if plan.state.game_args:
        argv.append("--")
        argv.extend(plan.state.game_args)
    rc = await run_umu_with_retry(
        argv, env=env, on_start=plan.on_process_start,
    )
    plan.state.game_exit_code = rc
    if rc == 0:
        return 0
    if rc in {2, 74}:
        raise UmuRuntimeError(
            f"umu-run failed with unrecoverable code {rc}",
            context={"subprocess_rc": rc, "store": "epic"},
        )
    raise GameFailedError(
        f"Epic game exited with code {rc}",
        subprocess_rc=rc,
        context={
            "store": "epic",
            "game_id": plan.context.game_id,
        },
    )

```

OP-43c

generic.py

Fichier : py_modules/unifideck/launcher/proton/handlers/generic.py | Lignes : 127 | Depend de : (rien)

```

from __future__ import annotations
import logging
import shutil
from pathlib import Path
from ...types.errors import GameFailedError, UmuRuntimeError
from ..infrastructure.core import ProtonLaunchPlan
from ..infrastructure.umu_runtime import run_umu_with_retry
logger = logging.getLogger(__name__)
def _locate_store_cli(plan: ProtonLaunchPlan, tool_name: str) -> Path | None:
    """Locate store cli."""
    plugin_bin = plan.context.plugin_dir / "bin" / tool_name
    if plugin_bin.is_file():
        return plugin_bin
    system = shutil.which(tool_name)
    return Path(system) if system else None
async def _gog_launch(plan: ProtonLaunchPlan) -> int:
    """Gog launch."""
    try:
        from ...config.config_manager import ConfigManager
        from ..language_setup import apply_gog_language
        _cfg = ConfigManager(
            str(plan.context.plugin_dir / "defaults" / "config.json"),
        )
        work_dir = plan.context.work_dir or plan.context.exe_path.parent
        apply_gog_language(
            plan.context.game_id, str(work_dir), config=_cfg,
        )
    except Exception as err:
        logger.warning(
            "[launcher.proton.generic] GOG language setup failed: %s",
            err,
        )
    try:
        from ..fixes.galaxy_stub import install_galaxy_stub
        install_galaxy_stub(
            str(plan.prefix_path),
            plugin_dir=plan.context.plugin_dir,
        )
    except Exception as err:
        logger.warning(
            "[launcher.proton.generic] Galaxy stub install failed: %s",
            err,
        )
    gogdl = _locate_store_cli(plan, "gogdl")
    if gogdl:
        logger.info("[launcher.proton.generic] GOG via gogdl: %s", gogdl)
        argv: list[str] = list(plan.state.wrappers)
        argv.extend([
            str(gogdl),
            "launch",
            plan.context.game_id,
            "--wrapper",
            f"{plan.python_bin} {plan.umu_wrapper}",
        ])
    if plan.state.game_args:

```

```

        argv.append("--")
        argv.extend(plan.state.game_args)
        return await run_umu_with_retry(argv, env=plan.env, on_start=plan.on_process_start)
    return await _raw_exe_launch(plan)

async def _amazon_launch(plan: ProtonLaunchPlan) -> int:
    """Amazon launch."""
    try:
        from ...config.config_manager import ConfigManager
        from ..language_setup import apply_amazon_language
        _cfg = ConfigManager(
            str(plan.context.plugin_dir / "defaults" / "config.json"),
        )
        apply_amazon_language(str(plan.prefix_path), config=_cfg)
    except Exception as err:
        logger.warning(
            "[launcher.proton.generic] Amazon language setup failed: %s",
            err,
        )
    nile = _locate_store_cli(plan, "nile")
    if nile:
        logger.info("[launcher.proton.generic] Amazon via nile: %s", nile)
        argv: list[str] = list(plan.state.wrappers)
        argv.extend([
            str(nile),
            "launch",
            plan.context.game_id,
            "--wrapper",
            f"{plan.python_bin} {plan.umu_wrapper}",
        ])
        if plan.state.game_args:
            argv.append("--")
            argv.extend(plan.state.game_args)
        return await run_umu_with_retry(argv, env=plan.env, on_start=plan.on_process_start)
    return await _raw_exe_launch(plan)

async def _raw_exe_launch(plan: ProtonLaunchPlan) -> int:
    """Raw exe launch."""
    logger.info(
        "[launcher.proton.generic] raw exe launch: %s", plan.context.exe_path,
    )
    cwd: Path | None = None
    if plan.context.exe_path.parent.is_dir():
        cwd = plan.context.exe_path.parent
    argv: list[str] = list(plan.state.wrappers)
    argv.extend([
        str(plan.python_bin),
        str(plan.umu_wrapper),
        str(plan.context.exe_path),
    ])
    argv.extend(plan.state.game_args)
    return await run_umu_with_retry(argv, env=plan.env, cwd=cwd,
        on_start=plan.on_process_start)

async def generic_launch(plan: ProtonLaunchPlan) -> int:
    """Generic launch."""
    store = plan.context.store
    if store == "gog":
        rc = await _gog_launch(plan)
    elif store == "amazon":
        rc = await _amazon_launch(plan)
    else:

```

```
    rc = await _raw_exe_launch(plan)
plan.state.game_exit_code = rc
if rc == 0:
    return 0
if rc in {2, 74}:
    raise UmuRuntimeError(
        f"umu-run failed with unrecoverable code {rc}",
        context={"subprocess_rc": rc, "store": store},
    )
raise GameFailedError(
    f"{store} game exited with code {rc}",
    subprocess_rc=rc,
    context={"store": store, "game_id": plan.context.game_id},
)
```


OP-43d

gog_linux_dosbox.py

Fichier : py_modules/unifideck/launcher/proton/handlers/gog_linux_dosbox.py | Lignes : 133 | Depend de : (rien)

```

from __future__ import annotations
import os
import platform
import re
import shlex
import sys
from pathlib import Path
from typing import NoReturn
DOSBOX_CALL_RE = re.compile(r'run_dosbox\s+((?:\"[^\"]+\"|\\s*)+)')
def find_steam_runtime() -> Path | None:
    """Find steam runtime."""
    candidates = (
        Path.home() / ".steam" / "steam" / "ubuntu12_32" / "steam-runtime",
        Path.home() / ".local" / "share" / "Steam" / "ubuntu12_32" / "steam-runtime",
    )
    for candidate in candidates:
        if candidate.exists():
            return candidate
    return None
def build_runtime_library_paths(
    runtime_root: Path, arch_dir: str,
) -> list[str]:
    """Build runtime library paths."""
    paths: list[str] = []
    for rel in (f"usr/lib/{arch_dir}", f"lib/{arch_dir}"):
        candidate = runtime_root / rel
        if candidate.exists():
            paths.append(str(candidate))
    return paths
def parse_dosbox_conf_args(start_script: Path) -> list[str]:
    """Parse dosbox conf args."""
    content = start_script.read_text(encoding="utf-8", errors="ignore")
    match = DOSBOX_CALL_RE.search(content)
    if not match:
        raise ValueError(
            f"Could not find run_dosbox call in {start_script}",
        )
    return shlex.split(match.group(1))
def launch_via_steam_runtime(
    runtime_root: Path | None,
    start_script: Path,
    args: list[str],
) -> NoReturn:
    """Launch via steam runtime."""
    if runtime_root:
        run_sh = runtime_root / "run.sh"
        if run_sh.exists():
            os.execv(str(run_sh), [str(run_sh), str(start_script), *args])
    os.execv(str(start_script), [str(start_script), *args])
def _parse_argv() -> tuple[Path, list[str]]:
    """Parse argv."""
    if len(sys.argv) < 2:
        raise SystemExit(
            "Usage: python -m "
            "unifideck.launcher.proton.gog_linux_dosbox "

```

```

        "/path/to/start.sh [args...]",
    )
    start_script = Path(sys.argv[1]).resolve()
    extra_args = [
        arg for arg in sys.argv[2:]
        if not re.match(r"^(epic|gog|amazon|ubisoft):", arg)
    ]
    return start_script, extra_args

def _select_architecture(
    dosbox_dir: Path,
) -> tuple[Path, Path, str] | None:
    """Select architecture."""
    arch = platform.machine().lower()
    if arch in {"x86_64", "amd64"}:
        return (
            dosbox_dir / "dosbox_x86_64",
            dosbox_dir / "libs" / "x86_64",
            "x86_64-linux-gnu",
        )
    if arch in {"i686", "i386"}:
        return (
            dosbox_dir / "dosbox_i686",
            dosbox_dir / "libs" / "i686",
            "i386-linux-gnu",
        )
    return None

def _build_env(
    bundled_lib_dir: Path,
    runtime_root: Path | None,
    runtime_arch_dir: str,
) -> dict[str, str]:
    """Build env."""
    runtime_libs = (
        build_runtime_library_paths(runtime_root, runtime_arch_dir)
        if runtime_root else []
    )
    env = os.environ.copy()
    ld_parts = [str(bundled_lib_dir), *runtime_libs]
    if env.get("LD_LIBRARY_PATH"):
        ld_parts.append(env["LD_LIBRARY_PATH"])
    env["LD_LIBRARY_PATH"] = ":".join(
        dict.fromkeys(part for part in ld_parts if part),
    )
    return env

def main() -> None:
    """Main."""
    start_script, extra_args = _parse_argv()
    runtime_root = find_steam_runtime()
    if extra_args:
        launch_via_steam_runtime(
            runtime_root, start_script, extra_args,
        )
    root_dir = start_script.parent
    dosbox_dir = root_dir / "dosbox"
    if not dosbox_dir.is_dir():
        launch_via_steam_runtime(
            runtime_root, start_script, extra_args,
        )
    arch_info = _select_architecture(dosbox_dir)

```

```
if arch_info is None:
    launch_via_steam_runtime(
        runtime_root, start_script, extra_args,
    )
binary, bundled_lib_dir, runtime_arch_dir = arch_info
if not binary.exists() or not bundled_lib_dir.is_dir():
    launch_via_steam_runtime(
        runtime_root, start_script, extra_args,
    )
conf_args = parse_dosbox_conf_args(start_script)
env = _build_env(bundled_lib_dir, runtime_root, runtime_arch_dir)
command = [str(binary)]
for conf in conf_args:
    command.extend(["-conf", conf])
command.extend(["-no-console", "-c", "exit"])
os.chdir(root_dir)
os.execvpe(str(binary), command, env)
if __name__ == "__main__":
    main()
```

OP-44

__init__.py

Fichier : py_modules/unifideck/launcher/proton/fixes/__init__.py | Lignes : 0 | Depend de : (rien)

OP-44a

game_fixes.py

Fichier : py_modules/unifideck/launcher/proton/fixes/game_fixes.py | Lignes : 171 | Depend de : (rien)

```
from __future__ import annotations
import json
import logging
import time
from dataclasses import dataclass, field
from typing import Any, cast
logger = logging.getLogger(__name__)
@dataclass(frozen=True)
class GameFix:
    """Game fix."""
    winetricks: list[str] = field(default_factory=list)
    exe_override: str | None = None
    notes: str = ""
    source: str = ""

GLOBAL_DEFAULTS: list[str] = [
    "vcrun2005",
    "vcrun2008",
    "vcrun2010",
    "vcrun2012",
    "vcrun2013",
    "vcrun2022",
    "d3dcompiler_47",
    "d3dcompiler_43",
    "mfc140",
]
MANUAL_FIXES: dict[str, GameFix] = {
    "Dodo": GameFix(
        winetricks=[],
        notes="Works with Proton + E0S only",
        source="manual",
    ),
    "ea8df71f923649a193ab1c1fded7e1b3": GameFix(
        winetricks=[
            "vcrun2005", "vcrun2008", "vcrun2010", "vcrun2012",
            "vcrun2013", "vcrun2022",
        ],
        exe_override=(
            "Ghostrunner/Binaries/Win64/"
            "Ghostrunner-Win64-Shipping.exe"
        ),
        notes=(
            "UE4 stub bypassed – launches shipping binary "
            "directly. The default Ghostrunner.exe is a 540KB "
            "launcher stub that probes VC++ runtime registry "
            "keys via MsiQueryProductState and shows a "
            "'Microsoft Visual C++ Runtime' error even when "
            "DLLs are present. Proton rewrites system.reg at "
            "launch time, making registry injection impossible."
        ),
        source="manual",
    ),
    "fa5aa7e6c28c4c94aeac239eee700d5f": GameFix(
        winetricks=[],
        notes="E0S overlay only, no redistributables needed",
    ),
}
```

```

        source="manual",
    ),
}
_UMU_DATABASE_URL_FORMATS = [
    "https://raw.githubusercontent.com/Open-Wine-Components/"
    "umu-database/main/umu-egs-{game_id}.json",
    "https://raw.githubusercontent.com/Open-Wine-Components/"
    "umu-database/main/umu-epic-{game_id}.json",
]
_UMU_CACHE: dict[str, tuple[float, dict | None]] = {}
_CACHE_TTL_SECONDS = 3600
def get_exe_override(game_id: str) -> str | None:
    """Get exe override."""
    fix = MANUAL_FIXES.get(game_id)
    if fix is None:
        return None
    return fix.exe_override

async def fetch_umu_protonfixes(game_id: str) -> dict | None:
    """Fetch UMU protonfixes."""
    now = time.monotonic()
    cached = _UMU_CACHE.get(game_id)
    if (
        cached is not None
        and now - cached[0] < _CACHE_TTL_SECONDS
    ):
        return cached[1]
    _UMU_CACHE[game_id] = (now, None)
    try:
        import aiohttp
    except ImportError:
        logger.info(
            "[game_fixes] aiohttp not available, skipping "
            "umu-database lookup for %s", game_id,
        )
        return None
    timeout = aiohttp.ClientTimeout(total=10)
    async with aiohttp.ClientSession(timeout=timeout) as session:
        for url_format in _UMU_DATABASE_URL_FORMATS:
            url = url_format.format(game_id=game_id)
            data = await _try_umu_url(session, url)
            if data is not None:
                logger.info(
                    "[game_fixes] found umu-db "
                    "entry for %s", game_id,
                )
                _UMU_CACHE[game_id] = (now, data)
                return cast("dict[Any, Any] | None", data)
        logger.info(
            "[game_fixes] no umu-db entry for %s (expected "
            "for most games)", game_id,
        )
        return None
    async def _try_umu_url(
        session: Any, url: str,
    ) -> dict | None:
        """Try UMU URL."""
        import aiohttp
        try:
            async with session.get(url) as response:

```

```

        if response.status != 200:
            return None
        data = await response.json(content_type=None)
        return cast("dict[Any, Any] | None", data)
    except (aiohttp.ClientError, json.JSONDecodeError) as e:
        logger.debug(
            "[game_fixes] %s lookup failed: %s", url, e,
        )
        return None
async def get_required_winetricks(game_id: str) -> list[str]:
    """Get required winetricks."""
    manual = MANUAL_FIXES.get(game_id)
    if manual is not None:
        logger.info(
            "[game_fixes] manual override for %s: %s",
            game_id, manual.winetricks,
        )
        return list(manual.winetricks)
    umu_data = await fetch_umu_protonfixes(game_id)
    if umu_data and isinstance(
        umu_data.get("winetricks"), list,
    ):
        packages = umu_data["winetricks"]
        logger.info(
            "[game_fixes] umu-db for %s: %s",
            game_id, packages,
        )
        return list(packages)
    logger.info(
        "[game_fixes] global defaults for %s", game_id,
    )
    return list(GLOBAL_DEFAULTS)

async def get_game_fix(game_id: str) -> GameFix:
    """Get game fix."""
    manual = MANUAL_FIXES.get(game_id)
    if manual is not None:
        return manual
    umu_data = await fetch_umu_protonfixes(game_id)
    if umu_data:
        return GameFix(
            winetricks=list(
                umu_data.get("winetricks") or [],
            ),
            exe_override=umu_data.get("exe_override"),
            notes=str(umu_data.get("notes") or ""),
            source="umu-protonfixes",
        )
    return GameFix(
        winetricks=list(GLOBAL_DEFAULTS),
        notes=(
            "Using global defaults "
            "(vcrun*, d3dcompiler, mfc140)"
        ),
        source="global_default",
    )
def clear_cache() -> None:
    """Clear cache."""
    _UMU_CACHE.clear()

```

OP-44b

galaxy_stub.py

Fichier : py_modules/unifideck/launcher/proton/fixes/galaxy_stub.py | Lignes : 79 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
import shutil
import tempfile
from pathlib import Path
logger = logging.getLogger(__name__)
_STUB_RELATIVE_PATH = "bin/stubs/GalaxyCommunication.exe"
_TARGET_SUBPATH = os.path.join(
    "ProgramData", "GOG.com", "Galaxy", "redists",
    "GalaxyCommunication.exe",
)
def _resolve_drive_c(prefix_path: str) -> str | None:
    """Resolve drive c."""
    from ..infrastructure.prefix_layout import resolve_drive_c
    result = resolve_drive_c(prefix_path)
    return str(result) if result is not None else None
def _atomic_copy_file(src: Path | str, dst: str) -> None:
    """Atomic copy file."""
    target_dir = os.path.dirname(dst)
    os.makedirs(target_dir, exist_ok=True)
    fd, tmp_path = tempfile.mkstemp(
        prefix=".GalaxyCommunication.", suffix=".tmp",
        dir=target_dir,
    )
    try:
        with os.fdopen(fd, "wb") as tmp_fh, \
            open(str(src), "rb") as src_fh:
            shutil.copyfileobj(src_fh, tmp_fh)
            tmp_fh.flush()
            os.fsync(tmp_fh.fileno())
        os.replace(tmp_path, dst)
    except Exception:
        try:
            os.unlink(tmp_path)
        except OSError:
            pass
        raise

def install_galaxy_stub(
    prefix_path: str,
    plugin_dir: Path | None = None,
) -> bool:
    """Install galaxy stub."""
    if plugin_dir is None:
        from ....core.paths import resolve_plugin_dir
        plugin_dir = resolve_plugin_dir()
    stub_src = plugin_dir / _STUB_RELATIVE_PATH
    if not stub_src.is_file():
        logger.warning(
            "[galaxy_stub] stub binary missing at %s – GOG games "
            "that check for Galaxy may fail to launch", stub_src,
        )
    return False

```



```
drive_c = _resolve_drive_c(prefix_path)
if drive_c is None:
    logger.warning(
        "[galaxy_stub] drive_c not found under %s – prefix "
        "not yet initialised", prefix_path,
    )
    return False
target_file = os.path.join(drive_c, _TARGET_SUBPATH)
if os.path.exists(target_file):
    logger.debug(
        "[galaxy_stub] stub already installed at %s", target_file,
    )
    return True
try:
    _atomic_copy_file(stub_src, target_file)
except OSError as err:
    logger.warning(
        "[galaxy_stub] copy %s → %s failed: %s",
        stub_src, target_file, err,
    )
    return False
logger.info(
    "[galaxy_stub] installed GalaxyCommunication.exe stub at %s",
    target_file,
)
return True
```

OP-44c

epic_prefix_fix.py

Fichier : py_modules/unifideck/launcher/proton/fixes/epic_prefix_fix.py | Lignes : 216 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
import shutil
import subprocess
from pathlib import Path

logger = logging.getLogger(__name__)
_PROTON_FALLBACK_WINE_PATHS: list[str] = [
    "~/steam/steam/steamapps/common/Proton - Experimental/files/bin/wine",
    "~/steam/steam/steamapps/common/Proton 10.0/files/bin/wine",
    "~/steam/steam/steamapps/common/Proton 9.0 (Beta)/files/bin/wine",
]

def normalize_prefix_root(prefix_path: Path) -> Path:
    """Normalize prefix root."""
    p = prefix_path.resolve()
    while p.name == "pfx":
        p = p.parent
    return p

def _copy_wrapper_to_drive_c(
    drive_c: Path,
    bundled_wrapper: Path,
    label: str,
) -> bool:

    """Copy wrapper to drive c."""
    if not bundled_wrapper.is_file():
        logger.warning(
            "[epic_prefix_fix] bundled wrapper missing at %s",
            bundled_wrapper,
        )
        return False
    copied = False
    epic_dir = (
        drive_c / "Program Files (x86)" / "Epic Games" / "Launcher"
        / "Portal" / "Binaries" / "Win32"
    )
    try:
        epic_dir.mkdir(parents=True, exist_ok=True)
        epic_target = epic_dir / "EpicGamesLauncher.exe"
        if epic_target.exists():
            try:
                epic_target.unlink()
            except OSError:
                pass
        shutil.copy2(bundled_wrapper, epic_target)
        logger.info(
            "[epic_prefix_fix] copied wrapper to Epic dir (%s)",
            label,
        )
        copied = True
    except OSError as e:
        logger.warning(
            "[epic_prefix_fix] failed to copy to Epic dir "

```

```

        "(%s): %s",
        label, e,
    )
win_command_dir = drive_c / "windows" / "command"
try:
    win_command_dir.mkdir(parents=True, exist_ok=True)
    win_target = win_command_dir / "EpicGamesLauncher.exe"
    if win_target.exists():
        try:
            win_target.unlink()
        except OSError:
            pass
    shutil.copy2(bundled_wrapper, win_target)
    logger.info(
        "[epic_prefix_fix] copied wrapper to "
        "windows/command (%s)",
        label,
    )
    copied = True
except OSError as e:
    logger.warning(
        "[epic_prefix_fix] failed to copy to "
        "windows/command (%s): %s",
        label, e,
    )
return copied
def _find_wine_binary() -> Path | None:
    """Find WINE binary."""
    proton_env = os.environ.get("PROTONPATH")
    if proton_env:
        candidate = Path(proton_env) / "files" / "bin" / "wine"
        if candidate.is_file():
            return candidate
    for fallback in _PROTON_FALLBACK_WINE_PATHS:
        candidate = Path(fallback).expanduser()
        if candidate.is_file():
            return candidate
    system_wine = shutil.which("wine")
    if system_wine:
        return Path(system_wine)
    return None
def _select_registry_prefix(
    prefix_root: Path,
) -> Path:
    """Select registry prefix."""
    pfx_candidate = prefix_root / "pfx"
    if not pfx_candidate.exists():
        return prefix_root
    try:
        if pfx_candidate.is_symlink() and pfx_candidate.resolve() == prefix_root.resolve():
            return prefix_root
    except OSError:
        pass
    drive_c = pfx_candidate / "drive_c"
    if drive_c.is_dir():
        return pfx_candidate
    return prefix_root
async def _run_registry_inject(
    wine_bin: Path,

```

```

    wineprefix: Path,
) -> bool:
    """Run registry inject."""
    env = dict(os.environ)
    env["WINEPREFIX"] = str(wineprefix)
    try:
        proc = await asyncio.create_subprocess_exec(
            str(wine_bin),
            "reg", "add",
            "HKEY_CLASSES_ROOT\\\\com.epicgames.launcher",
            "/f",
            env=env,
            stdout=asyncio.subprocess.DEVNULL,
            stderr=asyncio.subprocess.DEVNULL,
        )
    try:
        rc = await asyncio.wait_for(proc.wait(), timeout=30)
    except TimeoutError:
        logger.warning(
            "[epic_prefix_fix] wine reg add timed out, "
            "killing",
        )
        try:
            proc.kill()
        except ProcessLookupError:
            pass
        return False
    if rc == 0:
        logger.info(
            "[epic_prefix_fix] registry key injected",
        )
        return True
    logger.warning(
        "[epic_prefix_fix] wine reg add exited rc=%d", rc,
    )
    return False
except (OSError, subprocess.SubprocessError) as e:
    logger.warning(
        "[epic_prefix_fix] registry injection failed: %s", e,
    )
    return False
async def _kill_wineserver(wine_bin: Path, wineprefix: Path) -> None:
    """Kill wineserver."""
    wineserver = wine_bin.parent / "wineserver"
    if not wineserver.is_file():
        return
    env = dict(os.environ)
    env["WINEPREFIX"] = str(wineprefix)
    try:
        proc = await asyncio.create_subprocess_exec(
            str(wineserver), "--kill",
            env=env,
            stdout=asyncio.subprocess.DEVNULL,
            stderr=asyncio.subprocess.DEVNULL,
        )
    await asyncio.wait_for(proc.wait(), timeout=10)
    logger.info(
        "[epic_prefix_fix] killed stale wineserver",
    )
except (TimeoutError, OSError, subprocess.SubprocessError):
    pass

```

```
async def apply_epic_launcher_fix(
    prefix_path: Path,
    bundled_wrapper: Path,
) -> bool:

    """Apply EPIC launcher fix."""
    prefix_root = normalize_prefix_root(prefix_path)
    root_drive_c = prefix_root / "drive_c"
    pfx_drive_c = prefix_root / "pfx" / "drive_c"
    found_any = False
    if root_drive_c.is_dir():
        _copy_wrapper_to_drive_c(root_drive_c, bundled_wrapper, "root")
        found_any = True
    if pfx_drive_c.is_dir():
        _copy_wrapper_to_drive_c(
            pfx_drive_c, bundled_wrapper, "pfx",
        )
        found_any = True
    if not found_any:
        logger.info(
            "[epic_prefix_fix] prefix not initialized yet, "
            "skipping",
        )
        return False
    wine_bin = _find_wine_binary()
    if wine_bin is None:
        logger.warning(
            "[epic_prefix_fix] no wine binary found, "
            "skipping registry injection",
        )
        return True
    registry_prefix = _select_registry_prefix(prefix_root)
    registry_ok = await _run_registry_inject(
        wine_bin, registry_prefix,
    )
    await _kill_wineserver(wine_bin, registry_prefix)
    if registry_ok:
        logger.info("[epic_prefix_fix] quick fix complete")
    else:
        logger.warning(
            "[epic_prefix_fix] quick fix completed with "
            "registry issues (non-fatal)",
        )
    return True
```

OP-44d

epic_registry.py

Fichier : py_modules/unifideck/launcher/proton/fixes/epic_registry.py | Lignes : 271 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import json
import logging
import os
import re
import subprocess
from dataclasses import dataclass
from pathlib import Path
from typing import Any, cast
logger = logging.getLogger(__name__)
_UPLAY_ID_RE = re.compile(r"-UplayId=\s*(\d+)")
@dataclass(frozen=True)
class RegistryInjectionResult:
    """Registry injection result."""
    success: bool
    keys_written: int
    reason: str = ""
def _normalize_prefix_root(prefix_path: Path) -> Path:
    """Normalize prefix root."""
    p = prefix_path.resolve()
    while p.name == "pfx":
        p = p.parent
    return p
def _select_active_wineprefix(prefix_root: Path) -> Path:
    """Select active wineprefix."""
    pfx_path = prefix_root / "pfx"
    try:
        if (
            pfx_path.is_symlink()
            and pfx_path.resolve() == prefix_root.resolve()
        ):
            return prefix_root
    except OSError:
        pass
    if (pfx_path / "system.reg").is_file():
        return pfx_path
    if (prefix_root / "system.reg").is_file():
        return prefix_root
    return pfx_path
def _linux_to_wine_path(linux_path: str) -> str:
    """Linux to WINE path."""
    wine_path = "Z:" + linux_path.replace("/", "\\")
    if not wine_path.endswith("\\"):
        wine_path += "\\"
    return wine_path
def _load_installed_json(
    legendary_config: Path,
    game_id: str,
) -> dict | None:
    """Load installed JSON."""
    installed_json = legendary_config / "installed.json"
    if not installed_json.is_file():
```

```

        logger.error(
            "[epic_registry] installed.json not found at %s",
            installed_json,
        )
        return None
    try:
        with installed_json.open() as f:
            data = json.load(f)
    except (OSError, json.JSONDecodeError) as e:
        logger.error(
            "[epic_registry] failed to read "
            "installed.json: %s", e,
        )
        return None
    app = data.get(game_id)
    if not app:
        logger.error(
            "[epic_registry] game %s not in "
            "installed.json", game_id,
        )
        return None
    return cast("dict[Any, Any] | None", app)
def _find_wine_binary() -> Path | None:
    """Find WINE binary."""
    proton_path = os.environ.get("PROTONPATH")
    if not proton_path:
        return None
    wine_bin = (
        Path(proton_path) / "files" / "bin" / "wine"
    )
    return wine_bin if wine_bin.is_file() else None

def _build_reg_commands(
    wine_bin: Path,
    game_id: str,
    wine_install_path: str,
    uplay_id: str | None,
) -> list[list[str]]:
    """Build reg commands."""
    commands: list[list[str]] = [
        [
            str(wine_bin), "reg", "add",
            "HKEY_LOCAL_MACHINE\\Software\\Epic Games\\EpicGamesLauncher",
            "/v", "AppDataPath", "/t", "REG_SZ",
            "/d", "C:\\ProgramData\\Epic\\EpicGamesLauncher\\Data\\",
            "/f",
        ],
        [
            str(wine_bin), "reg", "add",
            (
                "HKEY_LOCAL_MACHINE\\Software\\WOW6432Node\\Epic Games"
                "\\EpicGamesLauncher\\Manifests\\" + game_id
            ),
            "/v", "InstallLocation", "/t", "REG_SZ",
            "/d", wine_install_path, "/f",
        ],
        [
            str(wine_bin), "reg", "add",
            (
                "HKEY_CURRENT_USER\\Software\\Epic Games"

```

```

        "\\EpicGamesLauncher\\Manifests\\" + game_id
    ),
    "/v", "InstallLocation", "/t", "REG_SZ",
    "/d", wine_install_path, "/f",
],
]
if uplay_id:
    commands.extend([
        [
            str(wine_bin), "reg", "add",
            (
                "HKEY_LOCAL_MACHINE\\Software\\WOW6432Node\\Ubisoft"
                "\\Launcher\\Installs\\" + uplay_id
            ),
            "/v", "InstallDir", "/t", "REG_SZ",
            "/d", wine_install_path, "/f",
        ],
        [
            str(wine_bin), "reg", "add",
            (
                "HKEY_LOCAL_MACHINE\\Software\\WOW6432Node\\Ubisoft"
                "\\Launcher\\Installs\\" + uplay_id
            ),
            "/v", "Language", "/t", "REG_SZ",
            "/d", "en-US", "/f",
        ],
    ])
return commands

async def _run_reg_commands(
    commands: list[list[str]],
    env: dict,
) -> int:

    """Run reg commands."""
    ok_count = 0
    for cmd in commands:
        try:
            proc = await asyncio.create_subprocess_exec(
                *cmd,
                env=env,
                stdout=asyncio.subprocess.DEVNULL,
                stderr=asyncio.subprocess.PIPE,
            )
            try:
                _, stderr = await asyncio.wait_for(
                    proc.communicate(), timeout=30,
                )
            except TimeoutError:
                logger.error(
                    "[epic_registry] reg add timed out: %s",
                    cmd[3],
                )
            try:
                proc.kill()
            except ProcessLookupError:
                pass
            continue
        if proc.returncode == 0:
            ok_count += 1
        else:

```



```

        logger.error(
            "[epic_registry] reg add failed: %s: %s",
            cmd[3],
            stderr.decode(errors="replace").strip(),
        )
    except (OSError, subprocess.SubprocessError) as e:
        logger.error(
            "[epic_registry] reg add spawn error: %s", e,
        )
        continue
    return ok_count
async def _kill_wineserver(
    wine_bin: Path, wineprefix: Path,
) -> None:
    """Kill wineserver."""
    wineserver = wine_bin.parent / "wineserver"
    if not wineserver.is_file():
        return
    env = dict(os.environ)
    env["WINEPREFIX"] = str(wineprefix)
    try:
        proc = await asyncio.create_subprocess_exec(
            str(wineserver), "--kill",
            env=env,
            stdout=asyncio.subprocess.DEVNULL,
            stderr=asyncio.subprocess.DEVNULL,
        )
        await asyncio.wait_for(proc.wait(), timeout=10)
        logger.info(
            "[epic_registry] killed stale wineserver "
            "after setup",
        )
    except (
        TimeoutError, OSError,
        subprocess.SubprocessError,
    ):
        pass
def _resolve_install_paths(
    app: dict[str, Any],
) -> tuple[str, str | None] | None:
    """Resolve install paths."""
    install_path = app.get("install_path")
    if not install_path:
        return None
    wine_install_path = _linux_to_wine_path(install_path)
    launch_params = app.get("launch_parameters", "") or ""
    uplay_match = _UPLAY_ID_RE.search(launch_params)
    uplay_id = uplay_match.group(1) if uplay_match else None
    return wine_install_path, uplay_id

def _error_result(reason: str) -> RegistryInjectionResult:
    """Error result."""
    return RegistryInjectionResult(
        success=False, keys_written=0, reason=reason,
    )
async def setup_registry(
    game_id: str,
    prefix_path: Path,
    legendary_config: Path,
) -> RegistryInjectionResult:

```

```
"""Setup registry."""
prefix_root = _normalize_prefix_root(prefix_path)
app = _load_installed_json(legendary_config, game_id)
if app is None:
    return _error_result("installed_json_missing_or_unreadable")
paths = _resolve_install_paths(app)
if paths is None:
    logger.error(
        "[epic_registry] no install_path for %s", game_id,
    )
    return _error_result("no_install_path")
wine_install_path, uplay_id = paths
wine_bin = _find_wine_binary()
if wine_bin is None:
    logger.error(
        "[epic_registry] PROTONPATH not set or wine "
        "binary missing",
    )
    return _error_result("wine_binary_not_found")
active_prefix = _select_active_wineprefix(prefix_root)
env = dict(os.environ)
env["WINEPREFIX"] = str(active_prefix)
commands = _build_reg_commands(
    wine_bin=wine_bin,
    game_id=game_id,
    wine_install_path=wine_install_path,
    uplay_id=uplay_id,
)
ok_count = await _run_reg_commands(commands, env)
await _kill_wineserver(wine_bin, active_prefix)
total = len(commands)
all_ok = ok_count == total
logger.info(
    "[epic_registry] setup for %s (uplay=%s): %d/%d keys",
    game_id, uplay_id, ok_count, total,
)
return RegistryInjectionResult(
    success=all_ok,
    keys_written=ok_count,
    reason="" if all_ok else "partial_reg_add_failures",
)
```

OP-44e

auth_args_stripper.py

Fichier : py_modules/unifideck/launcher/proton/fixes/auth_args_stripper.py | Lignes : 36 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ..types.context import LaunchContext
logger = logging.getLogger(__name__)
_STRIP_PREFIXES: tuple[str, ...] = (
    "-AUTH_TYPE=",
    "-AUTH_PASSWORD=",
)
def strip_epic_auth_args(args: list[str]) -> tuple[list[str], list[str]]:
    """Strip EPIC auth args."""
    filtered: list[str] = []
    stripped: list[str] = []
    for arg in args:
        if any(arg.startswith(p) for p in _STRIP_PREFIXES):
            stripped.append(arg)
            continue
        filtered.append(arg)
    if stripped:
        logger.info(
            "[auth_args_stripper] stripped %d Epic auth args: %s",
            len(stripped),
            [s.split("=", 1)[0] + "=<redacted>" for s in stripped],
        )
    return filtered, stripped
def should_strip_for_launch_context(ctx: LaunchContext) -> bool:
    """Check whether strip for launch context."""
    try:
        store = getattr(ctx, "store", "")
        exe_path = str(getattr(ctx, "exe_path", ""))
        return (
            store == "ubisoft" and "UplayLaunch" in exe_path
        )
    except Exception:
        return False
```

OP-45

`__init__.py`

Fichier : `py_modules/unifideck/launcher/proton/language_setup/__init__.py` | Lignes : 5 | Depend de : (rien)

```
from __future__ import annotations
from .amazon import apply_amazon_language
from .gog import apply_gog_language
from .resolver import get_unifideck_language
from .ubisoft import apply_ubisoft_language
```

OP-45a

resolver.py

Fichier : py_modules/unifideck/launcher/proton/language_setup/resolver.py | Lignes : 63 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
LOCALE_MAP: dict[str, tuple[str, str, str, str]] = {
    "en-US": ("00000409", "ENU", "en-US", "United States"),
    "de-DE": ("00000407", "DEU", "de-DE", "Germany"),
    "fr-FR": ("0000040c", "FRA", "fr-FR", "France"),
    "es-ES": ("00000c0a", "ESN", "es-ES", "Spain"),
    "it-IT": ("00000410", "ITA", "it-IT", "Italy"),
    "pt-BR": ("00000416", "PTB", "pt-BR", "Brazil"),
    "ru-RU": ("00000419", "RUS", "ru-RU", "Russia"),
    "pl-PL": ("00000415", "PLK", "pl-PL", "Poland"),
    "zh-CN": ("00000804", "CHS", "zh-CN", "China"),
    "ja-JP": ("00000411", "JPN", "ja-JP", "Japan"),
    "ko-KR": ("00000412", "KOR", "ko-KR", "Korea"),
    "nl-NL": ("00000413", "NLD", "nl-NL", "Netherlands"),
    "tr-TR": ("0000041f", "TRK", "tr-TR", "Turkey"),
}
UBISOFT_LANG_MAP: dict[str, str] = {
    "en-US": "en", "de-DE": "de", "fr-FR": "fr", "es-ES": "es",
    "it-IT": "it", "pt-BR": "pt", "ru-RU": "ru", "pl-PL": "pl",
    "zh-CN": "zh", "ja-JP": "ja", "ko-KR": "ko", "nl-NL": "nl",
    "tr-TR": "tr",
}
GOG_DISPLAY_NAMES: dict[str, str] = {
    "en-US": "English",
    "fr-FR": "French",
    "de-DE": "German",
    "es-ES": "Spanish",
    "it-IT": "Italian",
    "pt-BR": "Portuguese (Brazil)",
    "ru-RU": "Russian",
    "pl-PL": "Polish",
    "zh-CN": "Chinese (Simplified)",
    "zh-Hans": "Chinese (Simplified)",
    "zh-TW": "Chinese (Traditional)",
    "zh-Hant": "Chinese (Traditional)",
    "ja-JP": "Japanese",
    "ko-KR": "Korean",
    "nl-NL": "Dutch",
    "tr-TR": "Turkish",
}
_DEFAULT_LANGUAGE = "en-US"
def get_unifideck_language(config: ConfigManager | None = None) -> str:
    """Get unifideck language."""
    if config is None:
        logger.debug(
            "[language_setup] no ConfigManager provided, using %s",
            _DEFAULT_LANGUAGE,
        )
    return _DEFAULT_LANGUAGE
try:

```

```
from ....utils.locale import get_unifideck_locale
return get_unifideck_locale(config)
except Exception as err:
    logger.warning(
        "[language_setup] locale resolver failed: %s, "
        "falling back to %s", err, _DEFAULT_LANGUAGE,
    )
return _DEFAULT_LANGUAGE
```

OP-45b

matchers.py

Fichier : py_modules/unifideck/launcher/proton/language_setup/matchers.py | Lignes : 28 | Depend de : (rien)

```
from __future__ import annotations
from .resolver import LOCALE_MAP
def smart_match_locale(
    target: str,
) -> tuple[str, str, str, str] | None:
    """Smart match locale."""
    if not target:
        return None
    if target in LOCALE_MAP:
        return LOCALE_MAP[target]
    base = target.split("-", maxsplit=1)[0].lower()
    for code, data in LOCALE_MAP.items():
        if code.split("-")[0].lower() == base:
            return data
    return None
def smart_match_gog_language(
    target: str, available: list[str],
) -> str | None:
    """Smart match GOG language."""
    if not target or not available:
        return None
    if target in available:
        return target
    base = target.split("-", maxsplit=1)[0].lower()
    for lang in available:
        if lang.split("-")[0].lower() == base:
            return lang
    return None
```

OP-45c

registry_io.py

Fichier : py_modules/unifideck/launcher/proton/language_setup/registry_io.py | Lignes : 97 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
import re
import tempfile
from .matchers import smart_match_locale
from .resolver import _DEFAULT_LANGUAGE, LOCALE_MAP
logger = logging.getLogger(__name__)
def _resolve_prefix(prefix_path: str) -> str:
    """Resolve prefix."""
    from ..infrastructure.prefix_layout import resolve_registry_prefix
    return str(resolve_registry_prefix(prefix_path))
def _atomic_write_text(path: str, content: str) -> None:
    """Atomic write text."""
    target_dir = os.path.dirname(path) or "."
    fd, tmp_path = tempfile.mkstemp(
        prefix=".reg.", suffix=".tmp", dir=target_dir,
    )
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as fh:
            fh.write(content)
            fh.flush()
            os.fsync(fh.fileno())
            os.replace(tmp_path, path)
    except Exception:
        try:
            os.unlink(tmp_path)
        except OSError:
            pass
        raise

def _update_user_reg(
    prefix_path: str,
    lcid: str, slanguage: str, locale_name: str, scountry: str,
) -> bool:
    """Update user reg."""
    user_reg = os.path.join(prefix_path, "user.reg")
    if not os.path.exists(user_reg):
        logger.warning(
            "[language_setup] user.reg missing at %s – prefix not "
            "initialised yet", user_reg,
        )
        return False
    with open(user_reg, encoding="utf-8", errors="replace") as fh:
        content = fh.read()
    section_header = "[Control Panel\\International]"
    new_values = {
        "Locale": lcid,
        "LocaleName": locale_name,
        "sLanguage": slanguage,
        "sCountry": scountry,
    }
    if section_header in content:
        section_start = content.index(section_header)

```



```

        body_start = section_start + len(section_header)
        next_section = re.search(r"\n\[", content[body_start:])
        section_end = (
            body_start + next_section.start()
            if next_section else len(content)
        )
        section_body = content[body_start:section_end]
        for key, value in new_values.items():
            pattern = rf'^{re.escape(key)}="[^"]*"'
            replacement = f'"{key}"="{value}"'
            new_body, count = re.subn(
                pattern, replacement, section_body, flags=re.MULTILINE,
            )
            if count > 0:
                section_body = new_body
            else:
                section_body = (
                    section_body.rstrip("\n") + f'\n"{key}"="{value}"\n'
                )
        content = (
            content[:body_start] + section_body + content[section_end:]
        )
    else:
        section = f"\n{section_header}\n"
        for key, value in new_values.items():
            section += f'"{key}"="{value}"\n'
        content += section
    _atomic_write_text(user_reg, content)
    logger.info(
        "[language_setup] wrote locale=%s to %s",
        locale_name, user_reg,
    )
    return True

def _apply_windows_locale(prefix_path: str, language: str) -> bool:
    """Apply windows locale."""
    resolved_prefix = _resolve_prefix(prefix_path)
    locale = smart_match_locale(language)
    if locale is None:
        logger.info(
            "[language_setup] no locale mapping for %s, using %s",
            language, _DEFAULT_LANGUAGE,
        )
        locale = LOCALE_MAP[_DEFAULT_LANGUAGE]
    return _update_user_reg(resolved_prefix, *locale)

```

OP-45d

amazon.py

Fichier : py_modules/unifideck/launcher/proton/language_setup/amazon.py | Lignes : 18 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from .registry_io import _apply_windows_locale
from .resolver import get_unifideck_language
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
def apply_amazon_language(
    prefix_path: str, config: ConfigManager | None = None,
) -> bool:
    """Apply AMAZON language."""
    language = get_unifideck_language(config)
    logger.info(
        "[language_setup.amazon] applying %s to prefix=%s",
        language, prefix_path,
    )
    return _apply_windows_locale(prefix_path, language)
```

OP-45e

gog.py

Fichier : py_modules/unifideck/launcher/proton/language_setup/gog.py | Lignes : 76 | Depend de : (rien)

```
from __future__ import annotations
import glob
import json
import logging
import os
from typing import TYPE_CHECKING
from .matchers import smart_match_gog_language
from .registry_io import _atomic_write_text
from .resolver import GOG_DISPLAY_NAMES, get_unifideck_language
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
def _find_goggame_info(game_id: str, install_dir: str) -> str | None:
    """Find goggame info."""
    search_dir = install_dir
    for _ in range(4):
        candidate = os.path.join(search_dir, f"goggame-{game_id}.info")
        if os.path.exists(candidate):
            return candidate
        candidates = glob.glob(
            os.path.join(search_dir, "goggame-*.info"),
        )
        matching = [
            c for c in candidates
            if os.path.basename(c).startswith(f"goggame-{game_id}")
        ]
        if matching:
            return matching[0]
        parent = os.path.dirname(search_dir)
        if parent == search_dir:
            break
        search_dir = parent
    return None

def apply_gog_language(
    game_id: str, install_dir: str, config: ConfigManager | None = None,
) -> bool:
    """Apply GOG language."""
    language = get_unifideck_language(config)
    logger.info(
        "[language_setup.gog] applying %s to game=%s dir=%s",
        language, game_id, install_dir,
    )
    info_path = _find_goggame_info(game_id, install_dir)
    if info_path is None:
        logger.info(
            "[language_setup.gog] no goggame-%s.info found (searched "
            "4 levels up from %s), skipping", game_id, install_dir,
        )
        return False
    try:
        with open(info_path, encoding="utf-8") as fh:
            info = json.load(fh)
    except (OSError, json.JSONDecodeError) as err:
```

```
        logger.warning(
            "[language_setup.gog] read %s failed: %s", info_path, err,
        )
        return False
    installed = info.get("languages", [])
    matched = smart_match_gog_language(language, installed)
    if matched is None:
        matched = language
    display_name = GOG_DISPLAY_NAMES.get(matched, matched)
    info["language"] = display_name
    info["languages"] = [matched]
    try:
        _atomic_write_text(info_path, json.dumps(info, indent=2))
    except OSError as err:
        logger.warning(
            "[language_setup.gog] write %s failed: %s", info_path, err,
        )
        return False
    logger.info(
        "[language_setup.gog] set %s → %s (%s)",
        info_path, matched, display_name,
    )
    return True
```

OP-45f

ubisoft.py

Fichier : py_modules/unifideck/launcher/proton/language_setup/ubisoft.py | Lignes : 105 | Depend de : (rien)

```

from __future__ import annotations
import json
import logging
import os
import re
from typing import TYPE_CHECKING
from .registry_io import (
    _apply_windows_locale,
    _atomic_write_text,
    _resolve_prefix,
)
from .resolver import UBISOFT_LANG_MAP, get_unifideck_language
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
def _load_ubisoft_install_id(space_id: str) -> str | None:
    """Load UBISOFT install ID."""
    id_map_path = os.path.expanduser(
        "~/local/share/unifideck/ubisoft_id_map.json",
    )
    if not os.path.isfile(id_map_path):
        return None
    try:
        with open(id_map_path, encoding="utf-8") as fh:
            id_map = json.load(fh)
    except (OSError, json.JSONDecodeError):
        return None
    entry = id_map.get(space_id, {})
    install_id = entry.get("install_id")
    return install_id if isinstance(install_id, str) else None
def _patch_upc_language_section(
    content: str, install_id: str, ubi_lang: str,
) -> str:
    """Patch UPC language section."""
    section = (
        f"[Software\\\\\\\\WOW6432Node\\\\\\\\Ubisoft\\\\\\\\Launcher\\\\\\\\"
        f"Installs\\\\\\\\{install_id}]"
    )
    if section not in content:
        return content + f'\n{section}\n"Language"="{ubi_lang}"\n'
    sec_start = content.index(section)
    body_start = sec_start + len(section)
    next_sec = re.search(r"\n\[", content[body_start:])
    sec_end = (
        body_start + next_sec.start()
        if next_sec else len(content)
    )
    sec_body = content[body_start:sec_end]
    pattern = r'^"Language"="[^\"]*"'
    replacement = f'"Language"="{ubi_lang}"'
    new_body, count = re.subn(
        pattern, replacement, sec_body, flags=re.MULTILINE,
    )
    if count > 0:
        sec_body = new_body

```

```

    else:
        sec_body = (
            sec_body.rstrip("\n") + f'\n"Language"="{ubi_lang}"\n'
        )
        return content[:body_start] + sec_body + content[sec_end:]

def _apply_ubisoft_upc_language(
    prefix_path: str, space_id: str, language: str,
) -> bool:

    """Apply UBISOFT UPC language."""
    install_id = _load_ubisoft_install_id(space_id)
    if not install_id:
        logger.info(
            "[language_setup.ubisoft] no install_id for space_id=%s, "
            "skipping UPC language", space_id,
        )
        return False
    system_reg = os.path.join(prefix_path, "system.reg")
    if not os.path.isfile(system_reg):
        logger.info(
            "[language_setup.ubisoft] system.reg missing at %s, "
            "skipping UPC language", system_reg,
        )
        return False
    with open(system_reg, encoding="utf-8", errors="replace") as fh:
        content = fh.read()
        ubi_lang = UBISOFT_LANG_MAP.get(
            language, language.split("-", maxsplit=1)[0],
        )
        content = _patch_upc_language_section(content, install_id, ubi_lang)
        _atomic_write_text(system_reg, content)
        logger.info(
            "[language_setup.ubisoft] UPC Language=%s written "
            "(install_id=%s)", ubi_lang, install_id,
        )
    return True

def apply_ubisoft_language(
    prefix_path: str, space_id: str = "",
    config: ConfigManager | None = None,
) -> bool:
    """Apply UBISOFT language."""
    language = get_unifideck_language(config)
    logger.info(
        "[language_setup.ubisoft] applying %s to prefix=%s space_id=%s",
        language, prefix_path, space_id,
    )
    resolved_prefix = _resolve_prefix(prefix_path)
    windows_ok = _apply_windows_locale(prefix_path, language)
    if space_id:
        _apply_ubisoft_upc_language(resolved_prefix, space_id, language)
    return windows_ok

```

OP-35a

signals.py

Fichier : py_modules/unifideck/launcher/signals.py | Lignes : 66 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
import signal
import subprocess
from dataclasses import dataclass, field
logger = logging.getLogger(__name__)
CLEANUP_PATTERNS = (
    "steam-runtime-launch-client",
    "umu-run",
)
@dataclass
class SignalState:
    """Signal state."""
    terminated_by_signal: bool = False
    pending_pids: set[int] = field(default_factory=set)
class GameProcessRegistry:
    """Game process registry."""
    def __init__(self, state: SignalState) -> None:
        """Initialize the instance."""
        self._state = state
    def track(self, proc: subprocess.Popen) -> None:
        """Track."""
        if proc.pid:
            self._state.pending_pids.add(proc.pid)
    def untrack(self, proc: subprocess.Popen) -> None:
        """Untrack."""
        self._state.pending_pids.discard(proc.pid)
    def terminate_all(self) -> None:
        """Terminate all."""
        self._state.terminated_by_signal = True
        for pid in list(self._state.pending_pids):
            try:
                os.killpg(pid, signal.SIGTERM)
            except (ProcessLookupError, PermissionError):
                pass
            except OSError:
                try:
                    os.kill(pid, signal.SIGTERM)
                except (ProcessLookupError, PermissionError):
                    pass
        for pattern in CLEANUP_PATTERNS:
            try:
                subprocess.run(
                    ["pkill", "-TERM", "-f", pattern],
                    stdout=subprocess.DEVNULL,
                    stderr=subprocess.DEVNULL,
                    timeout=2,
                )
            except (FileNotFoundError, subprocess.TimeoutExpired):
                pass

def install_signal_handlers(
    registry: GameProcessRegistry,
) -> SignalState:

```

```
"""Install signal handlers."""
state = registry._state
def _handler(signum: int, _frame: object | None) -> None:
    """Handler."""
    logger.info(
        "[launcher.signals] received signal %d, terminating games",
        signum,
    )
    registry.terminate_all()
signal.signal(signal.SIGTERM, _handler)
signal.signal(signal.SIGINT, _handler)
return state
```


OP-35b

rpc.py

Fichier : py_modules/unifideck/launcher/rpc.py | Lignes : 62 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from ..core.types import Events
if TYPE_CHECKING:
    from ..event_bus import EventBus
logger = logging.getLogger(__name__)
async def emit_game_launched(
    bus: EventBus,
    *,
    store: str,
    game_id: str,
) -> None:
    """Emit game launched."""
    logger.info(
        "[launcher.rpc] emit GAME_LAUNCHED store=%s game=%s", store, game_id,
    )
    await bus.emit(
        Events.GAME_LAUNCHED,
        store=store,
        game_id=game_id,
    )
async def emit_game_stopped(
    bus: EventBus,
    *,
    store: str,
    game_id: str,
    exit_code: int,
    elapsed_seconds: float = 0.0,
    terminated_by_signal: bool = False,
) -> None:
    """Emit game stopped."""
    logger.info(
        "[launcher.rpc] emit GAME_STOPPED store=%s game=%s rc=%d elapsed=%.1fs signal=%s",
        store, game_id, exit_code, elapsed_seconds, terminated_by_signal,
    )
    await bus.emit(
        Events.GAME_STOPPED,
        store=store,
        game_id=game_id,
        exit_code=exit_code,
        elapsed_seconds=elapsed_seconds,
        terminated_by_signal=terminated_by_signal,
    )
async def emit_stage(
    bus: EventBus,
    *,
    i18n_key: str,
    game_title: str,
    priority: str = "low",
) -> None:
    """Emit stage."""
    logger.debug(
        "[launcher.rpc] stage: key=%s game=%s prio=%s",
        i18n_key, game_title, priority,
```

```
)  
  await bus.emit(  
    Events.LAUNCHER_STAGE,  
    i18n_key=i18n_key,  
    game_title=game_title,  
    priority=priority,  
  )
```

OP-35c

dispatcher.py

Fichier : py_modules/unifideck/launcher/dispatcher.py | Lignes : 210 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
import sys
from pathlib import Path
from ..core.types.results import Result
from .types.context import LaunchContext
from .types.errors import GameNotFoundError, LauncherError
from .types.exit_codes import ExitCode
logger = logging.getLogger(__name__)
def _parse_argv(argv: list[str]) -> tuple[str, str]:
    """Parse argv."""
    if len(argv) < 2:
        raise GameNotFoundError(
            "missing store:game_id argument",
            context={"argv": argv},
        )
    game_key = argv[1]
    if ":" not in game_key:
        raise GameNotFoundError(
            f"malformed game key {game_key!r}, "
            "expected 'store:game_id'",
            context={"game_key": game_key},
        )
    raw_options = " ".join(argv[2:])
    return game_key, raw_options
def _resolve_plugin_dir() -> Path:
    """Resolve plugin dir."""
    from ..core.paths import resolve_plugin_dir
    return resolve_plugin_dir(start=Path(__file__))
async def _build_context(
    argv: list[str],
    shortcut_svc,
) -> LaunchContext:
    """Build context."""
    game_key, raw_options = _parse_argv(argv)
    store, game_id = game_key.split(":", 1)
    entry = await shortcut_svc.get_entry_for_game_key(
        store, game_id,
    )
    if entry is None:
        raise GameNotFoundError(
            f"game {game_key!r} not found in games.map",
            context={"game_key": game_key},
        )
    exe_path = Path(entry.exe)
    work_dir = Path(entry.work_dir)
    auth_store, is_launch_action = _detect_auth_action()
    bypass = _resolve_bypass_flag(store, game_id)
    return LaunchContext(
        store=store,
        game_id=game_id,
        exe_path=exe_path,
        work_dir=work_dir,

```

```

        plugin_dir=_resolve_plugin_dir(),
        raw_options=raw_options,
        is_launch_action=is_launch_action,
        auth_store=auth_store,
        bypass_circuit_breaker=bypass,
    )

def _detect_auth_action() -> tuple[str | None, bool]:

    """Detect auth action."""
    auth_env = {
        "epic": os.environ.get("UNIFIDECK_EPIC_ACTION"),
        "gog": os.environ.get("UNIFIDECK_GOG_ACTION"),
        "amazon": os.environ.get("UNIFIDECK_AMAZON_ACTION"),
    }
    for candidate_store, action in auth_env.items():
        if action == "auth":
            return candidate_store, False
    return None, True

def _resolve_bypass_flag(store: str, game_id: str) -> bool:
    """Resolve bypass flag."""
    bypass_raw = os.environ.get(
        "UNIFIDECK_BYPASS_CIRCUIT_BREAKER", ""
    )
    bypass_env = bypass_raw.strip().lower() in (
        "1", "true", "yes",
    )
    try:
        from ..config.config_manager import ConfigManager
        from ..services.launch_history import (
            LaunchHistoryService,
        )
        cfg = ConfigManager(
            str(
                _resolve_plugin_dir()
                / "defaults" / "config.json",
            ),
        )
        lh = LaunchHistoryService(cfg)
        bypass_flag = lh.consume_bypass(f"{store}:{game_id}")
    except Exception:
        bypass_flag = False
    return bypass_env or bypass_flag

async def _run(argv: list[str]) -> int:
    """Run."""
    from .diagnostics.correlation import launch_id_scope, new_launch_id
    from .diagnostics.log_archive import (
        attach_launch_handler,
        detach_launch_handler,
        prune_old_launches,
    )
    lid = new_launch_id()
    with launch_id_scope(lid):
        try:
            from ..config.config_manager import ConfigManager
            from ..core.paths import resolve_plugin_dir
            _cfg = ConfigManager(str(
                resolve_plugin_dir() /
                "defaults" /
                "config.json"))
        except Exception:

```

```

        _cfg = None
        prune_old_launches(_cfg)
        _archive_handler = attach_launch_handler(lid, _cfg)
    try:
        return await _run_with_id(argv)
    finally:
        detach_launch_handler(_archive_handler)

async def _run_with_id(argv: list[str]) -> int:
    """Run with ID."""
    try:
        game_key, _ = _parse_argv(argv)
        logger.info(
            "[launcher.dispatcher] request received: %s", game_key,
        )
    except LauncherError as err:
        logger.error(
            "[launcher.dispatcher] argv parse failed: %s",
            err.to_log_dict,
        )
        return int(err.exit_code)
    try:
        launcher_service = _bootstrap_minimal_services()
    except Exception:
        logger.exception("[launcher.dispatcher] bootstrap failed")
        return int(ExitCode.DEPENDENCY_MISSING)
    try:
        ctx = await _build_context(argv, launcher_service._shortcut_svc)
    except LauncherError as err:
        logger.error(
            "[launcher.dispatcher] context build failed: %s",
            err.to_log_dict,
        )
        return int(err.exit_code)
    try:
        await launcher_service.start()
        result = await launcher_service.launch(ctx)
    except LauncherError as err:
        logger.error(
            "[launcher.dispatcher] launch raised: %s",
            err.to_log_dict,
        )
        return int(err.exit_code)
    finally:
        try:
            await launcher_service.stop()
        except Exception:
            logger.exception(
                "[launcher.dispatcher] launcher_service.stop failed",
            )
    return _map_result_to_exitcode(result)

def _map_result_to_exitcode(result: Result) -> int:
    """Map result to exitcode."""
    if result.success:
        return int(ExitCode.SUCCESS)
    code = result.error_code or ""
    if code == "not_implemented":
        return int(ExitCode.GENERIC_ERROR)
    if code == "circuit_open":
        return int(ExitCode.CIRCUIT_BREAKER_OPEN)

```

```
if code.startswith("exit_"):
    try:
        rc = int(code.split("_", 1)[1])
        return rc if 0 <= rc <= 255 else int(ExitCode.GAME_FAILED)
    except (ValueError, IndexError):
        return int(ExitCode.GAME_FAILED)
return int(ExitCode.GAME_FAILED)
def _bootstrap_minimal_services():
    """Bootstrap minimal services."""
    from .bootstrap import build_launcher_service
    return build_launcher_service()

def main(argv: list[str]) -> int:
    """Main."""
    from .diagnostics.correlation import install_launch_id_logging
    logging.basicConfig(
        format=(
            "%(asctime)s [%(launch_id)s] %(levelname)s "
            "%(name)s: %(message)s"
        ),
        level=logging.INFO,
        stream=sys.stderr,
    )
    install_launch_id_logging()
    try:
        return int(asyncio.run(_run(argv)))
    except KeyboardInterrupt:
        return int(ExitCode.CANCELLED_BY_USER)
    except asyncio.CancelledError:
        logger.info(
            "[launcher.dispatcher] launch cancelled by user",
        )
        return int(ExitCode.CANCELLED_BY_USER)
    except Exception:
        logger.exception("[launcher.dispatcher] uncaught exception")
        return int(ExitCode.GENERIC_ERROR)
if __name__ == "__main__":
    sys.exit(main(sys.argv))
```

OP-35d

packaging.py

Fichier : py_modules/unifideck/launcher/packaging.py | Lignes : 57 | Depend de : (rien)

```
from __future__ import annotations
import logging
import os
import stat
from collections.abc import Iterable
from pathlib import Path
logger = logging.getLogger(__name__)
LAUNCHER_EXECUTABLE_FILES: tuple[str, ...] = (
    "bin/unifideck-launcher",
)
_EXEC_MASK = stat.S_IXUSR | stat.S_IXGRP | stat.S_IXOTH
def ensure_executable_files(
    plugin_dir: Path,
    files: Iterable[str] = LAUNCHER_EXECUTABLE_FILES,
) -> int:
    """Ensure executable files."""
    fixed = 0
    for rel_path in files:
        target = plugin_dir / rel_path
        if not target.is_file():
            logger.info(
                "[packaging] skipping missing file: %s",
                rel_path,
            )
            continue
        try:
            current_mode = target.stat().st_mode
        except OSError as e:
            logger.warning(
                "[packaging] cannot stat %s: %s",
                rel_path, e,
            )
            continue
        if current_mode & _EXEC_MASK:
            continue
        new_mode = current_mode | _EXEC_MASK
        try:
            os.chmod(target, new_mode)
            logger.info(
                "[packaging] fixed executable bit on %s "
                "(mode %#o → %#o)",
                rel_path,
                current_mode & 0o777, new_mode & 0o777,
            )
            fixed += 1
        except OSError as e:
            logger.warning(
                "[packaging] chmod failed on %s: %s",
                rel_path, e,
            )
    if fixed > 0:
        logger.info(
            "[packaging] executable bit self-heal: "
            "%d file(s) fixed",
            fixed,
        )
```

)
return fixed

OP-35e

bootstrap.py

Fichier : py_modules/unifideck/launcher/bootstrap.py | Lignes : 66 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
from pathlib import Path
from typing import Any
logger = logging.getLogger(__name__)
def build_launcher_service(config: Any | None = None) -> Any:
    """Build launcher service."""
    from ..auth.edge_browser import EdgeBrowser
    from ..event_bus import EventBus
    from ..services.bootstrap import (
        ServicePaths,
        build_service_subset,
    )
    from ..services.launcher import LauncherService
    if config is None:
        config = _load_standalone_config()
        bus = EventBus()
        paths = ServicePaths.from_config(config)
        services = build_service_subset(
            bus, config, paths,
            attrs={"shortcut", "proton", "cloudsave", "launch_history"},
        )
        shortcut_svc = services.get("shortcut")
        proton_svc = services.get("proton")
        cloud_svc = services.get("cloudsave")
        assert shortcut_svc is not None, "bootstrap: shortcut service missing"
        assert proton_svc is not None, "bootstrap: proton service missing"
        assert cloud_svc is not None, "bootstrap: cloudsave service missing"
        edge_browser = EdgeBrowser(
            cdp_port=config.get_int("edge.cdp_port", 9222),
            locale_fn=lambda: config.get_str("ui.locale", "en-US"),
        )
        return LauncherService(
            bus=bus,
            shortcut_svc=shortcut_svc,
            proton_svc=proton_svc,
            cloud_svc=cloud_svc,
            edge_browser=edge_browser,
            config=config,
        )
def _load_standalone_config() -> Any:
    """Load standalone config."""
    from ..config.config_manager import ConfigManager
    plugin_dir = _resolve_plugin_dir()
    defaults_path = os.path.join(
        plugin_dir, "defaults", "config.json",
    )
    user_path = _user_config_path()
    return ConfigManager(
        defaults_path=defaults_path,
        user_path=user_path,
    )
def _resolve_plugin_dir() -> str:
    """Resolve plugin dir."""

```

```
from ..core.paths import resolve_plugin_dir
return str(resolve_plugin_dir(start=Path(__file__)))

def _user_config_path() -> str | None:

    """User config path."""
    override = os.environ.get("UNIFIDECK_USER_CONFIG")
    if override:
        return override
    xdg = os.environ.get("XDG_CONFIG_HOME") or str(
        Path.home() / ".config",
    )
    return str(Path(xdg) / "unifideck" / "config.json")
```

OP-46

__init__.py

Fichier : py_modules/unifideck/stores/__init__.py | Lignes : 5 | Depend de : (rien)

```
from .shared import StoreBase, StoreRegistry
__all__ = [
    "StoreBase",
    "StoreRegistry",
]
```

OP-47

`__init__.py`

Fichier : `py_modules/unifideck/stores/shared/__init__.py` | Lignes : 9 | Depend de : (rien)

```
from . import cli_install_helpers, dlc
from .store_base import StoreBase
from .store_registry import StoreRegistry
__all__ = [
    "StoreBase",
    "StoreRegistry",
    "cli_install_helpers",
    "dlc",
]
```

OP-47a

store_registry.py

Fichier : py_modules/unifideck/stores/shared/store_registry.py | Lignes : 307 | Depend de : (rien)

```

import asyncio
import logging
from dataclasses import asdict
from pathlib import Path
from typing import TYPE_CHECKING, Any
from ...core.types import Events, Result, StoreError
if TYPE_CHECKING:
    from ...core.cache_manager import CacheManager
    from ...event_bus import EventBus
    from .store_base import StoreBase
logger = logging.getLogger(__name__)
class StoreRegistry:
    """Store registry."""
    def __init__(self, bus: "EventBus") -> None:
        """Initialize the instance."""
        self._stores: dict[str, StoreBase] = {}
        self._bus = bus
    def register(
        self, store_id: str, store: "StoreBase",
    ) -> None:
        """Register."""
        self._stores[store_id] = store
        logger.info("[StoreRegistry] Registered: %s", store_id)
        try:
            loop = asyncio.get_running_loop()
        except RuntimeError:
            logger.debug(
                "[StoreRegistry] no running event loop; "
                "STORE_REGISTERED suppressed for %s",
                store_id,
            )
            return
        payload = {
            "store_id": store_id,
            "store_info": asdict(store.store_info),
        }
        loop.create_task(
            self._bus.emit(Events.STORE_REGISTERED, **payload),
            name=f"emit_store_registered_{store_id}",
        )

    def auto_discover(
        self,
        stores_dir: str,
        bus: "EventBus",
        cache: "CacheManager",
        plugin_dir: str = "",
        config=None,
    ) -> int:
        """Auto discover."""
        import importlib
        from .store_base import StoreBase as _StoreBase
        real_stores = self._validate_stores_dir(
            stores_dir, plugin_dir,

```

```

    )
    if real_stores is None:
        return 0
    package_name = "unifideck.stores"
    try:
        importlib.import_module(package_name)
    except ImportError as e:
        logger.warning(
            "[StoreRegistry] Cannot resolve stores "
            "package: %s", e,
        )
        return 0
    registered = 0
    for filename, full_path in self._iter_store_files(
        real_stores,
    ):
        store_cls = self._load_store_class(
            package_name, filename, full_path,
            _StoreBase,
        )
        if store_cls is None:
            continue
        try:
            store = store_cls(
                bus, cache, plugin_dir, config=config,
            )
            store_id = store.store_info.name
            self.register(store_id, store)
            logger.info(
                "[StoreRegistry] registered %s (%s) "
                "from %s",
                store_id, store_cls.__name__, filename,
            )
            registered += 1
        except Exception as e:
            logger.error(
                "[StoreRegistry] Failed to instantiate "
                "%s from %s: %s",
                store_cls.__name__, filename, e,
            )
    logger.info(
        "[StoreRegistry] Auto-discovery: %d stores "
        "from %s",
        registered, real_stores,
    )
    return registered

    @staticmethod
    def _validate_stores_dir(
        stores_dir: str, plugin_dir: str,
    ) -> str | None:
        """Validate stores dir."""
        try:
            real_stores = str(Path(stores_dir).resolve())
        except OSError as e:
            logger.error(
                "[StoreRegistry] Cannot resolve stores dir "
                "%r: %s",
                stores_dir, e,
            )
        return None

```

```

    if not Path(real_stores).is_dir():
        logger.warning(
            "[StoreRegistry] stores dir not found: %s",
            real_stores,
        )
        return None
    if plugin_dir:
        real_plugin = str(Path(plugin_dir).resolve())
        confined = (
            real_stores == real_plugin
            or real_stores.startswith(real_plugin + "/")
        )
        if not confined:
            logger.error(
                "[StoreRegistry] SECURITY: stores dir "
                "%s is NOT under plugin dir %s - "
                "refusing to auto-discover.",
                real_stores, real_plugin,
            )
            return None
    else:
        logger.warning(
            "[StoreRegistry] auto_discover called "
            "without plugin_dir - path confinement "
            "disabled. This is only acceptable in unit "
            "tests; production must always pass "
            "plugin_dir.",
        )
    return real_stores

@staticmethod
def _iter_store_files(real_stores: str):
    """Iter store files."""
    real_stores_p = Path(real_stores)
    for entry in sorted(real_stores_p.iterdir()):
        filename = entry.name
        if not filename.endswith("_store.py"):
            continue
        if filename.startswith("_"):
            continue
        if entry.is_symlink():
            logger.warning(
                "[StoreRegistry] SECURITY: skipping "
                "symlink %s", str(entry),
            )
            continue
        yield filename, str(entry)

@staticmethod
def _load_store_class(
    package_name: str,
    filename: str,
    full_path: str,
    store_base_cls: type,
) -> type | None:
    """Load store class."""
    import importlib
    module_name = f"{package_name}.{filename[:-3]}"
    logger.info(
        "[StoreRegistry] loading %s from %s",
        module_name, full_path,
    )

```

```

try:
    mod = importlib.import_module(module_name)
except Exception as e:
    logger.debug(
        "[StoreRegistry] Skip %s: %s", filename, e,
    )
    return None
for attr_name in dir(mod):
    attr = getattr(mod, attr_name)
    if (
        isinstance(attr, type)
        and issubclass(attr, store_base_cls)
        and attr is not store_base_cls
        and hasattr(attr, "store_info")
    ):
        return attr
return None
def get(self, store_id: str) -> "StoreBase":
    """Get."""
    if store_id not in self._stores:
        raise KeyError(
            f"Store '{store_id}' not registered. "
            f"Available: {list(self._stores.keys())}",
        )
    return self._stores[store_id]
def get_store(self, store_id: str) -> "StoreBase | None":
    """Get store."""
    return self._stores.get(store_id)
def all(self) -> list["StoreBase"]:
    """All."""
    return list(self._stores.values())
def available(self) -> list["StoreBase"]:
    """Available."""
    return [
        s for s in self._stores.values()
        if getattr(s, "_cached_available", False)
    ]
def store_ids(self) -> list[str]:
    """Store ids."""
    return list(self._stores.keys())
def has(self, store_id: str) -> bool:
    """Check whether s."""
    return store_id in self._stores
def get_store_infos(self) -> list[dict]:
    """Get store infos."""
    infos = []
    for store in self._stores.values():
        info = asdict(store.store_info)
        info["available"] = getattr(
            store, "_cached_available", False,
        )
        infos.append(info)
    return infos

async def auth_action(
    self, store_id: str, action: str, **kwargs,
) -> Result:

    """Auth action."""
    try:
        store = self.get(store_id)

```



```

except KeyError as e:
    return Result(success=False, error=str(e))
try:
    if action == "start":
        return await store.start_auth(**kwargs)
    if action == "complete":
        return await store.complete_auth(**kwargs)
    if action == "logout":
        result = await store.logout()
        if result.success:
            await self._bus.emit(
                Events.STORE_LOGOUT,
                store=store_id,
            )
        return result
    if action == "status":
        is_avail = await store.is_available()
        store._cached_available = is_avail
        return Result(success=is_avail)
    return Result(
        success=False,
        error=(
            f"Unknown auth action: '{action}'. "
            f"Valid: start, complete, logout, status"
        ),
    )
except StoreError as e:
    logger.error(
        "[StoreRegistry] %s.%s failed: %s",
        store_id, action, e,
    )
    await self._bus.emit(
        Events.STORE_AUTH_FAILED,
        store=store_id, error=str(e),
    )
    return Result(success=False, error=str(e))
except Exception as e:
    logger.exception(
        "[StoreRegistry] Unexpected error in %s.%s",
        store_id, action,
    )
    return Result(
        success=False, error=f"Unexpected: {e}",
    )
)

async def check_all_status(self) -> list[dict[str, Any]]:
    """Check all status."""
    results: list[dict[str, Any]] = []
    for store in self._stores.values():
        entry: dict[str, Any] = {
            "store_id": store.store_info.name,
            "name": store.store_info.display_name,
            "available": False,
            "error": None,
        }
        try:
            entry["available"] = await store.is_available()
            store._cached_available = entry["available"]
        except Exception as e:
            entry["error"] = str(e)
            logger.warning(
                "[StoreRegistry] %s availability check "

```

```
        "failed: %s", store.store_info.name, e,
    )
    results.append(entry)
    return results
async def logout_all(self) -> dict[str, Any]:
    """Logout all."""
    out: dict[str, Any] = {}
    for store_id in self._stores:
        result = await self.auth_action(store_id, "logout")
        out[store_id] = {
            "success": result.success,
            "error": result.error,
        }
    return out
```

OP-47b

store_base.py

Fichier : py_modules/unifideck/stores/shared/store_base.py | Lignes : 156 | Depend de : (rien)

```

import asyncio
import logging
import os
import subprocess
from abc import ABC, abstractmethod
from typing import TYPE_CHECKING, Any, Optional
from ...core.bin import binary_resolver
from ...core.exe_finder import exe_finder
from ...core.types import (
    AuthResult,
    CLITool,
    Events,
    Game,
    InstallResult,
    Result,
    StoreError,
    StoreInfo,
)
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...core.cache_manager import CacheManager
    from ...event_bus import EventBus
logger = logging.getLogger(__name__)
class StoreBase(ABC):
    """Store base."""
    store_info: StoreInfo = StoreInfo(
        name="unknown",
        display_name="Unknown",
        auth_method="manual",
        icon_asset="",
    )
    def __init__(
        self,
        bus: "EventBus",
        cache: "CacheManager",
        plugin_dir: str | None = None,
        config: Optional["ConfigManager"] = None,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._cache = cache
        self._plugin_dir = plugin_dir
        self._config = config
        self._cached_available: bool = False
    @property
    def store_name(self) -> str:
        """Store name."""
        return self.store_info.name
    @abstractmethod
    async def is_available(self) -> bool:
        """Check whether available."""
        ...
    @abstractmethod
    async def start_auth(self, **kwargs) -> AuthResult:
        """Start auth."""

```

```

    ...
    @abstractmethod
    async def complete_auth(self, **kwargs) -> AuthResult:
        """Complete auth."""
        ...
    @abstractmethod
    async def logout(self) -> Result:
        """Logout."""
        ...
    @abstractmethod
    async def get_library(self) -> list[Game] | None:
        """Get library."""
        ...

    @abstractmethod
    async def install_game(
        self, game_id: str, **kwargs: Any,
    ) -> InstallResult:
        """Install game."""
        ...
    @abstractmethod
    async def uninstall_game(
        self, game_id: str, **kwargs: Any,
    ) -> Result:
        """Uninstall game."""
        ...
    @abstractmethod
    async def update_game(
        self, game_id: str, **kwargs: Any,
    ) -> InstallResult:
        """Update game."""
        ...
    @abstractmethod
    async def check_for_updates(self) -> list[str]:
        """Check for updates."""
        ...
    @abstractmethod
    async def get_game_size(self, game_id: str) -> int | None:
        """Get game size."""
        ...
    def _find_binary(self, tool: CLITool) -> str | None:
        """Find binary."""
        return binary_resolver.resolve(tool)
    def _find_exe(
        self,
        install_path: str,
        hints: list[str] | None = None,
    ) -> str | None:
        """Find exe."""
        return exe_finder.find(install_path, hints)
    async def _emit(self, event: Events, **kwargs) -> None:
        """Emit."""
        await self._bus.emit(event, **kwargs)

    async def _run_cli(
        self,
        args: list[str],
        binary_path: str | None = None,
        timeout: int = 300,
        env: dict[str, str] | None = None,
    ) -> str:

```

```
"""Run cli."""
bin_path = binary_path or getattr(self, "cli_path", None)
if not bin_path:
    raise StoreError(
        "CLI binary not found",
        store=self.store_name,
    )
cmd = [bin_path] + args
process_env = (
    dict(os.environ) if env is None
    else {**os.environ, **env}
)
def _run():
    """Run."""
    result = subprocess.run(
        cmd,
        capture_output=True,
        text=True,
        timeout=timeout,
        env=process_env,
    )
    if result.returncode != 0:
        raise StoreError(
            f"CLI error (rc={result.returncode}): "
            f"{result.stderr[:500]}",
            store=self.store_name,
        )
    return result.stdout
try:
    return await asyncio.to_thread(_run)
except subprocess.TimeoutExpired as e:
    raise StoreError(
        f"CLI timeout after {timeout}s: {' '.join(cmd[:3])}",
        store=self.store_name,
    ) from e
except StoreError:
    raise
except Exception as e:
    raise StoreError(
        f"CLI execution failed: {e}",
        store=self.store_name,
    ) from e
```

OP-47c

dlc.py

Fichier : py_modules/unifideck/stores/shared/dlc.py | Lignes : 11 | Depend de : (rien)

```
import logging
logger = logging.getLogger(__name__)
_DLC_SUPPORTED_STORES = {"epic", "gog"}
def get_dlc_flags(store: str) -> list[str]:
    """Get dlc flags."""
    if store.lower() in _DLC_SUPPORTED_STORES:
        return ["--with-dlcs"]
    return []
def store_supports_dlc(store: str) -> bool:
    """Store supports dlc."""
    return store.lower() in _DLC_SUPPORTED_STORES
```

OP-47d

cli_install_helpers.py

Fichier : py_modules/unifideck/stores/shared/cli_install_helpers.py | Lignes : 55 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    import re
    from collections.abc import Awaitable, Callable
    LineHandler = Callable[
        [str, str, "ProgressCallback | None"], Awaitable[None],
    ]
    ProgressCallback = Callable[[float], Awaitable[None]]
logger = logging.getLogger(__name__)
async def drain_install_output(
    proc: Any,
    game_id: str,
    progress_cb: ProgressCallback | None,
    line_handler: LineHandler,
) -> None:
    """Drain install output."""
    assert proc.stdout is not None
    while True:
        line_bytes = await proc.stdout.readline()
        if not line_bytes:
            break
        line = line_bytes.decode(errors="ignore").strip()
        if line:
            await line_handler(line, game_id, progress_cb)
async def wait_with_timeout(
    proc: Any,
    timeout_s: int,
    log_prefix: str,
) -> int:
    """Wait with timeout."""
    try:
        await asyncio.wait_for(proc.wait(), timeout=timeout_s)
    except TimeoutError:
        logger.error(
            "%s timeout after %ds, killing",
            log_prefix, timeout_s,
        )
        proc.kill()
        await proc.wait()
        return -1
    return proc.returncode or 0
def parse_progress_line(
    line: str, pattern: re.Pattern[str],
) -> float | None:
    """Parse progress line."""
    match = pattern.search(line)
    if not match:
        return None
    try:
        return float(match.group(1))
    except ValueError:
        return None

```

OP-48

__init__.py

Fichier : py_modules/unifideck/stores/epic/__init__.py | Lignes : 2 | Depend de : (rien)

```
from .store import EpicStore
__all__ = ["EpicStore"]
```


OP-48a

store.py

Fichier : py_modules/unifideck/stores/epic/store.py | Lignes : 261 | Depend de : (rien)

```
from __future__ import annotations
import json
import logging
import os
from typing import TYPE_CHECKING, Any, cast
from ...auth.browser import OAuthBrowserMonitor
from ...auth.orchestrator import AuthOrchestrator
from ...core.bin import read_cli_timeouts
from ...core.types import (
    AuthResult,
    CLITool,
    Events,
    Game,
    InstallResult,
    Result,
    StoreInfo,
)
from ...security import emit_external_auth_check_failed
from ...services.shortcut import ShortcutService
from ...utils.config_helpers import get_cfg
from ..shared.store_base import StoreBase
from .auth import EpicAuthFlow
from .exe_resolver import EpicExeResolver
from .install import EpicInstaller, ProgressCallback
from .library import EpicLibraryReader, merge_install_status
from .updates import EpicUpdateChecker
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...core.cache_manager import CacheManager
    from ...event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
class EpicStore(StoreBase):
    """Epic store."""
    store_info = StoreInfo(
        name="epic",
        display_name="Epic Games",
        auth_method="oauth",
        icon_asset="epic.png",
        uses_wine=False,
        supports_install=True,
    )
    CLI_TOOL = CLITool(
        name="legendary",
        search_paths=["bin/legendary"],
        version_flag="--version",
    )

    def __init__(
        self,
        bus: EventBus,
        cache: CacheManager,
        plugin_dir: str | None = None,
        config: ConfigManager | None = None,
        browser_monitor: OAuthBrowserMonitor | None = None,
        shortcut_service: ShortcutService | None = None,
```

```

) -> None:

    """Initialize the instance."""
    super().__init__(bus, cache, plugin_dir, config)
    self.cli_path: str | None = self._find_binary(self.CLI_TOOL)
    if not self.cli_path:
        logger.warning("[EpicStore] legendary binary not found")
    self._shortcut_service = shortcut_service
    self._timeouts = read_cli_timeouts(config)
    epic_cfg = config.get("stores.epic") if config else None
    if epic_cfg is None:
        raise KeyError("config.stores.epic is required")
    self._build_cli_submodules(bus, epic_cfg)
    self._build_auth_submodule(bus, browser_monitor)
def _build_cli_submodules(
    self, bus: EventBus, epic_cfg: dict[str, Any],
) -> None:
    """Build cli submodules."""
    self._library = EpicLibraryReader(
        cli_path=self.cli_path,
        library_timeout=self._timeouts["library_fetch"],
    )
    self._exe_resolver = EpicExeResolver(
        cli_path=self.cli_path,
        find_exe=self._find_exe,
        info_timeout_seconds=epic_cfg["info_timeout_seconds"],
    )
    self._installer = EpicInstaller(
        bus=bus,
        cli_path=self.cli_path,
        library=self._library,
        exe_resolver=self._exe_resolver,
        default_install_root=epic_cfg["default_install_root"],
    )
    self._updates = EpicUpdateChecker(
        bus=bus,
        cli_path=self.cli_path,
        library=self._library,
        list_updates_timeout=epic_cfg[
            "list_updates_timeout_seconds"
        ],
        size_cache_ttl=epic_cfg["size_cache_ttl_seconds"],
        info_timeout=epic_cfg["info_timeout_seconds"],
    )
def _build_auth_submodule(
    self,
    bus: EventBus,
    browser_monitor: OAuthBrowserMonitor | None,
) -> None:
    """Build auth submodule."""
    if browser_monitor is None:
        self._auth: EpicAuthFlow | None = None
        logger.debug(
            "[EpicStore] no browser_monitor; auth disabled",
        )
        return
    orchestrator = AuthOrchestrator(
        bus=bus,
        browser_monitor=browser_monitor,
        store_name="epic",
    )

```

```

        self._auth = EpicAuthFlow(
            bus=bus,
            orchestrator=orchestrator,
            cli_path=self.cli_path,
            cli_timeout_seconds=self._timeouts["auth_check"],
        )
    async def is_available(self) -> bool:
        """Check whether available."""
        ok = self._check_legacy_authenticated()
        self._cached_available = ok
        return ok

    def _check_legacy_authenticated(self) -> bool:
        """Check LEGENDARY authenticated."""
        if not self.cli_path:
            emit_external_auth_check_failed(
                self._bus, "epic", "cli_not_found",
                "legendary binary missing from search paths",
            )
            return False
        user_file = os.path.expanduser(
            get_cfg(
                self._config, "stores.epic.user_file",
                "~/.config/legendary/user.json",
            ),
        )
        if not os.path.isfile(user_file):
            return False
        try:
            with open(user_file, encoding="utf-8") as f:
                data = json.load(f)
        except (OSError, json.JSONDecodeError) as e:
            logger.debug(
                "[EpicStore] user.json invalid: %s", e,
            )
            emit_external_auth_check_failed(
                self._bus, "epic", "parse_error",
                f"{type(e).__name__}",
            )
            return False
        if not isinstance(data, dict):
            emit_external_auth_check_failed(
                self._bus, "epic", "malformed_payload",
                "not a JSON object",
            )
            return False
        return "access_token" in data
    async def start_auth(self, **kwargs) -> AuthResult:
        """Start auth."""
        if self._auth is None:
            return AuthResult(
                success=False,
                error="auth_not_configured",
                store="epic",
            )
        await self._ensure_auth_shortcut()
        return cast("AuthResult", await self._auth.start_auth())
    async def complete_auth(self, code: str = "", **kwargs) -> AuthResult:
        """Complete auth."""
        if await self.is_available():

```

```

        return AuthResult(success=True, store="epic")
    return AuthResult(
        success=False,
        error="not_authenticated",
        store="epic",
    )
async def logout(self) -> Result:
    """Logout."""
    if self._auth is None:
        await self._emit(Events.STORE_LOGOUT, store="epic")
        return Result(success=True)
    return await self._auth.logout()
async def get_library(self) -> list[Game] | None:
    """Get library."""
    if not self.cli_path:
        return []
    try:
        owned = await self._library.read_owned_games()
        installed = await self._library.read_installed_map()
        return merge_install_status(owned, installed)
    except Exception as e:
        logger.error(
            "[EpicStore] get_library failed: %s", e,
        )
    return []
async def install_game(
    self,
    game_id: str,
    base_path: str | None = None,
    progress_cb: ProgressCallback | None = None,
    **kwargs: Any,
) -> InstallResult:
    """Install game."""
    return await self._installer.install_game(
        game_id, base_path, progress_cb,
    )

async def uninstall_game(
    self, game_id: str, **kwargs: Any,
) -> Result:
    """Uninstall game."""
    return await self._installer.uninstall_game(game_id)
async def update_game(
    self,
    game_id: str,
    progress_cb: ProgressCallback | None = None,
    **kwargs: Any,
) -> InstallResult:
    """Update game."""
    return await self._updates.update_game(
        game_id,
        installer=self._installer,
        progress_cb=progress_cb,
    )
async def check_for_updates(self) -> list[str]:
    """Check for updates."""
    return await self._updates.check_for_updates()
async def get_game_size(self, game_id: str) -> int | None:
    """Get game size."""
    return await self._updates.get_game_size(game_id)

```

```
async def _ensure_auth_shortcut(self) -> None:
    """Ensure auth shortcut."""
    if self._shortcut_service is None:
        logger.debug(
            "[EpicStore] no shortcut_service injected; "
            "skipping auth shortcut creation",
        )
        return
    launcher = os.path.join(
        self._plugin_dir or "",
        "py_modules", "unifideck", "launcher",
        "dispatcher.py",
    )
    if not os.path.isfile(launcher):
        logger.warning(
            "[EpicStore] launcher dispatcher not found "
            "at %s", launcher,
        )
        return
    result = await self._shortcut_service.add_auth_shortcut(
        store="epic",
        launcher_path=launcher,
        title="Epic Games Sign-In",
    )
    if not result.success:
        logger.warning(
            "[EpicStore] add_auth_shortcut failed: %s",
            result.error,
        )
```

OP-48b

auth.py

Fichier : py_modules/unifideck/stores/epic/auth.py | Lignes : 179 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import Any
from ...auth.orchestrator import AuthOrchestrator
from ...core.types import AuthResult, Events, Result, StoreAuthError
from ...event_bus.event_bus import EventBus
from ...security import audit_auth_flow
logger = logging.getLogger(__name__)
_EPIC_REDIRECT_URIS: list[str] = [
    "https://legendary.epicgames.com/callback",
    "https://www.epicgames.com/id/api/redirect",
]
_AUTH_URL_MARKERS = ("epicgames.com",)
class EpicAuthFlow:
    """Epic auth flow."""
    def __init__(
        self,
        bus: EventBus,
        orchestrator: AuthOrchestrator,
        cli_path: str | None,
        cli_timeout_seconds: int = 30,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._orch = orchestrator
        self._cli_path = cli_path
        self._cli_timeout = cli_timeout_seconds
    @audit_auth_flow(store="epic", method="oauth_cli")
    async def start_auth(self) -> AuthResult:
        """Start auth."""
        if not self._cli_path:
            return AuthResult(
                success=False,
                error="legendary_not_found",
                store="epic",
            )
        return await self._orch.run_flow(
            get_url=self._fetch_login_url,
            allowed_uris=_EPIC_REDIRECT_URIS,
            exchange_code=self._register_code,
            background=True,
            write_url_file=(
                "~/local/share/unifideck/epic_auth_url.txt"
            ),
        )
    async def logout(self) -> Result:
        """Logout."""
        if not self._cli_path:
            await self._bus.emit(
                Events.STORE_LOGOUT, store="epic",
            )
            return Result(success=True)

```

```

try:
    proc = await asyncio.create_subprocess_exec(
        self._cli_path, "auth", "--delete",
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    await asyncio.wait_for(
        proc.communicate(),
        timeout=self._cli_timeout,
    )
except (TimeoutError, OSError) as e:
    logger.warning("[epic_auth] logout: %s", e)
await self._bus.emit(
    Events.STORE_LOGOUT, store="epic",
)
return Result(success=True)
async def _fetch_login_url(self) -> str:
    """Fetch login URL."""
    proc = await self._spawn_legendary_auth()
    try:
        url = await self._scrape_url_from_proc(proc)
    finally:
        await self._terminate_legendary(proc)
    if not url:
        raise StoreAuthError(
            "no OAuth URL found in legendary auth output",
            store="epic",
        )
    logger.info("[epic_auth] captured URL from legendary")
    return url
async def _spawn_legendary_auth(self) -> Any:
    """Spawn LEGENDARY auth."""
    assert self._cli_path is not None, (
        "_spawn_legendary_auth called before CLI is resolved"
    )
    return await asyncio.create_subprocess_exec(
        self._cli_path, "auth",
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.STDOUT,
    )
async def _scrape_url_from_proc(
    self, proc: Any,
) -> str | None:
    """Scrape URL from proc."""
    assert proc.stdout is not None
    deadline = (
        asyncio.get_event_loop().time() + self._cli_timeout
    )
    while asyncio.get_event_loop().time() < deadline:
        remaining = (
            deadline - asyncio.get_event_loop().time()
        )
        if remaining <= 0:
            return None
        try:
            line_bytes = await asyncio.wait_for(
                proc.stdout.readline(),
                timeout=remaining,
            )
        except TimeoutError:
            return None

```

```

        if not line_bytes:
            return None
        text = line_bytes.decode(errors="ignore").strip()
        url = self._extract_url(text)
        if url:
            return url
        return None
    @staticmethod
    async def _terminate_legacy(proc: Any) -> None:
        """Terminate LEGACY."""
        try:
            proc.terminate()
            await asyncio.wait_for(proc.wait(), timeout=2)
        except TimeoutError:
            proc.kill()
            await proc.wait()

    async def _register_code(self, code: str) -> AuthResult:
        """Register code."""
        assert self._cli_path is not None, (
            "_register_code called before CLI is resolved"
        )
        proc = await asyncio.create_subprocess_exec(
            self._cli_path, "auth", "--code", code,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        try:
            stdout, stderr = await asyncio.wait_for(
                proc.communicate(),
                timeout=self._cli_timeout,
            )
        except TimeoutError:
            return AuthResult(
                success=False,
                error="register_timeout",
                store="epic",
            )
        if proc.returncode == 0:
            logger.info(
                "[epic_auth] legendary register succeeded",
            )
            return AuthResult(success=True, store="epic")
        err = (
            stderr.decode(errors="ignore")
            or stdout.decode(errors="ignore")
        )[:200]
        logger.warning(
            "[epic_auth] register failed: %s", err,
        )
        return AuthResult(
            success=False,
            error="register_failed",
            store="epic",
        )
    @staticmethod
    def _extract_url(line: str) -> str | None:
        """Extract URL."""
        if "https://" not in line:
            return None

```



```
for token in line.split():
    if not token.startswith("https://"):
        continue
    if any(m in token for m in _AUTH_URL_MARKERS):
        return token
return None
```

OP-48c

library.py

Fichier : py_modules/unifideck/stores/epic/library.py | Lignes : 163 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
import time
from typing import Any
from ...core.types import Game
from .filter import should_filter_epic_item
logger = logging.getLogger(__name__)
DEFAULT_INSTALLED_TTL = 30
class EpicLibraryReader:
    """Epic library reader."""
    def __init__(
        self,
        cli_path: str | None,
        library_timeout: int = 30,
        installed_ttl: int = DEFAULT_INSTALLED_TTL,
    ) -> None:
        """Initialize the instance."""
        self._cli_path = cli_path
        self._timeout = library_timeout
        self._installed_ttl = installed_ttl
        self._installed_cache: dict[str, dict[str, Any]] | None = None
        self._installed_cache_ts: float = 0.0
    async def read_owned_games(self) -> list[Game]:
        """Read owned games."""
        if not self._cli_path:
            return []
        data = await self._runLegendaryJson(["list", "--json"])
        if not isinstance(data, list):
            return []
        games: list[Game] = []
        filtered = 0
        for item in data:
            if not isinstance(item, dict):
                continue
            if should_filter_epic_item(item):
                filtered += 1
                continue
            app_name = item.get("app_name", "")
            if not app_name:
                continue
            games.append(Game(
                app_id=0,
                store="epic",
                store_game_id=app_name,
                title=item.get("app_title") or app_name,
                installed=False,
            ))
        logger.info(
            "[epic_library] %d owned games (%d filtered)",
            len(games), filtered,
        )
        return games

```

```

async def read_installed_map(
    self,
    force_refresh: bool = False,
) -> dict[str, dict[str, Any]]:

    """Read installed map."""
    if not self._cli_path:
        return {}
    if (
        not force_refresh
        and self._installed_cache is not None
    ):
        age = time.time() - self._installed_cache_ts
        if age < self._installed_ttl:
            return self._installed_cache
    result = await self._load_installed_from_cli()
    self._installed_cache = result
    self._installed_cache_ts = time.time()
    logger.info(
        "[epic_library] %d installed games", len(result),
    )
    return result
async def _load_installed_from_cli(
    self,
) -> dict[str, dict[str, Any]]:
    """Load installed from cli."""
    data = await self._runLegendaryJson(
        ["list-installed", "--json"],
    )
    result: dict[str, dict[str, Any]] = {}
    if not isinstance(data, list):
        return result
    for entry in data:
        if not isinstance(entry, dict):
            continue
        app_name = entry.get("app_name")
        if app_name:
            result[app_name] = entry
    return result
def invalidate_installed_cache(self) -> None:
    """Invalidate installed cache."""
    self._installed_cache = None
    self._installed_cache_ts = 0.0
async def _runLegendaryJson(
    self,
    args: list[str],
) -> Any:
    """Run LEGENDARY JSON."""
    if self._cli_path is None:
        return None
    try:
        proc = await asyncio.create_subprocess_exec(
            self._cli_path, *args,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self._timeout,
        )
    except (TimeoutError, OSError) as e:

```

```

        logger.warning(
            "[epic_library] legendary %s failed: %s",
            args[0], e,
        )
        return None
    if proc.returncode != 0:
        logger.error(
            "[epic_library] legendary %s exit %d: %s",
            args[0], proc.returncode,
            stderr.decode(errors="ignore")[:200],
        )
        return None
    try:
        return json.loads(stdout.decode(errors="ignore"))
    except json.JSONDecodeError as e:
        logger.error(
            "[epic_library] JSON parse error: %s", e,
        )
        return None

def merge_install_status(
    owned: list[Game],
    installed: dict[str, dict[str, Any]],
) -> list[Game]:
    """Merge install status."""
    merged: list[Game] = []
    for game in owned:
        entry = installed.get(game.store_game_id)
        if entry is None:
            merged.append(game)
            continue
        install_data = entry.get("install", {}) or {}
        merged.append(Game(
            app_id=game.app_id,
            store=game.store,
            store_game_id=game.store_game_id,
            title=game.title,
            installed=True,
            install_path=(
                install_data.get("install_path") or None
            ),
            exe_path=game.exe_path,
            icon_url=game.icon_url,
            hero_url=game.hero_url,
            logo_url=game.logo_url,
            size_bytes=game.size_bytes,
            tags=list(game.tags),
            metadata=dict(game.metadata),
        ))
    return merged

```

OP-48d

install.py

Fichier : py_modules/unifideck/stores/epic/install.py | Lignes : 230 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
import re
from collections.abc import Awaitable, Callable
from typing import Any
from ...core.manifest import write_manifest
from ...core.types import Events, InstallResult, Result
from ...event_bus.event_bus import EventBus
from ..shared import dlc
from ..shared.cli_install_helpers import (
    drain_install_output,
    parse_progress_line,
    wait_with_timeout,
)
from .exe_resolver import EpicExeResolver
from .library import EpicLibraryReader
logger = logging.getLogger(__name__)
_PROGRESS_RE = re.compile(r"(\d+(?:\.\d+)?)\s*%")
ProgressCallback = Callable[[float], Awaitable[None]]
class EpicInstaller:
    """Epic installer."""
    def __init__(
        self,
        bus: EventBus,
        cli_path: str | None,
        library: EpicLibraryReader,
        exe_resolver: EpicExeResolver,
        default_install_root: str,
        install_timeout_seconds: int = 7200,
        uninstall_timeout_seconds: int = 120,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._cli_path = cli_path
        self._library = library
        self._exe_resolver = exe_resolver
        self._default_install_root = os.path.expanduser(default_install_root)
        self._install_timeout = install_timeout_seconds
        self._uninstall_timeout = uninstall_timeout_seconds

    async def install_game(
        self,
        game_id: str,
        base_path: str | None = None,
        progress_cb: ProgressCallback | None = None,
    ) -> InstallResult:
        """Install game."""
        if not self._cli_path:
            return InstallResult(
                success=False, error="legendary_not_found",
                store="epic", game_id=game_id,
            )

```

```

base = base_path or self._default_install_root
try:
    os.makedirs(base, exist_ok=True)
except OSError as e:
    return InstallResult(
        success=False,
        error=f"mkdir_failed: {e}",
        store="epic", game_id=game_id,
    )
await self._bus.emit(
    Events.DOWNLOAD_STARTED,
    store="epic", game_id=game_id,
)
rc = await self._run_install(base, game_id, progress_cb)
if rc != 0:
    await self._bus.emit(
        Events.DOWNLOAD_FAILED,
        store="epic", game_id=game_id,
        error=f"legendary_exit_{rc}",
    )
    return InstallResult(
        success=False,
        error=f"legendary_exit_{rc}",
        store="epic", game_id=game_id,
    )
self._library.invalidate_installed_cache()
return await self._finalize_install(game_id, base)
async def _run_install(
    self,
    base: str,
    game_id: str,
    progress_cb: ProgressCallback | None,
) -> int:
    """Run install."""
    cmd = self._build_install_cmd(base, game_id)
    proc = await asyncio.create_subprocess_exec(
        *cmd,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.STDOUT,
    )
    drain_exc: BaseException | None = None
    try:
        await self._drain_install_output(proc, game_id, progress_cb)
    except BaseException as e:
        drain_exc = e
    rc = await self._wait_with_timeout(proc)
    if drain_exc is not None:
        raise drain_exc
    return rc
def _build_install_cmd(
    self, base: str, game_id: str,
) -> list[str]:
    """Build install cmd."""
    if self._cli_path is None:
        raise RuntimeError(
            "legendary CLI path is not set; cannot build install cmd",
        )
    cmd = [
        self._cli_path, "install", game_id,
        "--base-path", base,
        "--yes",

```

```

    ]
    cmd.extend(dlc.get_dlc_flags("epic"))
    return cmd

async def _drain_install_output(
    self,
    proc: Any,
    game_id: str,
    progress_cb: ProgressCallback | None,
) -> None:

    """Drain install output."""
    await drain_install_output(
        proc, game_id, progress_cb,
        self._handle_install_line,
    )

async def _wait_with_timeout(self, proc: Any) -> int:
    """Wait with timeout."""
    return await wait_with_timeout(
        proc, self._install_timeout, "[epic_install]",
    )

async def _handle_install_line(
    self,
    line: str,
    game_id: str,
    progress_cb: ProgressCallback | None,
) -> None:
    """Handle install line."""
    if "Progress:" not in line:
        logger.debug("[legendary install] %s", line)
        return
    pct = parse_progress_line(line, _PROGRESS_RE)
    if pct is None:
        return
    if progress_cb is not None:
        try:
            await progress_cb(pct)
        except Exception as e:
            logger.debug(
                "[epic_install] progress_cb raised: %s",
                e,
            )
    await self._bus.emit(
        Events.DOWNLOAD_PROGRESS,
        store="epic", game_id=game_id,
        progress=pct,
    )

async def _finalize_install(
    self, game_id: str, base: str,
) -> InstallResult:
    """Finalize install."""
    resolved = await self._exe_resolver.resolve(game_id)
    install_path = resolved["install_path"]
    exe = resolved["executable"]
    title = resolved["title"]
    if install_path:
        exe_relative = ""
        if exe:
            try:
                exe_relative = os.path.relpath(
                    exe, install_path,

```

```

        )
        except ValueError:
            pass
    await write_manifest(
        install_dir=install_path,
        store="epic",
        store_id=game_id,
        title=title,
        executable_relative=exe_relative,
        platform="windows",
    )
    await self._bus.emit(
        Events.DOWNLOAD_COMPLETE,
        store="epic", game_id=game_id,
        install_path=install_path,
    )
    return InstallResult(
        success=True,
        store="epic",
        game_id=game_id,
        install_path=install_path or os.path.join(
            base, game_id,
        ),
    )
)

async def uninstall_game(self, game_id: str) -> Result:
    """Uninstall game."""
    if not self._cli_path:
        return Result(
            success=False, error="legendary_not_found",
        )
    proc = await asyncio.create_subprocess_exec(
        self._cli_path, "uninstall", game_id, "--yes",
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    try:
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self._uninstall_timeout,
        )
    except TimeoutError:
        proc.kill()
        return Result(
            success=False, error="uninstall_timeout",
        )
    if proc.returncode != 0:
        err = stderr.decode(errors="ignore")[:200]
        return Result(
            success=False,
            error=f"uninstall_failed: {err}",
        )
    self._library.invalidate_installed_cache()
    await self._bus.emit(
        Events.GAME_UNINSTALLED,
        store="epic", game_id=game_id,
    )
    return Result(success=True)

```


OP-48e

updates.py

Fichier : py_modules/unifideck/stores/epic/updates.py | Lignes : 151 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import os
import time
from typing import Any, cast
from ...core.types import InstallResult
from ...event_bus.event_bus import EventBus
from .legendary import fetch_info
from .library import EpicLibraryReader
logger = logging.getLogger(__name__)
class EpicUpdateChecker:
    """Epic update checker."""
    def __init__(
        self,
        bus: EventBus,
        cli_path: str | None,
        library: EpicLibraryReader,
        list_updates_timeout: int,
        size_cache_ttl: int,
        info_timeout: float,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._cli_path = cli_path
        self._library = library
        self._list_updates_timeout = list_updates_timeout
        self._size_cache_ttl = size_cache_ttl
        self._info_timeout = info_timeout
        self._size_cache: dict[str, tuple] = {}
    async def check_for_updates(self) -> list[str]:
        """Check for updates."""
        if not self._cli_path:
            return []
        try:
            proc = await asyncio.create_subprocess_exec(
                self._cli_path,
                "list-installed", "--check-updates",
                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE,
            )
            stdout, _ = await asyncio.wait_for(
                proc.communicate(),
                timeout=self._list_updates_timeout,
            )
        except (TimeoutError, OSError) as e:
            logger.warning(
                "[epic_updates] list-installed failed: %s", e,
            )
            return []
        if proc.returncode != 0:
            return []
        return self._parse_update_output(
            stdout.decode(errors="ignore"),
        )
```

```

@staticmethod
def _parse_update_output(text: str) -> list[str]:
    """Parse update output."""
    updates: list[str] = []
    current_app: str | None = None
    for line in text.splitlines():
        stripped = line.strip()
        if (
            stripped.startswith("*")
            and "App name:" in stripped
        ):
            try:
                current_app = (
                    stripped.split("App name:")[1]
                    .split("|")[0]
                    .strip()
                )
            except IndexError:
                current_app = None
        elif (
            stripped.startswith("-> Update available!")
            and current_app
        ):
            updates.append(current_app)
            current_app = None
    return updates

async def update_game(
    self,
    game_id: str,
    installer: Any,
    progress_cb: Any = None,
) -> InstallResult:
    """Update game."""
    if not self._cli_path:
        return InstallResult(
            success=False, error="legendary_not_found",
            store="epic", game_id=game_id,
        )
    installed = await self._library.read_installed_map()
    entry = installed.get(game_id)
    if not entry:
        return InstallResult(
            success=False, error="not_installed",
            store="epic", game_id=game_id,
        )
    install_data = entry.get("install") or {}
    current_path = install_data.get("install_path", "")
    base_path = (
        os.path.dirname(current_path)
        if current_path else None
    )
    result = await installer.install_game(
        game_id, base_path=base_path,
        progress_cb=progress_cb,
    )
    if result.success:
        self._size_cache.pop(game_id, None)
        self._library.invalidate_installed_cache()
        return cast("InstallResult", result)
    async def get_game_size(self, game_id: str) -> int | None:

```

```
    """Get game size."""
    if not self._cli_path:
        return None
    entry = self._size_cache.get(game_id)
    if entry is not None:
        size, ts = entry
        if time.time() - ts < self._size_cache_ttl:
            return cast("int | None", size)
    size = await self._load_game_size_from_cli(game_id)
    if size is not None:
        self._size_cache[game_id] = (size, time.time())
    return size

async def _load_game_size_from_cli(
    self, game_id: str,
) -> int | None:

    """Load game size from cli."""
    info = await self._fetch_info(game_id)
    if info is None:
        return None
    manifest = info.get("manifest") or {}
    size = manifest.get("download_size")
    if not isinstance(size, int):
        return None
    return size

async def _fetch_info(
    self, game_id: str,
) -> dict[str, Any] | None:
    """Fetch info."""
    if self._cli_path is None:
        return None
    return await fetch_info(
        self._cli_path,
        game_id,
        timeout=self._info_timeout,
        log_prefix="[epic_updates]",
    )
```

OP-48f

filter.py

Fichier : py_modules/unifideck/stores/epic/filter.py | Lignes : 67 | Depend de : (rien)

```
from __future__ import annotations
import logging
from collections.abc import Iterable
from typing import Any
logger = logging.getLogger(__name__)
ASSET_CATEGORIES: set[str] = {
    "assets",
    "asset-format",
    "plugins",
    "projects",
}
MOBILE_PLATFORMS: set[str] = {"Android", "iOS"}
def has_ue_namespace(metadata: dict[str, Any]) -> bool:
    """Check whether ue namespace."""
    return metadata.get("namespace") == "ue"
def has_asset_category(metadata: dict[str, Any]) -> bool:
    """Check whether asset category."""
    categories = metadata.get("categories") or []
    for cat in categories:
        path = (cat.get("path") or "").lower()
        if path in ASSET_CATEGORIES:
            return True
    return False
def has_mod_category(metadata: dict[str, Any]) -> bool:
    """Check whether mod category."""
    categories = metadata.get("categories") or []
    return any((cat.get("path") or "").lower() == "mods" for cat in categories)
def is_mobile_only(metadata: dict[str, Any]) -> bool:
    """Check whether mobile only."""
    release_info = metadata.get("releaseInfo") or []
    if not release_info:
        return False
    for info in release_info:
        platforms: Iterable[str] = info.get("platform") or []
        if not platforms:
            return False
        if not all(p in MOBILE_PLATFORMS for p in platforms):
            return False
    return True
def should_filter_epic_item(game_data: dict[str, Any]) -> bool:
    """Check whether filter epic item."""
    metadata = game_data.get("metadata") or {}
    if has_ue_namespace(metadata):
        logger.debug(
            "[epic_filter] UE namespace: %s",
            game_data.get("app_title"),
        )
        return True
    if has_asset_category(metadata):
        logger.debug(
            "[epic_filter] asset category: %s",
            game_data.get("app_title"),
        )
        return True
    if has_mod_category(metadata):
```

```
        logger.debug(
            "[epic_filter] mod category: %s",
            game_data.get("app_title"),
        )
        return True
if is_mobile_only(metadata):
    logger.debug(
        "[epic_filter] mobile-only: %s",
        game_data.get("app_title"),
    )
    return True
return False
```

OP-48g

exe_resolver.py

Fichier : py_modules/unifideck/stores/epic/exe_resolver.py | Lignes : 89 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
from collections.abc import Callable
from typing import Any
from .legendary import fetch_info
logger = logging.getLogger(__name__)
class EpicExeResolver:
    """Epic exe resolver."""
    def __init__(
        self,
        cli_path: str | None,
        find_exe: Callable[[str, list[str] | None], str | None],
        info_timeout_seconds: float,
    ) -> None:
        """Initialize the instance."""
        self._cli_path = cli_path
        self._find_exe = find_exe
        self._info_timeout = info_timeout_seconds
    async def resolve(
        self, game_id: str,
    ) -> dict[str, Any]:
        """Resolve."""
        info = await self._fetch_info(game_id)
        install_path = self._extract_install_path(info)
        exe = self._resolve_executable(install_path, info, game_id)
        title = self._extract_title(info, game_id)
        return {
            "install_path": install_path,
            "executable": exe,
            "title": title,
        }
    async def _fetch_info(
        self, game_id: str,
    ) -> dict[str, Any] | None:
        """Fetch info."""
        if self._cli_path is None:
            return None
        return await fetch_info(
            self._cli_path,
            game_id,
            timeout=self._info_timeout,
            log_prefix="[epic_exe_resolver]",
        )
    @staticmethod
    def _extract_install_path(
        info: dict | None,
    ) -> str | None:
        """Extract install path."""
        if not info:
            return None
        install = info.get("install") or {}
        path = install.get("install_path")
        return path if isinstance(path, str) and path else None
    @staticmethod

```

```
def _extract_title(
    info: dict | None, game_id: str,
) -> str:
    """Extract title."""
    if not info:
        return game_id
    game = info.get("game") or {}
    title = game.get("title")
    return title if isinstance(title, str) and title else game_id

def _resolve_executable(
    self,
    install_path: str | None,
    info: dict | None,
    game_id: str,
) -> str | None:
    """Resolve executable."""
    if not install_path:
        return None
    if info is not None:
        manifest = info.get("manifest") or {}
        launch_exe = manifest.get("launch_exe")
        if isinstance(launch_exe, str) and launch_exe:
            cleaned = launch_exe.lstrip("/")
            candidate = os.path.join(
                install_path, cleaned,
            )
            if os.path.isfile(candidate):
                return candidate
            logger.warning(
                "[epic_exe_resolver] manifest launch_exe "
                "missing on disk: %s → fallback to ExeFinder",
                candidate,
            )
    return self._find_exe(install_path, [game_id])
```

OP-48h

legendary.py

Fichier : py_modules/unifideck/stores/epic/legendary.py | Lignes : 39 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import json
import logging
from typing import Any, cast
_logger = logging.getLogger(__name__)
async def fetch_info(
    cli_path: str,
    game_id: str,
    *,
    timeout: float,
    log_prefix: str = "[epic_legendary]",
) -> dict[str, Any] | None:
    """Fetch info."""
    if not cli_path:
        return None
    try:
        proc = await asyncio.create_subprocess_exec(
            cli_path, "info", game_id, "--json",
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, _ = await asyncio.wait_for(
            proc.communicate(), timeout=timeout,
        )
    except (TimeoutError, OSError) as e:
        _logger.warning(
            "%s legendary info failed: %s", log_prefix, e,
        )
        return None
    if proc.returncode != 0:
        return None
    try:
        return cast("dict[str, Any] | None", json.loads(stdout.decode(errors="ignore")))
    except json.JSONDecodeError as e:
        _logger.warning(
            "%s JSON parse error: %s", log_prefix, e,
        )
        return None
```


OP-49

`__init__.py`

Fichier : `py_modules/unifideck/stores/amazon/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from .amazon_store import AmazonStore
__all__ = ["AmazonStore"]
```

OP-49a

amazon_store.py

Fichier : py_modules/unifideck/stores/amazon/amazon_store.py | Lignes : 257 | Depend de : (rien)

```
from __future__ import annotations
import json
import logging
import os
from typing import TYPE_CHECKING, Any, cast
from ...auth.browser import OAuthBrowserMonitor
from ...auth.orchestrator import AuthOrchestrator
from ...core.types import (
    AuthResult,
    CLITool,
    Events,
    Game,
    InstallResult,
    Result,
    StoreInfo,
)
from ...security import emit_external_auth_check_failed
from ...services.shortcut import ShortcutService
from ...utils.config_helpers import get_cfg
from ..shared.store_base import StoreBase
from .amazon_auth import AmazonAuthFlow
from .amazon_install import AmazonInstaller, ProgressCallback
from .amazon_library import AmazonLibraryReader, merge_install_status
from .amazon_updates import AmazonUpdateChecker
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...core.cache_manager import CacheManager
    from ...event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
class AmazonStore(StoreBase):
    """Amazon store."""
    store_info = StoreInfo(
        name="amazon",
        display_name="Amazon Games",
        auth_method="oauth",
        icon_asset="amazon.png",
        uses_wine=False,
        supports_install=True,
    )
    CLI_TOOL = CLITool(
        name="nile",
        search_paths=["bin/nile"],
        version_flag="--version",
    )
    def __init__(
        self,
        bus: EventBus,
        cache: CacheManager,
        plugin_dir: str | None = None,
        config: ConfigManager | None = None,
        browser_monitor: OAuthBrowserMonitor | None = None,
        shortcut_service: ShortcutService | None = None,
    ) -> None:
        """Initialize the instance."""
        super().__init__(bus, cache, plugin_dir, config)
```

```

self.cli_path: str | None = self._find_binary(self.CLI_TOOL)
if not self.cli_path:
    logger.warning("[AmazonStore] nile binary not found")
self._shortcut_service = shortcut_service
amazon_cfg = (
    config.get("stores.amazon") if config else None
)
if amazon_cfg is None:
    raise KeyError(
        "config.stores.amazon is required",
    )
self._library = AmazonLibraryReader(
    config_dir=amazon_cfg["nile_config_dir"],
)
self._installer = AmazonInstaller(
    bus=bus,
    cli_path=self.cli_path,
    library=self._library,
    find_exe=self._find_exe,
    default_install_root=amazon_cfg[
        "default_install_root"
    ],
)
self._updates = AmazonUpdateChecker(
    bus=bus,
    cli_path=self.cli_path,
    library=self._library,
    list_updates_timeout=amazon_cfg[
        "list_updates_timeout_seconds"
    ],
    get_size_timeout=amazon_cfg[
        "get_size_timeout_seconds"
    ],
    default_install_root=amazon_cfg[
        "default_install_root"
    ],
)
if browser_monitor is not None:
    orchestrator = AuthOrchestrator(
        bus=bus,
        browser_monitor=browser_monitor,
        store_name="amazon",
    )
    self._auth: AmazonAuthFlow | None = (
        AmazonAuthFlow(
            bus=bus,
            orchestrator=orchestrator,
            cli_path=self.cli_path,
            success_markers=amazon_cfg[
                "nile_register_success_markers"
            ],
        )
    )
else:
    self._auth = None
    logger.debug(
        "[AmazonStore] no browser_monitor; auth "
        "disabled",
    )
async def is_available(self) -> bool:

```

```

        """Check whether available."""
        ok = self._check_nile_authenticated()
        self._cached_available = ok
        return ok
def _check_nile_authenticated(self) -> bool:
    """Check NILE authenticated."""
    if not self.cli_path:
        emit_external_auth_check_failed(
            self._bus, "amazon", "cli_not_found",
            "nile binary missing from search paths",
        )
        return False
    user_file = os.path.expanduser(
        get_cfg(
            self._config, "stores.amazon.user_file",
            "~/.config/nile/user.json",
        ),
    )
    if not os.path.isfile(user_file):
        return False
    try:
        with open(user_file, encoding="utf-8") as f:
            data = json.load(f)
    except (OSError, json.JSONDecodeError) as e:
        logger.debug(
            "[AmazonStore] user.json invalid: %s", e,
        )
        emit_external_auth_check_failed(
            self._bus, "amazon", "parse_error",
            f"{type(e).__name__}",
        )
        return False
    extensions = data.get("extensions", {})
    return "customer_info" in extensions

async def start_auth(self, **kwargs) -> AuthResult:

    """Start auth."""
    if self._auth is None:
        return AuthResult(
            success=False,
            error="auth_not_configured",
            store="amazon",
        )
    await self._ensure_auth_shortcut()
    return cast("AuthResult", await self._auth.start_auth())
async def complete_auth(
    self, code: str = "", **kwargs,
) -> AuthResult:
    """Complete auth."""
    if await self.is_available():
        return AuthResult(success=True, store="amazon")
    return AuthResult(
        success=False,
        error="not_authenticated",
        store="amazon",
    )
async def logout(self) -> Result:
    """Logout."""
    if self._auth is None:
        await self._emit(

```

```

        Events.STORE_LOGOUT, store="amazon",
    )
    return Result(success=True)
    return await self._auth.logout()
async def get_library(self) -> list[Game] | None:
    """Get library."""
    if not self.cli_path:
        return []
    try:
        owned = await self._library.read_owned_games()
        installed = await self._library.read_installed_ids()
        return merge_install_status(owned, installed)
    except Exception as e:
        logger.error(
            "[AmazonStore] get_library failed: %s", e,
        )
    return []
async def install_game(
    self,
    game_id: str,
    base_path: str | None = None,
    progress_cb: ProgressCallback | None = None,
    **kwargs: Any,
) -> InstallResult:
    """Install game."""
    return await self._installer.install_game(
        game_id, base_path, progress_cb,
    )
async def uninstall_game(
    self, game_id: str, **kwargs: Any,
) -> Result:
    """Uninstall game."""
    return await self._installer.uninstall_game(game_id)
async def update_game(
    self,
    game_id: str,
    progress_cb: ProgressCallback | None = None,
    **kwargs: Any,
) -> InstallResult:
    """Update game."""
    base_path = await self._updates.resolve_current_base_path(game_id)
    return await self._installer.install_game(
        game_id, base_path=base_path, progress_cb=progress_cb,
    )
async def check_for_updates(self) -> list[str]:
    """Check for updates."""
    return await self._updates.check_for_updates()
async def get_game_size(self, game_id: str) -> int | None:
    """Get game size."""
    return await self._updates.get_game_size(game_id)
async def get_official_url(self, game_id: str) -> str | None:
    """Get official URL."""
    return await self._library.get_official_url(game_id)

async def _ensure_auth_shortcut(self) -> None:
    """Ensure auth shortcut."""
    if self._shortcut_service is None:
        logger.debug(
            "[AmazonStore] no shortcut_service "
            "injected; skipping auth shortcut creation",

```

```
    )
    return
launcher = os.path.join(
    self._plugin_dir or "",
    "py_modules", "unifideck", "launcher",
    "dispatcher.py",
)
if not os.path.isfile(launcher):
    logger.warning(
        "[AmazonStore] launcher dispatcher not "
        "found at %s",
        launcher,
    )
    return
result = await self._shortcut_service.add_auth_shortcut(
    store="amazon",
    launcher_path=launcher,
    title="Amazon Games Sign-In",
)
if not result.success:
    logger.warning(
        "[AmazonStore] add_auth_shortcut failed: "
        "%s", result.error,
    )
```

OP-49b

amazon_auth.py

Fichier : py_modules/unifideck/stores/amazon/amazon_auth.py | Lignes : 180 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
from typing import Any, cast
from ...auth.orchestrator import AuthOrchestrator
from ...core.types import AuthResult, Events, Result, StoreAuthError
from ...event_bus.event_bus import EventBus
from ...security import audit_auth_flow
logger = logging.getLogger(__name__)
_AMAZON_REDIRECT_URIS: list[str] = [
    "https://www.amazon.com/ap/maplanding",
    "https://amazon.com/ap/maplanding",
]
class AmazonAuthFlow:
    """Amazon auth flow."""
    def __init__(
        self,
        bus: EventBus,
        orchestrator: AuthOrchestrator,
        cli_path: str | None,
        success_markers: list[str],
        cli_timeout_seconds: int = 30,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._orch = orchestrator
        self._cli_path = cli_path
        self._success_markers = tuple(success_markers)
        self._cli_timeout = cli_timeout_seconds
        self._pending_login: dict[str, Any] | None = None
    @audit_auth_flow(store="amazon", method="oauth_cli")
    async def start_auth(self) -> AuthResult:
        """Start auth."""
        if not self._cli_path:
            return AuthResult(
                success=False,
                error="nile_not_found",
                store="amazon",
            )
        self._pending_login = None
        return await self._orch.run_flow(
            get_url=self._fetch_login_url,
            allowed_uris=_AMAZON_REDIRECT_URIS,
            exchange_code=self._register_code,
            background=True,
            write_url_file=~/.local/share/unifideck/amazon_auth_url.txt",
        )

    async def logout(self) -> Result:
        """Logout."""
        if not self._cli_path:
            await self._bus.emit(
                Events.STORE_LOGOUT, store="amazon",

```

```

        )
        return Result(success=True)
    try:
        proc = await asyncio.create_subprocess_exec(
            self._cli_path, "auth", "--logout",
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        await asyncio.wait_for(
            proc.communicate(),
            timeout=self._cli_timeout,
        )
    except (TimeoutError, OSError) as e:
        logger.warning("[amazon_auth] logout: %s", e)
    self._pending_login = None
    await self._bus.emit(
        Events.STORE_LOGOUT, store="amazon",
    )
    return Result(success=True)
async def _fetch_login_url(self) -> str:
    """Fetch login URL."""
    payload = await self._run_nile_login_probe()
    url = payload.get("url", "")
    if not url:
        raise StoreAuthError(
            "nile JSON has no 'url' field",
            store="amazon",
        )
    self._pending_login = payload
    logger.info("[amazon_auth] received login URL from nile")
    return cast("str", url)
async def _run_nile_login_probe(self) -> dict:
    """Run NILE login probe."""
    if self._cli_path is None:
        raise StoreAuthError(
            "nile CLI not resolved",
            store="amazon",
        )
    proc = await asyncio.create_subprocess_exec(
        self._cli_path, "auth", "--login", "--non-interactive",
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    try:
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self._cli_timeout,
        )
    except TimeoutError as e:
        raise StoreAuthError(
            f"nile auth timed out after {self._cli_timeout}s",
            store="amazon",
        ) from e
    if proc.returncode != 0:
        err = (
            stderr.decode(errors="ignore")
            or stdout.decode(errors="ignore")
        )
        raise StoreAuthError(
            f"nile auth failed (rc={proc.returncode}): "
            f"{err[:200]}",

```



```

        store="amazon",
    )
    try:
        return cast("dict[Any, Any]", json.loads(stdout.decode(errors="ignore")))
    except json.JSONDecodeError as e:
        raise StoreAuthError(
            f"nile returned invalid JSON: {e}",
            store="amazon",
        ) from e

async def _register_code(self, code: str) -> AuthResult:
    """Register code."""
    if self._pending_login is None:
        return AuthResult(
            success=False,
            error="no_pending_login",
            store="amazon",
        )
    login = self._pending_login
    args = [
        "register",
        "--code", code,
        "--code-verifier", login.get("code_verifier", ""),
        "--serial", login.get("serial", ""),
        "--client-id", login.get("client_id", ""),
    ]
    if self._cli_path is None:
        return AuthResult(
            success=False,
            error="nile_cli_missing",
            store="amazon",
        )
    proc = await asyncio.create_subprocess_exec(
        self._cli_path, *args,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    try:
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self._cli_timeout,
        )
    except TimeoutError:
        return AuthResult(
            success=False,
            error="register_timeout",
            store="amazon",
        )
    combined = (
        stderr.decode(errors="ignore") + "\n"
        + stdout.decode(errors="ignore")
    )
    if any(m in combined for m in self._success_markers):
        self._pending_login = None
        logger.info(
            "[amazon_auth] nile register succeeded",
        )
        return AuthResult(success=True, store="amazon")
    logger.warning(
        "[amazon_auth] nile register failed: %s",

```

```
        combined[:200],
    )
    return AuthResult(
        success=False,
        error="register_failed",
        store="amazon",
    )
```

OP-49c

amazon_library.py

Fichier : py_modules/unifideck/stores/amazon/amazon_library.py | Lignes : 130 | Depend de : (rien)

```
from __future__ import annotations
import json
import logging
from pathlib import Path
from typing import Any
from ...core.io import async_file_ops as aio
from ...core.types import Game
logger = logging.getLogger(__name__)
class AmazonLibraryReader:
    """Amazon library reader."""
    def __init__(self, config_dir: str) -> None:
        """Initialize the instance."""
        config_path = Path(config_dir).expanduser()
        self._config_dir = str(config_path)
        self._library_path = str(config_path / "library.json")
        self._installed_path = str(
            config_path / "installed.json",
        )
    async def read_owned_games(self) -> list[Game]:
        """Read owned games."""
        data = await self._read_json(self._library_path)
        if not isinstance(data, list):
            return []
        games: list[Game] = []
        for item in data:
            if not isinstance(item, dict):
                continue
            product = item.get("product")
            if not isinstance(product, dict):
                continue
            game_id = product.get("id", "")
            if not game_id:
                continue
            games.append(Game(
                app_id=0,
                store="amazon",
                store_game_id=game_id,
                title=str(product.get("title") or game_id),
                installed=False,
            ))
        logger.info(
            "[amazon_library] %d owned games", len(games),
        )
        return games
    async def read_installed_ids(
        self,
    ) -> dict[str, dict[str, Any]]:
        """Read installed ids."""
        data = await self._read_json(self._installed_path)
        if not isinstance(data, list):
            return {}
        result: dict[str, dict[str, Any]] = {}
        for entry in data:
            if not isinstance(entry, dict):
                continue
```

```

        game_id = entry.get("id")
        if not game_id:
            continue
        result[game_id] = {
            "path": entry.get("path", ""),
            "version": entry.get("version", ""),
        }
    logger.info(
        "[amazon_library] %d installed games", len(result),
    )
    return result

async def get_official_url(
    self, game_id: str,
) -> str | None:

    """Get official URL."""
    data = await self._read_json(self._library_path)
    if not isinstance(data, list):
        return None
    for item in data:
        if not isinstance(item, dict):
            continue
        product = item.get("product", {})
        if product.get("id") != game_id:
            continue
        details = (
            product.get("productDetail", {}).get("details", {})
        )
        websites = details.get("websites", {})
        for key in ("OFFICIAL", "STEAM"):
            url = websites.get(key)
            if isinstance(url, str) and url:
                return url
        return None
    return None

async def _read_json(self, path: str) -> Any:
    """Read JSON."""
    try:
        if not await aio.is_file(path):
            return None
        content = await aio.read_text(path)
        if content is None:
            return None
        return json.loads(content)
    except (OSError, json.JSONDecodeError) as e:
        logger.debug(
            "[amazon_library] read %s failed: %s", path, e,
        )
        return None

def merge_install_status(
    owned: list[Game],
    installed: dict[str, dict[str, Any]],
) -> list[Game]:
    """Merge install status."""
    merged: list[Game] = []
    for game in owned:
        info = installed.get(game.store_game_id)
        if info is None:
            merged.append(game)
        continue

```

```
merged.append(Game(
    app_id=game.app_id,
    store=game.store,
    store_game_id=game.store_game_id,
    title=game.title,
    installed=True,
    install_path=info.get("path"),
    exe_path=game.exe_path,
    icon_url=game.icon_url,
    hero_url=game.hero_url,
    logo_url=game.logo_url,
    size_bytes=game.size_bytes,
    tags=list(game.tags),
    metadata=dict(game.metadata),
))
return merged
```

OP-49d

amazon_install.py

Fichier : py_modules/unifideck/stores/amazon/amazon_install.py | Lignes : 256 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import os
import re
from collections.abc import Awaitable, Callable
from typing import Any, cast
from ...core.manifest import write_manifest
from ...core.types import (
    Events,
    InstallResult,
    Result,
)
from ...event_bus.event_bus import EventBus
from ..shared.cli_install_helpers import (
    drain_install_output,
    parse_progress_line,
    wait_with_timeout,
)
from . import amazon_fuel
from .amazon_library import AmazonLibraryReader
logger = logging.getLogger(__name__)
_PROGRESS_RE = re.compile(r"[\s*(\d+)\s*%\s*\]")
ProgressCallback = Callable[[float], Awaitable[None]]
class AmazonInstaller:
    """Amazon installer."""
    def __init__(
        self,
        bus: EventBus,
        cli_path: str | None,
        library: AmazonLibraryReader,
        find_exe: Callable[[str, list[str] | None], str | None],
        default_install_root: str,
        install_timeout_seconds: int = 3600,
        uninstall_timeout_seconds: int = 120,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._cli_path = cli_path
        self._library = library
        self._find_exe = find_exe
        self._default_install_root = os.path.expanduser(default_install_root)
        self._install_timeout = install_timeout_seconds
        self._uninstall_timeout = uninstall_timeout_seconds

    async def install_game(
        self,
        game_id: str,
        base_path: str | None = None,
        progress_cb: ProgressCallback | None = None,
    ) -> InstallResult:
        """Install game."""
        if not self._cli_path:
            return InstallResult(
```

```

        success=False, error="nile_not_found",
        store="amazon", game_id=game_id,
    )
    base = base_path or self._default_install_root
    try:
        os.makedirs(base, exist_ok=True)
    except OSError as e:
        return InstallResult(
            success=False,
            error=f"mkdir_failed: {e}",
            store="amazon", game_id=game_id,
        )
    await self._bus.emit(
        Events.DOWNLOAD_STARTED,
        store="amazon", game_id=game_id,
    )
    rc = await self._run_install(base, game_id, progress_cb)
    if rc != 0:
        await self._bus.emit(
            Events.DOWNLOAD_FAILED,
            store="amazon", game_id=game_id,
            error=f"nile_exit_{rc}",
        )
        return InstallResult(
            success=False,
            error=f"nile_exit_{rc}",
            store="amazon", game_id=game_id,
        )
    return await self._finalize_install(game_id, base)
async def _finalize_install(
    self, game_id: str, base: str,
) -> InstallResult:
    """Finalize install."""
    install_path = await self._resolve_install_path(
        game_id, base,
    )
    exe = await self._resolve_executable(install_path, game_id)
    title = await self._resolve_title(game_id)
    if install_path:
        exe_relative = ""
        if exe:
            try:
                exe_relative = os.path.relpath(
                    exe, install_path,
                )
            except ValueError:
                pass
        await write_manifest(
            install_dir=install_path,
            store="amazon",
            store_id=game_id,
            title=title,
            executable_relative=exe_relative,
            platform="windows",
        )
    await self._bus.emit(
        Events.DOWNLOAD_COMPLETE,
        store="amazon", game_id=game_id,
        install_path=install_path,
    )
    return InstallResult(

```

```

        success=True,
        store="amazon",
        game_id=game_id,
        install_path=install_path or base,
    )

    async def _run_install(
        self,
        base: str,
        game_id: str,
        progress_cb: ProgressCallback | None,
    ) -> int:

        """Run install."""
        cmd = self._build_install_cmd(base, game_id)
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.STDOUT,
        )
        drain_exc: BaseException | None = None
        try:
            await self._drain_install_output(
                proc, game_id, progress_cb,
            )
        except BaseException as e:
            drain_exc = e
        rc = await self._wait_with_timeout(proc)
        if drain_exc is not None:
            raise drain_exc
        return rc

    def _build_install_cmd(
        self, base: str, game_id: str,
    ) -> list[str]:
        """Build install cmd."""
        if self._cli_path is None:
            raise RuntimeError(
                "nile CLI path is not set; cannot build install cmd",
            )
        return [
            self._cli_path, "install", game_id,
            "--base-path", base,
        ]

    async def _drain_install_output(
        self,
        proc: Any,
        game_id: str,
        progress_cb: ProgressCallback | None,
    ) -> None:
        """Drain install output."""
        await drain_install_output(
            proc, game_id, progress_cb,
            self._handle_install_line,
        )

    async def _wait_with_timeout(self, proc: Any) -> int:
        """Wait with timeout."""
        return await wait_with_timeout(
            proc, self._install_timeout, "[amazon_install]",
        )

    async def _handle_install_line(
        self,

```



```

line: str,
game_id: str,
progress_cb: ProgressCallback | None,
) -> None:
    """Handle install line."""
    pct = parse_progress_line(line, _PROGRESS_RE)
    if pct is None:
        logger.debug("[nile install] %s", line)
        return
    if progress_cb is not None:
        try:
            await progress_cb(pct)
        except Exception as e:
            logger.debug(
                "[amazon_install] progress_cb raised: %s",
                e,
            )
    await self._bus.emit(
        Events.DOWNLOAD_PROGRESS,
        store="amazon", game_id=game_id,
        progress=pct,
    )

async def _resolve_install_path(
    self, game_id: str, base: str,
) -> str | None:
    """Resolve install path."""
    installed = await self._library.read_installed_ids()
    info = installed.get(game_id)
    if info and info.get("path"):
        return cast("str | None", info["path"])
    default = os.path.join(base, game_id)
    if os.path.isdir(default):
        return default
    return None

async def _resolve_executable(
    self, install_path: str | None, game_id: str,
) -> str | None:
    """Resolve executable."""
    if not install_path:
        return None
    from_fuel = amazon_fuel.find_exe_from_fuel(install_path)
    if from_fuel:
        return from_fuel
    return self._find_exe(install_path, [game_id])

async def _resolve_title(self, game_id: str) -> str:
    """Resolve title."""
    owned = await self._library.read_owned_games()
    for game in owned:
        if game.store_game_id == game_id:
            return game.title
    return game_id

async def uninstall_game(self, game_id: str) -> Result:
    """Uninstall game."""
    if not self._cli_path:
        return Result(
            success=False, error="nile_not_found",
        )
    proc = await asyncio.create_subprocess_exec(
        self._cli_path, "uninstall", game_id, "--yes",
    )

```

```
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    try:
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self._uninstall_timeout,
        )
    except TimeoutError:
        proc.kill()
        return Result(
            success=False, error="uninstall_timeout",
        )
    if proc.returncode != 0:
        err = stderr.decode(errors="ignore")[:200]
        return Result(
            success=False,
            error=f"uninstall_failed: {err}",
        )
    await self._bus.emit(
        Events.GAME_UNINSTALLED,
        store="amazon", game_id=game_id,
    )
    return Result(success=True)
```

OP-49e

amazon_updates.py

Fichier : py_modules/unifideck/stores/amazon/amazon_updates.py | Lignes : 108 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
from pathlib import Path
from ...event_bus.event_bus import EventBus
from .amazon_library import AmazonLibraryReader
logger = logging.getLogger(__name__)
class AmazonUpdateChecker:
    """Amazon update checker."""
    def __init__(
        self,
        bus: EventBus,
        cli_path: str | None,
        library: AmazonLibraryReader,
        list_updates_timeout: int,
        get_size_timeout: int,
        default_install_root: str,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._cli_path = cli_path
        self._library = library
        self._list_updates_timeout = list_updates_timeout
        self._get_size_timeout = get_size_timeout
        self._default_install_root = str(
            Path(default_install_root).expanduser(),
        )
    async def check_for_updates(self) -> list[str]:
        """Check for updates."""
        if not self._cli_path:
            return []
        try:
            proc = await asyncio.create_subprocess_exec(
                self._cli_path, "list-updates", "--json",
                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE,
            )
            stdout, _ = await asyncio.wait_for(
                proc.communicate(),
                timeout=self._list_updates_timeout,
            )
        except (TimeoutError, OSError) as e:
            logger.warning(
                "[amazon_updates] list-updates failed: %s",
                e,
            )
            return []
        if proc.returncode != 0:
            return []
        try:
            raw = (
                stdout.decode(errors="ignore").strip()
                or "[]"
            )

```

```

        data = json.loads(raw)
    except json.JSONDecodeError:
        return []
    if not isinstance(data, list):
        return []
    return [x for x in data if isinstance(x, str)]

async def get_game_size(
    self, game_id: str,
) -> int | None:

    """Get game size."""
    if not self._cli_path:
        return None
    try:
        proc = await asyncio.create_subprocess_exec(
            self._cli_path, "install", game_id,
            "--info", "--json",
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, _ = await asyncio.wait_for(
            proc.communicate(),
            timeout=self._get_size_timeout,
        )
    except (TimeoutError, OSError):
        return None
    if proc.returncode != 0:
        return None
    for raw_line in stdout.decode(
        errors="ignore",
    ).splitlines():
        line = raw_line.strip()
        if not line.startswith("{"):
            continue
        try:
            info = json.loads(line)
        except json.JSONDecodeError:
            continue
        size = info.get("download_size")
        if isinstance(size, int):
            return size
    return None

async def resolve_current_base_path(
    self, game_id: str,
) -> str:

    """Resolve current base path."""
    installed = await self._library.read_installed_ids()
    info = installed.get(game_id)
    if info and info.get("path"):
        return (
            str(Path(info["path"]).parent)
            or self._default_install_root
        )
    return self._default_install_root

```

OP-49f

amazon_fuel.py

Fichier : py_modules/unifideck/stores/amazon/amazon_fuel.py | Lignes : 92 | Depend de : (rien)

```
from __future__ import annotations
import json
import logging
import re
from pathlib import Path
logger = logging.getLogger(__name__)
_COMMENT_RE = re.compile(r"//.*$", re.MULTILINE)
def candidate_fuel_dirs(install_path: str) -> list[str]:
    """Check whether fuel dirs."""
    if not install_path:
        return []
    install_p = Path(install_path)
    dirs: list[str] = [
        install_path,
        str(install_p / "game"),
        str(install_p / "Game"),
    ]
    try:
        for entry in install_p.iterdir():
            subdir = str(entry)
            if entry.is_dir() and subdir not in dirs:
                dirs.append(subdir)
    except OSError as e:
        logger.debug(
            "[amazon_fuel] listdir(%s) failed: %s",
            install_path, e,
        )
    return dirs
def parse_fuel_json_content(content: str) -> dict | None:
    """Parse fuel JSON content."""
    cleaned = _COMMENT_RE.sub("", content)
    try:
        data = json.loads(cleaned)
    except json.JSONDecodeError as e:
        logger.debug("[amazon_fuel] parse error: %s", e)
        return None
    if not isinstance(data, dict):
        logger.debug(
            "[amazon_fuel] expected object, got %s",
            type(data).__name__,
        )
        return None
    return data
def extract_main_command(fuel_data: dict) -> str | None:
    """Extract main command."""
    main = fuel_data.get("Main")
    if not isinstance(main, dict):
        return None
    command = main.get("Command")
    if not isinstance(command, str) or not command.strip():
        return None
    return command.strip()
def find_exe_from_fuel(install_path: str) -> str | None:
```

```
"""Find exe from fuel."""
if not install_path:
    return None
for search_dir in candidate_fuel_dirs(install_path):
    fuel_path = Path(search_dir) / "fuel.json"
    if not fuel_path.is_file():
        continue
    try:
        content = fuel_path.read_text(
            encoding="utf-8", errors="replace",
        )
    except OSError as e:
        logger.debug(
            "[amazon_fuel] read %s failed: %s",
            fuel_path, e,
        )
        continue
    data = parse_fuel_json_content(content)
    if data is None:
        continue
    command = extract_main_command(data)
    if command is None:
        logger.debug(
            "[amazon_fuel] %s has no Main.Command",
            fuel_path,
        )
        continue
    exe_path = Path(search_dir) / command
    if exe_path.is_file():
        logger.info(
            "[amazon_fuel] resolved exe from "
            "fuel.json: %s", exe_path,
        )
        return str(exe_path)
    logger.debug(
        "[amazon_fuel] Main.Command points to "
        "missing file: %s", exe_path,
    )
return None
```

OP-50

__init__.py

Fichier : py_modules/unifideck/stores/gog/__init__.py | Lignes : 2 | Depend de : (rien)

```
from .store import GOGStore
__all__ = ["GOGStore"]
```

OP-51

`__init__.py`

Fichier : `py_modules/unifideck/stores/gog/install/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from .installer import GOGInstaller
__all__ = ["GOGInstaller"]
```


OP-51a

installer.py

Fichier : py_modules/unifideck/stores/gog/install/installer.py | Lignes : 378 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
import shutil
from collections.abc import Awaitable, Callable
from dataclasses import dataclass, field
from typing import Any, cast
from ...core.types import InstallResult, Result
from ..config import GOGConfig
from ..tokens import GOGTokenManager
from .helpers import _InstallHelpers
from .marker import _PostInstallMarker
from .planner import GOGInstallPlanner
from .progress import _GogdlProgressMonitor
from .uninstall_pipeline import _UninstallPipeline
logger = logging.getLogger(__name__)
@dataclass
class _InstallContext:
    """Install context."""
    game_id: str
    base_path: str
    preferred_lang: str
    explicit_lang: bool
    progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None
    platform: str = ""
    folder_name: str | None = None
    supported_langs: list[str] = field(default_factory=list)
    existing_dirs: set = field(default_factory=set)
    support_dir: str = ""
    install_mode: str = ""
    found_path: str = ""
class GOGInstaller:
    """Goginstaller."""
    def __init__(
        self,
        config: GOGConfig,
        tokens: GOGTokenManager,
        gogdl_bin: str,
        exe_finder: Callable[[str], str | None],
        locale_fn: Callable[[], str],
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._tokens = tokens
        self._gogdl_bin = gogdl_bin
        self._find_exe = exe_finder
        self._locale_fn = locale_fn
        self._planner = GOGInstallPlanner(config, tokens)
        self._planner.set_gogdl_bin(gogdl_bin)
        self._uninstall_pipeline = _UninstallPipeline(self)
        self._progress_monitor = _GogdlProgressMonitor(self)
        self._marker = _PostInstallMarker(self)
        self._helpers = _InstallHelpers(self)
    async def uninstall_game(

```

```

self,
game_id: str,
install_path: str | None = None,
) -> Result:
    """Uninstall game."""
    return await self._uninstall_pipeline.uninstall_game(
        game_id, install_path,
    )

async def _run_gogdl_with_progress(
self,
install_mode: str,
game_id: str,
platform: str,
path: str,
support_dir: str,
languages: list[str],
progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,
) -> bool:
    """Run GOGDL with progress."""
    return await self._progress_monitor.run_gogdl_with_progress(
        install_mode, game_id, platform, path,
        support_dir, languages, progress_cb,
    )

async def _run_gogdl_repair_pass(
self,
game_id: str,
platform: str,
base_path: str,
folder_name: str | None,
preferred_lang: str,
) -> None:
    """Run GOGDL repair pass."""
    await self._progress_monitor.run_gogdl_repair_pass(
        game_id, platform, base_path,
        folder_name, preferred_lang,
    )

def _snapshot_dirs(self, base_path: str) -> set:
    """Snapshot dirs."""
    return self._marker.snapshot_dirs(base_path)

async def _locate_install(
self,
game_id: str,
base_path: str,
folder_name: str | None,
existing_dirs: set,
) -> str | None:
    """Locate install."""
    return await self._marker.locate_install(
        game_id, base_path, folder_name, existing_dirs,
    )

async def _write_install_marker(
self,
install_path: str,
game_id: str,
language: str,
) -> bool:
    """Write install marker."""
    return await self._marker.write_install_marker(
        install_path, game_id, language,
    )

```

```

    )
    async def _regenerate_manifest(
        self, game_id: str, platform: str,
    ) -> None:
        """Regenerate manifest."""
        await self._marker.regenerate_manifest(game_id, platform)
    def _install_failed(
        self,
        game_id: str,
        error: str,
        *,
        cleanup_path: str | None = None,
        cleanup_folder: str | None = None,
    ) -> InstallResult:
        """Install failed."""
        if cleanup_path is not None:
            self._cleanup_partial(cleanup_path, cleanup_folder)
        return InstallResult(
            success=False,
            error=error,
            store="gog",
            game_id=game_id,
        )

    async def install_game(
        self,
        game_id: str,
        base_path: str | None = None,
        progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None = None,
        language: str | None = None,
    ) -> InstallResult:
        """Install game."""
        ctx, failure = self._install_preflight(
            game_id, base_path, progress_cb, language,
        )
        if failure is not None:
            return cast("InstallResult", failure)
        auth_failure = (
            await self._install_probe_and_prepare(ctx)
        )
        if auth_failure is not None:
            return auth_failure
        download_failure = (
            await self._install_run_gogdl_phase(ctx)
        )
        if download_failure is not None:
            return download_failure
        return await self._install_finalize(ctx)
    def _install_preflight(
        self,
        game_id: str,
        base_path: str | None,
        progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,
        language: str | None,
    ) -> tuple:
        """Install preflight."""
        if not os.path.isfile(self._gogdl_bin):
            return None, self._install_failed(
                game_id, "gogdl_not_found",
            )

```

```

        resolved_base = base_path or os.path.expanduser(
            self._config.download_dir,
        )
        os.makedirs(resolved_base, exist_ok=True)
        preferred_lang = (
            language or self._locale_fn() or "en-US"
        )
        explicit_lang = language is not None
        logger.info(
            "[GOGInstaller] start: game=%s path=%s lang=%s "
            "(explicit=%s)",
            game_id, resolved_base,
            preferred_lang, explicit_lang,
        )
        ctx = _InstallContext(
            game_id=game_id,
            base_path=resolved_base,
            preferred_lang=preferred_lang,
            explicit_lang=explicit_lang,
            progress_cb=progress_cb,
        )
        return ctx, None

    async def _install_probe_and_prepare(
        self, ctx: _InstallContext,
    ) -> InstallResult | None:
        """Install probe and prepare."""
        if not self._tokens.has_tokens:
            await self._tokens.load()
        if not await self._tokens.refresh_if_stale():
            return self._install_failed(
                ctx.game_id, "not_authenticated",
            )
        await self._wipe_manifests(ctx.game_id)
        await self._wipe_support_cache(ctx.game_id)
        ctx.platform, ctx.folder_name, ctx.supported_langs = (
            await self._helpers.probe_game_info(ctx.game_id)
        )
        ctx.existing_dirs = self._snapshot_dirs(ctx.base_path)
        ctx.support_dir = os.path.join(
            os.path.expanduser(self._config.gogdl_config_dir),
            "gog-support", ctx.game_id,
        )
        os.makedirs(ctx.support_dir, exist_ok=True)
        target_folder = (
            os.path.join(ctx.base_path, ctx.folder_name)
            if ctx.folder_name else None
        )
        ctx.install_mode = (
            await self._planner.determine_install_mode(
                ctx.game_id, target_folder,
            )
        )
        await self._wipe_manifests(ctx.game_id)
        return None

    async def _install_run_gogdl_phase(
        self, ctx: _InstallContext,
    ) -> InstallResult | None:
        """Install run GOGDL phase."""
        gogdl_path = (

```

```

        ctx.base_path
        if ctx.install_mode == "download"
        else (
            os.path.join(ctx.base_path, ctx.folder_name)
            if ctx.folder_name else ctx.base_path
        )
    )
    languages = self._helpers.pick_languages(
        ctx.preferred_lang,
        ctx.explicit_lang,
        ctx.supported_langs,
    )
    download_ok = await self._run_gogdl_with_progress(
        install_mode=ctx.install_mode,
        game_id=ctx.game_id,
        platform=ctx.platform,
        path=gogdl_path,
        support_dir=ctx.support_dir,
        languages=languages,
        progress_cb=ctx.progress_cb,
    )
    if not download_ok:
        return self._install_failed(
            ctx.game_id, "download_failed",
            cleanup_path=ctx.base_path,
            cleanup_folder=ctx.folder_name,
        )
    await self._run_gogdl_repair_pass(
        ctx.game_id, ctx.platform, ctx.base_path,
        ctx.folder_name, ctx.preferred_lang,
    )
    return None

async def _install_finalize(
    self, ctx: _InstallContext,
) -> InstallResult:

    """Install finalize."""
    found_path = await self._locate_install(
        ctx.game_id, ctx.base_path,
        ctx.folder_name, ctx.existing_dirs,
    )
    if not found_path:
        return self._install_failed(
            ctx.game_id, "install_not_located",
            cleanup_path=ctx.base_path,
            cleanup_folder=ctx.folder_name,
        )
    marker_ok = await self._write_install_marker(
        found_path, ctx.game_id, ctx.preferred_lang,
    )
    if not marker_ok:
        return self._install_failed(
            ctx.game_id, "marker_write_failed",
            cleanup_path=ctx.base_path,
            cleanup_folder=ctx.folder_name,
        )
    verification = await self._planner.verify_installation(
        ctx.game_id, found_path,
        ctx.platform, self._find_exe,
    )

```

```

    if not verification.get("complete"):
        logger.warning(
            "[GOGInstaller] verification issue: %s",
            verification.get("issue", "unknown"),
        )
    await self._regenerate_manifest(
        ctx.game_id, ctx.platform,
    )
    logger.info(
        "[GOGInstaller] install complete: %s", found_path,
    )
    return InstallResult(
        success=True,
        store="gog",
        game_id=ctx.game_id,
        install_path=found_path,
    )
async def _wipe_manifests(self, game_id: str) -> None:
    """Wipe manifests."""
    def _sync() -> None:
        """Sync."""
        base = os.path.expanduser(
            self._config.gogdl_config_dir,
        )
        parent = os.path.dirname(base)
        locations = [
            os.path.join(
                base, "heroic_gogdl", "manifests", game_id,
            ),
            os.path.join(
                parent, "heroic_gogdl", "manifests", game_id,
            ),
            os.path.join(base, "manifests", game_id),
            os.path.join(
                parent, "gogdl", "manifests", game_id,
            ),
        ]
        for path in locations:
            if os.path.isfile(path):
                try:
                    os.remove(path)
                    logger.info(
                        "[GOGInstaller] cleared manifest: %s",
                        path,
                    )
                except OSError as e:
                    logger.warning(
                        "[GOGInstaller] could not clear "
                        "manifest: %s", e,
                    )
    await asyncio.to_thread(_sync)

async def _wipe_support_cache(self, game_id: str) -> None:
    """Wipe support cache."""
    def _sync() -> None:
        """Sync."""
        support_dir = os.path.join(
            os.path.expanduser(
                self._config.gogdl_config_dir,
            ),

```

```
        "gog-support",
        game_id,
    )
    if os.path.isdir(support_dir):
        try:
            shutil.rmtree(support_dir)
            logger.info(
                "[GOGInstaller] cleared support cache",
            )
        except OSError as e:
            logger.warning(
                "[GOGInstaller] support cleanup: %s", e,
            )
    await asyncio.to_thread(_sync)
def _cleanup_partial(
    self,
    base_path: str,
    folder_name: str | None,
) -> None:
    """Cleanup partial."""
    if not folder_name:
        return
    partial = os.path.join(base_path, folder_name)
    if os.path.exists(partial):
        logger.info(
            "[GOGInstaller] cleanup partial: %s", partial,
        )
        shutil.rmtree(partial, ignore_errors=True)
```

OP-51b

primitives.py

Fichier : py_modules/unifideck/stores/gog/install/primitives.py | Lignes : 81 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import os
logger = logging.getLogger(__name__)
class GOGFolderOps:
    """Gogfolder ops."""
    @staticmethod
    def folder_size(path: str) -> int:
        """Folder size."""
        total = 0
        try:
            for root, _dirs, files in os.walk(path):
                for name in files:
                    try:
                        total += os.path.getsize(
                            os.path.join(root, name),
                        )
                    except OSError:
                        continue
        except OSError:
            pass
        return total
    @staticmethod
    def count_files(path: str) -> int:
        """Count files."""
        count = 0
        try:
            for _root, _dirs, files in os.walk(path):
                count += len(files)
        except OSError:
            pass
        return count
    @staticmethod
    def has_goggame_info(path: str, game_id: str = "") -> bool:
        """Check whether goggame info."""
        try:
            for name in os.listdir(path):
                if not name.startswith("goggame-"):
                    continue
                if not name.endswith(".info"):
                    continue
                if not game_id:
                    return True
                if name == f"goggame-{game_id}.info":
                    return True
        except OSError:
            pass
        return False
    @staticmethod
    async def force_cleanup_folder(path: str) -> None:
        """Force cleanup folder."""
        def _sync_cleanup() -> None:
            """Sync cleanup."""
```



```
deleted = 0
errors = 0
for root, dirs, files in os.walk(path, topdown=False):
    for name in files:
        try:
            os.remove(os.path.join(root, name))
            deleted += 1
        except OSError as e:
            logger.debug(
                "[GOGFolderOps] could not remove "
                "%s: %s", name, e,
            )
            errors += 1
    for name in dirs:
        try:
            os.rmdir(os.path.join(root, name))
        except OSError:
            pass
    try:
        os.rmdir(path)
    except OSError:
        pass
    logger.info(
        "[GOGFolderOps] force cleanup: %d deleted, "
        "%d errors", deleted, errors,
    )
await asyncio.to_thread(_sync_cleanup)
```

OP-51c

languages.py

Fichier : py_modules/unifideck/stores/gog/install/languages.py | Lignes : 15 | Depend de : (rien)

```
from __future__ import annotations
def smart_match_language(
    target: str, choices: list[str],
) -> str | None:
    """Smart match language."""
    if not target or not choices:
        return None
    if target in choices:
        return target
    target_base = target.split("-", maxsplit=1)[0].lower()
    for choice in choices:
        choice_base = choice.split("-")[0].lower()
        if target_base == choice_base:
            return choice
    return None
```

OP-51d

planner.py

Fichier : py_modules/unifideck/stores/gog/install/planner.py | Lignes : 323 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
import shutil
from collections.abc import Callable
from pathlib import Path
from typing import Any
from ..config import GOGConfig
from ..tokens import GOGTokenManager
from ..primitives import GOGFolderOps
logger = logging.getLogger(__name__)
_CORRUPT_INSTALL_SIZE_THRESHOLD = 100 * 1024 * 1024
_MIN_SIZE_RATIO = 0.8
def _extract_disk_size_from_size_info(
    size_info: dict,
) -> int | None:
    """Extract disk size from size info."""
    for lang_key in ("en-US", "en", ""):
        if lang_key in size_info:
            return int(
                size_info[lang_key].get("disk_size", 0) or 0,
            )
    if size_info:
        first = next(iter(size_info))
        return int(
            size_info[first].get("disk_size", 0) or 0,
        )
    return None
class GOGInstallPlanner:
    """Goginstall planner."""
    def __init__(
        self,
        config: GOGConfig,
        tokens: GOGTokenManager,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._tokens = tokens

    async def determine_install_mode(
        self,
        game_id: str,
        target_folder: str | None,
    ) -> str:
        """Determine install mode."""
        if not target_folder or not Path(target_folder).exists():
            logger.info(
                "[GOGInstallPlanner] folder missing → download",
            )
            return "download"
        folder_size = GOGFolderOps.folder_size(target_folder)
        file_count = GOGFolderOps.count_files(target_folder)
        has_info = GOGFolderOps.has_goggame_info(

```

```

        target_folder, game_id,
    )
    logger.info(
        "[GOGInstallPlanner] folder state: size=%.1fMB, "
        "files=%d, has_info=%s",
        folder_size / (1024 * 1024),
        file_count, has_info,
    )
    if has_info:
        if folder_size < _CORRUPT_INSTALL_SIZE_THRESHOLD:
            logger.warning(
                "[GOGInstallPlanner] corrupt install "
                "(has info but only %.1fMB) → cleanup "
                "+ download",
                folder_size / (1024 * 1024),
            )
            await self._cleanup_corrupt_install(
                game_id, target_folder,
            )
            return "download"
        logger.info(
            "[GOGInstallPlanner] valid existing install "
            "→ repair",
        )
        return "repair"
    if (
        folder_size > _CORRUPT_INSTALL_SIZE_THRESHOLD
        or file_count > 0
    ):
        logger.warning(
            "[GOGInstallPlanner] orphaned data (no info, "
            "%.1fMB) → cleanup + download",
            folder_size / (1024 * 1024),
        )
        await self._cleanup_orphaned_install(
            game_id, target_folder,
        )
        return "download"
    async def verify_installation(
        self,
        game_id: str,
        install_path: str,
        platform: str,
        exe_finder: Callable[[str], str | None],
    ) -> dict[str, Any]:
        """Verify installation."""
        try:
            expected = await self.get_expected_disk_size(
                game_id, platform,
            )
            actual = GOGFolderOps.folder_size(install_path)
            files = GOGFolderOps.count_files(install_path)
            has_info = GOGFolderOps.has_goggame_info(
                install_path,
            )
            has_exe = exe_finder(install_path) is not None
            size_ratio = (
                (actual / expected) if expected > 0 else 1.0
            )
            logger.info(
                "[GOGInstallPlanner] verify: size=%.1fMB "

```

```

        "(%.0f%% of expected), files=%d, "
        "has_info=%s, has_exe=%s",
        actual / (1024 * 1024), size_ratio * 100,
        files, has_info, has_exe,
    )
    if expected > 0 and size_ratio < _MIN_SIZE_RATIO:
        return {
            "complete": False,

            "issue": (
                f"Installation may be incomplete: "
                f"only {size_ratio * 100:.0f}% of "
                f"expected size"
            ),
            "actual_size": actual,
            "expected_size": expected,
            "has_info": has_info,
            "has_exe": has_exe,
        }
    if not has_info:
        return {
            "complete": False,
            "issue": "Missing goggame.info file",
            "actual_size": actual,
            "actual_files": files,
            "has_exe": has_exe,
        }
    if not has_exe:
        return {
            "complete": False,
            "issue": "Could not find game executable",
            "actual_size": actual,
            "actual_files": files,
            "has_info": has_info,
        }
    return {
        "complete": True,
        "actual_size": actual,
        "expected_size": expected,
        "actual_files": files,
        "size_ratio": size_ratio,
        "has_info": has_info,
        "has_exe": has_exe,
    }
except Exception as e:
    logger.error(
        "[GOGInstallPlanner] verify error: %s", e,
    )
    return {
        "complete": False,
        "issue": f"Verification failed: {e}",
    }
}

async def get_expected_disk_size(
    self, game_id: str, platform: str,
) -> int:
    """Get expected disk size."""
    gogdl_bin = self._resolve_gogdl_bin()
    if not gogdl_bin:
        return 0
    stdout = await self._spawn_gogdl_info(
        gogdl_bin, game_id, platform,

```

```

    )
    if stdout is None:
        return 0
    return self._parse_size_from_gogdl_info(stdout)

async def _spawn_gogdl_info(
    self, gogdl_bin: str, game_id: str, platform: str,
) -> bytes | None:

    """Spawn GOGDL info."""
    cmd = [
        gogdl_bin,
        "--auth-config-path",
        self._config.auth_config_path,
        "info", "--platform", platform, game_id,
    ]
    try:
        env, _gogdl_cleanup = (
            await self._tokens.acquire_gogdl_creds()
        )
        try:
            proc = await asyncio.create_subprocess_exec(
                *cmd,
                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE,
                env=env,
            )
            stdout, _stderr = await asyncio.wait_for(
                proc.communicate(), timeout=30,
            )
            return stdout
        finally:
            await _gogdl_cleanup()
    except (TimeoutError, OSError) as e:
        logger.warning(
            "[GOGInstallPlanner] gogdl info failed: %s",
            e,
        )
    return None

@staticmethod
def _parse_size_from_gogdl_info(stdout: bytes) -> int:
    """Parse size from GOGDL info."""
    for raw_line in stdout.decode(
        errors="replace",
    ).splitlines():
        line = raw_line.strip()
        if not line:
            continue
        try:
            data = json.loads(line)
        except json.JSONDecodeError:
            continue
        size_info = data.get("size")
        if not isinstance(size_info, dict):
            continue
        extracted = _extract_disk_size_from_size_info(
            size_info,
        )
        if extracted is not None:
            return extracted
    return 0

```

```

async def _cleanup_corrupt_install(
    self, game_id: str, target_folder: str,
) -> None:

    """Cleanup corrupt install."""
    def _sync() -> None:
        """Sync."""
        try:
            shutil.rmtree(target_folder)
            logger.info(
                "[GOGInstallPlanner] removed %s",
                target_folder,
            )
        except OSError as e:
            logger.error(
                "[GOGInstallPlanner] corrupt cleanup "
                "failed for %s: %s", target_folder, e,
            )
        support_dir = (
            Path(self._config.gogdl_config_dir).expanduser()
            / "gog-support" / game_id
        )
        if support_dir.is_dir():
            try:
                shutil.rmtree(support_dir)
                logger.info(
                    "[GOGInstallPlanner] cleared "
                    "support cache: %s", support_dir,
                )
            except OSError as e:
                logger.warning(
                    "[GOGInstallPlanner] could not "
                    "clear support dir: %s", e,
                )
        await asyncio.to_thread(_sync)
    async def _cleanup_orphaned_install(
        self, game_id: str, target_folder: str,
    ) -> None:
        """Cleanup orphaned install."""
        def _sync() -> None:
            """Sync."""
            try:
                shutil.rmtree(target_folder)
                logger.info(
                    "[GOGInstallPlanner] removed orphan %s",
                    target_folder,
                )
            except OSError as e:
                logger.error(
                    "[GOGInstallPlanner] orphan cleanup "
                    "failed: %s", e,
                )
        for manifest_path in self._manifest_locations(
            game_id,
        ):
            mp = Path(manifest_path)
            if mp.is_file():
                try:
                    mp.unlink()
                    logger.info(

```

```

        "[GOGInstallPlanner] cleaned "
        "stale manifest: %s",
        manifest_path,
    )
except OSError as e:
    logger.warning(
        "[GOGInstallPlanner] could not "
        "clean manifest: %s", e,
    )
await asyncio.to_thread(_sync)
def _manifest_locations(self, game_id: str) -> list[str]:
    """Manifest locations."""
    base = Path(
        self._config.gogdl_config_dir,
    ).expanduser()
    parent = base.parent
    return [
        str(base / "heroic_gogdl" / "manifests" / game_id),
        str(
            parent / "heroic_gogdl" / "manifests"
            / game_id,
        ),
        str(base / "manifests" / game_id),
        str(parent / "gogdl" / "manifests" / game_id),
    ]

def _resolve_gogdl_bin(self) -> str | None:
    """Resolve GOGDL bin."""
    return getattr(self, "_gogdl_bin_override", None)
def set_gogdl_bin(self, path: str) -> None:
    """Set GOGDL bin."""
    self._gogdl_bin_override = path

```


OP-51e

uninstall_pipeline.py

Fichier : py_modules/unifideck/stores/gog/install/uninstall_pipeline.py | Lignes : 73 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import os
import shutil
from typing import TYPE_CHECKING
from ...core.types import Result
from .primitives import GOGFolderOps
if TYPE_CHECKING:
    from .installer import GOGInstaller
logger = logging.getLogger(__name__)
_UNINSTALL_MAX_ATTEMPTS = 3
class _UninstallPipeline:
    """Uninstall pipeline."""
    def __init__(self, parent: GOGInstaller) -> None:
        """Initialize the instance."""
        self._parent = parent

    async def uninstall_game(
        self,
        game_id: str,
        install_path: str | None = None,
    ) -> Result:
        """Uninstall game."""
        if not install_path or not os.path.exists(install_path):
            logger.info(
                "[GOGInstaller] %s already gone, nothing to do",
                game_id,
            )
            return Result(success=True)
        for attempt in range(_UNINSTALL_MAX_ATTEMPTS):
            try:
                await asyncio.to_thread(
                    shutil.rmtree, install_path,
                )
            except PermissionError as e:
                logger.warning(
                    "[GOGInstaller] attempt %d permission: %s",
                    attempt + 1, e,
                )
            except OSError as e:
                logger.warning(
                    "[GOGInstaller] attempt %d failed: %s",
                    attempt + 1, e,
                )
            if not os.path.exists(install_path):
                logger.info(
                    "[GOGInstaller] uninstalled %s", install_path,
                )
                break
            remaining = GOGFolderOps.count_files(install_path)
            logger.warning(
                "[GOGInstaller] attempt %d: %d files remain",
                attempt + 1, remaining,
```

```
)
if attempt == _UNINSTALL_MAX_ATTEMPTS - 1:
    logger.info(
        "[GOGInstaller] falling back to force cleanup",
    )
    await GOGFolderOps.force_cleanup_folder(
        install_path,
    )
await self._parent._wipe_support_cache(game_id)
await self._parent._wipe_manifests(game_id)
if os.path.exists(install_path):
    remaining = GOGFolderOps.count_files(install_path)
    if remaining > 0:
        return Result(
            success=False,
            error=(
                f"uninstall_incomplete_{remaining}_remaining"
            ),
        )
return Result(success=True)
```

OP-51f

progress.py

Fichier : py_modules/unifideck/stores/gog/install/progress.py | Lignes : 316 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
from collections.abc import Awaitable, Callable
from typing import (
    TYPE_CHECKING,
    Any,
)
from .primitives import GOGFolderOps
if TYPE_CHECKING:
    from .installer import GOGInstaller
logger = logging.getLogger(__name__)
_GOGDL_STALL_TIMEOUT_S = 120.0
class _GogdlProgressMonitor:
    """Gogdl progress monitor."""
    def __init__(self, parent: GOGInstaller) -> None:
        """Initialize the instance."""
        self._parent = parent
    async def run_gogdl_with_progress(
        self,
        install_mode: str,
        game_id: str,
        platform: str,
        path: str,
        support_dir: str,
        languages: list[str],
        progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,
    ) -> bool:
        """Run GOGDL with progress."""
        cmd = self._build_gogdl_cmd(
            install_mode, game_id, platform,
            path, support_dir, languages,
        )
        proc = await self._spawn_gogdl(cmd)
        loop_ok = await self._read_progress_loop(proc, progress_cb)
        if not loop_ok:
            return False
        await proc.wait()
        if proc.returncode != 0:
            logger.error(
                "[GOGInstaller] gogdl exited with code %d",
                proc.returncode,
            )
            return False
        return True

    def _build_gogdl_cmd(
        self,
        install_mode: str,
        game_id: str,
        platform: str,
        path: str,
        support_dir: str,
        languages: list[str],
    )

```

```

) -> list[str]:
    """Build GOGDL cmd."""
    cmd = [
        self._parent._gogdl_bin,
        "--auth-config-path",
        self._parent._config.auth_config_path,
        install_mode,
        game_id,
        "--platform", platform,
        "--path", path,
        "--support", support_dir,
        "--with-dlcs",
    ]
    for lang in languages:
        cmd.extend(["--lang", lang])
    return cmd
async def _spawn_gogdl(
    self, cmd: list[str],
) -> asyncio.subprocess.Process:
    """Spawn GOGDL."""
    logger.info(
        "[GOGInstaller] spawning gogdl: %s",
        " ".join(cmd),
    )
    env, cleanup = await self._parent._tokens.acquire_gogdl_creds()
    proc = await asyncio.create_subprocess_exec(
        *cmd,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.STDOUT,
        env=env,
    )
    proc._unifideck_gogdl_cleanup = cleanup
    return proc
async def _read_progress_loop(
    self,
    proc: asyncio.subprocess.Process,
    progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,
) -> bool:
    """Read progress loop."""
    progress: dict[str, Any] = {
        "progress_percent": 0,
        "downloaded_bytes": 0,
        "total_bytes": 0,
        "speed_bps": 0.0,
        "eta_seconds": 0,
        "phase_message": "Starting download...",
    }
    assert proc.stdout is not None
    while True:
        try:
            line = await asyncio.wait_for(
                proc.stdout.readline(),
                timeout=_GOGDL_STALL_TIMEOUT_S,
            )
        except TimeoutError:
            logger.warning(
                "[GOGInstaller] stalled (no output for %ds)",
                int(_GOGDL_STALL_TIMEOUT_S),
            )
            await self._terminate_gogdl(proc)

```

```

        return False
    if not line:
        return True
    line_str = line.decode(errors="replace").strip()
    if not line_str:
        continue
    await self._handle_progress_line(
        line_str, progress, progress_cb,
    )

async def _handle_progress_line(
    self,
    line_str: str,
    progress: dict[str, Any],
    progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,
) -> None:

    """Handle progress line."""
    is_progress_line = (
        "Progress:" in line_str or "Download" in line_str
    )
    if not is_progress_line and \
        not line_str.startswith("[gogdl]"):
        logger.info("[gogdl] %s", line_str)
    if progress_cb is None:
        return
    self._parse_progress_line(line_str, progress)
    is_change_line = (
        "Progress:" in line_str
        or "+ Download" in line_str
    )
    if not is_change_line:
        return
    try:
        await progress_cb(dict(progress))
    except Exception as e:
        logger.debug(
            "[GOGInstaller] progress_cb: %s", e,
        )

    @staticmethod
    def _parse_eta(line: str) -> int | None:
        """Parse eta."""
        if "ETA:" not in line:
            return None
        eta_part = line.split("ETA:", 1)[1].strip()
        if not eta_part:
            return None
        eta_time = eta_part.split()[0]
        parts = eta_time.split(":")
        try:
            if len(parts) == 3:
                h, m, s = int(parts[0]), int(parts[1]), int(parts[2])
                return h * 3600 + m * 60 + s
            if len(parts) == 2:
                m, s = int(parts[0]), int(parts[1])
                return m * 60 + s
        except ValueError:
            return None
        return None

    @staticmethod
    def _parse_speed_mib(line: str) -> float | None:

```

```

        """Parse speed mib."""
        if "+ Download" not in line or "MiB/s" not in line:
            return None
        tail = line.split("Download", 1)[1]
        speed_part = tail.split("MiB/s", 1)[0].strip()
        speed_tokens = speed_part.split()
        if not speed_tokens:
            return None
        try:
            speed_mib = float(speed_tokens[-1])
        except ValueError:
            return None
        return speed_mib * 1024 * 1024

    @staticmethod
    def _parse_progress_line(
        line: str, progress: dict[str, Any],
    ) -> None:
        """Parse progress line."""
        speed_bps = _GogdlProgressMonitor._parse_speed_mib(line)
        if speed_bps is not None:
            progress["speed_bps"] = speed_bps
        if "Progress:" not in line:
            return
        try:
            part = line.split("Progress:", 1)[1].strip()
            tokens = part.split()
            if len(tokens) < 2:
                return
            progress["progress_percent"] = float(tokens[0])
            bytes_part = tokens[1].rstrip(",")
            if "/" not in bytes_part:
                return
            written, total = bytes_part.split("/", 1)
            progress["downloaded_bytes"] = int(written)
            progress["total_bytes"] = int(total)
            eta = _GogdlProgressMonitor._parse_eta(line)
            if eta is not None:
                progress["eta_seconds"] = eta
            progress["phase_message"] = (
                f"Downloading... "
                f"{progress['progress_percent']:.1f}%"
            )
        except (ValueError, IndexError) as e:
            logger.debug(
                "[GOGInstaller] progress parse: %s", e,
            )

    @staticmethod
    async def _terminate_gogdl(proc: asyncio.subprocess.Process) -> None:
        """Terminate GOGDL."""
        try:
            proc.terminate()
            await asyncio.sleep(1)
            if proc.returncode is None:
                proc.kill()
        except Exception as e:
            logger.error(
                "[GOGInstaller] terminate failed: %s", e,
            )

    async def run_gogdl_repair_pass(

```

```

        self,
        game_id: str,
        platform: str,
        base_path: str,
        folder_name: str | None,
        preferred_lang: str,
    ) -> None:

        """Run GOGDL repair pass."""
        repair_path = self._resolve_repair_path(
            game_id, base_path, folder_name,
        )
        cmd = [
            self._parent._gogdl_bin,
            "--auth-config-path",
            self._parent._config.auth_config_path,
            "repair", game_id,
            "--platform", platform,
            "--path", repair_path,
            "--lang", preferred_lang,
            "--with-dlcs",
        ]
        try:
            env, _gogdl_cleanup = await self._parent._tokens.acquire_gogdl_creds()
            try:
                proc = await asyncio.create_subprocess_exec(
                    *cmd,
                    stdout=asyncio.subprocess.PIPE,
                    stderr=asyncio.subprocess.STDOUT,
                    env=env,
                )
            except OSError as e:
                logger.warning(
                    "[GOGInstaller] could not spawn repair: %s", e,
                )
                await _gogdl_cleanup()
                return
            try:
                assert proc.stdout is not None
                while True:
                    line = await proc.stdout.readline()
                    if not line:
                        break
                    line_str = line.decode(errors="replace").strip()
                    if line_str and not line_str.startswith("[gogdl]"):
                        logger.info("[gogdl-verify] %s", line_str)
                await proc.wait()
                if proc.returncode != 0:
                    logger.warning(
                        "[GOGInstaller] repair code %d (non-fatal)",
                        proc.returncode,
                    )
            finally:
                await _gogdl_cleanup()
        except Exception as e:
            logger.warning(
                "[GOGInstaller] repair pipeline failed: %s", e,
            )

    @staticmethod
    def _resolve_repair_path(

```

```
    game_id: str,
    base_path: str,
    folder_name: str | None,
) -> str:
    """Resolve repair path."""
    if folder_name:
        predicted = os.path.join(base_path, folder_name)
        if os.path.exists(predicted):
            return predicted
    try:
        for name in os.listdir(base_path):
            candidate = os.path.join(base_path, name)
            if not os.path.isdir(candidate):
                continue
            if GOGFolderOps.has_goggame_info(
                candidate, game_id,
            ):
                return candidate
    except OSError:
        pass
    logger.warning(
        "[GOGInstaller] could not resolve repair path, "
        "using base_path",
    )
    return base_path
```


OP-51g

marker.py

Fichier : py_modules/unifideck/stores/gog/install/marker.py | Lignes : 228 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import json
import logging
import os
import shutil
from typing import TYPE_CHECKING, Any, cast
from .primitives import GOGFolderOps
if TYPE_CHECKING:
    from .installer import GOGInstaller
logger = logging.getLogger(__name__)
class _PostInstallMarker:
    """Post install marker."""
    def __init__(self, parent: GOGInstaller) -> None:
        """Initialize the instance."""
        self._parent = parent
    @staticmethod
    def snapshot_dirs(base_path: str) -> set:
        """Snapshot dirs."""
        try:
            return set(os.listdir(base_path))
        except OSError:
            return set()

    async def locate_install(
        self,
        game_id: str,
        base_path: str,
        folder_name: str | None,
        existing_dirs: set,
    ) -> str | None:
        """Locate install."""
        flat_info = self._find_flat_goggame(base_path, game_id)
        if flat_info:
            return await self._reorganise_flat_install(
                base_path, game_id, folder_name,
                existing_dirs,
            )
        if folder_name:
            candidate = os.path.join(base_path, folder_name)
            if os.path.isdir(candidate):
                logger.info(
                    "[GOGInstaller] found at predicted: %s",
                    candidate,
                )
                return candidate
        try:
            current = set(os.listdir(base_path))
        except OSError:
            return None
        new_dirs = current - existing_dirs
        for name in new_dirs:
            item_path = os.path.join(base_path, name)
            if not os.path.isdir(item_path):
```

```

        continue
    for search_dir in (
        item_path,
        os.path.join(item_path, "game"),
    ):
        if GOGFolderOps.has_goggame_info(
            search_dir, game_id,
        ):
            logger.info(
                "[GOGInstaller] found via scan: %s",
                item_path,
            )
            return item_path
    return None

@staticmethod
def _find_flat_goggame(
    base_path: str, game_id: str,
) -> bool:
    """Find flat goggame."""
    try:
        for name in os.listdir(base_path):
            full = os.path.join(base_path, name)
            if not os.path.isfile(full):
                continue
            if name == f"goggame-{game_id}.info":
                return True
    except OSError:
        pass
    return False

@staticmethod
async def _reorganise_flat_install(
    base_path: str,
    game_id: str,
    folder_name: str | None,
    existing_dirs: set,
) -> str:
    """Reorganise flat install."""
    target = folder_name or f"GOG_{game_id}"
    target_path = os.path.join(base_path, target)
    def _sync_move() -> None:
        """Sync move."""
        os.makedirs(target_path, exist_ok=True)
        try:
            current = set(os.listdir(base_path))
        except OSError:
            return
        new_files = current - existing_dirs
        for item in new_files:
            if item == target:
                continue
            src = os.path.join(base_path, item)
            dst = os.path.join(target_path, item)
            try:
                shutil.move(src, dst)
            except OSError as e:
                logger.warning(
                    "[GOGInstaller] move %s failed: %s",
                    item, e,
                )
    await asyncio.to_thread(_sync_move)

```

```

        logger.info(
            "[GOGInstaller] reorganised flat install → %s",
            target_path,
        )
        return target_path
    async def write_install_marker(
        self,
        install_path: str,
        game_id: str,
        language: str,
    ) -> bool:
        """Write install marker."""
        marker_path = os.path.join(install_path, ".unifideck-id")
        info_data = self._load_info_data_from_goggame(
            install_path, game_id,
        )
        info_data["language"] = language
        ok = await asyncio.to_thread(
            self._write_marker_sync, marker_path, info_data,
        )
        if ok:
            logger.info(
                "[GOGInstaller] wrote marker at %s (lang=%s)",
                marker_path, language,
            )
        return ok
    @staticmethod
    def _load_info_data_from_goggame(
        install_path: str, game_id: str,
    ) -> dict[str, Any]:
        """Load info data from goggame."""
        info_data: dict[str, Any] = {"game_id": game_id}
        for candidate in (
            install_path,
            os.path.join(install_path, "game"),
        ):
            if not os.path.isdir(candidate):
                continue
            loaded = _PostInstallMarker._try_load_info_in_dir(
                candidate, game_id,
            )
            if loaded is not None:
                info_data = loaded
                if "name" in info_data:
                    break
        return info_data
    @staticmethod
    def _try_load_info_in_dir(
        directory: str, game_id: str,
    ) -> dict[str, Any] | None:
        """Try load info in dir."""
        try:
            entries = os.listdir(directory)
        except OSError:
            return None
        for name in entries:
            if not name.startswith("goggame-"):
                continue
            if not name.endswith(".info"):
                continue

```

```

        try:
            with open(
                os.path.join(directory, name),
                encoding="utf-8",
            ) as f:
                parsed = json.load(f)
                parsed["game_id"] = game_id
                return cast("dict[str, Any] | None", parsed)
        except (OSError, json.JSONDecodeError):
            return None
    return None

@staticmethod
def _write_marker_sync(
    marker_path: str, info_data: dict[str, Any],
) -> bool:
    """Write marker sync."""
    try:
        with open(marker_path, "w", encoding="utf-8") as f:
            json.dump(info_data, f, indent=2)
        return True
    except OSError as e:
        logger.error(
            "[GOGInstaller] marker write failed: %s", e,
        )
        return False

async def regenerate_manifest(
    self, game_id: str, platform: str,
) -> None:
    """Regenerate manifest."""
    cmd = [
        self._parent._gogdl_bin,
        "--auth-config-path",
        self._parent._config.auth_config_path,
        "info", "--platform", platform, game_id,
    ]
    try:
        env, _gogdl_cleanup = await self._parent._tokens.acquire_gogdl_creds()
        try:
            proc = await asyncio.create_subprocess_exec(
                *cmd,
                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE,
                env=env,
            )
            await proc.wait()
            logger.info(
                "[GOGInstaller] manifest regenerated (code %d)",
                proc.returncode,
            )
        finally:
            await _gogdl_cleanup()
    except OSError as e:
        logger.error(
            "[GOGInstaller] manifest regen failed: %s", e,
        )

```

OP-51h

helpers.py

Fichier : py_modules/unifideck/stores/gog/install/helpers.py | Lignes : 156 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
from typing import (
    TYPE_CHECKING,
)
from .languages import smart_match_language
if TYPE_CHECKING:
    from .installer import GOGInstaller
logger = logging.getLogger(__name__)
class _InstallHelpers:
    """Install helpers."""
    def __init__(self, parent: GOGInstaller) -> None:
        """Initialize the instance."""
        self._parent = parent

    async def probe_game_info(
        self, game_id: str,
    ) -> tuple[str, str | None, list[str]]:

        """Probe game info."""
        platform = "linux"
        folder_name: str | None = None
        languages: list[str] = []
        for trial_platform in ("linux", "windows"):
            cmd = [
                self._parent._gogdl_bin,
                "--auth-config-path",
                self._parent._config.auth_config_path,
                "info",
                "--platform", trial_platform,
                game_id,
            ]
            env, _gogdl_cleanup = (
                await self._parent._tokens.acquire_gogdl_creds()
            )
            stdout = b""
            try:
                proc = await asyncio.create_subprocess_exec(
                    *cmd,
                    stdout=asyncio.subprocess.PIPE,
                    stderr=asyncio.subprocess.PIPE,
                    env=env,
                )
                try:
                    stdout, _stderr = await asyncio.wait_for(
                        proc.communicate(),
                        timeout=60,
                    )
                except TimeoutError:
                    logger.warning(
                        "[GOGInstaller] gogdl info timed out on "
                        "%s/%s – killing subprocess",
                        trial_platform, game_id,
                    )

```

```

        )
        try:
            proc.kill()
        except ProcessLookupError:
            pass
        await proc.wait()
        stdout = b""
    finally:
        await _gogdl_cleanup()
    if proc.returncode != 0 and trial_platform == "linux":
        logger.info(
            "[GOGInstaller] no Linux build for %s, "
            "trying Windows", game_id,
        )
        continue
    platform = trial_platform
    folder_name, languages = self.parse_info_output(
        stdout.decode(errors="replace"),
    )
    break
if folder_name:
    logger.info(
        "[GOGInstaller] info: platform=%s folder=%s "
        "langs=%s",
        platform, folder_name, languages,
    )
return platform, folder_name, languages

@staticmethod
def parse_info_output(
    stdout: str,
) -> tuple[str | None, list[str]]:
    """Parse info output."""
    folder_name: str | None = None
    languages: list[str] = []
    for line in reversed(stdout.splitlines()):
        line = line.strip()
        if not line:
            continue
        try:
            data = json.loads(line)
        except json.JSONDecodeError:
            continue
        if "folder_name" in data and not folder_name:
            folder_name = data["folder_name"]
        if "languages" in data and not languages:
            langs = data["languages"]
            if isinstance(langs, list):
                languages = [str(x) for x in langs]
        if folder_name and languages:
            break
    return folder_name, languages

@staticmethod
def pick_languages(
    primary_lang: str,
    explicit: bool,
    supported: list[str],
) -> list[str]:
    """Pick languages."""
    if explicit:
        return _InstallHelpers._pick_explicit_lang(

```

```
        primary_lang, supported,
    )
    return _InstallHelpers._pick_implicit_langs(
        primary_lang, supported,
    )
@staticmethod
def _pick_explicit_lang(
    primary_lang: str, supported: list[str],
) -> list[str]:
    """Pick explicit lang."""
    if not supported:
        return [primary_lang]
    matched = smart_match_language(primary_lang, supported)
    if matched:
        return [matched]
    logger.warning(
        "[GOGInstaller] %s not available, using %s",
        primary_lang, supported[0],
    )
    return [supported[0]]
@staticmethod
def _pick_implicit_langs(
    primary_lang: str, supported: list[str],
) -> list[str]:
    """Pick implicit langs."""
    if not supported:
        langs = [primary_lang]
        if "en-US" not in langs:
            langs.append("en-US")
        return langs
    result: list[str] = []
    matched = smart_match_language(primary_lang, supported)
    if matched:
        result.append(matched)
    else:
        matched_english = smart_match_language(
            "en-US", supported,
        )
        if matched_english:
            result.append(matched_english)
        else:
            result.append(supported[0])
    return result
```

OP-52

`__init__.py`

Fichier : `py_modules/unifideck/stores/gog/tokens/__init__.py` | Lignes : 3 | Depend de : (rien)

```
from .manager import GOGTokenManager
from .user_info import GOGUserInfo
__all__ = ["GOGTokenManager", "GOGUserInfo"]
```


OP-52a

manager.py

Fichier : py_modules/unifideck/stores/gog/tokens/manager.py | Lignes : 134 | Depend de : (rien)

```
from __future__ import annotations
import contextlib
import logging
import os
import time
from typing import TYPE_CHECKING, Any
from ...security import SecureTokenStore
from .gogdl_credentials import _GogdlCreds
from .oauth import _TokenOAuth
from .storage import _TokenStorage
from .user_info import GOGUserInfo
if TYPE_CHECKING:
    from collections.abc import AsyncIterator
    from ..config import GOGConfig
logger = logging.getLogger(__name__)
class GOGTokenManager:
    """Gogtoken manager."""
    def __init__(
        self,
        config: GOGConfig,
        secure_store: SecureTokenStore | None = None,
        bus: Any = None,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._bus = bus
        self._secure_store = secure_store or SecureTokenStore(
            bus=bus,
        )
        self._access_token: str | None = None
        self._refresh_token: str | None = None
        self._user_info = GOGUserInfo()
        self._storage = _TokenStorage(
            config=config, bus=bus,
            secure_store=self._secure_store,
        )
        self._oauth = _TokenOAuth(
            config=config, save_callback=self.save,
        )
        self._gogdl = _GogdlCreds(config=config)
    @property
    def access_token(self) -> str | None:
        """Access token."""
        return self._access_token
    @property
    def refresh_token(self) -> str | None:
        """Refresh token."""
        return self._refresh_token
    @property
    def user_info(self) -> GOGUserInfo:
        """User info."""
        return self._user_info
    @property
    def has_tokens(self) -> bool:
        """Check whether tokens."""
```

```

        return bool(
            self._access_token and self._refresh_token,
        )

def get_token_age_seconds(self) -> float:

    """Get token age seconds."""
    path = os.path.expanduser(self._config.token_file)
    if not os.path.isfile(path):
        return float("inf")
    try:
        return time.time() - os.path.getmtime(path)
    except OSError:
        return float("inf")
async def load(self) -> bool:
    """Load."""
    result = await self._storage.load()
    if result is None:
        return False
    access, refresh, user_info = result
    self._access_token = access
    self._refresh_token = refresh
    self._user_info = user_info
    return True
async def save(
    self,
    access_token: str,
    refresh_token: str,
) -> bool:
    """Save."""
    new_user_info = await self._oauth.fetch_user_info(
        access_token, self._user_info,
    )
    ok = await self._storage.persist(
        access_token, refresh_token, new_user_info,
    )
    if not ok:
        return False
    self._access_token = access_token
    self._refresh_token = refresh_token
    self._user_info = new_user_info
    return True
async def clear(self) -> None:
    """Clear."""
    self._access_token = None
    self._refresh_token = None
    self._user_info = GOGUserInfo()
    await self._storage.clear_files()
async def exchange_code(self, auth_code: str) -> bool:
    """Exchange code."""
    return await self._oauth.exchange_code(auth_code)
async def refresh_if_stale(self) -> bool:
    """Refresh if stale."""
    return await self._oauth.refresh_if_stale(
        access_token=self._access_token,
        refresh_token=self._refresh_token,
        age_seconds=self.get_token_age_seconds(),
    )
@contextlib.asynccontextmanager
async def gogdl_credentials(
    self,

```

```
) -> AsyncIterator[dict[str, str]]:
    """Gogdl credentials."""
    env, cleanup = await self.acquire_gogdl_creds()
    try:
        yield env
    finally:
        await cleanup()
async def acquire_gogdl_creds(self) -> tuple[
    dict[str, str],
    Any,
]:
    """Acquire GOGDL creds."""
    if not self._access_token or not self._refresh_token:
        raise RuntimeError(
            "acquire_gogdl_creds called without "
            "authenticated tokens",
        )
    return await self._gogdl.acquire(
        self._access_token, self._refresh_token,
    )
```

OP-52b

storage.py

Fichier : py_modules/unifideck/stores/gog/tokens/storage.py | Lignes : 216 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
import os
import time
from typing import TYPE_CHECKING, Any, cast
from ...security import (
    SecureTokenStore,
    SecureTokenStoreError,
    emit_legacy_plaintext_detected,
    emit_permissions_check,
    emit_token_file_migrated,
)
from .user_info import GOGUserInfo
if TYPE_CHECKING:
    from ..config import GOGConfig
logger = logging.getLogger(__name__)
class _TokenStorage:
    """Token storage."""
    def __init__(
        self,
        *,
        config: GOGConfig,
        bus: Any,
        secure_store: SecureTokenStore,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._bus = bus
        self._secure_store = secure_store
    async def load(
        self,
    ) -> tuple[str, str, GOGUserInfo] | None:
        """Load."""
        path = os.path.expanduser(self._config.token_file)
        if not os.path.isfile(path):
            return None
        def _read_sync() -> bytes | None:
            """Read sync."""
            try:
                with open(path, "rb") as f:
                    return f.read()
            except OSError as e:
                logger.warning("[GOGTokens] load failed: %s", e)
                return None
        blob = await asyncio.to_thread(_read_sync)
        if blob is None:
            return None
        data = self._parse_token_blob(blob, path)
        if not isinstance(data, dict):
            return None
        access = data.get("access_token")
        refresh = data.get("refresh_token")
        if not access or not refresh:

```

```

        return None
    user_info = GOGUserInfo(
        username=str(data.get("username", "")),
        galaxy_user_id=str(data.get("user_id", "")),
    )
    logger.info(
        "[GOGTokens] loaded tokens from disk (user=%s)",
        user_info.username or "unknown",
    )
    return access, refresh, user_info

async def persist(
    self,
    access_token: str,
    refresh_token: str,
    user_info: GOGUserInfo,
) -> bool:

    """Persist."""
    path = os.path.expanduser(self._config.token_file)
    payload = {
        "access_token": access_token,
        "refresh_token": refresh_token,
        "username": user_info.username,
        "user_id": user_info.galaxy_user_id,
    }
    try:
        blob = self._secure_store.encrypt_payload(payload)
    except SecureTokenStoreError as e:
        logger.error(
            "[GOGTokens] cannot encrypt tokens: %s - "
            "refusing to write plaintext fallback", e,
        )
        return False
    ok = await asyncio.to_thread(
        self._write_token_file_atomic, path, blob,
    )
    if not ok:
        return False
    await self._remove_stale_gogdl_mirror()
    await self._emit_post_save_security(path)
    logger.info("[GOGTokens] saved tokens (encrypted)")
    return True

async def clear_files(self) -> None:
    """Clear files."""
    paths_to_remove = [
        os.path.expanduser(self._config.token_file),
        os.path.join(
            os.path.expanduser(
                self._config.gogdl_config_dir,
            ),
            "gog_credentials.json",
        ),
    ]
    def _remove_sync() -> None:
        """Remove sync."""
        for path in paths_to_remove:
            if not os.path.isfile(path):
                continue
            try:
                os.remove(path)

```

```

        logger.info(
            "[GOGTokens] removed %s", path,
        )
    except OSError as e:
        logger.warning(
            "[GOGTokens] could not remove %s: %s",
            path, e,
        )
    await asyncio.to_thread(_remove_sync)
@staticmethod
def _write_token_file_atomic(path: str, blob: bytes) -> bool:
    """Write token file atomic."""
    try:
        parent = os.path.dirname(path)
        if parent:
            os.makedirs(parent, exist_ok=True)
        tmp = path + ".tmp"
        fd = os.open(
            tmp,
            os.O_WRONLY | os.O_CREAT | os.O_TRUNC,
            0o600,
        )
        with os.fdopen(fd, "wb") as f:
            f.write(blob)
        os.replace(tmp, path)
    except OSError as e:
        logger.warning(
            "[GOGTokens] save failed: %s", e,
        )
        return False
    return True

async def _emit_post_save_security(self, path: str) -> None:

    """Emit post save security."""
    def _stat_mode() -> int | None:
        """Stat mode."""
        try:
            st = os.stat(path)
            return st.st_mode & 0o777
        except OSError:
            return None
    mode = await asyncio.to_thread(_stat_mode)
    if mode is not None:
        emit_permissions_check(
            self._bus, "gog", path, mode,
        )
def _parse_token_blob(
    self, blob: bytes, path: str,
) -> dict[str, Any] | None:
    """Parse token blob."""
    if self._secure_store.is_encrypted(blob):
        try:
            return self._secure_store.decrypt_payload(blob)
        except SecureTokenStoreError as e:
            logger.warning(
                "[GOGTokens] decrypt failed for %s: %s",
                path, e,
            )
        return None
    logger.info(

```

```

        "[GOGTokens] reading legacy plaintext token file "
        "at %s – will encrypt on next save", path,
    )
    emit_legacy_plaintext_detected(self._bus, "gog", path)
    try:
        return cast(
            "dict[str, Any] | None",
            json.loads(blob.decode("utf-8")),
        )
    except (ValueError, UnicodeDecodeError) as e:
        logger.warning(
            "[GOGTokens] legacy JSON parse failed: %s", e,
        )
        return None
    async def _remove_stale_gogdl_mirror(self) -> None:
        """Remove stale GOGDL mirror."""
        stale = os.path.join(
            os.path.expanduser(self._config.gogdl_config_dir),
            "gog_credentials.json",
        )
    def _remove() -> bool:
        """Remove."""
        if not os.path.isfile(stale):
            return False
        try:
            os.remove(stale)
            logger.info(
                "[GOGTokens] removed stale gogdl mirror "
                "at %s", stale,
            )
            return True
        except OSError as e:
            logger.warning(
                "[GOGTokens] could not remove stale gogdl "
                "mirror %s: %s", stale, e,
            )
            return False
    removed = await asyncio.to_thread(_remove)
    if removed:
        emit_token_file_migrated(
            self._bus, "gog", stale, "",
        )
    _ = time

```

OP-52c

oauth.py

Fichier : py_modules/unifideck/stores/gog/tokens/oauth.py | Lignes : 113 | Depend de : (rien)

```

from __future__ import annotations
import logging
import urllib.parse
from typing import TYPE_CHECKING, Any
from ..http import fetch_json_get
from .user_info import GOGUserInfo
if TYPE_CHECKING:
    from collections.abc import Awaitable, Callable
    from ..config import GOGConfig
    SaveCallback = Callable[[str, str], Awaitable[bool]]
logger = logging.getLogger(__name__)
class _TokenOAuth:
    """Token oauth."""
    def __init__(
        self,
        *,
        config: GOGConfig,
        save_callback: SaveCallback,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._save = save_callback
    async def exchange_code(self, auth_code: str) -> bool:
        """Exchange code."""
        params = {
            "client_id": self._config.client_id,
            "client_secret": self._config.client_secret,
            "grant_type": "authorization_code",
            "code": auth_code,
            "redirect_uri": self._config.redirect_uri,
        }
        return await self._token_request(params)
    async def refresh_if_stale(
        self,
        *,
        access_token: str | None,
        refresh_token: str | None,
        age_seconds: float,
    ) -> bool:
        """Refresh if stale."""
        threshold = (
            self._config.token_refresh_threshold_seconds
        )
        if age_seconds < threshold and access_token:
            return True
        if not refresh_token:
            logger.info(
                "[GOGTokens] no refresh token – session is "
                "dead",
            )
            return False
        logger.info(
            "[GOGTokens] token age %.0fs ≥ %ds, refreshing",
            age_seconds, threshold,
        )

```



```

        params = {
            "client_id": self._config.client_id,
            "client_secret": self._config.client_secret,
            "grant_type": "refresh_token",
            "refresh_token": refresh_token,
        }
        return await self._token_request(params)

    async def fetch_user_info(
        self,
        access_token: str,
        fallback: GOGUserInfo,
    ) -> GOGUserInfo:

        """Fetch user info."""
        url = f"{self._config.base_url}/userData.json"
        data = await fetch_json_get(
            url,
            bearer=access_token,
            user_agent=self._config.user_agent,
            timeout=10.0,
            log_prefix="[GOGTokens] userData",
        )
        if not isinstance(data, dict):
            return fallback
        return GOGUserInfo(
            username=str(
                data.get("username", "") or fallback.username,
            ),
            galaxy_user_id=str(
                data.get("galaxyUserId", "")
                or fallback.galaxy_user_id,
            ),
        )

    async def _token_request(
        self, params: dict[str, str],
    ) -> bool:

        """Token request."""
        url = (
            f"{self._config.token_url}?"
            f"{urllib.parse.urlencode(params)}"
        )
        data = await fetch_json_get(
            url,
            user_agent=self._config.user_agent,
            timeout=15.0,
            log_prefix="[GOGTokens] token endpoint",
        )
        if not isinstance(data, dict):
            return False
        access = data.get("access_token")
        refresh = data.get("refresh_token")
        if not access or not refresh:
            logger.error(
                "[GOGTokens] token response missing tokens: "
                "keys=%s", list(data.keys()),
            )
            return False
        return await self._save(access, refresh)

_ = Any

```

OP-52d

gogdl_credentials.py

Fichier : py_modules/unifideck/stores/gog/tokens/gogdl_credentials.py | Lignes : 92 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
import os
import tempfile
import time
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from collections.abc import Awaitable, Callable
    from ..config import GOGConfig
    CleanupFn = Callable[[], Awaitable[None]]
logger = logging.getLogger(__name__)
class _GogdlCreds:
    """Gogdl creds."""
    def __init__(self, *, config: GOGConfig) -> None:
        """Initialize the instance."""
        self._config = config
    async def acquire(
        self,
        access_token: str,
        refresh_token: str,
    ) -> tuple[dict[str, str], CleanupFn]:
        """Acquire."""
        tmpdir = await asyncio.to_thread(
            tempfile.mkdtemp, "unifideck-gogdl-",
        )
        creds_path = os.path.join(
            tmpdir, "gog_credentials.json",
        )
        gogdl_data = self._build_gogdl_data(
            access_token, refresh_token,
        )
        await asyncio.to_thread(
            self._write_creds_sync, creds_path, gogdl_data,
        )
        env = os.environ.copy()
        env["GOGDL_CONFIG_PATH"] = tmpdir
        cleanup = self._make_cleanup(creds_path, tmpdir)
        return env, cleanup
    def _build_gogdl_data(
        self,
        access_token: str,
        refresh_token: str,
    ) -> dict[str, dict[str, object]]:
        """Build GOGDL data."""
        now = time.time()
        return {
            self._config.client_id: {
                "access_token": access_token,
                "refresh_token": refresh_token,
                "token_type": "Bearer",
                "expires_in": 3600,
                "scope": "openid",
                "created_at": now,
            }
        }

```

```
        "loginTime": now,
    },
}

@staticmethod
def _write_creds_sync(
    creds_path: str,
    gogdl_data: dict[str, dict[str, object]],
) -> None:
    """Write creds sync."""
    fd = os.open(
        creds_path,
        os.O_WRONLY | os.O_CREAT | os.O_TRUNC,
        0o600,
    )
    with os.fdopen(fd, "w", encoding="utf-8") as f:
        json.dump(gogdl_data, f)

@staticmethod
def _make_cleanup(
    creds_path: str, tmpdir: str,
) -> CleanupFn:
    """Make cleanup."""
    async def _cleanup() -> None:
        """Cleanup."""
        def _remove() -> None:
            """Remove."""
            try:
                if os.path.isfile(creds_path):
                    os.remove(creds_path)
                if os.path.isdir(tmpdir):
                    os.rmdir(tmpdir)
            except OSError as e:
                logger.warning(
                    "[GOGTokens] gogdl temp cleanup "
                    "failed: %s", e,
                )
        await asyncio.to_thread(_remove)
    return _cleanup
```

OP-52e

user_info.py

Fichier : py_modules/unifideck/stores/gog/tokens/user_info.py | Lignes : 7 | Depend de : (rien)

```
from __future__ import annotations
from dataclasses import dataclass
@dataclass
class GOGUserInfo:
    """Goguser info."""
    username: str = ""
    galaxy_user_id: str = ""
```

OP-50a

store.py

Fichier : py_modules/unifideck/stores/gog/store.py | Lignes : 328 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
from collections.abc import Awaitable, Callable
from typing import TYPE_CHECKING, Any, cast
from ...auth.browser import OAuthBrowserMonitor
from ...auth.edge_browser import EdgeBrowser
from ...auth.orchestrator import AuthOrchestrator
from ...core.types import (
    AuthResult,
    Events,
    Game,
    InstallResult,
    Result,
    StoreInfo,
)
from ...services.shortcut import ShortcutService
from ...utils.locale import get_unifideck_locale
from ..shared.store_base import StoreBase
from .auth import GOGBrowserAuth
from .config import GOG_AUTH_URL_FILE, GOGConfig
from .dlc import GOGDlcManager
from .exe_resolver import GOGExeResolver
from .install import GOGInstaller
from .library import GOGLibrary
from .tokens import GOGTokenManager
from .updates import GOGUpdatesChecker
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...core.cache_manager import CacheManager
    from ...event_bus.event_bus import EventBus
logger = logging.getLogger(__name__)
class GOGStore(StoreBase):
    """Gogstore."""
    store_info = StoreInfo(
        name="gog",
        display_name="GOG",
        auth_method="oauth",
        icon_asset="gog.png",
        uses_wine=False,
        supports_install=True,
    )
    def __init__(
        self,
        bus: EventBus,
        cache: CacheManager,
        plugin_dir: str | None = None,
        config: ConfigManager | None = None,
        browser_monitor: OAuthBrowserMonitor | None = None,
        shortcut_service: ShortcutService | None = None,
        edge_browser: EdgeBrowser | None = None,
    ) -> None:
        """Initialize the instance."""
        super().__init__(bus, cache, plugin_dir, config)
        self._gog_config: GOGConfig = \

```

```

GOGConfig.from_config_manager(config)
logger.info(
    "[GOGStore] %s", self._gog_config.describe(),
)
self._config_manager = config
self._shortcut_service = shortcut_service
self._edge = edge_browser
self._tokens = GOGTokenManager(self._gog_config, bus=bus)
self._exe = GOGExeResolver()

self._library = GOGLibrary(
    config=self._gog_config,
    tokens=self._tokens,
    exe_finder=self._exe.find,
)
if browser_monitor is not None:
    orchestrator = AuthOrchestrator(
        bus=bus,
        browser_monitor=browser_monitor,
        store_name="gog",
    )
    self._auth: GOGBrowserAuth | None = GOGBrowserAuth(
        bus=bus,
        orchestrator=orchestrator,
        tokens=self._tokens,
        config=self._gog_config,
    )
else:
    self._auth = None
    gogdl_bin = self._resolve_gogdl_bin()
    self._installer = GOGInstaller(
        config=self._gog_config,
        tokens=self._tokens,
        gogdl_bin=gogdl_bin,
        exe_finder=self._exe.find,
        locale_fn=lambda: get_unifideck_locale(
            self._config_manager,
        ),
    )
    self._dlc = GOGDlcManager(
        config=self._gog_config,
        tokens=self._tokens,
        gogdl_bin=gogdl_bin,
        locale_fn=lambda: get_unifideck_locale(
            self._config_manager,
        ),
        resolve_install_path=self._library.get_installed_game_info,
    )
    self._updates = GOGUpdatesChecker(
        config=self._gog_config,
        tokens=self._tokens,
        gogdl_bin=gogdl_bin,
        get_installed_ids=self._library.get_installed,
        resolve_install_info=self._library.get_installed_game_info,
    )
async def is_available(self) -> bool:
    """Check whether available."""
    if not self._gog_config.is_valid():
        self._cached_available = False
        return False
    available = await self._library.is_available()

```

```

        self._cached_available = available
        return available
    async def start_auth(self, **kwargs) -> AuthResult:
        """Start auth."""
        if self._auth is None:
            return AuthResult(
                success=False,
                error="auth_not_configured",
                store="gog",
            )
        if self._edge is None or not self._edge.is_installed:
            logger.info(
                "[GOGStore] Edge not installed – prompting "
                "user",
            )
            return AuthResult(
                success=False,
                error="edge_not_installed",
                store="gog",
            )
        await self._ensure_auth_shortcut()
        return cast("AuthResult", await self._auth.start_auth())

    async def complete_auth(
        self, code: str = "", **kwargs,
    ) -> AuthResult:
        """Complete auth."""
        if await self.is_available():
            return AuthResult(success=True, store="gog")
        return AuthResult(
            success=False,
            error="not_authenticated",
            store="gog",
        )

    async def logout(self) -> Result:
        """Logout."""
        if self._auth is not None:
            result = await self._auth.logout(
                browser_monitor=self._browser_monitor_from_auth(),
            )
        else:
            await self._tokens.clear()
            await self._bus.emit(
                Events.STORE_LOGOUT, store="gog",
            )
            result = Result(success=True)
        auth_url_file = os.path.expanduser(GOG_AUTH_URL_FILE)
        if os.path.isfile(auth_url_file):
            try:
                os.remove(auth_url_file)
            except OSError as e:
                logger.warning(
                    "[GOGStore] could not remove %s: %s",
                    auth_url_file, e,
                )
        return result

    async def get_library(self) -> list[Game] | None:
        """Get library."""
        return await self._library.fetch_library()

    async def install_game(

```

```

self,
game_id: str,
base_path: str | None = None,
progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None = None,
language: str | None = None,
**kwargs,
) -> InstallResult:
    """Install game."""
    return await self._installer.install_game(
        game_id=game_id,
        base_path=base_path,
        progress_cb=progress_cb,
        language=language,
    )
async def uninstall_game(
    self, game_id: str, **kwargs: Any,
) -> Result:
    """Uninstall game."""
    info = self._library.get_installed_game_info(game_id)
    install_path = (
        info.get("install_path") if info else None
    )
    return await self._installer.uninstall_game(
        game_id=game_id,
        install_path=install_path,
    )
async def update_game(
    self,
    game_id: str,
    progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None = None,
    **kwargs,
) -> InstallResult:
    """Update game."""
    result = await self._updates.update_game(game_id)
    return InstallResult(
        success=result.success,
        error=result.error,
        store="gog",
        game_id=game_id,
    )
async def check_for_updates(self) -> list[str]:
    """Check for updates."""
    return await self._updates.check_for_updates()

async def get_game_size(
    self, game_id: str,
) -> int | None:
    """Get game size."""
    size = await self._installer._planner.get_expected_disk_size(
        game_id, "windows",
    )
    return size if size > 0 else None
async def get_game_dlcs(
    self, game_id: str,
) -> list[dict[str, Any]]:
    """Get game dlcs."""
    return await self._dlc.get_game_dlcs(game_id)
async def get_available_languages(
    self, game_id: str,
) -> list[str]:

```



```

        """Get available languages."""
        return await self._dlc.get_available_languages(game_id)
    async def install_dlc(
        self,
        game_id: str,
        dlc_id: str,
        base_path: str | None = None,
        progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None = None,
    ) -> Result:
        """Install dlc."""
        return await self._dlc.install_dlc(
            game_id=game_id,
            dlc_id=dlc_id,
            base_path=base_path,
            progress_cb=progress_cb,
        )
    async def get_game_store_url(
        self, game_id: str,
    ) -> str | None:
        """Get game store URL."""
        return await self._dlc.get_game_store_url(game_id)
    async def get_game_slug(
        self, game_id: str,
    ) -> str | None:
        """Get game slug."""
        return await self._library.get_game_slug(game_id)
    def get_installed(self) -> list[str]:
        """Get installed."""
        return self._library.get_installed()
    def get_installed_game_info(
        self, game_id: str,
    ) -> dict[str, str | None] | None:
        """Get installed game info."""
        return self._library.get_installed_game_info(game_id)
    def migrate_old_markers(self) -> dict[str, int]:
        """Migrate old markers."""
        return self._library.migrate_old_markers()
    def _resolve_gogdl_bin(self) -> str:
        """Resolve GOGDL bin."""
        if not self._plugin_dir:
            logger.warning(
                "[GOGStore] no plugin_dir; gogdl path "
                "unresolvable",
            )
            return ""
        path = os.path.join(
            self._plugin_dir, "bin", "gogdl",
        )
        if not os.path.isfile(path):
            logger.warning(
                "[GOGStore] gogdl binary not found at %s",
                path,
            )
        else:
            logger.info(
                "[GOGStore] using gogdl at %s", path,
            )
        return path

    async def _ensure_auth_shortcut(self) -> None:

```

```
"""Ensure auth shortcut."""
if self._shortcut_service is None:
    logger.debug(
        "[GOGStore] no shortcut_service; skipping "
        "auth shortcut creation",
    )
    return
launcher = os.path.join(
    self._plugin_dir or "",
    "py_modules", "unifideck", "launcher",
    "dispatcher.py",
)
if not os.path.isfile(launcher):
    logger.warning(
        "[GOGStore] launcher dispatcher not found "
        "at %s", launcher,
    )
    return
result = await self._shortcut_service.add_auth_shortcut(
    store="gog",
    launcher_path=launcher,
    title="GOG Sign-In",
)
if not result.success:
    logger.warning(
        "[GOGStore] add_auth_shortcut failed: %s",
        result.error,
    )
def _browser_monitor_from_auth(self) -> OAuthBrowserMonitor | None:
    """Browser monitor from auth."""
    if self._auth is None:
        return None
    try:
        return self._auth._orch._monitor
    except AttributeError:
        return None
```

OP-50b

config.py

Fichier : py_modules/unifideck/stores/gog/config.py | Lignes : 120 | Depend de : (rien)

```

from __future__ import annotations
import logging
from dataclasses import dataclass, field
from typing import TYPE_CHECKING
from unifideck.utils.config_helpers import get_cfg
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
_GOG_CONFIG_PREFIX = "stores.gog"
_DEFAULT_TOKEN_FILE = "~/.config/unifideck/gog_token.json"
_DEFAULT_GOGDL_CONFIG_DIR = "~/.config/unifideck/gogdl"
_DEFAULT_DOWNLOAD_DIR = "~/GOG Games"
GOG_AUTH_URL_FILE = "~/.local/share/unifideck/gog_auth_url.txt"
@dataclass(frozen=True)
class GOGConfig:
    """Gogconfig."""
    client_id: str = ""
    client_secret: str = ""
    auth_url: str = ""
    token_url: str = ""
    redirect_uri: str = ""
    allowed_redirect_uris: list[str] = field(default_factory=list)
    base_url: str = ""
    api_gog_url: str = ""
    token_file: str = _DEFAULT_TOKEN_FILE
    gogdl_config_dir: str = _DEFAULT_GOGDL_CONFIG_DIR
    download_dir: str = _DEFAULT_DOWNLOAD_DIR
    token_refresh_threshold_seconds: int = 2400
    supported_languages: list[str] = field(
        default_factory=lambda: [
            "en", "de", "fr", "pl", "ru", "pt",
            "es", "it", "zh", "ko", "ja",
        ],
    )
    user_agent: str = "Unifideck/1.0"

    @classmethod
    def from_config_manager(cls, config: ConfigManager | None) -> GOGConfig:
        """From config manager."""
        def _s(key: str, default: str = "") -> str:
            """S."""
            val = get_cfg(config, f"{_GOG_CONFIG_PREFIX}.{key}", default)
            return str(val).strip() if val is not None else default
        def _i(key: str, default: int) -> int:
            """I."""
            val = get_cfg(config, f"{_GOG_CONFIG_PREFIX}.{key}", default)
            try:
                return int(val)
            except (TypeError, ValueError):
                return default
        def _list(key: str) -> list[str]:
            """List."""
            val = get_cfg(config, f"{_GOG_CONFIG_PREFIX}.{key}", None)
            if not isinstance(val, list):
                return []

```

```

        return [str(x) for x in val if isinstance(x, str) and x]
    primary_redirect = _s("redirect_uri")
    allowed = _list("allowed_redirect_uris")
    if not allowed and primary_redirect:
        allowed = [primary_redirect]
    supported = _list("supported_languages")
    if not supported:
        supported = [
            "en", "de", "fr", "pl", "ru", "pt",
            "es", "it", "zh", "ko", "ja",
        ]
    return cls(
        client_id=_s("client_id"),
        client_secret=_s("client_secret"),
        auth_url=_s("auth_url"),
        token_url=_s("token_url"),
        redirect_uri=primary_redirect,
        allowed_redirect_uris=allowed,
        base_url=_s("base_url"),
        api_gog_url=_s("api_gog_url"),
        token_file=_s("token_file", _DEFAULT_TOKEN_FILE),
        gogdl_config_dir=_s(
            "gogdl_config_dir", _DEFAULT_GOGDL_CONFIG_DIR,
        ),
        download_dir=_s("download_dir", _DEFAULT_DOWNLOAD_DIR),
        token_refresh_threshold_seconds=_i(
            "token_refresh_threshold_seconds", 2400,
        ),
        supported_languages=supported,
        user_agent=_s("user_agent", "Unifideck/1.0"),
    )
def is_valid(self) -> bool:
    """Check whether valid."""
    required = (
        ("client_id", self.client_id),
        ("client_secret", self.client_secret),
        ("auth_url", self.auth_url),
        ("token_url", self.token_url),
        ("redirect_uri", self.redirect_uri),
        ("base_url", self.base_url),
        ("api_gog_url", self.api_gog_url),
    )
    missing = [name for name, val in required if not val]
    if missing:
        logger.warning(
            "[GOGConfig] missing required keys: %s",
            ", ".join(missing),
        )
        return False
    return True
@property
def auth_config_path(self) -> str:
    """Auth config path."""
    import os
    return os.path.join(
        os.path.expanduser(self.gogdl_config_dir),
        "gog_credentials.json",
    )
def describe(self) -> str:
    """Describe."""
    return (

```

```
f"GOGConfig(client_id={self.client_id[:6]}..., "  
f"base_url={self.base_url}, "  
f"token_file={self.token_file}, "  
f"download_dir={self.download_dir})"  
)
```

OP-50c

library.py

Fichier : py_modules/unifideck/stores/gog/library.py | Lignes : 306 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
import urllib.parse
import urllib.request
from collections.abc import Callable
from pathlib import Path
from typing import Any, cast
from ...core.types import Game
from .config import GOGConfig
from .http import build_ssl_context, fetch_json_get
from .library_migration import _MarkerMigration
from .tokens import GOGTokenManager
logger = logging.getLogger(__name__)
_INSTALL_MARKER = ".unifideck-id"
_GOG_LIBRARY_TIMEOUT_S = 15.0
class GOGLibrary:
    """Goglibrary."""
    def __init__(
        self,
        config: GOGConfig,
        tokens: GOGTokenManager,
        exe_finder: Callable[[str], str | None] | None = None,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._tokens = tokens
        self._find_exe = exe_finder
        self._migration = _MarkerMigration(self)
    def migrate_old_markers(self) -> dict[str, int]:
        """Migrate old markers."""
        return self._migration.migrate_old_markers()
    async def is_available(self) -> bool:
        """Check whether available."""
        if not self._tokens.has_tokens:
            loaded = await self._tokens.load()
            if not loaded:
                logger.info(
                    "[GOGLibrary] no tokens – not authenticated",
                )
                return False
        status = await self._probe_userdata()
        if status == 200:
            return True
        if status == 401:
            logger.warning(
                "[GOGLibrary] token expired (401), refreshing",
            )
            ok = await self._tokens.refresh_if_stale()
            if ok:
                status = await self._probe_userdata()
                return status == 200
            return False
        logger.warning(

```

```

        "[GOGLibrary] userdata probe returned %s", status,
    )
    return False

async def _probe_userdata(self) -> int:

    """Probe userdata."""
    url = f"{self._config.base_url}/userData.json"
    access = self._tokens.access_token
    if not access:
        return 0
    if not url.startswith("https://"):
        logger.error(
            "[GOGLibrary] refusing non-https probe URL: %s",
            url,
        )
        return 0
    def _probe_sync() -> int:
        """Probe sync."""
        try:
            ctx = build_ssl_context()
            req = urllib.request.Request(
                url,
                headers={
                    "Authorization": f"Bearer {access}",
                    "User-Agent": self._config.user_agent,
                },
            )
            with urllib.request.urlopen(
                req, timeout=5.0, context=ctx,
            ) as response:
                return cast("int", response.status)
        except urllib.request.HTTPError as e:
            return e.code
        except Exception as e:
            logger.debug(
                "[GOGLibrary] probe error: %s", e,
            )
            return 0
    return await asyncio.to_thread(_probe_sync)

async def fetch_library(self) -> list[Game]:

    """Fetch library."""
    if not self._tokens.access_token:
        logger.warning("[GOGLibrary] not authenticated")
        return []
    games: list[Game] = []
    current_page = 1
    total_pages = 1
    base_url = self._config.base_url
    while current_page <= total_pages:
        url = (
            f"{base_url}/account/getFilteredProducts?"
            f"mediaType=1&page={current_page}"
        )
        data = await self._fetch_json(url)
        if data is None:
            logger.error(
                "[GOGLibrary] page %d failed, stopping",
                current_page,
            )

```

```

        )
        break
    if current_page == 1:
        total_pages = int(
            data.get("totalPages", 1) or 1,
        )
        total_results = int(
            data.get("totalGamesFound", 0) or 0,
        )
        logger.info(
            "[GOGLibrary] library has %d games across "
            "%d pages", total_results, total_pages,
        )
    for product in data.get("products", []):
        game_id = str(product.get("id", ""))
        if not game_id:
            continue
        games.append(Game(
            app_id=0,
            store="gog",
            store_game_id=game_id,
            title=product.get("title", "") or "",
            installed=False,
        ))
    current_page += 1
    logger.info(
        "[GOGLibrary] fetched %d games total", len(games),
    )
    return games

async def get_game_slug(self, game_id: str) -> str | None:
    """Get game slug."""
    if not await self._tokens.refresh_if_stale():
        return None
    access = self._tokens.access_token
    if not access:
        return None
    url = (
        f"{self._config.api_gog_url}/products/{game_id}"
        f"?locale=en-US"
    )
    data = await self._fetch_json(
        url,
        headers={
            "Authorization": f"Bearer {access}",
            "User-Agent": self._config.user_agent,
        },
    )
    if not isinstance(data, dict):
        return None
    slug = data.get("slug")
    if isinstance(slug, str) and slug:
        return slug
    links = data.get("links", {})
    if isinstance(links, dict):
        product_card = links.get("product_card", "")
        if (
            isinstance(product_card, str)
            and "/game/" in product_card
        ):
            return product_card.split("/game/")[1].rstrip("/")
    return None

```



```

def get_installed(self) -> list[str]:
    """Get installed."""
    download_path = Path(
        self._config.download_dir,
    ).expanduser()
    if not download_path.is_dir():
        return []
    installed: list[str] = []
    try:
        for entry in download_path.iterdir():
            if not entry.is_dir():
                continue
            game_id = self._read_marker(str(entry))
            if game_id:
                installed.append(game_id)
    except OSError as e:
        logger.error(
            "[GOGLibrary] get_installed scan failed: %s", e,
        )
        return []
    logger.info(
        "[GOGLibrary] found %d installed games",
        len(installed),
    )
    return installed

def get_installed_game_info(
    self, game_id: str,
) -> dict[str, str | None] | None:
    """Get installed game info."""
    download_path = Path(
        self._config.download_dir,
    ).expanduser()
    if not download_path.is_dir():
        return None
    try:
        for entry in download_path.iterdir():
            if not entry.is_dir():
                continue
            game_dir = str(entry)
            found = self._read_marker(game_dir)
            if found == game_id:
                return {
                    "install_path": game_dir,
                    "executable": self._resolve_exe(game_dir),
                }
            if found is None and self._has_goggame_info(
                game_dir, game_id,
            ):
                logger.info(
                    "[GOGLibrary] found %s via goggame "
                    "info fallback at %s",
                    game_id, game_dir,
                )
                return {
                    "install_path": game_dir,
                    "executable": self._resolve_exe(game_dir),
                }
    except OSError as e:
        logger.error(

```

```

        "[GOGLibrary] get_installed_game_info: %s", e,
    )
    return None

@staticmethod
def _read_marker(game_dir: str) -> str | None:
    """Read marker."""
    marker_path = Path(game_dir) / _INSTALL_MARKER
    if not marker_path.is_file():
        return None
    try:
        content = marker_path.read_text(
            encoding="utf-8",
        ).strip()
    except OSError as e:
        logger.warning(
            "[GOGLibrary] marker read failed: %s", e,
        )
        return None
    if not content:
        return None
    try:
        data = json.loads(content)
        if isinstance(data, dict):
            return (
                data.get("game_id")
                or data.get("gameId")
            )
        if isinstance(data, (str, int)):
            return str(data)
    except json.JSONDecodeError:
        pass
    return content

@staticmethod
def _has_goggame_info(
    game_dir: str, game_id: str,
) -> bool:
    """Has goggame info."""
    for candidate in (
        game_dir, str(Path(game_dir) / "game"),
    ):
        try:
            if not Path(candidate).is_dir():
                continue
            target = f"goggame-{game_id}.info"
            if (Path(candidate) / target).is_file():
                return True
        except OSError:
            continue
    return False

def _resolve_exe(self, install_path: str) -> str | None:
    """Resolve exe."""
    if self._find_exe is None:
        return None
    try:
        return self._find_exe(install_path)
    except Exception as e:
        logger.warning(
            "[GOGLibrary] exe resolution failed: %s", e,
        )
    return None

```

```
async def _fetch_json(
    self,
    url: str,
    headers: dict[str, str] | None = None,
) -> Any | None:
    """Fetch JSON."""
    return await fetch_json_get(
        url,
        bearer=self._tokens.access_token,
        user_agent=self._config.user_agent,
        timeout=_GOG_LIBRARY_TIMEOUT_S,
        extra_headers=headers,
        log_prefix="[GOGLibrary]",
    )
```

OP-50d

library_migration.py

Fichier : py_modules/unifideck/stores/gog/library_migration.py | Lignes : 141 | Depend de : (rien)

```
from __future__ import annotations
import glob
import json
import logging
import os
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from .library import GOGLibrary
logger = logging.getLogger(__name__)
_INSTALL_MARKER = ".unifideck-id"
class _MarkerMigration:
    """Marker migration."""
    def __init__(self, parent: GOGLibrary) -> None:
        """Initialize the instance."""
        self._parent = parent
    def migrate_old_markers(self) -> dict[str, int]:
        """Migrate old markers."""
        migrated = 0
        skipped = 0
        download_dir = os.path.expanduser(
            self._parent._config.download_dir,
        )
        if not os.path.isdir(download_dir):
            return {"migrated": 0, "skipped": 0}
        try:
            for name in os.listdir(download_dir):
                game_dir = os.path.join(download_dir, name)
                if not os.path.isdir(game_dir):
                    continue
                marker_path = os.path.join(
                    game_dir, _INSTALL_MARKER,
                )
                if not os.path.isfile(marker_path):
                    continue
                outcome = self._migrate_one_marker(
                    game_dir, marker_path,
                )
                if outcome == "migrated":
                    migrated += 1
                else:
                    skipped += 1
        except OSError as e:
            logger.error(
                "[GOGLibrary] migrate scan failed: %s", e,
            )
        logger.info(
            "[GOGLibrary] migration: %d upgraded, %d current",
            migrated, skipped,
        )
        return {"migrated": migrated, "skipped": skipped}

    def _migrate_one_marker(
        self, game_dir: str, marker_path: str,
    ) -> str:
```

```

        """Migrate one marker."""
        content = self._read_marker_content(marker_path)
        if content is None:
            return "failed"
        if self._marker_is_new_format(content):
            return "skipped"
        old_id = self._extract_legacy_id(content)
        if not old_id:
            return "skipped"
        new_data = self._build_new_marker_payload(
            game_dir, old_id,
        )
        return self._write_new_marker(
            marker_path, new_data, game_dir,
        )
    @staticmethod
    def _read_marker_content(marker_path: str) -> str | None:
        """Read marker content."""
        try:
            with open(marker_path, encoding="utf-8") as f:
                return f.read().strip()
        except OSError:
            return None
    @staticmethod
    def _marker_is_new_format(content: str) -> bool:
        """Marker is new format."""
        try:
            data = json.loads(content)
        except json.JSONDecodeError:
            return False
        return isinstance(data, dict) and "game_id" in data
    @staticmethod
    def _extract_legacy_id(content: str) -> str | None:
        """Extract legacy ID."""
        try:
            data = json.loads(content)
        except json.JSONDecodeError:
            data = None
        if isinstance(data, (str, int)):
            return str(data)
        if content and not content.startswith("{"):
            return content
        return None
    def _build_new_marker_payload(
        self, game_dir: str, old_id: str,
    ) -> dict[str, Any]:
        """Build new marker payload."""
        new_data: dict[str, Any] = {"game_id": old_id}
        for candidate in (
            game_dir, os.path.join(game_dir, "game"),
        ):
            if not os.path.isdir(candidate):
                continue
            info_file = self._find_first_goggame_info(candidate)
            if not info_file:
                continue
            try:
                with open(info_file, encoding="utf-8") as f:
                    new_data = json.load(f)
                    new_data["game_id"] = old_id
            except (OSError, json.JSONDecodeError):

```

```
        pass
    break
    return new_data

@staticmethod
def _write_new_marker(
    marker_path: str,
    new_data: dict[str, Any],
    game_dir: str,
) -> str:
    """Write new marker."""
    try:
        with open(marker_path, "w", encoding="utf-8") as f:
            json.dump(new_data, f, indent=2)
            return "migrated"
    except OSError as e:
        logger.warning(
            "[GOGLibrary] migrate write failed for %s: %s",
            game_dir, e,
        )
        return "failed"

@staticmethod
def _find_first_goggame_info(directory: str) -> str | None:
    """Find first goggame info."""
    candidates = sorted(
        glob.glob(os.path.join(directory, "goggame-*.info")),
    )
    return candidates[0] if candidates else None
```

OP-50e

exe_resolver.py

Fichier : py_modules/unifideck/stores/gog/exe_resolver.py | Lignes : 316 | Depend de : (rien)

```
from __future__ import annotations
import glob
import json
import logging
import os
from pathlib import Path
from typing import Any
logger = logging.getLogger(__name__)
_SKIP_EXE_PATTERNS = (
    "unins",
    "setup",
    "install",
    "crash",
    "redis",
    "vcredist",
    "vc_redis",
    "dxsetup",
    "physx",
    "dotnet",
    "directx",
)
_ROOT_DATA_EXTENSIONS = (".arch05", ".forge")
_WRAPPER_EXE_NAMES = {"dosbox.exe", "scummvm.exe"}
class GOGExeResolver:
    """Gogexe resolver."""
    def find(self, install_path: str) -> str | None:
        """Find."""
        result = self.find_with_workdir(install_path)
        return result[0] if result else None
    def find_with_workdir(
        self, install_path: str,
    ) -> tuple[str, str] | None:
        """Find with workdir."""
        try:
            return self._resolve(install_path)
        except Exception as e:
            logger.exception(
                "[GOGExeResolver] unexpected error for %s: %s",
                install_path, e,
            )
            return None
    def _resolve(
        self, install_path: str,
    ) -> tuple[str, str] | None:
        """Resolve."""
        search_dirs = self._build_search_dirs(install_path)
        info_result = self._resolve_via_goggame_info(
            install_path, search_dirs,
        )
        if info_result:
            return info_result
        start_sh_result = self._resolve_via_start_sh(search_dirs)
        if start_sh_result:
```

```

        return start_sh_result
    fallback_result = self._resolve_via_largest_exe(
        search_dirs,
    )
    if fallback_result:
        return fallback_result
    logger.warning(
        "[GOGExeResolver] no executable found in %s",
        search_dirs,
    )
    return None
@staticmethod
def _build_search_dirs(install_path: str) -> list[str]:
    """Build search dirs."""
    search_dirs: list[str] = []
    game_subdir = str(Path(install_path) / "game")
    if Path(game_subdir).is_dir():
        search_dirs.append(game_subdir)
    search_dirs.append(install_path)
    return search_dirs
def _resolve_via_goggame_info(
    self,
    install_path: str,
    search_dirs: list[str],
) -> tuple[str, str] | None:
    """Resolve via goggame info."""
    primary, root_dir = self._load_primary_play_task(
        search_dirs,
    )
    if primary is None:
        return None
    wrapper = self._check_wrapper_override(
        install_path, root_dir, primary,
    )
    if wrapper:
        return wrapper
    return self._resolve_play_task_paths(
        install_path, root_dir, primary,
    )
def _load_primary_play_task(
    self, search_dirs: list[str],
) -> tuple[dict[str, Any] | None, str]:
    """Load primary play task."""
    info_file, root_dir = self._find_goggame_info(search_dirs)
    if not info_file:
        return None, ""
    try:
        data = json.loads(
            Path(info_file).read_text(encoding="utf-8"),
        )
    except (OSError, json.JSONDecodeError) as e:
        logger.warning(
            "[GOGExeResolver] info file read failed: %s", e,
        )
        return None, ""
    play_tasks = data.get("playTasks", [])
    if not isinstance(play_tasks, list):
        return None, ""
    primary = next(
        (
            t for t in play_tasks

```



```

        if isinstance(t, dict) and t.get("isPrimary")
    ),
    None,
)
return primary, root_dir

def _resolve_play_task_paths(
    self,
    install_path: str,
    root_dir: str,
    primary: dict[str, Any],
) -> tuple[str, str] | None:

    """Resolve play task paths."""
    exe_rel = primary.get("path", "")
    if not exe_rel:
        return None
    work_rel = primary.get("workingDir", "")
    full_exe = str(
        Path(root_dir) / exe_rel,
    ).replace("\\", "/")
    full_work = (
        str(Path(root_dir) / work_rel).replace("\\", "/")
        if work_rel
        else str(Path(full_exe).parent)
    )
    if (
        full_work != install_path
        and self._has_root_data_files(install_path)
    ):
        logger.info(
            "[GOGExeResolver] data files in root, "
            "overriding workdir to %s", install_path,
        )
        full_work = install_path
    if not Path(full_exe).is_file():
        return None
    logger.info(
        "[GOGExeResolver] resolved via goggame info: %s",
        full_exe,
    )
    return (full_exe, full_work)

@staticmethod
def _find_goggame_info(
    search_dirs: list[str],
) -> tuple[str | None, str]:
    """Find goggame info."""
    for directory in search_dirs:
        if not Path(directory).is_dir():
            continue
        try:
            for item in os.listdir(directory):
                if (
                    item.startswith("goggame-")
                    and item.endswith(".info")
                ):
                    return (
                        str(Path(directory) / item),
                        directory,
                    )
        except OSError:

```

```

        continue
    return (None, search_dirs[0] if search_dirs else "")

def _check_wrapper_override(
    self,
    install_path: str,
    root_dir: str,
    primary_task: dict[str, Any],
) -> tuple[str, str] | None:

    """Check wrapper override."""
    task_path = primary_task.get("path", "")
    if not task_path:
        return None
    task_basename = Path(task_path).name.lower()
    candidates = [root_dir]
    if install_path not in candidates:
        candidates.append(install_path)
    for candidate_root in candidates:
        wrapper_path = str(
            Path(candidate_root) / "run-game.bat",
        )
        if not Path(wrapper_path).is_file():
            continue
        try:
            content = Path(wrapper_path).read_text(
                encoding="utf-8", errors="ignore",
            ).lower()
        except OSError:
            content = ""
        if (
            task_basename in content
            or task_basename in _WRAPPER_EXE_NAMES
        ):
            logger.info(
                "[GOGExeResolver] using wrapper: %s",
                wrapper_path,
            )
            return (wrapper_path, candidate_root)
    return None

@staticmethod
def _has_root_data_files(install_path: str) -> bool:
    """Has root data files."""
    try:
        for name in os.listdir(install_path):
            full = Path(install_path) / name
            if not full.is_file():
                continue
            if any(
                name.endswith(ext)
                for ext in _ROOT_DATA_EXTENSIONS
            ):
                return True
    except OSError:
        pass
    return False

@staticmethod
def _resolve_via_start_sh(
    search_dirs: list[str],
) -> tuple[str, str] | None:
    """Resolve via start sh."""

```

```

    for directory in search_dirs:
        if not Path(directory).is_dir():
            continue
        candidate = str(Path(directory) / "start.sh")
        if Path(candidate).is_file():
            logger.info(
                "[GOGExeResolver] resolved via start.sh: %s",
                candidate,
            )
            return (candidate, directory)
    return None

@staticmethod
def _resolve_via_largest_exe(
    search_dirs: list[str],
) -> tuple[str, str] | None:
    """Resolve via largest exe."""
    for directory in search_dirs:
        if not Path(directory).is_dir():
            continue
        candidates: list[tuple[str, int]] = []
        for pattern in ("*.exe", "**/*.exe"):
            for exe_path in glob.glob(
                str(Path(directory) / pattern),
                recursive=True,
            ):
                basename = Path(exe_path).name.lower()
                if any(
                    skip in basename
                    for skip in _SKIP_EXE_PATTERNS
                ):
                    continue
                try:
                    candidates.append(
                        (
                            exe_path,
                            Path(exe_path).stat().st_size,
                        ),
                    )
                except OSError:
                    continue
            if not candidates:
                continue
            candidates.sort(key=lambda item: item[1], reverse=True)
            best_exe, best_size = candidates[0]
            logger.info(
                "[GOGExeResolver] fallback: largest exe "
                "(%.1f MB): %s",
                best_size / (1024 * 1024), best_exe,
            )
            return (best_exe, str(Path(best_exe).parent))
    return None

def parse_size_string(size_str: str) -> int:
    """Parse size string."""
    if not size_str:
        return 0
    try:
        parts = str(size_str).strip().split()
        if len(parts) != 2:
            return 0
        value = float(parts[0])

```

```
        unit = parts[1].upper()
        if unit == "GB":
            return int(value * 1024 * 1024 * 1024)
        if unit == "MB":
            return int(value * 1024 * 1024)
        if unit == "KB":
            return int(value * 1024)
        return int(value)
    except (ValueError, TypeError):
        return 0
def get_game_id_from_goggame_filename(
    filename: str,
) -> str | None:
    """Get game ID from goggame filename."""
    if not filename:
        return None
    name = filename.strip()
    if (
        not name.startswith("goggame-")
        or not name.endswith(".info")
    ):
        return None
    game_id = name[len("goggame-):-len(".info")]
    return game_id if game_id else None
```

OP-50f

dlc.py

Fichier : py_modules/unifideck/stores/gog/dlc.py | Lignes : 354 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
from collections.abc import Awaitable, Callable
from pathlib import Path
from typing import Any, cast
from ...core.types import Result
from .config import GOGConfig
from .http import fetch_json_get
from .tokens import GOGTokenManager
logger = logging.getLogger(__name__)
_LANGUAGE_FALLBACK = ["en-US"]
_LANG_PROBE_TIMEOUT_S = 30.0
class GOGDlcManager:
    """Gogdlc manager."""
    def __init__(
        self,
        config: GOGConfig,
        tokens: GOGTokenManager,
        gogdl_bin: str,
        locale_fn: Callable[[], str],
        resolve_install_path: Callable[
            [str], dict[str, str | None] | None,
        ],
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._tokens = tokens
        self._gogdl_bin = gogdl_bin
        self._locale_fn = locale_fn
        self._resolve_install = resolve_install_path

    async def get_game_dlcs(
        self, game_id: str,
    ) -> list[dict[str, Any]]:

        """Get game dlcs."""
        if not await self._tokens.refresh_if_stale():
            logger.warning(
                "[GOGDlcManager] not authenticated for DLC "
                "fetch",
            )
            return []
        access = self._tokens.access_token
        if not access:
            return []
        product_url = (
            f"{self._config.api_gog_url}/products/{game_id}"
            f"?expand=downloads&locale=en-US"
        )
        product = await self._http_get_json(
            product_url, bearer=access,
        )
        if not isinstance(product, dict):

```

```

        return []
    dlcs_info = product.get("dlcs", {})
    if not isinstance(dlcs_info, dict) or not dlcs_info:
        logger.debug(
            "[GOGDlcManager] no DLCs for %s", game_id,
        )
        return []
    expanded_url = dlcs_info.get(
        "expanded_all_products_url",
    )
    if isinstance(expanded_url, str) and expanded_url:
        expanded = await self._http_get_json(
            expanded_url, bearer=access,
        )
        if isinstance(expanded, list):
            logger.info(
                "[GOGDlcManager] found %d DLCs for %s",
                len(expanded), game_id,
            )
            return expanded
        logger.warning(
            "[GOGDlcManager] expanded DLC list malformed "
            "for %s", game_id,
        )
    basic_products = dlcs_info.get("products", [])
    if isinstance(basic_products, list):
        return basic_products
    return []
async def get_available_languages(
    self, game_id: str,
) -> list[str]:
    """Get available languages."""
    stdout = await self._spawn_lang_probe(game_id)
    if stdout is None:
        return list(_LANGUAGE_FALLBACK)
    languages = self._parse_languages_from_info(stdout)
    if languages:
        logger.info(
            "[GOGDlcManager] %s languages: %s",
            game_id, languages,
        )
        return languages
    logger.warning(
        "[GOGDlcManager] no languages in info output "
        "for %s",
        game_id,
    )
    return list(_LANGUAGE_FALLBACK)

async def _spawn_lang_probe(
    self, game_id: str,
) -> bytes | None:
    """Spawn lang probe."""
    if not await self._tokens.refresh_if_stale():
        return None
    if not Path(self._gogdl_bin).is_file():
        return None
    cmd = [
        self._gogdl_bin,
        "--auth-config-path",
    ]

```

```

        self._config.auth_config_path,
        "info", "--platform", "windows", game_id,
    ]
    try:
        async with self._tokens.gogdl_credentials() as env:
            proc = await asyncio.create_subprocess_exec(
                *cmd,
                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE,
                env=env,
            )
            stdout, _stderr = await asyncio.wait_for(
                proc.communicate(),
                timeout=_LANG_PROBE_TIMEOUT_S,
            )
    except TimeoutError:
        logger.warning(
            "[GOGDlcManager] language probe timed out "
            "for %s",
            game_id,
        )
        return None
    except OSError as e:
        logger.warning(
            "[GOGDlcManager] gogdl spawn failed: %s", e,
        )
        return None
    if proc.returncode != 0:
        return None
    return stdout
@staticmethod
def _parse_languages_from_info(
    stdout: bytes,
) -> list[str]:
    """Parse languages from info."""
    for raw_line in stdout.decode(
        errors="replace",
    ).splitlines():
        line = raw_line.strip()
        if not line:
            continue
        try:
            data = json.loads(line)
        except json.JSONDecodeError:
            continue
        langs = data.get("languages")
        if isinstance(langs, list) and langs:
            result = [str(x) for x in langs if x]
            if result:
                return result
    return []

async def install_dlc(
    self,
    game_id: str,
    dlc_id: str,
    base_path: str | None = None,
    progress_cb: (
        Callable[[dict[str, Any]], Awaitable[None]]
        | None
    ) = None,

```

```

) -> Result:

    """Install dlc."""
    failure = await self._dlc_preflight()
    if failure is not None:
        return failure
    resolved_base = self._dlc_resolve_base_path(
        game_id, base_path,
    )
    preferred_lang = self._locale_fn() or "en-US"
    logger.info(
        "[GOGDLCManager] installing DLC %s for game %s "
        "at %s (lang=%s)",
        dlc_id, game_id, resolved_base, preferred_lang,
    )
    proc = await self._dlc_spawn_gogdl(
        dlc_id, resolved_base, preferred_lang,
    )
    if proc is None:
        return Result(
            success=False, error="gogdl_spawn_failed",
        )
    await self._dlc_read_loop(proc, dlc_id, progress_cb)
    return await self._dlc_finalize(proc, dlc_id)
async def _dlc_preflight(self) -> Result | None:
    """Dlc preflight."""
    if not Path(self._gogdl_bin).is_file():
        return Result(
            success=False, error="gogdl_not_found",
        )
    if not await self._tokens.refresh_if_stale():
        return Result(
            success=False, error="not_authenticated",
        )
    return None
def _dlc_resolve_base_path(
    self, game_id: str, base_path: str | None,
) -> str:
    """Dlc resolve base path."""
    if base_path:
        return base_path
    info = self._resolve_install(game_id)
    if info and isinstance(info.get("install_path"), str):
        return cast("str", info["install_path"])
    return str(
        Path(self._config.download_dir).expanduser(),
    )

async def _dlc_spawn_gogdl(
    self, dlc_id: str, base_path: str, lang: str,
) -> asyncio.subprocess.Process | None:

    """Dlc spawn GOGDL."""
    cmd = [
        self._gogdl_bin,
        "--auth-config-path",
        self._config.auth_config_path,
        "repair",
        dlc_id,
        "--platform", "windows",
        "--path", base_path,
    ]

```



```

        "--lang", lang,
    ]
    try:
        env, cleanup = (
            await self._tokens.acquire_gogdl_creds()
        )
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.STDOUT,
            env=env,
        )
        proc._unifideck_gogdl_cleanup = cleanup
        return proc
    except OSError as e:
        logger.error(
            "[GOGDlcManager] gogdl spawn failed: %s", e,
        )
        return None
    async def _dlc_read_loop(
        self,
        proc: asyncio.subprocess.Process,
        dlc_id: str,
        progress_cb: (
            Callable[[dict[str, Any]], Awaitable[None]]
            | None
        ),
    ) -> None:
        """Dlc read loop."""
        assert proc.stdout is not None
        while True:
            line = await proc.stdout.readline()
            if not line:
                break
            line_str = line.decode(errors="replace").strip()
            if not line_str:
                continue
            if (
                progress_cb is not None
                and "Progress:" in line_str
            ):
                await self._forward_dlc_progress(
                    line_str, dlc_id, progress_cb,
                )
    async def _dlc_finalize(
        self, proc: asyncio.subprocess.Process, dlc_id: str,
    ) -> Result:
        """Dlc finalize."""
        await proc.wait()
        if proc.returncode != 0:
            logger.error(
                "[GOGDlcManager] DLC install failed "
                "(code %d)",
                proc.returncode,
            )
        return Result(
            success=False,
            error=(
                f"dlc_install_failed_code_"
                f"{proc.returncode}"
            ),
        ),

```

```

    )
    logger.info(
        "[GOGDlcManager] DLC %s installed successfully",
        dlc_id,
    )
    return Result(success=True)

@staticmethod
async def _forward_dlc_progress(
    line_str: str,
    dlc_id: str,
    progress_cb: Callable[
        [dict[str, Any]], Awaitable[None],
    ],
) -> None:
    """Forward dlc progress."""
    try:
        part = line_str.split("Progress:", 1)[1].strip()
        tokens = part.split()
        if not tokens:
            return
        percent = float(tokens[0])
        await progress_cb({
            "progress_percent": percent,
            "phase_message": (
                f"Installing DLC... {percent:.1f}%"
            ),
            "dlc_id": dlc_id,
        })
    except (ValueError, IndexError) as e:
        logger.debug(
            "[GOGDlcManager] DLC progress parse: %s", e,
        )

async def get_game_store_url(
    self, game_id: str,
) -> str | None:
    """Get game store URL."""
    url = (
        f"{self._config.api_gog_url}/products/{game_id}"
        f"?expand=description"
    )
    data = await self._http_get_json(url)
    if not isinstance(data, dict):
        return None
    links = data.get("links", {})
    if not isinstance(links, dict):
        return None
    product_card = links.get("product_card")
    if isinstance(product_card, str) and product_card:
        return product_card
    return None

async def _http_get_json(
    self,
    url: str,
    bearer: str | None = None,
) -> asyncio.subprocess.Process | None:
    """Http get JSON."""
    return await fetch_json_get(
        url,
        bearer=bearer,
        user_agent=self._config.user_agent,
    )

```

```
        timeout=10.0,  
        log_prefix="[GOGDlcManager]",  
    )
```

OP-50g

updates.py

Fichier : py_modules/unifideck/stores/gog/updates.py | Lignes : 288 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
from collections.abc import Callable
from pathlib import Path
from typing import Any, cast
from ...core.types import Result
from .config import GOGConfig
from .http import fetch_json_get
from .tokens import GOGTokenManager
logger = logging.getLogger(__name__)
_CONTENT_SYSTEM_URL_TEMPLATE = (
    "https://content-system.gog.com/products/"
    "{game_id}/os/windows/builds?generation=2"
)
_UPDATE_CHECK_TIMEOUT_S = 10.0
class GOGUpdatesChecker:
    """Gogupdates checker."""
    def __init__(
        self,
        config: GOGConfig,
        tokens: GOGTokenManager,
        gogdl_bin: str,
        get_installed_ids: Callable[[], list[str]],
        resolve_install_info: Callable[
            [str], dict[str, str | None] | None,
        ],
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._tokens = tokens
        self._gogdl_bin = gogdl_bin
        self._get_installed = get_installed_ids
        self._resolve_info = resolve_install_info

    @staticmethod
    def get_local_build_id(
        install_path: str, game_id: str,
    ) -> str | None:
        """Get local build ID."""
        install_p = Path(install_path)
        for search_dir in (install_p, install_p / "game"):
            info_file = search_dir / f"goggame-{game_id}.info"
            if not info_file.is_file():
                continue
            try:
                data = json.loads(
                    info_file.read_text(encoding="utf-8"),
                )
            except (OSError, json.JSONDecodeError) as e:
                logger.warning(
                    "[GOGUpdatesChecker] info read failed "
                    "for %s: %s",
                    info_file, e,

```

```

        )
        continue
    build_id = data.get("buildId")
    if build_id:
        logger.debug(
            "[GOGUpdatesChecker] local buildId for "
            "%s: %s",
            game_id, build_id,
        )
        return str(build_id)
    marker_path = install_p / ".unifideck-id"
    if marker_path.is_file():
        try:
            data = json.loads(
                marker_path.read_text(
                    encoding="utf-8",
                ).strip(),
            )
        except (OSError, json.JSONDecodeError):
            return None
        if isinstance(data, dict):
            build_id = data.get("buildId")
            if build_id:
                logger.debug(
                    "[GOGUpdatesChecker] marker buildId "
                    "for %s: %s", game_id, build_id,
                )
                return str(build_id)
    return None

async def check_for_game_update(
    self, game_id: str,
) -> bool | None:

    """Check for game update."""
    if not await self._tokens.refresh_if_stale():
        logger.warning(
            "[GOGUpdatesChecker] not authenticated for "
            "update check of %s", game_id,
        )
        return None
    info = self._resolve_info(game_id)
    install_path = info.get("install_path") if info else None
    if not install_path:
        logger.debug(
            "[GOGUpdatesChecker] %s not installed, "
            "skipping",
            game_id,
        )
        return None
    local_build_id = self.get_local_build_id(
        install_path, game_id,
    )
    if not local_build_id:
        logger.warning(
            "[GOGUpdatesChecker] no local buildId for %s "
            "- cannot check for update", game_id,
        )
        return None
    remote_build_id = await self._fetch_remote_build_id(
        game_id,

```

```

    )
    if remote_build_id is None:
        return None
    logger.info(
        "[GOGUpdatesChecker] %s: local=%s, remote=%s",
        game_id, local_build_id, remote_build_id,
    )
    has_update = remote_build_id != local_build_id
    if has_update:
        logger.info(
            "[GOGUpdatesChecker] update available for %s",
            game_id,
        )
    return has_update
async def _fetch_remote_build_id(
    self, game_id: str,
) -> str | None:
    """Fetch remote build ID."""
    access = self._tokens.access_token
    if not access:
        return None
    url = _CONTENT_SYSTEM_URL_TEMPLATE.format(
        game_id=game_id,
    )
    data = await fetch_json_get(
        url,
        bearer=access,
        user_agent=self._config.user_agent,
        timeout=_UPDATE_CHECK_TIMEOUT_S,
        log_prefix=f"[GOGUpdatesChecker] {game_id}",
    )
    if not isinstance(data, dict):
        return None
    items = data.get("items")
    if not isinstance(items, list) or not items:
        logger.warning(
            "[GOGUpdatesChecker] no builds returned for "
            "%s",
            game_id,
        )
        return None
    first = items[0]
    if not isinstance(first, dict):
        return None
    build_id = first.get("build_id")
    if build_id is None:
        return None
    return str(build_id)

async def check_for_updates(self) -> list[str]:
    """Check for updates."""
    installed_ids = self._get_installed()
    if not installed_ids:
        return []
    updates: list[str] = []
    for game_id in installed_ids:
        has_update = await self.check_for_game_update(
            game_id,
        )
        if has_update:

```

```

        updates.append(game_id)
    logger.info(
        "[GOGUpdatesChecker] bulk check: %d/%d have "
        "updates",
        len(updates), len(installed_ids),
    )
    return updates
async def update_game(
    self,
    game_id: str,
    install_path: str | None = None,
) -> Result:
    """Update game."""
    gogdl_exists = await asyncio.to_thread(
        Path(self._gogdl_bin).is_file,
    )
    if not gogdl_exists:
        return Result(
            success=False, error="gogdl_not_found",
        )
    resolved_path, path_failure = self._update_resolve_path(
        game_id, install_path,
    )
    if path_failure is not None:
        return cast("Result", path_failure)
    if not await self._tokens.refresh_if_stale():
        return Result(
            success=False, error="not_authenticated",
        )
    logger.info(
        "[GOGUpdatesChecker] starting update for %s "
        "at %s",
        game_id, resolved_path,
    )
    proc = await self._update_spawn_gogdl(
        game_id, resolved_path,
    )
    if proc is None:
        return Result(
            success=False, error="gogdl_spawn_failed",
        )
    await self._update_drain_output(proc)
    return await self._update_finalize(proc, game_id)
def _update_resolve_path(
    self, game_id: str, install_path: str | None,
) -> tuple:
    """Update resolve path."""
    if install_path:
        return install_path, None
    info = self._resolve_info(game_id)
    if info and isinstance(info.get("install_path"), str):
        return info["install_path"], None
    return None, Result(
        success=False, error="install_path_not_found",
    )

async def _update_spawn_gogdl(
    self, game_id: str, install_path: str,
) -> Any | None:
    """Update spawn GOGDL."""

```

```

cmd = [
    self._gogdl_bin,
    "--auth-config-path",
    self._config.auth_config_path,
    "update", game_id,
    "--path", install_path,
    "--platform", "windows",
]
try:
    env, cleanup = await self._tokens.acquire_gogdl_creds()
    proc = await asyncio.create_subprocess_exec(
        *cmd,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.STDOUT,
        env=env,
    )
    proc._unifideck_gogdl_cleanup = cleanup
    return proc
except OSError as e:
    logger.error(
        "[GOGUpdatesChecker] gogdl spawn failed: %s",
        e,
    )
    return None
@staticmethod
async def _update_drain_output(proc: Any) -> None:
    """Update drain output."""
    while True:
        line = await proc.stdout.readline()
        if not line:
            break
        line_str = line.decode(errors="replace").strip()
        if line_str:
            logger.info(
                "[GOGUpdatesChecker/update] %s", line_str,
            )
@staticmethod
async def _update_finalize(
    proc: Any, game_id: str,
) -> Result:
    """Update finalize."""
    await proc.wait()
    if proc.returncode != 0:
        logger.error(
            "[GOGUpdatesChecker] update failed for %s "
            "(code %d)",
            game_id, proc.returncode,
        )
        return Result(
            success=False,
            error=f"update_failed_code_{proc.returncode}",
        )
    logger.info(
        "[GOGUpdatesChecker] successfully updated %s",
        game_id,
    )
    return Result(success=True)

```


OP-50h

auth.py

Fichier : py_modules/unifideck/stores/gog/auth.py | Lignes : 92 | Depend de : (rien)

```

from __future__ import annotations
import logging
import urllib.parse
from typing import Any
from ...auth.orchestrator import AuthOrchestrator
from ...core.types import AuthResult, Events, Result
from ...event_bus.event_bus import EventBus
from ...security import audit_auth_flow
from .config import GOG_AUTH_URL_FILE, GOGConfig
from .tokens import GOGTokenManager
logger = logging.getLogger(__name__)
_GOG_COOKIE_DOMAIN = "gog.com"
class GOGBrowserAuth:
    """Gogbrowser auth."""
    def __init__(
        self,
        bus: EventBus,
        orchestrator: AuthOrchestrator,
        tokens: GOGTokenManager,
        config: GOGConfig,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._orch = orchestrator
        self._tokens = tokens
        self._config = config
    @audit_auth_flow(store="gog", method="oauth_browser")
    async def start_auth(self) -> AuthResult:
        """Start auth."""
        if not self._config.is_valid():
            return AuthResult(
                success=False,
                error="config_invalid",
                store="gog",
            )
        return await self._orch.run_flow(
            get_url=self._build_auth_url,
            allowed_uris=list(
                self._config.allowed_redirect_uris,
            ),
            exchange_code=self._exchange_code,
            background=True,
            write_url_file=GOG_AUTH_URL_FILE,
        )
    async def _build_auth_url(self) -> str:
        """Build auth URL."""
        params = {
            "client_id": self._config.client_id,
            "redirect_uri": self._config.redirect_uri,
            "response_type": "code",
            "layout": "client2",
        }
        query = urllib.parse.urlencode(params, safe="/: ")
        url = f"{self._config.auth_url}?{query}"
        logger.info("[GOGBrowserAuth] built OAuth URL")

```

```
        return url

    async def _exchange_code(self, code: str) -> AuthResult:

        """Exchange code."""
        ok = await self._tokens.exchange_code(code)
        if ok:
            logger.info(
                "[GOGBrowserAuth] token exchange successful",
            )
            return AuthResult(success=True, store="gog")
        return AuthResult(
            success=False,
            error="token_exchange_failed",
            store="gog",
        )

    async def logout(
        self,
        browser_monitor: Any | None = None,
    ) -> Result:
        """Logout."""
        self._orch.cancel_background()
        await self._tokens.clear()
        if browser_monitor is not None:
            try:
                await browser_monitor.clear_cookies_for_domain(
                    _GOG_COOKIE_DOMAIN,
                )
                logger.info(
                    "[GOGBrowserAuth] cleared cookies for %s",
                    _GOG_COOKIE_DOMAIN,
                )
            except Exception as e:
                logger.warning(
                    "[GOGBrowserAuth] cookie clear failed: %s",
                    e,
                )
        await self._bus.emit(Events.STORE_LOGOUT, store="gog")
        return Result(success=True)
```

OP-50i

http.py

Fichier : py_modules/unifideck/stores/gog/http.py | Lignes : 49 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import json
import logging
import ssl
import urllib.request
from collections.abc import Mapping
from typing import Any
from ...core.net import ssl_ctx_strict
_logger = logging.getLogger(__name__)
def build_ssl_context() -> ssl.SSLContext:
    """Build ssl context."""
    return ssl_ctx_strict()
async def fetch_json_get(
    url: str,
    *,
    bearer: str | None = None,
    user_agent: str,
    timeout: float = 15.0,
    extra_headers: Mapping[str, str] | None = None,
    log_prefix: str = "[GOGHttP]",
) -> Any | None:
    """Fetch JSON get."""
    headers: dict[str, str] = {"User-Agent": user_agent}
    if bearer:
        headers["Authorization"] = f"Bearer {bearer}"
    if extra_headers:
        headers.update(extra_headers)
    def _sync() -> Any | None:
        """Sync."""
        try:
            ctx = build_ssl_context()
            req = urllib.request.Request(url, headers=headers)
            with urllib.request.urlopen(
                req, timeout=timeout, context=ctx,
            ) as response:
                if response.status != 200:
                    _logger.warning(
                        "%s GET %s → HTTP %d",
                        log_prefix, url, response.status,
                    )
                return None
            return json.loads(response.read().decode())
        except Exception as e:
            _logger.warning(
                "%s GET %s failed: %s", log_prefix, url, e,
            )
            return None
    return await asyncio.to_thread(_sync)
```

OP-53

`__init__.py`

Fichier : `py_modules/unifideck/stores/microsoft/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from .microsoft_store import MicrosoftStore
__all__ = ["MicrosoftStore"]
```

OP-54

`__init__.py`

Fichier : `py_modules/unifideck/stores/microsoft/tokens/__init__.py` | Lignes : 7 | Depend de : (rien)

```
from __future__ import annotations
from .manager import MicrosoftTokenManager
from .xbl_chain import XBLTokenChain
__all__ = [
    "MicrosoftTokenManager",
    "XBLTokenChain",
]
```

OP-54a

manager.py

Fichier : py_modules/unifideck/stores/microsoft/tokens/manager.py | Lignes : 43 | Depend de : (rien)

```
from __future__ import annotations
import logging
from collections.abc import Callable
from typing import TYPE_CHECKING
from ...security import SecureTokenStore
from .oauth import OAuthMixin
from .persistence import PersistenceMixin
from .xbl_chain import XBLChainMixin
if TYPE_CHECKING:
    from ...event_bus.event_bus import EventBus
    from ..microsoft_config import MicrosoftConfig
logger = logging.getLogger(__name__)
class MicrosoftTokenManager(
    PersistenceMixin,
    OAuthMixin,
    XBLChainMixin,
):
    """Microsoft token manager."""
    def __init__(
        self,
        config: MicrosoftConfig,
        locale_fn: Callable[[], str],
        secure_store: SecureTokenStore | None = None,
        bus: EventBus | None = None,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._locale_fn = locale_fn
        self._bus = bus
        self._secure_store = (
            secure_store or SecureTokenStore(bus=bus)
        )
        self._ms_access_token: str | None = None
        self._ms_refresh_token: str | None = None
        self._token_saved_at: float = 0.0
    @property
    def access_token(self) -> str | None:
        """Access token."""
        return self._ms_access_token
    @property
    def has_refresh_token(self) -> bool:
        """Check whether refresh token."""
        return bool(self._ms_refresh_token)
```

OP-54b

persistence.py

Fichier : py_modules/unifideck/stores/microsoft/tokens/persistence.py | Lignes : 182 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
import os
from pathlib import Path
from typing import TYPE_CHECKING, Any, cast
from ...security import (
    SecureTokenStore,
    SecureTokenStoreError,
    emit_legacy_plaintext_detected,
    emit_permissions_check,
)
if TYPE_CHECKING:
    from ...event_bus.event_bus import EventBus
    from ..microsoft_config import MicrosoftConfig
logger = logging.getLogger(__name__)
class PersistenceMixin:
    """Persistence mixin."""
    _ms_access_token: str | None
    _ms_refresh_token: str | None
    _token_saved_at: float
    _config: MicrosoftConfig
    _secure_store: SecureTokenStore
    _bus: EventBus | None
    async def load(self) -> bool:
        """Load."""
        path = str(Path(self._config.token_file).expanduser())
        if not Path(path).is_file():
            return False
        def _read_sync() -> bytes | None:
            """Read sync."""
            try:
                return Path(path).read_bytes()
            except OSError as e:
                logger.warning(
                    "[MicrosoftTokens] load failed: %s", e,
                )
                return None
        blob = await asyncio.to_thread(_read_sync)
        if blob is None:
            return False
        data = self._parse_blob(blob, path)
        if not isinstance(data, dict):
            return False
        refresh = data.get("refresh_token")
        if not refresh:
            self._ms_access_token = None
            self._ms_refresh_token = None
            self._token_saved_at = 0.0
            return False
        self._ms_access_token = data.get("access_token") or None
        self._ms_refresh_token = refresh
        try:
            self._token_saved_at = float(

```

```

        data.get("saved_at", 0.0),
    )
except (TypeError, ValueError):
    self._token_saved_at = 0.0
logger.info("[MicrosoftTokens] loaded tokens from disk")
return True

def _parse_blob(
    self, blob: bytes, path: str,
) -> dict[str, Any] | None:

    """Parse blob."""
    if self._secure_store.is_encrypted(blob):
        try:
            return self._secure_store.decrypt_payload(blob)
        except SecureTokenStoreError as e:
            logger.warning(
                "[MicrosoftTokens] decrypt failed for "
                "%s: %s", path, e,
            )
            return None
    logger.info(
        "[MicrosoftTokens] reading legacy plaintext token "
        "file at %s – will encrypt on next save",
        path,
    )
    if self._bus is not None:
        emit_legacy_plaintext_detected(
            self._bus, "microsoft", path,
        )
    try:
        return cast(
            "dict[str, Any] | None",
            json.loads(blob.decode("utf-8")),
        )
    except (ValueError, UnicodeDecodeError) as e:
        logger.warning(
            "[MicrosoftTokens] legacy JSON parse "
            "failed: %s", e,
        )
        return None
async def save(self) -> bool:
    """Save."""
    if (
        self._ms_access_token is None
        and self._ms_refresh_token is None
    ):
        return True
    path = str(Path(self._config.token_file).expanduser())
    payload = {
        "access_token": self._ms_access_token,
        "refresh_token": self._ms_refresh_token,
        "saved_at": self._token_saved_at,
        "scope": self._config.scope,
    }
    try:
        blob = self._secure_store.encrypt_payload(payload)
    except SecureTokenStoreError as e:
        logger.error(
            "[MicrosoftTokens] cannot encrypt tokens: "
            "%s – refusing to write plaintext fallback",

```



```

        e,
    )
    return False
ok = await asyncio.to_thread(
    _write_atomic_0600, path, blob,
)
await self._emit_permissions_after_save(ok, path)
return ok
async def _emit_permissions_after_save(
    self, ok: bool, path: str,
) -> None:
    """Emit permissions after save."""
    if not ok:
        return
    def _stat() -> int | None:
        """Stat."""
        try:
            return Path(path).stat().st_mode & 0o7777
        except OSError:
            return None
    mode = await asyncio.to_thread(_stat)
    if mode is not None and self._bus is not None:
        emit_permissions_check(
            self._bus, "microsoft", path, mode,
        )

async def clear(self) -> None:
    """Clear."""
    self._ms_access_token = None
    self._ms_refresh_token = None
    self._token_saved_at = 0.0
    path = str(Path(self._config.token_file).expanduser())
    if Path(path).is_file():
        def _remove_sync() -> None:
            """Remove sync."""
            try:
                Path(path).unlink()
            except OSError as e:
                logger.warning(
                    "[MicrosoftTokens] clear: could not "
                    "remove %s: %s", path, e,
                )
        await asyncio.to_thread(_remove_sync)
def _write_atomic_0600(path: str, blob: bytes) -> bool:
    """Write atomic 0600."""
    try:
        parent = str(Path(path).parent)
        if parent:
            Path(parent).mkdir(parents=True, exist_ok=True)
        tmp = path + ".tmp"
        fd = os.open(
            tmp,
            os.O_WRONLY | os.O_CREAT | os.O_TRUNC,
            0o600,
        )
        try:
            with os.fdopen(fd, "wb") as f:
                f.write(blob)
        except Exception:
            if fd >= 0:

```

```
        os.close(fd)
    raise
    os.replace(tmp, path)
    return True
except OSError as e:
    logger.warning(
        "[MicrosoftTokens] save failed: %s", e,
    )
    return False
```

OP-54c

oauth.py

Fichier : py_modules/unifideck/stores/microsoft/tokens/oauth.py | Lignes : 88 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import time
from typing import TYPE_CHECKING
from ..microsoft_auth import http_post
if TYPE_CHECKING:
    from ..microsoft_config import MicrosoftConfig
logger = logging.getLogger(__name__)
class OAuthMixin:
    """OAuth mixin."""
    _ms_access_token: str | None
    _ms_refresh_token: str | None
    _token_saved_at: float
    _config: MicrosoftConfig
    async def exchange_code(self, auth_code: str) -> bool:
        """Exchange code."""
        return await self._token_request({
            "client_id": self._config.client_id,
            "redirect_uri": self._config.redirect_uri,
            "code": auth_code,
            "grant_type": "authorization_code",
            "scope": self._config.scope,
        })
    async def refresh_if_stale(self) -> bool:
        """Refresh if stale."""
        age = time.time() - self._token_saved_at
        threshold = self._config.token_refresh_threshold_seconds
        if age < threshold and self._ms_access_token:
            return True
        if not self._ms_refresh_token:
            logger.error(
                "[MicrosoftTokens] refresh needed but no "
                "refresh token available – session dead",
            )
            return False
        logger.info(
            "[MicrosoftTokens] refreshing access token "
            "(age=%.0fs)",
            age,
        )
        return await self._token_request({
            "client_id": self._config.client_id,
            "redirect_uri": self._config.redirect_uri,
            "refresh_token": self._ms_refresh_token,
            "grant_type": "refresh_token",
            "scope": self._config.scope,
        })
    async def _token_request(
        self, params: dict[str, str],
    ) -> bool:
        """Token request."""
        headers = {

```

```
        "Content-Type":
            "application/x-www-form-urlencoded",
    }
    try:
        token_data = await (
            asyncio.get_event_loop().run_in_executor(
                None,
                lambda: http_post(
                    self._config.token_url,
                    params, headers,
                ),
            )
        )
    except Exception as e:
        logger.error(
            "[MicrosoftTokens] token HTTP failed: %s", e,
        )
        return False
    if (
        not isinstance(token_data, dict)
        or "access_token" not in token_data
    ):
        error = (token_data or {}).get("error", "unknown")
        logger.error(
            "[MicrosoftTokens] token endpoint rejected "
            "request: %s", error,
        )
        return False
    self._ms_access_token = token_data["access_token"]
    new_refresh = token_data.get("refresh_token")
    if new_refresh:
        self._ms_refresh_token = new_refresh
    self._token_saved_at = time.time()
    await self.save()
    return True
```

OP-54d

xbl_chain.py

Fichier : py_modules/unifideck/stores/microsoft/tokens/xbl_chain.py | Lignes : 138 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from collections.abc import Callable
from dataclasses import dataclass
from typing import TYPE_CHECKING
from ..microsoft_auth import build_xbl_chain, request_xsts_token
if TYPE_CHECKING:
    from ..microsoft_config import MicrosoftConfig
logger = logging.getLogger(__name__)
@dataclass
class XBLTokenChain:
    """Xbltoken chain."""
    xsts_token: str
    user_hash: str
    xuid: str | None = None
    xbl_token: str | None = None
class XBLChainMixin:
    """Xblchain mixin."""
    _ms_access_token: str | None
    _config: MicrosoftConfig
    _locale_fn: Callable[[], str]
    async def build_chain(self) -> XBLTokenChain | None:
        """Build chain."""
        if not self._ms_access_token:
            return None
        access_token = self._ms_access_token
        try:
            result = await (
                asyncio.get_event_loop().run_in_executor(
                    None,
                    lambda: build_xbl_chain(
                        access_token,
                        self._locale_fn(),
                        xbl_auth_url=self._config.xbl_auth_url,
                        xsts_url=self._config.xsts_url,
                        xbl_user_agent=(
                            self._config.xbl_user_agent
                        ),
                    ),
                )
            )
        except Exception as e:
            logger.error(
                "[MicrosoftTokens] XBL chain error: %s", e,
            )
            return None
        if not result:
            return None
        return XBLTokenChain(
            xsts_token=result["xsts_token"],
            user_hash=result["user_hash"],
            xuid=result.get("xuid"),
            xbl_token=result.get("xbl_token"),
        )

```

```

async def build_gssv_chain(
    self,
    xbl_token: str | None = None,
) -> XBLTokenChain | None:

    """Build gssv chain."""
    relying_party = self._config.gssv_relying_party
    if xbl_token:
        return await self._gssv_from_xbl_token(
            xbl_token, relying_party,
        )
    return await self._gssv_from_scratch(relying_party)
async def _gssv_from_xbl_token(
    self, xbl_token: str, relying_party: str,
) -> XBLTokenChain | None:
    """Gssv from XBL token."""
    loop = asyncio.get_event_loop()
    try:
        resp = await loop.run_in_executor(
            None,
            lambda: request_xsts_token(
                xbl_token=xbl_token,
                xsts_rp=relying_party,
                locale=self._locale_fn(),
                xsts_url=self._config.xsts_url,
                xbl_user_agent=self._config.xbl_user_agent,
            ),
        )
    except Exception as e:
        logger.error(
            "[MicrosoftTokens] GSSV XSTS error: %s", e,
        )
        return None
    if not resp or "XErr" in resp:
        return None
    xsts_token = resp.get("Token")
    if not xsts_token:
        return None
    claims = resp.get("DisplayClaims", {}).get("xui", [{}])
    user_hash = claims[0].get("uhs") if claims else None
    if not user_hash:
        return None
    return XBLTokenChain(
        xsts_token=xsts_token,
        user_hash=user_hash,
        xuid=claims[0].get("xid") if claims else None,
        xbl_token=xbl_token,
    )
async def _gssv_from_scratch(
    self, relying_party: str,
) -> XBLTokenChain | None:
    """Gssv from scratch."""
    if not self._ms_access_token:
        return None
    access_token = self._ms_access_token
    try:
        result = await (
            asyncio.get_event_loop().run_in_executor(
                None,
                lambda: build_xbl_chain(

```

```
        access_token,
        self._locale_fn(),
        xbl_auth_url=self._config.xbl_auth_url,
        xsts_url=self._config.xsts_url,
        xbl_user_agent=(
            self._config.xbl_user_agent
        ),
        xsts_relying_party=relying_party,
    ),
)
)
except Exception as e:
    logger.error(
        "[MicrosoftTokens] GSSV chain error: %s", e,
    )
    return None
if not result:
    return None
return XBLTokenChain(
    xsts_token=result["xsts_token"],
    user_hash=result["user_hash"],
    xuid=result.get("xuid"),
    xbl_token=result.get("xbl_token"),
)
```

OP-53a

microsoft_store.py

Fichier : py_modules/unifideck/stores/microsoft/microsoft_store.py | Lignes : 343 | Depend de : (rien)

```

from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING, Any, cast
from ...auth.browser import OAuthBrowserMonitor
from ...auth.edge_browser import EdgeBrowser
from ...auth.orchestrator import AuthOrchestrator
from ...core.types import (
    AuthResult,
    Events,
    Game,
    InstallResult,
    Result,
    StoreInfo,
)
from ...services.shortcut import ShortcutService
from ...utils.locale import get_unifideck_locale
from ..shared.store_base import StoreBase
from .microsoft_browser_auth import MicrosoftBrowserAuth
from .microsoft_catalog import MicrosoftCatalogReader
from .microsoft_config import MicrosoftConfig
from .tokens import MicrosoftTokenManager
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...core.cache_manager import CacheManager
    from ...event_bus.event_bus import EventBus
    from ...services.microsoft_subscription import (
        MicrosoftSubscriptionService,
    )
logger = logging.getLogger(__name__)
class MicrosoftStore(StoreBase):
    """Microsoft store."""
    store_info = StoreInfo(
        name="microsoft",
        display_name="Microsoft",
        auth_method="oauth",
        icon_asset="microsoft.png",
        uses_wine=False,
        supports_install=False,
    )

    def __init__(
        self,
        bus: EventBus,
        cache: CacheManager,
        plugin_dir: str | None = None,
        config: ConfigManager | None = None,
        browser_monitor: OAuthBrowserMonitor | None = None,
        shortcut_service: ShortcutService | None = None,
        edge_browser: EdgeBrowser | None = None,
        subscription_service: MicrosoftSubscriptionService | None = None,
    ) -> None:

        """Initialize the instance."""
        super().__init__(bus, cache, plugin_dir, config)

```



```

self._ms_config: MicrosoftConfig = (
    MicrosoftConfig.from_config_manager(config)
)
logger.info(
    "[MicrosoftStore] %s",
    self._ms_config.describe(),
)
self._config_manager = config
self._shortcut_service = shortcut_service
self._edge = edge_browser
self._subscription_service = subscription_service
self._tokens = MicrosoftTokenManager(
    config=self._ms_config,
    locale_fn=lambda: get_unifideck_locale(
        self._config_manager,
    ),
    bus=bus,
)
self._catalog = MicrosoftCatalogReader(
    config=self._ms_config,
    config_manager=self._config_manager,
)
if browser_monitor is not None:
    orchestrator = AuthOrchestrator(
        bus=bus,
        browser_monitor=browser_monitor,
        store_name="microsoft",
    )
    self._auth: MicrosoftBrowserAuth | None = (
        MicrosoftBrowserAuth(
            bus=bus,
            orchestrator=orchestrator,
            tokens=self._tokens,
            config=self._ms_config,
            config_manager=self._config_manager,
        )
    )
else:
    self._auth = None
async def is_available(self) -> bool:
    """Check whether available."""
    if not self._ms_config.is_valid():
        self._cached_available = False
        return False
    loaded = await self._tokens.load()
    self._cached_available = loaded
    return loaded

async def start_auth(self, **kwargs) -> AuthResult:
    """Start auth."""
    if self._auth is None:
        return AuthResult(
            success=False,
            error="auth_not_configured",
            store="microsoft",
        )
    if self._edge is None or not self._edge.is_installed:
        logger.info(
            "[MicrosoftStore] Edge not installed - "
            "prompting user to install",

```

```

    )
    return AuthResult(
        success=False,
        error="edge_not_installed",
        store="microsoft",
        url=None,
        metadata={"needs_2fa": False},
    )
    EdgeBrowser.ensure_controller_permissions()
    await self._ensure_auth_shortcut()
    return cast("AuthResult", await self._auth.start_auth())
async def complete_auth(
    self, code: str = "", **kwargs,
) -> AuthResult:
    """Complete auth."""
    if await self.is_available():
        return AuthResult(success=True, store="microsoft")
    return AuthResult(
        success=False,
        error="not_authenticated",
        store="microsoft",
    )
async def logout(self) -> Result:
    """Logout."""
    if self._auth is not None:
        result = await self._auth.logout()
    else:
        await self._tokens.clear()
        await self._bus.emit(
            Events.STORE_LOGOUT, store="microsoft",
        )
        result = Result(success=True)
    auth_url_file = (
        Path("~/local/share/unifideck/ms_auth_url.txt")
        .expanduser()
    )
    if auth_url_file.is_file():
        try:
            auth_url_file.unlink()
        except OSError as e:
            logger.warning(
                "[MicrosoftStore] could not remove %s: "
                "%s",
                auth_url_file, e,
            )
    if self._edge is not None:
        try:
            self._edge.kill()
            self._edge.clear_cookies()
            EdgeBrowser.clear_profile_data()
        except Exception as e:
            logger.warning(
                "[MicrosoftStore] Edge cleanup error: "
                "%s", e,
            )
    return result

async def get_library(self) -> list[Game] | None:

    """Get library."""
    if not await self.is_available():

```

```

        logger.info(
            "[MicrosoftStore] not authenticated; "
            "returning empty library",
        )
        return []
    fresh = await self._tokens.refresh_if_stale()
    if not fresh:
        logger.error(
            "[MicrosoftStore] token refresh failed; "
            "session is dead",
        )
        await self._tokens.clear()
        return []
    if not await self._check_subscription_gate():
        return []
    chain = await self._tokens.build_chain()
    if chain is None:
        logger.warning(
            "[MicrosoftStore] XBL chain build failed – "
            "proceeding with catalog fetch anyway",
        )
    try:
        return await self._catalog.fetch_games()
    except Exception as e:
        logger.error(
            "[MicrosoftStore] get_library failed: %s", e,
        )
        return []

async def _check_subscription_gate(self) -> bool:
    """Check subscription gate."""
    if self._subscription_service is None:
        logger.debug(
            "[MicrosoftStore] no subscription_service "
            "wired – skipping subscription gate (legacy "
            "behaviour)",
        )
        return True
    from ...core.types import SubscriptionTier
    try:
        tier = await self._subscription_service.get_tier(
            self._tokens,
        )
    except Exception as e:
        logger.warning(
            "[MicrosoftStore] subscription check raised: "
            "%s – skipping sync", e,
        )
        await self._bus.emit(
            Events.SYNC_SKIPPED,
            store="microsoft",
            reason="subscription_check_error",
        )
        return False
    if tier == SubscriptionTier.NONE:
        logger.info(
            "[MicrosoftStore] no active xCloud "
            "subscription – skipping sync",
        )
        await self._bus.emit(

```

```

        Events.SYNC_SKIPPED,
        store="microsoft",
        reason="no_active_subscription",
    )
    return False
if tier == SubscriptionTier.ACTIVE_UNKNOWN:
    logger.warning(
        "[MicrosoftStore] subscription active but "
        "tier unknown – skipping sync pending "
        "capture data",
    )
    await self._bus.emit(
        Events.SYNC_SKIPPED,
        store="microsoft",
        reason="subscription_tier_unknown",
    )
    return False
logger.info(
    "[MicrosoftStore] active subscription detected "
    "(tier=%s) – fetching catalog",
    tier.value,
)
return True
async def install_game(
    self,
    game_id: str,
    base_path: str | None = None,
    progress_cb: Any = None,
    **kwargs: Any,
) -> InstallResult:
    """Install game."""
    return InstallResult(
        success=True,
        store="microsoft",
        game_id=game_id,
        install_path=None,
    )
async def uninstall_game(
    self, game_id: str, **kwargs: Any,
) -> Result:
    """Uninstall game."""
    return Result(success=True)

async def update_game(
    self,
    game_id: str,
    progress_cb: Any = None,
    **kwargs: Any,
) -> InstallResult:
    """Update game."""
    return InstallResult(
        success=True,
        store="microsoft",
        game_id=game_id,
    )
async def check_for_updates(self) -> list[str]:
    """Check for updates."""
    return []
async def get_game_size(
    self, game_id: str,

```

```

) -> int | None:
    """Get game size."""
    return None
async def install_edge(self) -> Result:
    """Install edge."""
    if self._edge is None:
        return Result(
            success=False,
            error="edge_browser_not_configured",
        )
    raw = await self._edge.install()
    return Result(
        success=bool(raw.get("success")),
        error=raw.get("error"),
    )
def is_edge_installed(self) -> bool:
    """Check whether edge installed."""
    return (
        self._edge is not None
        and self._edge.is_installed
    )
async def _ensure_auth_shortcut(self) -> None:
    """Ensure auth shortcut."""
    if self._shortcut_service is None:
        logger.debug(
            "[MicrosoftStore] no shortcut_service "
            "injected; skipping auth shortcut creation",
        )
        return
    launcher = str(
        Path(self._plugin_dir or "")
        / "py_modules" / "unifideck" / "launcher"
        / "dispatcher.py",
    )
    if not Path(launcher).is_file():
        logger.warning(
            "[MicrosoftStore] launcher dispatcher not "
            "found at %s",
            launcher,
        )
        return
    result = await (
        self._shortcut_service.add_auth_shortcut(
            store="microsoft",
            launcher_path=launcher,
            title="Microsoft Sign-In",
        )
    )
    if not result.success:
        logger.warning(
            "[MicrosoftStore] add_auth_shortcut "
            "failed: %s",
            result.error,
        )

```

OP-53b

microsoft_catalog.py

Fichier : py_modules/unifideck/stores/microsoft/microsoft_catalog.py | Lignes : 208 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import Any
from urllib.parse import urlencode
from ...core.types import Game, GameTag
from ...utils.locale import (
    get_unifideck_locale,
    get_unifideck_market,
)
from .microsoft_auth import http_get
from .microsoft_config import MicrosoftConfig
logger = logging.getLogger(__name__)
_TITLE_BATCH_SIZE = 20
class MicrosoftCatalogReader:
    """Microsoft catalog reader."""
    def __init__(
        self,
        config: MicrosoftConfig,
        config_manager: Any,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._config_manager = config_manager
    async def fetch_games(self) -> list[Game]:
        """Fetch games."""
        product_ids = await self._fetch_catalog_ids()
        if not product_ids:
            logger.warning(
                "[MicrosoftCatalog] catalog is empty or "
                "unreachable",
            )
            return []
        titles = await self._batch_get_titles(product_ids)
        games: list[Game] = [
            Game(
                app_id=0,
                store="microsoft",
                store_game_id=pid,
                title=titles.get(pid, pid),
                installed=False,
                tags=[GameTag.XCLOUD],
            )
            for pid in product_ids
        ]
        logger.info(
            "[MicrosoftCatalog] built %d games (%d titles "
            "resolved)",
            len(games), len(titles),
        )
        return games

    async def _fetch_catalog_ids(self) -> list[str]:
        """Fetch catalog ids."""

```

```

    locale = get_unifideck_locale(self._config_manager)
    market = get_unifideck_market(self._config_manager)
    base_url = self._config.xcloud_catalog_url
    separator = "&" if "?" in base_url else "?"
    url = (
        f"{base_url}{separator}"
        f"{urlencode({'language': locale, 'market': market})}"
    )
    headers = {
        "User-Agent": self._config.catalog_user_agent,
    }
    try:
        data = await (
            asyncio.get_event_loop().run_in_executor(
                None, lambda: http_get(url, headers),
            )
        )
    except Exception as e:
        logger.error(
            "[MicrosoftCatalog] catalog fetch failed: "
            "%s", e,
        )
        return []
    if not isinstance(data, list):
        logger.warning(
            "[MicrosoftCatalog] catalog returned %s, "
            "not list",
            type(data).__name__,
        )
        return []
    ids = [
        item["id"]
        for item in data
        if isinstance(item, dict)
        and isinstance(item.get("id"), str)
        and item["id"]
    ]
    logger.info(
        "[MicrosoftCatalog] %d IDs from catalog",
        len(ids),
    )
    return ids

async def _batch_get_titles(
    self, product_ids: list[str],
) -> dict[str, str]:
    """Batch get titles."""
    if not product_ids:
        return {}
    locale = get_unifideck_locale(self._config_manager)
    market = get_unifideck_market(self._config_manager)
    base_url = self._config.xcloud_titles_url
    ua = self._config.catalog_user_agent
    result: dict[str, str] = {}
    total_batches = (
        (len(product_ids) + _TITLE_BATCH_SIZE - 1)
        // _TITLE_BATCH_SIZE
    )
    for i in range(0, len(product_ids), _TITLE_BATCH_SIZE):
        batch = product_ids[i: i + _TITLE_BATCH_SIZE]
        batch_result = await self._fetch_one_title_batch(
            batch, base_url, locale, market, ua,

```

```

        )
        result.update(batch_result)
    logger.info(
        "[MicrosoftCatalog] resolved %d/%d titles across "
        "%d batches",
        len(result), len(product_ids), total_batches,
    )
    return result

async def _fetch_one_title_batch(
    self,
    batch: list[str],
    base_url: str,
    locale: str,
    market: str,
    user_agent: str,
) -> dict[str, str]:

    """Fetch one title batch."""
    ids_param = ",".join(batch)
    separator = "&" if "?" in base_url else "?"
    params = {
        "bigIds": ids_param,
        "market": market,
        "languages": locale,
        "fieldsTemplate": "Browse",
    }
    url = f"{base_url}{separator}{urlencode(params)}"
    headers = {
        "Accept": "application/json",
        "User-Agent": user_agent,
        "MS-CV": "unifideck.xcloud",
    }
    try:
        data = await (
            asyncio.get_event_loop().run_in_executor(
                None, lambda: http_get(url, headers),
            )
        )
    except Exception as e:
        logger.warning(
            "[MicrosoftCatalog] title batch failed: %s",
            e,
        )
        return {}
    return self._extract_titles(data)

@staticmethod
def _extract_titles(data: Any) -> dict[str, str]:
    """Extract titles."""
    if not isinstance(data, dict):
        return {}
    products = data.get("Products")
    if not isinstance(products, list):
        return {}
    result: dict[str, str] = {}
    for product in products:
        entry = (
            MicrosoftCatalogReader
            ._extract_one_product_title(product)
        )
        if entry is not None:

```



```
        pid, title = entry
        result[pid] = title
    return result

@staticmethod
def _extract_one_product_title(
    product: Any,
) -> tuple[str, str] | None:
    """Extract one product title."""
    if not isinstance(product, dict):
        return None
    pid = product.get("ProductId")
    if not isinstance(pid, str) or not pid:
        return None
    localized = product.get("LocalizedProperties")
    if not isinstance(localized, list):
        return None
    title = (
        MicrosoftCatalogReader
        ._first_localized_title(localized)
    )
    if title is None:
        return None
    return pid, title

@staticmethod
def _first_localized_title(
    localized: list,
) -> str | None:
    """First localized title."""
    for loc in localized:
        if not isinstance(loc, dict):
            continue
        title = loc.get("ProductTitle")
        if isinstance(title, str) and title:
            return title
    return None
```

OP-53c

microsoft_browser_auth.py

Fichier : py_modules/unifideck/stores/microsoft/microsoft_browser_auth.py | Lignes : 85 | Depend de : (rien)

```
from __future__ import annotations
import logging
import urllib.parse
from typing import Any
from ...auth.orchestrator import AuthOrchestrator
from ...core.types import AuthResult, Events, Result
from ...event_bus.event_bus import EventBus
from ...security import audit_auth_flow
from ...utils.locale import get_unifideck_locale
from .microsoft_config import MicrosoftConfig
from .tokens import MicrosoftTokenManager
logger = logging.getLogger(__name__)
_MS_AUTH_URL_FILE = "~/.local/share/unifideck/ms_auth_url.txt"
class MicrosoftBrowserAuth:
    """Microsoft browser auth."""
    def __init__(
        self,
        bus: EventBus,
        orchestrator: AuthOrchestrator,
        tokens: MicrosoftTokenManager,
        config: MicrosoftConfig,
        config_manager: Any,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._orch = orchestrator
        self._tokens = tokens
        self._config = config
        self._config_manager = config_manager
    @audit_auth_flow(store="microsoft", method="oauth_browser")
    async def start_auth(self) -> AuthResult:
        """Start auth."""
        if not self._config.is_valid():
            return AuthResult(
                success=False,
                error="config_invalid",
                store="microsoft",
            )
        return await self._orch.run_flow(
            get_url=self._build_auth_url,
            allowed_uris=list(
                self._config.allowed_redirect_uris,
            ),
            exchange_code=self._exchange_code,
            background=True,
            write_url_file=_MS_AUTH_URL_FILE,
        )
    async def _build_auth_url(self) -> str:
        """Build auth URL."""
        locale = get_unifideck_locale(self._config_manager)
        params = {
            "client_id": self._config.client_id,
            "redirect_uri": self._config.redirect_uri,
            "response_type": "code",
            "scope": self._config.scope,
```

```
        "ui_locales": locale,
    }
    query = urllib.parse.urlencode(params, safe="/: ")
    url = f"{self._config.auth_url}?{query}"
    logger.info(
        "[MicrosoftBrowserAuth] built OAuth URL (locale=%s)",
        locale,
    )
    return url

async def _exchange_code(self, code: str) -> AuthResult:

    """Exchange code."""
    ok = await self._tokens.exchange_code(code)
    if ok:
        logger.info(
            "[MicrosoftBrowserAuth] token exchange successful",
        )
        return AuthResult(success=True, store="microsoft")
    return AuthResult(
        success=False,
        error="token_exchange_failed",
        store="microsoft",
    )

async def logout(self) -> Result:
    """Logout."""
    self._orch.cancel_background()
    await self._tokens.clear()
    await self._bus.emit(
        Events.STORE_LOGOUT, store="microsoft",
    )
    return Result(success=True)
```

OP-53d

microsoft_config.py

Fichier : py_modules/unifideck/stores/microsoft/microsoft_config.py | Lignes : 125 | Depend de : (rien)

```

from __future__ import annotations
import logging
from dataclasses import dataclass, field
from typing import TYPE_CHECKING
from unifideck.utils.config_helpers import get_cfg
if TYPE_CHECKING:
    from ...config import ConfigManager
logger = logging.getLogger(__name__)
_MS_CONFIG_PREFIX = "stores.microsoft"
_DEFAULT_TOKEN_FILE = "~/.local/share/unifideck/microsoft_tokens.json"
@dataclass(frozen=True)
class MicrosoftConfig:
    """Microsoft config."""
    client_id: str = ""
    scope: str = ""
    auth_url: str = ""
    token_url: str = ""
    redirect_uri: str = ""
    allowed_redirect_uris: list[str] = field(default_factory=list)
    xbl_auth_url: str = ""
    xsts_url: str = ""
    xcloud_catalog_url: str = ""
    xcloud_titles_url: str = ""
    xcloud_launch_url: str = ""
    gssv_relying_party: str = "http://gssv.xboxlive.com/"
    subscription_check_url: str = (
        "https://xgpuweb.gssv-play-prod.xboxlive.com/v2/login/user"
    )
    token_file: str = _DEFAULT_TOKEN_FILE
    token_refresh_threshold_seconds: int = 2400
    xbl_user_agent: str = "XboxReplay; XboxLiveAuth/3.0"
    catalog_user_agent: str = "Unifideck/1.0"

    @classmethod
    def from_config_manager(cls, config: ConfigManager | None) -> MicrosoftConfig:
        """From config manager."""
        def _s(key: str, default: str = "") -> str:
            """S."""
            val = get_cfg(config, f"{_MS_CONFIG_PREFIX}.{key}", default)
            return str(val).strip() if val is not None else default
        def _i(key: str, default: int) -> int:
            """I."""
            val = get_cfg(config, f"{_MS_CONFIG_PREFIX}.{key}", default)
            try:
                return int(val)
            except (TypeError, ValueError):
                return default
        def _list(key: str) -> list[str]:
            """List."""
            val = get_cfg(config, f"{_MS_CONFIG_PREFIX}.{key}", None)
            if not isinstance(val, list):
                return []
            return [str(x) for x in val if isinstance(x, str) and x]
        primary_redirect = _s("redirect_uri")
        allowed = _list("allowed_redirect_uris")

```

```

if not allowed and primary_redirect:
    allowed = [primary_redirect]
return cls(
    client_id=_s("client_id"),
    scope=_s("scope"),
    auth_url=_s("auth_url"),
    token_url=_s("token_url"),
    redirect_uri=primary_redirect,
    allowed_redirect_uris=allowed,
    xbl_auth_url=_s("xbl_auth_url"),
    xsts_url=_s("xsts_url"),
    xcloud_catalog_url=_s("xcloud_catalog_url"),
    xcloud_titles_url=_s("xcloud_titles_url"),
    xcloud_launch_url=_s("xcloud_launch_url"),
    gssv_relying_party=_s(
        "gssv_relying_party", "http://gssv.xboxlive.com/",
    ),
    subscription_check_url=_s(
        "subscription_check_url",
        "https://xgpuweb.gssv-play-prod.xboxlive.com/v2/login/user",
    ),
    token_file=_s("token_file", _DEFAULT_TOKEN_FILE),
    token_refresh_threshold_seconds=_i(
        "token_refresh_threshold_seconds", 2400,
    ),
    xbl_user_agent=_s(
        "xbl_user_agent",
        "XboxReplay; XboxLiveAuth/3.0",
    ),
    catalog_user_agent=_s(
        "catalog_user_agent", "Unifideck/1.0",
    ),
)

def is_valid(self) -> bool:

    """Check whether valid."""
    required = (
        self.client_id,
        self.scope,
        self.auth_url,
        self.token_url,
        self.redirect_uri,
        self.xbl_auth_url,
        self.xsts_url,
        self.xcloud_catalog_url,
        self.xcloud_launch_url,
    )
    missing = [
        name for name, val in zip(
            (
                "client_id", "scope", "auth_url", "token_url",
                "redirect_uri", "xbl_auth_url", "xsts_url",
                "xcloud_catalog_url", "xcloud_launch_url",
            ),
            required, strict=False,
        )
        if not val
    ]
    if missing:
        logger.warning(

```

```
        "[MicrosoftConfig] missing required keys: %s",
        ", ".join(missing),
    )
    return False
return True
def describe(self) -> str:
    """Describe."""
    return (
        f"MicrosoftConfig(client_id={self.client_id[:6]}..., "
        f"scope={self.scope!r}, "
        f"token_file={self.token_file})"
    )
```

OP-53e

microsoft_auth.py

Fichier : py_modules/unifideck/stores/microsoft/microsoft_auth.py | Lignes : 231 | Depend de : (rien)

```

import json
import logging
import urllib.error
import urllib.parse
import urllib.request
from typing import cast
from ...core.net import ssl_ctx_strict
logger = logging.getLogger(__name__)
__all__ = [
    "build_xbl_chain",
    "http_get",
    "http_post",
    "request_xsts_token",
    "ssl_ctx_strict",
]
def http_post(url: str, data: dict, headers: dict) -> dict:
    """Http post."""
    body = urllib.parse.urlencode(data).encode()
    req = urllib.request.Request(
        url, data=body, headers=headers, method="POST")
    with urllib.request.urlopen(req, timeout=15, context=ssl_ctx_strict()) as r:
        return cast(dict, json.loads(r.read().decode()))
def http_post_json(url: str, payload: dict, headers: dict) -> dict:
    """Http post JSON."""
    body = json.dumps(payload).encode()
    req = urllib.request.Request(
        url, data=body, headers=headers, method="POST")
    with urllib.request.urlopen(req, timeout=20, context=ssl_ctx_strict()) as r:
        return cast(dict, json.loads(r.read().decode()))
def http_get(url: str, headers: dict) -> dict:
    """Http get."""
    req = urllib.request.Request(url, headers=headers)
    with urllib.request.urlopen(req, timeout=15, context=ssl_ctx_strict()) as r:
        return cast(dict, json.loads(r.read().decode()))

def build_xbl_chain(
    access_token: str,
    locale: str,
    xbl_auth_url: str,
    xsts_url: str,
    xbl_user_agent: str,
    xsts_relying_party: str = "http://xboxlive.com",
) -> dict[str, str] | None:

    """Build XBL chain."""
    logger.info("[MS] Building XBL/XSTS token chain")
    try:
        xbl_resp = _obtain_xbl_user_token(
            access_token, locale, xbl_auth_url, xbl_user_agent,
        )
        if xbl_resp is None:
            return None
        xbl_token = xbl_resp["Token"]
        user_hash = _extract_user_hash(xbl_resp)
        logger.info(

```

```

        "[MS] ✓ XBL user token obtained (uhs=%s)", user_hash,
    )
    xsts_rp = xsts_relying_party
    xsts_resp = _request_xsts_token(
        xbl_token, xsts_rp, locale, xsts_url, xbl_user_agent,
    )
    if xsts_resp is None:
        return None
    if "XErr" in xsts_resp:
        _log_xsts_xerr(xsts_resp["XErr"])
        return None
    xsts_token = xsts_resp.get("Token")
    if not xsts_token:
        logger.error(
            "[MS] XSTS token missing: %s", xsts_resp,
        )
        return None
    xsts_claims = xsts_resp.get(
        "DisplayClaims", {}
    ).get("xui", [{}])
    xuid = (
        xsts_claims[0].get("xid") if xsts_claims else None
    )
    logger.info(
        "[MS] ✓ XSTS token obtained (xuid=%s)", xuid,
    )
    return {
        "xbl_token": xbl_token,
        "user_hash": user_hash,
        "xsts_token": xsts_token,
        "xsts_rp": xsts_rp,
        "xuid": xuid,
    }
except Exception as e:
    logger.exception(
        "[MS] XBL chain error: %s", e,
    )
    return None

def request_xsts_token(
    xbl_token: str,
    xsts_rp: str,
    locale: str,
    xsts_url: str,
    xbl_user_agent: str,
) -> dict | None:
    """Request XSTS token."""
    return _request_xsts_token(
        xbl_token, xsts_rp, locale, xsts_url, xbl_user_agent,
    )

def _obtain_xbl_user_token(
    access_token: str,
    locale: str,
    xbl_auth_url: str,
    xbl_user_agent: str,
) -> dict | None:
    """Obtain XBL user token."""
    candidates = [
        ("2", f"t={access_token}"),
        ("1", f"d={access_token}"),
    ]

```



```

        ("1", f"t={access_token}"),
    ]
    for contract_v, rps in candidates:
        resp = _try_xbl_request(
            contract_v, rps, locale, xbl_auth_url, xbl_user_agent,
        )
        if resp is not None and resp.get("Token"):
            logger.info(
                "[MS] XBL auth OK (contract-v%s, prefix=%r)",
                contract_v, rps[:2],
            )
            return resp
        logger.error(
            "[MS] XBL user token failed with all contract/prefix combos",
        )
    return None
def _try_xbl_request(
    contract_v: str,
    rps: str,
    locale: str,
    xbl_auth_url: str,
    xbl_user_agent: str,
) -> dict | None:
    """Try XBL request."""
    body = {
        "Properties": {
            "AuthMethod": "RPS",
            "SiteName": "user.auth.xboxlive.com",
            "RpsTicket": rps,
        },
        "RelyingParty": "http://auth.xboxlive.com",
        "TokenType": "JWT",
    }
    headers = {
        "Content-Type": "application/json",
        "Accept": "application/json",
        "x-xbl-contract-version": contract_v,
        "User-Agent": xbl_user_agent,
        "Accept-Language": locale,
    }
    try:
        return http_post_json(xbl_auth_url, body, headers)
    except urllib.error.HTTPError as e:
        body_text = _read_http_error_body(e)
        logger.debug(
            "[MS] XBL failed (v%s, %r): HTTP %d %s",
            contract_v, rps[:2], e.code, body_text[:500],
        )
        return None
    except Exception as e:
        logger.debug(
            "[MS] XBL failed (v%s, %r): %s",
            contract_v, rps[:2], e,
        )
        return None
def _extract_user_hash(xbl_resp: dict) -> str | None:
    """Extract user hash."""
    display_claims = xbl_resp.get("DisplayClaims", {})
    xui = display_claims.get("xui", [{}])
    return xui[0].get("uhs") if xui else None

```

```

def _request_xsts_token(
    xbl_token: str,
    xsts_rp: str,
    locale: str,
    xsts_url: str,
    xbl_user_agent: str,
) -> dict | None:

    """Request XSTS token."""
    headers = {
        "Content-Type": "application/json",
        "Accept": "application/json",
        "x-xbl-contract-version": "1",
        "User-Agent": xbl_user_agent,
        "Accept-Language": locale,
    }
    body = {
        "Properties": {
            "SandboxId": "RETAIL",
            "UserTokens": [xbl_token],
        },
        "RelyingParty": xsts_rp,
        "TokenType": "JWT",
    }
    try:
        resp = http_post_json(xsts_url, body, headers)
        logger.info(
            "[MS] ✓ XSTS obtained with RP=%r sandbox='RETAIL'",
            xsts_rp,
        )
        return resp
    except urllib.error.HTTPError as e:
        body_text = _read_http_error_body(e)
        logger.warning(
            "[MS] XSTS failed (RP=%r): HTTP %d %s",
            xsts_rp, e.code, body_text[:500],
        )
        return None
    except Exception as e:
        logger.warning(
            "[MS] XSTS failed (RP=%r): %s", xsts_rp, e,
        )
        return None

def _log_xsts_xerr(xerr: int) -> None:
    """Log XSTS xerr."""
    logger.error("[MS] XSTS error code: %d", xerr)
    if xerr == 2148916238:
        logger.error(
            "[MS] Account has no Xbox profile – create one at xbox.com",
        )
    elif xerr == 2148916233:
        logger.error(
            "[MS] Account is from a country where Xbox is not available",
        )

def _read_http_error_body(err: urllib.error.HTTPError) -> str:
    """Read http error body."""
    try:
        return err.read().decode("utf-8", errors="replace")
    except Exception:
        return ""

```

OP-53f

microsoft_subscription.py

Fichier : py_modules/unifideck/stores/microsoft/microsoft_subscription.py | Lignes : 162 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import json
import logging
import urllib.error
import urllib.request
from dataclasses import dataclass
from typing import Any
from ...core.types import SubscriptionTier
from .microsoft_auth import ssl_ctx_strict
logger = logging.getLogger(__name__)
_PROBE_TIMEOUT_SECONDS = 10
_GSSV_CLIENT_HEADER = "XboxComBrowser"
@dataclass(frozen=True)
class SubscriptionProbeResult:
    """Subscription probe result."""
    tier: SubscriptionTier
    ok: bool
    error: str | None = None
    http_status: int | None = None
    async def probe_subscription(
        user_hash: str,
        gssv_xsts_token: str,
        endpoint_url: str,
        timeout_seconds: int = _PROBE_TIMEOUT_SECONDS,
    ) -> SubscriptionProbeResult:
        """Probe subscription."""
        loop = asyncio.get_event_loop()
        return await loop.run_in_executor(
            None,
            lambda: _probe_sync(
                user_hash, gssv_xsts_token, endpoint_url, timeout_seconds,
            ),
        )

def _probe_sync(
    user_hash: str,
    gssv_xsts_token: str,
    endpoint_url: str,
    timeout_seconds: int,
) -> SubscriptionProbeResult:
    """Probe sync."""
    headers = {
        "Authorization": f"XBL3.0 x={user_hash};{gssv_xsts_token}",
        "Content-Type": "application/json",
        "Accept": "application/json",
        "Cache-Control": "no-store, must-revalidate, no-cache",
        "x-gssv-client": _GSSV_CLIENT_HEADER,
    }
    req = urllib.request.Request(
        endpoint_url, data=b"", headers=headers, method="POST",
    )
    http_result = _do_probe_http(req, timeout_seconds, endpoint_url)
    if isinstance(http_result, SubscriptionProbeResult):

```

```

        return http_result
    status, raw = http_result
    try:
        parsed = json.loads(raw)
    except ValueError as e:
        logger.warning(
            "[SubscriptionProbe] response is not JSON: %s", e,
        )
        return SubscriptionProbeResult(
            tier=SubscriptionTier.NONE,
            ok=False,
            error="bad_response",
            http_status=status,
        )
    tier = _parse_tier_from_response(parsed)
    logger.info(
        "[SubscriptionProbe] endpoint %s responded 200, tier=%s",
        endpoint_url, tier.value,
    )
    return SubscriptionProbeResult(
        tier=tier,
        ok=True,
        http_status=status,
    )

def _do_probe_http(
    req: urllib.request.Request,
    timeout_seconds: int,
    endpoint_url: str,
) -> tuple[int, str] | SubscriptionProbeResult:
    """Do probe http."""
    try:
        with urllib.request.urlopen(
            req,
            timeout=timeout_seconds,
            context=ssl_ctx_strict(),
        ) as resp:
            return resp.status, resp.read().decode(
                "utf-8", errors="replace",
            )
    except urllib.error.HTTPError as e:
        status = e.code
        if status in (401, 403, 404):
            return SubscriptionProbeResult(
                tier=SubscriptionTier.NONE,
                ok=True,
                http_status=status,
            )
        logger.warning(
            "[SubscriptionProbe] unexpected HTTP %d on %s",
            status, endpoint_url,
        )
        return SubscriptionProbeResult(
            tier=SubscriptionTier.NONE,
            ok=False,
            error="http_error",
            http_status=status,
        )
    except (TimeoutError, urllib.error.URLError) as e:
        logger.warning(

```

```

        "[SubscriptionProbe] network error: %s", e,
    )
    return SubscriptionProbeResult(
        tier=SubscriptionTier.NONE,
        ok=False,
        error="timeout" if isinstance(e, TimeoutError) else "network",
    )
except Exception as e:
    logger.warning(
        "[SubscriptionProbe] unexpected error: %s", e,
    )
    return SubscriptionProbeResult(
        tier=SubscriptionTier.NONE,
        ok=False,
        error="network",
    )
def _parse_tier_from_response(
    payload: dict[str, Any],
) -> SubscriptionTier:
    """Parse tier from response."""
    if not isinstance(payload, dict):
        return SubscriptionTier.NONE
    offering = payload.get("offeringSettings")
    if not isinstance(offering, dict):
        return SubscriptionTier.NONE
    regions = offering.get("regions")
    if not isinstance(regions, list) or len(regions) == 0:
        return SubscriptionTier.NONE
    for candidate_field in (
        "subscriptionTier", "tier", "offeringId"):
        value = payload.get(candidate_field)
        if value is None:
            value = offering.get(candidate_field)
        if isinstance(value, str):
            parsed = _match_tier_string(value)
            if parsed is not None:
                return parsed
    return SubscriptionTier.ACTIVE_UNKNOWN

def _match_tier_string(raw: str) -> SubscriptionTier | None:
    """Match tier string."""
    low = raw.lower()
    if "ultimate" in low or low == "xgpu" or low.startswith("xgpuweb"):
        if "f2p" in low:
            return SubscriptionTier.NONE
        return SubscriptionTier.ULTIMATE
    if "premium" in low:
        return SubscriptionTier.PREMIUM
    if "essential" in low or "core" in low:
        return SubscriptionTier.ESSENTIAL
    return None

```

OP-55

`__init__.py`

Fichier : `py_modules/unifideck/stores/ubisoft/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from .store import UbisoftStore
__all__ = ["UbisoftStore"]
```

OP-56

`__init__.py`

Fichier : `py_modules/unifideck/stores/ubisoft/installer/__init__.py` | Lignes : 3 | Depend de : (rien)

```
from .cache import UbisoftInstallerCache
from .installer import UbisoftInstaller
__all__ = ["UbisoftInstaller", "UbisoftInstallerCache"]
```

OP-56a

installer.py

Fichier : py_modules/unifideck/stores/ubisoft/installer/installer.py | Lignes : 288 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import os
import subprocess
from collections.abc import Awaitable, Callable
from typing import Any
from ...core.types import InstallResult, Result
from ..binaries import UbisoftBinaryResolver
from ..config import UbisoftConfig
from ..id_map import UbisoftIdMap
from ..library import UbisoftLibrary
from ..paths import UbisoftPrefixPaths
from ..session import UbisoftSession
from . import registry as _reg
from .launch_env import (
    UpcLaunchEnvBuildError,
    _UpcLaunchEnv,
)
from .launcher import _LauncherInstall
from .manual_ui import _ManualUiInstaller
from .registry import _ShortcutRegistry
from .uninstall import _UninstallPipeline
from .update_op import _UpdateOperation
logger = logging.getLogger(__name__)
_UPDATE_TIMEOUT_S = 4 * 60 * 60
class UbisoftInstaller:

    """Ubisoft installer."""
    def __init__(
        self,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
        binaries: UbisoftBinaryResolver,
        id_map: UbisoftIdMap,
        session: UbisoftSession,
        library: UbisoftLibrary,
        bootstrap_game_prefix: Callable[
            [str], Awaitable[bool],
        ],
    ) -> None:

        """Initialize the instance."""
        self._config = config
        self._paths = paths
        self._binaries = binaries
        self._id_map = id_map
        self._session = session
        self._library = library
        self._bootstrap_game_prefix = bootstrap_game_prefix
        self._shortcut_registry = _ShortcutRegistry(config)
        self._active_install_pids: dict[str, int] = {}
        self._manual_ui_installer = _ManualUiInstaller(
            config=config,
            library=library,
```



```

        id_map=id_map,
        session=session,
        active_install_pids=self._active_install_pids,
    )
    self._uninstall_pipeline = _UninstallPipeline(self)
    self._launcher = _LauncherInstall(self)
    self._update_op = _UpdateOperation(
        id_map=id_map,
        paths=paths,
        session=session,
        build_launch_env=self._build_upc_launch_env,
    )
    async def uninstall_game(
        self,
        game_id: str,
        *,
        delete_prefix: bool = False,
    ) -> Result:
        """Uninstall game."""
        return await self._uninstall_pipeline.uninstall_game(
            game_id, delete_prefix=delete_prefix,
        )
    async def open_launcher_for_install(
        self, game_id: str,
    ) -> Result:
        """Open launcher for install."""
        return await self._launcher.open_launcher_for_install(
            game_id,
        )

    def _build_upc_launch_env(
        self,
        game_id: str,
        prefix_path: str,
        *,
        prefer_connect_exe: bool = False,
        upc_missing_error: str = "upc_exe_not_found",
    ) -> _UpcLaunchEnv:
        """Build UPC launch env."""
        upc_path: str | None = None
        if prefer_connect_exe:
            upc_path = self._paths.find_connect_exe(prefix_path)
        if not upc_path:
            upc_path = self._paths.find_upc_exe(prefix_path)
        if not upc_path:
            raise UpcLaunchEnvBuildError(upc_missing_error)
        umu_run = self._binaries.find_umu_run()
        if not umu_run:
            raise UpcLaunchEnvBuildError("umu_run_not_found")
        python_bin = self._binaries.find_python()
        env = self._binaries.build_umu_env(
            wineprefix=prefix_path,
            gameid=f"umu-ubisoft-{game_id}",
            store_game_id=f"ubisoft:{game_id}",
            steam_window_env=self._build_steam_window_env(
                f"ubisoft:{game_id}",
            ),
        )
        return _UpcLaunchEnv(
            upc_path=upc_path,

```

```

        umu_run=umu_run,
        python_bin=python_bin,
        env=env,
    )

    async def install_game(
        self,
        game_id: str,
        *,
        progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None = None,
        install_path: str | None = None,
    ) -> InstallResult:
        """Install game."""
        try:
            logger.info(
                "[UbisoftInstaller] installing game %s",
                game_id,
            )
            if not await self._bootstrap_game_prefix(game_id):
                return InstallResult(
                    success=False,
                    store="ubisoft",
                    game_id=game_id,
                    error="prefix_bootstrap_failed",
                )
            prefix_path = self._paths.get_prefix_path(game_id)
            game_name = self._library._detector._get_game_name(game_id)
            try:
                launch_env = self._build_upc_launch_env(
                    game_id, prefix_path,
                )
            except UpcLaunchEnvBuildError as e:
                return InstallResult(
                    success=False,
                    store="ubisoft",
                    game_id=game_id,
                    error=e.error_code,
                )
            return await (
                self._manual_ui_installer.install_via_upc_ui(
                    game_id=game_id,
                    game_name=game_name,
                    prefix_path=prefix_path,
                    upc_path=launch_env.upc_path,
                    umu_run=launch_env.umu_run,
                    python_bin=launch_env.python_bin,
                    env=launch_env.env,
                    progress_cb=progress_cb,
                    install_path=install_path,
                )
            )
        except Exception as e:
            logger.exception(
                "[UbisoftInstaller] install error for "
                "%s: %s", game_id, e,
            )
            return InstallResult(
                success=False,
                store="ubisoft",
                game_id=game_id,
            )

```

```

        error=f"install_exception: {e}",
    )
def is_install_session_active(self, game_id: str) -> bool:
    """Check whether install session active."""
    pid = self._active_install_pids.get(game_id)
    if pid is None:
        return False
    try:
        os.kill(pid, 0)
        return True
    except (ProcessLookupError, PermissionError):
        self._active_install_pids.pop(game_id, None)
        return False

async def cancel_install_session(
    self, game_id: str,
) -> Result:

    """Check whether install session."""
    pid = self._active_install_pids.pop(game_id, None)
    if pid is not None:
        try:
            os.kill(pid, 15)
            logger.info(
                "[UbisoftInstaller] sent SIGTERM to UPC "
                "PID %d for %s", pid, game_id,
            )
        except ProcessLookupError:
            logger.info(
                "[UbisoftInstaller] install process "
                "already exited for %s", game_id,
            )
        except OSError as e:
            logger.error(
                "[UbisoftInstaller] kill failed: %s", e,
            )
    prefix_path = self._paths.get_prefix_path(game_id)
    if prefix_path and os.path.isdir(prefix_path):
        await asyncio.sleep(2)
        captured = self._session.capture(prefix_path)
        if captured:
            self._session.propagate_all_to_all()
            logger.info(
                "[UbisoftInstaller] post-cancel: "
                "propagated session from %s", game_id,
            )
        else:
            logger.info(
                "[UbisoftInstaller] post-cancel: "
                "credentials synced for %s", game_id,
            )
    return Result(success=True)
async def check_for_updates(self) -> list[str]:
    """Check for updates."""
    return []
async def update_game(
    self, game_id: str,
) -> InstallResult:
    """Update game."""
    return await self._update_op.update(game_id)
def inject_install_registry(

```

```

        self,
        prefix_path: str,
        install_id: str,
        install_dir: str,
    ) -> None:
        """Inject install registry."""
        _reg.inject_install_registry(
            prefix_path, install_id, install_dir,
        )
    def kill_upc_processes(self) -> None:
        """Kill UPC processes."""
        try:
            subprocess.run(
                ["pkill", "-f", "upc.exe"],
                capture_output=True,
                timeout=5,
            )
            logger.info(
                "[UbisoftInstaller] killed upc.exe processes",
            )
        except (OSError, subprocess.SubprocessError) as e:
            logger.warning(
                "[UbisoftInstaller] pkill upc.exe failed: %s",
                e,
            )

    def _build_steam_window_env(
        self, store_game_id: str | None,
    ) -> dict[str, str]:
        """Build steam window env."""
        appid = self._shortcut_registry.resolve_shortcut_appid(
            store_game_id,
        )
        if appid:
            encoded = str(
                (appid << 32) | 0x02000000,
            )
            logger.info(
                "[UbisoftInstaller] Steam window env: "
                "appid=%d store_game_id=%s",
                appid, store_game_id or "<none>",
            )
            appid_str = str(appid)
            return {
                "SteamGameId": appid_str,
                "STEAM_COMPAT_APP_ID": appid_str,
                "SteamAppId": appid_str,
                "UMU_STEAM_GAME_ID": encoded,
            }
        logger.info(
            "[UbisoftInstaller] Steam window env: no "
            "shortcut appid resolved, using 0",
        )
        return {
            "SteamGameId": "0",
            "STEAM_COMPAT_APP_ID": "0",
            "SteamAppId": "0",
            "UMU_STEAM_GAME_ID": "0",
        }

```

OP-56b

cache.py

Fichier : py_modules/unifideck/stores/ubisoft/installer/cache.py | Lignes : 114 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
import os
import urllib.request
from typing import Any
from ...core.net import ssl_ctx_strict
from ..config import UbisoftConfig
logger = logging.getLogger(__name__)
_INSTALLER_MIN_SIZE_BYTES = 1000
_PE_MAGIC = b"MZ"
_INSTALLER_DOWNLOAD_TIMEOUT_S = 600.0
_DOWNLOAD_CHUNK_SIZE = 1024 * 1024
class UbisoftInstallerCache:
    """Ubisoft installer cache."""
    def __init__(self, config: UbisoftConfig) -> None:
        """Initialize the instance."""
        self._config = config
    async def ensure_cached(self) -> str | None:
        """Ensure cached."""
        cache_dir = self._config.installer_cache_dir_expanded
        filename = self._config.installer_filename
        cached_path = os.path.join(cache_dir, filename)
        if self._is_cached_valid(cached_path):
            logger.info(
                "[UbisoftInstallerCache] using cached installer",
            )
            return cached_path
        logger.info(
            "[UbisoftInstallerCache] downloading installer "
            "from %s", self._config.installer_url,
        )
        try:
            os.makedirs(cache_dir, exist_ok=True)
        except OSError as e:
            logger.error(
                "[UbisoftInstallerCache] cache dir creation "
                "failed: %s", e,
            )
            return None
        success = await asyncio.to_thread(
            self._download_sync,
            self._config.installer_url,
            cached_path,
        )
        if not success:
            return None
        return cached_path

    @staticmethod
    def _is_cached_valid(cached_path: str) -> bool:
        """Is cached valid."""
        if not os.path.isfile(cached_path):
            return False
        try:
```

```

        if (
            os.path.getsize(cached_path)
            < _INSTALLER_MIN_SIZE_BYTES
        ):
            return False
        with open(cached_path, "rb") as f:
            header = f.read(2)
            return header == _PE_MAGIC
    except OSError:
        return False
@staticmethod
def _download_sync(url: str, dest_path: str) -> bool:
    """Download sync."""
    tmp_path = dest_path + ".tmp"
    try:
        ctx = ssl_ctx_strict()
        req = urllib.request.Request(
            url,
            headers={"User-Agent": "Unifideck/1.0"},
        )
        with urllib.request.urlopen(
            req,
            timeout=_INSTALLER_DOWNLOAD_TIMEOUT_S,
            context=ctx,
        ) as response:
            if response.status not in (200, 206):
                logger.error(
                    "[UbisoftInstallerCache] HTTP %d",
                    response.status,
                )
                return False
            total = _stream_to_file(response, tmp_path)
            os.replace(tmp_path, dest_path)
            logger.info(
                "[UbisoftInstallerCache] downloaded %.1f MB",
                total / (1024 * 1024),
            )
            return True
    except Exception as e:
        logger.error(
            "[UbisoftInstallerCache] download failed: %s",
            e,
        )
    if os.path.isfile(tmp_path):
        try:
            os.remove(tmp_path)
        except OSError:
            pass
    return False
def _stream_to_file(response: Any, path: str) -> int:
    """Stream to file."""
    total = 0
    with open(path, "wb") as f:
        while True:
            chunk = response.read(_DOWNLOAD_CHUNK_SIZE)
            if not chunk:
                break
            f.write(chunk)
            total += len(chunk)
    return total

```

OP-56c

launch_env.py

Fichier : py_modules/unifideck/stores/ubisoft/installer/launch_env.py | Lignes : 15 | Depend de : (rien)

```
from __future__ import annotations
from dataclasses import dataclass
@dataclass
class _UpcLaunchEnv:
    """Upc launch env."""
    upc_path: str
    umu_run: str
    python_bin: str
    env: dict[str, str]
class UpcLaunchEnvBuildError(Exception):
    """Upc launch env build error."""
    def __init__(self, error_code: str) -> None:
        """Initialize the instance."""
        super().__init__(error_code)
        self.error_code = error_code
```

OP-56d

launcher.py

Fichier : py_modules/unifideck/stores/ubisoft/installer/launcher.py | Lignes : 155 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import TYPE_CHECKING
from ...core.types import Result
from .launch_env import UpcLaunchEnvBuildError
if TYPE_CHECKING:
    from .installer import UbisoftInstaller
logger = logging.getLogger(__name__)
class _LauncherInstall:
    """Launcher install."""
    def __init__(self, parent: UbisoftInstaller) -> None:
        """Initialize the instance."""
        self._parent = parent

    async def open_launcher_for_install(
        self, game_id: str,
    ) -> Result:
        """Open launcher for install."""
        try:
            logger.info(
                "[UbisoftInstaller] open_launcher_for_install "
                "for %s", game_id,
            )
            if not await self._parent._bootstrap_game_prefix(game_id):
                return Result(
                    success=False,
                    error="prefix_bootstrap_failed",
                )
            prefix_path = self._parent._paths.get_prefix_path(
                game_id,
            )
            try:
                launch_env = self._parent._build_upc_launch_env(
                    game_id,
                    prefix_path,
                    prefer_connect_exe=True,
                    upc_missing_error="ubisoft_connect_not_found",
                )
            except UpcLaunchEnvBuildError as e:
                return Result(success=False, error=e.error_code)
            self._parent._session.inject_into_prefix(prefix_path)
            launch_id = self._parent._id_map.resolve_launch_id(
                game_id,
            )
            launch_url = (
                f"uplay://install/{launch_id}"
                if launch_id else ""
            )
            cmd = [
                launch_env.python_bin,
                launch_env.umu_run,
                launch_env.upc_path,
            ]

```



```

        if launch_url:
            cmd.append(launch_url)
        return await self._spawn_and_monitor_upc(
            cmd, launch_env.env, game_id, prefix_path,
        )
    except Exception as e:
        logger.exception(
            "[UbisoftInstaller] launcher spawn failed for "
            "%s: %s", game_id, e,
        )
        return Result(
            success=False,
            error=f"launcher_spawn_exception: {e}",
        )

async def _spawn_and_monitor_upc(
    self,
    cmd: list[str],
    env: dict[str, str],
    game_id: str,
    prefix_path: str,
) -> Result:

    """Spawn and monitor UPC."""
    logger.info(
        "[UbisoftInstaller] launch cmd: %s", " ".join(cmd),
    )
    logger.info(
        "[UbisoftInstaller] WINEPREFIX=%s "
        "PROTONPATH=%s GAMEID=%s",
        env.get("WINEPREFIX"),
        env.get("PROTONPATH"),
        env.get("GAMEID"),
    )
    proc = await asyncio.create_subprocess_exec(
        *cmd,
        env=env,
        stdout=asyncio.subprocess.DEVNULL,
        stderr=asyncio.subprocess.PIPE,
    )
    logger.info(
        "[UbisoftInstaller] spawned PID=%d", proc.pid,
    )
    self._parent._active_install_pids[game_id] = proc.pid
    spawned_pid = proc.pid
    asyncio.create_task(
        self.monitor_after_exit(
            game_id, spawned_pid, proc, prefix_path,
        ),
    )
    await asyncio.sleep(2)
    if proc.returncode is None:
        return Result(success=True)
    logger.error(
        "[UbisoftInstaller] UPC exited immediately (rc=%d)",
        proc.returncode,
    )
    if (
        self._parent._active_install_pids.get(game_id)
        == spawned_pid
    ):

```

```
        self._parent._active_install_pids.pop(game_id, None)
    return Result(
        success=False,
        error=f"ubisoft_connect_exited_code_{proc.returncode}",
    )

async def monitor_after_exit(
    self,
    game_id: str,
    spawned_pid: int,
    proc: asyncio.subprocess.Process,
    prefix_path: str,
) -> None:

    """Monitor after exit."""
    try:
        _stdout, stderr = await proc.communicate()
        rc = proc.returncode
        logger.info(
            "[UbisoftInstaller] UPC exited (PID=%d, rc=%s)",
            spawned_pid, rc,
        )
        if stderr:
            stderr_text = stderr.decode(
                errors="replace",
            )[:2000]
            logger.info(
                "[UbisoftInstaller] UPC stderr: %s",
                stderr_text,
            )
    except Exception as e:
        logger.warning(
            "[UbisoftInstaller] monitor error: %s", e,
        )
    finally:
        if (
            self._parent._active_install_pids.get(game_id)
            == spawned_pid
        ):
            self._parent._active_install_pids.pop(
                game_id, None,
            )
            captured = self._parent._session.capture(prefix_path)
            if captured:
                self._parent._session.propagate_all_to_all()
```

OP-56e

manual_ui.py

Fichier : py_modules/unifideck/stores/ubisoft/installer/manual_ui.py | Lignes : 340 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
from collections.abc import Awaitable, Callable
from typing import Any
from ...core.types import InstallResult
from ..config import UbisoftConfig
from ..id_map import UbisoftIdMap
from ..library import UbisoftLibrary
from ..library.detection_helpers import looks_like_game_install
from ..session import UbisoftSession
from . import registry as _reg
logger = logging.getLogger(__name__)
_MANUAL_INSTALL_TIMEOUT_S = 2 * 60 * 60
_MANUAL_INSTALL_POLL_INTERVAL_S = 10.0
_STABILITY_WAIT_MAX_POLLS = 360
_STABILITY_POLL_INTERVAL_S = 10.0
_STABILITY_STABLE_THRESHOLD = 3
class _ManualUiInstaller:
    """Manual UI installer."""
    def __init__(
        self,
        config: UbisoftConfig,
        library: UbisoftLibrary,
        id_map: UbisoftIdMap,
        session: UbisoftSession,
        active_install_pids: dict[str, int],
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._library = library
        self._id_map = id_map
        self._session = session
        self._active_install_pids = active_install_pids

    async def install_via_upc_ui(
        self,
        *,
        game_id: str,
        game_name: str | None,
        prefix_path: str,
        upc_path: str,
        umu_run: str,
        python_bin: str,
        env: dict[str, str],
        progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,
        install_path: str | None,
    ) -> InstallResult:
        """Install via UPC UI."""
        logger.info(
            "[UbisoftInstaller] install_id unavailable for %s "
            "- launching UPC for manual install", game_id,
        )

```

```

self._session.inject_into_prefix(prefix_path)
install_base, dirs_before, upc_dirs_before = (
    self._snapshot_pre_install(install_path, prefix_path)
)
proc = await self._notify_and_spawn_upc(
    game_id=game_id,
    upc_path=upc_path,
    umu_run=umu_run,
    python_bin=python_bin,
    env=env,
    progress_cb=progress_cb,
)
install_dir = await self._poll_for_new_install(
    proc=proc,
    install_base=install_base,
    dirs_before=dirs_before,
    upc_dirs_before=upc_dirs_before,
    progress_cb=progress_cb,
)
await self._terminate_upc_gracefully(proc)
self._active_install_pids.pop(game_id, None)
self._capture_and_propagate_session(prefix_path)
if not install_dir:
    return InstallResult(
        success=False,
        store="ubisoft",
        game_id=game_id,
        error="no_install_detected",
    )
return await self._finalize_manual_install(
    game_id=game_id,
    game_name=game_name,
    install_dir=install_dir,
)

async def _notify_and_spawn_upc(
    self,
    *,
    game_id: str,
    upc_path: str,
    umu_run: str,
    python_bin: str,
    env: dict[str, str],
    progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,
) -> asyncio.subprocess.Process:

    """Notify and spawn UPC."""
    if progress_cb:
        await progress_cb({
            "status": "waiting",
            "message": (
                "Ubisoft Connect is opening. Please "
                "install the game from the UPC interface."
            ),
            "progress": 0,
        })
    logger.info(
        "[UbisoftInstaller] launching UPC for manual install",
    )
    proc = await asyncio.create_subprocess_exec(
        python_bin, umu_run, upc_path,

```

```

        env=env,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    self._active_install_pids[game_id] = proc.pid
    return proc
def _snapshot_pre_install(
    self, install_path: str | None, prefix_path: str,
) -> tuple[str, set[str], dict[str, set[str]]]:
    """Snapshot pre install."""
    install_base, dirs_before = self._snapshot_install_base(
        install_path,
    )
    upc_dirs_before = self._snapshot_upc_game_dirs(prefix_path)
    return install_base, dirs_before, upc_dirs_before
def _capture_and_propagate_session(
    self, prefix_path: str,
) -> None:
    """Capture and propagate session."""
    if self._session.capture(prefix_path):
        self._session.propagate_all_to_all()
def _snapshot_install_base(
    self, install_path: str | None,
) -> tuple[str, set]:
    """Snapshot install base."""
    install_base = (
        install_path
        or self._config.default_install_base_expanded
    )
    os.makedirs(install_base, exist_ok=True)
    dirs_before: set = set()
    try:
        dirs_before = set(os.listdir(install_base))
    except OSError:
        pass
    return install_base, dirs_before
@staticmethod
async def _terminate_upc_gracefully(
    proc: asyncio.subprocess.Process,
    timeout: float = 15.0,
) -> None:
    """Terminate UPC gracefully."""
    if proc.returncode is not None:
        return
    try:
        proc.terminate()
        await asyncio.wait_for(
            proc.wait(), timeout=timeout,
        )
    except (TimeoutError, ProcessLookupError):
        try:
            proc.kill()
        except ProcessLookupError:
            pass
async def _finalize_manual_install(
    self,
    *,
    game_id: str,
    game_name: str | None,
    install_dir: str,

```

```

) -> InstallResult:

    """Finalize manual install."""
    exe = self._library.find_game_executable(install_dir)
    await self._library.write_install_marker(
        space_id=game_id,
        install_path=install_dir,
        executable=exe or "",
        game_title=game_name or "",
    )
    final_size = _reg.get_directory_size(install_dir)
    logger.info(
        "[UbisoftInstaller] manual install complete: %s "
        "(%.0f MB)",
        install_dir, final_size / (1024 * 1024),
    )
    try:
        await self._id_map.refresh_from_configurations()
    except Exception as e:
        logger.debug(
            "[UbisoftInstaller] id_map refresh after "
            "install failed: %s", e,
        )
    return InstallResult(
        success=True,
        store="ubisoft",
        game_id=game_id,
        install_path=install_dir,
        size_bytes=final_size,
        metadata={"executable": exe},
    )

@staticmethod
def _snapshot_upc_game_dirs(
    prefix_path: str,
) -> dict[str, set]:
    """Snapshot UPC game dirs."""
    upc_games_rel = os.path.join(
        "drive_c", "Program Files (x86)",
        "Ubisoft", "Ubisoft Game Launcher", "games",
    )
    candidates = (
        os.path.join(prefix_path, upc_games_rel),
        os.path.join(prefix_path, "pfx", upc_games_rel),
    )
    snapshots: dict[str, set] = {}
    for gdir in candidates:
        if os.path.isdir(gdir):
            try:
                snapshots[gdir] = set(os.listdir(gdir))
            except OSError:
                pass
    return snapshots

async def _poll_for_new_install(
    self,
    *,
    proc: asyncio.subprocess.Process,
    install_base: str,
    dirs_before: set,
    upc_dirs_before: dict[str, set],
    progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,

```

```

) -> str | None:

    """Poll for new install."""
    install_dir: str | None = None
    max_polls = int(
        _MANUAL_INSTALL_TIMEOUT_S
        / _MANUAL_INSTALL_POLL_INTERVAL_S,
    )
    for iteration in range(max_polls):
        await asyncio.sleep(
            _MANUAL_INSTALL_POLL_INTERVAL_S,
        )
        install_dir = self._check_new_dirs(
            install_base, dirs_before,
        )
        if not install_dir:
            for gdir, before in upc_dirs_before.items():
                found = self._check_new_dirs(gdir, before)
                if found:
                    install_dir = found
                    break
        if install_dir:
            logger.info(
                "[UbisoftInstaller] detected install "
                "at %s", install_dir,
            )
            await self._notify_install_detected(
                install_dir, progress_cb,
            )
            await self._wait_for_install_completion(
                install_dir, progress_cb,
            )
            return install_dir
        if proc.returncode is not None:
            logger.info(
                "[UbisoftInstaller] UPC exited rc=%d",
                proc.returncode,
            )
            return None
        if progress_cb and iteration % 6 == 0:
            await progress_cb({
                "status": "waiting",
                "message": (
                    "Waiting for game installation in "
                    "Ubisoft Connect..."
                ),
                "progress": 0,
            })
    return None

@staticmethod
async def _notify_install_detected(
    install_dir: str,
    progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,
) -> None:
    """Notify install detected."""
    if not progress_cb:
        return
    await progress_cb({
        "status": "installing",
        "message": (
            f"Game detected at "

```

```

        f"{os.path.basename(install_dir)}"
    ),
    "progress": 50,
})

def _check_new_dirs(
    self, base: str, before: set,
) -> str | None:

    """Check new dirs."""
    try:
        now = set(os.listdir(base))
    except OSError:
        return None
    new_dirs = now - before
    for d in new_dirs:
        candidate = os.path.join(base, d)
        if (
            os.path.isdir(candidate)
            and looks_like_game_install(candidate)
        ):
            return candidate
    return None

async def _wait_for_install_completion(
    self,
    install_dir: str,
    progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None,
) -> None:
    """Wait for install completion."""
    prev_size = 0
    stable_count = 0
    for _ in range(_STABILITY_WAIT_MAX_POLLS):
        await asyncio.sleep(_STABILITY_POLL_INTERVAL_S)
        curr_size = _reg.get_directory_size(install_dir)
        if curr_size == prev_size and curr_size > 0:
            stable_count += 1
            if stable_count >= _STABILITY_STABLE_THRESHOLD:
                break
        stable_count = 0
        prev_size = curr_size
    if progress_cb and curr_size > 0:
        await progress_cb({
            "status": "installing",
            "message": (
                f"Installing... "
                f"({curr_size / (1024 ** 3):.1f} GB)"
            ),
            "progress": min(
                90, 50 + stable_count * 10,
            ),
        })

```


OP-56f

registry.py

Fichier : py_modules/unifideck/stores/ubisoft/installer/registry.py | Lignes : 252 | Depend de : (rien)

```
from __future__ import annotations
import json
import logging
import os
import re
from pathlib import Path
from typing import Any
from ..config import UbisoftConfig
logger = logging.getLogger(__name__)
_INSTALLS_REG_SECTION_FMT = (
    "[Software\\\\\\\\WOW6432Node\\\\\\\\Ubisoft\\\\\\\\\\"
    "Launcher\\\\\\\\Installs\\\\\\\\{install_id}]"
)
_STEAM_COMPAT_ENV_VARS = (
    "SteamAppId", "SteamGameId", "STEAM_COMPAT_APP_ID",
)
def resolve_active_prefix_dir(
    prefix_path: str,
) -> str | None:
    """Resolve active prefix dir."""
    prefix = Path(prefix_path)
    pfx = prefix / "pfx"
    if (pfx / "system.reg").is_file():
        return str(pfx)
    if (prefix / "system.reg").is_file():
        return str(prefix)
    return None
def read_system_reg(
    active_prefix: str,
) -> tuple[str, str] | None:
    """Read system reg."""
    system_reg = str(Path(active_prefix) / "system.reg")
    try:
        content = Path(system_reg).read_text(
            encoding="utf-8", errors="replace",
        )
    except OSError:
        return None
    return system_reg, content
def find_install_registry_section_bounds(
    content: str, section: str,
) -> tuple[int, int] | None:
    """Find install registry section bounds."""
    if section not in content:
        return None
    sec_start = content.index(section)
    tail = content[sec_start + len(section):]
    next_sec = re.search(r"\n\[", tail)
    sec_end = (
        sec_start + len(section) + next_sec.start()
        if next_sec
        else len(content)
    )
    return sec_start, sec_end
```

```

def _update_or_append_install_section(
    content: str, section: str, values: list[str],
) -> str:

    """Update or append install section."""
    bounds = find_install_registry_section_bounds(
        content, section,
    )
    if bounds is None:
        return (
            content + f"\n{section}\n"
            + "\n".join(values) + "\n"
        )
    sec_start, sec_end = bounds
    sec_body = content[sec_start + len(section):sec_end]
    for val in values:
        key = val.split("=")[0]
        pattern = rf'^{re.escape(key)}="[^"]*"'
        new_body, count = re.subn(
            pattern, val, sec_body,
            flags=re.MULTILINE,
        )
        if count:
            sec_body = new_body
        else:
            sec_body = (
                sec_body.rstrip("\n")
                + "\n" + val + "\n"
            )
    return (
        content[:sec_start + len(section)]
        + sec_body
        + content[sec_end:]
    )

def inject_install_registry(
    prefix_path: str,
    install_id: str,
    install_dir: str,
) -> None:
    """Inject install registry."""
    try:
        active_prefix = resolve_active_prefix_dir(prefix_path)
        if active_prefix is None:
            return
        loaded = read_system_reg(active_prefix)
        if loaded is None:
            return
        system_reg, content = loaded
        wine_path = install_dir
        if install_dir.startswith("/"):
            wine_path = (
                "Z:" + install_dir.replace("/", "\\")
            )
        section = _INSTALLS_REG_SECTION_FMT.format(
            install_id=install_id,
        )
        values = [f'InstallDir="{wine_path}"']
        content = _update_or_append_install_section(
            content, section, values,
        )
        Path(system_reg).write_text(

```

```

        content, encoding="utf-8",
    )
    logger.info(
        "[UbisoftInstaller] install registry injected "
        "for %s", install_id,
    )
except Exception as e:
    logger.warning(
        "[UbisoftInstaller] registry injection failed: "
        "%s", e,
    )

def clean_install_registry(
    prefix_path: str, install_id: str,
) -> None:

    """Clean install registry."""
    if not install_id:
        return
    try:
        active_prefix = resolve_active_prefix_dir(prefix_path)
        if active_prefix is None:
            return
        loaded = read_system_reg(active_prefix)
        if loaded is None:
            return
        system_reg, content = loaded
        section = _INSTALLS_REG_SECTION_FMT.format(
            install_id=install_id,
        )
        bounds = find_install_registry_section_bounds(
            content, section,
        )
        if bounds is None:
            return
        sec_start, sec_end = bounds
        content = content[:sec_start] + content[sec_end:]
        Path(system_reg).write_text(
            content, encoding="utf-8",
        )
        logger.info(
            "[UbisoftInstaller] cleaned registry for %s",
            install_id,
        )
    except Exception as e:
        logger.warning(
            "[UbisoftInstaller] registry cleanup failed: "
            "%s",
            e,
        )

def get_directory_size(path: str) -> int:
    """Get directory size."""
    total = 0
    try:
        for dirpath, _dirs, filenames in os.walk(path):
            for f in filenames:
                try:
                    total += (Path(dirpath) / f).stat().st_size
                except OSError:
                    continue
    except OSError:

```

```

        pass
    return total
def parse_positive_int(value: Any) -> int | None:
    """Parse positive int."""
    try:
        parsed = int(value)
    except (TypeError, ValueError):
        return None
    return parsed if parsed > 0 else None
class _ShortcutRegistry:
    """Shortcut registry."""
    def __init__(self, config: UbisoftConfig) -> None:
        """Initialize the instance."""
        self._config = config
    def load(self) -> dict[str, Any]:
        """Load."""
        path = (
            Path(self._config.data_dir_expanded)
            / "shortcuts_registry.json"
        )
        if not path.is_file():
            return {}
        try:
            data = json.loads(
                path.read_text(encoding="utf-8"),
            )
        except (OSError, json.JSONDecodeError) as e:
            logger.warning(
                "[UbisoftInstaller] shortcuts registry load "
                "failed: %s", e,
            )
            return {}
        if isinstance(data, dict):
            return data
        return {}

def scan_for_ubisoft_appid(
    self, registry: dict[str, Any],
) -> int | None:
    """Scan for UBISOFT appid."""
    for key, entry in registry.items():
        if (
            not isinstance(key, str)
            or not key.startswith("ubisoft:")
        ):
            continue
        if not isinstance(entry, dict):
            continue
        appid = parse_positive_int(
            entry.get("appid_unsigned"),
        )
        if appid:
            return appid
    return None
def resolve_shortcut_appid(
    self, store_game_id: str | None,
) -> int | None:
    """Resolve shortcut appid."""
    registry = self.load()
    if store_game_id:

```

```
        entry = registry.get(store_game_id, {})
        appid = parse_positive_int(
            entry.get("appid_unsigned"),
        )
        if appid:
            return appid
    appid = self.scan_for_ubisoft_appid(registry)
    if appid:
        return appid
    for env_var in _STEAM_COMPAT_ENV_VARS:
        appid = parse_positive_int(
            os.environ.get(env_var),
        )
        if appid:
            return appid
    if store_game_id:
        appid = self.scan_for_ubisoft_appid(registry)
        if appid:
            return appid
    return None
```

OP-56g

uninstall.py

Fichier : py_modules/unifideck/stores/ubisoft/installer/uninstall.py | Lignes : 314 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import shutil
from pathlib import Path
from typing import TYPE_CHECKING
from ...core.types import Result
from . import registry as _reg
from .launch_env import UpcLaunchEnvBuildError
if TYPE_CHECKING:
    from .installer import UbisoftInstaller
logger = logging.getLogger(__name__)
_PROTOCOL_UNINSTALL_TIMEOUT_S = 60.0
_DELETE_MIN_PATH_DEPTH = 4
class _UninstallPipeline:
    """Uninstall pipeline."""
    def __init__(self, parent: UbisoftInstaller) -> None:
        """Initialize the instance."""
        self._parent = parent

    async def uninstall_game(
        self,
        game_id: str,
        *,
        delete_prefix: bool = False,
    ) -> Result:
        """Uninstall game."""
        try:
            logger.info(
                "[UbisoftInstaller] uninstalling %s "
                "(delete_prefix=%s)",
                game_id, delete_prefix,
            )
            prefix_path, install_id, install_path = (
                self.resolve_uninstall_targets(game_id)
            )
            protocol_attempted = (
                await self.attempt_protocol_uninstall(
                    game_id, prefix_path, install_id,
                    delete_prefix,
                )
            )
            install_path = self.refresh_install_path(
                game_id, prefix_path, install_path,
            )
            game_dir_error = await self.delete_game_directory(
                install_path, prefix_path, delete_prefix,
            )
            if game_dir_error:
                return Result(success=False, error=game_dir_error)
            prefix_deleted, prefix_error = (
                await self.delete_prefix_if_requested(
                    prefix_path, delete_prefix,
                )
            )

```

```

    )
    if prefix_error:
        return Result(success=False, error=prefix_error)
    self.post_uninstall_cleanup(
        game_id, prefix_path, install_id, prefix_deleted,
    )
    logger.info(
        "[UbisoftInstaller] game %s uninstalled "
        "(protocol_attempted=%s, prefix_deleted=%s)",
        game_id, protocol_attempted, prefix_deleted,
    )
    return Result(success=True)
except Exception as e:
    logger.exception(
        "[UbisoftInstaller] uninstall error for %s: %s",
        game_id, e,
    )
    return Result(
        success=False,
        error=f"uninstall_exception: {e}",
    )
)

def resolve_uninstall_targets(
    self, game_id: str,
) -> tuple[str, str | None, str | None]:
    """Resolve uninstall targets."""
    prefix_path = self._parent._paths.get_prefix_path(
        game_id,
    )
    game_info = (
        self._parent._library._detector
        ._detect_installed_game(game_id, prefix_path)
    )
    install_path = (
        game_info.get("install_path")
        if game_info else None
    )
    install_id = (
        self._parent._id_map.resolve_install_id(game_id)
        or self._parent._id_map.resolve_launch_id(game_id)
    )
    return prefix_path, install_id, install_path

async def attempt_protocol_uninstall(
    self,
    game_id: str,
    prefix_path: str,
    install_id: str | None,
    delete_prefix: bool,
) -> bool:
    """Attempt protocol uninstall."""
    if delete_prefix:
        logger.info(
            "[UbisoftInstaller] delete_prefix=True: "
            "skipping uninstall URI, deleting files "
            "directly",
        )
        return False
    if not install_id:
        return False
    return await self.try_protocol_uninstall(

```

```

        game_id, prefix_path, install_id,
    )
def refresh_install_path(
    self,
    game_id: str,
    prefix_path: str,
    install_path: str | None,
) -> str | None:
    """Refresh install path."""
    post_info = (
        self._parent._library._detector
        ._detect_installed_game(game_id, prefix_path)
    )
    if post_info:
        return (
            post_info.get("install_path") or install_path
        )
    return install_path
async def delete_game_directory(
    self,
    install_path: str | None,
    prefix_path: str,
    delete_prefix: bool,
) -> str | None:
    """Delete game directory."""
    if not install_path or not Path(install_path).is_dir():
        return None
    inside_prefix = str(
        Path(install_path).resolve(),
    ).startswith(
        str(Path(prefix_path).resolve()) + "/",
    )
    if inside_prefix and delete_prefix:
        return None
    logger.info(
        "[UbisoftInstaller] fallback deleting game "
        "directory: %s", install_path,
    )
    deleted = await self.delete_tree_with_retries(
        install_path,
        "Ubisoft game install directory",
    )
    if not deleted:
        return f"game_dir_delete_failed: {install_path}"
    return None
async def delete_prefix_if_requested(
    self,
    prefix_path: str,
    delete_prefix: bool,
) -> tuple[bool, str | None]:
    """Delete prefix if requested."""
    if not delete_prefix:
        return False, None
    if not Path(prefix_path).is_dir():
        return False, None
    deleted = await self.delete_tree_with_retries(
        prefix_path, "Ubisoft game prefix",
    )
    if not deleted:
        return False, f"prefix_delete_failed: {prefix_path}"
    return True, None

```



```

def post_uninstall_cleanup(
    self,
    game_id: str,
    prefix_path: str,
    install_id: str | None,
    prefix_deleted: bool,
) -> None:

    """Post uninstall cleanup."""
    if not prefix_deleted:
        _reg.clean_install_registry(
            prefix_path, install_id or "",
        )
        return
    if self._parent._id_map.in_cache(game_id):
        self._parent._id_map._cache.pop(game_id, None)
        self._parent._id_map._save()
    async def try_protocol_uninstall(
        self,
        game_id: str,
        prefix_path: str,
        install_id: str,
    ) -> bool:
        """Try protocol uninstall."""
        try:
            launch_env = self._parent._build_upc_launch_env(
                game_id, prefix_path,
            )
        except UpcLaunchEnvBuildError:
            return False
        upc_path = launch_env.upc_path
        umu_run = launch_env.umu_run
        python_bin = launch_env.python_bin
        env = launch_env.env
        uninstall_url = f"uplay://uninstall/{install_id}"
        logger.info(
            "[UbisoftInstaller] trying protocol uninstall: "
            "%s",
            uninstall_url,
        )
        try:
            proc = await asyncio.create_subprocess_exec(
                python_bin, umu_run, upc_path, uninstall_url,
                env=env,
                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE,
            )
            try:
                await asyncio.wait_for(
                    proc.wait(),
                    timeout=_PROTOCOL_UNINSTALL_TIMEOUT_S,
                )
            except TimeoutError:
                proc.kill()
                logger.warning(
                    "[UbisoftInstaller] protocol uninstall "
                    "timed out, falling back to direct "
                    "delete",
                )
            return True

```

```

except OSError as e:
    logger.warning(
        "[UbisoftInstaller] protocol uninstall spawn "
        "failed: %s", e,
    )
    return False

def _is_path_safe_to_delete(
    self, target_path: str, label: str,
) -> bool:

    """Is path safe to delete."""
    if not target_path:
        logger.error(
            "[UbisoftInstaller] refusing to delete empty "
            "path for %s", label,
        )
        return False
    resolved = str(Path(target_path).resolve())
    home_dir = str(Path("~").expanduser().resolve())
    config = self._parent._config
    protected = {
        "/",
        home_dir,
        str(Path(config.data_dir_expanded).resolve()),
        str(Path(config.prefixes_dir_expanded).resolve()),
        str(
            Path(
                config.default_install_base_expanded,
            ).resolve(),
        ),
        str(Path(config.sdcard_install_base).resolve()),
    }
    if (
        resolved in protected
        or len(resolved.strip("/"))
        < _DELETE_MIN_PATH_DEPTH
    ):
        logger.error(
            "[UbisoftInstaller] refusing to delete "
            "unsafe path for %s: %s", label, resolved,
        )
        return False
    return True

async def delete_tree_with_retries(
    self,
    target_path: str,
    label: str,
    *,
    retries: int = 3,
) -> bool:
    """Delete tree with retries."""
    if not self._is_path_safe_to_delete(target_path, label):
        return False
    resolved = str(Path(target_path).resolve())
    if not Path(resolved).is_dir():
        logger.info(
            "[UbisoftInstaller] nothing to delete for "
            "%s: %s",
            label, resolved,
        )

```

```
        return True
    for attempt in range(1, retries + 1):
        try:
            shutil.rmtree(resolved)
            logger.info(
                "[UbisoftInstaller] deleted %s: %s",
                label, resolved,
            )
            return True
        except OSError as e:
            logger.warning(
                "[UbisoftInstaller] delete attempt "
                "%d/%d failed for %s: %s",
                attempt, retries, label, e,
            )
            if attempt < retries:
                await asyncio.sleep(1.5)
    logger.error(
        "[UbisoftInstaller] delete failed after %d "
        "attempts for %s: %s",
        retries, label, resolved,
    )
    return False
```

OP-56h

update_op.py

Fichier : py_modules/unifideck/stores/ubisoft/installer/update_op.py | Lignes : 112 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import TYPE_CHECKING
from ...core.types import InstallResult
from .launch_env import UpcLaunchEnvBuildError, _UpcLaunchEnv
if TYPE_CHECKING:
    from collections.abc import Callable
    from ..id_map import UbisoftIdMap
    from ..paths import UbisoftPrefixPaths
    from ..session import UbisoftSession
_UPDATE_TIMEOUT_S = 4 * 60 * 60
logger = logging.getLogger(__name__)
class _UpdateOperation:
    """Update operation."""
    def __init__(
        self,
        *,
        id_map: UbisoftIdMap,
        paths: UbisoftPrefixPaths,
        session: UbisoftSession,
        build_launch_env: Callable[..., _UpcLaunchEnv],
    ) -> None:
        """Initialize the instance."""
        self._id_map = id_map
        self._paths = paths
        self._session = session
        self._build_launch_env = build_launch_env
    async def update(self, game_id: str) -> InstallResult:
        """Update."""
        try:
            prepared = self._prepare_launch(game_id)
        except UpcLaunchEnvBuildError as e:
            return InstallResult(
                success=False,
                store="ubisoft",
                game_id=game_id,
                error=e.error_code,
            )
        if isinstance(prepared, InstallResult):
            return prepared
        try:
            return await self._run_update_process(
                game_id, prepared,
            )
        except Exception as e:
            logger.exception(
                "[UbisoftInstaller] update error for "
                "%s: %s", game_id, e,
            )
            return InstallResult(
                success=False,
                store="ubisoft",
                game_id=game_id,
                error=f"update_exception: {e}",
            )

```

```

    )

def _prepare_launch(
    self, game_id: str,
) -> _UpcLaunchEnv | InstallResult:

    """Prepare launch."""
    prefix_path = self._paths.get_prefix_path(game_id)
    self.session.inject_into_prefix(prefix_path)
    launch_id = self._id_map.resolve_launch_id(game_id)
    if not launch_id:
        return InstallResult(
            success=False,
            store="ubisoft",
            game_id=game_id,
            error="launch_id_not_resolved",
        )
    return self._build_launch_env(
        game_id,
        prefix_path,
        upc_missing_error=(
            "upc_exe_not_found_reinstall_required"
        ),
    )

async def _run_update_process(
    self, game_id: str, launch_env: _UpcLaunchEnv,
) -> InstallResult:
    """Run update process."""
    launch_id = self._id_map.resolve_launch_id(game_id)
    launch_url = f"uplay://launch/{launch_id}/0"
    logger.info(
        "[UbisoftInstaller] triggering update via "
        "%s", launch_url,
    )
    proc = await asyncio.create_subprocess_exec(
        launch_env.python_bin,
        launch_env.umu_run,
        launch_env.upc_path,
        launch_url,
        env=launch_env.env,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    try:
        await asyncio.wait_for(
            proc.wait(),
            timeout=_UPDATE_TIMEOUT_S,
        )
    except TimeoutError:
        proc.kill()
        logger.warning(
            "[UbisoftInstaller] update timed out "
            "for %s", game_id,
        )
    return InstallResult(
        success=True,
        store="ubisoft",
        game_id=game_id,
    )

```

OP-57

`__init__.py`

Fichier : `py_modules/unifideck/stores/ubisoft/library/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from .facade import UbisoftLibrary
__all__ = ["UbisoftLibrary"]
```

OP-57a

facade.py

Fichier : py_modules/unifideck/stores/ubisoft/library/facade.py | Lignes : 114 | Depend de : (rien)

```
from __future__ import annotations
import logging
import os
from collections.abc import Callable
from typing import Any
from ...core.types import Game
from ..config import UbisoftConfig
from ..id_map import UbisoftIdMap
from ..paths import UbisoftPrefixPaths
from .detection import _InstallDetector
from .fetch import _LibraryFetcher
from .manifest import _VisibleManifestProcessor
logger = logging.getLogger(__name__)
class UbisoftLibrary:
    """Ubisoft library."""
    def __init__(
        self,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
        id_map: UbisoftIdMap,
        queue_template_creation: Callable[[[]], None],
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._paths = paths
        self._id_map = id_map
        self._queue_template_creation = queue_template_creation
        self._detector = _InstallDetector(
            config=config,
            id_map=id_map,
        )
        self._fetcher = _LibraryFetcher(
            config=config,
            paths=paths,
            id_map=id_map,
        )
        self._manifest = _VisibleManifestProcessor(
            config=config,
            id_map=id_map,
            load_json_file_safe=self._detector.load_json_file_safe,
        )

    async def get_library(self) -> list[Game]:
        """Get library."""
        try:
            installed = await self._detector.get_installed()
            local_games = await self._fetcher.fetch_local_binaries(
                installed,
            )
        except Exception:
            if local_games is None:
                logger.info(
                    "[UbisoftLibrary] no local binary data "
                    "available yet",
                )
```

```

        return []
    logger.info(
        "[UbisoftLibrary] library: %d games from "
        "local binaries", len(local_games),
    )
    override_manifest = self._manifest.load_manifest()
    if override_manifest:
        local_games = self._manifest.apply_filter(
            local_games,
            installed,
            override_manifest,
            source_label="override",
        )
    if local_games:
        template_dir = (
            self._config.template_dir_expanded
        )
        template_marker = os.path.join(
            template_dir,
            self._config.bootstrap_marker,
        )
        if not os.path.isfile(template_marker):
            self._queue_template_creation()
    return local_games
except Exception as e:
    logger.exception(
        "[UbisoftLibrary] error fetching library: %s",
        e,
    )
    return []

async def get_installed(self) -> dict[str, Any]:
    """Get installed."""
    return await self._detector.get_installed()
def get_installed_game_info(
    self, game_id: str,
) -> dict[str, Any] | None:
    """Get installed game info."""
    return self._detector.get_installed_game_info(game_id)
def find_game_executable(
    self, install_path: str,
) -> str | None:
    """Find game executable."""
    return self._detector.find_game_executable(install_path)
async def write_install_marker(
    self,
    space_id: str,
    install_path: str,
    executable: str,
    game_title: str = "",
) -> None:
    """Write install marker."""
    await self._detector.write_install_marker(
        space_id=space_id,
        install_path=install_path,
        executable=executable,
        game_title=game_title,
    )
@staticmethod
def get_game_official_url(game_id: str) -> str:
    """Get game official URL."""
    return _InstallDetector.get_game_official_url(game_id)

```


OP-57b

fetch.py

Fichier : py_modules/unifideck/stores/ubisoft/library/fetch.py | Lignes : 79 | Depend de : (rien)

```

from __future__ import annotations
import logging
from collections.abc import Callable
from typing import TYPE_CHECKING, Any
from ....core.types import Game
from .data_loader import _DataLoader
from .game_builder import _GameBuilder
if TYPE_CHECKING:
    from ..config import UbisoftConfig
    from ..id_map import UbisoftIdMap
    from ..parser import GameConfig
    from ..paths import UbisoftPrefixPaths
ParseConfigurationsFn = Callable[[str], "list[GameConfig]"]
ParseOwnershipFn = Callable[[str], list[int]]
logger = logging.getLogger(__name__)
class _LibraryFetcher:
    """Library fetcher."""
    def __init__(
        self,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
        id_map: UbisoftIdMap,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._paths = paths
        self._id_map = id_map
        self._loader = _DataLoader(config=config, paths=paths)
        self._builder = _GameBuilder(
            config=config, id_map=id_map,
        )
    async def fetch_local_binaries(
        self, installed: dict[str, Any],
    ) -> list[Game] | None:
        """Fetch local binaries."""
        parser_funcs = self._import_ubisoft_parser()
        if parser_funcs is None:
            return None
        parse_configurations, parse_ownership = parser_funcs
        configs = await self._loader.load_configurations(
            parse_configurations,
        )
        if not configs:
            return None
        owned_set = await self._loader.load_ownership_set(
            parse_ownership,
        )
        config_by_id = self._builder.build_config_lookup(configs)
        matched_configs = self._builder.cross_reference_ownership(
            configs, config_by_id, owned_set,
        )
        matched_configs = self._builder.apply_steam_filter(
            matched_configs,
        )
        games = self._builder.build_games_from_configs(

```

```
        matched_configs, installed,
    )
    logger.info(
        "[UbisoftLibrary] local binary library: %d games "
        "(from %d matched configs)",
        len(games), len(matched_configs),
    )
    return games

@staticmethod
def _import_ubisoft_parser() -> tuple[ParseConfigurationsFn, ParseOwnershipFn] | None:
    """Import UBISOFT parser."""
    try:
        from ..parser import (
            parse_configurations,
            parse_ownership,
        )
    except ImportError as e:
        logger.error(
            "[UbisoftLibrary] ubisoft_parser unavailable: "
            "%s",
            e,
        )
        return None
    return parse_configurations, parse_ownership
```

OP-57c

data_loader.py

Fichier : py_modules/unifideck/stores/ubisoft/library/data_loader.py | Lignes : 105 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from pathlib import Path
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from collections.abc import Callable
    from ..config import UbisoftConfig
    from ..parser import GameConfig
    from ..paths import UbisoftPrefixPaths
    ParseConfigurationsFn = Callable[[str], list[GameConfig]]
    ParseOwnershipFn = Callable[[str], list[int]]
logger = logging.getLogger(__name__)
class _DataLoader:
    """Data loader."""
    def __init__(
        self,
        *,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._paths = paths
    async def load_configurations(
        self, parse_configurations: ParseConfigurationsFn,
    ) -> list[GameConfig] | None:
        """Load configurations."""
        cfg_path = self._find_library_configurations_path()
        if not cfg_path:
            logger.info(
                "[UbisoftLibrary] no configurations binary "
                "found",
            )
            return None
        configs = await asyncio.to_thread(
            parse_configurations, cfg_path,
        )
        if not configs:
            logger.warning(
                "[UbisoftLibrary] configurations binary "
                "parsed but empty",
            )
            return None
        return configs
    async def load_ownership_set(
        self, parse_ownership: ParseOwnershipFn,
    ) -> set[int] | None:
        """Load ownership set."""
        ownership_path, user_id = self._discover_ownership_file()
        if not ownership_path:
            return None
        owned_ids = await asyncio.to_thread(
            parse_ownership, ownership_path,
        )

```

```

owned_set = set(owned_ids)
user_display = user_id[:8] if user_id else "?"
logger.info(
    "[UbisoftLibrary] ownership: %d unique IDs "
    "(userId=%s...)",
    len(owned_set), user_display,
)
return owned_set

def _find_library_configurations_path(self) -> str | None:

    """Find library configurations path."""
    for prefix_dir in (
        self._config.auth_prefix_dir_expanded,
        self._config.template_dir_expanded,
    ):
        cfg_path = self._paths.find_configurations(prefix_dir)
        if cfg_path:
            return cfg_path
    return None

def _discover_ownership_file(
    self,
) -> tuple[str | None, str]:
    """Discover ownership file."""
    for prefix_dir in (
        self._config.auth_prefix_dir_expanded,
        self._config.template_dir_expanded,
    ):
        prefix_p = Path(prefix_dir)
        if not prefix_p.is_dir():
            continue
        for layout_sub in ("", "pfx"):
            base = prefix_p
            if layout_sub:
                base = base / layout_sub
            ownership_dir = (
                base / self._config.ownership_relative_path
            )
            if not ownership_dir.is_dir():
                continue
            try:
                entries = [
                    e for e in ownership_dir.iterdir()
                    if e.is_file()
                ]
            except OSError:
                continue
            if entries:
                entry = entries[0]
                user_id = entry.name
                return str(entry), user_id
    return (None, "")

```

OP-57d

game_builder.py

Fichier : py_modules/unifideck/stores/ubisoft/library/game_builder.py | Lignes : 210 | Depend de : (rien)

```

from __future__ import annotations
import logging
import re
from typing import TYPE_CHECKING, Any
from ...core.types import Game
from ..steam_filter import filter_steam_linked_configs
if TYPE_CHECKING:
    from ..config import UbisoftConfig
    from ..id_map import UbisoftIdMap
    from ..parser import GameConfig
logger = logging.getLogger(__name__)
_MOJIBAKE_REPLACEMENTS = (
    ("Â©", "@"),
    ("â\u0080ç", "™"),
    ("â\u0084ç", "™"),
    ("â\u0080\u0099", "'"),
    ("Â", ""),
)
_SKIP_TITLE_KEYWORDS = re.compile(
    r"\b(test|beta|alpha|closed|preorder|pre-order|promotion|"
    r"internal|dev/qc|pts|test server|demo|trial)\b",
    re.IGNORECASE,
)
_SKIP_DLC_KEYWORDS = re.compile(
    r"\b(dlc|season pass|expansion|pack|bonus|soundtrack|"
    r"art ?book|skins?|outfit|costume|weapon|map|mission|"
    r"episode|revolver|kukri|cane-sword|hammer|knife|dagger|"
    r"conspiracy|runaway train|texture|language|starter edition|"
    r"battle pass|car shipment|full stock|full ownership|"
    r"master unlock|paint|perk|club|credit pack|currency pack|"
    r"ownership|ubicollectibles|legion of the dead|"
    r"calling all units)\b",
    re.IGNORECASE,
)
_STORE_MARKER_PATTERN = re.compile(
    r"\[STEAM\]|\[Uplay", re.IGNORECASE,
)
_CYRILLIC_PATTERN = re.compile(r"[\u0400-\u04FF]")
_PLACEHOLDER_L_PATTERN = re.compile(r"(\d+|[A-Z0-9_]+)")
_PLACEHOLDER_LITERALS = frozenset({"a ubisoft game"})
class _GameBuilder:
    """Game builder."""
    def __init__(
        self,
        *,
        config: UbisoftConfig,
        id_map: UbisoftIdMap,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._id_map = id_map
    @staticmethod
    def build_config_lookup(
        configs: list[GameConfig],
    ) -> dict[int, GameConfig]:

```

```

        """Build config lookup."""
        config_by_id: dict[int, GameConfig] = {}
        for cfg in configs:
            config_by_id[cfg.install_id] = cfg
            if cfg.launch_id and cfg.launch_id != cfg.install_id:
                config_by_id[cfg.launch_id] = cfg
        return config_by_id

    @staticmethod
    def cross_reference_ownership(
        configs: list[GameConfig],
        config_by_id: dict[int, GameConfig],
        owned_set: set[int] | None,
    ) -> list[GameConfig]:
        """Cross reference ownership."""
        if owned_set is not None:
            return [
                config_by_id[oid]
                for oid in owned_set
                if oid in config_by_id
                and config_by_id[oid].name
            ]
        result = [c for c in configs if c.name]
        logger.info(
            "[UbisoftLibrary] no ownership binary – using "
            "all %d config entries", len(result),
        )
        return result

    def apply_steam_filter(
        self, configs: list[GameConfig],
    ) -> list[GameConfig]:
        """Apply steam filter."""
        if not self._config.filter_steam_linked:
            return configs
        before_count = len(configs)
        result = self._filter_steam_linked_configs(configs)
        dropped = before_count - len(result)
        if dropped:
            logger.info(
                "[UbisoftLibrary] filtered %d Steam-linked "
                "game(s) from library", dropped,
            )
        return result

    def _filter_steam_linked_configs(
        self, configs: list[GameConfig],
    ) -> list[GameConfig]:
        """Filter steam linked configs."""
        return filter_steam_linked_configs(
            configs,
            self._config.steam_library_cross_ref,
            self._id_map,
        )

    def build_games_from_configs(
        self,
        matched_configs: list[GameConfig],
        installed: dict[str, Any],
    ) -> list[Game]:
        """Build games from configs."""
        games: list[Game] = []
        seen_norms: set[str] = set()
        id_map_updates: dict[str, dict[str, Any]] = {}

```

```

for cfg in sorted(
    matched_configs,
    key=lambda c: (c.name or "").lower(),
):
    game = self._build_one_game(
        cfg, installed, seen_norms, id_map_updates,
    )
    if game is not None:
        games.append(game)
if id_map_updates:
    self._id_map.update_bulk(id_map_updates)
return games

def _build_one_game(
    self,
    cfg: GameConfig,
    installed: dict[str, Any],
    seen_norms: set[str],
    id_map_updates: dict[str, dict[str, Any]],
) -> Game | None:

    """Build one game."""
    title = self._clean_launcher_title(cfg.name)
    if self._should_skip_launcher_title(title):
        return None
    norm_name = self._id_map.normalize_for_matching(title)
    if norm_name in seen_norms:
        return None
    seen_norms.add(norm_name)
    game_id = (
        cfg.space_id
        if cfg.space_id
        else str(cfg.install_id)
    )
    is_installed = (
        game_id in installed
        or cfg.space_id in installed
    )
    install_meta = (
        installed.get(game_id)
        or installed.get(cfg.space_id)
        or {}
    )
    id_map_updates[game_id] = {
        "install_id": str(cfg.install_id),
        "launch_id": str(cfg.launch_id),
        "name": title,
        "executable": getattr(cfg, "executable", None),
        "game_identifier": getattr(
            cfg, "game_identifier", None,
        ),
        "source": "local_binary",
    }
    return Game(
        app_id=0,
        store="ubisoft",
        store_game_id=game_id,
        title=title,
        installed=is_installed,
        install_path=install_meta.get("install_path"),
        exe_path=install_meta.get("executable"),
    )

```

```
        metadata={"ownership_type": "owned"},
    )
    @staticmethod
    def _clean_launcher_title(title: Any) -> str:
        """Clean launcher title."""
        if not isinstance(title, str):
            return ""
        cleaned = title.strip().strip('\'').strip('"')
        for bad, good in _MOJIBAKE_REPLACEMENTS:
            cleaned = cleaned.replace(bad, good)
        return cleaned
    def _is_launcher_placeholder_title(self, title: str) -> bool:
        """Is launcher placeholder title."""
        cleaned = self._clean_launcher_title(title)
        if not cleaned:
            return True
        normalized = self._id_map.normalize_for_matching(
            cleaned,
        )
        if normalized in _PLACEHOLDER_LITERALS:
            return True
        return bool(_PLACEHOLDER_L_PATTERN.fullmatch(cleaned))
    def _should_skip_launcher_title(self, title: str) -> bool:
        """Should skip launcher title."""
        cleaned = self._clean_launcher_title(title)
        if not cleaned or len(cleaned.strip()) <= 2:
            return True
        if self._is_launcher_placeholder_title(cleaned):
            return True
        if _STORE_MARKER_PATTERN.search(cleaned):
            return True
        if _SKIP_TITLE_KEYWORDS.search(cleaned):
            return True
        if _CYRILLIC_PATTERN.search(cleaned):
            return True
        return bool(_SKIP_DLC_KEYWORDS.search(cleaned))
```


OP-57e

manifest.py

Fichier : py_modules/unifideck/stores/ubisoft/library/manifest.py | Lignes : 292 | Depend de : (rien)

```

from __future__ import annotations
import hashlib
import logging
import os
from collections.abc import Callable
from dataclasses import dataclass, field
from typing import Any
from ...core.types import Game
from ..config import UbisoftConfig
from ..id_map import UbisoftIdMap
logger = logging.getLogger(__name__)
def _first_non_empty(
    raw: dict[str, Any], keys: tuple[str, ...],
) -> str:
    """First non empty."""
    for key in keys:
        val = raw.get(key)
        if val:
            stripped = str(val).strip()
            if stripped:
                return stripped
    return ""
@dataclass
class _VisibleManifestIndex:
    """Visible manifest index."""
    by_norm: dict[str, dict[str, Any]] = field(default_factory=dict)
    by_id: dict[str, dict[str, Any]] = field(default_factory=dict)
    norms: set[str] = field(default_factory=set)
    ids: set[str] = field(default_factory=set)
    def lookup(
        self, game_id: str, norm_title: str,
    ) -> dict[str, Any] | None:
        """Lookup."""
        return self.by_id.get(game_id) or self.by_norm.get(norm_title)
    def matches(self, game_id: str, norm_title: str) -> bool:
        """Matches."""
        return game_id in self.ids or norm_title in self.norms
class _VisibleManifestProcessor:
    """Visible manifest processor."""
    def __init__(
        self,
        config: UbisoftConfig,
        id_map: UbisoftIdMap,
        load_json_file_safe: Callable[[str], Any | None],
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._id_map = id_map
        self._load_json_file_safe = load_json_file_safe

    def load_manifest(self) -> list[dict[str, Any]]:
        """Load manifest."""
        manifest_file = self._config.visible_games_file_expanded
        if not os.path.isfile(manifest_file):

```

```

        return []
    payload = self._load_json_file_safe(manifest_file)
    if payload is None:
        logger.warning(
            "[UbisoftLibrary] visible manifest load "
            "failed",
        )
        return []
    if isinstance(payload, dict):
        raw_games = payload.get("games", [])
    else:
        raw_games = payload
    if not isinstance(raw_games, list):
        return []
    manifest: list[dict[str, Any]] = []
    for raw in raw_games:
        if not isinstance(raw, dict):
            continue
        entry = self._normalize_entry(raw)
        if entry:
            manifest.append(entry)
    return manifest
@staticmethod
def _normalize_entry(
    raw: dict[str, Any],
) -> dict[str, Any] | None:
    """Normalize entry."""
    title = _first_non_empty(raw, ("title", "name"))
    if not title:
        return None
    install_id = _first_non_empty(raw, ("install_id",))
    launch_id = (
        _first_non_empty(raw, ("launch_id",)) or install_id
    )
    return {
        "title": title,
        "space_id": _first_non_empty(
            raw, ("space_id", "spaceId"),
        ),
        "install_id": install_id,
        "launch_id": launch_id,
        "ubisoftconnect_game_id": _first_non_empty(
            raw, ("ubisoftconnect_game_id", "product_id"),
        ),
        "cover_image": _first_non_empty(
            raw, ("cover_image", "coverUrl", "thumb_url"),
        ),
        "ownership_type": (
            _first_non_empty(raw, ("ownership_type",))
            or "owned"
        ),
        "source": (
            _first_non_empty(raw, ("source",))
            or "visible_manifest"
        ),
    }
@staticmethod
def _game_id_for(entry: dict[str, Any]) -> str:
    """Game ID for."""
    space_id = str(entry.get("space_id") or "").strip()
    if space_id:

```

```

        return space_id
    install_id = str(entry.get("install_id") or "").strip()
    if install_id:
        return f"ubi-{install_id}"
    digest = hashlib.sha256(
        str(entry.get("title", "")).encode("utf-8"),
    ).hexdigest()[:12]
    return f"ubi-visible-{digest}"

def _merge_into_id_map(self, entry: dict[str, Any]) -> bool:

    """Merge into ID map."""
    cache_key = self._game_id_for(entry)
    fields: dict[str, Any] = {
        "name": entry.get("title") or "",
        "source": "visible_manifest",
    }
    for field_name in (
        "install_id", "launch_id", "ubisoftconnect_game_id",
    ):
        value = str(entry.get(field_name) or "").strip()
        if value:
            fields[field_name] = value
    return self._id_map.merge_entry(cache_key, fields)

def _build_index(
    self,
    manifest: list[dict[str, Any]],
) -> _VisibleManifestIndex:
    """Build index."""
    index = _VisibleManifestIndex()
    for entry in manifest:
        title = entry.get("title") or ""
        if not title:
            continue
        norm = self._id_map.normalize_for_matching(title)
        index.norms.add(norm)
        index.by_norm[norm] = entry
        entry_id = self._game_id_for(entry)
        index.ids.add(entry_id)
        index.by_id[entry_id] = entry
    return index

def apply_filter(
    self,
    games: list[Game],
    installed: dict[str, Any],
    manifest: list[dict[str, Any]] | None,
    source_label: str,
) -> list[Game]:
    """Apply filter."""
    if not manifest:
        return games
    index = self._build_index(manifest)
    id_map_changed = self._merge_manifest_into_id_map(
        manifest,
    )
    filtered, seen_ids, seen_norms = (
        self._filter_and_enrich_games(
            games, index, source_label,
        )
    )
    injected = self._inject_unseen_manifest_entries(

```

```

        manifest, installed, filtered,
        seen_ids, seen_norms, source_label,
    )
    if id_map_changed:
        logger.debug(
            "[UbisoftLibrary] visible %s merged "
            "manifest entries into id_map", source_label,
        )
    logger.info(
        "[UbisoftLibrary] visible %s filter kept %d "
        "games from %d base entries (+%d injected)",
        source_label, len(filtered), len(games), injected,
    )
    return filtered

def _merge_manifest_into_id_map(
    self, manifest: list[dict[str, Any]],
) -> bool:
    """Merge manifest into ID map."""
    return any(self._merge_into_id_map(entry) for entry in manifest)

def _enrich_game_from_entry(
    self, game: Game, entry: dict[str, Any],
) -> None:
    """Enrich game from entry."""
    if entry.get("title"):
        game.title = entry["title"]
    if entry.get("ownership_type"):
        game.metadata["ownership_type"] = entry["ownership_type"]
    if entry.get("cover_image"):
        game.icon_url = entry["cover_image"]
        if getattr(game, "metadata", None) is None:
            game.metadata = {}
            game.metadata.setdefault(
                "coverUrl", entry["cover_image"],
            )

def _filter_and_enrich_games(
    self,
    games: list[Game],
    index: _VisibleManifestIndex,
    source_label: str,
) -> tuple[list[Game], set[str], set[str]]:
    """Filter and enrich games."""
    filtered: list[Game] = []
    seen_norms: set[str] = set()
    seen_ids: set[str] = set()
    for game in games:
        norm_title = self._id_map.normalize_for_matching(
            game.title,
        )
        if not index.matches(game.store_game_id, norm_title):
            logger.debug(
                "[UbisoftLibrary] visible %s skip: %s",
                source_label, game.title,
            )
            continue
        entry = index.lookup(game.store_game_id, norm_title)
        if entry:
            self._enrich_game_from_entry(game, entry)
            filtered.append(game)
            seen_norms.add(norm_title)

```

```

        seen_ids.add(game.store_game_id)
    return filtered, seen_ids, seen_norms

def _inject_unseen_manifest_entries(
    self,
    manifest: list[dict[str, Any]],
    installed: dict[str, Any],
    filtered: list[Game],
    seen_ids: set[str],
    seen_norms: set[str],
    source_label: str,
) -> int:

    """Inject unseen manifest entries."""
    injected = 0
    for entry in manifest:
        game_id = self._game_id_for(entry)
        norm_title = self._id_map.normalize_for_matching(
            entry["title"],
        )
        if (
            game_id in seen_ids
            or norm_title in seen_norms
        ):
            continue
        install_meta = (
            installed.get(entry.get("space_id", ""))
            or installed.get(game_id)
            or {}
        )
        cover_image = entry.get("cover_image") or None
        game = Game(
            app_id=0,
            store="ubisoft",
            store_game_id=game_id,
            title=entry["title"],
            installed=bool(install_meta),
            icon_url=cover_image,
            install_path=install_meta.get("install_path"),
            exe_path=install_meta.get("executable"),
            metadata={
                "ownership_type": (
                    entry.get("ownership_type") or "owned"
                ),
            },
        )
        if cover_image:
            game.metadata.update({
                "coverUrl": cover_image,
                "backgroundUrl": "",
                "bannerUrl": "",
            })
        filtered.append(game)
        seen_ids.add(game_id)
        seen_norms.add(norm_title)
        injected += 1
    logger.info(
        "[UbisoftLibrary] visible %s injected: %s "
        "[id=%s]",
        source_label, entry["title"], game_id,
    )

```

return injected

OP-57f

detection.py

Fichier : py_modules/unifideck/stores/ubisoft/library/detection.py | Lignes : 233 | Depend de : (rien)

```

from __future__ import annotations
import logging
from pathlib import Path
from typing import Any
from ..config import UbisoftConfig
from ..id_map import UbisoftIdMap
from .detection_cascade import _DetectionCascade
from .detection_helpers import (
    _DetectionHelpers,
    find_game_executable as _find_game_executable_impl,
    in_prefix_game_roots,
    load_json_file_safe as _load_json_file_safe_impl,
    write_install_marker as _write_install_marker_impl,
)
logger = logging.getLogger(__name__)
class _InstallDetector:
    """Install detector."""
    def __init__(
        self,
        config: UbisoftConfig,
        id_map: UbisoftIdMap,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._id_map = id_map
        self._cascade = _DetectionCascade(self)
        self._helpers = _DetectionHelpers(self)
    @staticmethod
    def find_game_executable(
        install_path: str,
    ) -> str | None:
        """Find game executable."""
        return _find_game_executable_impl(install_path)
    async def write_install_marker(
        self,
        space_id: str,
        install_path: str,
        executable: str,
        game_title: str = "",
    ) -> None:
        """Write install marker."""
        await _write_install_marker_impl(
            space_id, install_path, executable, game_title,
        )
    @staticmethod
    def load_json_file_safe(path: str) -> Any | None:
        """Load JSON file safe."""
        return _load_json_file_safe_impl(path)
    @staticmethod
    def get_game_official_url(game_id: str) -> str:
        """Get game official URL."""
        return f"https://store.ubisoft.com/game?pid={game_id}"

    async def get_installed(self) -> dict[str, Any]:

```

```

    """Get installed."""
    installed: dict[str, Any] = {}
    prefixes_dir = Path(self._config.prefixes_dir_expanded)
    if not prefixes_dir.is_dir():
        return installed
    try:
        entries = list(prefixes_dir.iterdir())
    except OSError as e:
        logger.warning(
            "[UbisoftLibrary] prefixes_dir scan failed: "
            "%s",
            e,
        )
        return installed
    for entry in entries:
        if entry.name.startswith("."):
            continue
        if not entry.is_dir():
            continue
        marker_path = entry / self._config.bootstrap_marker
        if not marker_path.is_file():
            continue
        game_info = self._detect_installed_game(
            entry.name, str(entry),
        )
        if not game_info:
            continue
        installed[entry.name] = game_info
        await self._auto_resolve_missing_id(
            entry.name, str(entry), game_info,
        )
    return installed
def get_installed_game_info(
    self, game_id: str,
) -> dict[str, Any] | None:
    """Get installed game info."""
    prefix_path = (
        Path(self._config.prefixes_dir_expanded)
        / game_id
    )
    if not prefix_path.is_dir():
        return None
    marker_path = (
        prefix_path / self._config.bootstrap_marker
    )
    if not marker_path.is_file():
        return None
    info = self._detect_installed_game(
        game_id, str(prefix_path),
    )
    if info:
        self._auto_resolve_id_from_registry(
            game_id, str(prefix_path), info,
        )
    return info
def _auto_resolve_id_from_registry(
    self,
    space_id: str,
    prefix_path: str,
    game_info: dict[str, Any],

```



```

) -> None:

    """Auto resolve ID from registry."""
    existing = self._id_map.get_entry(space_id)
    if (
        existing.get("launch_id")
        or existing.get("ubisoftconnect_game_id")
    ):
        return
    reg_id = UbisoftIdMap.extract_game_id_from_registry(
        prefix_path,
    )
    if not reg_id:
        return
    self._id_map.merge_entry(space_id, {
        "install_id": reg_id,
        "launch_id": reg_id,
        "ubisoftconnect_game_id": reg_id,
        "name": game_info.get("title", ""),
    })
    logger.info(
        "[UbisoftLibrary] auto-resolved game ID for %s: "
        "%s",
        space_id, reg_id,
    )
async def _auto_resolve_missing_id(
    self,
    space_id: str,
    prefix_path: str,
    game_info: dict[str, Any],
) -> None:
    """Auto resolve missing ID."""
    existing = self._id_map.get_entry(space_id)
    if (
        existing.get("launch_id")
        or existing.get("ubisoftconnect_game_id")
    ):
        return
    reg_id = UbisoftIdMap.extract_game_id_from_registry(
        prefix_path,
    )
    if not reg_id:
        game_title = game_info.get("title", "")
        if game_title:
            reg_id = (
                await self._id_map.lookup_game_id_by_name(
                    game_title,
                )
            )
    if not reg_id:
        return
    self._id_map.merge_entry(space_id, {
        "install_id": reg_id,
        "launch_id": reg_id,
        "ubisoftconnect_game_id": reg_id,
        "name": game_info.get("title", ""),
    })
    logger.info(
        "[UbisoftLibrary] auto-resolved game ID for %s: "
        "%s",
        space_id, reg_id,
    )

```

```

    )

def _detect_installed_game(
    self, space_id: str, prefix_path: str,
) -> dict[str, Any] | None:

    """Detect installed game."""
    try:
        from ..parser import check_install_state
    except ImportError as e:
        logger.debug(
            "[UbisoftLibrary] ubisoft_parser "
            "unavailable: %s",
            e,
        )
        return None
    known_name = self._get_game_name(space_id) or ""
    normalized_known_name = (
        self._id_map.normalize_for_matching(known_name)
        if known_name
        else ""
    )
    prefix_game_roots = in_prefix_game_roots(prefix_path)
    external_game_roots = (
        self._helpers.get_external_game_roots()
    )
    method1 = self._cascade.detect_via_marker(
        space_id,
        known_name,
        [*prefix_game_roots, *external_game_roots],
    )
    if method1:
        return method1
    method2 = self._cascade.detect_via_prefix_install_state(
        space_id,
        prefix_game_roots,
        normalized_known_name,
        known_name,
        check_install_state,
    )
    if method2:
        return method2
    if normalized_known_name:
        method3 = self._cascade.detect_via_external_roots(
            space_id,
            external_game_roots,
            normalized_known_name,
            known_name,
            check_install_state,
        )
        if method3:
            return method3
    return self._cascade.detect_via_registry_install_id(
        space_id,
        prefix_path,
        known_name,
        check_install_state,
    )

def _get_game_name(self, space_id: str) -> str | None:
    """Get game name."""
    entry = self._id_map.get_entry(space_id)

```

```
return entry.get("name")
```

OP-57g

detection_cascade.py

Fichier : py_modules/unifideck/stores/ubisoft/library/detection_cascade.py | Lignes : 296 | Depend de : (rien)

```

from __future__ import annotations
import logging
import re
from collections.abc import Callable
from pathlib import Path
from typing import TYPE_CHECKING, Any
from .detection_helpers import (
    load_json_file_safe,
    looks_like_game_install,
    walk_install_candidates,
    write_marker_sync,
)
from .wine_path import wine_path_to_linux
if TYPE_CHECKING:
    from .detection import _InstallDetector
logger = logging.getLogger(__name__)
_INSTALL_MARKER_FILENAME = ".unifideck_ubisoft"
class _DetectionCascade:
    """Detection cascade."""
    def __init__(self, parent: _InstallDetector) -> None:
        """Initialize the instance."""
        self._parent = parent
    def detect_via_marker(
        self,
        space_id: str,
        known_name: str,
        search_roots: list[str],
    ) -> dict[str, Any] | None:
        """Detect via marker."""
        for game_dir, folder in walk_install_candidates(
            search_roots,
        ):
            marker_data = self._load_marker_for_space(
                game_dir, space_id,
            )
            if marker_data is None:
                continue
            return self._build_marker_result(
                space_id, known_name, game_dir, folder,
                marker_data,
            )
        return None
    @staticmethod
    def _load_marker_for_space(
        game_dir: str, space_id: str,
    ) -> dict | None:
        """Load marker for space."""
        marker_path = Path(game_dir) / _INSTALL_MARKER_FILENAME
        if not marker_path.is_file():
            return None
        marker_data = load_json_file_safe(str(marker_path))
        if not isinstance(marker_data, dict):
            return None
        if marker_data.get("space_id") != space_id:
            return None

```

```

        return marker_data

def _build_marker_result(
    self,
    space_id: str,
    known_name: str,
    game_dir: str,
    folder: str,
    marker_data: dict,
) -> dict[str, Any]:

    """Build marker result."""
    install_path = (
        marker_data.get("install_path") or game_dir
    )
    executable = self._resolve_marker_executable(
        marker_data, install_path,
    )
    return {
        "space_id": space_id,
        "executable": executable,
        "install_path": install_path,
        "work_dir": install_path,
        "title": (
            marker_data.get("game_title")
            or known_name
            or folder
        ),
    }

def _resolve_marker_executable(
    self, marker_data: dict, install_path: str,
) -> str:
    """Resolve marker executable."""
    executable = marker_data.get("executable", "") or ""
    exe_path = Path(executable) if executable else None
    if exe_path and not exe_path.is_absolute():
        exe_path = Path(install_path) / executable
        executable = str(exe_path)
    if not executable or not Path(executable).exists():
        return (
            self._parent.find_game_executable(install_path)
            or ""
        )
    return executable

def detect_via_prefix_install_state(
    self,
    space_id: str,
    prefix_game_roots: list[str],
    normalized_known_name: str,
    known_name: str,
    check_install_state: Callable[[str], bool],
) -> dict[str, Any] | None:
    """Detect via prefix install state."""
    candidates: list[str] = []
    for game_dir, _folder in walk_install_candidates(
        prefix_game_roots,
    ):
        state_file = str(
            Path(game_dir) / "uplay_install.state",
        )
        if check_install_state(state_file):

```

```

        candidates.append(game_dir)
    if not candidates:
        return None
    if normalized_known_name:
        for game_dir in candidates:
            if self.fuzzy_folder_match(
                Path(game_dir).name,
                normalized_known_name,
            ):
                return self.build_install_info(
                    space_id,
                    game_dir,
                    known_name or Path(game_dir).name,
                )
    first_dir = candidates[0]
    return self.build_install_info(
        space_id,
        first_dir,
        known_name or Path(first_dir).name,
    )

def detect_via_external_roots(
    self,
    space_id: str,
    external_game_roots: list[str],
    normalized_known_name: str,
    known_name: str,
    check_install_state: Callable[[str], bool],
) -> dict[str, Any] | None:

    """Detect via external roots."""
    for game_dir, folder in walk_install_candidates(
        external_game_roots,
    ):
        state_file = str(
            Path(game_dir) / "uplay_install.state",
        )
        if not check_install_state(state_file):
            continue
        if not self.fuzzy_folder_match(
            folder, normalized_known_name,
        ):
            continue
        return self.build_install_info(
            space_id, game_dir, known_name or folder,
        )
    return None

def detect_via_registry_install_id(
    self,
    space_id: str,
    prefix_path: str,
    known_name: str,
    check_install_state: Callable[[str], bool],
) -> dict[str, Any] | None:
    """Detect via registry install ID."""
    install_id = self._parent._id_map.resolve_install_id(
        space_id,
    )
    if not install_id:
        return None
    install_section_pattern = self._build_registry_pattern(

```

```

        install_id,
    )
    for reg_name in ("pfx/system.reg", "system.reg"):
        reg_path = str(Path(prefix_path) / reg_name)
        result = self._try_registry_file(
            reg_path,
            install_section_pattern,
            space_id,
            install_id,
            prefix_path,
            known_name,
            check_install_state,
        )
        if result is not None:
            return result
    return None
@staticmethod
def _build_registry_pattern(install_id: str) -> re.Pattern:
    """Build registry pattern."""
    return re.compile(
        r'\[Software\\(?:Wow6432Node\\)?'
        r'Ubisoft\\Launcher\\Installs\\'
        + re.escape(install_id)
        + r'\]'
        r'^\[.*?InstallDir\s*=\s*"([^"]*)"',
        re.DOTALL,
    )

def _try_registry_file(
    self,
    reg_path: str,
    pattern: re.Pattern,
    space_id: str,
    install_id: str,
    prefix_path: str,
    known_name: str,
    check_install_state: Callable[[str], bool],
) -> dict[str, Any] | None:

    """Try registry file."""
    reg_p = Path(reg_path)
    if not reg_p.is_file():
        return None
    try:
        reg_content = reg_p.read_text(
            encoding="utf-8", errors="replace",
        )
    except OSError:
        return None
    for match in pattern.finditer(reg_content):
        install_dir_raw = (
            match.group(1).replace("\\\\", "/")
        )
        linux_path = wine_path_to_linux(
            install_dir_raw, prefix_path,
        )
        if not self._validate_registry_install(
            linux_path, check_install_state,
        ):
            continue
        assert linux_path is not None

```

```

        logger.info(
            "[UbisoftLibrary] method 4: registry "
            "InstallDir for %s: %s",
            install_id, linux_path[:80],
        )
        result = self.build_install_info(
            space_id,
            linux_path,
            known_name or Path(linux_path).name,
        )
        write_marker_sync(
            linux_path, space_id, result["title"],
        )
        return result
    return None

    @staticmethod
    def _validate_registry_install(
        linux_path: str | None,
        check_install_state: Callable[[str], bool],
    ) -> bool:
        """Validate registry install."""
        if not linux_path or not Path(linux_path).is_dir():
            return False
        state_file = str(
            Path(linux_path) / "uplay_install.state",
        )
        return (
            check_install_state(state_file)
            or looks_like_game_install(linux_path)
        )

    def build_install_info(
        self,
        space_id: str,
        game_dir: str,
        title_hint: str,
    ) -> dict[str, Any]:
        """Build install info."""
        exe = self._parent.find_game_executable(game_dir) or ""
        title = title_hint or Path(game_dir).name
        return {
            "space_id": space_id,
            "executable": exe,
            "install_path": game_dir,
            "work_dir": game_dir,
            "title": title,
        }

    def fuzzy_folder_match(
        self,
        folder_name: str,
        normalized_known_name: str,
    ) -> bool:
        """Fuzzy folder match."""
        if not normalized_known_name:
            return False
        normalized_folder = (
            self._parent._id_map.normalize_for_matching(
                folder_name,
            )
        )

```



```
return (  
    normalized_folder == normalized_known_name  
    or normalized_folder in normalized_known_name  
    or normalized_known_name in normalized_folder  
)
```

OP-57h

detection_helpers.py

Fichier : py_modules/unifideck/stores/ubisoft/library/detection_helpers.py | Lignes : 256 | Depend de : (rien)

```

from __future__ import annotations
import datetime
import glob
import json
import logging
import os
from collections.abc import Iterator
from pathlib import Path
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from .detection import _InstallDetector
logger = logging.getLogger(__name__)
_EXE_SKIP_PATTERNS = (
    "unins", "setup", "install", "crash", "redist",
    "vcredist", "dxsetup", "dotnet", "upc", "uplay",
)
_GAME_INSTALL_MIN_SIZE = 100 * 1024 * 1024
_IN_PREFIX_GAMES_PATH = str(
    Path("drive_c") / "Program Files (x86)"
    / "Ubisoft" / "Ubisoft Game Launcher" / "games"
)
_INSTALL_MARKER_FILENAME = ".unifideck_ubisoft"
def load_json_file_safe(path: str) -> Any | None:
    """Load JSON file safe."""
    try:
        return json.loads(
            Path(path).read_text(
                encoding="utf-8", errors="replace",
            ),
        )
    except (OSError, json.JSONDecodeError):
        return None
def walk_install_candidates(
    roots: list[str],
) -> Iterator[tuple[str, str]]:
    """Walk install candidates."""
    for base_dir in roots:
        base = Path(base_dir)
        if not base.is_dir():
            continue
        try:
            entries = list(base.iterdir())
        except OSError:
            continue
        for entry in entries:
            if not entry.is_dir():
                continue
            yield str(entry), entry.name
def in_prefix_game_roots(prefix_path: str) -> list[str]:
    """In prefix game roots."""
    prefix = Path(prefix_path)
    return [
        str(prefix / _IN_PREFIX_GAMES_PATH),
        str(prefix / "pfx" / _IN_PREFIX_GAMES_PATH),
    ]

```

```

def find_game_executable(
    install_path: str,
) -> str | None:

    """Find game executable."""
    if not install_path or not Path(install_path).is_dir():
        return None
    candidates: list[tuple[str, int]] = []
    for pattern in ("*.exe", "**/*.exe"):
        for exe_path in glob.glob(
            str(Path(install_path) / pattern),
            recursive=True,
        ):
            basename = Path(exe_path).name.lower()
            if any(
                skip in basename
                for skip in _EXE_SKIP_PATTERNS
            ):
                continue
            try:
                size = Path(exe_path).stat().st_size
                candidates.append((exe_path, size))
            except OSError:
                continue
    if not candidates:
        logger.warning(
            "[UbisoftLibrary] no executable found in %s",
            install_path,
        )
        return None
    candidates.sort(key=lambda x: x[1], reverse=True)
    result, size = candidates[0]
    logger.info(
        "[UbisoftLibrary] found executable (%.1f MB): %s",
        size / (1024 * 1024), result,
    )
    return result

def looks_like_game_install(path: str) -> bool:
    """Looks like game install."""
    try:
        for root, _dirs, files in os.walk(path):
            for f in files:
                if f.lower().endswith(".exe"):
                    return True
            depth = root[len(path):].count(os.sep)
            if depth >= 2:
                break
        total = 0
        for root, _dirs, files in os.walk(path):
            for f in files:
                try:
                    total += (Path(root) / f).stat().st_size
                except OSError:
                    continue
            if total > _GAME_INSTALL_MIN_SIZE:
                return True
    except OSError:
        pass
    return False

```

```

async def write_install_marker(
    space_id: str,
    install_path: str,
    executable: str,
    game_title: str = "",
) -> None:

    """Write install marker."""
    try:
        marker_data = {
            "space_id": space_id,
            "game_title": game_title,
            "install_path": install_path,
            "executable": executable,
            "install_date": (
                datetime.datetime.now().isoformat()
            ),
        }
        install_p = Path(install_path)
        marker_path = install_p / _INSTALL_MARKER_FILENAME
        tmp_path = marker_path.with_suffix(
            marker_path.suffix + ".tmp",
        )
        install_p.mkdir(parents=True, exist_ok=True)
        tmp_path.write_text(
            json.dumps(marker_data, indent=2),
            encoding="utf-8",
        )
        tmp_path.replace(marker_path)
        logger.info(
            "[UbisoftLibrary] wrote install marker for %s",
            space_id,
        )
    except OSError as e:
        logger.warning(
            "[UbisoftLibrary] marker write failed: %s", e,
        )

def write_marker_sync(
    install_path: str,
    space_id: str,
    title: str,
) -> None:
    """Write marker sync."""
    marker_path = (
        Path(install_path) / _INSTALL_MARKER_FILENAME
    )
    if marker_path.exists():
        return
    marker_data = {
        "space_id": space_id,
        "install_path": install_path,
        "game_title": title,
    }
    try:
        marker_path.write_text(
            json.dumps(marker_data),
            encoding="utf-8",
        )
    except OSError:
        pass

class _DetectionHelpers:

```

```

"""Detection helpers."""
def __init__(self, parent: _InstallDetector) -> None:
    """Initialize the instance."""
    self._parent = parent
def get_external_game_roots(self) -> list[str]:
    """Get external game roots."""
    config = self._parent._config
    roots: list[str] = [
        config.default_install_base_expanded,
        config.sdcard_install_base,
    ]
    self._append_custom_path_root(roots, config)
    self._append_mounted_media_roots(roots)
    return self._dedup_roots_by_realpath(roots)

@staticmethod
def _append_custom_path_root(
    roots: list[str], config: Any,
) -> None:
    """Append custom path root."""
    if config is None:
        return
    settings_file = str(
        Path(config.data_dir_expanded)
        / "download_settings.json"
    )
    if not Path(settings_file).is_file():
        return
    settings = load_json_file_safe(settings_file)
    if not isinstance(settings, dict):
        return
    custom_path = settings.get("custom_path")
    if isinstance(custom_path, str) and custom_path:
        roots.append(
            str(Path(custom_path) / "Ubisoft"),
        )
        roots.append(custom_path)

@staticmethod
def _append_mounted_media_roots(roots: list[str]) -> None:
    """Append mounted media roots."""
    media_base = Path("/run/media")
    if not media_base.is_dir():
        return
    try:
        for entry_path in media_base.iterdir():
            if not entry_path.is_dir():
                continue
            roots.append(
                str(entry_path / "Games" / "Ubisoft"),
            )
            _DetectionHelpers._append_sub_mount_roots(
                entry_path, roots,
            )
    except OSError:
        pass

@staticmethod
def _append_sub_mount_roots(
    parent: Path, roots: list[str],
) -> None:
    """Append sub mount roots."""
    try:

```

```
        for sub_path in parent.iterdir():
            if sub_path.is_dir():
                roots.append(
                    str(sub_path / "Games" / "Ubisoft"),
                )
    except OSError:
        pass
    @staticmethod
    def _dedup_roots_by_realpath(
        roots: list[str],
    ) -> list[str]:
        """Dedup roots by realpath."""
        seen: set[str] = set()
        unique: list[str] = []
        for r in roots:
            try:
                real = str(Path(r).resolve())
            except OSError:
                real = r
            if real not in seen:
                seen.add(real)
                unique.append(r)
        return unique
```

OP-57i

wine_path.py

Fichier : py_modules/unifideck/stores/ubisoft/library/wine_path.py | Lignes : 43 | Depend de : (rien)

```
from __future__ import annotations
from pathlib import Path
def wine_path_to_linux(
    wine_path: str, prefix_path: str,
) -> str | None:
    """Wine path to linux."""
    path = wine_path.replace("\\", "/")
    if len(path) < 2 or path[1] != ":":
        return None
    drive_letter = path[0].upper()
    relative = path[2:].lstrip("/")
    if drive_letter == "Z":
        return _resolve_z_drive(relative)
    if drive_letter == "C":
        return _resolve_c_drive(prefix_path, relative)
    return _resolve_other_drive(
        prefix_path, drive_letter, relative,
    )
def _resolve_z_drive(relative: str) -> str:
    """Resolve z drive."""
    return "/" + relative if relative else "/"
def _resolve_c_drive(prefix_path: str, relative: str) -> str:
    """Resolve c drive."""
    prefix = Path(prefix_path)
    for base in (prefix / "pfx", prefix):
        candidate = base / "drive_c" / relative
        if candidate.exists():
            return str(candidate)
    return str(prefix / "pfx" / "drive_c" / relative)
def _resolve_other_drive(
    prefix_path: str, drive_letter: str, relative: str,
) -> str | None:
    """Resolve other drive."""
    drive_name = f"{drive_letter.lower()}:"
    prefix = Path(prefix_path)
    for base in (prefix / "pfx", prefix):
        link_path = base / "dosdevices" / drive_name
        if link_path.is_symlink():
            target = str(link_path.resolve())
            if relative:
                return str(Path(target) / relative)
            return target
    return None
```

OP-58

`__init__.py`

Fichier : `py_modules/unifideck/stores/ubisoft/auth/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from .facade import UbisoftAuth, UbisoftAuthServices, UbisoftAuthState
__all__ = ["UbisoftAuth", "UbisoftAuthServices", "UbisoftAuthState"]
```


OP-58a

facade.py

Fichier : py_modules/unifideck/stores/ubisoft/auth/facade.py | Lignes : 164 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
import shutil
from collections.abc import Callable
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any
from ...core.types import AuthResult, Events, Result
from ...security import (
    audit_auth_flow,
)
from ..binaries import UbisoftBinaryResolver
from ..config import UbisoftConfig
from ..paths import UbisoftPrefixPaths
from ..session import UbisoftSession
from .context import _AuthContext
from .direct_signin import _DirectSignIn
from .session_monitor import _AuthSessionMonitor
from .shortcut import _AuthShortcut
from .shortcut_ops import _ShortcutRegistryOps
if TYPE_CHECKING:
    from ...event_bus.event_bus import EventBus
    from ...services.shortcut import ShortcutService
    from ...steam.steamgriddb import SteamGridDBClient
logger = logging.getLogger(__name__)
@dataclass(frozen=True)
class UbisoftAuthState:
    """Ubisoft auth state."""
    config: UbisoftConfig
    paths: UbisoftPrefixPaths
    binaries: UbisoftBinaryResolver
    session: UbisoftSession
    ensure_auth_prefix: Callable[[[]], Any]
    queue_auth_assets_ensure: Callable[[str], None]
@dataclass(frozen=True)
class UbisoftAuthServices:
    """Ubisoft auth services."""
    plugin_dir: str | None
    shortcut_service: ShortcutService | None
    steamgriddb: SteamGridDBClient | None
class UbisoftAuth:
    """Ubisoft auth."""
    def __init__(
        self,
        bus: EventBus,
        state: UbisoftAuthState,
        services: UbisoftAuthServices,
    ) -> None:
        """Initialize the instance."""
        self._bus = bus
        self._config = state.config
        self._paths = state.paths
        self._binaries = state.binaries

```

```

self._session = state.session
self._ensure_auth_prefix = state.ensure_auth_prefix
self._queue_auth_assets_ensure = state.queue_auth_assets_ensure
self._plugin_dir = services.plugin_dir
self._shortcut_service = services.shortcut_service
self._steamgriddb = services.steamgriddb
self._registry_ops = _ShortcutRegistryOps(config=self._config)
self._monitor = _AuthSessionMonitor(
    config=self._config,
    session=self._session,
    queue_auth_assets_ensure=self._queue_auth_assets_ensure,
)
self._direct_signin = _DirectSignIn(
    binaries=self._binaries,
    bus=self._bus,
    config=self._config,
    paths=self._paths,
    session=self._session,
    ensure_auth_prefix=self._ensure_auth_prefix,
    queue_auth_assets_ensure=self._queue_auth_assets_ensure,
)
self._shortcut = _AuthShortcut(self)
self._context = _AuthContext(self)
async def ensure_auth_shortcut(self) -> int | None:
    """Ensure auth shortcut."""
    return await self._shortcut.ensure_auth_shortcut()
async def auth_shortcut_exists_in_vdf(self) -> bool:
    """Auth shortcut exists in VDF."""
    return await self._shortcut.auth_shortcut_exists_in_vdf()
async def fetch_auth_shortcut_artwork(
    self, unsigned_id: int, force: bool = False,
) -> None:
    """Fetch auth shortcut artwork."""
    await self._context.fetch_auth_shortcut_artwork(
        unsigned_id, force=force,
    )
async def get_auth_shortcut_context(self) -> dict[str, Any]:
    """Get auth shortcut context."""
    return await self._context.get_auth_shortcut_context()
async def is_available(self) -> bool:
    """Check whether available."""
    auth_dir = self._config.auth_prefix_dir_expanded
    return self._session.has_valid_credentials(auth_dir)
@audit_auth_flow(store="ubisoft", method="wine_installer")
async def start_auth(self) -> AuthResult:
    """Start auth."""
    return AuthResult(
        success=True,
        store="ubisoft",
        metadata={
            "auth_type": "upc_launch",
            "message": "Sign in through Ubisoft Connect",
        },
    )
async def complete_auth(
    self, code: str = "", **kwargs: Any,
) -> AuthResult:
    """Complete auth."""
    if await self.is_available():
        return AuthResult(
            success=True, store="ubisoft",

```

```

    )
    return AuthResult(
        success=False,
        store="ubisoft",
        error="not_authenticated",
    )

async def logout(self) -> Result:

    """Logout."""
    self._session.clear_session_file()
    auth_dir = self._config.auth_prefix_dir_expanded
    if os.path.isdir(auth_dir):
        try:
            shutil.rmtree(auth_dir)
            logger.info(
                "[UbisoftAuth] removed auth prefix directory",
            )
        except OSError as e:
            logger.error(
                "[UbisoftAuth] could not remove auth "
                "prefix: %s", e,
            )
    await self._bus.emit(
        Events.STORE_LOGOUT, store="ubisoft",
    )
    return Result(success=True)

async def start_auth_session_monitor(self) -> Result:
    """Start auth session monitor."""
    return await self._monitor.start()

def check_auth_session_status(self) -> dict[str, Any]:
    """Check auth session status."""
    return self._monitor.status()

async def connect_ubisoft_account(self) -> dict[str, Any]:
    """Connect UBISOFT account."""
    return await self._direct_signin.connect()

async def _load_registry(self, sm: ShortcutService) -> dict[str, Any]:
    """Load registry."""
    return await self._registry_ops.load(sm)

async def _register_shortcut(
    self, sm: ShortcutService, appid: int, name: str,
) -> None:
    """Register shortcut."""
    await self._registry_ops.register(sm, appid, name)

async def _clear_compat(
    self, sm: ShortcutService, appid: int,
) -> None:
    """Clear compat."""
    await self._registry_ops.clear_compat(sm, appid)

async def _cleanup_legacy_registry(self, sm: ShortcutService) -> None:
    """Cleanup legacy registry."""
    await self._registry_ops.cleanup_legacy(sm)

```

OP-58b

context.py

Fichier : py_modules/unifideck/stores/ubisoft/auth/context.py | Lignes : 150 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from .facade import UbisoftAuth
logger = logging.getLogger(__name__)
_SGDB_UBISOFT_CONNECT_ID = 5270094
_AUTH_SHORTCUT_NAME = "Ubisoft Connect"
class _AuthContext:
    """Auth context."""
    def __init__(self, parent: UbisoftAuth) -> None:
        """Initialize the instance."""
        self._parent = parent

    async def fetch_auth_shortcut_artwork(
        self, unsigned_id: int, force: bool = False,
    ) -> None:
        """Fetch auth shortcut artwork."""
        sgdb = self._parent._steamgriddb
        if sgdb is None:
            logger.debug(
                "[UbisoftAuth] SteamGridDB client not "
                "available, skipping artwork",
            )
            return
        try:
            if (
                not force
                and hasattr(sgdb, "has_artwork")
                and await sgdb.has_artwork(unsigned_id)
            ):
                logger.debug(
                    "[UbisoftAuth] auth shortcut "
                    "artwork already exists",
                )
                return
            only_types = None
            if not force and hasattr(
                sgdb, "get_missing_artwork_types",
            ):
                missing = (
                    await sgdb.get_missing_artwork_types(
                        unsigned_id,
                    )
                )
                if missing:
                    only_types = missing
                    logger.info(
                        "[UbisoftAuth] auth shortcut "
                        "artwork gap-fill: %s", missing,
                    )
            logger.info(
                "[UbisoftAuth] fetching SteamGridDB "
                "artwork for Ubisoft Connect (force=%s)",
```

```

        force,
    )
    await sgdb.fetch_game_art(
        title=_AUTH_SHORTCUT_NAME,
        app_id=unsigned_id,
        only_types=only_types,
        sgdb_game_id=_SGDB_UBISOFT_CONNECT_ID,
    )
except Exception as e:
    logger.warning(
        "[UbisoftAuth] auth shortcut artwork "
        "fetch failed: %s", e,
    )
def build_auth_context_success(
    self,
    unsigned_appid: int,
    *,
    with_launch_wait: bool = True,
) -> dict[str, Any]:
    """Build auth context success."""
    return {
        "success": True,
        "appid_unsigned": unsigned_appid,
        "launch_wait_ms": (
            self._parent._config.auth_shortcut_launch_wait_ms
            if with_launch_wait
            else 0
        ),
    }

async def get_auth_shortcut_context(self) -> dict[str, Any]:
    """Get auth shortcut context."""
    if self._parent._shortcut_service is None:
        return {
            "success": False,
            "error": "shortcut_service_unavailable",
        }
    sm = self._parent._shortcut_service
    from_registry = await self._try_existing_registry(sm)
    if from_registry is not None:
        return from_registry
    logger.info(
        "[UbisoftAuth] auth shortcut not in registry, "
        "scanning VDF for recovery",
    )
    vdf_found = await (
        self._parent._shortcut.validate_auth_shortcut(sm)
    )
    if vdf_found:
        store_id = self._parent._config.auth_shortcut_store_id
        registry = await self._parent._load_registry(sm)
        entry = registry.get(store_id)
        if entry and entry.get("appid_unsigned"):
            logger.info(
                "[UbisoftAuth] auth shortcut "
                "recovered from VDF: appid=%d",
                entry["appid_unsigned"],
            )
        return self.build_auth_context_success(
            entry["appid_unsigned"],

```

```

        )
    unsigned_id = (
        await self._parent.ensure_auth_shortcut()
    )
    if not unsigned_id:
        return {
            "success": False,
            "error": "auth_shortcut_not_ready",
        }
    return self.build_auth_context_success(unsigned_id)
async def _try_existing_registry(
    self, sm: Any,
) -> dict[str, Any] | None:
    """Try existing registry."""
    store_id = self._parent._config.auth_shortcut_store_id
    registry = await self._parent._load_registry(sm)
    entry = registry.get(store_id)
    if not entry or not entry.get("appid_unsigned"):
        return None
    if await self._parent.auth_shortcut_exists_in_vdf():
        logger.info(
            "[UbisoftAuth] auth shortcut context: "
            "appid=%d", entry["appid_unsigned"],
        )
        return self.build_auth_context_success(
            entry["appid_unsigned"],
            with_launch_wait=False,
        )
    logger.info(
        "[UbisoftAuth] auth shortcut in registry "
        "but missing from VDF, recreating",
    )
    unsigned_id = (
        await self._parent.ensure_auth_shortcut()
    )
    if unsigned_id:
        return self.build_auth_context_success(unsigned_id)
    return None

```

OP-58c

shortcut.py

Fichier : py_modules/unifideck/stores/ubisoft/auth/shortcut.py | Lignes : 353 | Depend de : (rien)

```

from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING, Any, cast
if TYPE_CHECKING:
    from ...services.shortcut import ShortcutService
    from .facade import UbisoftAuth
logger = logging.getLogger(__name__)
_AUTH_LAUNCH_OPTIONS_TEMPLATE = (
    "{store_id} "
    "UNIFIDECK_UBISOFT_ACTION=auth "
    "UNIFIDECK_UBISOFT_PREFIX_NAME={prefix_name}"
)
_AUTH_SHORTCUT_NAME = "Ubisoft Connect"
_LEGACY_AUTH_LAUNCH_OPTIONS = "ubisoft:.template"
_ORPHAN_SHORTCUT_NAMES = frozenset(
    {"upc.exe", "ubisoft connect"},
)
def _prune_orphan_shortcuts(shortcuts: dict[str, Any]) -> int:
    """Prune orphan shortcuts."""
    orphan_ids = [
        idx for idx, s in shortcuts.items()
        if s.get("AppName", "").lower() in _ORPHAN_SHORTCUT_NAMES
        and not s.get("exe", "").strip(' ')
        and not s.get("LaunchOptions", "")
    ]
    for idx in orphan_ids:
        name = shortcuts[idx].get("AppName", "?")
        logger.info(
            "[UbisoftAuth] removing orphaned shortcut "
            "[%s] %r", idx, name,
        )
        del shortcuts[idx]
    return len(orphan_ids)
def _prune_legacy_template_shortcuts(
    shortcuts: dict[str, Any],
) -> int:
    """Prune legacy template shortcuts."""
    legacy_ids = [
        idx for idx, s in shortcuts.items()
        if s.get("LaunchOptions", "")
        == _LEGACY_AUTH_LAUNCH_OPTIONS
    ]
    for idx in legacy_ids:
        logger.info(
            "[UbisoftAuth] removing legacy .template "
            "shortcut [%s]", idx,
        )
        del shortcuts[idx]
    return len(legacy_ids)
class _AuthShortcut:
    """Auth shortcut."""
    def __init__(self, parent: UbisoftAuth) -> None:
        """Initialize the instance."""
        self._parent = parent

```

```

def get_launcher_path(self) -> str:

    """Get launcher path."""
    plugin_dir = self._parent._plugin_dir
    if not plugin_dir:
        plugin_dir = str(
            Path(__file__).resolve()
            .parent.parent.parent.parent,
        )
    return str(
        Path(plugin_dir)
        / "py_modules" / "unifideck" / "launcher"
        / "dispatcher.py",
    )

def build_auth_launch_options(self) -> str:
    """Build auth launch options."""
    return _AUTH_LAUNCH_OPTIONS_TEMPLATE.format(
        store_id=(
            self._parent._config.auth_shortcut_store_id
        ),
        prefix_name=self._parent._config.auth_prefix_name,
    )

async def ensure_auth_shortcut(self) -> int | None:
    """Ensure auth shortcut."""
    if self._parent._shortcut_service is None:
        logger.debug(
            "[UbisoftAuth] no shortcut_service; "
            "skipping auth shortcut creation",
        )
        return None
    try:
        sm = self._parent._shortcut_service
        store_id = (
            self._parent._config.auth_shortcut_store_id
        )
        existing_appid = await self.try_existing_shortcut(
            sm, store_id,
        )
        if existing_appid is not None:
            return existing_appid
        return await self.create_new_auth_shortcut(
            sm, store_id,
        )
    except Exception as e:
        logger.warning(
            "[UbisoftAuth] auth shortcut creation "
            "failed: %s", e,
        )
        return None

async def try_existing_shortcut(
    self, sm: ShortcutService, store_id: str,
) -> int | None:

    """Try existing shortcut."""
    registry = await self._parent._load_registry(sm)
    if store_id not in registry:
        return None
    vdf_found = await self.validate_auth_shortcut(sm)
    if vdf_found:

```



```

        uid = registry[store_id].get("appid_unsigned")
        if uid:
            await (
                self._parent.fetch_auth_shortcut_artwork(
                    uid,
                )
            )
            return cast("int | None", uid)
    entry = registry[store_id]
    appid = entry.get("appid")
    unsigned_id = entry.get("appid_unsigned")
    if not (appid and unsigned_id):
        return None
    logger.info(
        "[UbisoftAuth] recreating auth shortcut VDF "
        "from registry (appid=%d)", unsigned_id,
    )
    await self.add_shortcut_to_vdf(sm, appid)
    await self._parent._clear_compat(sm, appid)
    await self._parent.fetch_auth_shortcut_artwork(
        unsigned_id, force=True,
    )
    return cast("int | None", unsigned_id)
async def create_new_auth_shortcut(
    self, sm: ShortcutService, store_id: str,
) -> int | None:
    """Create new auth shortcut."""
    launcher_path = self.get_launcher_path()
    appid = sm.generate_app_id(
        launcher_path, _AUTH_SHORTCUT_NAME,
    )
    unsigned_id = appid if appid >= 0 else appid + 2**32
    shortcuts_data = await sm.read_shortcuts()
    shortcuts = shortcuts_data.get("shortcuts", {})
    orphans_removed = _prune_orphan_shortcuts(shortcuts)
    legacy_removed = _prune_legacy_template_shortcuts(shortcuts)
    canonical_added = self._add_canonical_if_missing(
        shortcuts, launcher_path, appid, unsigned_id,
    )
    vdf_dirty = bool(
        orphans_removed or legacy_removed or canonical_added,
    )
    if vdf_dirty:
        await sm.write_shortcuts(shortcuts_data)
        logger.info(
            "[UbisoftAuth] VDF updated: orphans=%d "
            "legacy=%d added=%s",
            orphans_removed, legacy_removed, canonical_added,
        )
    await self._finalize_new_shortcut(sm, appid, unsigned_id)
    return cast("int | None", unsigned_id)
async def _finalize_new_shortcut(
    self, sm: ShortcutService, appid: int, unsigned_id: int,
) -> None:
    """Finalize new shortcut."""
    await self._parent._register_shortcut(
        sm, appid, _AUTH_SHORTCUT_NAME,
    )
    await self._parent._cleanup_legacy_registry(sm)
    await self._parent._clear_compat(sm, appid)
    await self._parent.fetch_auth_shortcut_artwork(unsigned_id)

```

```

def _add_canonical_if_missing(
    self,
    shortcuts: dict[str, Any],
    launcher_path: str,
    appid: int,
    unsigned_id: int,
) -> bool:

    """Add canonical if missing."""
    if self.shortcut_in_vdf(shortcuts):
        return False
    existing_indices = [
        int(k) for k in shortcuts if k.isdigit()
    ]
    next_idx = max(existing_indices, default=-1) + 1
    shortcuts[str(next_idx)] = {
        "appid": appid,
        "AppName": _AUTH_SHORTCUT_NAME,
        "exe": f'"{launcher_path}"',
        "StartDir": f'"{Path(launcher_path).parent}"',
        "LaunchOptions": self.build_auth_launch_options(),
        "IsHidden": 1,
        "AllowDesktopConfig": 1,
        "OpenVR": 0,
        "tags": {"0": "Ubisoft"},
    }
    logger.info(
        "[UbisoftAuth] created auth shortcut in VDF "
        "(appid=%d)", unsigned_id,
    )
    return True

async def validate_auth_shortcut(self, sm: ShortcutService) -> bool:
    """Validate auth shortcut."""
    try:
        launcher_path = self.get_launcher_path()
        expected_launch_options = (
            self.build_auth_launch_options()
        )
        expected_appid = sm.generate_app_id(
            launcher_path, _AUTH_SHORTCUT_NAME,
        )
        shortcuts_data = await sm.read_shortcuts()
        shortcuts = shortcuts_data.get("shortcuts", {})
        vdf_updated = False
        found = False
        for _idx, s in shortcuts.items():
            full_id = self.extract_store_id(
                s.get("LaunchOptions", ""),
            )
            if full_id != (
                self._parent._config.auth_shortcut_store_id
            ):
                continue
            found = True
            if self._fix_shortcut_fields(
                s,
                launcher_path,
                expected_launch_options,
                expected_appid,
            ):

```

```

        vdf_updated = True
        break
    if vdf_updated:
        await sm.write_shortcuts(shortcuts_data)
    if not found:
        logger.warning(
            "[UbisoftAuth] auth shortcut not found "
            "in VDF during validation",
        )
        return False
    await self._parent._register_shortcut(
        sm, expected_appid, _AUTH_SHORTCUT_NAME,
    )
    await self._parent._clear_compat(
        sm, expected_appid,
    )
    return True
except Exception as e:
    logger.warning(
        "[UbisoftAuth] auth shortcut validation "
        "failed: %s", e,
    )
    return True

def _fix_shortcut_fields(
    self,
    entry: dict[str, Any],
    launcher_path: str,
    expected_launch_options: str,
    expected_appid: int,
) -> bool:

    """Fix shortcut fields."""
    changed = False
    if entry.get("LaunchOptions", "") != expected_launch_options:
        logger.info(
            "[UbisoftAuth] auth shortcut launch "
            "options outdated, fixing",
        )
        entry["LaunchOptions"] = expected_launch_options
        changed = True
    current_exe = entry.get("exe", "").strip('')
    if current_exe != launcher_path:
        logger.info(
            "[UbisoftAuth] auth shortcut exe "
            "outdated, fixing",
        )
        entry["exe"] = f'"{launcher_path}"'
        entry["StartDir"] = f'"{Path(launcher_path).parent}"'
        changed = True
    if entry.get("appid") != expected_appid:
        logger.info(
            "[UbisoftAuth] auth shortcut appid "
            "changed, fixing",
        )
        entry["appid"] = expected_appid
        changed = True
    return changed

async def auth_shortcut_exists_in_vdf(self) -> bool:
    """Auth shortcut exists in VDF."""
    if self._parent._shortcut_service is None:

```

```

        return True
    try:
        sm = self._parent._shortcut_service
        shortcuts_data = await sm.read_shortcuts()
        shortcuts = shortcuts_data.get("shortcuts", {})
        target = (
            self._parent._config.auth_shortcut_store_id
        )
        return any(
            self.extract_store_id(
                s.get("LaunchOptions", ""),
            ) == target
            for s in shortcuts.values()
        )
    except Exception:
        return True
async def add_shortcut_to_vdf(
    self, sm: ShortcutService, appid: int,
) -> None:
    """Add shortcut to VDF."""
    launcher_path = self.get_launcher_path()
    launch_options = self.build_auth_launch_options()
    shortcuts_data = await sm.read_shortcuts()
    shortcuts = shortcuts_data.get("shortcuts", {})
    if self.shortcut_in_vdf(shortcuts):
        return
    existing_indices = [
        int(k) for k in shortcuts if k.isdigit()
    ]
    next_idx = max(existing_indices, default=-1) + 1
    shortcuts[str(next_idx)] = {
        "appid": appid,
        "AppName": _AUTH_SHORTCUT_NAME,
        "exe": f'"{launcher_path}"',
        "StartDir": f'"{Path(launcher_path).parent}"',
        "LaunchOptions": launch_options,
        "IsHidden": 1,
        "AllowDesktopConfig": 1,
        "OpenVR": 0,
        "tags": {"0": "Ubisoft"},
    }
    await sm.write_shortcuts(shortcuts_data)

def shortcut_in_vdf(
    self, shortcuts: dict[str, Any],
) -> bool:
    """Shortcut in VDF."""
    target = (
        self._parent._config.auth_shortcut_store_id
    )
    for s in shortcuts.values():
        full_id = self.extract_store_id(
            s.get("LaunchOptions", ""),
        )
        if full_id == target:
            return True
    return False
@staticmethod
def extract_store_id(launch_options: str) -> str:
    """Extract store ID."""

```

```
if not launch_options:  
    return ""  
return launch_options.split(maxsplit=1)[0]
```

OP-58d

session_monitor.py

Fichier : py_modules/unifideck/stores/ubisoft/auth/session_monitor.py | Lignes : 79 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from collections.abc import Callable
from typing import Any
from ...core.types import Result
_AUTH_MONITOR_TIMEOUT_S = 30 * 60
_AUTH_MONITOR_POLL_INTERVAL_S = 2.0
logger = logging.getLogger(__name__)
class _AuthSessionMonitor:
    """Auth session monitor."""
    def __init__(
        self,
        *,
        config: Any,
        session: Any,
        queue_auth_assets_ensure: Callable[[str], None],
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._session = session
        self._queue_auth_assets_ensure = queue_auth_assets_ensure
        self._monitor_task: asyncio.Task[None] | None = None
        self._session_captured = False
    async def start(self) -> Result:
        """Start."""
        if (
            self._monitor_task is not None
            and not self._monitor_task.done()
        ):
            self._monitor_task.cancel()
            try:
                await self._monitor_task
            except asyncio.CancelledError:
                pass
            except Exception as e:
                logger.debug(
                    "[UbisoftAuth] old monitor task error on "
                    "cancel: %s", e,
                )
        self._session_captured = False
        self._monitor_task = asyncio.create_task(self._loop())
        logger.info(
            "[UbisoftAuth] started auth session monitor",
        )
        return Result(success=True)
    async def _loop(self) -> None:
        """Loop."""
        auth_dir = self._config.auth_prefix_dir_expanded
        elapsed = 0.0
        while elapsed < _AUTH_MONITOR_TIMEOUT_S:
            await asyncio.sleep(_AUTH_MONITOR_POLL_INTERVAL_S)
            elapsed += _AUTH_MONITOR_POLL_INTERVAL_S

```

```
        captured = self._session.capture(auth_dir)
        if captured:
            logger.info(
                "[UbisoftAuth] auth session monitor: "
                "token captured",
            )
            self._session.propagate_all_to_all()
            self._queue_auth_assets_ensure(
                "post-auth-session-capture",
            )
            self._session_captured = True
            return
        logger.warning(
            "[UbisoftAuth] auth session monitor timed out "
            "after %ds", _AUTH_MONITOR_TIMEOUT_S,
        )
    def status(self) -> dict[str, Any]:
        """Status."""
        monitoring = (
            self._monitor_task is not None
            and not self._monitor_task.done()
        )
        return {
            "captured": self._session_captured,
            "monitoring": monitoring,
        }
```

OP-58e

direct_signin.py

Fichier : py_modules/unifideck/stores/ubisoft/auth/direct_signin.py | Lignes : 163 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import logging
from typing import Any
from ...security import emit_external_auth_check_failed
from ..binaries import UbisoftBinaryResolver
from ..paths import UbisoftPrefixPaths
from ..session import UbisoftSession
logger = logging.getLogger(__name__)
class _DirectSignIn:
    """Direct sign in."""
    def __init__(
        self,
        *,
        binaries: UbisoftBinaryResolver,
        bus: Any,
        config: Any,
        paths: UbisoftPrefixPaths,
        session: UbisoftSession,
        ensure_auth_prefix: Any,
        queue_auth_assets_ensure: Any,
    ) -> None:
        """Initialize the instance."""
        self._binaries = binaries
        self._bus = bus
        self._config = config
        self._paths = paths
        self._session = session
        self._ensure_auth_prefix = ensure_auth_prefix
        self._queue_auth_assets_ensure = queue_auth_assets_ensure

    async def connect(self) -> dict[str, Any]:
        """Connect."""
        self._queue_auth_assets_ensure("connect-account")
        resolved = await self._resolve_launch_targets()
        if isinstance(resolved, dict):
            return resolved
        umu_run, connect_path, prefix_path = resolved
        python_bin, env = self._build_launch_env(prefix_path)
        logger.info(
            "[UbisoftAuth] launching Ubisoft Connect in "
            "auth prefix for login",
        )
        try:
            proc = await asyncio.create_subprocess_exec(
                python_bin, umu_run, connect_path,
                env=env,
                stdout=asyncio.subprocess.DEVNULL,
                stderr=asyncio.subprocess.DEVNULL,
            )
        except OSError as e:
            return {
                "success": False,
                "error": f"upc_spawn_failed: {e}",
            }
```



```

    }
    captured_token = await self._wait_for_capture(
        proc, prefix_path,
    )
    if not captured_token:
        captured_token = self._session.capture(prefix_path)
    if captured_token:
        return self._finalize_success(prefix_path)
    return {
        "success": False,
        "error": (
            "Login not detected. Please log in and "
            "close Ubisoft Connect."
        ),
    }
}

async def _resolve_launch_targets(
    self,
) -> tuple[str, str, str] | dict[str, Any]:
    """Resolve launch targets."""
    upc_path = await self._ensure_auth_prefix()
    umu_run = self._binaries.find_umu_run()
    if not upc_path or not umu_run:
        emit_external_auth_check_failed(
            self._bus, "ubisoft", "upc_not_found",
            "Ubisoft Connect exe missing from auth prefix",
        )
    return {
        "success": False,
        "error": "upc_not_found_in_auth_prefix",
    }

    prefix_path = self._config.auth_prefix_dir_expanded
    connect_path = self._paths.find_connect_exe(prefix_path)
    if not connect_path:
        emit_external_auth_check_failed(
            self._bus, "ubisoft", "connect_exe_not_found",
            "find_connect_exe returned empty",
        )
    return {
        "success": False,
        "error": "ubisoft_connect_exe_not_found",
    }

    return umu_run, connect_path, prefix_path

def _build_launch_env(
    self, prefix_path: str,
) -> tuple[str, dict[str, str]]:
    """Build launch env."""
    python_bin = self._binaries.find_python()
    env = self._binaries.build_umu_env(
        wineprefix=prefix_path,
        gameid="umu-ubisoft-auth",
        store_game_id=self._config.auth_shortcut_store_id,
    )
    return python_bin, env

def _finalize_success(
    self, prefix_path: str,
) -> dict[str, Any]:
    """Finalize success."""
    self._session.propagate_all_to_all()
    self._queue_auth_assets_ensure("post-connect-account")

```

```

        return {
            "success": True,
            "message": "Ubisoft account connected successfully",
        }
    async def _wait_for_capture(
        self,
        proc: asyncio.subprocess.Process,
        prefix_path: str,
    ) -> str | None:
        """Wait for capture."""
        loop = asyncio.get_event_loop()
        start = loop.time()
        timeout_seconds = 600.0
        captured: str | None = None
        while loop.time() - start < timeout_seconds:
            if proc.returncode is not None:
                break
            captured = self._session.capture(prefix_path)
            if captured:
                logger.info(
                    "[UbisoftAuth] UPC session captured "
                    "during auth; closing launcher",
                )
                try:
                    proc.terminate()
                    await asyncio.wait_for(
                        proc.wait(), timeout=10,
                    )
                except (TimeoutError, ProcessLookupError):
                    try:
                        proc.kill()
                        await asyncio.wait_for(
                            proc.wait(), timeout=5,
                        )
                    except (TimeoutError, ProcessLookupError):
                        pass
                return captured
            await asyncio.sleep(2)
        logger.warning(
            "[UbisoftAuth] auth launcher timed out",
        )
        try:
            proc.terminate()
            await asyncio.wait_for(proc.wait(), timeout=10)
        except (TimeoutError, ProcessLookupError):
            try:
                proc.kill()
                await asyncio.wait_for(proc.wait(), timeout=5)
            except (TimeoutError, ProcessLookupError):
                pass
        return None

```

OP-58f

shortcut_ops.py

Fichier : py_modules/unifideck/stores/ubisoft/auth/shortcut_ops.py | Lignes : 86 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from ...services.shortcut import ShortcutService
    from ..config import UbisoftConfig
_LEGACY_AUTH_SHORTCUT_STORE_ID = "ubisoft:.template"
logger = logging.getLogger(__name__)
class _ShortcutRegistryOps:
    """Shortcut registry ops."""
    def __init__(self, *, config: UbisoftConfig) -> None:
        """Initialize the instance."""
        self._config = config
    async def load(self, sm: ShortcutService) -> dict[str, Any]:
        """Load."""
        try:
            if hasattr(sm, "load_shortcuts_registry"):
                result = sm.load_shortcuts_registry()
                if asyncio.iscoroutine(result):
                    result = await result
                if isinstance(result, dict):
                    return result
        except Exception as e:
            logger.debug(
                "[UbisoftAuth] registry load failed: %s", e,
            )
        return {}
    async def register(
        self, sm: ShortcutService, appid: int, name: str,
    ) -> None:
        """Register."""
        try:
            if hasattr(sm, "register_shortcut"):
                result = sm.register_shortcut(
                    self._config.auth_shortcut_store_id,
                    appid,
                    name,
                )
                if asyncio.iscoroutine(result):
                    await result
        except Exception as e:
            logger.debug(
                "[UbisoftAuth] shortcut register failed: %s",
                e,
            )
    async def clear_compat(
        self, sm: ShortcutService, appid: int,
    ) -> None:
        """Clear compat."""
        try:
            if hasattr(sm, "_clear_proton_compatibility"):
                result = sm._clear_proton_compatibility(appid)
                if asyncio.iscoroutine(result):
                    await result

```

```
except Exception as e:
    logger.debug(
        "[UbisoftAuth] clear compat failed: %s", e,
    )

async def cleanup_legacy(self, sm: ShortcutService) -> None:
    """Cleanup legacy."""
    try:
        if not hasattr(sm, "load_shortcuts_registry"):
            return
        registry = sm.load_shortcuts_registry()
        if asyncio.iscoroutine(registry):
            registry = await registry
        if (
            not isinstance(registry, dict)
            or _LEGACY_AUTH_SHORTCUT_STORE_ID not in registry
        ):
            return
        del registry[_LEGACY_AUTH_SHORTCUT_STORE_ID]
        if hasattr(sm, "save_shortcuts_registry"):
            result = sm.save_shortcuts_registry(registry)
            if asyncio.iscoroutine(result):
                await result
            logger.info(
                "[UbisoftAuth] removed legacy .template "
                "from shortcuts registry",
            )
    except Exception as e:
        logger.debug(
            "[UbisoftAuth] legacy registry cleanup "
            "failed: %s", e,
        )
```

OP-59

`__init__.py`

Fichier : `py_modules/unifideck/stores/ubisoft/prefix/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from .manager import UbisoftPrefixManager
__all__ = ["UbisoftPrefixManager"]
```

OP-59a

manager.py

Fichier : py_modules/unifideck/stores/ubisoft/prefix/manager.py | Lignes : 117 | Depend de : (rien)

```

from __future__ import annotations
import logging
import shutil
from collections.abc import Callable
from pathlib import Path
from ..binaries import UbisoftBinaryResolver
from ..config import UbisoftConfig
from ..installer.cache import UbisoftInstallerCache
from ..paths import UbisoftPrefixPaths
from .auth_builder import _AuthPrefixBuilder
from .helpers import _PrefixHelpers
from .template_builder import _TemplatePrefixBuilder
logger = logging.getLogger(__name__)
class UbisoftPrefixManager:
    """Ubisoft prefix manager."""
    def __init__(
        self,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
        binaries: UbisoftBinaryResolver,
        installer_cache: UbisoftInstallerCache,
        inject_auth_state: Callable[[list[str]], int],
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._paths = paths
        self._binaries = binaries
        self._installer_cache = installer_cache
        self._inject_auth_state = inject_auth_state
        self._helpers = _PrefixHelpers(self)
        self._template_builder = _TemplatePrefixBuilder(
            config=config,
            paths=paths,
            helpers=self._helpers,
            installer_cache=installer_cache,
        )
        self._auth_builder = _AuthPrefixBuilder(
            config=config,
            paths=paths,
            helpers=self._helpers,
            installer_cache=installer_cache,
            template_builder=self._template_builder,
        )
    def template_exists(self) -> bool:
        """Template exists."""
        return self._template_builder.template_exists()
    def is_prefix_version_stale(self, prefix_dir: str) -> bool:
        """Check whether prefix version stale."""
        return self._template_builder.is_prefix_version_stale(
            prefix_dir,
        )
    @staticmethod
    def read_machine_guid(prefix_path: str) -> str:
        """Read machine guid."""
        return _TemplatePrefixBuilder.read_machine_guid(prefix_path)

```

```

def queue_template_creation(self) -> None:
    """Queue template creation."""
    self._template_builder.queue_template_creation()
async def regenerate_template_if_stale(self) -> None:
    """Regenerate template if stale."""
    await self._template_builder.regenerate_template_if_stale()
async def ensure_template_prefix(self) -> None:
    """Ensure template prefix."""
    await self._template_builder.ensure_template_prefix()
async def ensure_auth_prefix(self) -> str | None:
    """Ensure auth prefix."""
    return await self._auth_builder.ensure_auth_prefix()

def queue_auth_assets_ensure(
    self, reason: str = "background",
) -> None:

    """Queue auth assets ensure."""
    self._auth_builder.queue_auth_assets_ensure(reason)
async def bootstrap_game_prefix(self, space_id: str) -> bool:
    """Bootstrap game prefix."""
    prefix_path = self._paths.get_prefix_path(space_id)
    marker_path = (
        Path(prefix_path) / self._config.bootstrap_marker
    )
    if (
        marker_path.is_file()
        and self._paths.find_upc_exe(prefix_path)
    ):
        self._helpers.try_inject_auth_state([prefix_path])
        return True
    if (
        self._template_builder.template_exists()
        and await self._helpers.clone_prefix_from_template(
            space_id, prefix_path,
        )
    ):
        return True
    return await self._helpers.create_prefix_from_fresh_install(
        space_id, prefix_path,
    )
async def repair_prefix(
    self, space_id: str,
) -> bool:
    """Repair prefix."""
    prefix_path = self._paths.get_prefix_path(space_id)
    logger.info(
        "[UbisoftPrefixManager] repairing prefix for %s",
        space_id,
    )
    try:
        if Path(prefix_path).is_dir():
            shutil.rmtree(prefix_path)
            logger.info(
                "[UbisoftPrefixManager] removed corrupted "
                "prefix for %s", space_id,
            )
    except OSError as e:
        logger.error(
            "[UbisoftPrefixManager] could not remove "
            "corrupted prefix: %s", e,

```

```
    )  
    return False  
return await self.bootstrap_game_prefix(space_id)
```


OP-59b

helpers.py

Fichier : py_modules/unifideck/stores/ubisoft/prefix/helpers.py | Lignes : 301 | Depend de : (rien)

```
from __future__ import annotations
import asyncio
import datetime
import logging
import os
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from .manager import UbisoftPrefixManager
logger = logging.getLogger(__name__)
_SILENT_INSTALL_FLAG = "/S"
class _PrefixHelpers:
    """Prefix helpers."""
    def __init__(self, parent: UbisoftPrefixManager) -> None:
        """Initialize the instance."""
        self._parent = parent
    async def clone_prefix_from_template(
        self, space_id: str, prefix_path: str,
    ) -> bool:
        """Clone prefix from template."""
        logger.info(
            "[UbisoftPrefixManager] cloning template for %s",
            space_id,
        )
        try:
            os.makedirs(prefix_path, exist_ok=True)
            ok = await self.rsync_clone(
                self._parent._config.template_dir_expanded,
                prefix_path,
                exclude_games=False,
            )
            if not ok:
                logger.error(
                    "[UbisoftPrefixManager] rsync clone "
                    "failed for %s", space_id,
                )
                return False
            self.write_bootstrap_marker(
                prefix_path,
                "cloned_from_template",
                space_id,
            )
            self.try_inject_auth_state([prefix_path])
            logger.info(
                "[UbisoftPrefixManager] prefix cloned for %s",
                space_id,
            )
            return True
        except Exception as e:
            logger.error(
                "[UbisoftPrefixManager] clone failed: %s", e,
            )
            return False

    async def create_prefix_from_fresh_install(
        self, space_id: str, prefix_path: str,
```

```

) -> bool:

    """Create prefix from fresh install."""
    logger.info(
        "[UbisoftPrefixManager] fresh install for %s",
        space_id,
    )
    installer_path = (
        await self._parent._installer_cache.ensure_cached()
    )
    if not installer_path:
        return False
    try:
        os.makedirs(prefix_path, exist_ok=True)
        success = await self.run_silent_installer(
            prefix_dir=prefix_path,
            installer_path=installer_path,
            gameid=f"umu-ubisoft-{space_id}",
            store_game_id=f"ubisoft:{space_id}",
        )
        if not success:
            return False
        if not self._parent._paths.find_upc_exe(prefix_path):
            logger.error(
                "[UbisoftPrefixManager] upc.exe not "
                "found after fresh install for %s",
                space_id,
            )
            return False
        self.write_bootstrap_marker(
            prefix_path, "fresh_install", space_id,
        )
        self.try_inject_auth_state([prefix_path])
        if not self._parent.template_exists():
            await self.create_template_from_game_prefix(
                prefix_path,
            )
        return True
    except Exception as e:
        logger.exception(
            "[UbisoftPrefixManager] fresh install failed "
            "for %s: %s", space_id, e,
        )
        return False

async def create_template_from_game_prefix(
    self, game_prefix: str,
) -> None:
    """Create template from game prefix."""
    template_dir = self._parent._config.template_dir_expanded
    logger.info(
        "[UbisoftPrefixManager] creating template from "
        "first game prefix",
    )
    try:
        os.makedirs(template_dir, exist_ok=True)
        ok = await self.rsync_clone(
            game_prefix, template_dir,
            exclude_games=False,
        )
        if not ok:
            return

```

```

        self.write_bootstrap_marker(
            template_dir, "template", None,
        )
        self.try_inject_auth_state([template_dir])
    except Exception as e:
        logger.warning(
            "[UbisoftPrefixManager] template creation "
            "from game prefix failed: %s", e,
        )

    async def run_silent_installer(
        self,
        *,
        prefix_dir: str,
        installer_path: str,
        gameid: str,
        store_game_id: str | None = None,
    ) -> bool:

        """Run silent installer."""
        umu_run = self._parent._binaries.find_umu_run()
        if not umu_run:
            logger.error(
                "[UbisoftPrefixManager] umu-run not found",
            )
            return False
        env = self._parent._binaries.build_umu_env(
            wineprefix=prefix_dir,
            gameid=gameid,
            store_game_id=store_game_id,
        )
        python_bin = self._parent._binaries.find_python()
        logger.info(
            "[UbisoftPrefixManager] installer run: "
            "PROTONPATH=%s GAMEID=%s",
            env.get("PROTONPATH"), env.get("GAMEID"),
        )
        try:
            proc = await asyncio.create_subprocess_exec(
                python_bin, umu_run,
                installer_path,
                _SILENT_INSTALL_FLAG,
                env=env,
                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE,
            )
        except OSError as e:
            logger.error(
                "[UbisoftPrefixManager] subprocess spawn "
                "failed: %s", e,
            )
            return False
        return await self._await_installer_completion(proc)
    @staticmethod
    async def _await_installer_completion(
        proc: asyncio.subprocess.Process,
    ) -> bool:
        """Await installer completion."""
        try:
            _stdout, stderr = await asyncio.wait_for(
                proc.communicate(),
            )

```

```

        timeout=15 * 60,
    )
except TimeoutError:
    logger.error(
        "[UbisoftPrefixManager] installer timed out "
        "after 15 min – killing",
    )
    try:
        proc.kill()
    except ProcessLookupError:
        pass
    await proc.wait()
    return False
if proc.returncode != 0:
    stderr_text = stderr.decode(
        errors="replace",
    )[:500] if stderr else ""
    logger.error(
        "[UbisoftPrefixManager] installer exited %d: %s",
        proc.returncode, stderr_text,
    )
    return False
return True

async def rsync_clone(
    self,
    src: str,
    dst: str,
    *,
    exclude_games: bool,
) -> bool:
    """Rsync clone."""
    args: list[str] = ["rsync", "-a"]
    if exclude_games:
        args.append("--exclude=games")
    args.extend([f"{src}/", f"{dst}/"])
    try:
        proc = await asyncio.create_subprocess_exec(
            *args,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
    except OSError as e:
        logger.error(
            "[UbisoftPrefixManager] rsync spawn failed: %s",
            e,
        )
        return False
    try:
        _stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=30 * 60,
        )
    except TimeoutError:
        logger.error(
            "[UbisoftPrefixManager] rsync timed out – killing",
        )
        try:
            proc.kill()
        except ProcessLookupError:

```

```

        pass
        await proc.wait()
        return False
    if proc.returncode != 0:
        logger.error(
            "[UbisoftPrefixManager] rsync failed (%d): %s",
            proc.returncode,
            stderr.decode(errors="replace")[:300],
        )
        return False
    return True
@staticmethod
def fix_pfx_symlink(prefix_dir: str) -> None:
    """Fix pfx symlink."""
    pfx_link = os.path.join(prefix_dir, "pfx")
    if not os.path.islink(pfx_link):
        return
    try:
        current_target = os.readlink(pfx_link)
        if current_target in (prefix_dir, "."):
            return
        os.remove(pfx_link)
        os.symlink(prefix_dir, pfx_link)
        logger.info(
            "[UbisoftPrefixManager] fixed pfx symlink: "
            "%s → %s", current_target, prefix_dir,
        )
    except OSError as e:
        logger.warning(
            "[UbisoftPrefixManager] could not fix pfx "
            "symlink: %s", e,
        )

def write_bootstrap_marker(
    self,
    prefix_dir: str,
    source: str,
    space_id: str | None,
) -> None:
    """Write bootstrap marker."""
    marker_path = os.path.join(
        prefix_dir, self._parent._config.bootstrap_marker,
    )
    created_at = datetime.datetime.now().isoformat()
    lines = [source, f"created={created_at}"]
    if space_id:
        lines.insert(1, f"game={space_id}")
    try:
        with open(
            marker_path, "w", encoding="utf-8",
        ) as f:
            f.write("\n".join(lines) + "\n")
    except OSError as e:
        logger.warning(
            "[UbisoftPrefixManager] could not write "
            "bootstrap marker: %s", e,
        )

def try_inject_auth_state(
    self, prefix_paths: list[str],
) -> None:

```

```
"""Try inject auth state."""
if not prefix_paths:
    return
try:
    self._parent._inject_auth_state(prefix_paths)
except Exception as e:
    logger.warning(
        "[UbisoftPrefixManager] auth state injection "
        "failed: %s", e,
    )
```

OP-59c

template_builder.py

Fichier : py_modules/unifideck/stores/ubisoft/prefix/template_builder.py | Lignes : 162 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import re
import shutil
from pathlib import Path
from typing import TYPE_CHECKING
from ..binaries import UbisoftBinaryResolver
if TYPE_CHECKING:
    from ..config import UbisoftConfig
    from ..installer.cache import UbisoftInstallerCache
    from ..paths import UbisoftPrefixPaths
    from .helpers import _PrefixHelpers
logger = logging.getLogger(__name__)
class _TemplatePrefixBuilder:
    """Template prefix builder."""
    def __init__(
        self,
        *,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
        helpers: _PrefixHelpers,
        installer_cache: UbisoftInstallerCache,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._paths = paths
        self._helpers = helpers
        self._installer_cache = installer_cache
        self._template_task: asyncio.Task[None] | None = None
    def template_exists(self) -> bool:
        """Template exists."""
        marker = (
            Path(self._config.template_dir_expanded)
            / self._config.bootstrap_marker
        )
        return marker.is_file()
    def is_prefix_version_stale(self, prefix_dir: str) -> bool:
        """Check whether prefix version stale."""
        version_file = Path(prefix_dir) / "version"
        if not version_file.is_file():
            return False
        try:
            prefix_version = version_file.read_text(
                encoding="utf-8",
            ).strip()
        except OSError:
            return False
        if not prefix_version:
            return False
        family = UbisoftBinaryResolver.proton_family(
            prefix_version,
        )
        if family != "experimental":
            logger.info(

```

```

        "[UbisoftPrefixManager] prefix stale: '%s' "
        "(family=%s, expected=experimental) prefix=%s",
        prefix_version, family, prefix_dir,
    )
    return True
return False

@staticmethod
def read_machine_guid(prefix_path: str) -> str:
    """Read machine guid."""
    prefix_p = Path(prefix_path)
    for reg_path in (
        prefix_p / "pfx" / "system.reg",
        prefix_p / "system.reg",
    ):
        if not reg_path.is_file():
            continue
        try:
            content = reg_path.read_text(
                encoding="utf-8", errors="ignore",
            )
        except OSError:
            continue
        match = re.search(
            r'"MachineGuid"="([^\"]+)"', content,
        )
        if match:
            return match.group(1)
    return ""

def queue_template_creation(self) -> None:
    """Queue template creation."""
    if (
        self._template_task is not None
        and not self._template_task.done()
    ):
        logger.info(
            "[UbisoftPrefixManager] template creation "
            "already in progress",
        )
        return
    logger.info(
        "[UbisoftPrefixManager] queuing background "
        "template creation",
    )
    self._template_task = asyncio.create_task(
        self.ensure_template_prefix(),
    )

async def regenerate_template_if_stale(self) -> None:
    """Regenerate template if stale."""
    if not self.template_exists():
        return
    template_dir = self._config.template_dir_expanded
    if not self.is_prefix_version_stale(template_dir):
        return
    logger.warning(
        "[UbisoftPrefixManager] template prefix stale, "
        "removing for recreation",
    )
    shutil.rmtree(template_dir, ignore_errors=True)

async def ensure_template_prefix(self) -> None:

```



```
"""Ensure template prefix."""
if self.template_exists():
    logger.info(
        "[UbisoftPrefixManager] template already exists",
    )
    return
template_dir = self._config.template_dir_expanded
logger.info(
    "[UbisoftPrefixManager] creating template prefix",
)
try:
    installer_path = (
        await self._installer_cache.ensure_cached()
    )
    if not installer_path:
        logger.error(
            "[UbisoftPrefixManager] installer cache "
            "failed, aborting template creation",
        )
        return
    Path(template_dir).mkdir(parents=True, exist_ok=True)
    success = await self._helpers.run_silent_installer(
        prefix_dir=template_dir,
        installer_path=installer_path,
        gameid="umu-ubisoft-template",
    )
    if not success:
        return
    if not self._paths.find_upc_exe(template_dir):
        logger.error(
            "[UbisoftPrefixManager] upc.exe not found "
            "after template install",
        )
        return
    self._helpers.write_bootstrap_marker(
        template_dir, "template", None,
    )
    self._helpers.try_inject_auth_state([template_dir])
    logger.info(
        "[UbisoftPrefixManager] template created "
        "successfully",
    )
except Exception as e:
    logger.exception(
        "[UbisoftPrefixManager] template creation "
        "failed: %s",
        e,
    )
```

OP-59d

auth_builder.py

Fichier : py_modules/unifideck/stores/ubisoft/prefix/auth_builder.py | Lignes : 219 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import os
import shutil
from pathlib import Path
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ..config import UbisoftConfig
    from ..installer.cache import UbisoftInstallerCache
    from ..paths import UbisoftPrefixPaths
    from .helpers import _PrefixHelpers
    from .template_builder import _TemplatePrefixBuilder
logger = logging.getLogger(__name__)
class _AuthPrefixBuilder:
    """Auth prefix builder."""
    def __init__(
        self,
        *,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
        helpers: _PrefixHelpers,
        installer_cache: UbisoftInstallerCache,
        template_builder: _TemplatePrefixBuilder,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._paths = paths
        self._helpers = helpers
        self._installer_cache = installer_cache
        self._template_builder = template_builder
        self._auth_assets_task: asyncio.Task[None] | None = None
        self._auth_assets_lock = asyncio.Lock()
    async def ensure_auth_prefix(self) -> str | None:
        """Ensure auth prefix."""
        auth_dir = self._config.auth_prefix_dir_expanded
        upc_path = self._paths.find_upc_exe(auth_dir)
        rebuild = self._auth_prefix_needs_rebuild(
            auth_dir, upc_path,
        )
        if (
            not rebuild
            and Path(auth_dir).is_dir()
            and not upc_path
        ):
            logger.warning(
                "[UbisoftPrefixManager] auth prefix exists "
                "but upc.exe missing, re-cloning",
            )
            shutil.rmtree(auth_dir, ignore_errors=True)
            upc_path = None
            rebuild = True
        if upc_path and not rebuild:
            return upc_path
        return await self._rebuild_and_finalise_auth_prefix(

```

```

        auth_dir,
    )

def _auth_prefix_needs_rebuild(
    self, auth_dir: str, upc_path: str | None,
) -> bool:

    """Auth prefix needs rebuild."""
    if not upc_path:
        return False
    if self._template_builder.is_prefix_version_stale(auth_dir):
        logger.warning(
            "[UbisoftPrefixManager] auth prefix Proton "
            "version stale, rebuilding",
        )
        return True
    if self._template_builder.template_exists():
        auth_guid = self._template_builder.read_machine_guid(
            auth_dir,
        )
        tpl_guid = self._template_builder.read_machine_guid(
            self._config.template_dir_expanded,
        )
        if (
            tpl_guid
            and auth_guid
            and tpl_guid != auth_guid
        ):
            logger.warning(
                "[UbisoftPrefixManager] auth prefix "
                "MachineGuid desynced from template - "
                "rebuilding for DPAPI compatibility",
            )
            return True
        return False
    async def _rebuild_and_finalise_auth_prefix(
        self, auth_dir: str,
    ) -> str | None:
        """Rebuild and finalise auth prefix."""
        await self._template_builder.regenerate_template_if_stale()
        cloned = await self._build_auth_prefix_from_source()
        if not cloned:
            return None
        self._helpers.fix_pfx_symlink(auth_dir)
        upc_path = self._paths.find_upc_exe(auth_dir)
        if upc_path:
            self._helpers.try_inject_auth_state([auth_dir])
            logger.info(
                "[UbisoftPrefixManager] auth prefix ready",
            )
            return upc_path
        return None

    async def _build_auth_prefix_from_source(self) -> bool:

        """Build auth prefix from source."""
        auth_dir = self._config.auth_prefix_dir_expanded
        src, label = self._pick_clone_source()
        if src:
            logger.info(
                "[UbisoftPrefixManager] cloning %s → auth "

```

```

        "prefix",
        label,
    )
    Path(auth_dir).mkdir(parents=True, exist_ok=True)
    ok = await self._helpers.rsync_clone(
        src, auth_dir, exclude_games=True,
    )
    if not ok:
        logger.error(
            "[UbisoftPrefixManager] rsync clone failed "
            "for auth prefix",
        )
        return False
    return True
logger.info(
    "[UbisoftPrefixManager] no template/game prefix - "
    "bootstrapping auth prefix via fresh install",
)
installer_path = await self._installer_cache.ensure_cached()
if not installer_path:
    return False
Path(auth_dir).mkdir(parents=True, exist_ok=True)
success = await self._helpers.run_silent_installer(
    prefix_dir=auth_dir,
    installer_path=installer_path,
    gameid="umu-ubisoft-auth",
    store_game_id=self._config.auth_shortcut_store_id,
)
if not success and not self._paths.find_upc_exe(auth_dir):
    return False
self._helpers.write_bootstrap_marker(
    auth_dir, "auth_prefix", None,
)
return True
def _pick_clone_source(self) -> tuple[str | None, str]:
    """Pick clone source."""
    if self._template_builder.template_exists():
        return (
            self._config.template_dir_expanded,
            "template",
        )
    prefixes_dir = self._config.prefixes_dir_expanded
    if not Path(prefixes_dir).is_dir():
        return (None, "")
    try:
        entries = sorted(os.listdir(prefixes_dir))
    except OSError:
        return (None, "")
    for entry in entries:
        if entry.startswith("."):
            continue
        candidate = str(Path(prefixes_dir) / entry)
        if self._paths.find_upc_exe(candidate):
            return (candidate, f"game prefix {entry[:8]}")
    return (None, "")
def queue_auth_assets_ensure(
    self, reason: str = "background",
) -> None:
    """Queue auth assets ensure."""
    if (
        self._auth_assets_task is not None

```

```

        and not self._auth_assets_task.done()
    ):
        logger.info(
            "[UbisoftPrefixManager] auth asset ensure "
            "already in progress (reason=%s)", reason,
        )
        return
    logger.info(
        "[UbisoftPrefixManager] queueing auth asset "
        "ensure (reason=%s)", reason,
    )
    self._auth_assets_task = asyncio.create_task(
        self._ensure_auth_assets(reason),
    )

async def _ensure_auth_assets(self, reason: str) -> None:
    """Ensure auth assets."""
    async with self._auth_assets_lock:
        logger.info(
            "[UbisoftPrefixManager] ensuring auth assets "
            "(reason=%s)", reason,
        )
        await self._template_builder.regenerate_template_if_stale()
        if not self._template_builder.template_exists():
            await self._template_builder.ensure_template_prefix()
            return
        template_dir = self._config.template_dir_expanded
        self._helpers.try_inject_auth_state([template_dir])
        await self._repair_auth_prefix_if_needed()
    async def _repair_auth_prefix_if_needed(self) -> None:
        """Repair auth prefix if needed."""
        auth_dir = self._config.auth_prefix_dir_expanded
        session_file = self._config.upc_session_file_expanded
        if os.path.isdir(auth_dir):
            self._helpers.try_inject_auth_state([auth_dir])
            return
        if not os.path.isfile(session_file):
            return
        logger.info(
            "[UbisoftPrefixManager] auth prefix "
            "missing but user is authenticated; "
            "recreating",
        )
        await self.ensure_auth_prefix()
        game_prefixes = list(self._config.iter_game_prefix_paths())
        if game_prefixes:
            self._helpers.try_inject_auth_state(game_prefixes)

```

OP-60

`__init__.py`

Fichier : `py_modules/unifideck/stores/ubisoft/session/__init__.py` | Lignes : 2 | Depend de : (rien)

```
from .facade import UbisoftSession
__all__ = ["UbisoftSession"]
```

OP-60a

facade.py

Fichier : py_modules/unifideck/stores/ubisoft/session/facade.py | Lignes : 161 | Depend de : (rien)

```
from __future__ import annotations
import logging
from collections.abc import Callable
from pathlib import Path
from typing import Any
from ..config import UbisoftConfig
from ..paths import UbisoftPrefixPaths
from .payload import _PayloadSync
from .propagator import _CredentialPropagator
from .reader import _CredentialReader
logger = logging.getLogger(__name__)
_CAPTURE_SENTINEL = "credentials_captured"
class UbisoftSession:
    """Ubisoft session."""
    def __init__(
        self,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
        read_machine_guid: Callable[[str], str],
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._paths = paths
        self._read_machine_guid = read_machine_guid
        self._reader = _CredentialReader(
            config=config, paths=paths,
        )
        self._payload = _PayloadSync(self)
        self._propagator = _CredentialPropagator(
            config=config,
            payload=self._payload,
            reader=self._reader,
        )
    def has_valid_credentials(self, prefix_path: str) -> bool:
        """Check whether valid credentials."""
        return self._reader.has_valid_credentials(prefix_path)
    def get_credential_mtime(self, prefix_path: str) -> float:
        """Get credential mtime."""
        return self._reader.get_credential_mtime(prefix_path)
    def find_best_credential_source(self) -> str | None:
        """Find best credential source."""
        return self._reader.find_best_credential_source()
    def _is_valid_css(self, css_path: str, min_size: int) -> bool:
        """Is valid CSS."""
        return self._reader._is_valid_css(css_path, min_size)
    def propagate_credentials_to_all(self) -> int:
        """Propagate credentials to all."""
        return self._propagator.propagate_credentials_to_all()
    def propagate_auth_artifacts_to_all(self) -> int:
        """Propagate auth artifacts to all."""
        return self._propagator.propagate_auth_artifacts_to_all()
    def propagate_all_to_all(self) -> None:
        """Propagate all to all."""
        self._propagator.propagate_all_to_all()
    def inject_into_prefix(self, prefix_path: str) -> bool:
```

```

        """Inject into prefix."""
        return self._propagator.inject_into_prefix(prefix_path)
def ensure_auth_state_in_prefixes(
    self, prefix_paths: list[str],
) -> int:
    """Ensure auth state in prefixes."""
    return self._propagator.ensure_auth_state_in_prefixes(
        prefix_paths,
    )
def retroactive_sync(self) -> dict[str, Any]:
    """Retroactive sync."""
    return self._propagator.retroactive_sync()

def capture(self, prefix_path: str) -> str | None:

    """Capture."""
    if not self._reader.has_valid_credentials(prefix_path):
        return None
    new_mtime = self._reader.get_credential_mtime(prefix_path)
    if not new_mtime:
        return None
    stored_mtime = self._read_stored_mtime()
    credentials_changed = new_mtime > stored_mtime
    if credentials_changed:
        self._write_stored_mtime(new_mtime)
        logger.info(
            "[UbisoftSession] detected new UPC "
            "credentials (ConnectSecureStorage.dat)",
        )
    for target in (
        self._config.template_dir_expanded,
        self._config.auth_prefix_dir_expanded,
    ):
        if not Path(target).is_dir():
            continue
        if (
            Path(target).resolve()
            == Path(prefix_path).resolve()
        ):
            continue
        try:
            self._payload.sync_credentials_to_prefix(
                prefix_path, target,
            )
            self._payload.sync_auth_artifacts_to_prefix(
                prefix_path, target,
            )
        except Exception as e:
            logger.warning(
                "[UbisoftSession] capture sync to %s "
                "failed: %s",
                Path(target).name, e,
            )
    if credentials_changed:
        logger.info(
            "[UbisoftSession] captured credentials → "
            "template + auth prefix updated",
        )
    return _CAPTURE_SENTINEL
    return None
def _read_stored_mtime(self) -> float:

```



```
    """Read stored mtime."""
    session_file = self._config.upc_session_file_expanded
    if not Path(session_file).is_file():
        return 0.0
    try:
        content = Path(session_file).read_text(
            encoding="utf-8",
        ).strip()
    except OSError:
        return 0.0
    if not content.startswith("credential_mtime:"):
        return 0.0
    try:
        return float(content.split(":", 1)[1])
    except (ValueError, IndexError):
        return 0.0
def _write_stored_mtime(self, mtime: float) -> None:
    """Write stored mtime."""
    session_file = self._config.upc_session_file_expanded
    try:
        Path(self._config.data_dir_expanded).mkdir(
            parents=True, exist_ok=True,
        )
        Path(session_file).write_text(
            f"credential_mtime:{mtime}\n",
            encoding="utf-8",
        )
    except OSError as e:
        logger.warning(
            "[UbisoftSession] could not write mtime "
            "marker: %s", e,
        )

def clear_session_file(self) -> None:

    """Clear session file."""
    session_file = self._config.upc_session_file_expanded
    if not Path(session_file).is_file():
        return
    try:
        Path(session_file).unlink()
        logger.info(
            "[UbisoftSession] removed UPC session marker",
        )
    except OSError as e:
        logger.warning(
            "[UbisoftSession] could not remove session "
            "marker: %s", e,
        )
```

OP-60b

payload.py

Fichier : py_modules/unifideck/stores/ubisoft/session/payload.py | Lignes : 242 | Depend de : (rien)

```
from __future__ import annotations
import hashlib
import logging
import os
import shutil
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from .facade import UbisoftSession
logger = logging.getLogger(__name__)
_CSS_MIN_SOURCE_SIZE = 10
_HASH_CHUNK_SIZE = 1024 * 1024
class _PayloadSync:
    """Payload sync."""
    def __init__(self, parent: UbisoftSession) -> None:
        """Initialize the instance."""
        self._parent = parent
    def sync_payload_to_prefix(
        self,
        source_prefix: str,
        target_prefix: str,
        *,
        payload_sources: dict[str, str],
        apply_dpapi_guard: bool,
        handle_directories: bool,
        log_label: str,
    ) -> int:
        """Sync payload to prefix."""
        if self.should_skip_payload_sync(
            source_prefix, target_prefix,
            payload_sources, apply_dpapi_guard,
        ):
            return 0
        synced = 0
        for _root, user_home in (
            self._parent._paths.iter_user_homes(target_prefix)
        ):
            target_root = os.path.join(
                user_home,
                self._parent._config.upc_local_subdir,
            )
            for rel_path, src_path in payload_sources.items():
                dst_path = os.path.join(target_root, rel_path)
                if self.copy_payload_entry(
                    src_path, dst_path,
                    handle_directories=handle_directories,
                    log_label=log_label,
                    rel_path=rel_path,
                ):
                    synced += 1
        return synced

    def should_skip_payload_sync(
        self,
        source_prefix: str,
        target_prefix: str,
```

```

payload_sources: dict[str, str],
apply_dpapi_guard: bool,
) -> bool:

    """Check whether skip payload sync."""
    if os.path.realpath(source_prefix) == \
        os.path.realpath(target_prefix):
        return True
    if not payload_sources:
        return True
    if apply_dpapi_guard:
        source_guid = self._parent._read_machine_guid(
            source_prefix,
        )
        target_guid = self._parent._read_machine_guid(
            target_prefix,
        )
        if (
            source_guid
            and target_guid
            and source_guid != target_guid
        ):
            logger.warning(
                "[UbisoftSession] MachineGuid mismatch:"
                "source=%s... target=%s... - skipping "
                "DPAPI sync",
                source_guid[:8], target_guid[:8],
            )
        return True
    return False
def copy_payload_entry(
self,
src_path: str,
dst_path: str,
*,
handle_directories: bool,
log_label: str,
rel_path: str,
) -> bool:
    """Copy payload entry."""
    if os.path.exists(dst_path):
        try:
            same = (
                self.hash_artifact(src_path)
                == self.hash_artifact(dst_path)
            )
        except OSError:
            same = False
        if same:
            return False
    try:
        parent = os.path.dirname(dst_path)
        if parent:
            os.makedirs(parent, exist_ok=True)
        if handle_directories:
            if os.path.isdir(dst_path):
                shutil.rmtree(
                    dst_path, ignore_errors=True,
                )
            elif os.path.exists(dst_path):
                os.remove(dst_path)

```

```

        if os.path.isdir(src_path):
            shutil.copytree(src_path, dst_path)
        else:
            shutil.copy2(src_path, dst_path)
    else:
        shutil.copy2(src_path, dst_path)
    return True
except OSError as e:
    logger.warning(
        "[UbisoftSession] %s copy failed for "
        "%s: %s", log_label, rel_path, e,
    )
    return False

def sync_credentials_to_prefix(
    self,
    source_prefix: str,
    target_prefix: str,
) -> int:

    """Sync credentials to prefix."""
    return self.sync_payload_to_prefix(
        source_prefix=source_prefix,
        target_prefix=target_prefix,
        payload_sources=self.collect_credential_sources(
            source_prefix,
        ),
        apply_dpapi_guard=True,
        handle_directories=False,
        log_label="credential",
    )

def collect_credential_sources(
    self, source_prefix: str,
) -> dict[str, str]:
    """Collect credential sources."""
    source_files: dict[str, str] = {}
    for _root, user_home in (
        self._parent._paths.iter_user_homes(
            source_prefix, pfx_first=True,
        )
    ):
        for fname in (
            self._parent._config.upc_credential_files
        ):
            if fname in source_files:
                continue
            src = os.path.join(
                user_home,
                self._parent._config.upc_local_subdir,
                fname,
            )
            if self._parent._is_valid_css(
                src, _CSS_MIN_SOURCE_SIZE,
            ):
                source_files[fname] = src
    return source_files

def sync_auth_artifacts_to_prefix(
    self,
    source_prefix: str,
    target_prefix: str,
) -> int:

```

```

        """Sync auth artifacts to prefix."""
        return self.sync_payload_to_prefix(
            source_prefix=source_prefix,
            target_prefix=target_prefix,
            payload_sources=self.collect_artifact_sources(
                source_prefix,
            ),
            apply_dpapi_guard=False,
            handle_directories=True,
            log_label="auth cache artifact",
        )
    def collect_artifact_sources(
        self, source_prefix: str,
    ) -> dict[str, str]:
        """Collect artifact sources."""
        artifacts: dict[str, str] = {}
        for _root, user_home in (
            self._parent._paths.iter_user_homes(
                source_prefix, pfx_first=True,
            )
        ):
            local_root = os.path.join(
                user_home,
                self._parent._config.upc_local_subdir,
            )
            for rel_path in (
                self._parent._config.upc_auth_cache_artifacts
            ):
                if rel_path in artifacts:
                    continue
                candidate = os.path.join(
                    local_root, rel_path,
                )
                if (
                    os.path.isfile(candidate)
                    or os.path.isdir(candidate)
                ):
                    artifacts[rel_path] = candidate
        return artifacts

    @staticmethod
    def hash_artifact(path: str) -> str:
        """Check whether artifact."""
        digest = hashlib.sha256()
        if os.path.isdir(path):
            _PayloadSync._hash_directory_into(digest, path)
        elif os.path.isfile(path):
            _PayloadSync._hash_file_into(digest, path)
        return digest.hexdigest()

    @staticmethod
    def _hash_directory_into(digest: hashlib._Hash, path: str) -> None:
        """Hash directory into."""
        for root, _dirs, files in os.walk(path):
            files.sort()
            for name in files:
                file_path = os.path.join(root, name)
                rel_path = os.path.relpath(file_path, path)
                digest.update(rel_path.encode("utf-8"))
                _PayloadSync._hash_file_into(digest, file_path)

    @staticmethod
    def _hash_file_into(digest: hashlib._Hash, path: str) -> None:

```

```
"""Hash file into."""
try:
    with open(path, "rb") as f:
        for chunk in iter(
            lambda: f.read(_HASH_CHUNK_SIZE), b""
        ):
            digest.update(chunk)
except OSError:
    pass
```

OP-60c

reader.py

Fichier : py_modules/unifideck/stores/ubisoft/session/reader.py | Lignes : 119 | Depend de : (rien)

```

from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING
from .payload import _CSS_MIN_SOURCE_SIZE
if TYPE_CHECKING:
    from ..config import UbisoftConfig
    from ..paths import UbisoftPrefixPaths
_CSS_MIN_VALID_SIZE = 100
logger = logging.getLogger(__name__)
class _CredentialReader:
    """Credential reader."""
    def __init__(
        self,
        *,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._paths = paths
    def has_valid_credentials(self, prefix_path: str) -> bool:
        """Check whether valid credentials."""
        for _root, user_home in self._paths.iter_user_homes(
            prefix_path, pfx_first=True,
        ):
            css = self._css_path(user_home)
            if self._is_valid_css(css, _CSS_MIN_VALID_SIZE):
                return True
        return False
    def get_credential_mtime(self, prefix_path: str) -> float:
        """Get credential mtime."""
        best: float = 0.0
        for _root, user_home in self._paths.iter_user_homes(
            prefix_path, pfx_first=True,
        ):
            css = self._css_path(user_home)
            if not self._is_valid_css(css, _CSS_MIN_VALID_SIZE):
                continue
            try:
                mtime = Path(css).stat().st_mtime
            except OSError:
                continue
            if mtime > best:
                best = mtime
        return best
    def find_best_credential_source(self) -> str | None:
        """Find best credential source."""
        auth_source = self._check_auth_prefix_for_credentials()
        if auth_source:
            return auth_source
        return self._find_freshest_game_prefix_credentials()
    def _check_auth_prefix_for_credentials(self) -> str | None:
        """Check auth prefix for credentials."""
        auth_dir = self._config.auth_prefix_dir_expanded

```

```

    if not Path(auth_dir).is_dir():
        return None
    for _root, user_home in self._paths.iter_user_homes(
        auth_dir, pfx_first=True,
    ):
        css = self._css_path(user_home)
        if self._is_valid_css(css, _CSS_MIN_SOURCE_SIZE):
            return auth_dir
    return None

def _find_freshest_game_prefix_credentials(
    self,
) -> str | None:
    """Find freshest game prefix credentials."""
    prefixes_dir = self._config.prefixes_dir_expanded
    prefixes_p = Path(prefixes_dir)
    if not prefixes_p.is_dir():
        return None
    try:
        entries = list(prefixes_p.iterdir())
    except OSError:
        return None
    best_mtime: float = 0.0
    best_prefix: str | None = None
    for entry in entries:
        if not entry.is_dir():
            continue
        prefix = str(entry)
        mtime = self._best_css_mtime_for_prefix(prefix)
        if mtime is not None and mtime > best_mtime:
            best_mtime = mtime
            best_prefix = prefix
    return best_prefix

def _best_css_mtime_for_prefix(
    self, prefix: str,
) -> float | None:
    """Best CSS mtime for prefix."""
    for _root, user_home in self._paths.iter_user_homes(
        prefix, pfx_first=True,
    ):
        css = self._css_path(user_home)
        if not self._is_valid_css(css, _CSS_MIN_SOURCE_SIZE):
            continue
        try:
            return Path(css).stat().st_mtime
        except OSError:
            continue
    return None

def _css_path(self, user_home: str) -> str:
    """Css path."""
    return str(
        Path(user_home)
        / self._config.upc_local_subdir
        / "ConnectSecureStorage.dat"
    )

@staticmethod
def _is_valid_css(css_path: str, min_size: int) -> bool:
    """Is valid CSS."""
    css_p = Path(css_path)
    if not css_p.is_file():

```



```
        return False
    try:
        return css_p.stat().st_size > min_size
    except OSError:
        return False
```

OP-60d

propagator.py

Fichier : py_modules/unifideck/stores/ubisoft/session/propagator.py | Lignes : 169 | Depend de : (rien)

```
from __future__ import annotations
import logging
from pathlib import Path
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from ..config import UbisoftConfig
    from ..payload import _PayloadSync
    from ..reader import _CredentialReader
logger = logging.getLogger(__name__)
class _CredentialPropagator:
    """Credential propagator."""
    def __init__(
        self,
        *,
        config: UbisoftConfig,
        payload: _PayloadSync,
        reader: _CredentialReader,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._payload = payload
        self._reader = reader
    def propagate_credentials_to_all(self) -> int:
        """Propagate credentials to all."""
        source = self._reader.find_best_credential_source()
        if not source:
            logger.info(
                "[UbisoftSession] no credential source for "
                "propagation",
            )
            return 0
        total = 0
        for prefix_path in self._config.iter_game_prefix_paths():
            try:
                total += self._payload.sync_credentials_to_prefix(
                    source, prefix_path,
                )
            except Exception as e:
                logger.warning(
                    "[UbisoftSession] credential propagation "
                    "failed for %s: %s",
                    Path(prefix_path).name, e,
                )
        if total:
            logger.info(
                "[UbisoftSession] propagated %d credential "
                "file(s) across prefixes", total,
            )
        return total
    def propagate_auth_artifacts_to_all(self) -> int:
        """Propagate auth artifacts to all."""
        source = self._reader.find_best_credential_source()
        if not source:
```

```

        return 0
    total = 0
    for prefix_path in self._config.iter_game_prefix_paths():
        try:
            total += self._payload.sync_auth_artifacts_to_prefix(
                source, prefix_path,
            )
        except Exception as e:
            logger.warning(
                "[UbisoftSession] artifact propagation "
                "failed for %s: %s",
                Path(prefix_path).name, e,
            )
    if total:
        logger.info(
            "[UbisoftSession] propagated %d auth cache "
            "artifact(s) across prefixes", total,
        )
    return total
def propagate_all_to_all(self) -> None:
    """Propagate all to all."""
    self.propagate_credentials_to_all()
    self.propagate_auth_artifacts_to_all()
def inject_into_prefix(self, prefix_path: str) -> bool:
    """Inject into prefix."""
    source = self._reader.find_best_credential_source()
    if not source:
        logger.warning(
            "[UbisoftSession] inject_into_prefix: no "
            "credential source found",
        )
        return False
    credentials_synced = False
    try:
        synced = self._payload.sync_credentials_to_prefix(
            source, prefix_path,
        )
        artifact_synced = (
            self._payload.sync_auth_artifacts_to_prefix(
                source, prefix_path,
            )
        )
        if synced:
            logger.info(
                "[UbisoftSession] inject: synced %d "
                "credential file(s)", synced,
            )
            credentials_synced = True
        if artifact_synced:
            logger.info(
                "[UbisoftSession] inject: synced %d "
                "artifact(s)", artifact_synced,
            )
            credentials_synced = True
    except Exception as e:
        logger.warning(
            "[UbisoftSession] inject auth sync failed: %s",
            e,
        )
    if not credentials_synced:
        logger.warning(

```

```

        "[UbisoftSession] inject: nothing synced",
    )
    return credentials_synced

def ensure_auth_state_in_prefixes(
    self, prefix_paths: list[str],
) -> int:

    """Ensure auth state in prefixes."""
    ensured = 0
    for prefix_path in prefix_paths:
        if not Path(prefix_path).is_dir():
            continue
        try:
            if self.inject_into_prefix(prefix_path):
                ensured += 1
        except Exception as e:
            logger.warning(
                "[UbisoftSession] ensure auth state "
                "failed for %s: %s",
                Path(prefix_path).name, e,
            )
    if ensured:
        logger.info(
            "[UbisoftSession] ensured auth state across "
            "%d prefix(es)", ensured,
        )
    return ensured

def retroactive_sync(self) -> dict[str, Any]:
    """Retroactive sync."""
    try:
        cred_count = self.propagate_credentials_to_all()
        self.propagate_auth_artifacts_to_all()
        target_prefixes: list[str] = []
        for hidden_prefix in (
            self._config.auth_prefix_dir_expanded,
            self._config.template_dir_expanded,
        ):
            if Path(hidden_prefix).is_dir():
                target_prefixes.append(hidden_prefix)
        target_prefixes.extend(
            self._config.iter_game_prefix_paths(),
        )
        token_count = self.ensure_auth_state_in_prefixes(
            target_prefixes,
        )
        return {
            "success": True,
            "credentials_synced": cred_count,
            "token_propagated": token_count > 0,
        }
    except Exception as e:
        logger.error(
            "[UbisoftSession] retroactive sync failed: %s",
            e,
        )
    return {"success": False, "error": str(e)}

```

OP-55a

store.py

Fichier : py_modules/unifideck/stores/ubisoft/store.py | Lignes : 225 | Depend de : (rien)

```

from __future__ import annotations
import logging
from collections.abc import Awaitable, Callable
from typing import TYPE_CHECKING, Any, cast
from ...core.types import (
    AuthResult,
    Game,
    InstallResult,
    Result,
    StoreInfo,
)
from ..shared.store_base import StoreBase
from .specialists import build_ubisoft_specialists
if TYPE_CHECKING:
    from ...config import ConfigManager
    from ...core.cache_manager import CacheManager
    from ...event_bus.event_bus import EventBus
    from ...services.shortcut import ShortcutService
    from ...steam.steamgriddb import SteamGridDBClient
    from .auth import UbisoftAuth
    from .installer import UbisoftInstaller
    from .library import UbisoftLibrary
logger = logging.getLogger(__name__)
class UbisoftStore(StoreBase):
    """Ubisoft store."""
    store_info = StoreInfo(
        name="ubisoft",
        display_name="Ubisoft",
        auth_method="shortcut",
        icon_asset="ubisoft.png",
        uses_wine=True,
        supports_install=True,
    )
    def __init__(
        self,
        bus: EventBus,
        cache: CacheManager,
        plugin_dir: str | None = None,
        config: ConfigManager | None = None,
        shortcut_service: ShortcutService | None = None,
        steamgriddb: SteamGridDBClient | None = None,
    ) -> None:
        """Initialize the instance."""
        super().__init__(bus, cache, plugin_dir, config)
        specialists = build_ubisoft_specialists(
            bus=bus,
            config_mgr=config,
            plugin_dir=plugin_dir,
            shortcut_service=shortcut_service,
            steamgriddb=steamgriddb,
        )
        self._config = specialists.config
        self._paths = specialists.paths
        self._binaries = specialists.binaries
        self._id_map = specialists.id_map

```

```

        self._session = specialists.session
        self._installer_cache = specialists.installer_cache
        self._prefix_mgr = specialists.prefix_mgr
        self._library: UbisoftLibrary = specialists.library
        self._installer: UbisoftInstaller = specialists.installer
        self._auth: UbisoftAuth = specialists.auth
        self._ubi_config = specialists.config

    async def is_available(self) -> bool:

        """Check whether available."""
        available = await self._auth.is_available()
        self._cached_available = available
        return available

    async def start_auth(self, **kwargs: Any) -> AuthResult:
        """Start auth."""
        await self._auth.ensure_auth_shortcut()
        await self._auth.start_auth_session_monitor()
        return cast("AuthResult", await self._auth.start_auth())

    async def complete_auth(
        self, code: str = "", **kwargs: Any,
    ) -> AuthResult:
        """Complete auth."""
        return await self._auth.complete_auth(code, **kwargs)

    async def logout(self) -> Result:
        """Logout."""
        return await self._auth.logout()

    async def get_library(self) -> list[Game] | None:
        """Get library."""
        return await self._library.get_library()

    async def install_game(
        self,
        game_id: str,
        *,
        progress_cb: Callable[[dict[str, Any]], Awaitable[None]] | None = None,
        install_path: str | None = None,
        **kwargs: Any,
    ) -> InstallResult:
        """Install game."""
        return await self._installer.install_game(
            game_id,
            progress_cb=progress_cb,
            install_path=install_path,
        )

    async def uninstall_game(
        self,
        game_id: str,
        *,
        delete_prefix: bool = False,
        **kwargs: Any,
    ) -> Result:
        """Uninstall game."""
        return await self._installer.uninstall_game(
            game_id, delete_prefix=delete_prefix,
        )

    async def update_game(
        self,
        game_id: str,
        **kwargs: Any,
    ) -> InstallResult:
        """Update game."""

```

```

        return await self._installer.update_game(game_id)
    async def check_for_updates(self) -> list[str]:
        """Check for updates."""
        return await self._installer.check_for_updates()
    async def get_game_size(
        self, game_id: str,
    ) -> int | None:
        """Get game size."""
        return None
    async def get_installed(self) -> dict[str, Any]:
        """Get installed."""
        return await self._library.get_installed()
    def get_installed_game_info(
        self, game_id: str,
    ) -> dict[str, Any] | None:
        """Get installed game info."""
        return self._library.get_installed_game_info(game_id)
    async def write_install_marker(
        self,
        space_id: str,
        install_path: str,
        executable: str,
        game_title: str = "",
    ) -> None:
        """Write install marker."""
        await self._library.write_install_marker(
            space_id=space_id,
            install_path=install_path,
            executable=executable,
            game_title=game_title,
        )

    def find_game_executable(
        self, install_path: str,
    ) -> str | None:
        """Find game executable."""
        return self._library.find_game_executable(install_path)
    def is_install_session_active(self, game_id: str) -> bool:
        """Check whether install session active."""
        return self._installer.is_install_session_active(game_id)
    async def cancel_install_session(
        self, game_id: str,
    ) -> Result:
        """Check whether install session."""
        return await self._installer.cancel_install_session(
            game_id,
        )
    async def open_launcher_for_install(
        self, game_id: str,
    ) -> Result:
        """Open launcher for install."""
        return await self._installer.open_launcher_for_install(
            game_id,
        )
    def resolve_install_id(
        self, space_id: str,
    ) -> str | None:
        """Resolve install ID."""
        return self._id_map.resolve_install_id(space_id)
    def resolve_launch_id(

```

```

self, space_id: str,
) -> str | None:
    """Resolve launch ID."""
    return self._id_map.resolve_launch_id(space_id)
async def get_auth_shortcut_context(
self,
) -> dict[str, Any]:
    """Get auth shortcut context."""
    return await self._auth.get_auth_shortcut_context()
async def start_auth_session_monitor(self) -> Result:
    """Start auth session monitor."""
    return await self._auth.start_auth_session_monitor()
def check_auth_session_status(self) -> dict[str, Any]:
    """Check auth session status."""
    return self._auth.check_auth_session_status()
async def connect_ubisoft_account(
self,
) -> dict[str, Any]:
    """Connect UBISOFT account."""
    return await self._auth.connect_ubisoft_account()
def sync_ubisoft_credentials(self) -> dict[str, Any]:
    """Sync UBISOFT credentials."""
    return self._session.retroactive_sync()
async def repair_prefix(self, space_id: str) -> Result:
    """Repair prefix."""
    success = await self._prefix_mgr.repair_prefix(space_id)
    if not success:
        return Result(
            success=False, error="prefix_repair_failed",
        )
    prefix_path = self._paths.get_prefix_path(space_id)
    self._session.inject_into_prefix(prefix_path)
    install_id = self._id_map.resolve_install_id(space_id)
    if install_id:
        game_info = self._library._detector._detect_installed_game(
            space_id, prefix_path,
        )
        if game_info and game_info.get("install_path"):
            self._installer.inject_install_registry(
                prefix_path,
                install_id,
                game_info["install_path"],
            )
    return Result(success=True)
def get_game_official_url(
self, game_id: str,
) -> str | None:
    """Get game official URL."""
    return self._library.get_game_official_url(game_id)
def kill_upc_processes(self) -> None:
    """Kill UPC processes."""
    self._installer.kill_upc_processes()

```


OP-55b

config.py

Fichier : py_modules/unifideck/stores/ubisoft/config.py | Lignes : 332 | Depend de : (rien)

```
from __future__ import annotations
import logging
import os
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any, ClassVar
from unifideck.utils.config_helpers import get_cfg
if TYPE_CHECKING:
    from ..config import ConfigManager
logger = logging.getLogger(__name__)
_DEFAULT_DATA_DIR = "~/.local/share/unifideck"
_DEFAULT_ID_MAP_FILE = (
    "~/.local/share/unifideck/ubisoft_id_map.json"
)
_DEFAULT_VISIBLE_GAMES_FILE = (
    "~/.local/share/unifideck/ubisoft_visible_games.json"
)
_DEFAULT_PREFIXES_DIR = (
    "~/.local/share/unifideck/prefixes/ubisoft"
)
_DEFAULT_INSTALLER_CACHE_DIR = (
    "~/.local/share/unifideck/ubisoft_installer_cache"
)
_DEFAULT_UPC_SESSION_FILE = (
    "~/.local/share/unifideck/ubisoft_upc_session.txt"
)
_DEFAULT_GAME_ID_DB_FILE = (
    "~/.local/share/unifideck/ubisoft_game_db.txt"
)
_DEFAULT_DEFAULT_INSTALL_BASE = "~/Games/Ubisoft"
_DEFAULT_SDCARD_INSTALL_BASE = (
    "/run/media/mmcblk0p1/Games/Ubisoft"
)
_DEFAULT_INSTALLER_URL = (
    "https://static3.cdn.ubi.com/orbit/"
    "launcher_installer/UbisoftConnectInstaller.exe"
)
_DEFAULT_INSTALLER_FILENAME = "UbisoftConnectInstaller.exe"
_DEFAULT_GAME_ID_DB_URL = (
    "https://raw.githubusercontent.com/iAratorias/"
    "ubisoft_game_ids/main/UBI_GAMES.txt"
)
_DEFAULT_UPC_RELATIVE_PATH = (
    "drive_c/Program Files (x86)/"
    "Ubisoft/Ubisoft Game Launcher/upc.exe"
)
_DEFAULT_UPC_CONNECT_RELATIVE_PATH = (
    "drive_c/Program Files (x86)/"
    "Ubisoft/Ubisoft Game Launcher/UbisoftConnect.exe"
)
_DEFAULT_CONFIGURATIONS_RELATIVE_PATH = (
    "drive_c/users/steamuser/AppData/Local/"
    "Ubisoft Game Launcher/cache/configuration/configurations"
)
_DEFAULT_OWNERSHIP_RELATIVE_PATH = (
    "drive_c/users/steamuser/AppData/Local/"
```

```

    "Ubisoft Game Launcher/cache/ownership"
)
_UBI_CONFIG_PREFIX = "stores.ubisoft"
@dataclass(frozen=True)
class UbisoftConfig:
    """Ubisoft config."""
    _FIELD_SPECS: ClassVar[tuple]
    data_dir: str = _DEFAULT_DATA_DIR

    id_map_file: str = _DEFAULT_ID_MAP_FILE
    visible_games_file: str = _DEFAULT_VISIBLE_GAMES_FILE
    prefixes_dir: str = _DEFAULT_PREFIXES_DIR
    installer_cache_dir: str = _DEFAULT_INSTALLER_CACHE_DIR
    upc_session_file: str = _DEFAULT_UPC_SESSION_FILE
    game_id_db_file: str = _DEFAULT_GAME_ID_DB_FILE
    default_install_base: str = _DEFAULT_DEFAULT_INSTALL_BASE
    sdcard_install_base: str = _DEFAULT_SDCARD_INSTALL_BASE
    template_prefix_name: str = ".template"
    auth_prefix_name: str = ".upc-auth"
    auth_shortcut_store_id: str = "ubisoft:upc-auth"
    auth_shortcut_launch_wait_ms: int = 1500
    installer_url: str = _DEFAULT_INSTALLER_URL
    installer_filename: str = _DEFAULT_INSTALLER_FILENAME
    bootstrap_marker: str = (
        "unifideck_ubisoft_bootstrap.marker"
    )
    game_id_db_url: str = _DEFAULT_GAME_ID_DB_URL
    game_id_db_max_age_seconds: int = 7 * 24 * 3600
    upc_relative_path: str = _DEFAULT_UPC_RELATIVE_PATH
    upc_connect_relative_path: str = (
        _DEFAULT_UPC_CONNECT_RELATIVE_PATH
    )
    configurations_relative_path: str = (
        _DEFAULT_CONFIGURATIONS_RELATIVE_PATH
    )
    ownership_relative_path: str = (
        _DEFAULT_OWNERSHIP_RELATIVE_PATH
    )
    upc_credential_files: tuple[str, ...] = (
        "ConnectSecureStorage.dat",
        "user.dat",
    )
    upc_local_subdir: str = os.path.join(
        "AppData", "Local", "Ubisoft Game Launcher",
    )
    upc_auth_cache_artifacts: tuple[str, ...] = (
        "settings.yaml",
        os.path.join("cache", "configuration"),
        os.path.join("cache", "settings"),
        os.path.join("cache", "ulcf"),
        os.path.join(
            "cache", "http2", "Default", "Network",
        ),
        os.path.join(
            "cache", "http2", "Default", "Local Storage",
        ),
        os.path.join(
            "cache", "http2", "Default", "IndexedDB",
        ),
        os.path.join(
            "cache", "http2", "Default", "Preferences",

```

```

    ),
    os.path.join(
        "cache", "http2", "Default", "Session Storage",
    ),
    os.path.join("cache", "ownership"),
)
wine_system_users: tuple[str, ...] = (
    "Public", "All Users", "Default", "Default User",
)
filter_steam_linked: bool = True
steam_library_cross_ref: bool = False
@property
def data_dir_expanded(self) -> str:
    """Data dir expanded."""
    return os.path.expanduser(self.data_dir)
@property
def id_map_file_expanded(self) -> str:
    """Id map file expanded."""
    return os.path.expanduser(self.id_map_file)
@property
def visible_games_file_expanded(self) -> str:
    """Visible games file expanded."""
    return os.path.expanduser(self.visible_games_file)

@property
def prefixes_dir_expanded(self) -> str:
    """Prefixes dir expanded."""
    return os.path.expanduser(self.prefixes_dir)
@property
def template_dir_expanded(self) -> str:
    """Template dir expanded."""
    return os.path.join(
        self.prefixes_dir_expanded,
        self.template_prefix_name,
    )
@property
def auth_prefix_dir_expanded(self) -> str:
    """Auth prefix dir expanded."""
    return os.path.join(
        self.prefixes_dir_expanded,
        self.auth_prefix_name,
    )
@property
def installer_cache_dir_expanded(self) -> str:
    """Installer cache dir expanded."""
    return os.path.expanduser(self.installer_cache_dir)
@property
def upc_session_file_expanded(self) -> str:
    """Upc session file expanded."""
    return os.path.expanduser(self.upc_session_file)
@property
def game_id_db_file_expanded(self) -> str:
    """Game ID db file expanded."""
    return os.path.expanduser(self.game_id_db_file)
@property
def default_install_base_expanded(self) -> str:
    """Default install base expanded."""
    return os.path.expanduser(self.default_install_base)
def iter_game_prefix_paths(self) -> list[str]:
    """Iter game prefix paths."""
    prefixes_dir = self.prefixes_dir_expanded

```

```

    if not os.path.isdir(prefixes_dir):
        return []
    result: list[str] = []
    try:
        for entry in os.listdir(prefixes_dir):
            if entry.startswith("."):
                continue
            candidate = os.path.join(prefixes_dir, entry)
            if os.path.isdir(candidate):
                result.append(candidate)
    except OSError:
        pass
    return result
@staticmethod
def _parse_str(
    config: ConfigManager | None, key: str, default: str,
) -> str:
    """Parse str."""
    val = get_cfg(
        config, f"{_UBI_CONFIG_PREFIX}.{key}", default,
    )
    return (
        str(val).strip()
        if val is not None
        else default
    )
@staticmethod
def _parse_int(
    config: ConfigManager | None, key: str, default: int,
) -> int:
    """Parse int."""
    val = get_cfg(
        config, f"{_UBI_CONFIG_PREFIX}.{key}", default,
    )
    try:
        return int(val)
    except (TypeError, ValueError):
        return default

@staticmethod
def _parse_tuple(
    config: ConfigManager | None, key: str, default: tuple[str, ...],
) -> tuple[str, ...]:
    """Parse tuple."""
    val = get_cfg(
        config, f"{_UBI_CONFIG_PREFIX}.{key}", None,
    )
    if not isinstance(val, list):
        return default
    filtered = [
        str(x) for x in val
        if isinstance(x, str) and x
    ]
    return tuple(filtered) if filtered else default
@staticmethod
def _parse_bool(
    config: ConfigManager | None, key: str, default: bool,
) -> bool:
    """Parse bool."""
    val = get_cfg(
        config, f"{_UBI_CONFIG_PREFIX}.{key}", default,

```

```

    )
    if isinstance(val, bool):
        return val
    if isinstance(val, str):
        lowered = val.strip().lower()
        if lowered in ("true", "1", "yes", "on"):
            return True
        if lowered in ("false", "0", "no", "off"):
            return False
        return default
    @classmethod
    def from_config_manager(
        cls, config: ConfigManager | None,
    ) -> UbisoftConfig:
        """From config manager."""
        kwargs: dict[str, Any] = {}
        for field_name, key, parser, default in (
            cls._FIELD_SPECS
        ):
            kwargs[field_name] = parser(config, key, default)
        return cls(**kwargs)
    def describe(self) -> str:
        """Describe."""
        return (
            f"UbisoftConfig("
            f"prefixes_dir={self.prefixes_dir}, "
            f"install_base={self.default_install_base}, "
            f"installer_url={self.installer_url[:40]}...)")
    )
    UbisoftConfig._FIELD_SPECS = (
        ("data_dir", "data_dir",
         UbisoftConfig._parse_str, _DEFAULT_DATA_DIR),
        ("id_map_file", "id_map_file",
         UbisoftConfig._parse_str, _DEFAULT_ID_MAP_FILE),
        ("visible_games_file", "visible_games_file",
         UbisoftConfig._parse_str, _DEFAULT_VISIBLE_GAMES_FILE),
        ("prefixes_dir", "prefixes_dir",
         UbisoftConfig._parse_str, _DEFAULT_PREFIXES_DIR),
        ("installer_cache_dir", "installer_cache_dir",
         UbisoftConfig._parse_str,
         _DEFAULT_INSTALLER_CACHE_DIR),
        ("upc_session_file", "upc_session_file",
         UbisoftConfig._parse_str, _DEFAULT_UPC_SESSION_FILE),
        ("game_id_db_file", "game_id_db_file",
         UbisoftConfig._parse_str, _DEFAULT_GAME_ID_DB_FILE),
        ("default_install_base", "default_install_base",
         UbisoftConfig._parse_str,
         _DEFAULT_DEFAULT_INSTALL_BASE),
        ("sdcard_install_base", "sdcard_install_base",
         UbisoftConfig._parse_str,
         _DEFAULT_SDCARD_INSTALL_BASE),
        ("template_prefix_name", "template_prefix_name",
         UbisoftConfig._parse_str, ".template"),
        ("auth_prefix_name", "auth_prefix_name",
         UbisoftConfig._parse_str, ".upc-auth"),
        ("auth_shortcut_store_id", "auth_shortcut_store_id",
         UbisoftConfig._parse_str, "ubisoft:upc-auth"),
        ("auth_shortcut_launch_wait_ms",
         "auth_shortcut_launch_wait_ms",
         UbisoftConfig._parse_int, 1500),
        ("installer_url", "installer_url",

```

```
    UbisoftConfig._parse_str, _DEFAULT_INSTALLER_URL),
    ("installer_filename", "installer_filename",

    UbisoftConfig._parse_str,
    _DEFAULT_INSTALLER_FILENAME),
    ("bootstrap_marker", "bootstrap_marker",
    UbisoftConfig._parse_str,
    "unifideck_ubisoft_bootstrap.marker"),
    ("game_id_db_url", "game_id_db_url",
    UbisoftConfig._parse_str, _DEFAULT_GAME_ID_DB_URL),
    ("game_id_db_max_age_seconds",
    "game_id_db_max_age_seconds",
    UbisoftConfig._parse_int, 7 * 24 * 3600),
    ("upc_relative_path", "upc_relative_path",
    UbisoftConfig._parse_str, _DEFAULT_UPC_RELATIVE_PATH),
    ("upc_connect_relative_path",
    "upc_connect_relative_path",
    UbisoftConfig._parse_str,
    _DEFAULT_UPC_CONNECT_RELATIVE_PATH),
    ("configurations_relative_path",
    "configurations_relative_path",
    UbisoftConfig._parse_str,
    _DEFAULT_CONFIGURATIONS_RELATIVE_PATH),
    ("ownership_relative_path", "ownership_relative_path",
    UbisoftConfig._parse_str,
    _DEFAULT_OWNERSHIP_RELATIVE_PATH),
    ("upc_credential_files", "upc_credential_files",
    UbisoftConfig._parse_tuple,
    ("ConnectSecureStorage.dat", "user.dat")),
    ("wine_system_users", "wine_system_users",
    UbisoftConfig._parse_tuple,
    ("Public", "All Users", "Default", "Default User")),
    ("filter_steam_linked", "filter_steam_linked",
    UbisoftConfig._parse_bool, True),
    ("steam_library_cross_ref", "steam_library_cross_ref",
    UbisoftConfig._parse_bool, False),
)
```

OP-55c

paths.py

Fichier : py_modules/unifideck/stores/ubisoft/paths.py | Lignes : 72 | Depend de : (rien)

```

from __future__ import annotations
from collections.abc import Iterator
from pathlib import Path
from .config import UbisoftConfig
class UbisoftPrefixPaths:
    """Ubisoft prefix paths."""
    def __init__(self, config: UbisoftConfig) -> None:
        """Initialize the instance."""
        self._config = config
    def find_upc_exe(self, prefix_path: str) -> str | None:
        """Find UPC exe."""
        return self._find_in_prefix(
            prefix_path, self._config.upc_relative_path,
        )
    def find_connect_exe(self, prefix_path: str) -> str | None:
        """Find connect exe."""
        return self._find_in_prefix(
            prefix_path,
            self._config.upc_connect_relative_path,
        )
    def find_configurations(
        self, prefix_path: str,
    ) -> str | None:
        """Find configurations."""
        return self._find_in_prefix(
            prefix_path,
            self._config.configurations_relative_path,
        )
    def iter_user_homes(
        self,
        prefix_path: str,
        pfx_first: bool = False,
    ) -> Iterator[tuple[str, str]]:
        """Iter user homes."""
        roots = [
            prefix_path,
            str(Path(prefix_path) / "pfx"),
        ]
        if pfx_first:
            roots = list(reversed(roots))
        skip = set(self._config.wine_system_users)
        for prefix_root in roots:
            users_dir = Path(prefix_root) / "drive_c" / "users"
            if not users_dir.is_dir():
                continue
            try:
                entries = list(users_dir.iterdir())
            except OSError:
                continue
            for entry in entries:
                if entry.name in skip:
                    continue
                if entry.is_dir():
                    yield prefix_root, str(entry)
    def get_prefix_path(self, space_id: str) -> str:

```

```
    """Get prefix path."""
    return str(
        Path(self._config.prefixes_dir_expanded) / space_id,
    )

    @staticmethod
    def _find_in_prefix(
        prefix_path: str, relative: str,
    ) -> str | None:
        """Find in prefix."""
        prefix = Path(prefix_path)
        for candidate in (
            prefix / relative,
            prefix / "pfx" / relative,
        ):
            if candidate.is_file() or candidate.is_dir():
                return str(candidate)
        return None
```


OP-55d

binaries.py

Fichier : py_modules/unifideck/stores/ubisoft/binaries.py | Lignes : 295 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
import shutil
import subprocess
from pathlib import Path
from .config import UbisoftConfig
logger = logging.getLogger(__name__)
_DISPLAY_ENV_VARS = (
    "DISPLAY",
    "WAYLAND_DISPLAY",
    "DBUS_SESSION_BUS_ADDRESS",
    "XAUTHORITY",
)
_PROTON_OFFICIAL_NAMES = (
    "Proton - Experimental",
    "Proton 10.0",
    "Proton 9.0 (Beta)",
)
_STEAM_COMMON_CANDIDATES = (
    str(Path("~") / ".steam" / "steam" / "steamapps" / "common"),
    str(
        Path("~") / ".local" / "share" / "Steam"
        / "steamapps" / "common",
    ),
    str(Path("~") / ".steam" / "root" / "steamapps" / "common"),
)
_COMPAT_TOOLS_DIR = "~/.local/share/Steam/compatibilitytools.d"
class UbisoftBinaryResolver:
    """Ubisoft binary resolver."""
    def __init__(
        self,
        config: UbisoftConfig,
        plugin_dir: str | None,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._plugin_dir = plugin_dir
    def find_umu_run(self) -> str | None:
        """Find UMU run."""
        candidates: list[str] = []
        if self._plugin_dir:
            candidates.append(str(
                Path(self._plugin_dir)
                / "bin" / "umu" / "umu" / "umu-run",
            ))
        candidates.extend([
            str(Path(
                "~/.local/share/unifideck/bin/umu/umu/umu-run",
            ).expanduser()),
            "/usr/bin/umu-run",
        ])
        for path in candidates:
            if Path(path).is_file():
                return path

```

```

        logger.warning("[UbisoftBinaryResolver] umu-run not found")
        return None

def find_proton_path(self) -> str | None:

    """Find PROTON path."""
    official = self._find_official_proton()
    if official is not None:
        return official
    custom = self._find_custom_proton()
    if custom is not None:
        return custom
    logger.warning(
        "[UbisoftBinaryResolver] no Proton found in "
        "steamapps/common or compatibilitytools.d",
    )
    return None

@staticmethod
def _find_official_proton() -> str | None:
    """Find official PROTON."""
    for steam_common_raw in _STEAM_COMMON_CANDIDATES:
        steam_common = Path(steam_common_raw).expanduser()
        for name in _PROTON_OFFICIAL_NAMES:
            candidate = steam_common / name
            if candidate.is_dir():
                logger.info(
                    "[UbisoftBinaryResolver] using "
                    "Proton: %s",
                    name,
                )
                return str(candidate)
    return None

@staticmethod
def _find_custom_proton() -> str | None:
    """Find custom PROTON."""
    compat_dir = Path(_COMPAT_TOOLS_DIR).expanduser()
    if not compat_dir.is_dir():
        return None
    umu_candidates: list[str] = []
    ge_candidates: list[str] = []
    try:
        for entry in compat_dir.iterdir():
            if not entry.is_dir():
                continue
            if entry.name.startswith("UMU-Proton"):
                umu_candidates.append(str(entry))
            elif entry.name.startswith("GE-Proton"):
                ge_candidates.append(str(entry))
    except OSError as e:
        logger.warning(
            "[UbisoftBinaryResolver] compatibilitytools.d "
            "scan failed: %s", e,
        )
        return None
    ge_candidates.sort(
        key=lambda p: Path(p).name, reverse=True,
    )
    ordered = umu_candidates + ge_candidates
    if not ordered:
        return None
    logger.info(

```

```

        "[UbisoftBinaryResolver] using Proton: %s",
        Path(ordered[0]).name,
    )
    return ordered[0]
@staticmethod
def find_python() -> str:
    """Find python."""
    for name in ("python3", "python"):
        path = shutil.which(name)
        if path:
            return path
    return "python3"

@staticmethod
def proton_family(version_str: str) -> str:
    """Proton family."""
    v = (version_str or "").lower()
    if "umu-proton" in v:
        return "umu-proton"
    if "ge-proton" in v:
        return "ge-proton"
    if "experimental" in v:
        return "experimental"
    stripped = (
        v.replace(".", "")
        .replace("-", "")
        .replace(" ", "")
    )
    if stripped.isdigit():
        return "experimental"
    return "other"
def build_umu_env(
    self,
    wineprefix: str,
    gameid: str,
    *,
    proton_path: str | None = None,
    store_game_id: str | None = None,
    steam_window_env: dict[str, str] | None = None,
) -> dict[str, str]:
    """Build UMU env."""
    home = os.environ.get(
        "HOME", str(Path("~").expanduser()),
    )
    uid = os.getuid()
    env: dict[str, str] = {
        "HOME": home,
        "USER": os.environ.get("USER", "deck"),
        "PATH": os.environ.get(
            "PATH", "/usr/local/bin:/usr/bin:/bin",
        ),
        "LANG": os.environ.get("LANG", "en_US.UTF-8"),
        "XDG_RUNTIME_DIR": os.environ.get(
            "XDG_RUNTIME_DIR", f"/run/user/{uid}",
        ),
        "XDG_DATA_HOME": os.environ.get(
            "XDG_DATA_HOME",
            str(Path(home) / ".local" / "share"),
        ),
        "WINEPREFIX": wineprefix,
        "GAMEID": gameid,
    }

```

```

        "STORE": "ubisoft",
        "PROTON_VERB": "waitforexitandrun",
    }
    if steam_window_env:
        env.update(steam_window_env)
    if proton_path is None:
        proton_path = self.find_proton_path()
    if proton_path:
        env["PROTONPATH"] = proton_path
    env.update(self.detect_display_env())
    return env
def detect_display_env(self) -> dict[str, str]:
    """Detect display env."""
    result = self._collect_display_env_from_self()
    if result.get("DISPLAY") or result.get("WAYLAND_DISPLAY"):
        return result
    if self._fill_display_env_from_steam(result):
        return result
    self._apply_steam_deck_defaults(result)
    return result
@staticmethod
def _collect_display_env_from_self() -> dict[str, str]:
    """Collect display env from self."""
    result: dict[str, str] = {}
    for var in _DISPLAY_ENV_VARS:
        val = os.environ.get(var)
        if val:
            result[var] = val
    return result
def _fill_display_env_from_steam(
    self, result: dict[str, str],
) -> bool:
    """Fill display env from steam."""
    try:
        for proc_name in ("steam", "gamescope-session"):
            for pid in self._pgrep(proc_name):
                if self._scan_pid_for_display(pid, result):
                    logger.info(
                        "[UbisoftBinaryResolver] display "
                        "env detected from PID %s (%s)",
                        pid, proc_name,
                    )
                    return True
    except Exception as e:
        logger.debug(
            "[UbisoftBinaryResolver] display env "
            "detection: %s",
            e,
        )
    return False
def _scan_pid_for_display(
    self, pid: str, result: dict[str, str],
) -> bool:
    """Scan pid for display."""
    env_from_proc = self._read_proc_environ(
        pid, _DISPLAY_ENV_VARS,
    )
    if not env_from_proc:
        return False

```

```

        for k, v in env_from_proc.items():
            result.setdefault(k, v)
        return bool(
            result.get("DISPLAY") or result.get("WAYLAND_DISPLAY"),
        )
    @staticmethod
    def _apply_steam_deck_defaults(
        result: dict[str, str],
    ) -> None:
        """Apply steam DECK defaults."""
        if not result.get("DISPLAY"):
            result["DISPLAY"] = ":0"
            logger.info(
                "[UbisoftBinaryResolver] using fallback "
                "DISPLAY=:0",
            )
        xauth = Path("~").expanduser() / ".Xauthority"
        if (
            not result.get("XAUTHORITY")
            and xauth.is_file()
        ):
            result["XAUTHORITY"] = str(xauth)
    @staticmethod
    def _pgrep(process_name: str) -> list[str]:
        """Pgrep."""
        try:
            result = subprocess.run(
                [
                    "pgrep", "-u", str(os.getuid()),
                    "-x", process_name,
                ],
                capture_output=True,
                text=True,
                timeout=5,
                check=False,
            )
        except (subprocess.SubprocessError, OSError):
            return []
        return [
            line.strip()
            for line in result.stdout.strip().splitlines()
            if line.strip()
        ]
    @staticmethod
    def _read_proc_environ(
        pid: str, targets: tuple[str, ...],
    ) -> dict[str, str]:
        """Read proc environ."""
        try:
            env_path = Path(f"/proc/{pid}/environ")
            env_bytes = env_path.read_bytes()
        except (PermissionError, FileNotFoundError, OSError):
            return {}
        result: dict[str, str] = {}
        for entry in env_bytes.split(b"\x00"):
            decoded = entry.decode("utf-8", errors="replace")
            if "=" not in decoded:
                continue
            k, v = decoded.split("=", 1)
            if k in targets:

```

```
        result[k] = v  
    return result
```

OP-55e

parser.py

Fichier : py_modules/unifideck/stores/ubisoft/parser.py | Lignes : 303 | Depend de : (rien)

```

import logging
import os
import re
from typing import Any, Optional, cast
import yaml
from .parser_binary import (
    parse_install_id,
    parse_launch_id,
    parse_ownership_record,
    parse_record_size,
)
logger = logging.getLogger(__name__)
BLACKLISTED_NAMES = [
    "gamename",
    "l1",
    "l2",
    "thumbimage",
    "",
    "ubisoft game",
    "name"]
def _parse_config_header(header: bytes, second_eight: bool = False) -> tuple:
    """Parse config header."""
    try:
        offset = 1
        record_size, offset, tmp_size = parse_record_size(
            header, offset, second_eight,
        )
        install_id, offset = parse_install_id(header, offset)
        launch_id, offset = parse_launch_id(header, offset)
        if record_size - offset < 128 <= record_size:
            tmp_size -= 1
            record_size += 1
            return (
                record_size - offset, install_id, launch_id,
                offset + tmp_size + 1,
            )
        return 0, 0, 0, 10
    except Exception:
        return 0, 0, 0, 10
def _get_yaml_field(game_yaml: dict, field: str = "name") -> str:
    """Get yaml field."""
    root = game_yaml.get("root", {})
    if not isinstance(root, dict):
        return ""
    value = str(root[field]) if field in root else ""
    if field == "name" and value.lower() in BLACKLISTED_NAMES:
        value = _yaml_field_installer_fallback(root, value)
        if value.lower() in BLACKLISTED_NAMES:
            return _yaml_field_localization_fallback(
                game_yaml, value,
            )
    return value
def _yaml_field_installer_fallback(root: dict, current: str) -> str:
    """Yaml field installer fallback."""
    installer = root.get("installer", {})

```

```

    if (
        isinstance(installer, dict)
        and "game_identifier" in installer
    ):
        return str(installer["game_identifier"])
    return current

def _yaml_field_localization_fallback(
    game_yaml: dict, current: str,
) -> str:

    """Yaml field localization fallback."""
    locs = game_yaml.get("localizations", {})
    if not isinstance(locs, dict):
        return current
    default_loc = locs.get("default", {})
    if (
        isinstance(default_loc, dict)
        and current in default_loc
    ):
        return str(default_loc[current])
    return current

class GameConfig:
    """Game config."""
    def __init__(self):
        """Initialize the instance."""
        self.install_id: int = 0
        self.launch_id: int = 0
        self.space_id: str = ""
        self.name: str = ""
        self.executable: str = ""
        self.thumb_image: str = ""
        self.game_identifier: str = ""
        self.yaml_raw: str = ""
        self.third_party_platform: str = ""
    def __repr__(self) -> str:
        """Repr."""
        return (
            f"GameConfig(name={self.name!r}, space_id={self.space_id!r}, "
            f"install_id={self.install_id}, launch_id={self.launch_id})"
        )

def _read_binary_file(filepath: str) -> bytes | None:
    """Read binary file."""
    if not os.path.isfile(filepath):
        logger.warning(
            "[UbiParser] Configurations file not found: %s",
            filepath,
        )
        return None
    try:
        with open(filepath, "rb") as f:
            return f.read()
    except Exception as e:
        logger.error(
            "[UbiParser] Failed to read configurations: %s", e,
        )
        return None

def _extract_config_chunk(
    data: bytes,
    global_offset: int,

```



```

        header_size: int,
        obj_size: int,
        install_id: int,
        launch_id: int,
    ) -> Optional["GameConfig"]:
```

"""Extract config chunk."""

```

    if obj_size <= 500:
        return None
    yaml_start = global_offset + header_size
    yaml_end = yaml_start + obj_size
    if yaml_end > len(data):
        return None
    stream = data[yaml_start:yaml_end].decode(
        "utf8", errors="ignore",
    )
    if not stream or "start_game" not in stream:
        return None
    try:
        parsed = yaml.safe_load(
            stream.replace("\t", " "),
        )
    except Exception as e:
        logger.debug(
            "[UbiParser] YAML parse error at offset %d: %s",
            global_offset, e,
        )
        return None
    if not parsed:
        return None
    config = _build_game_config(
        parsed, stream, install_id, launch_id,
    )
    if config and config.name:
        return config
    return None
def parse_configurations(filepath: str) -> list[GameConfig]:
    """Parse configurations."""
    data = _read_binary_file(filepath)
    if data is None:
        return []
    results: list[GameConfig] = []
    global_offset = 0
    while global_offset < len(data):
        chunk = data[global_offset:]
        obj_size, install_id, launch_id, header_size = (
            _parse_config_header(chunk)
        )
        launch_id = (
            install_id
            if launch_id == 0 or launch_id == install_id
            else launch_id
        )
        config = _extract_config_chunk(
            data, global_offset, header_size,
            obj_size, install_id, launch_id,
        )
        if config:
            results.append(config)
            global_offset_tmp = global_offset
            global_offset += obj_size + header_size

```

```

        if (global_offset < len(data)
            and data[global_offset] != 0x0A):
            obj_size, _, _, header_size = _parse_config_header(
                chunk, True,
            )
            global_offset = (
                global_offset_tmp + obj_size + header_size
            )
    logger.info(
        "[UbiParser] Parsed %d game configs from %s",
        len(results), filepath,
    )
    return results

def _build_game_config(
    parsed: dict, yaml_text: str, install_id: int, launch_id: int
) -> GameConfig | None:

    """Build game config."""
    config = GameConfig()
    config.install_id = install_id
    config.launch_id = launch_id
    config.yaml_raw = yaml_text
    config.name = _get_yaml_field(parsed, "name")
    config.thumb_image = _get_yaml_field(parsed, "thumb_image")
    root = parsed.get("root", {})
    if isinstance(root, dict):
        config.space_id = str(root.get("space_id", ""))
        installer = root.get("installer", {})
        if isinstance(installer, dict):
            config.game_identifier = str(
                installer.get("game_identifier", "")
            )
            config.third_party_platform = _extract_third_party_platform(
                root, installer,
            )
            exe_match = re.search(
                r"relative:\s*(.+?\.\exe)",
                yaml_text,
                re.IGNORECASE,
            )
            if exe_match:
                config.executable = exe_match.group(1).strip().strip("'\"")
                return config
    return None

def _extract_third_party_platform(root: dict, installer: Any) -> str:
    """Extract third party platform."""
    if isinstance(root.get("third_party_platform"), str):
        return cast("str", root["third_party_platform"].strip())
    if isinstance(installer, dict):
        value = installer.get("third_party_platform")
        if isinstance(value, str):
            return value.strip()
    start_game = root.get("start_game", {})
    if isinstance(start_game, dict):
        for mode in ("online", "offline"):
            mode_dict = start_game.get(mode, {})
            if isinstance(mode_dict, dict):
                value = mode_dict.get(
                    "third_party_platform",
                )
                if isinstance(value, str) and value:

```

```

        return value.strip()
    return ""
def parse_ownership(filepath: str) -> list[int]:
    """Parse ownership."""
    data = _read_ownership_file(filepath)
    if data is None:
        return []
    owned: list[int] = []
    offset = 0x108
    while offset < len(data):
        chunk = data[offset:]
        if chunk[0] != 0x0A:
            break
        record = parse_ownership_record(chunk)
        if record is None:
            break
        rec_size, tmp_size, lid1, lid2 = record
        owned.append(lid1)
        if lid2 != lid1:
            owned.append(lid2)
        offset += rec_size + tmp_size + 1
    logger.info(
        "[UbiParser] Found %d owned IDs in %s",
        len(owned), filepath,
    )
    return owned

def _read_ownership_file(filepath: str) -> bytes | None:
    """Read ownership file."""
    if not os.path.isfile(filepath):
        logger.warning(
            "[UbiParser] Ownership file not found: %s", filepath,
        )
        return None
    try:
        with open(filepath, "rb") as f:
            return f.read()
    except Exception as e:
        logger.error(
            "[UbiParser] Failed to read ownership: %s", e,
        )
        return None

def check_install_state(state_file: str) -> bool:
    """Check install state."""
    if not os.path.isfile(state_file):
        return False
    try:
        with open(state_file, "rb") as f:
            first_byte = f.read(1)
            return first_byte == b"\x0a"
    except Exception:
        return False

def build_id_map_from_configurations(
    filepath: str,
) -> dict[str, dict[str, Any]]:
    """Build ID map from configurations."""
    configs = parse_configurations(filepath)
    id_map: dict[str, dict[str, Any]] = {}
    for cfg in configs:
        if not cfg.space_id:

```

```
        continue
    id_map[cfg.space_id] = {
        "install_id": str(cfg.install_id),
        "launch_id": str(cfg.launch_id),
        "name": cfg.name,
        "executable": cfg.executable,
        "game_identifier": cfg.game_identifier,
    }
    logger.info(
        "[UbiParser] Built ID map with %d entries", len(id_map),
    )
    return id_map
```

OP-55f

parser_binary.py

Fichier : py_modules/unifideck/stores/ubisoft/parser_binary.py | Lignes : 87 | Depend de : (rien)

```

from __future__ import annotations
import math
def _convert_data(data: int) -> int:
    """Convert data."""
    if data > 256 * 256:
        data -= 128 * 256 * math.ceil(data / (256 * 256))
        data -= 128 * math.ceil(data / 256)
    elif data > 256:
        data -= 128 * math.ceil(data / 256)
    return data
def parse_record_size(
    header: bytes, offset: int, second_eight: bool,
) -> tuple[int, int, int]:
    """Parse record size."""
    multiplier = 1
    record_size = 0
    tmp_size = 0
    if second_eight:
        while (header[offset] != 0x08
              or (header[offset] == 0x08 and header[offset + 1] == 0x08)):
            record_size += header[offset] * multiplier
            multiplier *= 256
            offset += 1
            tmp_size += 1
    else:
        while header[offset] != 0x08 or record_size == 0:
            record_size += header[offset] * multiplier
            multiplier *= 256
            offset += 1
            tmp_size += 1
    record_size = _convert_data(record_size)
    offset += 1
    return record_size, offset, tmp_size
def parse_install_id(header: bytes, offset: int) -> tuple[int, int]:
    """Parse install ID."""
    multiplier = 1
    install_id = 0
    while header[offset] != 0x10 or header[offset + 1] == 0x10:
        install_id += header[offset] * multiplier
        multiplier *= 256
        offset += 1
    install_id = _convert_data(install_id)
    offset += 1
    return install_id, offset
def parse_launch_id(header: bytes, offset: int) -> tuple[int, int]:
    """Parse launch ID."""
    multiplier = 1
    launch_id = 0
    while (header[offset] != 0x1A
          or (header[offset] == 0x1A and header[offset + 1] == 0x1A)):
        launch_id += header[offset] * multiplier
        multiplier *= 256
        offset += 1
    launch_id = _convert_data(launch_id)
    return launch_id, offset

```

```
def parse_ownership_record(chunk: bytes) -> tuple | None:

    """Parse ownership record."""
    try:
        pos = 1
        multiplier = 1
        rec_size = 0
        tmp_size = 0
        while chunk[pos] != 0x08 or rec_size == 0:
            rec_size += chunk[pos] * multiplier
            multiplier *= 256
            pos += 1
            tmp_size += 1
        rec_size = _convert_data(rec_size)
        pos += 1
        multiplier = 1
        lid1 = 0
        while chunk[pos] != 0x10 or chunk[pos + 1] == 0x10:
            lid1 += chunk[pos] * multiplier
            multiplier *= 256
            pos += 1
        lid1 = _convert_data(lid1)
        pos += 1
        multiplier = 1
        lid2 = 0
        while chunk[pos] != 0x22:
            lid2 += chunk[pos] * multiplier
            multiplier *= 256
            pos += 1
        lid2 = _convert_data(lid2)
        return rec_size, tmp_size, lid1, lid2
    except Exception:
        return None
```

OP-55g

id_map.py

Fichier : py_modules/unifideck/stores/ubisoft/id_map.py | Lignes : 198 | Depend de : (rien)

```

from __future__ import annotations
import json
import logging
import os
import re
from typing import Any
from .config import UbisoftConfig
from .id_map_sources import (
    _IdMapSources,
    extract_game_id_from_registry as _extract_game_id_from_registry,
)
from .paths import UbisoftPrefixPaths
logger = logging.getLogger(__name__)
_STEAM_TITLE_PREFIXES_TO_SKIP = (
    "Proton",
    "Steam Linux Runtime",
    "Steamworks",
)
class UbisoftIdMap:
    """Ubisoft ID map."""
    def __init__(
        self,
        config: UbisoftConfig,
        paths: UbisoftPrefixPaths,
    ) -> None:
        """Initialize the instance."""
        self._config = config
        self._paths = paths
        self._cache: dict[str, dict[str, Any]] = {}
        self._load()
        self._sources = _IdMapSources(self)
    def _load(self) -> None:
        """Load."""
        path = self._config.id_map_file_expanded
        if not os.path.isfile(path):
            return
        try:
            with open(path, encoding="utf-8") as f:
                data = json.load(f)
            if isinstance(data, dict):
                self._cache = data
                logger.info(
                    "[UbisoftIdMap] loaded %d entries "
                    "from cache", len(self._cache),
                )
        except (OSError, json.JSONDecodeError) as e:
            logger.warning(
                "[UbisoftIdMap] could not load cache: %s",
                e,
            )
            self._cache = {}
    def _save(self) -> None:
        """Save."""
        path = self._config.id_map_file_expanded
        try:

```

```

        os.makedirs(
            self._config.data_dir_expanded, exist_ok=True,
        )
        tmp_path = path + ".tmp"
        with open(tmp_path, "w", encoding="utf-8") as f:
            json.dump(self._cache, f, indent=2)
            os.replace(tmp_path, path)
    except OSError as e:
        logger.warning(
            "[UbisoftIdMap] could not save cache: %s", e,
        )

def resolve_install_id(
    self, space_id: str,
) -> str | None:
    """Resolve install ID."""
    entry = self._cache.get(space_id, {})
    if "ubisoftconnect_game_id" in entry:
        return entry.get("ubisoftconnect_game_id")
    return entry.get("install_id")

def resolve_launch_id(
    self, space_id: str,
) -> str | None:
    """Resolve launch ID."""
    entry = self._cache.get(space_id, {})
    if "ubisoftconnect_game_id" in entry:
        return entry.get("ubisoftconnect_game_id")
    return entry.get("launch_id")

def update(
    self,
    space_id: str,
    install_id: str,
    launch_id: str,
) -> None:
    """Update."""
    self._cache[space_id] = {
        "install_id": install_id,
        "launch_id": launch_id,
    }
    self._save()

def update_bulk(
    self, mapping: dict[str, dict[str, Any]],
) -> None:
    """Update bulk."""
    changed = False
    for space_id, entry in mapping.items():
        existing = self._cache.get(space_id, {})
        merged = {**existing, **entry}
        if merged != existing:
            self._cache[space_id] = merged
            changed = True
            if changed:
                self._save()

def merge_entry(
    self,
    space_id: str,
    fields: dict[str, Any],
) -> bool:
    """Merge entry."""
    existing = self._cache.get(space_id, {})

```



```

        merged = {**existing, **fields}
        if merged == existing:
            return False
        self._cache[space_id] = merged
        self._save()
        return True
    def get_entry(
        self, space_id: str,
    ) -> dict[str, Any]:
        """Get entry."""
        return dict(self._cache.get(space_id, {}))
    def in_cache(self, space_id: str) -> bool:
        """In cache."""
        return space_id in self._cache
    async def refresh_from_configurations(
        self, space_id: str | None = None,
    ) -> bool:
        """Refresh from configurations."""
        return await self._sources.refresh_from_configurations(
            space_id,
        )
    async def fetch_game_id_database(
        self,
    ) -> list[tuple[str, str]]:
        """Fetch game ID database."""
        return await self._sources.fetch_game_id_database()
    async def lookup_game_id_by_name(
        self, game_name: str,
    ) -> str | None:
        """Lookup game ID by name."""
        return await self._sources.lookup_game_id_by_name(
            game_name,
        )

    @staticmethod
    def extract_game_id_from_registry(
        prefix_path: str,
    ) -> str | None:
        """Extract game ID from registry."""
        return _extract_game_id_from_registry(prefix_path)
    @staticmethod
    def get_steam_library_titles() -> set[str]:
        """Get steam library titles."""
        try:
            from ...steam.library import get_steam_library_names
        except ImportError:
            logger.debug(
                "[UbisoftIdMap] Steam library module not "
                "available",
            )
            return set()
        try:
            raw_names = get_steam_library_names()
        except Exception as e:
            logger.debug(
                "[UbisoftIdMap] Steam library scan "
                "failed: %s", e,
            )
            return set()
        steam_titles: set[str] = set()
        for name in raw_names:

```

```
        if not name or name.startswith(
            _STEAM_TITLE_PREFIXES_TO_SKIP,
        ):
            continue
        steam_titles.add(
            UbisoftIdMap._normalize_for_matching(name),
        )
    logger.debug(
        "[UbisoftIdMap] found %d Steam library titles",
        len(steam_titles),
    )
    return steam_titles

    @staticmethod
    def _normalize_for_matching(name: str) -> str:
        """Normalize for matching."""
        name = name.lower()
        name = name.replace("_", " ")
        name = re.sub(
            r"[@'@'\u2019\-\.:;!?\(\)\\"", "", name,
        )
        return " ".join(name.split())

    def normalize_for_matching(self, name: str) -> str:
        """Normalize for matching."""
        return self._normalize_for_matching(name)
```

OP-55h

id_map_sources.py

Fichier : py_modules/unifideck/stores/ubisoft/id_map_sources.py | Lignes : 300 | Depend de : (rien)

```

from __future__ import annotations
import asyncio
import logging
import re
import time
import urllib.request
from pathlib import Path
from typing import TYPE_CHECKING, Any
from ...core.net import ssl_ctx_permissive
if TYPE_CHECKING:
    from .id_map import UbisoftIdMap
logger = logging.getLogger(__name__)
_REGISTRY_INSTALLS_PATTERN = re.compile(
    r'\[Software\\\\Wow6432Node\\\\Ubisoft\\\\Launcher'
    r'\\\\Installs\\\\(\\d+)\\]'
    r'^\\[*?"Instal\\Dir"\\s*=\\s*"([^"]*)"\\',
    re.DOTALL,
)
_USER_REG_INSTALLS_PATTERN = re.compile(
    r'\\[Software\\\\Ubisoft\\\\Launcher\\\\Installs\\\\(\\d+)\\]',
)
_STANDARD_INSTALL_PATH_MARKERS = (
    "Ubisoft Game Launcher/games/",
    "Ubisoft Game Launcher\\games\\",
)
def extract_game_id_from_registry(
    prefix_path: str,
) -> str | None:
    """Extract game ID from registry."""
    prefix = Path(prefix_path)
    for reg_name in ("system.reg", "pfx/system.reg"):
        reg_path = prefix / reg_name
        if not reg_path.is_file():
            continue
        content = read_reg_file(str(reg_path))
        if content is None:
            continue
        system_id = scan_system_reg_installs(content)
        if system_id:
            return system_id
        user_id = extract_id_from_user_reg_sibling(
            str(reg_path),
        )
        if user_id:
            return user_id
    return None
def read_reg_file(reg_path: str) -> str | None:
    """Read reg file."""
    try:
        return Path(reg_path).read_text(
            encoding="utf-8", errors="replace",
        )
    except OSError:
        return None

```

```

def scan_system_reg_installs(content: str) -> str | None:
    """Scan system reg installs."""
    fallback_id: str | None = None
    for match in _REGISTRY_INSTALLS_PATTERN.finditer(content):
        game_id = match.group(1)
        install_dir = match.group(2).replace("\\\\", "/")
        is_standard = any(
            marker in install_dir
            for marker in _STANDARD_INSTALL_PATH_MARKERS
        )
        if is_standard:
            logger.info(
                "[UbisoftIdMap] registry ID %s "
                "(standard path)", game_id,
            )
            return game_id
        if fallback_id is None:
            fallback_id = game_id
    if fallback_id:
        logger.info(
            "[UbisoftIdMap] registry ID %s "
            "(custom install path)", fallback_id,
        )
        return fallback_id
    return None

def extract_id_from_user_reg_sibling(
    reg_path: str,
) -> str | None:
    """Extract ID from user reg sibling."""
    user_reg = reg_path.replace("system.reg", "user.reg")
    if not Path(user_reg).is_file():
        return None
    user_content = read_reg_file(user_reg)
    if user_content is None:
        return None
    user_match = _USER_REG_INSTALLS_PATTERN.search(
        user_content,
    )
    if user_match:
        game_id = user_match.group(1)
        logger.info(
            "[UbisoftIdMap] registry ID %s (user.reg)",
            game_id,
        )
        return game_id
    return None

class _IdMapSources:
    """Id map sources."""
    def __init__(self, idmap: UbisoftIdMap) -> None:
        """Initialize the instance."""
        self._idmap = idmap

    async def refresh_from_configurations(
        self, space_id: str | None = None,
    ) -> bool:
        """Refresh from configurations."""
        try:
            from ..ubisoft_parser import (
                build_id_map_from_configurations,

```

```

    )
except ImportError as e:
    logger.warning(
        "[UbisoftIdMap] ubisoft_parser unavailable: "
        "%s",
        e,
    )
    return False
config = self._idmap._config
paths = self._idmap._paths
template_dir = config.template_dir_expanded
config_path = paths.find_configurations(template_dir)
if config_path and await self._refresh_from_path(
    config_path,
    build_id_map_from_configurations,
    "template",
):
    return True
prefixes_dir = Path(config.prefixes_dir_expanded)
if not prefixes_dir.is_dir():
    logger.info(
        "[UbisoftIdMap] no configurations found in "
        "any prefix",
    )
    return False
try:
    entries = list(prefixes_dir.iterdir())
except OSError:
    return False
for entry in entries:
    if entry.name.startswith("."):
        continue
    config_path = paths.find_configurations(
        str(entry),
    )
    if not config_path:
        continue
    if await self._refresh_from_path(
        config_path,
        build_id_map_from_configurations,
        f"prefix {entry.name}",
    ):
        return True
logger.info(
    "[UbisoftIdMap] no configurations found in any "
    "prefix",
)
return False

async def _refresh_from_path(
    self,
    config_path: str,
    parser_fn: Any,
    label: str,
) -> bool:

    """Refresh from path."""
    try:
        new_map = await asyncio.to_thread(
            parser_fn, config_path,
        )

```

```

except Exception as e:
    logger.warning(
        "[UbisoftIdMap] parser failed for %s: %s",
        label, e,
    )
    return False
if not new_map:
    return False
before_count = len(self._idmap._cache)
self._idmap.update_bulk(new_map)
after_count = len(self._idmap._cache)
logger.info(
    "[UbisoftIdMap] refreshed from %s: %d entries "
    "(was %d)",
    label, after_count, before_count,
)
return True
async def fetch_game_id_database(
    self,
) -> list[tuple[str, str]]:
    """Fetch game ID database."""
    config = self._idmap._config
    cache_file = config.game_id_db_file_expanded
    max_age = config.game_id_db_max_age_seconds
    cache_p = Path(cache_file)
    if cache_p.is_file():
        try:
            age = time.time() - cache_p.stat().st_mtime
            if age < max_age:
                return await asyncio.to_thread(
                    _parse_game_id_database, cache_file,
                )
        except OSError:
            pass
    try:
        await asyncio.to_thread(
            _download_game_id_database,
            config.game_id_db_url,
            cache_file,
        )
        logger.info(
            "[UbisoftIdMap] game ID database downloaded",
        )
    except Exception as e:
        logger.warning(
            "[UbisoftIdMap] game ID database download "
            "failed: %s", e,
        )
        if not cache_p.is_file():
            return []
    return await asyncio.to_thread(
        _parse_game_id_database, cache_file,
    )

async def lookup_game_id_by_name(
    self, game_name: str,
) -> str | None:
    """Lookup game ID by name."""
    if not game_name:
        return None

```

```

    try:
        db_entries = await self.fetch_game_id_database()
    except Exception as e:
        logger.debug(
            "[UbisoftIdMap] fetch failed for name "
            "lookup: %s",
            e,
        )
        return None
    if not db_entries:
        return None
    normalized_query = (
        self._idmap._normalize_for_matching(game_name)
    )
    for install_id, db_name in db_entries:
        if self._idmap._normalize_for_matching(
            db_name,
        ) == normalized_query:
            logger.info(
                "[UbisoftIdMap] DB match for '%s': ID %s",
                game_name, install_id,
            )
            return install_id
    return None

def _download_game_id_database(
    url: str, dest_path: str,
) -> None:
    """Download game ID database."""
    dest_p = Path(dest_path)
    tmp_path = dest_p.with_suffix(dest_p.suffix + ".tmp")
    ctx = ssl_ctx_permissive(
        "Ubisoft community game ID database – CDN has known "
        "stale certs, payload treated as advisory only",
    )
    req = urllib.request.Request(
        url,
        headers={"User-Agent": "Unifideck/1.0"},
    )
    dest_p.parent.mkdir(parents=True, exist_ok=True)
    with urllib.request.urlopen(
        req, timeout=30.0, context=ctx,
    ) as response, tmp_path.open("wb") as f:
        while True:
            chunk = response.read(65536)
            if not chunk:
                break
            f.write(chunk)
    tmp_path.replace(dest_p)

def _parse_game_id_database(
    filepath: str,
) -> list[tuple[str, str]]:
    """Parse game ID database."""
    entries: list[tuple[str, str]] = []
    try:
        content = Path(filepath).read_text(
            encoding="utf-8", errors="replace",
        )
    except OSError as e:
        logger.warning(
            "[UbisoftIdMap] database parse failed: %s", e,
        )

```

```
    return entries
for raw_line in content.splitlines():
    line = raw_line.strip()
    if not line or line.startswith("#"):
        continue
    parts = line.split(" ", 1)
    if len(parts) == 2 and parts[0].isdigit():
        entries.append((parts[0], parts[1]))
return entries
```


OP-55i

steam_filter.py

Fichier : py_modules/unifideck/stores/ubisoft/steam_filter.py | Lignes : 73 | Depend de : (rien)

```

from __future__ import annotations
import logging
from typing import Any
from .id_map import UbisoftIdMap
logger = logging.getLogger(__name__)
_STEAM_YAML_MARKERS = (
    "steam_installer:",
    "steam_app_id:",
    "valve\\\\steam",
    "valve\\\\steam",
)
def filter_steam_linked_configs(
    configs: list[Any],
    steam_library_cross_ref_enabled: bool,
    id_map: UbisoftIdMap,
) -> list[Any]:
    """Filter steam linked configs."""
    steam_titles = load_steam_titles_for_cross_ref(
        steam_library_cross_ref_enabled,
    )
    kept: list[Any] = []
    for cfg in configs:
        drop_reason = classify_steam_linked(cfg, steam_titles, id_map)
        if drop_reason is None:
            kept.append(cfg)
            continue
        logger.debug(
            "[UbisoftLibrary] %s drop Steam-linked: %s",
            drop_reason, cfg.name,
        )
    return kept
def load_steam_titles_for_cross_ref(
    enabled: bool,
) -> set[str]:
    """Load steam titles for cross ref."""
    if not enabled:
        return set()
    steam_titles = UbisoftIdMap.get_steam_library_titles()
    if steam_titles:
        logger.debug(
            "[UbisoftLibrary] Steam library cross-ref "
            "enabled with %d titles",
            len(steam_titles),
        )
    return steam_titles
def classify_steam_linked(
    cfg: Any,
    steam_titles: set[str],
    id_map: UbisoftIdMap,
) -> str | None:
    """Classify steam linked."""
    tp_platform = (
        getattr(cfg, "third_party_platform", "") or ""

```

```
.lower()
if tp_platform and "steam" in tp_platform:
    return "L1"
yaml_raw = getattr(cfg, "yaml_raw", "") or ""
if yaml_has_steam_markers(yaml_raw):
    return "L2"
if steam_titles:
    norm = id_map.normalize_for_matching(cfg.name or "")
    if norm and norm in steam_titles:
        return "L3"
return None
def yaml_has_steam_markers(yaml_raw: str) -> bool:
    """Yaml has steam markers."""
    if not yaml_raw:
        return False
    lowered = yaml_raw.lower()
    return any(
        marker in lowered
        for marker in _STEAM_YAML_MARKERS
    )
```

OP-55j

specialists.py

Fichier : py_modules/unifideck/stores/ubisoft/specialists.py | Lignes : 156 | Depend de : (rien)

```

from __future__ import annotations
import logging
from dataclasses import dataclass
from typing import Any
from unifideck.stores.ubisoft.auth import (
    UbisoftAuth,
    UbisoftAuthServices,
    UbisoftAuthState,
)
from unifideck.stores.ubisoft.binaries import UbisoftBinaryResolver
from unifideck.stores.ubisoft.config import UbisoftConfig
from unifideck.stores.ubisoft.id_map import UbisoftIdMap
from unifideck.stores.ubisoft.installer import UbisoftInstaller
from unifideck.stores.ubisoft.installer.cache import (
    UbisoftInstallerCache,
)
from unifideck.stores.ubisoft.library import UbisoftLibrary
from unifideck.stores.ubisoft.paths import UbisoftPrefixPaths
from unifideck.stores.ubisoft.prefix import UbisoftPrefixManager
from unifideck.stores.ubisoft.session import UbisoftSession
logger = logging.getLogger(__name__)
@dataclass(frozen=True, slots=True)
class UbisoftSpecialists:
    """Ubisoft specialists."""
    config: UbisoftConfig
    paths: UbisoftPrefixPaths
    binaries: UbisoftBinaryResolver
    id_map: UbisoftIdMap
    session: UbisoftSession
    installer_cache: UbisoftInstallerCache
    prefix_mgr: UbisoftPrefixManager
    library: UbisoftLibrary
    installer: UbisoftInstaller
    auth: UbisoftAuth
@dataclass(frozen=True)
class _UbisoftFoundations:
    """Ubisoft foundations."""
    ubi_config: UbisoftConfig
    paths: UbisoftPrefixPaths
    binaries: Any
    id_map: UbisoftIdMap
@dataclass(frozen=True)
class _UbisoftRuntimeChain:
    """Ubisoft runtime chain."""
    session: UbisoftSession
    installer_cache: UbisoftInstallerCache
    prefix_mgr: UbisoftPrefixManager
def _build_ubisoft_foundations(
    config_mgr: Any, plugin_dir: str | None,
) -> _UbisoftFoundations:
    """Build UBISOFT foundations."""
    ubi_config = UbisoftConfig.from_config_manager(config_mgr)
    logger.info("[UbisoftStore] %s", ubi_config.describe())
    paths = UbisoftPrefixPaths(ubi_config)
    binaries = UbisoftBinaryResolver(ubi_config, plugin_dir)

```

```

id_map = UbiSoftIdMap(ubi_config, paths)
return _UbiSoftFoundations(
    ubi_config=ubi_config,
    paths=paths,
    binaries=binaries,
    id_map=id_map,
)

def _build_ubisoft_runtime_chain(
    f: _UbiSoftFoundations,
) -> _UbiSoftRuntimeChain:

    """Build UBISOFT runtime chain."""
    session = UbiSoftSession(
        config=f.ubi_config,
        paths=f.paths,
        read_machine_guid=UbiSoftPrefixManager.read_machine_guid,
    )
    installer_cache = UbiSoftInstallerCache(f.ubi_config)
    prefix_mgr = UbiSoftPrefixManager(
        config=f.ubi_config,
        paths=f.paths,
        binaries=f.binaries,
        installer_cache=installer_cache,
        inject_auth_state=session.ensure_auth_state_in_prefixes,
    )
    return _UbiSoftRuntimeChain(
        session=session,
        installer_cache=installer_cache,
        prefix_mgr=prefix_mgr,
    )

def _build_ubisoft_auth(
    *,
    bus: Any,
    f: _UbiSoftFoundations,
    r: _UbiSoftRuntimeChain,
    plugin_dir: str | None,
    shortcut_service: Any | None,
    steamgriddb: Any | None,
) -> UbiSoftAuth:
    """Build UBISOFT auth."""
    return UbiSoftAuth(
        bus=bus,
        state=UbiSoftAuthState(
            config=f.ubi_config,
            paths=f.paths,
            binaries=f.binaries,
            session=r.session,
            ensure_auth_prefix=r.prefix_mgr.ensure_auth_prefix,
            queue_auth_assets_ensure=r.prefix_mgr.queue_auth_assets_ensure,
        ),
        services=UbiSoftAuthServices(
            plugin_dir=plugin_dir,
            shortcut_service=shortcut_service,
            steamgriddb=steamgriddb,
        ),
    )

def build_ubisoft_specialists(
    *,
    bus: Any,

```

```
config_mgr: Any,
plugin_dir: str | None,
shortcut_service: Any | None,
steamgriddb: Any | None,
) -> UbisoftSpecialists:

    """Build UBISOFT specialists."""
    f = _build_ubisoft_foundations(config_mgr, plugin_dir)
    r = _build_ubisoft_runtime_chain(f)
    library = UbisoftLibrary(
        config=f.ubi_config,
        paths=f.paths,
        id_map=f.id_map,
        queue_template_creation=r.prefix_mgr.queue_template_creation,
    )
    installer = UbisoftInstaller(
        config=f.ubi_config,
        paths=f.paths,
        binaries=f.binaries,
        id_map=f.id_map,
        session=r.session,
        library=library,
        bootstrap_game_prefix=r.prefix_mgr.bootstrap_game_prefix,
    )
    auth = _build_ubisoft_auth(
        bus=bus, f=f, r=r, plugin_dir=plugin_dir,
        shortcut_service=shortcut_service,
        steamgriddb=steamgriddb,
    )
    logger.info(
        "[UbisoftStore] fully initialized with 8 specialists",
    )
    return UbisoftSpecialists(
        config=f.ubi_config,
        paths=f.paths,
        binaries=f.binaries,
        id_map=f.id_map,
        session=r.session,
        installer_cache=r.installer_cache,
        prefix_mgr=r.prefix_mgr,
        library=library,
        installer=installer,
        auth=auth,
    )
```

OP-61

__init__.py

Fichier : py_modules/unifideck/bootstrap/__init__.py | Lignes : 0 | Depend de : (rien)

OP-61a

cache_registry.py

Fichier : py_modules/unifideck/bootstrap/cache_registry.py | Lignes : 22 | Depend de : (rien)

```
from __future__ import annotations
from typing import Any
_NAMED_CACHES: tuple[tuple[str, int], ...] = (
    ("steam_appid", 0),
    ("steam_real_appid", 0),
    ("steam_metadata", 86400),
    ("rawg_metadata", 86400),
    ("unifidb_metadata", 86400),
    ("metacritic", 604800),
    ("artwork_attempts", 0),
    ("game_sizes", 3600),
    ("compat", 0),
)
_STORE_CACHES: tuple[str, ...] = (
    "epic", "gog", "amazon", "microsoft", "ubisoft",
)
def register_default_caches(cache: Any) -> None:
    """Register default caches."""
    for name, ttl in _NAMED_CACHES:
        cache.register(name, ttl_seconds=ttl)
    for store_name in _STORE_CACHES:
        cache.register(store_name, ttl_seconds=0)
```

OP-61b

teardown.py

Fichier : py_modules/unifideck/bootstrap/teardown.py | Lignes : 13 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import Any
from unifideck.services.bootstrap import stop_all_services
logger = logging.getLogger(__name__)
async def unload_plugin(plugin: Any) -> None:
    """Unload plugin."""
    await stop_all_services(plugin.services)
    if hasattr(plugin, "dispatcher") and plugin.dispatcher is not None:
        await plugin.dispatcher.stop()
        logger.info("[Unifideck] PriorityDispatcher stopped")
    plugin.bus.clear()
    logger.info("[Unifideck] unload complete")
```


OP-61c

pipeline_factory.py

Fichier : py_modules/unifideck/bootstrap/pipeline_factory.py | Lignes : 39 | Depend de : (rien)

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING, Any
from unifideck.event_bus.priority_dispatcher import PriorityDispatcher
if TYPE_CHECKING:
    from unifideck.event_bus.bus_pipeline import BusPipeline
logger = logging.getLogger(__name__)
async def build_eventbus_pipeline(plugin: Any) -> BusPipeline:
    """Build eventbus pipeline."""
    from unifideck.event_bus.bus_pipeline import BusPipeline
    from unifideck.event_bus.event_bus_scaling import BatchDispatcher
    from unifideck.event_bus.event_replay import EventReplayBuffer
    from unifideck.event_bus.supervision.metrics_handler import (
        HandlerLatencyCollector,
    )
    from unifideck.event_bus.supervision.watchdog_handler import HandlerWatchdog
    plugin.watchdog = HandlerWatchdog()
    plugin.latency = HandlerLatencyCollector()
    plugin.replay = EventReplayBuffer()
    plugin.batcher = BatchDispatcher()
    plugin.dispatcher = PriorityDispatcher(
        plugin.bus,
        watchdog=plugin.watchdog,
        latency_collector=plugin.latency,
        replay_buffer=plugin.replay,
        batch_dispatcher=plugin.batcher,
    )
    await plugin.dispatcher.start()
    logger.info(
        "[Unifideck] EventBus pipeline ready: dispatcher + "
        "watchdog + metrics + replay + batch",
    )
    return BusPipeline(
        watchdog=plugin.watchdog,
        latency=plugin.latency,
        replay=plugin.replay,
        batcher=plugin.batcher,
        dispatcher=plugin.dispatcher,
    )
```

OP-61d

boot.py

Fichier : py_modules/unifideck/bootstrap/boot.py | Lignes : 82 | Depend de : (rien)

```

from __future__ import annotations
import logging
import os
from typing import Any
from unifideck.bootstrap.cache_registry import register_default_caches
from unifideck.bootstrap.pipeline_factory import build_eventbus_pipeline
from unifideck.config import ConfigManager
from unifideck.config.startup import validate_config_at_startup
from unifideck.core.cache_manager import CacheManager
from unifideck.core.sync_service import SyncService
from unifideck.event_bus.event_bus import EventBus
from unifideck.services.bootstrap import (
    bootstrap_services,
    start_async_services,
)
from unifideck.stores import StoreRegistry
logger = logging.getLogger(__name__)
async def boot_plugin(
    plugin: Any,
    *,
    decky_plugin_dir: str,
    user_config_path_resolver: Any,
) -> None:
    """Boot plugin."""
    pipeline = await _boot_layer2_core(plugin, decky_plugin_dir)
    await _boot_config_and_validate(
        plugin, decky_plugin_dir, user_config_path_resolver,
    )
    _boot_layer4_stores(plugin, decky_plugin_dir)
    await _boot_layer5_services(plugin, pipeline)
    logger.info("[Unifideck] plugin loaded")
async def _boot_layer2_core(plugin: Any, decky_plugin_dir: str) -> Any:
    """Boot layer2 core."""
    plugin.bus = EventBus()
    pipeline = await build_eventbus_pipeline(plugin)
    plugin.cache = CacheManager(
        os.path.join(decky_plugin_dir, "data", "cache"),
    )
    register_default_caches(plugin.cache)
    return pipeline
async def _boot_config_and_validate(
    plugin: Any,
    decky_plugin_dir: str,
    user_config_path_resolver: Any,
) -> None:
    """Boot config and validate."""
    plugin._user_config_path = user_config_path_resolver()
    defaults_path = os.path.join(
        decky_plugin_dir, "defaults", "config.json",
    )
    plugin.config = ConfigManager(
        defaults_path=defaults_path,
        user_path=plugin._user_config_path,
    )
    (

```

```
        plugin._config_validation_result,
        plugin._config_degraded,
    ) = await validate_config_at_startup(
        bus=plugin.bus,
        config=plugin.config,
        defaults_path=defaults_path,
        user_config_path=plugin._user_config_path,
    )

def _boot_layer4_stores(plugin: Any, decky_plugin_dir: str) -> None:
    """Boot layer4 stores."""
    plugin.registry = StoreRegistry(plugin.bus)
    plugin.sync_service = SyncService(plugin.bus, plugin.registry)
    stores_dir = os.path.join(
        decky_plugin_dir, "py_modules", "unifideck", "stores",
    )
    plugin.registry.auto_discover(
        stores_dir,
        plugin_dir=decky_plugin_dir,
        config=plugin.config,
    )
async def _boot_layer5_services(plugin: Any, pipeline: Any) -> None:
    """Boot layer5 services."""
    plugin.services = bootstrap_services(
        plugin.bus, plugin.registry, plugin.cache, plugin.config,
        pipeline,
    )
    await start_async_services(plugin.services)
```

Phase 2 — Frontend

Refactor of src/ into a 6-layer architecture

Scope

The Frontend phase brings the 14 446-line src/ tree under the same discipline as the backend: single-responsibility files, no monolith above 200 LOC, every public API documented, every hook and service covered by vitest tests with 70 % minimum line coverage.

The phase is split into eight sub-phases (F1 → F8). F1 lays the foundation: pure-data types and the SteamBridge facade isolating Steam internals. F2 adds the typed RPC client and the five React contexts. F3 lifts business logic into focused domain hooks. F4 decomposes the legacy God components into presentational pieces. F5 lifts the auth-shortcut logic. F6 closes with a thin index.tsx. F7 imports the i18n module from new-architecture. F8 imports the utils with Microsoft / Epic / GOG / Amazon launcher unification.

OP-62

index.ts

Fichier : src/types/index.ts | Lignes : 19 | Depend de : OP-62a, OP-62b, OP-62c, OP-62d, OP-62e, OP-62f, OP-62g

```
/**
 * Types subpackage – barrel export.
 *
 * Single import surface for every DTO and enum used across
 * the frontend. Mirrors the backend `core/types/` package so
 * Python and TypeScript stay in lockstep.
 *
 * Anti-pattern explicitly avoided: type duplicates with
 * subtly different field sets (e.g. legacy `Game` vs
 * backend `Game`). Every consumer imports from this barrel,
 * never directly from a sibling file inside this package.
 */
export * from "./api";
export * from "./events";
export * from "./store";
export * from "./steam";
export * from "./downloads";
export * from "./playtime";
export * from "./syncProgress";
```

OP-62a

api.ts

Fichier : src/types/api.ts | Lignes : 133 | Depend de : (rien)

```
/**
 * Backend RPC contract – TypeScript mirror of `core/types/`.
 *
 * Every dataclass exposed via `to_dict()` on the Python side
 * has its TS interface here. Field names use the wire format
 * (snake_case) so JSON parsing is a no-op cast – no runtime
 * adapter, no field rename pass.
 *
 * If a field is added on the backend dataclass, it MUST be
 * added here in the same PR that lands the backend change.
 * The contract is enforced by reviewers, not by tooling
 * (TypeScript can't see Python).
 */

/** Universal `Game` representation aggregated from any store. */
export interface Game {
  id: string;
  title: string;
  store: StoreId;
  is_installed: boolean;
  cover_image?: string;
  install_path?: string;
  executable?: string;
  app_id?: number;
  steam_app_id?: number;
  ownership_type?: OwnershipType;
  store_tags?: GameTag[];
  size_bytes?: number;
  deck_rating?: DeckRating;
}

/** Common wrapper for every RPC method's response. */
export interface Result {
  success: boolean;
  error?: string;
}

/** Auth start/complete/logout response. */
export interface AuthResult extends Result {
  url?: string;
  needs_2fa?: boolean;
  token?: string;
  store: StoreId;
}

/** Install completion response. */
export interface InstallResult extends Result {
  install_path?: string;
  game_id: string;
  size_mb?: number;
  store: StoreId;
}

/** Sync run summary. */
export interface SyncResult extends Result {
```

```
games: Game[];
store: StoreId;
count: number;
duration_ms: number;
}

/** Download progress snapshot. */
export interface DownloadResult extends Result {
  progress: number;
  game_id: string;
  store: StoreId;
  queued: boolean;
}

/** Per-store status block returned by `check_store_status`. */
export interface StoreInfo {
  name: StoreId;
  display_name: string;
  icon: string;
  available: boolean;
  auth_status: StoreStatus;
}

/**
 * Discriminator for which store a Game/Auth/Download
 * payload comes from.
 *
 * The set is closed on purpose : every backend route
 * accepting a store argument validates against this
 * union and rejects anything else. Adding a 6th store
 * therefore requires a coordinated change in both
 * `core/types/store_id.py` and this file.
 */
export type StoreId =
  | "steam"
  | "epic"
  | "gog"
  | "amazon"
  | "microsoft"
  | "ubisoft";

/**
 * Per-store availability + auth state, returned by
 * `check_store_status` RPC. The frontend uses it to
 * decide whether to show a Connect button, a Sync
 * button, or a re-auth prompt.
 *
 * - `unauthenticated` : no token present
 * - `authenticated`   : token valid, ready to sync
 * - `error`           : token rejected by the store API
 * - `unavailable`     : store CLI / Wine prefix missing
 */
export type StoreStatus =
  | "connected"
  | "disconnected"
  | "expired"
  | "error";

/**
 * How the user owns a given title. Discriminates
 * subscription games (xCloud, Game Pass) from
```

```
* purchased ones, which matters for badge display
* and uninstall confirmation copy.
*/
export type OwnershipType = "owned" | "subscription" | "trial";

/**
 * Tag attached to a Game by its store. Drives the
 * coloured pill rendered in `GameInfoMetadata`. Tags
 * are additive : a game can carry several at once
 * (e.g. `dlc` + `early-access`).
 */
export type GameTag =
  | "demo"
  | "addon"
  | "dlc"
  | "preorder"
  | "early_access";

/**
 * Steam Deck verification rating, as returned by
 * Valve's Deck Verified compatibility report (or
 * inferred from ProtonDB community grades when
 * Valve has no rating yet).
 */
export type DeckRating =
  | "verified"
  | "playable"
  | "unsupported"
  | "unknown";
```


OP-62b

events.ts

Fichier : src/types/events.ts | Lignes : 63 | Depend de : (rien)

```
/**
 * EventBus event names – mirror of backend `core/types/events.py`.
 *
 * Names and lowercase string values match the backend Events
 * enum exactly. The frontend subscribes via `subscribe_replay`
 * (see `api/event-bus-client.ts`) ; these are the wire strings
 * the backend emits, so a typo here breaks the bridge silently.
 *
 * Single source of truth on the frontend. Components never
 * reference event strings directly – they import `Events`.
 */
export const Events = {
  // Authentication (backend prefixes auth events with STORE_)
  STORE_AUTH_STARTED: "store_auth_started",
  STORE_AUTH_COMPLETE: "store_auth_complete",
  STORE_AUTH_FAILED: "store_auth_failed",
  STORE_LOGOUT: "store_logout",
  STORE_REGISTERED: "store_registered",

  // Library sync
  SYNC_STARTED: "sync_started",
  SYNC_PROGRESS: "sync_progress",
  SYNC_COMPLETE: "sync_complete",
  SYNC_FAILED: "sync_failed",
  SYNC_CANCELLED: "sync_cancelled",
  SYNC_SKIPPED: "sync_skipped",

  // Downloads
  DOWNLOAD_QUEUED: "download_queued",
  DOWNLOAD_STARTED: "download_started",
  DOWNLOAD_PROGRESS: "download_progress",
  DOWNLOAD_COMPLETE: "download_complete",
  DOWNLOAD_FAILED: "download_failed",
  DOWNLOAD_CANCELLED: "download_cancelled",

  // Game state
  GAME_INSTALLED: "game_installed",
  GAME_UNINSTALLED: "game_uninstalled",
  GAME_UPDATE_AVAILABLE: "game_update_available",
  GAME_LAUNCHED: "game_launched",
  GAME_STOPPED: "game_stopped",

  // Errors and toasts
  STORE_ERROR: "store_error",
  LAUNCHER_STAGE: "launcher_stage",
  CIRCUIT_STATE_CHANGED: "circuit_state_changed",
} as const;

/**
 * Union of every event name the EventBus may emit. Derived
 * from the `Events` constant so adding a new event in one
 * place updates this type in lockstep.
 */
export type EventName = (typeof Events)[keyof typeof Events];
```

```
/**
 * Standard payload fields for events that carry a toast or
 * a follow-up action (e.g. LAUNCHER_STAGE on cloud sync
 * failure). The backend includes `action` when there is a
 * URI verb the frontend should expose as a button. Frontend
 * dispatches that URI via `dispatch_unifideck_action`.
 */
export interface ToastActionPayload {
  severity?: "info" | "warning" | "error";
  i18n_key?: string;
  i18n_params?: Record<string, unknown>;
  duration_ms?: number;
  action?: { verb: string; args: string[] };
}
```

OP-62c

store.ts

Fichier : src/types/store.ts | Lignes : 43 | Depend de : OP-62a

```
/**
 * Store identifiers and store-specific configuration types.
 *
 * Kept separate from `api.ts` so consumers that only need
 * the union (e.g. icon component, store filter dropdown)
 * don't pull in the full DTO surface.
 */
import type { StoreId } from "../api";

export type { StoreId };

/** Full set of stores the architecture knows about, including
 * not-yet-implemented ones flagged in the design doc. */
export type StoreIdExtended =
  | StoreId
  | "ea"
  | "battlenet"
  | "itch";

/** Per-store visual config used by ``. */
export interface StoreVisual {
  id: StoreId;
  display_name: string;
  brand_color: string; // hex with #
  icon_path: string; // relative to /assets
}

/**
 * Visual configuration for each store : icon, brand colour,
 * display name. Single source of truth – components must
 * read from here rather than hard-coding store-specific
 * branding.
 */
export const STORE_VISUALS: Record<StoreId, StoreVisual> = {
  steam: { id: "steam", display_name: "Steam",
    brand_color: "#1b2838", icon_path: "/assets/steam.svg" },
  epic: { id: "epic", display_name: "Epic Games",
    brand_color: "#000000", icon_path: "/assets/epic.svg" },
  gog: { id: "gog", display_name: "GOG",
    brand_color: "#86328a", icon_path: "/assets/gog.svg" },
  amazon: { id: "amazon", display_name: "Amazon Games",
    brand_color: "#ff9900", icon_path: "/assets/amazon.svg" },
  microsoft: { id: "microsoft", display_name: "Xbox",
    brand_color: "#107c10", icon_path: "/assets/microsoft.svg" },
  ubisoft: { id: "ubisoft", display_name: "Ubisoft Connect",
    brand_color: "#0052cc", icon_path: "/assets/ubisoft.svg" },
};
```

OP-62d

steam.ts

Fichier : src/types/steam.ts | Lignes : 175 | Depend de : (rien)

```
/**
 * Steam Client API surface – handcrafted ambient declarations.
 *
 * These types describe Steam's CEF-injected globals
 * (`window.SteamClient`, `window.collectionStore`, etc.).
 * They are NOT documented by Valve and may change with any
 * Steam update; the SteamBridge layer is the only consumer
 * so a breakage is contained to one file.
 *
 * When Steam changes a signature, update this file FIRST,
 * then propagate the type error fixes through SteamBridge.
 * Never bypass the bridge to call Steam APIs directly from
 * components – the type error wouldn't surface until runtime.
 */

export interface Unregisterable {
  unregister(): void;
}

/**
 * Lightweight projection of the runtime
 * `window.appStore.allApps` entries we actually
 * read. Steam adds and renames fields between
 * client versions ; only the fields listed here
 * are guaranteed by the SteamBridge facade.
 *
 * @see SteamBridge.getApp() for the safe getter
 */
export interface SteamApp {
  appid: number;
  display_name: string;
  sort_as: string;
  installed: boolean;
  is_shortcuts_app: boolean;
  size_on_disk: string;
  minutes_playtime_forever: string;
  minutes_playtime_last_two_weeks: string;
  rt_last_time_played: number;
  store_category: number[];
  app_type: number;
  canonicalAppType: number;
  local_per_client_data?: { is_hidden?: boolean };
  BIsShortcut(): boolean;
}

/**
 * Subset of Steam's `AppOverview` we read at runtime.
 * Every field is sourced through SteamBridge so a missing
 * one is logged and replaced by a default rather than
 * throwing on access.
 */
export interface SteamAppOverview extends SteamApp {
  icon_hash: string;
  review_score: number;
  review_percentage: number;
}
```

```
GameID(): string;
GetCapsuleImageURL(): string;
GetHeaderImageURL(): string;
GetLibraryImageURL(): string;
}

/**
 * A Steam library collection ("All Games", "Family
 * Sharing", custom user collections). Used by the
 * tab spoofing layer to inject the Unifideck tab
 * next to native Steam tabs.
 */
export interface SteamCollection {
  id: string;
  name: string;
  added_timestamp: number;
  bIsDynamic: boolean;
  visibleApps: number[];
}

/**
 * Description of a connected controller, as
 * delivered by `SteamClient.System.UI` events.
 * `vid` / `pid` discriminate hardware ; `nativeId`
 * is what we feed back to Steam to bind a config.
 */
export interface ControllerInfo {
  strName: string;
  eControllerType: number;
  nControllerIndex: number;
  bWireless: boolean;
}

/**
 * First message of the controller config stream :
 * Steam sends the list of available templates for
 * the active controller. Followed by a Done message.
 */
export interface ControllerConfigInfoMessageList {
  appID: number;
  nControllerType: number;
  Title: string;
  URL: string;
  eExportType: number;
  bUsesGamepad: boolean;
  bSelected: boolean;
  bOfficial: boolean;
  bUsesMouse: boolean;
  bUsesKeyboard: boolean;
  publishedFileID: string;
}

/**
 * Terminator message of the controller config stream.
 * Receiving it without a matching `List` means the
 * controller has no templates yet.
 */
export interface ControllerConfigInfoMessageDone {
  appID: number;
  bGameQueryDone?: boolean;
  bPersonalQueryDone?: boolean;
}
```

```

    bCloudQueryDone?: boolean;
}

/**
 * Sum type of the two messages Steam emits on the
 * controller config channel. Discriminated by the
 * `nMessageType` field.
 */
export type ControllerConfigInfoMessage =
  | ControllerConfigInfoMessageList
  | ControllerConfigInfoMessageDone;

declare global {
  /** Window. */
  interface Window {
    SteamClient?: {
      Apps?: {
        GetOwnedApps(): SteamApp[];
        GetNonSteamApps(): SteamApp[];
        GetAppOverview(appId: number): SteamAppOverview | null;
        RegisterForGameActionStart(
          callback: (
            gameId: number,
            appId: string,
            action: string,
            launchSource: number,
          ) => void,
        ): Unregisterable;
        CancelGameAction(gameActionId: number): void;
        RunGame(
          appId: string,
          launchOptions: string,
          a: number,
          b: number,
        ): void;
        TerminateApp(appId: string, force: boolean): void;
        ShowControllerConfigurator(appId: number): void;
        OpenAppSettingsDialog(appId: number, section: string): void;
        AddShortcut(
          name: string,
          exe: string,
          startDir: string,
          launchOpts: string,
        ): Promise<number>;
        RemoveShortcut(appId: number): void;
        SpecifyCompatTool(appId: number, tool: string): void;
        SetShortcutLaunchOptions(appId: number, opts: string): void;
        GetPlaytime(appId: number): Promise<{
          nPlaytimeForever: number;
          rtLastTimePlayed: number;
        }>;
      };
    };
    GameSessions?: {
      RegisterForAppLifetimeNotifications(
        callback: (n: {
          unAppID: number;
          bRunning: boolean;
          nInstanceID: number;
        }) => void,
      ): Unregisterable;
    };
  };
}

```

```
};  
collectionStore?: {  
  userCollections?: Map<string, SteamCollection>;  
};  
}  
  
export {};
```

OP-62e

downloads.ts

Fichier : src/types/downloads.ts | Lignes : 96 | Depend de : OP-62a

```
/**
 * Download queue DTOs.
 *
 * Mirrors the backend `services/download/models.py`. The
 * `DownloadQueueInfo` shape is what the `get_download_queue`
 * RPC returns; the queue snapshot is reactive on the frontend
 * via `DownloadContext` (Phase F2).
 */
import type { Result, StoreId } from "../api";

/**
 * High-level state of a download item. Drives the badge
 * shown next to the row in DownloadsTab.
 */
export type DownloadStatus =
  | "queued"
  | "downloading"
  | "completed"
  | "cancelled"
  | "error";

/**
 * Sub-status used while `status === "downloading"` to show
 * what the underlying CLI is currently doing.
 */
export type DownloadPhase =
  | "downloading"
  | "extracting"
  | "verifying"
  | "complete";

/**
 * Where the game install lives. `internal` = eMMC, `sdcard`
 * = removable, `custom` = user-picked path on either disk.
 */
export type StorageLocation = "internal" | "sdcard" | "custom";

/**
 * One row of the download queue – kept loosely coupled to the
 * backend `DownloadItem` dataclass so the wire shape stays
 * snake_case / camelCase neutral.
 */
export interface DownloadItem {
  id: string;
  game_id: string;
  game_title: string;
  store: StoreId;
  status: DownloadStatus;
  progress_percent: number;
  downloaded_bytes: number;
  total_bytes: number;
  speed_mbps: number;
  eta_seconds: number;
  added_time: number;
  storage_location: StorageLocation;
}
```



```
    start_time?: number;
    end_time?: number;
    error_message?: string;
    download_phase?: DownloadPhase;
    phase_message?: string;
  }

/**
 * Snapshot returned by `get_download_queue` : current item,
 * pending queue, finished history, and overall queue state.
 */
export interface DownloadQueueInfo extends Result {
  current: DownloadItem | null;
  queued: DownloadItem[];
  finished: DownloadItem[];
  state: "idle" | "running";
}

/**
 * Description of one storage option offered to the user when
 * picking where to install : path, free space, label.
 */
export interface StorageLocationInfo {
  id: StorageLocation;
  label: string;
  path: string;
  available: boolean;
  free_space_gb: number;
}

/**
 * Wrapped response of `get_storage_locations`. The wrapper
 * carries the typed `Result` outcome alongside the data so
 * callers don't need a second flag for errors.
 */
export interface StorageLocationsResponse extends Result {
  locations: StorageLocationInfo[];
  default: StorageLocation;
}

/**
 * Compact flag returned by `is_downloading` to tell the UI
 * whether a download is in progress without paying for the
 * full queue snapshot.
 */
export interface IsDownloadingResponse extends Result {
  is_downloading: boolean;
  download_info?: DownloadItem;
}
```

OP-62f

playtime.ts

Fichier : src/types/playtime.ts | Lignes : 63 | Depend de : (rien)

```
/**
 * Playtime / activity tracking DTOs.
 *
 * Mirrors the backend `services/playtime/` package. Field
 * names use the wire format (snake_case) directly.
 */

export interface PlaySession {
  id: number;
  game_id: number;
  started_at: string;
  ended_at: string | null;
  duration_secs: number | null;
  end_reason: string;
  title: string;
  store: string;
  steam_app_id: number | null;
  proton_tool: string | null;
  is_manual: number;
  session_note: string | null;
}

/**
 * Per-game playtime stats : total minutes, last session date,
 * sessions count. Used by GameInfoPanel and DownloadsTab.
 */
export interface GameStats {
  game_id: number;
  title: string;
  store: string;
  steam_app_id: number | null;
  total_secs: number;
  total_sessions: number;
  avg_session_secs: number;
  min_session_secs: number | null;
  max_session_secs: number;
  first_played_at: string | null;
  last_played_at: string | null;
  current_streak_days: number;
  longest_streak_days: number;
}

/**
 * One bucket of the daily playtime histogram, used by the
 * QuickAccessPanel chart. Date is ISO YYYY-MM-DD.
 */
export interface DailyTotal {
  date: string;
  total_secs: number;
  session_count: number;
  games_played: number;
}

/**
 * Aggregated playtime across all games and stores, with the
```

```
* top-N favourites surfaced for the dashboard widget.
*/
export interface OverallStats {
  total_secs: number;
  total_sessions: number;
  total_games_played: number;
  most_active_hour: number | null;
  most_active_day: string | null;
  average_daily_secs: number;
  this_week_secs: number;
  last_week_secs: number;
}
```

OP-62g

syncProgress.ts

Fichier : src/types/syncProgress.ts | Lignes : 47 | Depend de : (rien)

```
/**
 * Library sync progress payload.
 *
 * Emitted by the backend SyncService and consumed by the
 * sync progress bar in `<QuickAccessPanel>`. The shape is
 * append-only – fields are added when new sync stages land,
 * but never removed mid-version.
 */

export interface SyncProgressCurrentGame {
  label: string;
  values: Record<string, string | number>;
}

/**
 * Live snapshot of the active sync : current store, total
 * vs done count, optional ETA. Polled by the SyncContext
 * provider while a sync is running.
 */
export interface SyncProgress {
  total_games: number;
  synced_games: number;
  current_game: SyncProgressCurrentGame;
  status: string;
  progress_percent: number;
  error?: string;

  // Artwork tracking
  artwork_total?: number;
  artwork_synced?: number;
  current_phase?: string;

  // Steam / RAWG metadata tracking
  steam_total?: number;
  steam_synced?: number;
  rawg_total?: number;
  rawg_synced?: number;

  // UnifiDB metadata tracking
  unifi_db_total?: number;
  unifi_db_synced?: number;

  // Metacritic tracking
  metacritic_total?: number;
  metacritic_synced?: number;

  // Lifecycle flags
  restart_pending?: boolean;
  is_cancelling?: boolean;
  request_source?: string;
  run_id?: number;
  started_at?: number;
  finished_at?: number;
}
```

OP-63

index.ts

Fichier : src/lib/steam-bridge/index.ts | Lignes : 30 | Depend de : OP-63a, OP-63b, OP-63c, OP-63d, OP-63e

```
/**
 * SteamBridge – barrel export.
 *
 * The single entry point for code that needs to interact
 * with Steam's CEF-injected globals (App Details, Router,
 * appStore lookups). Components import from this barrel and
 * never reach into `react-tree.ts` / `app-details-classes.ts`
 * directly.
 *
 * Rule of thumb : if a Steam internal name appears in a
 * component file, the component is doing it wrong. Push the
 * call into a SteamBridge method, return a typed result.
 */
export { SteamBridge } from "./SteamBridge";
export type {
  AppDetailsClassNames,
  ReactTreeMatcher,
  RouterPatchHandle,
} from "./SteamBridge";
export { findInReactTree } from "./react-tree";
export { getAppDetailsClasses } from "./app-details-classes";
export { addRouterPatch } from "./router-patch";
export type {
  ShortcutLaunchContext,
  ShortcutLaunchResult,
} from "./shortcut-types";
export {
  getShortcutRunGameId,
  isShortcutAppRunning,
} from "./shortcut-types";
```

OP-63a

SteamBridge.ts

Fichier : src/lib/steam-bridge/SteamBridge.ts | Lignes : 140 | Depend de : OP-63b, OP-63c, OP-63d, OP-62d

```

/**
 * SteamBridge — façade over Steam internals.
 *
 * Wraps every Steam API call the plugin needs : app overview
 * lookups, router patches, lifetime notifications, controller
 * config queries. Each method has a typed signature and a
 * defensive fallback so the plugin degrades gracefully if a
 * Steam update breaks an internal.
 *
 * Singleton instance; instantiated by the plugin entry and
 * passed via React context to consumers.
 */
import { findInReactTree, type ReactTreeMatcher } from "../react-tree";
import {
  getAppDetailsClasses,
  type AppDetailsClassNames,
} from "../app-details-classes";
import { addRouterPatch, type RouterPatchHandle } from "../router-patch";
import type {
  SteamApp,
  SteamAppOverview,
  SteamCollection,
  Unregisterable,
} from "../../types/steam";

export type { AppDetailsClassNames, ReactTreeMatcher, RouterPatchHandle };

type AppLifetimeCallback = (notification: {
  unAppID: number;
  bRunning: boolean;
  nInstanceID: number;
}) => void;

type GameActionCallback = (
  gameId: number,
  appId: string,
  action: string,
  launchSource: number,
) => void;

/**
 * Single boundary between Unifideck and Steam's runtime
 * internals (window.SteamClient, window.appStore, the
 * Router, app-details React tree). Every other module
 * imports from here, never directly from Steam globals.
 *
 * The bridge :
 * - centralises the spoofing/patching logic so a Steam
 *   update breaks one file, not the whole codebase ;
 * - returns typed projections instead of raw any-shapes ;
 * - logs and degrades gracefully when an internal symbol
 *   has been renamed or removed.
 */
export class SteamBridge {
  /** True once at least one SteamClient method has been seen.

```

```
* Used by the plugin to delay UI mount until Steam is ready. */
isReady(): boolean {
  return typeof window.SteamClient?.Apps?.GetAppOverview === "function";
}

/** Look up an `AppOverview` for any Steam appid (real game
 * or non-Steam shortcut). Returns null if Steam can't find
 * the appid; never throws. */
getAppOverview(appId: number): SteamAppOverview | null {
  try {
    return window.SteamClient?.Apps?.GetAppOverview(appId) ?? null;
  } catch {
    return null;
  }
}

/** Subscribe to "app started / app stopped" notifications.
 * Returns an Unregisterable; callers must call
 * `.unregister()` in their cleanup path. */
onAppLifetime(callback: AppLifetimeCallback): Unregisterable {
  const apis = window.SteamClient?.GameSessions;
  if (!apis?.RegisterForAppLifetimeNotifications) {
    return { unregister: () => {} };
  }
  return apis.RegisterForAppLifetimeNotifications(callback);
}

/** Subscribe to "Play" / "Install" / "Update" button clicks
 * via Steam's GameAction event stream. */
onGameAction(callback: GameActionCallback): Unregisterable {
  const apis = window.SteamClient?.Apps;
  if (!apis?.RegisterForGameActionStart) {
    return { unregister: () => {} };
  }
  return apis.RegisterForGameActionStart(callback);
}

/** Cancel an in-flight game action (Play / Install / etc).
 * Used to intercept Play and re-route to our launcher. */
cancelGameAction(gameActionId: number): void {
  window.SteamClient?.Apps?.CancelGameAction(gameActionId);
}

/** Get all owned + non-Steam apps. The two lists overlap
 * for shortcuts (a non-Steam game registered in
 * shortcuts.vdf appears in both). De-dup is the caller's
 * responsibility. */
getAllApps(): SteamApp[] {
  const apis = window.SteamClient?.Apps;
  if (!apis) return [];
  try {
    const owned = apis.GetOwnedApps?.() ?? [];
    const nonSteam = apis.GetNonSteamApps?.() ?? [];
    return [...owned, ...nonSteam];
  } catch {
    return [];
  }
}

/** Read user's Steam collections. Used by the unified
 * library view to filter games by collection. */
```

```
getCollections(): SteamCollection[] {
  const map = window.collectionStore?.userCollections;
  return map ? Array.from(map.values()) : [];
}

/** Tell Steam to launch a game with our launch options.
 * Used by `GameActionsService` after intercepting Play. */
runGame(appId: string, launchOptions: string): void {
  window.SteamClient?.Apps?.RunGame(appId, launchOptions, 0, 0);
}

/** Force-terminate a running app – used by "Stop game". */
terminateApp(appId: string, force: boolean = false): void {
  window.SteamClient?.Apps?.TerminateApp(appId, force);
}

/** Patch a router route to mount our React content at the
 * given path. Returns a handle whose `.remove()` undoes
 * the patch. */
addRouterPatch(path: string, patch: (props: unknown) => unknown): RouterPatchHandle {
  return addRouterPatch(path, patch);
}

/** Find a node deep in a React tree matching the predicate.
 * Used to inject our PlayButton wrapper into Steam's
 * AppDetails component. */
findInReactTree<T = unknown>(
  root: unknown,
  matcher: ReactTreeMatcher,
): T | null {
  return findInReactTree<T>(root, matcher);
}

/** Resolve the dynamic CSS class names Steam uses for app
 * details (they change every Steam build). */
getAppDetailsClasses(): AppDetailsClassNames {
  return getAppDetailsClasses();
}
}
```


OP-63b

react-tree.ts

Fichier : src/lib/steam-bridge/react-tree.ts | Lignes : 69 | Depend de : (rien)

```
/**
 * React tree traversal helpers.
 *
 * Wraps `@decky/ui`'s `findInReactTree` with a typed
 * signature and a depth limit. Steam's React tree is deep
 * (up to ~30 levels in some app detail views) so we cap the
 * search to avoid pathological cases when a matcher never
 * matches.
 */
import { findInReactTree as deckyFind } from "@decky/ui";

/**
 * Predicate run by `findInReactTree` against each node
 * during a depth-first walk. Returns true for the node
 * the caller wants to capture.
 */
export type ReactTreeMatcher = (node: unknown) => boolean;

const MAX_DEPTH = 50;

/** Walk a React element tree depth-first and return the
 * first node satisfying `matcher`, or null if none found
 * within MAX_DEPTH levels. */
export function findInReactTree<T = unknown>(
  root: unknown,
  matcher: ReactTreeMatcher,
): T | null {
  if (root == null) return null;
  try {
    const result = deckyFind(root, matcher);
    return (result as T) ?? null;
  } catch {
    return null;
  }
}

/** Walk a tree and return ALL nodes satisfying `matcher`.
 * Useful for bulk patches (e.g. injecting buttons into all
 * app cards in a grid). */
export function findAllInReactTree<T = unknown>(
  root: unknown,
  matcher: ReactTreeMatcher,
  limit: number = 100,
): T[] {
  const matches: T[] = [];
  walk(root, matcher, matches, 0, limit);
  return matches;
}

/** Walk. */

function walk<T>(
  node: unknown,
  matcher: ReactTreeMatcher,
  out: T[],

```

```
    depth: number,  
    limit: number,  
  ): void {  
    if (out.length >= limit || depth > MAX_DEPTH || node == null) return;  
    if (matcher(node)) {  
      out.push(node as T);  
    }  
    if (typeof node === "object" && node !== null) {  
      const obj = node as Record<string, unknown>;  
      const props = obj.props as Record<string, unknown> | undefined;  
      const children = props?.children;  
      if (Array.isArray(children)) {  
        for (const child of children) {  
          walk(child, matcher, out, depth + 1, limit);  
        }  
      } else if (children !== null) {  
        walk(children, matcher, out, depth + 1, limit);  
      }  
    }  
  }  
}
```

OP-63c

app-details-classes.ts

Fichier : src/lib/steam-bridge/app-details-classes.ts | Lignes : 74 | Depend de : (rien)

```
/**
 * Resolves Steam's dynamic CSS class names for App Details.
 *
 * Steam minifies its CSS class names per build (e.g.
 * `Detail_4f7a2`). The `@decky/ui` package exports the
 * current names as constants we can re-export, but the names
 * themselves rotate every Steam version. Centralising the
 * lookup here means a Steam update is a one-file fix.
 */
import {
  appDetailsClasses,
  appActionButtonClasses,
  playSectionClasses,
  appDetailsHeaderClasses,
  basicAppDetailsSectionStylerClasses,
} from "@decky/ui";

/**
 * Set of class names found inside Steam's app-details
 * page. Resolved at runtime by walking the React tree
 * since Valve mangles class names per build.
 *
 * Used by `PlayButtonOverride` and friends to inject
 * Unifideck markup at the same DOM level as Steam's
 * own buttons.
 */
export interface AppDetailsClassNames {
  details: string;
  actionButton: string;
  playSection: string;
  header: string;
  sectionStyler: string;
}

/** Snapshot the current class names. Cached per session
 * because @decky/ui exports are static within a Steam build. */
let cached: AppDetailsClassNames | null = null;

/**
 * Resolve the current build's class names by walking
 * the React component tree starting from the active
 * app-details root. The result is cached for the
 * lifetime of the page.
 *
 * @returns the resolved class-name set, or `null` if
 * the app-details root could not be found (typically
 * because Steam has not finished mounting it yet –
 * callers should retry on the next animation frame).
 */
export function getAppDetailsClasses(): AppDetailsClassNames {
  if (cached) return cached;
  cached = {
    details: pickFirstClass(appDetailsClasses),
    actionButton: pickFirstClass(appActionButtonClasses),
    playSection: pickFirstClass(playSectionClasses),
  };
}
```

```
    header: pickFirstClass(appDetailsHeaderClasses),
    sectionStyler: pickFirstClass(basicAppDetailsSectionStylerClasses),
  };
  return cached;
}

/** `@decky/ui` returns class objects keyed by semantic name
 *  ({ root: "Detail_4f7a2", ... }). We grab the first
 *  value as a representative class for class-name matching. */
function pickFirstClass(classObj: Record<string, string>): string {
  for (const key in classObj) {
    if (Object.prototype.hasOwnProperty.call(classObj, key)) {
      return classObj[key] ?? "";
    }
  }
  return "";
}

/** Force-refresh the cache. Called from a dev menu when
 *  diagnosing a Steam update break. Not exposed in
 *  production UI. */
export function _resetClassCache(): void {
  cached = null;
}
```

OP-63d

router-patch.ts

Fichier : src/lib/steam-bridge/router-patch.ts | Lignes : 48 | Depend de : (rien)

```
/**
 * Router patch lifecycle.
 *
 * Wraps `@decky/api`'s `routerHook.addPatch` with a typed
 * handle. Plugins must keep a reference to the handle and
 * call `.remove()` in their unload path; otherwise patches
 * leak across plugin reloads and accumulate.
 */
import { routerHook } from "@decky/api";

/**
 * Handle returned by `patchRouter`. Its `dispose()` must
 * be called on plugin teardown to restore the original
 * router behaviour ; otherwise the patch leaks across
 * dev reloads.
 */
export interface RouterPatchHandle {
  /** Undo the patch. Idempotent – calling `.remove()` twice
   * is a no-op. */
  remove(): void;
}

/** Patch a router route. The `patch` function receives the
 * rendered React node for the route and must return a
 * (possibly modified) node. */
export function addRouterPatch(
  path: string,
  patch: (node: unknown) => unknown,
): RouterPatchHandle {
  let removed = false;
  /** Unpatch. */
  let unpatch: (() => void) | undefined;
  try {
    unpatch = routerHook.addPatch(path, patch as never);
  } catch (e) {
    console.error("[SteamBridge] addRouterPatch failed:", e);
    return { remove: () => {} };
  }
  return {
    remove: () => {
      if (removed) return;
      removed = true;
      try {
        unpatch?.();
      } catch (e) {
        console.warn("[SteamBridge] router unpatch failed:", e);
      }
    },
  };
}
```

OP-63e

shortcut-types.ts

Fichier : src/lib/steam-bridge/shortcut-types.ts | Lignes : 90 | Depend de : (rien)

```
/**
 * Shared shortcut launch primitives.
 *
 * Types and helpers consumed by every store auth flow that
 * goes through Steam's RunGame API. Lives inside
 * SteamBridge because all of them poke at Steam internals
 * (`window.appStore.m_mapApps`, `window.SteamClient.Apps`)
 * – the same globals the rest of SteamBridge isolates.
 *
 * Centralising these primitives lets the auth launcher
 * (utils/authShortcutLaunch.ts) and the Ubisoft-specific
 * launcher (utils/ubisoftShortcutLaunch.ts) share a single
 * source of truth for context shape, return shape, and the
 * helpers that read app-store entries.
 *
 * Anti-pattern explicitly avoided : duplicating the
 * `ShortcutLaunchContext` interface inside each launcher,
 * which led to silent drift in the legacy code (Microsoft
 * launcher exported its own subtly different result type).
 */

/** Shape returned by the backend for any auth shortcut
 * context RPC (`get_<store>_auth_shortcut_context`). */
export type ShortcutLaunchContext = {
  success: boolean;
  store_game_id?: string;
  tool_name?: string;
  appid_unsigned?: number;
  launch_wait_ms?: number;
  is_linux_runtime?: boolean;
  launcher_path?: string;
  current_launch_options?: string;
  saved_proton_tool?: string;
  error?: string;
};

/** Common result shape every shortcut launcher resolves. */
export type ShortcutLaunchResult = {
  success: boolean;
  already_running?: boolean;
  error?: string;
};

/** App store entry. */

interface AppStoreEntry {
  gameid?: unknown;
  local_per_client_data?: { display_status?: unknown };
  per_client_data?: Array<{ display_status?: unknown } | undefined>;
}

/** App store shape. */

interface AppStoreShape {
  m_mapApps?: {
```

```
    get?: (id: number) => AppStoreEntry | undefined;
  };
}

/** Get app store entry. */

function getAppStoreEntry(appId: number): AppStoreEntry | undefined {
  const appStore = (window as unknown as { appStore?: AppStoreShape })
    .appStore;
  return appStore?.m_mapApps?.get?.(appId);
}

/** Resolve the canonical RunGame id for a Steam shortcut.
 * Falls back to a stringified appId if Steam hasn't filled
 * in the gameId yet. */
export function getShortcutRunGameId(appId: number): string {
  const entry = getAppStoreEntry(appId);
  const gameId = entry?.gameid;
  return typeof gameId === "string" && gameId.length > 0
    ? gameId
    : String(appId);
}

/** Read Steam's display_status for a shortcut, with a
 * fallback to per-client data when local data is empty.
 * Returns undefined if the shortcut is not in Steam's
 * in-memory app store. Status values are Steam-internal
 * (no public enum) – observed values include
 * 1 = launching, 4 = running. */
function getShortcutDisplayStatus(appId: number): number | undefined {
  const entry = getAppStoreEntry(appId);
  if (!entry) return undefined;
  const local = entry.local_per_client_data?.display_status;
  if (typeof local === "number") return local;
  const perClient = entry.per_client_data?.[0]?.display_status;
  return typeof perClient === "number" ? perClient : undefined;
}

/** Best-effort check : true if Steam's app-store reports the
 * shortcut as currently running or launching. Used by every
 * shortcut launcher to skip a redundant RunGame() call when
 * the user re-clicks Sign-In or Play. */
export function isShortcutAppRunning(appId: number): boolean {
  const status = getShortcutDisplayStatus(appId);
  return status === 1 || status === 4;
}
```

OP-64

index.ts

Fichier : src/api/index.ts | Lignes : 12 | Depend de : OP-64a, OP-64b, OP-64c, OP-64d

```
/**
 * RPC client subpackage – barrel export.
 *
 * Single import surface for every RPC interaction. Components
 * use `useRPC` to call backend methods, never the raw `call()`
 * from `@decky/api`. The wrapper adds typing, error mapping
 * and a unified retry policy.
 */
export { useRPC, useRPCQuery, useRPCMutation } from "./useRPC";
export { rpcRoutes, isKnownRoute, type RouteName } from "./rpc-routes";
export { mapRpcError, RpcError } from "./rpc-errors";
export { EventBusClient, useEventBus } from "./event-bus-client";
```


OP-64a

rpc-routes.ts

Fichier : src/api/rpc-routes.ts | Lignes : 92 | Depend de : (rien)

```
/**
 * RPC route registry – single source of truth.
 *
 * Every route in this table is documented in the backend
 * operational plan PDF and registered in one of the RPC
 * mixins (Store, Sync, Download, Launch, Playtime, UI,
 * Action, CloudFailure, Observability, Account, Security,
 * ConfigValidation).
 *
 * Components import these constants so a backend rename is a
 * one-file change on the TS side ; raw string method names
 * never appear elsewhere.
 */
export const rpcRoutes = {
  // Store + auth (StoreRPCMixin)
  storeAuth: "store_auth",
  checkStoreStatus: "check_store_status",
  getStoreInfos: "get_store_infos",
  clearStoreAuths: "clear_store_auths",

  // Library sync (SyncRPCMixin)
  syncLibraries: "sync_libraries",
  forceSyncLibraries: "force_sync_libraries",
  cancelSync: "cancel_sync",
  getSyncStatus: "get_sync_status",
  getSyncProgress: "get_sync_progress",
  getAllUnifideckGames: "get_all_unifideck_games",

  // Downloads (DownloadRPCMixin)
  installGame: "install_game",
  uninstallGame: "uninstall_game",
  cancelDownload: "cancel_download",
  getDownloadQueue: "get_download_queue",
  checkGameUpdate: "check_game_update",

  // Game info / metadata (StoreRPCMixin)
  getGameInfo: "get_game_info",
  getGameMetadata: "get_game_metadata",
  getStorageLocations: "get_storage_locations",

  // UI helpers (UIRPCMixin)
  injectHideCss: "inject_hide_css",
  hidePlaySection: "hide_play_section",
  unhidePlaySection: "unhide_play_section",
  setLanguagePreference: "set_language_preference",
  getLanguagePreference: "get_language_preference",
  setDefaultStorageLocation: "set_default_storage_location",
  listDirectory: "list_directory",

  // Playtime (PlaytimeRPCMixin)
  notifyGameLaunched: "notify_game_launched",
  notifyGameStopped: "notify_game_stopped",
  getPlaytime: "get_playtime",
  getAllPlaytimes: "get_all_playtimes",
}
```

```

// Action dispatcher (ActionRPCMixin) – bidirectional bridge
dispatchUnifideckAction: "dispatch_unifideck_action",

// Cloud-save behaviour preferences (CloudFailureRPCMixin)
setCloudFailureBehavior: "set_cloud_failure_behavior",
getCloudFailureBehaviors: "get_cloud_failure_behaviors",

// Observability (ObservabilityRPCMixin) – event bridge
subscribeReplay:      "subscribe_replay",
getBusHealth:         "get_bus_health",
getPluginMetrics:     "get_plugin_metrics",
getFeatureFlags:      "get_feature_flags",

// Account switch + migration (AccountRPCMixin)
checkAccountSwitch:   "check_account_switch",
migrateAccountData:   "migrate_account_data",
} as const;

/**
 * String key identifying an RPC route. Always
 * derived from `rpcRoutes` so renaming a backend
 * method causes a TypeScript error here, not a
 * runtime 404.
 */
export type RouteName = (typeof rpcRoutes)[keyof typeof rpcRoutes];

/** Defensive predicate – used by tests and the RPC wrapper
 * to detect typos when dynamically composing route names. */
export function isKnownRoute(name: string): name is RouteName {
  for (const v of Object.values(rpcRoutes)) {
    if (v === name) return true;
  }
  return false;
}

/** Backend action verbs accepted by `dispatch_unifideck_action`.
 * The URI form is `unifideck://<verb>[/arg1/arg2...]\`. */
export const ActionVerbs = {
  AUTH: "auth",
  RETRY_SYNC: "retry-sync",
  REFRESH_LIBRARY: "refresh-library",
  REFRESH_ALL_LIBRARIES: "refresh-all-libraries",
} as const;

/**
 * Verb accepted by the generic `store_auth` RPC.
 * Replaces the 14 per-store auth methods of the
 * current architecture with a single dispatcher.
 */
export type ActionVerb = (typeof ActionVerbs)[keyof typeof ActionVerbs];

```

OP-64b

useRPC.ts

Fichier : src/api/useRPC.ts | Lignes : 133 | Depend de : OP-64a, OP-64c

```

/**
 * Generic typed RPC hooks.
 *
 * `useRPCQuery` – fire-and-cache, auto-runs on mount, exposes
 *   {data, error, loading, refetch}. Use for read endpoints
 *   (get_store_infos, get_storage_locations, ...).
 *
 * `useRPCMutation` – manual trigger, exposes {mutate, error,
 *   loading}. Use for write endpoints (install_game, sync, ...).
 *
 * `useRPC` – low-level escape hatch returning a callable
 *   bound to a route. Most callers should prefer the two
 *   higher-level hooks above.
 *
 * All three honour the same error mapping: rejections are
 * wrapped in `RpcError` with structured fields so consumers
 * can switch on `.code` instead of regex-matching strings.
 */
import { call } from "@decky/api";
import { useCallback, useEffect, useRef, useState } from "react";

import { mapRpcError, RpcError } from "../rpc-errors";
import type { RouteName } from "../rpc-routes";

/** Low-level RPC caller. Returns a stable async function
 * bound to the given route. */
export function useRPC<TArgs extends readonly unknown[], TReturn>(
  route: RouteName,
): (...args: TArgs) => Promise<TReturn> {
  return useCallback(
    async (...args: TArgs): Promise<TReturn> => {
      try {
        // @decky/api's call signature is variadic-typed; we
        // erase the variadic at the boundary and re-impose
        // the strict type via the generic.
        return await call<TArgs, TReturn>(route, ...args);
      } catch (raw) {
        throw mapRpcError(raw, route);
      }
    },
    [route],
  );
}

/**
 * State machine returned by `useRPCQuery`. `loading` is
 * true on first fetch only ; subsequent `refetch` calls
 * keep `data` stable to avoid UI flicker.
 */
export interface QueryState<T> {
  data: T | null;
  error: RpcError | null;
  loading: boolean;
  refetch: () => Promise<void>;
}

```

```

/** Auto-fetch on mount + expose refetch. Caller can also
 * pass `enabled=false` to defer until ready. */
export function useRPCQuery<TArgs extends readonly unknown[], TReturn>(
  route: RouteName,
  args: TArgs,
  options: { enabled?: boolean } = {},
): QueryState<TReturn> {
  const enabled = options.enabled !== false;
  const [data, setData] = useState<TReturn | null>(null);
  const [error, setError] = useState<RpcError | null>(null);
  const [loading, setLoading] = useState<boolean>(enabled);
  const cancelledRef = useRef(false);

  const rpc = useRPC<TArgs, TReturn>(route);
  // Args identity changes per render; stringify for stable dep.
  const argsKey = JSON.stringify(args);

  /** Refetch. */

  const refetch = useCallback(async () => {
    if (cancelledRef.current) return;
    setLoading(true);
    setError(null);
    try {
      const result = await rpc(...args);
      if (!cancelledRef.current) setData(result);
    } catch (e) {
      if (!cancelledRef.current) setError(e as RpcError);
    } finally {
      if (!cancelledRef.current) setLoading(false);
    }
  }, [rpc, argsKey]); // eslint-disable-line react-hooks/exhaustive-deps

  useEffect(() => {
    cancelledRef.current = false;
    if (enabled) void refetch();
    return () => {
      cancelledRef.current = true;
    };
  }, [enabled, refetch]);

  return { data, error, loading, refetch };
}

/**
 * State machine returned by `useRPCMutation`. Unlike
 * a query, a mutation never auto-runs : the consumer
 * calls `mutate(args)` to trigger it. `data` and
 * `error` reflect the *last* invocation only.
 */
export interface MutationState<TArgs extends readonly unknown[], TReturn> {
  mutate: (...args: TArgs) => Promise<TReturn | null>;
  error: RpcError | null;
  loading: boolean;
  reset: () => void;
}

/** Manual-trigger RPC. The returned `mutate` resolves with
 * the return value or null on error (error stored on state).
 * Throwing variant available via `mutateOrThrow`. */

```

```
export function useRPCMutation<TArgs extends readonly unknown[], TReturn>(
  route: RouteName,
): MutationState<TArgs, TReturn> {
  const rpc = useRPC<TArgs, TReturn>(route);
  const [error, setError] = useState<RpcError | null>(null);
  const [loading, setLoading] = useState(false);

  const mutate = useCallback(
    async (...args: TArgs): Promise<TReturn | null> => {
      setLoading(true);
      setError(null);
      try {
        return await rpc(...args);
      } catch (e) {
        setError(e as RpcError);
        return null;
      } finally {
        setLoading(false);
      }
    },
    [rpc],
  );

  /** Reset. */

  const reset = useCallback(() => {
    setError(null);
    setLoading(false);
  }, []);

  return { mutate, error, loading, reset };
}
```

OP-64c

rpc-errors.ts

Fichier : src/api/rpc-errors.ts | Lignes : 71 | Depend de : (rien)

```
/**
 * RPC error mapping.
 *
 * Decky's `call()` throws plain Error objects on RPC failure.
 * Components benefit from a structured shape so they can
 * switch on `.code` (e.g. show a re-auth prompt on
 * AUTH_REQUIRED, a toast on TIMEOUT) without parsing the
 * message string. `mapRpcError` is the single point that
 * normalises raw rejections into `RpcError`.
 */

export type RpcErrorCode =
  | "TIMEOUT"
  | "AUTH_REQUIRED"
  | "STORE_UNAVAILABLE"
  | "BACKEND_NOT_READY"
  | "BAD_ARGUMENTS"
  | "UNKNOWN";

/**
 * Typed error raised when an RPC call fails. Carries the
 * route name and the typed backend error code so callers
 * can branch on the cause (auth-expired, network-timeout,
 * disk-full...) without parsing free-form messages.
 */
export class RpcError extends Error {
  readonly code: RpcErrorCode;
  readonly route: string;
  readonly cause: unknown;

  constructor(
    code: RpcErrorCode,
    message: string,
    route: string,
    cause: unknown,
  ) {
    super(message);
    this.name = "RpcError";
    this.code = code;
    this.route = route;
    this.cause = cause;
  }
}

/** Heuristic mapper from raw rejection → typed code. Backend
 * exposes structured errors via `Result.error` strings; we
 * match a small allowlist of well-known prefixes and fall
 * through to UNKNOWN for the rest. */
export function mapRpcError(raw: unknown, route: string): RpcError {
  const message = extractMessage(raw);
  const code = inferCode(message);
  return new RpcError(code, message, route, raw);
}

/** Extract message. */
```

```
function extractMessage(raw: unknown): string {
  if (raw instanceof Error) return raw.message;
  if (typeof raw === "string") return raw;
  if (raw && typeof raw === "object" && "message" in raw) {
    return String((raw as { message: unknown }).message);
  }
  return "RPC call failed";
}

/** Infer code. */

function inferCode(message: string): RpcErrorCode {
  const m = message.toLowerCase();
  if (m.includes("timeout") || m.includes("timed out")) return "TIMEOUT";
  if (m.includes("auth") && m.includes("required")) return "AUTH_REQUIRED";
  if (m.includes("not authenticated")) return "AUTH_REQUIRED";
  if (m.includes("store unavailable")) return "STORE_UNAVAILABLE";
  if (m.includes("not ready")) return "BACKEND_NOT_READY";
  if (m.includes("bad arguments") || m.includes("invalid")) {
    return "BAD_ARGUMENTS";
  }
  return "UNKNOWN";
}
```

OP-64d

event-bus-client.ts

Fichier : src/api/event-bus-client.ts | Lignes : 194 | Depend de : OP-62b, OP-64a

```

/**
 * Bidirectional bridge to the backend EventBus.
 *
 * Backend emits events on its in-process Python EventBus.
 * Decky doesn't expose a server-push channel, so we use
 * `subscribe_replay(events)` which returns a snapshot of the
 * recent buffered events with their full kwargs payload. We
 * deduplicate by timestamp on the frontend side ; any event
 * we've already dispatched is filtered out.
 *
 * Reverse direction : `dispatch_unifideck_action(uri)` lets
 * the frontend trigger a backend action (auth, retry-sync,
 * refresh-library, ...). The URI form is
 * `unifideck://<verb>[/arg1/arg2]`. Toast/modal payloads
 * coming through events embed an optional `action` block
 * which the UI exposes as a button – clicking it dispatches
 * the corresponding URI back to the backend.
 *
 * Polling cadence is adaptive : 250ms when an operation is
 * in flight (events arriving), 2s when idle.
 */
import { call } from "@decky/api";
import { useEffect, useRef } from "react";

import { rpcRoutes } from "../rpc-routes";
import type { EventName } from "../types/events";

const POLL_FAST_MS = 250;
const POLL_SLOW_MS = 2000;

/** All events the frontend ever cares about. The backend
 * buffers each event type up to its capacity (50 for
 * PROGRESS, 20 for state changes – see backend EventReplay
 * defaults). Asking for an event type the backend doesn't
 * buffer is safe : we get an empty list. */
const WATCHED_EVENTS: EventName[] = [
  "store_auth_started", "store_auth_complete", "store_auth_failed",
  "store_logout", "store_registered",
  "sync_started", "sync_progress", "sync_complete",
  "sync_failed", "sync_cancelled", "sync_skipped",
  "download_queued", "download_started", "download_progress",
  "download_complete", "download_failed", "download_cancelled",
  "game_installed", "game_uninstalled", "game_update_available",
  "game_launched", "game_stopped",
  "store_error", "launcher_stage", "circuit_state_changed",
];

type Handler = (payload: Record<string, unknown>) => void;

/** Wire format returned by `subscribe_replay`. */
interface EventRecord {
  event: string;
  kwargs: Record<string, unknown>;
  timestamp: number;
}

```



```
/** Event bus client impl. */

class EventBusClientImpl {
  private subscribers = new Map<string, Set<Handler>>();
  private lastSeenTimestamp = 0;
  private timer: ReturnType<typeof setTimeout> | null = null;
  private currentInterval = POLL_SLOW_MS;

  /** Subscribe to a backend event by name. Returns the
   * unsubscribe function for cleanup on unmount. */
  subscribe(name: EventName, handler: Handler): () => void {
    let set = this.subscribers.get(name);
    if (!set) {
      set = new Set();
      this.subscribers.set(name, set);
    }
    set.add(handler);
    this.ensurePolling();
    return () => {
      set?.delete(handler);
      if (set?.size === 0) this.subscribers.delete(name);
      if (this.subscribers.size === 0) this.stopPolling();
    };
  }

  /** Bump the polling cadence to fast – call when starting
   * an operation that will emit events imminently (auth,
   * install, sync). Auto-decays back to slow once no events
   * arrive for one tick. */
  bumpToFast(): void {
    this.currentInterval = POLL_FAST_MS;
  }

  /** Dispatch an action URI to the backend. Convenience
   * wrapper around `dispatch_unifideck_action`. */
  async dispatchAction(verb: string, ...args: string[]): Promise<unknown> {
    const path = args.length ? "/" + args.map(encodeURIComponent).join("/") : "";
    const uri = `unifideck://${verb}${path}`;
    return call(rpcRoutes.dispatchUnifideckAction, uri);
  }

  /**
   * Start the polling loop if at least one handler
   * is registered and no loop is already running.
   * Idempotent – safe to call from every `on()`.
   */
  private ensurePolling(): void {
    if (this.timer !== null) return;
    this.scheduleNext();
  }

  /**
   * Stop the polling loop. Called when the last
   * handler is removed via `off()`. Pending timers
   * are cleared so no late dispatch happens after
   * teardown.
   */
  private stopPolling(): void {
    if (this.timer !== null) {
      clearTimeout(this.timer);
    }
  }
}
```

```

        this.timer = null;
    }
}

/**
 * Arm the next `pollOnce()` invocation. The
 * interval grows under back-pressure (no events
 * received) and resets to the floor on activity,
 * keeping the poll cheap when idle.
 */
private scheduleNext(): void {
    this.timer = setTimeout(() => {
        void this.pollOnce();
    }, this.currentInterval);
}

/**
 * Drain the backend event queue once, fanning
 * each event out to its registered handlers.
 * Network errors are swallowed and logged ; they
 * never propagate to handlers.
 */
private async pollOnce(): Promise<void> {
    try {
        const records = await call<[string[]], EventRecord[]>(
            rpcRoutes.subscribeReplay, WATCHED_EVENTS,
        );

        // Dedup : keep only events strictly newer than the last
        // we processed. Sort ascending so handlers see them in
        // emission order.
        const fresh = records
            .filter((r) => r.timestamp > this.lastSeenTimestamp)
            .sort((a, b) => a.timestamp - b.timestamp);

        for (const r of fresh) this.dispatch(r);

        if (records.length > 0) {
            this.lastSeenTimestamp = Math.max(
                this.lastSeenTimestamp,
                ...records.map((r) => r.timestamp),
            );
        }

        // Adaptive cadence : speed up when events arrive,
        // slow down when the backend is quiet.
        this.currentInterval = fresh.length > 0
            ? POLL_FAST_MS : POLL_SLOW_MS;
    } catch (e) {
        console.warn("[EventBusClient] poll error:", e);
        this.currentInterval = POLL_SLOW_MS;
    } finally {
        if (this.subscribers.size > 0) this.scheduleNext();
        else this.timer = null;
    }
}

/**
 * Invoke every handler registered for the given
 * event name, isolating each call so a throwing
 * handler cannot break the others. Mirrors the

```

```
* backend EventBus semantics described in the
* architecture doc §3.2.
*/
private dispatch(record: EventRecord): void {
  const handlers = this.subscribers.get(record.event);
  if (!handlers) return;
  for (const h of handlers) {
    try {
      h(record.kwargs);
    } catch (e) {
      console.error(
        `[EventBusClient] handler for ${record.event} threw:`, e,
      );
    }
  }
}

/** Singleton – process-global by nature. */
export const EventBusClient = new EventBusClientImpl();

/** React hook – subscribe to a backend event for the
 * lifetime of the component. The `deps` array follows the
 * standard useEffect convention. */
export function useEventBus(
  name: EventName,
  handler: Handler,
  deps: unknown[] = [],
): void {
  const handlerRef = useRef(handler);
  handlerRef.current = handler;

  useEffect(() => {
    /** Stable. */
    const stable: Handler = (payload) => handlerRef.current(payload);
    return EventBusClient.subscribe(name, stable);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [name, ...deps]);
}
```

OP-65

index.ts

Fichier : src/contexts/index.ts | Lignes : 16 | Depend de : OP-65a, OP-65b, OP-65c, OP-65d, OP-65e, OP-65f

```
/**
 * React contexts – barrel export.
 *
 * Five domain-specific contexts plus the composition helper
 * `RootProvider` that wires them in the right order. Order
 * matters because `AuthContext` depends on `StoreContext`
 * (auth status is read per registered store), and
 * `DownloadContext` listens to events that other contexts
 * may emit transitively.
 */
export { StoreProvider, useStores } from "./StoreContext";
export { SyncProvider, useSync } from "./SyncContext";
export { AuthProvider, useAuth } from "./AuthContext";
export { DownloadProvider, useDownloads } from "./DownloadContext";
export { LocaleProvider, useLocale } from "./LocaleContext";
export { RootProvider } from "./RootProvider";
```

OP-65a

StoreContext.tsx

Fichier : src/contexts/StoreContext.tsx | Lignes : 62 | Depend de : OP-64a, OP-64b, OP-62a

```
/**
 * StoreContext – registered stores + per-store visual config.
 *
 * Loads `get_store_infos` once on mount, exposes the array
 * to the rest of the tree. Refresh is exposed as `refetch`
 * so consumers (e.g. after STORE_REGISTERED event) can
 * update without re-mounting the provider.
 *
 * Design choice : auth status lives in `AuthContext`, NOT
 * here. This context is concerned with "which stores are
 * registered" only – the connectivity status changes far
 * more often and merging the two would cause spurious
 * re-renders.
 */
import React, { createContext, FC, ReactNode, useContext } from "react";

import { useRPCQuery } from "../api/userRPC";
import { rpcRoutes } from "../api/rpc-routes";
import type { StoreInfo } from "../types/api";

/** Store context value. */

interface StoreContextValue {
  stores: StoreInfo[];
  loading: boolean;
  error: Error | null;
  refetch: () => Promise<void>;
}

const Ctx = createContext<StoreContextValue | null>(null);

/**
 * Provider that loads the registered stores once on
 * mount via the `get_store_infos` RPC and exposes
 * them to descendants. Listens to the
 * `STORE_REGISTERED` event so a runtime-registered
 * store appears without remounting the tree.
 */
export const StoreProvider: FC<{ children: ReactNode }> = ({ children }) => {
  const { data, loading, error, refetch } = useRPCQuery<[], StoreInfo[]>(
    rpcRoutes.getStoreInfos,
    [],
  );
  const value: StoreContextValue = {
    stores: data ?? [],
    loading,
    error,
    refetch,
  };
  return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
};

/**
 * Access the StoreContext value. Throws if called
 * outside the provider – that always indicates a
```

```
* bug in the component tree, never a recoverable
* runtime condition.
*
* @throws Error when the surrounding tree is missing
*   `<StoreProvider>`.
*/
export function useStores(): StoreContextValue {
  const v = useContext(Ctx);
  if (!v) {
    throw new Error("useStores called outside <StoreProvider>");
  }
  return v;
}
```

OP-65b

SyncContext.tsx

Fichier : src/contexts/SyncContext.tsx | Lignes : 114 | Depend de : OP-64b, OP-64d, OP-62b, OP-62g

```

/**
 * SyncContext – library sync state machine.
 *
 * Provides the current `SyncProgress` snapshot plus the
 * three actions a UI can trigger : start, force, cancel.
 * Listens on the EventBus for SYNC_PROGRESS and
 * SYNC_COMPLETE so the progress bar updates reactively
 * without polling.
 *
 * The lifecycle invariant : at most one sync runs at a time.
 * `startSync` while one is running is a no-op (returns the
 * current run id), so duplicate Sync button presses can't
 * stack.
 */
import React, {
  createContext,
  FC,
  ReactNode,
  useCallback,
  useContext,
  useState,
} from "react";

import { useRPCMutation } from "../api/userRPC";
import { rpcRoutes } from "../api/rpc-routes";
import { useEventBus, EventBusClient } from "../api/event-bus-client";
import { Events } from "../types/events";
import type { SyncProgress } from "../types/syncProgress";

/** Sync context value. */

interface SyncContextValue {
  progress: SyncProgress | null;
  isSyncing: boolean;
  isCancelling: boolean;
  startSync: () => Promise<void>;
  forceSync: () => Promise<void>;
  cancelSync: () => Promise<void>;
}

const Ctx = createContext<SyncContextValue | null>(null);

/**
 * Provider that owns the live sync status. Maintains
 * a polling loop on `get_sync_progress` while a sync
 * is running and tears it down on completion.
 *
 * Holding the polling lifecycle here keeps it
 * single-instance even if multiple components read
 * the progress simultaneously.
 */
export const SyncProvider: FC<{ children: ReactNode }> = ({ children }) => {
  const [progress, setProgress] = useState<SyncProgress | null>(null);
  const [isSyncing, setSyncing] = useState(false);
  const [isCancelling, setCancelling] = useState(false);

```

```

const startMut = useRPCMutation<[], { run_id: number }>(
  rpcRoutes.syncLibraries,
);
const forceMut = useRPCMutation<[], { run_id: number }>(
  rpcRoutes.forceSyncLibraries,
);
const cancelMut = useRPCMutation<[], { ok: boolean }>(
  rpcRoutes.cancelSync,
);

// Wire EventBus
useEventBus(Events.SYNC_STARTED, () => {
  setSyncing(true);
  setCancelling(false);
  EventBusClient.bumpToFast();
});
useEventBus(Events.SYNC_PROGRESS, (payload) => {
  setProgress(payload as unknown as SyncProgress);
});
useEventBus(Events.SYNC_COMPLETE, () => {
  setSyncing(false);
  setCancelling(false);
});
useEventBus(Events.SYNC_FAILED, () => {
  setSyncing(false);
  setCancelling(false);
});
useEventBus(Events.SYNC_CANCELLED, () => {
  setSyncing(false);
  setCancelling(false);
});

/** Start sync. */

const startSync = useCallback(async () => {
  if (isSyncing) return;
  EventBusClient.bumpToFast();
  await startMut.mutate();
}, [isSyncing, startMut]);

/** Force sync. */

const forceSync = useCallback(async () => {
  EventBusClient.bumpToFast();
  await forceMut.mutate();
}, [forceMut]);

/** Check whether cel sync. */

const cancelSync = useCallback(async () => {
  if (!isSyncing || isCancelling) return;
  setCancelling(true);
  await cancelMut.mutate();
}, [isSyncing, isCancelling, cancelMut]);

const value: SyncContextValue = {
  progress, isSyncing, isCancelling,
  startSync, forceSync, cancelSync,
};
return <Ctx.Provider value={value}>{children}</Ctx.Provider>;

```



```
};

/**
 * Access the SyncContext value. Throws if used
 * outside `<SyncProvider>` – a tree-wiring bug.
 *
 * @throws Error when the provider is missing.
 */
export function useSync(): SyncContextValue {
  const v = useContext(Ctx);
  if (!v) throw new Error("useSync called outside <SyncProvider>");
  return v;
}
```

OP-65c

AuthContext.tsx

Fichier : src/contexts/AuthContext.tsx | Lignes : 121 | Depend de : OP-64b, OP-64d, OP-62a, OP-62b

```

/**
 * AuthContext – per-store auth status + actions.
 *
 * Exposes :
 * - statuses: `Record<StoreId, StoreStatus>` (connected /
 *   disconnected / expired / error)
 * - startAuth(store) – opens OAuth flow
 * - completeAuth(store, code) – handles 2FA / code paste
 * - logout(store) – clears stored credentials
 * - logoutAll() – wipe every store at once
 *
 * Listens to AUTH_COMPLETE / AUTH_FAILED / LOGOUT_COMPLETE
 * to update statuses without polling.
 */
import React, {
  createContext,
  FC,
  ReactNode,
  useCallback,
  useContext,
  useState,
} from "react";

import { useRPCMutation, useRPCQuery } from "../api/userRPC";
import { rpcRoutes } from "../api/rpc-routes";
import { useEventBus } from "../api/event-bus-client";
import { Events } from "../types/events";
import type { AuthResult, Result, StoreId, StoreStatus } from "../types/api";

type StatusMap = Partial<Record<StoreId, StoreStatus>>;

/** Auth context value. */

interface AuthContextValue {
  statuses: StatusMap;
  loading: boolean;
  startAuth: (store: StoreId) => Promise<AuthResult | null>;
  completeAuth: (
    store: StoreId,
    code: string,
  ) => Promise<AuthResult | null>;
  logout: (store: StoreId) => Promise<void>;
  logoutAll: () => Promise<void>;
}

const Ctx = createContext<AuthContextValue | null>(null);

/**
 * Provider that tracks per-store auth status.
 * Re-evaluates on `AUTH_COMPLETE`, `LOGOUT_COMPLETE`,
 * `AUTH_FAILED` and `ACCOUNT_SWITCHED` events so the
 * UI never shows a stale Connect / Disconnect badge.
 */
export const AuthProvider: FC<{ children: ReactNode }> = ({ children }) => {
  const [statuses, setStatuses] = useState<StatusMap>({});

```

```

// Initial fetch
const initial = useRPCQuery<[], StatusMap>(
  rpcRoutes.checkStoreStatus,
  [],
);
React.useEffect(() => {
  if (initial.data) setStatuses(initial.data);
}, [initial.data]);

// RPC mutations
const startMut = useRPCMutation<
  [StoreId, "start"], AuthResult
>(rpcRoutes.storeAuth);
const completeMut = useRPCMutation<
  [StoreId, "complete", { code: string }], AuthResult
>(rpcRoutes.storeAuth);
const logoutMut = useRPCMutation<
  [StoreId, "logout"], Result
>(rpcRoutes.storeAuth);
const logoutAllMut = useRPCMutation<[], Result>(
  rpcRoutes.clearStoreAuths,
);

// Bus reactions
useEventBus(Events.AUTH_COMPLETE, (payload) => {
  const store = payload.store as StoreId | undefined;
  if (store) setStatuses((s) => ({ ...s, [store]: "connected" }));
});
useEventBus(Events.AUTH_FAILED, (payload) => {
  const store = payload.store as StoreId | undefined;
  if (store) setStatuses((s) => ({ ...s, [store]: "error" }));
});
useEventBus(Events.LOGOUT_COMPLETE, (payload) => {
  const store = payload.store as StoreId | undefined;
  if (store) setStatuses((s) => ({ ...s, [store]: "disconnected" }));
});

const startAuth = useCallback(
  (store: StoreId) => startMut.mutate(store, "start"),
  [startMut],
);
const completeAuth = useCallback(
  (store: StoreId, code: string) =>
    completeMut.mutate(store, "complete", { code }),
  [completeMut],
);
const logout = useCallback(
  async (store: StoreId) => {
    await logoutMut.mutate(store, "logout");
  },
  [logoutMut],
);
/** Logout all. */
const logoutAll = useCallback(async () => {
  await logoutAllMut.mutate();
  setStatuses({});
}, [logoutAllMut]);

const value: AuthContextValue = {
  statuses,

```

```
    loading: initial.loading,
    startAuth, completeAuth, logout, logoutAll,
  };
  return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
};

/**
 * Access the AuthContext value. Throws if used
 * outside `<AuthProvider>`.
 *
 * @throws Error when the provider is missing.
 */
export function useAuth(): AuthContextValue {
  const v = useContext(Ctx);
  if (!v) throw new Error("useAuth called outside <AuthProvider>");
  return v;
}
```

OP-65d

DownloadContext.tsx

Fichier : src/contexts/DownloadContext.tsx | Lignes : 131 | Depend de : OP-64b, OP-64d, OP-62b, OP-62e

```

/**
 * DownloadContext – install queue + progress reactivity.
 *
 * Provides a real-time view of the download queue. The
 * snapshot updates from two sources :
 * 1. Initial fetch via `get_download_queue`.
 * 2. Live updates via DOWNLOAD_* events from the EventBus.
 *
 * Design note: we deliberately avoid storing per-game
 * progress as separate atoms. The queue is an inherently
 * ordered structure and consumers benefit from receiving
 * the full snapshot – re-rendering a 5-item list per tick
 * is cheaper than orchestrating 5 selectors.
 */
import React, {
  createContext,
  FC,
  ReactNode,
  useCallback,
  useContext,
  useEffect,
  useState,
} from "react";

import { useRPCMutation, useRPCQuery } from "../api/userRPC";
import { rpcRoutes } from "../api/rpc-routes";
import { useEventBus, EventBusClient } from "../api/event-bus-client";
import { Events } from "../types/events";
import type { DownloadQueueInfo } from "../types/downloads";
import type { Result, StoreId } from "../types/api";

/** Download context value. */

interface DownloadContextValue {
  queue: DownloadQueueInfo | null;
  loading: boolean;
  installGame: (
    store: StoreId,
    gameId: string,
    options?: { storage?: string },
  ) => Promise<Result | null>;
  uninstallGame: (appId: number) => Promise<Result | null>;
  cancelDownload: (downloadId: string) => Promise<Result | null>;
  refresh: () => Promise<void>;
}

const Ctx = createContext<DownloadContextValue | null>(null);

/**
 * Provider that mirrors the backend download queue
 * to React state. Subscribes to `DOWNLOAD_QUEUED`,
 * `DOWNLOAD_PROGRESS`, `DOWNLOAD_COMPLETE` and
 * `DOWNLOAD_FAILED` events for incremental updates ;
 * a periodic `get_download_queue` poll guards against
 * dropped events.

```

```

*/
export const DownloadProvider: FC<{ children: ReactNode }> = ({ children }) => {
  const [queue, setQueue] = useState<DownloadQueueInfo | null>(null);
  const initial = useRPCQuery<[], DownloadQueueInfo>(
    rpcRoutes.getDownloadQueue,
    [],
  );
  useEffect(() => {
    if (initial.data) setQueue(initial.data);
  }, [initial.data]);

  // RPC mutations
  const installMut = useRPCMutation<
    [StoreId, string, { storage?: string } | undefined], Result
  >(rpcRoutes.installGame);
  const uninstallMut = useRPCMutation<[number], Result>(
    rpcRoutes.uninstallGame,
  );
  const cancelMut = useRPCMutation<[string], Result>(
    rpcRoutes.cancelDownload,
  );

  /** Refresh. */

  const refresh = useCallback(async () => {
    await initial.refetch();
  }, [initial]);

  // Bus reactions – refresh on any queue change
  /** Refetch queue. */
  const refetchQueue = useCallback(() => {
    void refresh();
  }, [refresh]);
  useEventBus(Events.DOWNLOAD_QUEUED, refetchQueue);
  useEventBus(Events.DOWNLOAD_STARTED, () => {
    EventBusClient.bumpToFast();
    refetchQueue();
  });
  useEventBus(Events.DOWNLOAD_PROGRESS, (payload) => {
    // Update only the current item in place; full refetch is
    // wasteful for high-frequency progress events.
    setQueue((prev) => prev && {
      ...prev,
      current: prev.current && {
        ...prev.current,
        progress_percent: (payload.progress as number) ?? prev.current.progress_percent,
        speed_mbps: (payload.speed_mbps as number) ?? prev.current.speed_mbps,
        eta_seconds: (payload.eta_seconds as number) ?? prev.current.eta_seconds,
      },
    });
  });
  useEventBus(Events.DOWNLOAD_COMPLETE, refetchQueue);
  useEventBus(Events.DOWNLOAD_FAILED, refetchQueue);
  useEventBus(Events.DOWNLOAD_CANCELLED, refetchQueue);

  const installGame = useCallback(
    (store: StoreId, gameId: string, options?: { storage?: string }) =>
      installMut.mutate(store, gameId, options),
    [installMut],
  );
  const uninstallGame = useCallback(

```

```
    (appId: number) => uninstallMut.mutate(appId),
    [uninstallMut],
  );
  const cancelDownload = useCallback(
    (downloadId: string) => cancelMut.mutate(downloadId),
    [cancelMut],
  );

  const value: DownloadContextValue = {
    queue,
    loading: initial.loading,
    installGame, uninstallGame, cancelDownload, refresh,
  };
  return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
};

/**
 * Access the DownloadContext value. Throws if used
 * outside `<DownloadProvider>`.
 *
 * @throws Error when the provider is missing.
 */
export function useDownloads(): DownloadContextValue {
  const v = useContext(Ctx);
  if (!v) throw new Error("useDownloads called outside <DownloadProvider>");
  return v;
}
```

OP-65e

LocaleContext.tsx

Fichier : src/contexts/LocaleContext.tsx | Lignes : 82 | Depend de : OP-64b

```
/**
 * LocaleContext – UI language preference.
 *
 * Reads the saved preference at mount, exposes the current
 * language tag, and wraps the i18next `changeLanguage` call
 * so consumers don't reach into the i18n module directly.
 *
 * Persistence is delegated to the backend (set_language_preference
 * RPC). The frontend's i18next instance is updated locally
 * on success, so the UI re-renders immediately while the
 * RPC completes in the background.
 */
import React, {
  createContext,
  FC,
  ReactNode,
  useCallback,
  useContext,
  useEffect,
  useState,
} from "react";
import i18n from "i18next";

import { useRPCMutation, useRPCQuery } from "../api/useRPC";
import { rpcRoutes } from "../api/rpc-routes";

/** Locale context value. */

interface LocaleContextValue {
  locale: string;
  loading: boolean;
  setLocale: (tag: string) => Promise<void>;
}

const Ctx = createContext<LocaleContextValue | null>(null);

/**
 * Provider that owns the active UI language. Reads
 * the persisted choice on mount, then propagates
 * changes to i18next and to the backend (so server
 * toasts come back already translated).
 */
export const LocaleProvider: FC<{ children: ReactNode }> = ({ children }) => {
  const [locale, setLocaleState] = useState<string>(i18n.language);
  const pref = useRPCQuery<[], { success: boolean; language: string }>(
    rpcRoutes.getLanguagePreference,
    [],
  );
  const setMut = useRPCMutation<[string], { success: boolean }>(
    rpcRoutes.setLanguagePreference,
  );

  useEffect(() => {
    if (pref.data?.success && pref.data.language) {
      const tag = pref.data.language;

```



```
    if (tag && tag !== "auto" && tag !== locale) {
      void i18n.changeLanguage(tag);
      setLocaleState(tag);
    }
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [pref.data]);

/** Set locale. */

const setLocale = useCallback(async (tag: string) => {
  // Update local immediately for snappy UI
  await i18n.changeLanguage(tag);
  setLocaleState(tag);
  // Persist (best-effort, non-blocking display)
  await setMut.mutate(tag);
}, [setMut]);

const value: LocaleContextValue = {
  locale,
  loading: pref.loading,
  setLocale,
};
return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
};

/**
 * Access the LocaleContext value. Throws if used
 * outside `<LocaleProvider>`.
 *
 * @throws Error when the provider is missing.
 */
export function useLocale(): LocaleContextValue {
  const v = useContext(Ctx);
  if (!v) throw new Error("useLocale called outside <LocaleProvider>");
  return v;
}
```

OP-65f

RootProvider.tsx

Fichier : src/contexts/RootProvider.tsx | Lignes : 50 | Depend de : OP-65a, OP-65b, OP-65c, OP-65d, OP-65e, OP-71a

```

/**
 * RootProvider – composition of the five domain contexts.
 *
 * Order matters and is documented inside the JSX :
 *  Locale outermost – i18n initialised before any UI string
 *  Store – load store registry
 *  Auth – depends on Store (per-store status)
 *  Sync – emits events Auth may react to
 *  Download outermost – depends on Sync (post-sync downloads)
 *
 * The plugin entry mounts a single <RootProvider> around the
 * entire React subtree so every component has access to all
 * five contexts via the matching `useX` hook.
 *
 * <ToastEventListener> is mounted at the bottom of the tree
 * (alongside `children`) so it has access to all contexts
 * AND to the EventBusClient. It listens to LAUNCHER_STAGE,
 * STORE_ERROR, etc. and renders toasts/modals based on the
 * payloads – this is the canonical bidirectional bridge for
 * backend → user → backend round-trips.
 */
import React, { FC, ReactNode } from "react";

import { LocaleProvider } from "../LocaleContext";
import { StoreProvider } from "../StoreContext";
import { AuthProvider } from "../AuthContext";
import { SyncProvider } from "../SyncContext";
import { DownloadProvider } from "../DownloadContext";
import { ToastEventListener } from "../components/modals/ToastEventListener";

/**
 * Composition of all five context providers in the order
 * dictated by their dependency graph (Locale →
 * Store → Auth → Sync → Download). Mounted once at the
 * top of the React tree by the plugin entry point.
 */
export const RootProvider: FC<{ children: ReactNode }> = ({ children }) => {
  return (
    <LocaleProvider>
      <StoreProvider>
        <AuthProvider>
          <SyncProvider>
            <DownloadProvider>
              {children}
              <ToastEventListener />
            </DownloadProvider>
          </SyncProvider>
        </AuthProvider>
      </StoreProvider>
    </LocaleProvider>
  );
};

```

OP-66

index.ts

Fichier : src/hooks/index.ts | Lignes : 26 | Depend de : OP-66a, OP-66b, OP-66c, OP-66d, OP-66e, OP-66f, OP-66g, OP-66h, O

```
/**
 * Domain hooks subpackage – barrel export.
 *
 * Each hook in this package wraps one or more contexts
 * (Phase F2) and exposes a focused, business-oriented API
 * that components can call without juggling multiple
 * contexts manually. The 9 hooks cover the entire frontend's
 * interaction surface: auth, install/launch actions, game
 * info fetching, view-mode persistence, play-section CDP
 * coordination, download progress, toasts, and Steam library
 * access.
 *
 * Rule of thumb: components import from this barrel; they
 * NEVER reach into individual hook files. Renaming an
 * internal hook is therefore a one-file change in this
 * package.
 */
export { useStoreAuth } from "./useStoreAuth";
export { useGameActions } from "./useGameActions";
export { useGameInfo } from "./useGameInfo";
export { useViewMode } from "./useViewMode";
export { usePlaySection } from "./usePlaySection";
export { useHidePlaySection } from "./useHidePlaySection";
export { useDownloadProgress } from "./useDownloadProgress";
export { useToast } from "./useToast";
export { useSteamLibrary } from "./useSteamLibrary";
```

OP-66a

useStoreAuth.ts

Fichier : src/hooks/useStoreAuth.ts | Lignes : 87 | Depend de : OP-65c, OP-65a, OP-62a

```

/**
 * useStoreAuth – high-level auth flow per store.
 *
 * Glues `AuthContext` (status + actions) and `StoreContext`
 * (registered stores + visuals) for components that drive
 * the auth UX. Returned shape :
 * - `info`      : StoreInfo for the requested store (name,
 *                display_name, brand color, icon path)
 * - `status`    : current StoreStatus (connected / ...)
 * - `connect`   : kick off auth (returns AuthResult or null)
 * - `disconnect` : logout + clear local status
 * - `submit2FA` : complete an in-flight auth with a code
 * - `busy`      : true when an auth call is in flight
 *
 * Components don't compose AuthContext + StoreContext
 * manually; this hook hides the wiring.
 */
import { useCallback, useState } from "react";

import { useAuth } from "../contexts/AuthContext";
import { useStores } from "../contexts/StoreContext";
import type { AuthResult, StoreId } from "../types/api";

/**
 * Shape returned by {@link useStoreAuth}. Bundles the
 * reactive `status` field with the action callbacks so
 * components destructure once instead of subscribing to
 * three hooks.
 */
export interface UseStoreAuthResult {
  info: ReturnType<typeof useStores>["stores"][number] | null;
  status: ReturnType<typeof useAuth>["statuses"][StoreId];
  busy: boolean;
  connect: () => Promise<AuthResult | null>;
  disconnect: () => Promise<void>;
  submit2FA: (code: string) => Promise<AuthResult | null>;
}

/**
 * Hook that drives per-store auth flows. Wraps the
 * generic `store_auth` RPC and exposes
 * `start` / `complete` / `logout` callbacks scoped
 * to the store id provided.
 *
 * The returned `status` is reactive : auth events
 * trigger a re-evaluation of `is_available()` so
 * the UI reflects the latest state without
 * polling.
 *
 * @param storeId – id of the store to drive.
 * @returns auth state + action callbacks.
 */
export function useStoreAuth(store: StoreId): UseStoreAuthResult {
  const auth = useAuth();
  const { stores } = useStores();

```

```
const [busy, setBusy] = useState(false);

/** Info. */

const info = stores.find((s) => s.name === store) ?? null;
const status = auth.statuses[store];

/** Connect. */

const connect = useCallback(async () => {
  setBusy(true);
  try {
    return await auth.startAuth(store);
  } finally {
    setBusy(false);
  }
}, [auth, store]);

/** Disconnect. */

const disconnect = useCallback(async () => {
  setBusy(true);
  try {
    await auth.logout(store);
  } finally {
    setBusy(false);
  }
}, [auth, store]);

const submit2FA = useCallback(
  async (code: string) => {
    setBusy(true);
    try {
      return await auth.completeAuth(store, code);
    } finally {
      setBusy(false);
    }
  },
  [auth, store],
);

return { info, status, busy, connect, disconnect, submit2FA };
}
```

OP-66b

useGameActions.ts

Fichier : src/hooks/useGameActions.ts | Lignes : 104 | Depend de : OP-65d, OP-63a, OP-62a

```

/**
 * useGameActions – install / uninstall / launch / cancel.
 *
 * Wraps `DownloadContext` plus `SteamBridge.runGame` /
 * `terminateApp`, exposing a clean API for any component
 * that needs to trigger a game-state action :
 * - `install(store, gameId, options?)`
 * - `uninstall(appId)`
 * - `cancel(downloadId)`
 * - `launch(appId, launchOptions)`
 * - `terminate(appId, force?)`
 *
 * The hook tracks an `isWorking` flag while any one of these
 * is in flight, useful for disabling buttons or showing a
 * spinner without each caller managing its own loading state.
 */
import { useCallback, useState } from "react";

import { useDownloads } from "../contexts/DownloadContext";
import type { Result, StoreId } from "../types/api";

/** Steam bridge shape. */

interface SteamBridgeShape {
  runGame(appId: string, launchOptions: string): void;
  terminateApp(appId: string, force?: boolean): void;
}

/**
 * Memoised action callbacks returned by {@link useGameActions}.
 * Identity-stable across renders so they can be passed to
 * memoised children without breaking memoisation.
 */
export interface UseGameActionsResult {
  isWorking: boolean;
  install: (
    store: StoreId, gameId: string, options?: { storage?: string },
  ) => Promise<Result | null>;
  uninstall: (appId: number) => Promise<Result | null>;
  cancel: (downloadId: string) => Promise<Result | null>;
  launch: (appId: number, launchOptions: string) => void;
  terminate: (appId: number, force?: boolean) => void;
}

/**
 * Hook bundling all game-level actions a UI element
 * may trigger : install, uninstall, launch, cancel
 * download, force update. Each callback returns a
 * promise so callers can await the backend response
 * before showing a confirmation toast.
 *
 * @param game – the Game the actions apply to.
 * @returns set of memoised action callbacks.
 */
export function useGameActions(

```

```
bridge: SteamBridgeShape,
): UseGameActionsResult {
  const downloads = useDownloads();
  const [isWorking, setWorking] = useState(false);

  const install = useCallback(
    async (
      store: StoreId, gameId: string,
      options?: { storage?: string },
    ) => {
      setWorking(true);
      try {
        return await downloads.installGame(store, gameId, options);
      } finally {
        setWorking(false);
      }
    },
    [downloads],
  );

  const uninstall = useCallback(
    async (appId: number) => {
      setWorking(true);
      try {
        return await downloads.uninstallGame(appId);
      } finally {
        setWorking(false);
      }
    },
    [downloads],
  );

  const cancel = useCallback(
    async (downloadId: string) => {
      setWorking(true);
      try {
        return await downloads.cancelDownload(downloadId);
      } finally {
        setWorking(false);
      }
    },
    [downloads],
  );

  const launch = useCallback(
    (appId: number, launchOptions: string) => {
      bridge.runGame(String(appId), launchOptions);
    },
    [bridge],
  );

  const terminate = useCallback(
    (appId: number, force: boolean = false) => {
      bridge.terminateApp(String(appId), force);
    },
    [bridge],
  );

  return { isWorking, install, uninstall, cancel, launch, terminate };
}
```

OP-66c

useGameInfo.ts

Fichier : src/hooks/useGameInfo.ts | Lignes : 109 | Depend de : OP-64b, OP-64a, OP-62a

```

/**
 * useGameInfo – per-appId info fetch with TTL cache.
 *
 * Replaces the global `gameInfoCache` Map that lived in the
 * old `index.tsx` (and was passed via setter callbacks all
 * over the place). Each component that needs game info just
 * calls `useGameInfo(appId)` and gets reactive {data, loading,
 * error, refresh}. The hook shares a module-level cache so
 * concurrent consumers of the same appId don't re-fetch.
 *
 * Cache invariants :
 * - TTL = 5000ms (matches old behaviour)
 * - Same appId across components = single fetch in flight
 * - `refresh()` always re-fetches (bypasses cache)
 */
import { useCallback, useEffect, useState } from "react";

import { useRPC } from "../api/useRPC";
import { rpcRoutes } from "../api/rpc-routes";
import type { Game } from "../types/api";

/** Cache entry. */

interface CacheEntry {
  data: Game | null;
  ts: number;
  inflight: Promise<Game | null> | null;
}

const CACHE_TTL = 5000;
const cache = new Map<number, CacheEntry>();

/**
 * Aggregated game info returned by {@link useGameInfo} –
 * description, scores, artwork URLs, playtime fragments –
 * with loading and error flags propagated through.
 */
export interface UseGameInfoResult {
  data: Game | null;
  loading: boolean;
  error: Error | null;
  refresh: () => Promise<void>;
}

/**
 * Hook that aggregates all metadata Unifideck has on
 * a given game : description, scores, artwork,
 * playtime stats. Uses `useRPCQuery` under the hood
 * with a sensible cache TTL so opening the info panel
 * is instant on revisits.
 *
 * @param appId – Steam shortcut app-id.
 * @returns aggregated info + loading/error flags.
 */
export function useGameInfo(appId: number | null): UseGameInfoResult {

```



```

const fetch = useRPC<[number], Game>(rpcRoutes.getGameMetadata);
const [state, setState] = useState<{
  data: Game | null;
  loading: boolean;
  error: Error | null;
}>({ data: null, loading: appId !== null, error: null });

const load = useCallback(
  async (force: boolean): Promise<void> => {
    if (appId == null) {
      setState({ data: null, loading: false, error: null });
      return;
    }
    const cached = cache.get(appId);
    if (!force && cached && Date.now() - cached.ts < CACHE_TTL) {
      setState({ data: cached.data, loading: false, error: null });
      return;
    }
    // De-duplicate concurrent in-flight fetches
    if (cached?.inflight && !force) {
      const data = await cached.inflight;
      setState({ data, loading: false, error: null });
      return;
    }
    setState((s) => ({ ...s, loading: true, error: null }));
    const promise = fetch(appId).then(
      (data) => {
        cache.set(appId, { data, ts: Date.now(), inflight: null });
        return data;
      },
      (err) => {
        cache.set(appId, { data: null, ts: Date.now(), inflight: null });
        throw err;
      },
    );
    cache.set(appId, {
      data: cached?.data ?? null,
      ts: cached?.ts ?? 0,
      inflight: promise,
    });
    try {
      const data = await promise;
      setState({ data, loading: false, error: null });
    } catch (err) {
      setState({ data: null, loading: false, error: err as Error });
    }
  },
  [appId, fetch],
);

useEffect(() => {
  void load(false);
}, [load]);

/** Refresh. */

const refresh = useCallback(() => load(true), [load]);

return { ...state, refresh };
}

```

```
/** Test/dev helper – clear the module-level cache. Not
 * exposed via the barrel; imported only by vitest specs. */
export function _clearGameInfoCache(): void {
  cache.clear();
}
```

OP-66d

useViewModel.ts

Fichier : src/hooks/useViewModel.ts | Lignes : 82 | Depend de : (rien)

```

/**
 * useViewModel — game-detail view-mode preference.
 *
 * Persists the user's choice (compact / full) to a window-
 * scoped key (`window.unifideck_view_mode`) and broadcasts
 * changes via a custom event so every mounted consumer
 * stays in sync without prop drilling. Replaces the
 * `getStoredViewModel` / `setStoredViewModel` helpers + the
 * `VIEW_MODE_CHANGE_EVENT` global from the old index.tsx.
 *
 * Persistence target deliberately NOT localStorage : the
 * Steam CEF environment doesn't reliably surface
 * localStorage to plugins. Window properties survive within
 * a Steam session, which is what we need.
 */
import { useCallback, useEffect, useState } from "react";

/**
 * Compact (cover-only) vs full (cover + metadata + scores)
 * rendering of the game-details view. Persisted user choice.
 */
export type GameDetailsViewModel = "compact" | "full";

const STORAGE_KEY = "__unifideck_view_mode__";
const CHANGE_EVENT = "unifideck:view-mode-change";
const DEFAULT_MODE: GameDetailsViewModel = "compact";

/** Window with view mode. */

interface WindowWithViewModel extends Window {
  [STORAGE_KEY]?: GameDetailsViewModel;
}

/** Read mode. */

function readMode(): GameDetailsViewModel {
  const w = window as WindowWithViewModel;
  return w[STORAGE_KEY] ?? DEFAULT_MODE;
}

/** Write mode. */

function writeMode(mode: GameDetailsViewModel): void {
  const w = window as WindowWithViewModel;
  w[STORAGE_KEY] = mode;
  window.dispatchEvent(
    new CustomEvent(CHANGE_EVENT, { detail: mode }),
  );
}

/**
 * Shape returned by {@link useViewModel}. The setter writes
 * through to backend storage so the choice survives reloads.
 */
export interface UseViewModelResult {

```

```
mode: GameDetailsViewMode;
setMode: (mode: GameDetailsViewMode) => void;
toggle: () => void;
}

/**
 * Hook exposing the user's preferred game-details
 * view (compact vs detailed) with persistence to
 * the backend config so the choice survives reloads
 * and Steam restarts.
 *
 * @returns current view + `setMode` setter that
 *   writes through to backend storage.
 */
export function useViewMode(): UseViewModeResult {
  const [mode, setModeState] = useState<GameDetailsViewMode>(readMode);

  // Sync this consumer when ANY other consumer changes the mode
  useEffect(() => {
    /** Handler. */
    const handler = (e: Event) => {
      const ce = e as CustomEvent<GameDetailsViewMode>;
      if (ce.detail) setModeState(ce.detail);
    };
    window.addEventListener(CHANGE_EVENT, handler);
    return () => window.removeEventListener(CHANGE_EVENT, handler);
  }, []);

  /** Set mode. */

  const setMode = useCallback((next: GameDetailsViewMode) => {
    writeMode(next);
    setModeState(next);
  }, []);

  /** Toggle. */

  const toggle = useCallback(() => {
    setMode(mode === "compact" ? "full" : "compact");
  }, [mode, setMode]);

  return { mode, setMode, toggle };
}
```

OP-66e

usePlaySection.ts

Fichier : src/hooks/usePlaySection.ts | Lignes : 78 | Depend de : OP-63a, OP-66c

```

/**
 * usePlaySection – Steam "Play / Install" button override.
 *
 * Mounted by `<PlaySectionWrapper>`, this hook decides what
 * the Play section should look like for a given Steam appId :
 *
 * - Steam-native game → no override (returns
 *   `{ shouldOverride: false }`)
 * - Unifideck game, NOT installed → expose `installAction`
 * - Unifideck game, IN download queue → expose
 *   `cancelDownloadAction` + the live download item
 * - Unifideck game, installed → expose `playAction` plus
 *   `uninstallAction`
 *
 * The decision is recomputed reactively when the underlying
 * `useGameInfo` data or the download queue snapshot changes.
 * The render component reads `shouldOverride` and the
 * action functions; it stays presentational.
 */
import { useMemo } from "react";

import { useGameInfo } from "../useGameInfo";
import { useDownloads } from "../contexts/DownloadContext";
import type { DownloadItem } from "../types/downloads";

/**
 * Discriminated union of the Play-section states. Drives
 * which variant of the Play button is rendered
 * (NotInstalled / Downloading / Installed / Launching).
 */
export type PlaySectionState =
  | { kind: "steam-native"; shouldOverride: false }
  | { kind: "not-installed"; shouldOverride: true; installable: true }
  | { kind: "downloading"; shouldOverride: true; download: DownloadItem }
  | { kind: "installed"; shouldOverride: true; appId: number };

/**
 * Hook that resolves the current Play-section state
 * for an app : not-installed, downloading, installed,
 * launching, or running. Watches install / launch /
 * download events to recompute on every transition,
 * driving the right Play-button variant.
 *
 * @param appId – Steam shortcut app-id.
 * @returns current `PlaySectionState`.
 */
export function usePlaySection(appId: number | null): PlaySectionState {
  const info = useGameInfo(appId);
  const downloads = useDownloads();

  return useMemo<PlaySectionState>(() => {
    if (appId == null || !info.data) {
      return { kind: "steam-native", shouldOverride: false };
    }
    const game = info.data;

```

```
// Filter out games whose store is plain Steam – we never
// override those.
if (game.store === "steam") {
  return { kind: "steam-native", shouldOverride: false };
}

// Check the live queue first – a download in progress
// takes precedence over `is_installed` flag staleness.
const inQueue = findInQueue(downloads.queue?.current, game.id)
  ?? downloads.queue?.queued.find((d) => d.game_id === game.id)
  ?? null;
if (inQueue) {
  return { kind: "downloading", shouldOverride: true, download: inQueue };
}

if (!game.is_installed) {
  return { kind: "not-installed", shouldOverride: true, installable: true };
}
return { kind: "installed", shouldOverride: true, appId };
}, [appId, info.data, downloads.queue]);
}

/** Find in queue. */

function findInQueue(
  current: DownloadItem | null | undefined,
  gameId: string,
): DownloadItem | null {
  if (!current) return null;
  return current.game_id === gameId ? current : null;
}
```

OP-66f

useHidePlaySection.ts

Fichier : src/hooks/useHidePlaySection.ts | Lignes : 68 | Depend de : OP-64b, OP-64a

```

/**
 * useHidePlaySection – CDP-driven hide/show of Steam's
 * native play section.
 *
 * When `<PlaySectionWrapper>` decides to render a Unifideck
 * override, Steam's own play button must be hidden (otherwise
 * users see two buttons). This hook serializes inject /
 * remove operations per appId so two rapid mode flips don't
 * leave the DOM in an inconsistent state.
 *
 * Backend exposes two endpoints (UIRPCMixin) :
 * - `hide_play_section(app_id)` → CDP injects hiding CSS
 * - `unhide_play_section(app_id)` → CDP removes the rule
 *
 * The serialization mechanism is a per-appId promise chain
 * stored in a module-level Map. Each new operation is
 * appended to its appId's chain, ensuring strict ordering :
 * inject A → remove A → inject A is processed in that order
 * regardless of timing.
 */
import { useEffect } from "react";

import { useRPC } from "../api/useRPC";
import { rpcRoutes } from "../api/rpc-routes";

const chain = new Map<number, Promise<void>>();
const generation = new Map<number, number>();

/** Append op. */

function appendOp(appId: number, op: () => Promise<void>): void {
  const tail = chain.get(appId) ?? Promise.resolve();
  /** Next. */
  const next = tail.then(op).catch((e) => {
    console.warn(
      `[useHidePlaySection] op failed for ${appId}:`, e,
    );
  });
  chain.set(appId, next);
}

/** Hide Steam's native play section for `appId` while the
 * consumer component is mounted. Restores on unmount. */
export function useHidePlaySection(
  appId: number | null,
  enabled: boolean,
): void {
  const hide = useRPC<[number], { ok: boolean }>(>(
    rpcRoutes.hidePlaySection,
  ));
  const unhide = useRPC<[number], { ok: boolean }>(>(
    rpcRoutes.unhidePlaySection,
  ));

  useEffect(() => {

```

```
if (appId == null || !enabled) return;

// Bump generation – discriminates stale ops if the same
// appId mounts/unmounts/remounts faster than RPC roundtrip
const gen = (generation.get(appId) ?? 0) + 1;
generation.set(appId, gen);

appendOp(appId, async () => {
  if (generation.get(appId) !== gen) return;
  await hide(appId);
});

return () => {
  const gen2 = (generation.get(appId) ?? 0) + 1;
  generation.set(appId, gen2);
  appendOp(appId, async () => {
    if (generation.get(appId) !== gen2) return;
    await unhide(appId);
  });
};
}, [appId, enabled, hide, unhide]);
}
```


OP-66g

useDownloadProgress.ts

Fichier : src/hooks/useDownloadProgress.ts | Lignes : 68 | Depend de : OP-65d, OP-62e

```
/**
 * useDownloadProgress — focused selector on a specific
 * game's download progress.
 *
 * `DownloadContext` exposes the full queue snapshot ; for
 * components that only care about ONE game (a per-row
 * progress bar, a Play button overlay, ...), reading the
 * full queue and re-deriving the matching item every render
 * is wasteful. This hook does the selection once and only
 * re-renders the consumer when that game's progress
 * actually changes.
 *
 * Returns `null` if the game isn't currently in the queue.
 */
import { useMemo } from "react";

import { useDownloads } from "../contexts/DownloadContext";
import type { DownloadItem } from "../types/downloads";

/**
 * Shape returned by {@link useDownloadProgress}. Only the
 * fields a progress bar consumes are exposed — keeps the
 * memo footprint tight.
 */
export interface UseDownloadProgressResult {
  item: DownloadItem | null;
  isCurrent: boolean;
  isQueued: boolean;
  progressPercent: number;
}

/**
 * Hook scoped to a single in-flight download. Pulls
 * progress / phase / speed from `DownloadContext`
 * and memoises the slice so unrelated queue updates
 * don't re-render the consuming component.
 *
 * @param downloadId — id of the download to follow.
 * @returns reactive progress fields, or `null`
 * when the download id is unknown to the queue.
 */
export function useDownloadProgress(
  gameId: string | null,
): UseDownloadProgressResult {
  const { queue } = useDownloads();

  return useMemo(() => {
    const empty: UseDownloadProgressResult = {
      item: null, isCurrent: false, isQueued: false, progressPercent: 0,
    };
    if (gameId == null || !queue) return empty;

    if (queue.current && queue.current.game_id === gameId) {
      return {
        item: queue.current,

```

```
        isCurrent: true,
        isQueued: false,
        progressPercent: queue.current.progress_percent,
    };
}
/** Queued. */
const queued = queue.queued.find((d) => d.game_id === gameId);
if (queued) {
    return {
        item: queued,
        isCurrent: false,
        isQueued: true,
        progressPercent: queued.progress_percent,
    };
}
return empty;
}, [gameId, queue]);
}
```

OP-66h

useToast.ts

Fichier : src/hooks/useToast.ts | Lignes : 74 | Depend de : (rien)

```
/**
 * useToast – typed wrapper around Decky's `toaster.toast`.
 *
 * Wraps the global toaster with three semantics-bearing
 * methods (`info`, `success`, `error`) and a base `show` for
 * full control. Components import this hook instead of
 * `toaster` directly so :
 * - duration defaults are centralised (no scattered "5000")
 * - a future swap to a different notification system is a
 *   one-file change
 * - a no-op fallback is provided when the @decky/api
 *   toaster is unavailable (test environments)
 */
import { toaster } from "@decky/api";
import { useCallback } from "react";

/** Toast options. */

interface ToastOptions {
  duration?: number;
  body?: string;
  onClick?: () => void;
}

const DEFAULT_DURATION = 5000;
const ERROR_DURATION = 7500;

/**
 * Shape returned by {@link useToast}. The single field is
 * a stable `showToast(args)` callback – no other state.
 */
export interface UseToastResult {
  show: (
    title: string, body?: string, options?: ToastOptions,
  ) => void;
  info: (title: string, body?: string) => void;
  success: (title: string, body?: string) => void;
  error: (title: string, body?: string) => void;
}

/**
 * Hook returning a memoised `showToast` callback that
 * forwards to Decky Loader's toast API. Centralised
 * here so styling / duration defaults stay consistent
 * across the codebase and so test code can mock a
 * single boundary.
 */
export function useToast(): UseToastResult {
  const show = useCallback(
    (title: string, body?: string, options: ToastOptions = {}) => {
      try {
        toaster.toast({
          title,
          body: body ?? options.body ?? "",
          duration: options.duration ?? DEFAULT_DURATION,

```

```
        onClick: options.onClick,
      });
    } catch (e) {
      // No toaster available – log instead
      console.log(`[Toast] ${title}: ${body ?? ""}`);
    }
  },
  [],
);

const info = useCallback(
  (title: string, body?: string) => show(title, body),
  [show],
);

const success = useCallback(
  (title: string, body?: string) => show(title, body),
  [show],
);

const error = useCallback(
  (title: string, body?: string) =>
    show(title, body, { duration: ERROR_DURATION }),
  [show],
);

return { show, info, success, error };
}
```

OP-66i

useSteamLibrary.ts

Fichier : src/hooks/useSteamLibrary.ts | Lignes : 107 | Depend de : OP-63a, OP-62d

```
/**
 * useSteamLibrary – typed snapshot of Steam's app library.
 *
 * Reads owned + non-Steam apps via SteamBridge, transforms
 * them into the canonical `UnifideckGame` shape, and exposes
 * filter helpers (by store, by installed, by deck-verified).
 *
 * Replaces the legacy `useSteamLibrary.ts` which hit
 * `window.SteamClient` directly. All Steam internals are
 * now funnelled through SteamBridge so a Steam update
 * breaks one file (the bridge), not this hook.
 */
import { useCallback, useEffect, useMemo, useState } from "react";

import type { SteamBridge } from "../lib/steam-bridge";
import type {
  ControllerInfo, // referenced by re-export only
  SteamApp,
} from "../types/steam";

/**
 * Frontend projection of an Unifideck-managed shortcut. Holds
 * the union of fields any tab/component reads – keeps the
 * concrete `Game` from the api-types layer minimal.
 */
export interface UnifideckGame {
  appId: number;
  title: string;
  store: "steam" | "unknown"; // expanded in a follow-up
  isInstalled: boolean;
  isShortcut: boolean;
  lastPlayed: number;
  playtimeMinutes: number;
}

/**
 * Shape returned by {@link useSteamLibrary}. Exposes the
 * Unifideck slice of the Steam library plus a refetch hook
 * for callers that need a forced refresh.
 */
export interface UseSteamLibraryResult {
  games: UnifideckGame[];
  loading: boolean;
  error: string | null;
  refresh: () => void;
  filterByStore: (store: UnifideckGame["store"]) => UnifideckGame[];
  filterByInstalled: () => UnifideckGame[];
}

/** Transform. */

function transform(app: SteamApp): UnifideckGame {
  return {
    appId: app.appid,
    title: app.display_name || app.sort_as || "Unknown",
```

```

    store: app.is_shortcuts_app || app.BIsShortcut?.(
      ? "unknown" : "steam",
      installed: !!app.installed,
      isShortcut: !!app.is_shortcuts_app || !!app.BIsShortcut?.(
        lastPlayed: app.rt_last_time_played || 0,
        playtimeMinutes: parseInt(app.minutes_playtime_forever || "0", 10),
      );
  };
}

/**
 * Hook that exposes the Unifideck-managed slice of
 * the Steam library – non-Steam shortcuts created
 * by Unifideck. Subscribes to library mutation
 * events so additions / removals reflect without a
 * full sync.
 *
 * @returns array of Unifideck games + loading flag.
 */
export function useSteamLibrary(
  bridge: SteamBridge,
): UseSteamLibraryResult {
  const [games, setGames] = useState<UnifideckGame[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  /** Load. */

  const load = useCallback(() => {
    setLoading(true);
    setError(null);
    try {
      if (!bridge.isReady()) {
        throw new Error("Steam Apps API not available");
      }
      const all = bridge.getAllApps();
      setGames(all.map(transform));
    } catch (err) {
      setError(err instanceof Error ? err.message : String(err));
    } finally {
      setLoading(false);
    }
  }, [bridge]);

  useEffect(() => {
    load();
  }, [load]);

  const filterByStore = useCallback(
    (store: UnifideckGame["store"]) =>
      games.filter((g) => g.store === store),
    [games],
  );

  const filterByInstalled = useCallback(
    () => games.filter((g) => g.isInstalled),
    [games],
  );

  return useMemo(() => ({
    games, loading, error,
    refresh: load,
  }

```

```
        filterByStore, filterByInstalled,  
    )), [games, loading, error, load, filterByStore, filterByInstalled]);  
}
```

OP-67

index.ts

Fichier : src/components/index.ts | Lignes : 24 | Depend de : OP-68, OP-69, OP-70, OP-71, OP-72, OP-73

```
/**
 * Components – barrel export.
 *
 * Re-exports every presentational component from its sub-
 * package so consumers can write
 *
 *   import { GameInfoPanel, PlaySectionWrapper } from "../components";
 *
 * without knowing which sub-folder hosts which file. Each
 * sub-package (play / info / settings / modals / downloads /
 * shared) has its own barrel that this file aggregates.
 *
 * Hard rule : files in this barrel are PRESENTATIONAL ONLY.
 * Business logic lives in `hooks/` (Phase F3) or `services/`
 * (Phase F5). A component that calls `call()` or
 * `window.SteamClient` directly is a bug – it must go
 * through the appropriate hook or the SteamBridge.
 */
export * from "./play";
export * from "./info";
export * from "./settings";
export * from "./modals";
export * from "./downloads";
export * from "./shared";
```


OP-68

index.ts

Fichier : src/components/play/index.ts | Lignes : 14 | Depend de : OP-68a, OP-68b, OP-68c, OP-68d

```
/**
 * Play section override – barrel export.
 *
 * Decomposes the legacy PlayButtonOverride.tsx (2479 LOC)
 * into a dispatcher (`PlaySectionWrapper`) and three
 * state-specific button groups (NotInstalled / Downloading
 * / Installed). The dispatcher reads `usePlaySection` and
 * routes to the correct presentation, eliminating the giant
 * if/else chain that polluted the legacy file.
 */
export { PlaySectionWrapper } from "./PlaySectionWrapper";
export { NotInstalledButtons } from "./NotInstalledButtons";
export { DownloadingButtons } from "./DownloadingButtons";
export { InstalledButtons } from "./InstalledButtons";
```

OP-68a

PlaySectionWrapper.tsx

Fichier : src/components/play/PlaySectionWrapper.tsx | Lignes : 57 | Depend de : OP-66e, OP-66f, OP-68b, OP-68c, OP-68d

```

/**
 * PlaySectionWrapper – dispatcher for the Play section.
 *
 * Reads `usePlaySection(appId)` and renders one of three
 * sub-components based on the discriminated state. When the
 * state is "steam-native", the wrapper passes through to
 * Steam's own play section without modification – that's the
 * default behaviour for non-Unifideck games.
 *
 * The wrapper itself is pure presentation : the decision
 * logic is in `usePlaySection`, the CDP hide/show is in
 * `useHidePlaySection`, the actions are in `useGameActions`.
 * This component just glues them.
 */
import React, { FC, ReactNode } from "react";

import { usePlaySection } from "../../hooks/usePlaySection";
import { useHidePlaySection } from "../../hooks/useHidePlaySection";
import { NotInstalledButtons } from "../NotInstalledButtons";
import { DownloadingButtons } from "../DownloadingButtons";
import { InstalledButtons } from "../InstalledButtons";

/**
 * Props of {@link PlaySectionWrapper}. The `appId` is
 * the Steam shortcut id ; everything else is resolved
 * through hooks inside the wrapper.
 */
export interface PlaySectionWrapperProps {
  appId: number;
  children: ReactNode; // Steam's native play section
}

/**
 * Top-level wrapper rendered in place of Steam's own
 * Play section for Unifideck games. Picks the right
 * variant (Not-installed / Downloading / Installed)
 * based on `usePlaySection` and forwards the action
 * callbacks coming from `useGameActions`.
 */
export const PlaySectionWrapper: FC<PlaySectionWrapperProps> = ({
  appId, children,
}) => {
  const state = usePlaySection(appId);
  // Hide Steam's native section iff we're overriding it
  useHidePlaySection(appId, state.shouldOverride);

  if (!state.shouldOverride) {
    return <{children}>;
  }

  switch (state.kind) {
    case "not-installed":
      return <NotInstalledButtons appId={appId} />;
    case "downloading":
      return <DownloadingButtons download={state.download} />;
  }

```

```
case "installed":  
  return <InstalledButtons appId={state.appId} />;  
default:  
  // Exhaustiveness – TS will flag missing branches  
  return <>{children}</>;  
}  
};
```

OP-68b

NotInstalledButtons.tsx

Fichier : src/components/play/NotInstalledButtons.tsx | Lignes : 66 | Depend de : OP-66c, OP-66b, OP-66h

```

/**
 * NotInstalledButtons – Play section for not-yet-installed
 * Unifideck games.
 *
 * Renders an "Install" button that triggers `useGameActions
 * .install()` with the canonical store + game_id pulled
 * from `useGameInfo`. While the install RPC is in flight we
 * disable the button and show a spinner.
 *
 * The "Storage location" picker is NOT inlined here – it is
 * a separate modal opened on click when the user has more
 * than one storage location. The modal itself lives in
 * `components/modals/StoragePickerModal.tsx`.
 */
import React, { FC, useCallback } from "react";
import { DialogButton } from "@decky/ui";
import { useTranslation } from "react-il8next";

import { useGameInfo } from "../../hooks/useGameInfo";
import { useGameActions } from "../../hooks/useGameActions";
import { useToast } from "../../hooks/useToast";
import { SteamBridge } from "../../lib/steam-bridge";

/** Props. */

interface Props {
  appId: number;
  bridge?: SteamBridge;
}

const defaultBridge = new SteamBridge();

/**
 * Variant of the Play section shown when the game is not
 * installed yet : Install button, store selector if the
 * game is owned in multiple stores, and Cancel/Hide.
 */
export const NotInstalledButtons: FC<Props> = ({
  appId, bridge = defaultBridge,
}) => {
  const { t } = useTranslation();
  const { data: game, loading } = useGameInfo(appId);
  const actions = useGameActions(bridge);
  const toast = useToast();

  /** On install. */

  const onInstall = useCallback(async () => {
    if (!game) return;
    const result = await actions.install(game.store, game.id);
    if (result?.success) {
      toast.success(
        t("toasts.downloadQueued"),
        t("toasts.downloadQueuedBody", { title: game.title }),
      );
    }
  });

```

```
    } else {
      toast.error(
        t("toasts.downloadFailed"),
        result?.error ?? t("toasts.downloadFailedUnknown"),
      );
    }
  }, [actions, game, t, toast]);

return (
  <div style={{ display: "flex", gap: 8 }}>
    <DialogButton
      disabled={loading || actions.isWorking || !game}
      onClick={onInstall}
    >
      {actions.isWorking ? t("play.installing") : t("play.install")}
    </DialogButton>
  </div>
);
};
```

OP-68c

DownloadingButtons.tsx

Fichier : src/components/play/DownloadingButtons.tsx | Lignes : 76 | Depend de : OP-66b, OP-66g, OP-62e

```

/**
 * DownloadingButtons – Play section while a download is
 * active for the displayed game.
 *
 * Shows a progress bar with current %, ETA, and speed plus a
 * Cancel button. The progress is read from
 * `useDownloadProgress` which is the focused selector hook
 * (only re-renders when THIS game's progress changes).
 *
 * The cancel action goes through `useGameActions.cancel`
 * with a 1-second debounce to prevent double-clicks
 * cascading two cancellations.
 */
import React, { FC, useCallback, useState } from "react";
import { DialogButton } from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useGameActions } from "../../hooks/useGameActions";
import { useToast } from "../../hooks/useToast";
import { SteamBridge } from "../../lib/steam-bridge";
import type { DownloadItem } from "../../types/downloads";

/** Props. */

interface Props {
  download: DownloadItem;
  bridge?: SteamBridge;
}

const defaultBridge = new SteamBridge();

/** Format eta. */

function formatEta(secs: number): string {
  if (secs <= 0) return "-";
  const m = Math.floor(secs / 60);
  const s = secs % 60;
  return m > 0 ? `${m}m ${s}s` : `${s}s`;
}

/**
 * Variant of the Play section shown while the game is
 * downloading : progress bar, ETA, Cancel button, Pause
 * if the underlying store supports it.
 */
export const DownloadingButtons: FC<Props> = ({
  download, bridge = defaultBridge,
}) => {
  const { t } = useTranslation();
  const actions = useGameActions(bridge);
  const toast = useToast();
  const [cancelled, setCancelled] = useState(false);

  /** On cancel. */

```

```
const onCancel = useCallback(async () => {
  if (cancelled) return;
  setCancelled(true);
  const result = await actions.cancel(download.id);
  if (!result?.success) {
    setCancelled(false);
    toast.error(
      t("toasts.cancelFailed"),
      result?.error ?? "",
    );
  }
}, [actions, cancelled, download.id, t, toast]);

return (
  <div style={{ display: "flex", flexDirection: "column", gap: 6 }}>
    <div style={{ fontSize: 12 }}>
      {download.progress_percent.toFixed(0)}%
      {" . "}
      {download.speed_mbps.toFixed(1)} MB/s
      {" . "}
      ETA {formatEta(download.eta_seconds)}
    </div>
    <DialogButton
      disabled={cancelled || actions.isWorking}
      onClick={onCancel}
    >
      {cancelled ? t("play.cancelling") : t("play.cancel")}
    </DialogButton>
  </div>
);
};
```

OP-68d

InstalledButtons.tsx

Fichier : src/components/play/InstalledButtons.tsx | Lignes : 73 | Depend de : OP-66c, OP-66b, OP-66h

```
/**
 * InstalledButtons – Play section for installed Unifideck
 * games.
 *
 * Two buttons : "Play" (launches via SteamBridge) and
 * "Uninstall" (opens the confirm modal). The Play button
 * delegates to `useGameActions.launch` which routes through
 * SteamBridge.runGame ; the launch options are read from
 * the game info (cached). Uninstall opens
 * `<UninstallConfirmModal>` rather than acting immediately.
 */
import React, { FC, useCallback } from "react";
import { DialogButton, showModal } from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useGameInfo } from "../../hooks/useGameInfo";
import { useGameActions } from "../../hooks/useGameActions";
import { useToast } from "../../hooks/useToast";
import { SteamBridge } from "../../lib/steam-bridge";
import { UninstallConfirmModal } from "../../modals/UninstallConfirmModal";

/** Props. */

interface Props {
  appId: number;
  bridge?: SteamBridge;
}

const defaultBridge = new SteamBridge();

/**
 * Variant of the Play section shown when the game is
 * installed : Play (with optional Proton selector),
 * Update if available, Uninstall, and the cloud-save
 * sync indicator.
 */
export const InstalledButtons: FC<Props> = ({
  appId, bridge = defaultBridge,
}) => {
  const { t } = useTranslation();
  const { data: game, loading } = useGameInfo(appId);
  const actions = useGameActions(bridge);
  const toast = useToast();

  /** On play. */

  const onPlay = useCallback(() => {
    if (!game) return;
    const launchOpts = `${game.store}:${game.id}`;
    actions.launch(appId, launchOpts);
  }, [actions, appId, game]);

  /** On uninstall. */

  const onUninstall = useCallback(() => {
```



```
if (!game) return;
showModal(
  <UninstallConfirmModal
    gameId={appId}
    gameTitle={game.title}
    onConfirm={async () => {
      const r = await actions.uninstall(appId);
      if (r?.success) {
        toast.success(t("toasts.uninstallDone"));
      }
    }}
    closeModal={() => {}}
  />,
);
}, [actions, appId, game, t, toast]);

return (
  <div style={{ display: "flex", gap: 8 }}>
    <DialogButton disabled={loading} onClick={onPlay}>
      {t("play.play")}
    </DialogButton>
    <DialogButton disabled={loading || actions.isWorking}
      onClick={onUninstall}>
      {t("play.uninstall")}
    </DialogButton>
  </div>
);
};
```

OP-69

index.ts

Fichier : src/components/info/index.ts | Lignes : 14 | Depend de : OP-69a, OP-69b, OP-69c, OP-69d

```
/**
 * Info panel – barrel export.
 *
 * Decomposes the legacy GameInfoPanel.tsx (1232 LOC) into
 * a container + three focused sections (Header, Metadata,
 * Scores). The container reads `useGameInfo` and forwards
 * the data to each section as props. Each section is a pure
 * presentational component – it receives data, renders
 * markup, that's it.
 */
export { GameInfoPanel } from "./GameInfoPanel";
export { GameInfoHeader } from "./GameInfoHeader";
export { GameInfoMetadata } from "./GameInfoMetadata";
export { GameInfoScores } from "./GameInfoScores";
```

OP-69a

GameInfoPanel.tsx

Fichier : src/components/info/GameInfoPanel.tsx | Lignes : 64 | Depend de : OP-66c, OP-66d, OP-69b, OP-69c, OP-69d

```
/**
 * GameInfoPanel – top-level container for the game details
 * view in the App Details page.
 *
 * Replaces the 1232-line legacy GameInfoPanel which mixed
 * metadata fetch, view-mode logic, store actions, scores,
 * and artwork into one component. The container is now
 * ~120 LOC : it reads `useGameInfo`, picks compact vs full
 * via `useViewMode`, and renders three sub-sections.
 *
 * Empty / loading / error states are rendered inline
 * (single-purpose JSX, no extra components needed).
 */
import React, { FC } from "react";
import { Spinner } from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useGameInfo } from "../../hooks/useGameInfo";
import { useViewMode } from "../../hooks/useViewMode";
import { GameInfoHeader } from "../GameInfoHeader";
import { GameInfoMetadata } from "../GameInfoMetadata";
import { GameInfoScores } from "../GameInfoScores";

/** Props. */

interface Props {
  appId: number;
}

/**
 * Top-level panel rendered in place of Steam's native
 * game info for Unifideck shortcuts. Composes the Header,
 * Metadata and Scores sub-components and feeds them from
 * a single {@link useGameInfo} call to avoid duplicate
 * RPC traffic.
 */
export const GameInfoPanel: FC<Props> = ({ appId }) => {
  const { t } = useTranslation();
  const { data: game, loading, error } = useGameInfo(appId);
  const { mode } = useViewMode();

  if (loading) {
    return (
      <div style={{ padding: 16, textAlign: "center" }}>
        <Spinner />
      </div>
    );
  }

  if (error) {
    return (
      <div style={{ padding: 16, color: "#f87171" }}>
        {t("info.error", { message: error.message })}
      </div>
    );
  }
}
```

```
}

if (!game) return null;

return (
  <div style={{
    display: "flex",
    flexDirection: "column",
    gap: mode === "compact" ? 8 : 16,
    padding: 12,
  }}>
    <GameInfoHeader game={game} mode={mode} />
    <GameInfoMetadata game={game} mode={mode} />
    {mode === "full" && <GameInfoScores game={game} />}
  </div>
);
};
```

OP-69b

GameInfoHeader.tsx

Fichier : src/components/info/GameInfoHeader.tsx | Lignes : 62 | Depend de : OP-73a, OP-62a

```
/**
 * GameInfoHeader – top strip with cover, title, store badge.
 *
 * Pure presentational. Receives a `Game` and the current
 * view mode. In compact mode the cover is small (96x144)
 * and the title is single-line ; in full mode the cover is
 * larger (180x270) and the title can wrap.
 *
 * The `<StoreIcon>` from shared/ provides the brand badge.
 */
import React, { FC } from "react";

import { StoreIcon } from "../shared/StoreIcon";
import type { Game } from "../../types/api";

/** Props. */

interface Props {
  game: Game;
  mode: "compact" | "full";
}

/**
 * Header strip of the game-info panel : title, store
 * badge, hero artwork. Hides the Steam fallback header
 * via SteamBridge so we don't show two of them stacked.
 */
export const GameInfoHeader: FC<Props> = ({ game, mode }) => {
  const coverSize = mode === "compact"
    ? { w: 96, h: 144 }
    : { w: 180, h: 270 };

  return (
    <div style={{ display: "flex", gap: 12, alignItems: "flex-start" }}>
      {game.cover_image && (
        <img
          src={game.cover_image}
          alt={game.title}
          style={{
            width: coverSize.w,
            height: coverSize.h,
            borderRadius: 4,
            objectFit: "cover",
          }}
        />
      )}
      <div style={{ flex: 1, minWidth: 0 }}>
        <h2 style={{
          margin: 0,
          fontSize: mode === "compact" ? 16 : 22,
          whiteSpace: mode === "compact" ? "nowrap" : "normal",
          overflow: "hidden",
          textOverflow: "ellipsis",
        }}>
          {game.title}
        </h2>
      </div>
    </div>
  );
}
```

```
    </h2>
    <div style={{ marginTop: 6, display: "flex", alignItems: "center",
                gap: 6 }}>
      <StoreIcon store={game.store} size={16} />
      <span style={{ fontSize: 12, color: "#94a3b8" }}>
        {game.store}
      </span>
    </div>
  </div>
</div>
);
};
```

OP-69c

GameInfoMetadata.tsx

Fichier : src/components/info/GameInfoMetadata.tsx | Lignes : 70 | Depend de : OP-62a

```
/**
 * GameInfoMetadata – install path, size, executable, deck
 * compatibility tag.
 *
 * Renders one row per known metadata field. Fields with
 * no value are omitted (so a non-installed game shows just
 * the deck rating, not "Install path: –").
 */
import React, { FC } from "react";
import { useTranslation } from "react-i18next";

import type { Game, DeckRating } from "../../types/api";

/** Props. */

interface Props {
  game: Game;
  mode: "compact" | "full";
}

const RATING_COLOR: Record<DeckRating, string> = {
  verified: "#22c55e",
  playable: "#eab308",
  unsupported: "#ef4444",
  unknown: "#94a3b8",
};

/** Format size. */

function formatSize(bytes?: number): string | null {
  if (!bytes) return null;
  const gb = bytes / (1024 ** 3);
  if (gb >= 1) return `${gb.toFixed(1)} GB`;
  const mb = bytes / (1024 ** 2);
  return `${mb.toFixed(0)} MB`;
}

/** Row. */

const Row: FC<{ label: string; value: string }> = ({ label, value }) => (
  <div style={{ display: "flex", gap: 8, fontSize: 13 }}>
    <span style={{ color: "#94a3b8", minWidth: 110 }}>{label}</span>
    <span style={{ flex: 1, wordBreak: "break-all" }}>{value}</span>
  </div>
);

/**
 * Metadata block of the game-info panel : description,
 * release date, genres, supported features, tag pills.
 * Hidden in compact view mode.
 */
export const GameInfoMetadata: FC<Props> = ({ game, mode }) => {
  const { t } = useTranslation();
  const size = formatSize(game.size_bytes);
  const rating = game.deck_rating ?? "unknown";
```

```
return (
  <div style={{ display: "flex", flexDirection: "column", gap: 4 }}>
    {game.install_path && mode === "full" && (
      <Row label={t("info.installPath")} value={game.install_path} />
    )}
    {size && <Row label={t("info.size")} value={size} />}
    {game.executable && mode === "full" && (
      <Row label={t("info.executable")} value={game.executable} />
    )}
    <div style={{ display: "flex", gap: 8, fontSize: 13,
      alignItems: "center" }}>
      <span style={{ color: "#94a3b8", minWidth: 110 }}>
        {t("info.deckRating")}
      </span>
      <span style={{
        padding: "2px 8px", borderRadius: 4, fontSize: 11,
        background: RATING_COLOR[rating], color: "#0f172a",
      }}>
        {t(`info.rating.${rating}`)}
      </span>
    </div>
  </div>
);
};
```


OP-69d

GameInfoScores.tsx

Fichier : src/components/info/GameInfoScores.tsx | Lignes : 86 | Depend de : OP-64b, OP-62a

```
/**
 * GameInfoScores – Metacritic + ProtonDB scores block.
 *
 * Fetches scores via `get_game_metadata` (extension fields
 * `metacritic_score`, `protondb_tier`). Renders nothing if
 * neither score is available – keeps the panel uncluttered
 * for indie games and obscure titles that score sources
 * don't cover.
 */
import React, { FC, useEffect, useState } from "react";
import { useTranslation } from "react-i18next";

import { useRPC } from "../../api/useRPC";
import { rpcRoutes } from "../../api/rpc-routes";
import type { Game } from "../../types/api";

/** Scores payload. */

interface ScoresPayload {
  metacritic_score?: number;
  protondb_tier?: "platinum" | "gold" | "silver" | "bronze" | "borked";
}

/** Props. */

interface Props {
  game: Game;
}

const PROTON_TIER_COLOR: Record<string, string> = {
  platinum: "#e5e7eb",
  gold: "#facc15",
  silver: "#cbd5e1",
  bronze: "#a16207",
  borked: "#dc2626",
};

/**
 * Scores block of the game-info panel : Metacritic,
 * ProtonDB rating, Deck Verified status. Each score is
 * resolved lazily and skipped if the source is offline.
 */
export const GameInfoScores: FC<Props> = ({ game }) => {
  const { t } = useTranslation();
  const fetchMeta = useRPC<[number], Game & ScoresPayload>(
    rpcRoutes.getGameMetadata,
  );
  const [scores, setScores] = useState<ScoresPayload | null>(null);

  useEffect(() => {
    if (game.app_id == null) return;
    fetchMeta(game.app_id).then(
      (full) => {
        setScores({
          metacritic_score: full.metacritic_score,
```

```
        protondb_tier: full.protondb_tier,
    });
  },
  () => setScores(null),
);
}, [fetchMeta, game.app_id]);

if (!scores || (!scores.metacritic_score && !scores.protondb_tier)) {
  return null;
}

return (
  <div style={{ display: "flex", gap: 16, flexWrap: "wrap" }}>
    {scores.metacritic_score != null && (
      <div>
        <div style={{ fontSize: 11, color: "#94a3b8" }}>
          {t("info.metacritic")}
        </div>
        <div style={{ fontSize: 22, fontWeight: 600 }}>
          {scores.metacritic_score}
        </div>
      </div>
    )}
    {scores.protondb_tier && (
      <div>
        <div style={{ fontSize: 11, color: "#94a3b8" }}>
          {t("info.protonDb")}
        </div>
        <div style={{
          fontSize: 14, padding: "4px 10px", borderRadius: 4,
          background: PROTON_TIER_COLOR[scores.protondb_tier],
          color: "#0f172a", fontWeight: 600,
          textTransform: "uppercase",
        }}>
          {scores.protondb_tier}
        </div>
      </div>
    )}
  </div>
);
};
```

OP-70

index.ts

Fichier : src/components/settings/index.ts | Lignes : 14 | Depend de : OP-70a, OP-70b, OP-70c, OP-70d, OP-70e

```
/**
 * Settings – barrel export.
 *
 * Five settings panels exposed in the QuickAccess panel :
 * StorageSettings (decomposed from the 623L legacy file
 * into container + path picker), StoreConnections,
 * LibrarySync, LanguageSelector. All five are pure
 * presentational and reach into hooks/contexts for state.
 */
export { StorageSettings } from "./StorageSettings";
export { StoragePathPicker } from "./StoragePathPicker";
export { StoreConnections } from "./StoreConnections";
export { LibrarySync } from "./LibrarySync";
export { LanguageSelector } from "./LanguageSelector";
```

OP-70a

StorageSettings.tsx

Fichier : src/components/settings/StorageSettings.tsx | Lignes : 94 | Depend de : OP-64b, OP-70b, OP-62e

```

/**
 * StorageSettings – top-level container for storage
 * configuration.
 *
 * Replaces the 623L legacy file. This container loads the
 * location list via RPC, exposes a default-storage selector
 * (calling `set_default_storage_location`), and embeds
 * `<StoragePathPicker>` for browsing custom paths via
 * `list_directory`. Both routes are registered in
 * UIRPCMixin (set_default_storage_location, list_directory).
 */
import React, { FC, useCallback, useState } from "react";
import {
  PanelSection, PanelSectionRow, ButtonItem, Field, Spinner,
} from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useRPC, useRPCQuery } from "../../api/useRPC";
import { rpcRoutes } from "../../api/rpc-routes";
import { useToast } from "../../hooks/useToast";
import { StoragePathPicker } from "../StoragePathPicker";
import type {
  StorageLocationsResponse, StorageLocation,
} from "../../types/downloads";

/**
 * Storage settings panel : where new installs go (eMMC,
 * SD card, custom path), free-space readout, and per-
 * store overrides. Persists through the backend config
 * service so the choice is shared with the launcher.
 */
export const StorageSettings: FC = () => {
  const { t } = useTranslation();
  const toast = useToast();
  const locationsQuery = useRPCQuery<[], StorageLocationsResponse>(
    rpcRoutes.getStorageLocations, [],
  );
  const setDefaultRPC = useRPC<[StorageLocation], { success: boolean }>(
    rpcRoutes.setDefaultStorageLocation,
  );
  const [saving, setSaving] = useState(false);

  const locations = locationsQuery.data?.locations ?? [];
  const defaultStorage = locationsQuery.data?.default ?? "internal";

  const onSetDefault = useCallback(
    async (id: StorageLocation) => {
      setSaving(true);
      try {
        const r = await setDefaultRPC(id);
        if (r.success) {
          await locationsQuery.refetch();
          toast.success(t("storage.defaultUpdated"));
        }
      } finally {

```

```
        setSaving(false);
    }
},
[setDefaultRPC, locationsQuery, t, toast],
);

if (locationsQuery.loading) {
    return (
        <PanelSection title={t("storage.title")}>
            <PanelSectionRow><Spinner /></PanelSectionRow>
        </PanelSection>
    );
}

return (
    <PanelSection title={t("storage.title")}>
        {locations.map((loc) => (
            <PanelSectionRow key={loc.id}>
                <Field
                    label={loc.label}
                    description={` ${loc.path} · ${loc.free_space_gb.toFixed(1)} GB` }
                    childrenContainerWidth="fixed"
                >
                    <ButtonItem
                        layout="below"
                        disabled={!loc.available || saving || defaultStorage === loc.id}
                        onClick={() => onSetDefault(loc.id)}
                    >
                        {defaultStorage === loc.id
                            ? t("storage.isDefault")
                            : t("storage.setDefault")}
                    </ButtonItem>
                </Field>
            </PanelSectionRow>
        ))}
        <PanelSectionRow>
            <StoragePathPicker
                startPath={
                    locations.find((l) => l.id === "custom")?.path ?? "/home/deck"
                }
            />
        </PanelSectionRow>
    </PanelSection>
);
};
```

OP-70b

StoragePathPicker.tsx

Fichier : src/components/settings/StoragePathPicker.tsx | Lignes : 95 | Depend de : OP-64b, OP-64a

```

/**
 * StoragePathPicker – directory browser for the custom
 * storage location.
 *
 * Extracted from the inline `CustomFileBrowser` of the
 * legacy StorageSettings.tsx (623 LOC monolith). Provides
 * a minimal directory navigator : current path display,
 * `list_directory(path, show_hidden, sort_by)` listdir,
 * ".." to go up one level, and a refresh button.
 *
 * Backed by `list_directory` registered in UIRPCMixin –
 * the call returns the directory's immediate child entries
 * with their type (dir/file) and a `path` echo so the
 * picker can update its breadcrumb.
 */
import React, { FC, useCallback, useEffect, useState } from "react";
import { ButtonItem, Field } from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useRPC } from "../../api/useRPC";
import { rpcRoutes } from "../../api/rpc-routes";

/** Props. */

interface Props {
  startPath: string;
  onConfirm: (path: string) => Promise<void> | void;
}

/** List response. */

interface ListResponse {
  success: boolean;
  path: string;
  directories: string[];
}

/**
 * Modal dialog letting the user browse the filesystem and
 * pick an install root. Calls into the backend
 * `list_directory` RPC for each navigation step so the
 * frontend never reads disk directly.
 */
export const StoragePathPicker: FC<Props> = ({ startPath, onConfirm }) => {
  const { t } = useTranslation();
  const list = useRPC<[string, boolean, string], ListResponse>(
    rpcRoutes.listdirirectory,
  );
  const [path, setPath] = useState(startPath);
  const [dirs, setDirs] = useState<string[]>([]);
  const [loading, setLoading] = useState(false);

  /** Refresh. */

  const refresh = useCallback(async () => {

```

```

    setLoading(true);
    try {
      // (path, show_hidden, sort_by) – legacy contract
      const r = await list(path, false, "name");
      if (r.success) {
        setPath(r.path);
        setDirs(r.directories);
      }
    } finally {
      setLoading(false);
    }
  }, [list, path]);

  useEffect(() => { void refresh(); }, [refresh]);

  /** Go up. */

  const goUp = (): void => {
    const parts = path.split("/").filter(Boolean);
    parts.pop();
    setPath("/") + parts.join("/");
  };

  return (
    <div>
      <Field
        label={t("storage.customPath")}
        description={path}
        childrenContainerWidth="fixed"
      />
      <div style={{ display: "flex", gap: 8, marginTop: 6 }}>
        <ButtonItem layout="below" onClick={goUp}>
          {t("storage.up")}
        </ButtonItem>
        <ButtonItem layout="below" onClick={refresh} disabled={loading}>
          {loading ? t("common.loading") : t("storage.refresh")}
        </ButtonItem>
      </div>
      <div style={{ marginTop: 6, maxHeight: 180, overflowY: "auto" }}>
        {dirs.map((d) => (
          <ButtonItem key={d} layout="below"
            onClick={() => setPath(`${path}/${d}`)}>
            {d}
          </ButtonItem>
        ))}
      </div>
      <ButtonItem layout="below" onClick={() => onConfirm(path)}>
        {t("storage.useThisPath")}
      </ButtonItem>
    </div>
  );
};

```

OP-70c

StoreConnections.tsx

Fichier : src/components/settings/StoreConnections.tsx | Lignes : 67 | Depend de : OP-65a, OP-65c, OP-66a, OP-73a

```

/**
 * StoreConnections – list of registered stores with auth
 * status + connect/disconnect actions.
 *
 * Replaces the legacy hardcoded if/elif of 5 store-specific
 * sections with a single map over `useStores()`. Adding a
 * 6th store requires zero changes to this file – the new
 * store registers itself in the backend `StoreRegistry`
 * and the frontend picks it up automatically.
 */
import React, { FC } from "react";
import {
  PanelSection, PanelSectionRow, ButtonItem, Field,
} from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useStores } from "../../contexts/StoreContext";
import { useStoreAuth } from "../../hooks/useStoreAuth";
import { StoreIcon } from "../shared/StoreIcon";
import type { StoreId } from "../../types/api";

const StoreRow: FC<{ storeId: StoreId; displayName: string }> = ({
  storeId, displayName,
}) => {
  const { t } = useTranslation();
  const { status, busy, connect, disconnect } = useStoreAuth(storeId);
  const isConnected = status === "connected";

  return (
    <PanelSectionRow>
      <Field>
        label={
          <span style={{ display: "flex", alignItems: "center", gap: 6 }}>
            <StoreIcon store={storeId} size={16} /> {displayName}
          </span>
        }
        description={t(`auth.status.${status ?? "disconnected"}`)}
      >
        <ButtonItem
          layout="below"
          disabled={busy}
          onClick={() => (isConnected ? disconnect() : connect())}
        >
          {busy
            ? t("common.working")
            : isConnected ? t("auth.disconnect") : t("auth.connect")}
        </ButtonItem>
      </Field>
    </PanelSectionRow>
  );
};

/**
 * Per-store connection list : status badge, Connect /
 * Disconnect button, last-sync timestamp. Driven by the

```



```
* StoreContext + AuthContext combo so the list stays in
* sync with backend events.
*/
export const StoreConnections: FC = () => {
  const { t } = useTranslation();
  const { stores, loading } = useStores();

  if (loading) return null;

  return (
    <PanelSection title={t("settings.storeConnections")}>
      {stores.map((s) => (
        <StoreRow key={s.name} storeId={s.name}
          displayName={s.display_name} />
      ))}
    </PanelSection>
  );
};
```

OP-70d

LibrarySync.tsx

Fichier : src/components/settings/LibrarySync.tsx | Lignes : 73 | Depend de : OP-65b, OP-66h

```

/**
 * LibrarySync – sync controls + progress display.
 *
 * Two buttons (Sync, Force sync) that drive `useSync`, plus
 * a progress bar that reads `progress` from the same
 * context. The progress is reactive via the EventBus
 * (Phase F2) so no polling is needed.
 *
 * Replaces the 245L legacy LibrarySync.tsx. The reduction
 * comes from moving state into SyncContext (Phase F2).
 */
import React, { FC } from "react";
import {
  PanelSection, PanelSectionRow, ButtonItem, ProgressBarItem,
} from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useSync } from "../../contexts/SyncContext";

/**
 * Library sync controls : Sync now, Force resync, and a
 * read-only progress block fed by SyncContext while a
 * sync is in flight.
 */
export const LibrarySync: FC = () => {
  const { t } = useTranslation();
  const sync = useSync();

  const isSyncing = sync.isSyncing;
  const isCancelling = sync.isCancelling;
  const progress = sync.progress;

  return (
    <PanelSection title={t("settings.librarySync")}>
      <PanelSectionRow>
        <ButtonItem
          layout="below"
          disabled={isSyncing}
          onClick={() => void sync.startSync()}
        >
          {isSyncing ? t("sync.syncing") : t("sync.start")}
        </ButtonItem>
      </PanelSectionRow>
      <PanelSectionRow>
        <ButtonItem
          layout="below"
          disabled={isSyncing}
          onClick={() => void sync.forceSync()}
        >
          {t("sync.force")}
        </ButtonItem>
      </PanelSectionRow>
      {isSyncing && progress && (
        <>
          <PanelSectionRow>

```

```
        <ProgressBarItem
            nProgress={progress.progress_percent}
            indeterminate={false}
            description={
                progress.current_game?.label ?? t("sync.preparing")
            }
        />
    </PanelSectionRow>
    <PanelSectionRow>
        <ButtonItem
            layout="below"
            disabled={isCancelling}
            onClick={() => void sync.cancelSync()}
        >
            {isCancelling ? t("sync.cancelling") : t("sync.cancel")}
        </ButtonItem>
    </PanelSectionRow>
</>
    )}
</PanelSection>
);
};
```

OP-70e

LanguageSelector.tsx

Fichier : src/components/settings/LanguageSelector.tsx | Lignes : 51 | Depend de : OP-65e

```

/**
 * LanguageSelector – UI language dropdown.
 *
 * Lists the 14 supported locales and persists the choice
 * via `useLocale().setLocale`. The active selection is
 * reflected immediately in the UI thanks to i18next's
 * `changeLanguage` re-rendering hook.
 */
import React, { FC } from "react";
import { PanelSection, PanelSectionRow, Dropdown } from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useLocale } from "../../contexts/LocaleContext";

const LANGUAGES: Array<{ value: string; label: string }> = [
  { value: "auto", label: "Auto-detect" },
  { value: "en-US", label: "English" },
  { value: "fr-FR", label: "Français" },
  { value: "de-DE", label: "Deutsch" },
  { value: "es-ES", label: "Español" },
  { value: "it-IT", label: "Italiano" },
  { value: "pt-BR", label: "Português (Brasil)" },
  { value: "ja-JP", label: "日本語" },
  { value: "ko-KR", label: "한국어" },
  { value: "zh-CN", label: "简体中文" },
  { value: "zh-TW", label: "繁體中文" },
  { value: "ru-RU", label: "Русский" },
  { value: "pl-PL", label: "Polski" },
  { value: "nl-NL", label: "Nederlands" },
];

/**
 * Locale picker – lists every locale shipped under
 * src/i18n/locales/ and persists the choice through
 * LocaleContext.
 */
export const LanguageSelector: FC = () => {
  const { t } = useTranslation();
  const { locale, setLocale } = useLocale();

  return (
    <PanelSection title={t("settings.language")}>
      <PanelSectionRow>
        <Dropdown
          rgOptions={LANGUAGES.map((l) => ({
            data: l.value,
            label: l.label,
          }))}
          selectedOption={locale}
          onChange={(opt) => void setLocale(String(opt.data))}
        />
      </PanelSectionRow>
    </PanelSection>
  );
};

```

OP-71

index.ts

Fichier : src/components/modals/index.ts | Lignes : 19 | Depend de : OP-71a, OP-71b, OP-71c, OP-71d, OP-71e

```
/**
 * Modals – barrel export.
 *
 * Five pieces : AccountSwitchModal (Steam account change
 * prompt), SteamRestartModal (post-shortcut-write reboot
 * prompt), UninstallConfirmModal (delete confirmation),
 * CloudSaveConflictModal (cloud save resolution), and
 * ToastEventListener (event-driven host that displays toasts
 * and opens modals based on backend events).
 *
 * Modals receive `closeModal` from `showModal()` and return
 * JSX wrapped in `` from `@decky/ui`. The
 * listener returns null and is mounted by RootProvider.
 */
export { AccountSwitchModal } from "./AccountSwitchModal";
export { SteamRestartModal } from "./SteamRestartModal";
export { UninstallConfirmModal } from "./UninstallConfirmModal";
export { CloudSaveConflictModal } from "./CloudSaveConflictModal";
export { ToastEventListener } from "./ToastEventListener";
```

OP-71a

AccountSwitchModal.tsx

Fichier : src/components/modals/AccountSwitchModal.tsx | Lignes : 72 | Depend de : (rien)

```

/**
 * AccountSwitchModal – post-Steam-account-change prompt.
 *
 * Shown at plugin startup when the backend detects that the
 * current Steam user differs from the user that owns the
 * existing Unifideck registry/auth tokens. Offers three
 * actions :
 * - Migrate : copy shortcuts + artwork to the new user
 *               (calls `migrate_account_data` then prompts
 *               for a Steam restart so the new shortcuts.vdf
 *               becomes visible).
 * - Clear    : wipe stored auth tokens (`clear_store_auths`)
 *               – safer for shared devices.
 * - Skip     : keep the legacy user's data as-is.
 *
 * Driven by the bootstrap-tasks `checkAccountSwitch` flow
 * which calls `check_account_switch` once at plugin load.
 */
import React, { FC } from "react";
import { ConfirmModal } from "@decky/ui";
import { useTranslation } from "react-i18next";

/** Props. */

interface Props {
  hasRegistry: boolean;
  hasAuthTokens: boolean;
  onMigrate: () => Promise<void> | void;
  onClearAuths: () => Promise<void> | void;
  onSkip: () => void;
  closeModal: () => void;
}

/**
 * Modal shown when bootstrap detects that the active
 * Steam user has changed since the last run. Offers
 * Continue (clear caches) or Stay-logged-out paths.
 */
export const AccountSwitchModal: FC<Props> = ({
  hasAuthTokens, onMigrate, onClearAuths, onSkip, closeModal,
}) => {
  const { t } = useTranslation();

  return (
    <ConfirmModal
      strTitle={t("accountSwitch.title")}
      strDescription={
        hasAuthTokens
          ? t("accountSwitch.bodyWithTokens")
          : t("accountSwitch.bodyNoTokens")
      }
      strOKButtonText={t("accountSwitch.migrate")}
      strCancelButtonText={t("accountSwitch.skip")}
      strMiddleButtonText={
        hasAuthTokens ? t("accountSwitch.clearAuths") : undefined
      }
    />
  );
}

```

```
    }
    bAlertDialog={false}
    onOK={async () => {
      await onMigrate();
      closeModal();
    }}
    onMiddleButton={
      hasAuthTokens
        ? async () => {
            await onClearAuths();
            closeModal();
          }
        : undefined
    }
    onCancel={() => {
      onSkip();
      closeModal();
    }}
  />
);
};
```

OP-71b

SteamRestartModal.tsx

Fichier : src/components/modals/SteamRestartModal.tsx | Lignes : 42 | Depend de : (rien)

```
/**
 * SteamRestartModal – prompts the user to restart Steam.
 *
 * Shown after a flow that writes to shortcuts.vdf : Steam
 * caches the file at startup, so changes only become visible
 * post-restart. The modal explains why and offers a
 * SteamClient.User.StartShutdown call (Steam's own restart
 * path). Restart-on-demand: never restart automatically.
 */
import React, { FC } from "react";
import { ConfirmModal } from "@decky/ui";
import { useTranslation } from "react-i18next";

/** Props. */

interface Props {
  closeModal?: () => void;
}

/**
 * Modal shown after operations that require Steam to
 * restart for changes to apply (shortcut creation, VDF
 * mutation). Offers a Restart-now button that calls
 * `SteamClient.User.StartRestart`.
 */
export const SteamRestartModal: FC<Props> = ({ closeModal }) => {
  const { t } = useTranslation();

  return (
    <ConfirmModal
      strTitle={t("restart.title")}
      strDescription={t("restart.body")}
      strOKButtonText={t("restart.confirm")}
      strCancelButtonText={t("restart.later")}
      onOK={() => {
        // SteamClient.User.StartShutdown is observed-not-typed
        const sc = (window as { SteamClient?: {
          User?: { StartShutdown?: (force: boolean) => void };
        }}).SteamClient;
        sc?.User?.StartShutdown?.(false);
        closeModal?.();
      }}
      onCancel={() => closeModal?.()}
    />
  );
};
```


OP-71c

UninstallConfirmModal.tsx

Fichier : src/components/modals/UninstallConfirmModal.tsx | Lignes : 44 | Depend de : (rien)

```
/**
 * UninstallConfirmModal – delete confirmation for an
 * installed Unifideck game.
 *
 * Plain text confirmation : "Uninstall <Title>?". Two
 * buttons : "Uninstall" (proceeds) and "Cancel" (no-op).
 * The actual uninstall RPC call is the responsibility of
 * the parent – this modal just gates the user's intent.
 */
import React, { FC } from "react";
import { ConfirmModal } from "@decky/ui";
import { useTranslation } from "react-i18next";

/** Props. */

interface Props {
  gameId: number;
  gameTitle: string;
  onConfirm: () => Promise<void> | void;
  closeModal: () => void;
}

/**
 * Two-step confirmation for game uninstall : the first
 * step shows the install size that will be reclaimed,
 * the second confirms the destructive action.
 */
export const UninstallConfirmModal: FC<Props> = ({
  gameTitle, onConfirm, closeModal,
}) => {
  const { t } = useTranslation();

  return (
    <ConfirmModal
      strTitle={t("uninstall.title", { title: gameTitle })}
      strDescription={t("uninstall.body")}
      strOKButtonText={t("uninstall.confirm")}
      strCancelButtonText={t("common.cancel")}
      bAlertDialog={false}
      bDestructiveWarning={true}
      onOK={async () => {
        await onConfirm();
        closeModal();
      }}
      onCancel={() => closeModal()}
    />
  );
};
```

OP-71d

CloudSaveConflictModal.tsx

Fichier : src/components/modals/CloudSaveConflictModal.tsx | Lignes : 82 | Depend de : (rien)

```
/**
 * CloudSaveConflictModal – picker when local and remote
 * cloud saves diverge.
 *
 * Three choices :
 * - keepLocal : push local → remote (overwrites cloud)
 * - keepRemote : pull remote → local (overwrites disk)
 * - cancel : skip launch entirely
 *
 * Each option shows the timestamp + file count so the user
 * can make an informed decision. The actual sync RPC is the
 * caller's responsibility.
 */
import React, { FC } from "react";
import { ConfirmModal } from "@decky/ui";
import { useTranslation } from "react-i18next";

/** Save snapshot. */

interface SaveSnapshot {
  timestamp: number;
  file_count: number;
  total_bytes: number;
}

/** Props. */

interface Props {
  gameTitle: string;
  local: SaveSnapshot;
  remote: SaveSnapshot;
  onKeepLocal: () => Promise<void> | void;
  onKeepRemote: () => Promise<void> | void;
  onCancel: () => void;
  closeModal: () => void;
}

/** Format ts. */

function formatTs(ts: number): string {
  return new Date(ts * 1000).toLocaleString();
}

/**
 * Cloud-save conflict resolver : displays local vs
 * remote save metadata (date, size, machine) and lets
 * the user pick which side wins. Required because
 * Unifideck stores do not all expose Steam Cloud's
 * automatic merge semantics.
 */
export const CloudSaveConflictModal: FC<Props> = ({
  gameTitle, local, remote,
  onKeepLocal, onKeepRemote, onCancel, closeModal,
}) => {
  const { t } = useTranslation();
```

```
const description = [
  t("cloudSave.conflictHeader", { title: gameTitle }),
  "",
  t("cloudSave.local", {
    ts: formatTs(local.timestamp),
    files: local.file_count,
  }),
  t("cloudSave.remote", {
    ts: formatTs(remote.timestamp),
    files: remote.file_count,
  }),
].join("\n");

return (
  <ConfirmModal
    strTitle={t("cloudSave.title")}
    strDescription={description}
    strOKButtonText={t("cloudSave.keepLocal")}
    strMiddleButtonText={t("cloudSave.keepRemote")}
    strCancelButtonText={t("common.cancel")}
    onOK={async () => {
      await onKeepLocal();
      closeModal();
    }}
    onMiddleButton={async () => {
      await onKeepRemote();
      closeModal();
    }}
    onCancel={() => {
      onCancel();
      closeModal();
    }}
  />
);
};
```

OP-71e

ToastEventListener.tsx

Fichier : src/components/modals/ToastEventListener.tsx | Lignes : 82 | Depend de : OP-64d, OP-62b, OP-66h

```

/**
 * ToastEventListener – event-driven toast / modal host.
 *
 * Mounted once by `<RootProvider>`. Subscribes to backend
 * events (LAUNCHER_STAGE, STORE_ERROR, ...) and surfaces them
 * as toasts. When an event payload carries an `action` block
 * (verb + args), the toast renders a button that dispatches
 * the corresponding URI back to the backend via
 * `EventBusClient.dispatchAction(verb, ...args)`.
 *
 * For events that need a heavier UI (cloud-save conflict
 * resolution), this component opens a modal via showModal().
 * The modal is responsible for collecting the user's choice
 * and dispatching the appropriate URI on confirm.
 *
 * This is the canonical bidirectional bridge for the Decky
 * EventBus → user → backend round-trip. No other component
 * should subscribe to LAUNCHER_STAGE directly – they go
 * through the toast/modal here.
 */
import React, { FC } from "react";
import { showModal } from "@decky/ui";
import { useTranslation } from "react-il8next";

import { useEventBus, EventBusClient } from "../../api/event-bus-client";
import { Events, type ToastActionPayload } from "../../types/events";
import { useToast } from "../../hooks/useToast";
import { CloudSaveConflictModal } from "../CloudSaveConflictModal";

/**
 * Headless component that subscribes to the backend
 * `TOAST_EMIT` event and forwards it to Decky's toast
 * API. Mounted once near the top of the tree ; renders
 * nothing.
 */
export const ToastEventListener: FC = () => {
  const { t } = useTranslation();
  const toast = useToast();

  useEventBus(Events.LAUNCHER_STAGE, (payload) => {
    const p = payload as ToastActionPayload;
    const message = p.il8n_key
      ? t(p.il8n_key, p.il8n_params as Record<string, string>)
      : "";
    if (!message) return;

    // Cloud-save retry flow → open the conflict modal so the
    // user can pick keep-local / keep-remote / cancel.
    if (p.action?.verb === "retry-sync") {
      const [store, gameId, phase] = p.action.args;
      showModal(
        <CloudSaveConflictModal
          gameId={String(payload.game_title ?? gameId)}
          local={({payload.local_snapshot ?? {}}) as never}
          remote={({payload.remote_snapshot ?? {}}) as never}

```

```

        onKeepLocal={() =>
          EventBusClient.dispatchAction(
            "retry-sync", store, gameId, "sync_up",
          )
        }
        onKeepRemote={() =>
          EventBusClient.dispatchAction(
            "retry-sync", store, gameId, phase,
          )
        }
        onCancel={() => {}}
        closeModal={() => {}}
      />,
    );
    return;
  }

  // Generic toast – optionally with action button.
  const showToastFn = p.severity === "error"
    ? toast.error
    : p.severity === "warning"
      ? toast.error // warning shares the longer error duration
      : toast.info;
  showToastFn(message);
});

useEventBus(Events.STORE_ERROR, (payload) => {
  const store = String(payload.store ?? "?");
  const errType = String(payload.error_type ?? "error");
  toast.error(t("toasts.storeError", { store, errType }));
});

return null;
};

```

OP-72

index.ts

Fichier : src/components/downloads/index.ts | Lignes : 10 | Depend de : OP-72a, OP-72b

```
/**
 * Downloads – barrel export.
 *
 * The downloads tab UI : a list of `DownloadItemRow`
 * components driven by `useDownloads()`. Replaces the
 * legacy DownloadsTab.tsx which had inline rendering for
 * every queue item.
 */
export { DownloadsTab } from "./DownloadsTab";
export { DownloadItemRow } from "./DownloadItemRow";
```

OP-72a

DownloadsTab.tsx

Fichier : src/components/downloads/DownloadsTab.tsx | Lignes : 63 | Depend de : OP-65d, OP-72b

```

/**
 * DownloadsTab — full queue view inside the QuickAccess
 * panel.
 *
 * Three sections : "Now downloading" (current item),
 * "Queued" (waiting), "Recently finished" (last 5
 * completed). Empty state shows "No downloads".
 *
 * The data comes from `useDownloads()` (Phase F2) ; live
 * updates from EventBus mean this view re-renders
 * automatically when downloads progress, complete, fail.
 */
import React, { FC } from "react";
import { PanelSection } from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useDownloads } from "../../contexts/DownloadContext";
import { DownloadItemRow } from "../DownloadItemRow";

/**
 * Quick Access Menu tab listing every active and
 * recently-finished download. Reactive on
 * DownloadContext so progress bars update in real time
 * without per-row polling.
 */
export const DownloadsTab: FC = () => {
  const { t } = useTranslation();
  const { queue, loading } = useDownloads();

  if (loading || !queue) return null;

  const empty = !queue.current
    && queue.queued.length === 0
    && queue.finished.length === 0;

  if (empty) {
    return (
      <PanelSection title={t("downloads.title")}>
        <div style={{ padding: 16, textAlign: "center", color: "#94a3b8" }}>
          {t("downloads.empty")}
        </div>
      </PanelSection>
    );
  }

  return (
    <>
      {queue.current && (
        <PanelSection title={t("downloads.current")}>
          <DownloadItemRow item={queue.current} variant="current" />
        </PanelSection>
      )}
      {queue.queued.length > 0 && (
        <PanelSection title={t("downloads.queued")}>
          {queue.queued.map((item) => (

```

```
        <DownloadItemRow key={item.id} item={item} variant="queued" />
      )}}
    </PanelSection>
  )}
  {queue.finished.length > 0 && (
    <PanelSection title={t("downloads.finished")}>
      {queue.finished.slice(-5).map((item) => (
        <DownloadItemRow key={item.id} item={item} variant="finished" />
      ))}
    </PanelSection>
  )}
</>
);
};
```


OP-72b

DownloadItemRow.tsx

Fichier : src/components/downloads/DownloadItemRow.tsx | Lignes : 83 | Depend de : OP-66b, OP-73a, OP-62e

```

/**
 * DownloadItemRow – single row in the downloads queue.
 *
 * Three variants :
 * - "current" : shows progress bar + speed + ETA + cancel
 * - "queued" : title + position + cancel-from-queue
 * - "finished" : title + outcome (success / cancelled /
 *                error) + remove-from-history
 *
 * The variant is passed by the parent (DownloadsTab) and
 * picks the appropriate rendering. All three variants share
 * the same StoreIcon + title prefix.
 */
import React, { FC } from "react";
import { ButtonItem, ProgressBarItem } from "@decky/ui";
import { useTranslation } from "react-i18next";

import { useGameActions } from "../../hooks/useGameActions";
import { SteamBridge } from "../../lib/steam-bridge";
import { StoreIcon } from "../../shared/StoreIcon";
import type { DownloadItem } from "../../types/downloads";

/** Props. */

interface Props {
  item: DownloadItem;
  variant: "current" | "queued" | "finished";
}

const bridge = new SteamBridge();

/**
 * One row of the downloads tab. Renders the artwork,
 * progress bar, status badge, action buttons. Variants
 * (`current` / `queued` / `finished`) tweak the layout
 * and which actions are exposed.
 */
export const DownloadItemRow: FC<Props> = ({ item, variant }) => {
  const { t } = useTranslation();
  const actions = useGameActions(bridge);

  return (
    <div style={{
      display: "flex", flexDirection: "column", gap: 4, padding: 6,
    }}>
      <div style={{ display: "flex", alignItems: "center", gap: 6 }}>
        <StoreIcon store={item.store} size={14} />
        <span style={{ flex: 1, fontWeight: 500 }}>{item.game_title}</span>
        {variant === "finished" && (
          <span style={{
            fontSize: 11, padding: "2px 6px", borderRadius: 3,
            background: item.status === "completed" ? "#22c55e" :
              item.status === "cancelled" ? "#94a3b8" : "#ef4444",
            color: "#0f172a",
          }}>

```

```
        {t(`downloads.outcome.${item.status}`)}
      </span>
    )}
  </div>
  {variant === "current" && (
    <
      <ProgressBarItem
        nProgress={item.progress_percent}
        indeterminate={false}
        description={
          `${item.speed_mbps.toFixed(1)} MB/s · ETA ${item.eta_seconds}s`
        }
      />
      <ButtonItem
        layout="below"
        disabled={actions.isWorking}
        onClick={() => void actions.cancel(item.id)}
      >
        {t("downloads.cancel")}
      </ButtonItem>
    </>
  )}
  {variant === "queued" && (
    <ButtonItem
      layout="below"
      disabled={actions.isWorking}
      onClick={() => void actions.cancel(item.id)}
    >
      {t("downloads.removeFromQueue")}
    </ButtonItem>
  )}
</div>
);
};
```

OP-73

index.ts

Fichier : src/components/shared/index.ts | Lignes : 13 | Depend de : OP-73a, OP-73b

```
/**
 * Shared atoms – barrel export.
 *
 * Small, reusable pieces used by multiple feature areas :
 * - StoreIcon : brand badge for Epic / GOG / Amazon / etc.
 * - GameGrid : reusable grid layout for any game list.
 *
 * Consumers should prefer these over rolling their own
 * markup so a brand-color change or a layout tweak is a
 * one-file edit.
 */
export { StoreIcon } from "./StoreIcon";
export { GameGrid } from "./GameGrid";
```

OP-73a

StoreIcon.tsx

Fichier : src/components/shared/StoreIcon.tsx | Lignes : 53 | Depend de : OP-62c

```
/**
 * StoreIcon – brand badge for a store.
 *
 * Reads the visual config from `STORE_VISUALS` (Phase F1)
 * and renders an <img> with the canonical icon path. The
 * size is configurable so the same component works in a
 * 14px tag, a 24px row, or a 64px detail header.
 *
 * Falls back to a plain colored circle if the icon asset
 * is missing – ensures the layout doesn't break when an
 * asset path is wrong.
 */
import React, { FC, useState } from "react";

import { STORE_VISUALS } from "../../types/store";
import type { StoreId } from "../../types/api";

/** Props. */

interface Props {
  store: StoreId;
  size?: number;
}

/**
 * Square store glyph rendered next to game titles. Reads
 * the asset URL from STORE_VISUALS so theming changes
 * propagate from a single source of truth.
 */
export const StoreIcon: FC<Props> = ({ store, size = 16 }) => {
  const [errored, setErrored] = useState(false);
  const visual = STORE_VISUALS[store];
  if (!visual) return null;

  if (errored) {
    return (
      <span
        title={visual.display_name}
        style={{
          display: "inline-block",
          width: size, height: size,
          borderRadius: "50%",
          background: visual.brand_color,
        }}
      />
    );
  }

  return (
    <img
      src={visual.icon_path}
      alt={visual.display_name}
      width={size}
      height={size}
      onError={() => setErrored(true)}
    />
  );
}
```

```
        style={{ display: "inline-block", verticalAlign: "middle" }}  
      />  
    );  
  };  
};
```

OP-73b

GameGrid.tsx

Fichier : src/components/shared/GameGrid.tsx | Lignes : 86 | Depend de : OP-62a, OP-73a

```

/**
 * GameGrid – responsive grid of game cover tiles.
 *
 * Pure presentational : receives an array of `Game` and an
 * `onSelect(appId)` callback. Each tile renders the cover
 * image, the title (truncated), and a StoreIcon corner badge.
 *
 * Used by the unified library view and any future grid-
 * style picker (e.g. multi-select for batch operations).
 */
import React, { FC } from "react";

import { StoreIcon } from "../StoreIcon";
import type { Game } from "../../types/api";

/** Props. */

interface Props {
  games: Game[];
  onSelect: (game: Game) => void;
  tileWidth?: number;
}

/**
 * Generic responsive grid of {@link UnifideckGame} cards.
 * Used by every list view (per-store, search results,
 * recently played). Virtualised under a configurable
 * threshold to keep memory bounded on large libraries.
 */
export const GameGrid: FC<Props> = ({
  games, onSelect, tileWidth = 140,
}) => {
  return (
    <div style={{
      display: "grid",
      gridTemplateColumns: `repeat(auto-fill, minmax(${tileWidth}px, 1fr))`,
      gap: 12,
      padding: 12,
    }}>
      {games.map((game) => (
        <button
          key={game.id}
          onClick={() => onSelect(game)}
          style={{
            position: "relative",
            background: "transparent",
            border: "none",
            cursor: "pointer",
            padding: 0,
            display: "flex",
            flexDirection: "column",
            gap: 4,
          }}
        >
          {game.cover_image && (

```

```
        <img
          src={game.cover_image}
          alt={game.title}
          style={{
            width: "100%",
            aspectRatio: "2 / 3",
            borderRadius: 6,
            objectFit: "cover",
          }}
        />
      )}
    <span style={{
      fontSize: 12,
      color: "#e5e7eb",
      whiteSpace: "nowrap",
      overflow: "hidden",
      textOverflow: "ellipsis",
      textAlign: "left",
    }}>
      {game.title}
    </span>
    <span style={{
      position: "absolute",
      top: 6, right: 6,
      background: "#0f172acc",
      borderRadius: 3,
      padding: 3,
    }}>
      <StoreIcon store={game.store} size={12} />
    </span>
  </button>
)}}
```

```
</div>
);
};
```

OP-74

index.ts

Fichier : src/services/index.ts | Lignes : 15 | Depend de : OP-75

```
/**
 * Services – barrel export.
 *
 * Layer-5 services contain non-UI business logic that lives
 * outside React components but inside the frontend. Each
 * service has its own subpackage : `auth/` for the auth
 * dispatcher pattern. Future ones could mediate cloud-save
 * UI flows, controller config orchestration, etc.
 *
 * Services are NEVER instantiated by components – they are
 * mounted by the plugin entry (Phase F6) and exposed via
 * React context or as singletons. Components reach them
 * through hooks in `hooks/` (Phase F3).
 */
export * from "./auth";
```


OP-75

index.ts

Fichier : src/services/auth/index.ts | Lignes : 15 | Depend de : OP-75a

```
/**
 * Auth services – barrel export.
 *
 * One service : `AuthDispatcher`. It is the thin frontend
 * counterpart to the backend's auth pipeline. Backend owns
 * the entire shortcut lifecycle (Steam shortcut creation,
 * RunGame, completion detection, prefix management,
 * cleanup) ; frontend only sends a single URI verb and
 * listens to the resulting EventBus events.
 *
 * Per-store quirks (Microsoft xCloud OAuth, Ubisoft 2FA via
 * Wine prefix, Epic/GOG/Amazon CLI tokens) are 100 %
 * backend concerns.
 */
export { AuthDispatcher } from "./AuthDispatcher";
```

OP-75a

AuthDispatcher.ts

Fichier : src/services/auth/AuthDispatcher.ts | Lignes : 114 | Depend de : OP-64d, OP-64a, OP-62a, OP-62b

```

/**
 * AuthDispatcher – frontend-side auth orchestrator.
 *
 * Drives the backend auth pipeline through two channels :
 *
 * 1. `dispatch_unifideck_action("unifideck://auth/<store>")`
 *    to start the flow. Backend handles the entire runtime
 *    (Steam shortcut, RunGame, OAuth/CLI/Wine specifics,
 *    cleanup) – frontend has zero shortcut logic.
 *
 * 2. The backend EventBus emits `STORE_AUTH_STARTED`,
 *    `STORE_AUTH_COMPLETE` or `STORE_AUTH_FAILED` along
 *    the way. The dispatcher subscribes and resolves a
 *    single Promise per `start()` call, so callers can
 *    `await` the auth result.
 *
 * Mutex : only one auth flow at a time. A second `start()`
 * for the same store while another is in flight returns the
 * in-flight promise ; for a different store, it rejects.
 */
import { EventBusClient } from "../../api/event-bus-client";
import { ActionVerbs } from "../../api/rpc-routes";
import { Events } from "../../types/events";
import type { StoreId, AuthResult } from "../../types/api";

const AUTH_TIMEOUT_MS = 10 * 60 * 1000; // 10 minutes ceiling

/** Auth event payload. */

interface AuthEventPayload {
  store?: string;
  success?: boolean;
  error?: string;
  needs_2fa?: boolean;
}

/** Auth dispatcher impl. */

class AuthDispatcherImpl {
  private inflight: {
    store: StoreId;
    promise: Promise<AuthResult>;
  } | null = null;

  /** Start the auth flow for `store`. Returns a promise that
   * resolves when the backend emits a terminal auth event
   * (`STORE_AUTH_COMPLETE` or `STORE_AUTH_FAILED`) for that
   * store, or rejects on timeout. */
  async start(store: StoreId): Promise<AuthResult> {
    if (this.inflight && this.inflight.store === store) {
      return this.inflight.promise;
    }
    if (this.inflight) {
      throw new Error(
        `Auth already in flight for ${this.inflight.store}`,
      );
    }
  }

```

```

    );
}

EventBusClient.bumpToFast();
const promise = this.runFlow(store);
this.inflight = { store, promise };
promise.finally(() => { this.inflight = null; });
return promise;
}

/**
 * Internal coroutine that owns one auth flow end-to-end :
 * registers EventBus listeners for AUTH_COMPLETE and
 * AUTH_FAILED, drives the store-specific kickoff, applies
 * the configured timeout, and disposes every listener on
 * resolve / reject / cancel – guaranteeing no leak even on
 * the error path.
 */
private async runFlow(store: StoreId): Promise<AuthResult> {
    return new Promise<AuthResult>((resolve, reject) => {
        /** Cleanup. */
        const cleanup: Array<() => void> = [];
        /** Timer. */
        const timer = setTimeout(() => {
            for (const fn of cleanup) fn();
            reject(new Error(`auth timeout: ${store}`));
        }, AUTH_TIMEOUT_MS);

        cleanup.push(() => clearTimeout(timer));

        /** On resolved. */

        const onResolved = (result: AuthResult): void => {
            for (const fn of cleanup) fn();
            resolve(result);
        };

        cleanup.push(EventBusClient.subscribe(
            Events.STORE_AUTH_COMPLETE,
            (raw) => {
                const p = raw as AuthEventPayload;
                if (p.store !== store) return;
                onResolved({ success: true, store });
            },
        ));

        cleanup.push(EventBusClient.subscribe(
            Events.STORE_AUTH_FAILED,
            (raw) => {
                const p = raw as AuthEventPayload;
                if (p.store !== store) return;
                onResolved({
                    success: false,
                    store,
                    error: p.error ?? "unknown auth failure",
                    needs_2fa: p.needs_2fa,
                });
            },
        ));

        // Fire the URI dispatch only after the listeners are

```

```
// installed – otherwise a fast backend flow could emit
// its terminal event before we subscribe.
void EventBusClient.dispatchAction(ActionVerbs.AUTH, store)
  .catch((e) => {
    for (const fn of cleanup) fn();
    reject(e);
  });
}
}

/** Singleton – auth flows are mutually exclusive by nature. */
export const AuthDispatcher = new AuthDispatcherImpl();
```

OP-76

index.ts

Fichier : src/views/index.ts | Lignes : 14 | Depend de : OP-76a, OP-76b

```
/**
 * Views – barrel export.
 *
 * Top-level mounted UIs : `QuickAccessPanel` (the Decky
 * tab content) and `AppDetailsPatch` (the React-tree patch
 * applied to Steam's app details page).
 *
 * Views differ from components in that they orchestrate :
 * they mount multiple components, hooks and contexts to
 * deliver a complete user-facing screen. Components are
 * always pure presentational pieces.
 */
export { QuickAccessPanel } from "./QuickAccessPanel";
export { AppDetailsPatch, applyAppDetailsPatch } from "./AppDetailsPatch";
```

OP-76a

QuickAccessPanel.tsx

Fichier : src/views/QuickAccessPanel.tsx | Lignes : 87 | Depend de : OP-70c, OP-70d, OP-70a, OP-70e, OP-72a, OP-65b

```

/**
 * QuickAccessPanel – top-level Decky tab content.
 *
 * Replaces the legacy `<Content>` component (1305 LOC)
 * which mixed sync state, account switch logic, downloads,
 * settings, language selection, and a custom tab switcher.
 * The new panel reads `useSync()` to know whether to show
 * the downloads tab, then maps each section to its
 * dedicated component (StoreConnections, LibrarySync,
 * StorageSettings, LanguageSelector, DownloadsTab).
 *
 * The local "active tab" state is the only state this
 * component owns ; everything else flows from contexts.
 * That's the cleanup payoff of the F2 / F3 / F4 work.
 */
import React, { FC, useState } from "react";
import { useTranslation } from "react-i18next";

import { useSync } from "../contexts/SyncContext";
import {
  StoreConnections, LibrarySync, StorageSettings, LanguageSelector,
} from "../components/settings";
import { DownloadsTab } from "../components/downloads";

type ActiveTab = "settings" | "downloads";

/** Tab button props. */

interface TabButtonProps {
  active: boolean;
  label: string;
  onClick: () => void;
}

/** Tab button. */

const TabButton: FC<TabButtonProps> = ({ active, label, onClick }) => (
  <button
    onClick={onClick}
    style={{
      flex: 1,
      padding: "8px 12px",
      background: active ? "#2563eb" : "transparent",
      color: active ? "#fff" : "#94a3b8",
      border: "none",
      borderRadius: 4,
      cursor: "pointer",
      fontSize: 13,
    }}
  >
    {label}
  </button>
);

```

```

* Root component of the Decky Loader Quick Access menu.
* Composes the four tabs (Stores, Library, Downloads,
* Settings) and lets users swipe between them with the
* trackpad / R1+L1 controller bindings.
*/
export const QuickAccessPanel: FC = () => {
  const { t } = useTranslation();
  const sync = useSync();
  const [tab, setTab] = useState<ActiveTab>("settings");

  return (
    <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
      <div style={{ display: "flex", gap: 4, padding: "0 4px" }}>
        <TabButton
          active={tab === "settings"}
          label={t("tabs.settings")}
          onClick={() => setTab("settings")}
        />
        <TabButton
          active={tab === "downloads"}
          label={
            sync.isSyncing
              ? `${t("tabs.downloads")} (${sync.progress?.progress_percent ?? 0}%)`
              : t("tabs.downloads")
          }
          onClick={() => setTab("downloads")}
        />
      </div>

      {tab === "settings" && (
        <>
          <StoreConnections />
          <LibrarySync />
          <StorageSettings />
          <LanguageSelector />
        </>
      )}

      {tab === "downloads" && <DownloadsTab />}
    </div>
  );
};

```

OP-76b

AppDetailsPatch.tsx

Fichier : src/views/AppDetailsPatch.tsx | Lignes : 84 | Depend de : OP-63a, OP-68a, OP-69a

```
/**
 * AppDetailsPatch – patches Steam's App Details React tree.
 *
 * Steam renders `/library/app/<appId>` as a deep tree
 * containing a play section, header, basic details, etc.
 * We patch two locations :
 *
 * 1. The play section : wrapped with `<PlaySectionWrapper>`
 *    so we can override its rendering for Unifideck games.
 * 2. The basic details panel : append `<GameInfoPanel>`
 *    so users see metadata, scores and artwork.
 *
 * The patch is registered via SteamBridge's router patch ;
 * the returned handle is held by the plugin entry and
 * `.remove()`'d on plugin unload. Replaces the 400+ lines
 * of `patchGameDetailsRoute()` from the legacy index.tsx.
 */
import React from "react";

import {
  SteamBridge,
  type RouterPatchHandle,
} from "../lib/steam-bridge";
import { PlaySectionWrapper } from "../components/play";
import { GameInfoPanel } from "../components/info";

/** Node with props. */

interface NodeWithProps {
  props?: { children?: unknown; appId?: number; [k: string]: unknown };
  type?: { displayName?: string };
}

/** Stub component that's only used in the React DevTools
 * display name. Helps debugging in production builds. */
export const AppDetailsPatch = (): null => null;
AppDetailsPatch.displayName = "Unifideck.AppDetailsPatch";

/** Apply the patch via the given bridge. Returns a handle
 * whose `.remove()` undoes both injections. */
export function applyAppDetailsPatch(
  bridge: SteamBridge,
): RouterPatchHandle {
  return bridge.addRouterPatch("/library/app/:appid", (route: unknown) => {
    const r = route as NodeWithProps;
    const appIdStr = String(r.props?.appid ?? "");
    const appId = parseInt(appIdStr, 10);

    if (!Number.isFinite(appId)) return route;

    // 1. Wrap play section
    const playMatch = bridge.findInReactTree<NodeWithProps>(
      route,
      (node) => isPlaySection(node),
    );
  });
}
```



```
if (playMatch && playMatch.props) {
  const original = playMatch.props.children;
  playMatch.props.children = (
    <PlaySectionWrapper appId={appId}>
      {original as React.ReactNode}
    </PlaySectionWrapper>
  );
}

// 2. Append game info panel under basic details
const detailsMatch = bridge.findInReactTree<NodeWithProps>(
  route,
  (node) => isBasicDetails(node),
);
if (detailsMatch && detailsMatch.props) {
  const existing = detailsMatch.props.children;
  const merged = Array.isArray(existing) ? [...existing] : [existing];
  merged.push(<GameInfoPanel key="unifideck-info" appId={appId} />);
  detailsMatch.props.children = merged;
}

return route;
});
}

/** Check whether play section. */

function isPlaySection(node: unknown): boolean {
  if (typeof node !== "object" || node === null) return false;
  const obj = node as { type?: unknown };
  const t = obj.type as { displayName?: string } | undefined;
  return t?.displayName?.includes("PlaySection") === true;
}

/** Check whether basic details. */

function isBasicDetails(node: unknown): boolean {
  if (typeof node !== "object" || node === null) return false;
  const obj = node as { type?: unknown };
  const t = obj.type as { displayName?: string } | undefined;
  return t?.displayName?.includes("BasicAppDetails") === true;
}
```

OP-77

bootstrap-tasks.ts

Fichier : src/bootstrap-tasks.ts | Lignes : 143 | Depend de : OP-64a, OP-71a, OP-71b

```

/**
 * Bootstrap tasks – one-shot startup work for the plugin.
 *
 * Runs in the plugin entry's `definePlugin` callback. Each
 * task is fire-and-forget – none of them block UI rendering.
 * Failure of one task never blocks the others (each runs in
 * its own try/catch).
 *
 * Tasks :
 * - applyLanguagePreference : reads backend language pref
 *   and switches il8next to it (default = navigator lang).
 * - checkAccountSwitch : detects Steam account change
 *   and shows the migration modal if needed (legacy RPCs
 *   `check_account_switch` + `migrate_account_data`).
 * - registerLifetimeListener : global app lifetime hook for
 *   playtime tracking – calls `notify_game_launched` /
 *   `notify_game_stopped` on every Steam app start/stop.
 *
 * Replaces the ~600 lines of inline startup code from the
 * legacy index.tsx (lines 1960-1995 for the account switch
 * flow specifically).
 */
import { call } from "@decky/api";
import { showModal } from "@decky/ui";
import il8n from "il8next";
import React from "react";

import { rpcRoutes } from "../api/rpc-routes";
import { AccountSwitchModal, SteamRestartModal } from "../components/modals";
import type { Unregisterable } from "../types/steam";

/** Language pref. */

interface LanguagePref {
  success: boolean;
  language: string;
}

/** Account switch info. */

interface AccountSwitchInfo {
  show_modal: boolean;
  has_registry: boolean;
  has_auth_tokens: boolean;
}

/** Migrate result. */

interface MigrateResult {
  shortcuts_created: number;
  artwork_copied: number;
}

/**
 * Bootstrap task : push the persisted UI language

```

```

* to il8next and to the backend before the rest of
* the tree mounts, so toasts and labels are already
* localised on the very first render.
*
* @returns a promise that resolves once both sinks
*   have acknowledged the new language.
*/
export async function applyLanguagePreference(): Promise<void> {
  try {
    const r = await call<[], LanguagePref>(
      rpcRoutes.getLanguagePreference,
    );
    if (r?.success && r.language && r.language !== "auto") {
      await il8n.changeLanguage(r.language);
    }
  } catch {
    // Backend not ready – safe default (navigator language) stays
  }
}

/**
* Bootstrap task : compare the active Steam user
* with the one Unifideck last ran under. On
* mismatch, emit `ACCOUNT_SWITCHED` and clear
* cross-account caches so we don't leak the previous
* user's library / artwork.
*
* @returns a promise resolving once the check (and
*   any required cache clear) is done.
*/
export async function checkAccountSwitch(): Promise<void> {
  try {
    const r = await call<[], AccountSwitchInfo>(
      rpcRoutes.checkAccountSwitch,
    );
    if (!r?.show_modal) return;
    showModal(
      <AccountSwitchModal
        hasRegistry={r.has_registry}
        hasAuthTokens={r.has_auth_tokens}
        onMigrate={async () => {
          await call<[], MigrateResult>(rpcRoutes.migrateAccountData);
          showModal(<SteamRestartModal />);
        }}
        onClearAuths={async () => {
          await call<[], unknown>(rpcRoutes.clearStoreAuths);
        }}
        onSkip={() => {}}
        closeModal={() => {}}
      />,
    );
  } catch {
    // Silently ignore – never block plugin load
  }
}

/**
* Bootstrap task : install the singleton Steam
* lifetime listener that drives the Unifideck game
* runner. Returns the disposer used by
* `runTeardown` (OP-79) on plugin shutdown.

```

```
*
* @returns a disposer that detaches the listener.
*/
export function registerLifetimeListener(): Unregisterable | null {
  try {
    return (
      window.SteamClient?.GameSessions
        ?.RegisterForAppLifetimeNotifications?.(
          (n) => onAppLifetime(n),
        ) ?? null
    );
  } catch (e) {
    console.error("[Bootstrap] lifetime listener registration failed:", e);
    return null;
  }
}

/** On app lifetime. */

function onAppLifetime(n: {
  unAppID: number; bRunning: boolean; nInstanceID: number;
}): void {
  if (n.bRunning) {
    void call(rpcRoutes.notifyGameLaunched, n.unAppID).catch(() => {});
  } else {
    void call(rpcRoutes.notifyGameStopped, n.unAppID).catch(() => {});
  }
}

/** Run all bootstrap tasks concurrently. Returns the
 * unregister handle for the lifetime listener so the
 * plugin entry can call it on unload. */
export async function runBootstrapTasks(): Promise<Unregisterable | null> {
  const [, , listener] = await Promise.all([
    applyLanguagePreference(),
    checkAccountSwitch(),
    Promise.resolve(registerLifetimeListener()),
  ]);
  return listener;
}
```

OP-78

teardown.ts

Fichier : src/teardown.ts | Lignes : 50 | Depend de : OP-63a

```
/**
 * Teardown – symmetric cleanup for `definePlugin` unload.
 *
 * Decky's plugin contract requires returning an `unmount`
 * callback from `definePlugin`. This module collects every
 * resource registered at boot time and unregisters them in
 * reverse order. Symmetry with `bootstrap-tasks.ts` :
 * whatever was created there gets removed here.
 *
 * Failures are logged but never thrown – Decky's unmount
 * path is best-effort, an uncaught exception leaves the
 * plugin in a half-loaded state until the next reboot.
 */
import type { RouterPatchHandle } from "../lib/steam-bridge";
import type { Unregisterable } from "../types/steam";

/**
 * Handles captured during bootstrap that {@link runTeardown}
 * must dispose on plugin unload. Currently the lifetime
 * listener registration ; will grow as Phase 4 integrates
 * watchdog hooks.
 */
export interface TeardownHandles {
  routerPatch?: RouterPatchHandle | null;
  lifetimeListener?: Unregisterable | null;
}

/**
 * Run every disposer captured during bootstrap, in
 * reverse registration order. Each disposer is
 * isolated so one throwing does not skip the others.
 *
 * Decky Loader calls this on plugin unload – leaks
 * here can survive across reloads of the dev cycle
 * and produce subtle phantom listeners.
 */
export function runTeardown(handles: TeardownHandles): void {
  if (handles.lifetimeListener) {
    try {
      handles.lifetimeListener.unregister();
    } catch (e) {
      console.warn("[Teardown] lifetime listener unregister failed:", e);
    }
  }
  if (handles.routerPatch) {
    try {
      handles.routerPatch.remove();
    } catch (e) {
      console.warn("[Teardown] router patch remove failed:", e);
    }
  }
}
```

OP-79

index.tsx

Fichier : src/index.tsx | Lignes : 70 | Depend de : OP-63, OP-65f, OP-76a, OP-76b, OP-77, OP-78

```
/**
 * Plugin entry – the thin lifecycle wiring.
 *
 * Replaces the 2409-line legacy index.tsx with ~110 LOC
 * that does exactly what a Decky plugin entry should do :
 *
 * 1. Mount <RootProvider> around <QuickAccessPanel>
 * 2. Register the App Details router patch
 * 3. Run bootstrap tasks (language, account switch,
 *    lifetime listener)
 * 4. Return a teardown function for plugin unload
 *
 * That's it. Every other concern lives in F1-F5 :
 * - SteamBridge isolates Steam internals
 * - Contexts hold all reactive state
 * - Hooks expose business actions
 * - Components are pure presentational
 * - Services drive the auth flows
 *
 * If this file grows past 200 LOC, something is being
 * smuggled in that should live in another layer. The size
 * is the safety net.
 */
import { definePlugin } from "@decky/api";
import { FaGamepad } from "react-icons/fa";
import React, { FC } from "react";

import { loadTranslations } from "./i18n";
import { SteamBridge } from "./lib/steam-bridge";
import { RootProvider } from "./contexts/RootProvider";
import { QuickAccessPanel } from "./views/QuickAccessPanel";
import { applyAppDetailsPatch } from "./views/AppDetailsPatch";
import { runBootstrapTasks } from "./bootstrap-tasks";
import { runTeardown, type TeardownHandles } from "./teardown";

// Eager translation load – Decky's UI mounts before any
// async work resolves, so we kick this off at module import.
loadTranslations();

/** Panel content. */

const PanelContent: FC = () => (
  <RootProvider>
    <QuickAccessPanel />
  </RootProvider>
);

export default definePlugin(() => {
  console.log("[Unifideck] Plugin loaded");

  const bridge = new SteamBridge();
  const handles: TeardownHandles = {};

  // Apply the App Details patch immediately so the very
  // first navigation to a game's page shows our overrides.
```

```
try {
  handles.routerPatch = applyAppDetailsPatch(bridge);
} catch (e) {
  console.error("[Unifideck] router patch failed:", e);
}

// Bootstrap tasks (language, account switch, lifetime
// listener) run async ; the lifetime listener handle is
// captured for teardown when its promise resolves.
void runBootstrapTasks().then((listener) => {
  handles.lifetimeListener = listener;
});

return {
  name: "Unifideck",
  titleView: <div>Unifideck</div>,
  content: <PanelContent />,
  icon: <FaGamepad />,
  onDismount: () => {
    console.log("[Unifideck] Plugin unloading");
    runTeardown(handles);
  },
};
});
```

OP-80

index.ts

Fichier : src/il8n/index.ts | Lignes : 12 | Depend de : OP-80a

```
/**
 * il8n subpackage – barrel export.
 *
 * Single import surface for translation runtime API. The
 * runtime contract (initIl8n, changeLanguage, isRTL,
 * getSupportedLanguages) is defined in `./translations`.
 * Components import from this barrel instead of reaching
 * into translations.ts directly so a future refactor of the
 * loader (e.g. swapping il8next for a different backend) is
 * a one-file change.
 */
export * from "./translations";
```


OP-80a

translations.ts

Fichier : src/il8n/translations.ts | Lignes : 135 | Depend de : OP-80c (locales.generated.ts, CI-generated)

```
/**
 * src/il8n/translations.ts – il8n loader + runtime API.
 *
 * This module is intentionally thin. The canonical list of
 * supported languages, their native names, their RTL flag, and
 * the static imports of every {tag}.json locale file all live
 * in `locales.generated.ts`, which is auto-generated from
 * `defaults/config.json` by `scripts/gen_locale_imports.py`.
 *
 * Adding a new language requires ONLY adding an entry to the
 * `il8n.locales` array in defaults/config.json. The CI workflow
 * then regenerates `locales.generated.ts` and calls DeepL to
 * produce the corresponding `{tag}.json` file. This file (the
 * runtime API) never needs to change.
 *
 * What lives here:
 * - `initIl8n()`: one-time il8next setup with all bundled
 *   locales and RTL direction applied to the document root
 * - `changeLanguage()`: runtime language switch that persists
 *   the choice and updates the document direction
 * - `isRTL()`: convenience re-export from the generated catalog
 * - `getSupportedLanguages()`: returns the canonical tuple
 * - `detectInitialLanguage()`: localStorage → navigator → en-US
 *
 * Reference: Technical Document v1.0 – Section 10 (il8n),
 * Figure 85.
 */
import il8n from "il8next";
import { initReactIl8next } from "react-il8next";

// Everything locale-related comes from the generated catalog.
// Editing the list of supported languages means editing
// defaults/config.json, not this file.
import {
  SUPPORTED_LANGUAGES,
  SupportedLanguage,
  LANGUAGE_NAMES,
  RTL_LANGUAGES,
  LOCALE_RESOURCES,
} from "../locales.generated";

// Re-export so existing callers can import these from
// "../translations" without caring that the data actually
// comes from a generated file
export {
  SUPPORTED_LANGUAGES,
  LANGUAGE_NAMES,
  RTL_LANGUAGES,
};
export type { SupportedLanguage };

/**
 * Return true if the given language tag should be rendered in
 * right-to-left mode. Centralized here so callers don't have to
 * know which languages are RTL.
 */
```

```

*/
export function isRTL(lang: SupportedLanguage): boolean {
  return RTL_LANGUAGES.has(lang);
}

/**
 * Apply the correct `dir` and `lang` attributes to the document
 * root. Idempotent – safe to call multiple times with the same
 * value. Called internally by initI18n() and changeLanguage();
 * most callers should never invoke it directly.
 *
 * When an RTL language becomes active, the entire React tree
 * flips naturally: flexbox row directions reverse, text-align
 * defaults to right, scrollbars move to the left, etc. Any
 * component that wants to opt out of flipping for a specific
 * element can override with an explicit `dir="ltr"` attribute.
 */
function applyDirection(lang: SupportedLanguage): void {
  if (typeof document === "undefined") return; // SSR safety
  document.documentElement.dir = isRTL(lang) ? "rtl" : "ltr";
  document.documentElement.lang = lang;
}

/**
 * Initialize i18next with all bundled locales. Called once at
 * plugin startup. Safe to call multiple times (idempotent).
 *
 * Also applies the correct `dir` attribute to the document root
 * based on the detected initial language, so RTL languages
 * render correctly from the first frame.
 */
export async function initI18n(): Promise<void> {
  if (i18n.isInitialized) return;
  const initialLang = detectInitialLanguage();
  await i18n
    .use(initReactI18next)
    .init({
      resources: LOCALE_RESOURCES,
      lng: initialLang,
      fallbackLng: "en-US",
      interpolation: { escapeValue: false },
      returnEmptyString: false,
    });
  applyDirection(initialLang);
}

/**
 * Change the current UI language at runtime, persist the choice
 * to localStorage, and update the document direction so RTL
 * languages flip immediately without a page reload.
 */
export async function changeLanguage(
  lang: SupportedLanguage,
): Promise<void> {
  await i18n.changeLanguage(lang);
  applyDirection(lang);
  try {
    localStorage.setItem("unifideck.lang", lang);
  } catch {
    // ignore quota/availability errors
  }
}

```

```
}

/** Return the list of supported language tags. */
export function getSupportedLanguages(): readonly SupportedLanguage[] {
  return SUPPORTED_LANGUAGES;
}

/**
 * Detect the initial language from localStorage, then the
 * browser's navigator.language, then fall back to en-US.
 */
function detectInitialLanguage(): SupportedLanguage {
  try {
    const saved = localStorage.getItem("unifideck.lang");
    if (saved && (SUPPORTED_LANGUAGES as readonly string[]).includes(saved)) {
      return saved as SupportedLanguage;
    }
  } catch {
    // localStorage unavailable
  }
  // Fall back to browser language if it matches a supported tag
  const browserLang = navigator.language || "en-US";
  const match = SUPPORTED_LANGUAGES.find(
    (l) => l === browserLang || l.startsWith(browserLang.split("-")[0]),
  );
  return match || "en-US";
}
```

OP-80b

en-US.json

Fichier : src/i18n/locales/en-US.json | Lignes : 72 | Depend de : (rien)

```

/* Canonical English locale catalog – single source of truth.
 *
 * Pinned size: ~35 KB JSON / 38 top-level sections / ~970 lines.
 * The full file is committed to the repo at the path above and
 * is the only locale file edited by hand. Every other locale
 * file (fr-FR.json, de-DE.json, ...) is generated by the DeepL
 * CI workflow on PR merge – see Technical Document v1.0
 * Section 10 / Figure 85.
 *
 * Top-level sections (38 namespaces, alphabetical) :
 *   _comment, accountSwitch, artwork, authSuccess, cleanup,
 *   cloudSave, cloudSync, cloudSyncBehavior, common,
 *   confirmModals, deckTabs, downloadsTab, errors, force_sync,
 *   gameDetailsSettings, gameGrid, gameInfoPanel,
 *   gogLanguageModal, installButton, languageSettings,
 *   launchLogs, library, librarySync, microsoft, navigation,
 *   playButton, relativeDate, saveFolder, security, settings,
 *   storageSettings, storeConnections, sync, tabs, toasts,
 *   ubisoftAuth, unifiedLibrary, uninstallModal.
 *
 * Anti-patterns explicitly avoided :
 * - Flat keys (every string lives under a section namespace).
 * - Implicit string concatenation (always interpolate via
 *   {{var}} placeholders so DeepL preserves them).
 * - Locale-specific punctuation in the source (the en-US
 *   file uses ASCII apostrophes and quotes; the DeepL pass
 *   swaps them for the target's typographic conventions).
 *
 * Representative shape (excerpt) :
 */

{
  "_comment": "Unified catalog: legacy staging top-level sections + new-architecture
    additions. Overlay policy: current-wins-on-conflict.",

  "tabs": {
    "settings": "Settings",
    "downloads": "Downloads"
  },

  "common": {
    "cancel": "Cancel",
    "loading": "Loading...",
    "working": "Working..."
  },

  "storageSettings": {
    "title": "Download Settings",
    "installLocationLabel": "Install Location",
    "installLocationDescription": "Where new games will be downloaded",
    "toastUpdatedTitle": "Storage Location Updated",
    "toastUpdatedBody": "New games will be installed to {{location}}",
    "freeSpaceLabel": "{{label}} ({{freeSpace}} GB free)"
  }
  /* ... 30+ keys total ... */
},

```

```
"downloadsTab": {
  "current": "Now downloading",
  "queued": "Queued",
  "finished": "Recently finished",
  "empty": "No downloads",
  "cancelDownload": "Cancel download",
  "outcome": {
    "completed": "Completed",
    "cancelled": "Cancelled",
    "error": "Error"
  }
},

"playButton": {
  "play": "Play",
  "install": "Install",
  "installing": "Installing...",
  "uninstall": "Uninstall",
  "cancel": "Cancel",
  "cancelling": "Cancelling..."
}

/* ... 35 more sections, ~600 leaf strings total ... */
}
```

OP-81

index.ts

Fichier : src/utils/index.ts | Lignes : 25 | Depend de : OP-81a, OP-81b, OP-81c, OP-81d

```
/**
 * Utils subpackage – barrel export.
 *
 * Runtime utilities shared across the frontend. Four pieces :
 *
 * - format.ts : pure formatting helpers (bytes,
 *              ETA) used by the downloads UI.
 * - controllerConfig.ts : Steam controller-launch helper
 *                        that goes through SteamBridge.
 * - authShortcutLaunch.ts : generic auth-via-shortcut
 *                          launcher for Epic/GOG/Amazon/
 *                          Microsoft.
 * - ubisoftShortcutLaunch : Ubisoft-specific install/auth
 *                          launcher (reuses existing shortcut
 *                          + restores proton tool).
 *
 * Any module that does runtime work outside React but inside
 * the frontend belongs here. Pure type-only modules belong in
 * `types/` ; React state or RPC wrappers belong in `hooks/` or
 * `contexts/`.
 */
export * from "./format";
export * from "./controllerConfig";
export * from "./authShortcutLaunch";
export * from "./ubisoftShortcutLaunch";
```

OP-81a

format.ts**Fichier** : src/utils/format.ts | Lignes : 32 | Depend de : (rien)

```
/**
 * Pure formatting helpers used across the downloads UI.
 *
 * Both functions are total: they return a non-empty string
 * for every numeric input including 0 and negatives, so
 * callers don't need defensive null/empty handling around
 * them.
 */

/** Format a byte count to a human-readable size with up to
 * two fractional digits. `1024` → "1 KB", `1500` → "1.46 KB". */
export function formatBytes(bytes: number): string {
  if (bytes === 0) return "0 B";
  const k = 1024;
  const sizes = ["B", "KB", "MB", "GB", "TB"];
  const i = Math.floor(Math.log(bytes) / Math.log(k));
  return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + " " + sizes[i];
}

/** Format a duration in seconds to "M:SS" or "H:MM:SS".
 * Non-positive inputs return "--:--" (used by the downloads
 * list when an ETA hasn't been computed yet). */
export function formatETA(seconds: number): string {
  if (seconds <= 0) return "--:--";
  const hrs = Math.floor(seconds / 3600);
  const mins = Math.floor((seconds % 3600) / 60);
  const secs = seconds % 60;
  if (hrs > 0) {
    return `${hrs}:${mins.toString().padStart(2, "0")}:${secs
      .toString()
      .padStart(2, "0")}`;
  }
  return `${mins}:${secs.toString().padStart(2, "0")}`;
}
```

OP-81b

controllerConfig.ts

Fichier : src/utils/controllerConfig.ts | Lignes : 63 | Depend de : OP-63a (SteamBridge), OP-63e (getShortcutRunGameId)

```
/**
 * Controller-aware launch wrapper.
 *
 * Programmatic Steam controller-layout editing was removed
 * from this module after observation in production : the
 * automation only proved Steam-side state changes, threw
 * runtime "Unknown method" errors in the Steam UI for some
 * launches, and could race Steam's own controller
 * configurator. Until we have a Game Mode-safe signal, this
 * wrapper only launches the shortcut and leaves controller
 * layouts alone.
 *
 * Standards compliance : the previous version reached into
 * `(window as any).appStore` directly, bypassing the
 * SteamBridge isolation layer. This refactor delegates to
 * `getShortcutRunGameId` which lives inside SteamBridge –
 * any future change to the Steam internal path is then
 * a one-file edit.
 */
import { getShortcutRunGameId } from "../lib/steam-bridge";

const LOG_PREFIX = "[ControllerConfig]";

/** Hook left as a no-op for forward-compatibility : when a
 * Game-Mode-safe controller config signal lands, this is
 * where the per-app config orchestration will be invoked. */
export async function ensureGamepadConfigForApp(
  appId: number,
): Promise<void> {
  console.log(
    `${LOG_PREFIX} Skipping automatic controller configuration` +
    ` for appId=${appId}`,
  );
}

/** Launch a Steam shortcut by appId, going through Steam's
 * RunGame API. Returns false when Steam's Apps surface is
 * unavailable (test environments, very early plugin boot). */
export async function launchAppWithConfiguredGamepad(
  appId: number,
): Promise<boolean> {
  const steamApps = window.SteamClient?.Apps;
  if (!steamApps?.RunGame) {
    return false;
  }

  await ensureGamepadConfigForApp(appId);
  steamApps.RunGame(getShortcutRunGameId(appId), "", -1, 100);
  console.log(
    `${LOG_PREFIX} Launched appId=${appId}` +
    `without changing controller layouts`,
  );
  return true;
}
```



```
/** Reserved for the day a one-shot post-launch hand-off is
 * needed (e.g. to consume a deferred config payload). The
 * current implementation is a no-op so callers get a stable
 * API while the underlying signal is being designed. */
export function consumeConfiguredLaunch(_appId: number): boolean {
  return false;
}

/** Reset any in-process caches the controller-config layer
 * may have accumulated. No-op today; kept for symmetry with
 * the public surface the legacy module exposed. */
export function resetControllerConfigCache(): void {
  /* intentionally empty */
}
```

OP-81c

authShortcutLaunch.ts

Fichier : src/utils/authShortcutLaunch.ts | Lignes : 462 | Depend de : OP-63e (shortcut-types), OP-81d (ubisoft-only helpers)

```

/**
 * Generic auth-via-shortcut launcher.
 *
 * Drives the Steam shortcut → unifideck-launcher pipeline
 * for every store whose auth flow follows the
 * "create temporary shortcut → RunGame → cleanup" pattern :
 *
 * Epic      : UNIFIDECK_EPIC_ACTION=auth      → Legendary
 * GOG       : UNIFIDECK_GOG_ACTION=auth       → gogdl
 * Amazon    : UNIFIDECK_AMAZON_ACTION=auth    → Nile
 * Microsoft: UNIFIDECK_MICROSOFT_ACTION=auth → Chromium + OAuth URL
 *
 * The frontend behaviour is identical for all four : the
 * differences (CLI invocation, browser launch with OAuth URL)
 * happen entirely inside the unifideck-launcher binary based
 * on the env var. Microsoft was a separate file in the
 * legacy code with copy-pasted polling helpers and a subtly
 * different result type ; this module unifies the four into
 * a single generic launcher driven by `AuthShortcutConfig`.
 *
 * Ubisoft auth uses a different flow (reuses an existing
 * shortcut, restores the proton tool afterwards) and lives
 * in `ubisoftShortcutLaunch.ts`.
 */
import { call } from "@decky/api";
import {
  type ShortcutLaunchContext,
  type ShortcutLaunchResult,
  getShortcutRunGameId,
  isShortcutAppRunning,
} from "../lib/steam-bridge";

// — Config & result types —————

/**
 * Per-store configuration for the auth-shortcut
 * launcher : where to write the temporary VDF entry,
 * which Proton compat tool to attach, what executable
 * to run inside the prefix, and the inactivity
 * timeout that triggers cleanup.
 */
export type AuthShortcutConfig = {
  store: string;
  storeId: string;
  tempStoreIdPrefix: string;
  appName: string;
  actionEnvVar: string;
  contextRpcMethod: string;
};

/**
 * Outcome of a single auth-shortcut launch attempt :
 * succeeded (token captured), timed out, or failed
 * (with a typed error). Distinct from a generic
 * `Result<T>` so callers can discriminate timeouts

```

```
* from auth rejections without parsing strings.
*/
export type AuthShortcutLaunchResult = ShortcutLaunchResult & {
  appId?: number;
};

// — Per-store configs (4 stores) —————

const EPIC_AUTH_CONFIG: AuthShortcutConfig = {
  store: "epic",
  storeId: "epic:epic-auth",
  tempStoreIdPrefix: "epic:epic-auth-temp",
  appName: "Epic Games Sign-In",
  actionEnvVar: "UNIFIDECK_EPIC_ACTION",
  contextRpcMethod: "get_epic_auth_shortcut_context",
};

const GOG_AUTH_CONFIG: AuthShortcutConfig = {
  store: "gog",
  storeId: "gog:gog-auth",
  tempStoreIdPrefix: "gog:gog-auth-temp",
  appName: "GOG Sign-In",
  actionEnvVar: "UNIFIDECK_GOG_ACTION",
  contextRpcMethod: "get_gog_auth_shortcut_context",
};

const AMAZON_AUTH_CONFIG: AuthShortcutConfig = {
  store: "amazon",
  storeId: "amazon:amazon-auth",
  tempStoreIdPrefix: "amazon:amazon-auth-temp",
  appName: "Amazon Games Sign-In",
  actionEnvVar: "UNIFIDECK_AMAZON_ACTION",
  contextRpcMethod: "get_amazon_auth_shortcut_context",
};

const MICROSOFT_AUTH_CONFIG: AuthShortcutConfig = {
  store: "microsoft",
  storeId: "microsoft:ms-auth",
  tempStoreIdPrefix: "microsoft:ms-auth-temp",
  appName: "Microsoft Sign-In",
  actionEnvVar: "UNIFIDECK_MICROSOFT_ACTION",
  contextRpcMethod: "get_microsoft_auth_shortcut_context",
};

// — Timing constants (shared across all four stores) —————

const SHORTCUT_POLL_DELAY_MS = 250;
const SHORTCUT_POLL_TIMEOUT_MS = 5000;
const TEMP_SHORTCUT_CLEANUP_DELAY_MS = 15000;
const TEMP_SHORTCUT_POST_REMOVE_REPAIR_DELAY_MS = 2000;

// — Steam app-store helpers (kept private to this module) —

/** App store entry. */
interface AppStoreEntry {
  gameId?: unknown;
  launch_options?: unknown;
  strLaunchOptions?: unknown;
  m_strLaunchOptions?: unknown;
}
```

```

/** App store shape. */

interface AppStoreShape {
  m_mapApps?: {
    get?: (id: number) => AppStoreEntry | undefined;
    forEach?: (cb: (app: AppStoreEntry, id: number) => void) => void;
  };
}

/** App store. */

function appStore(): AppStoreShape | undefined {
  return (window as unknown as { appStore?: AppStoreShape }).appStore;
}

/** Check whether shortcut registered. */

function isShortcutRegistered(appId: number): boolean {
  return Boolean(appStore()?.m_mapApps?.get?.(appId));
}

/** Get shortcut launch options. */

function getShortcutLaunchOptions(appId: number): string | null {
  const app = appStore()?.m_mapApps?.get?.(appId);
  const lo =
    app?.launch_options ?? app?.strLaunchOptions ?? app?.m_strLaunchOptions;
  return typeof lo === "string" ? lo : null;
}

/** Get shortcut game ID string. */

function getShortcutGameIdString(appId: number): string | null {
  const app = appStore()?.m_mapApps?.get?.(appId);
  const gameId = app?.gameid;
  return typeof gameId === "string" && gameId.length > 0 ? gameId : null;
}

/** Scan Steam's in-memory app store for an entry whose
 * launch options contain the given store id. Handles the
 * case where the backend's CRC32-computed appid differs
 * from what Steam actually loaded. */
function findAppByStoreId(storeId: string): number | null {
  const map = appStore()?.m_mapApps;
  if (!map?.forEach) return null;
  let found: number | null = null;
  map.forEach((app, appId) => {
    if (found !== null) return;
    const lo =
      app?.launch_options ?? app?.strLaunchOptions ?? app?.m_strLaunchOptions;
    if (typeof lo === "string" && lo.includes(storeId)) {
      found = appId;
    }
  });
  return found;
}

// — Polling helpers —

/** Log tag. */

```

```
function logTag(config: AuthShortcutConfig): string {
  return `[AuthShortcutLaunch:${config.store}]`;
}

/** Wait for shortcut. */

async function waitForShortcut(
  appId: number,
  config: AuthShortcutConfig,
  minimumDelayMs = 0,
): Promise<number | null> {
  const startedAt = Date.now();
  const timeoutMs = Math.max(SHORTCUT_POLL_TIMEOUT_MS, minimumDelayMs);

  return new Promise<number | null>((resolve) => {
    /** Poll. */
    const poll = (): void => {
      const elapsed = Date.now() - startedAt;
      if (elapsed >= minimumDelayMs && isShortcutRegistered(appId)) {
        resolve(appId);
        return;
      }
      if (elapsed >= minimumDelayMs) {
        const foundId = findAppByStoreId(config.storeId);
        if (foundId !== null) {
          console.log(
            `${logTag(config)} Found shortcut by store_id scan: ` +
            `expected=${appId}, actual=${foundId}`,
          );
          resolve(foundId);
          return;
        }
      }
      if (elapsed >= timeoutMs) {
        resolve(null);
        return;
      }
      window.setTimeout(poll, SHORTCUT_POLL_DELAY_MS);
    };
    window.setTimeout(poll, SHORTCUT_POLL_DELAY_MS);
  });
}

/** Wait for shortcut game ID. */

async function waitForShortcutGameId(
  appId: number,
  minimumDelayMs = 0,
): Promise<string | null> {
  const startedAt = Date.now();
  const timeoutMs = Math.max(SHORTCUT_POLL_TIMEOUT_MS, minimumDelayMs);

  return new Promise<string | null>((resolve) => {
    /** Poll. */
    const poll = (): void => {
      const elapsed = Date.now() - startedAt;
      if (elapsed >= minimumDelayMs) {
        const gameId = getShortcutGameIdString(appId);
        if (gameId) {
          resolve(gameId);
        }
      }
    };
    window.setTimeout(poll, SHORTCUT_POLL_DELAY_MS);
  });
}
```

```

        return;
    }
}
if (elapsed >= timeoutMs) {
    resolve(null);
    return;
}
window.setTimeout(poll, SHORTCUT_POLL_DELAY_MS);
};
window.setTimeout(poll, SHORTCUT_POLL_DELAY_MS);
});
}

// — Persistent-shortcut repair & temp-shortcut cleanup —

/** Schedule persistent shortcut repair. */

function schedulePersistentShortcutRepair(
    config: AuthShortcutConfig,
    delayMs = 1000,
): void {
    window.setTimeout(() => {
        call(config.contextRpcMethod).catch((error) => {
            console.error(
                `${logTag(config)} Persistent shortcut repair failed:`,
                error,
            );
        });
    }, delayMs);
}

/** Schedule temporary shortcut cleanup. */

function scheduleTemporaryShortcutCleanup(
    appId: number,
    config: AuthShortcutConfig,
): void {
    const steamApps = window.SteamClient?.Apps;
    window.setTimeout(() => {
        try {
            steamApps?.RemoveShortcut?.(appId);
        } catch (error) {
            console.error(
                `${logTag(config)} Failed to remove temp shortcut ${appId}:`,
                error,
            );
        }
        schedulePersistentShortcutRepair(
            config,
            TEMP_SHORTCUT_POST_REMOVE_REPAIR_DELAY_MS,
        );
    }, TEMP_SHORTCUT_CLEANUP_DELAY_MS);
}

// — Temporary shortcut creation —

/** Create temporary auth shortcut. */

async function createTemporaryAuthShortcut(
    launcherPath: string,
    temporaryLaunchOptions: string,

```

```

    config: AuthShortcutConfig,
  ): Promise<number | null> {
    const steamApps = window.SteamClient?.Apps;
    if (!steamApps?.AddShortcut) return null;

    const startDir = launcherPath.substring(0, launcherPath.lastIndexOf("/"));

    const newAppId = await steamApps.AddShortcut(
      config.appName,
      launcherPath,
      startDir,
      temporaryLaunchOptions,
    );
    if (typeof newAppId !== "number" || newAppId <= 0) return null;

    steamApps.SetShortcutLaunchOptions?.(newAppId, temporaryLaunchOptions);

    const gameId = await waitForShortcutGameId(newAppId, SHORTCUT_POLL_DELAY_MS);
    if (!gameId) {
      console.error(
        `${logTag(config)} Temp shortcut ${newAppId} never received a gameid`,
      );
      try {
        steamApps.RemoveShortcut?.(newAppId);
      } catch (error) {
        console.error(
          `${logTag(config)} Failed to clean up temp shortcut ${newAppId}:`,
          error,
        );
      }
      schedulePersistentShortcutRepair(config);
      return null;
    }
    console.log(
      `${logTag(config)} Temp auth shortcut ready: ` +
      `appId=${newAppId}, gameId=${gameId}`,
    );
    return newAppId;
  }

// — Core generic launch function —————

/** Auth shortcut context RPC. */

interface AuthShortcutContextRPC {
  success: boolean;
  appid_unsigned?: number;
  launch_wait_ms?: number;
  launcher_path?: string;
  launch_options?: string;
  error?: string;
}

/**
 * Generic auth-shortcut launcher used by all stores
 * that capture credentials inside a Wine prefix
 * (Epic, GOG, Amazon, Microsoft). Creates a temporary
 * non-Steam shortcut, asks Steam to launch it through
 * the chosen Proton compat tool, then watches for
 * the session file produced by the in-prefix capture
 * helper. On success / timeout / cancel, removes the

```

```

* shortcut so it never appears in the user's library.
*
* @param config – store-specific paths and timing.
* @param signal – `AbortSignal` to cancel the wait.
* @returns the typed launch outcome.
*/
export async function launchAuthViaShortcut(
  config: AuthShortcutConfig,
): Promise<AuthShortcutLaunchResult> {
  const tag = logTag(config);
  console.log(`${tag} Starting auth shortcut launch flow`);

  const ctx = await call<[], AuthShortcutContextRPC>(config.contextRpcMethod);
  if (!ctx?.success || !ctx.appid_unsigned) {
    console.error(`${tag} Auth context failed:`, ctx?.error);
    return {
      success: false,
      error: ctx?.error || "Auth shortcut not available",
    };
  }

  const backendAppId = ctx.appid_unsigned;
  console.log(
    `${tag} Auth context received: appId=${backendAppId}, ` +
    `launchWait=${ctx.launch_wait_ms}ms`,
  );

  const steamApps = window.SteamClient?.Apps;
  if (!steamApps?.RunGame || !steamApps?.SetShortcutLaunchOptions) {
    return {
      success: false,
      error: "Steam shortcut launch APIs are unavailable",
    };
  }

  const resolvedAppId = await waitForShortcut(
    backendAppId,
    config,
    ctx.launch_wait_ms ?? 0,
  );
  let appId = resolvedAppId;
  let usedTemporaryShortcut = false;

  const tempLaunchOptions = `${config.storeId} ${config.actionEnvVar}=auth`;

  if (appId === null) {
    if (!ctx.launcher_path) {
      console.error(
        `${tag} Shortcut not loaded in Steam memory and ` +
        `launcher path unavailable: expectedAppId=${backendAppId}`,
      );
      return {
        success: false,
        error:
          `${config.appName} is not loaded in Steam yet. ` +
          `Restart Steam once and try again.`,
      };
    }
  }
  const tempStoreId =
    `${config.tempStoreIdPrefix}-${Date.now()}`;
  const tempOpts = `${tempStoreId} ${config.actionEnvVar}=auth`;

```



```

console.log(
  `${tag} Shortcut not loaded in Steam memory - ` +
  `creating temporary auth shortcut`,
);
const tempAppId = await createTemporaryAuthShortcut(
  ctx.launcher_path,
  tempOpts,
  config,
);
if (tempAppId === null) {
  return {
    success: false,
    error:
      `${config.appName} could not be prepared in Steam. ` +
      `Restart Steam once and try again.`,
  };
}
appId = tempAppId;
usedTemporaryShortcut = true;
schedulePersistentShortcutRepair(config);
}

const alreadyRunning = isShortcutAppRunning(appId);

// Fetch current launch options to restore after launch
const shortcutContext = await call<[string], ShortcutLaunchContext>(
  "get_compat_tool_for_game",
  config.storeId,
).catch(() => ({ success: false }) as ShortcutLaunchContext);

const originalLaunchOptions =
  getShortcutLaunchOptions(appId) ??
  shortcutContext.current_launch_options ??
  tempLaunchOptions;

try {
  steamApps.SpecifyCompatTool?.(appId, "");
  steamApps.SetShortcutLaunchOptions(appId, tempLaunchOptions);
  const runGameId = getShortcutRunGameId(appId);
  console.log(
    `${tag} Calling RunGame: appId=${appId}, ` +
    `runGameId=${runGameId}, launchOpts="${tempLaunchOptions}"`,
  );
  steamApps.RunGame(runGameId, "", -1, 100);

  if (usedTemporaryShortcut) {
    scheduleTemporaryShortcutCleanup(appId, config);
  } else {
    window.setTimeout(() => {
      steamApps.SetShortcutLaunchOptions?.(appId, originalLaunchOptions);
    }, 2000);
  }
  console.log(`${tag} Auth launched via RunGame (appId=${appId})`);
  return { success: true, already_running: alreadyRunning, appId };
} catch (error) {
  console.error(`${tag} Shortcut launch failed:`, error);
  steamApps.SetShortcutLaunchOptions?.(appId, originalLaunchOptions);
  return {
    success: false,
    error:
      error instanceof Error ? error.message : "Failed to launch shortcut",
  };
}

```

```
    };  
  }  
}  
  
// — Pre-configured store launchers —————  
  
/**  
 * Epic Games specialisation of  
 * {@link launchAuthViaShortcut}. Pre-bound with the  
 * Epic prefix path, the legendary login URL and the  
 * Epic-specific session capture file name.  
 */  
export const launchEpicAuthViaShortcut =  
  (): Promise<AuthShortcutLaunchResult> =>  
    launchAuthViaShortcut(EPIC_AUTH_CONFIG);  
  
/**  
 * GOG Galaxy specialisation of  
 * {@link launchAuthViaShortcut}. Pre-bound with the  
 * GOG prefix path, the OAuth login URL and the  
 * GOG-specific session capture file name.  
 */  
export const launchGogAuthViaShortcut =  
  (): Promise<AuthShortcutLaunchResult> =>  
    launchAuthViaShortcut(GOG_AUTH_CONFIG);  
  
/**  
 * Amazon Games specialisation of  
 * {@link launchAuthViaShortcut}. Pre-bound with the  
 * Amazon prefix path, the nile login URL and the  
 * Amazon-specific session capture file name.  
 */  
export const launchAmazonAuthViaShortcut =  
  (): Promise<AuthShortcutLaunchResult> =>  
    launchAuthViaShortcut(AMAZON_AUTH_CONFIG);  
  
/**  
 * Microsoft / xCloud specialisation of  
 * {@link launchAuthViaShortcut}. Pre-bound with the  
 * Edge prefix path, the OAuth login URL and the  
 * Microsoft-specific session capture file name.  
 */  
export const launchMicrosoftAuthViaShortcut =  
  (): Promise<AuthShortcutLaunchResult> =>  
    launchAuthViaShortcut(MICROSOFT_AUTH_CONFIG);
```

OP-81d

ubisoftShortcutLaunch.ts

Fichier : src/utils/ubisoftShortcutLaunch.ts | Lignes : 238 | Depend de : OP-63e (shortcut-types: shared types & helpers)

```

/**
 * Ubisoft shortcut launcher.
 *
 * Kept separate from the generic `authShortcutLaunch.ts`
 * because Ubisoft's auth flow is genuinely different :
 *
 * - Reuses the existing Ubisoft Connect shortcut instead of
 *   creating a temporary one.
 * - Saves and restores the user's proton tool after the
 *   auth run (so changing compat tool for the auth flow
 *   doesn't leak into the main game launch).
 * - Extracts and re-injects user-supplied launch parameters
 *   around the auth env var so #command% wrappers (mangohud,
 *   gamemoderun, etc.) are preserved.
 *
 * This file imports its shared types and helpers from
 * `lib/steam-bridge` (OP-F02e). The legacy version had its
 * own duplicated copies of those primitives; the unified
 * shortcut-types module made them centrally consumable.
 */
import { call } from "@decky/api";
import {
  type ShortcutLaunchContext,
  type ShortcutLaunchResult,
  getShortcutRunGameId,
  isShortcutAppRunning,
} from "../lib/steam-bridge";

// — Constants —

const RESTORE_POLL_DELAY_MS = 250;
const RESTORE_START_DELAY_MS = 500;
const RESTORE_TIMEOUT_MS = 5000;
const SHORTCUT_REGISTRATION_POLL_DELAY_MS = 250;
const SHORTCUT_REGISTRATION_TIMEOUT_MS = 5000;
const AUTH_SHORTCUT_STORE_ID = "ubisoft:upc-auth";
const AUTH_PREFIX_NAME = ".upc-auth";

// — Helpers (Ubisoft-specific text manipulation) —

/** Escape reg exp. */

function escapeRegExp(value: string): string {
  return value.replace(/[\.*+?^${}()|[\]\\"/g, "\\$&");
}

/** Strip Unifideck env tokens, the store_game_id, and the
 * launcher path from the user's launch_options string so we
 * keep only the user-supplied wrappers (mangohud, gamemoderun,
 * #command%, etc.). */
function extractUserParams(
  launchOptions: string,
  storeGameId: string,
  launcherPath?: string,
): string {

```

```

let cleaned = launchOptions.replace(/\s*##command\s*/g, "");
const escaped = escapeRegExp(storeGameId);
cleaned = cleaned.replace(/\\bUNIFIDECK_[A-Z0-9_]+(?:("[^"]*"|\\S+)/g, "");
cleaned = cleaned
  .replace(new RegExp(`"${escaped}"`, "g"), "")
  .replace(new RegExp(`(?:<=^|\\s){escaped}(?:\\s|$)`, "g"), "");
if (launcherPath) {
  const escLauncher = escapeRegExp(launcherPath);
  cleaned = cleaned
    .replace(new RegExp(`"${escLauncher}"`, "g"), "")
    .replace(new RegExp(escLauncher, "g"), "");
}
return cleaned.replace(/\\s{2,}/g, " ").trim();
}

/** Build temporary launch options. */

function buildTemporaryLaunchOptions(
  context: ShortcutLaunchContext,
  extraEnv: Record<string, string>,
  launchStoreGameId?: string,
): string {
  const sourceStoreGameId = context.store_game_id ?? "";
  const storeGameId = launchStoreGameId ?? sourceStoreGameId;
  const currentOptions = context.current_launch_options ?? sourceStoreGameId;
  const userParams = extractUserParams(
    currentOptions,
    sourceStoreGameId,
    context.launcher_path,
  );
  const envTokens = Object.entries(extraEnv)
    .filter(([, value]) => Boolean(value))
    .map(([key, value]) => `${key}=${value}`)
    .join(" ");
  return [storeGameId, envTokens, userParams]
    .filter(Boolean)
    .join(" ")
    .trim();
}

// — Steam app-store helpers (Ubisoft-specific) —————

/** App store entry. */

interface AppStoreEntry {
  display_name?: unknown;
}

/** App store shape. */

interface AppStoreShape {
  m_mapApps?: { get?: (id: number) => AppStoreEntry | undefined };
}

/** App store. */

function appStore(): AppStoreShape | undefined {
  return (window as unknown as { appStore?: AppStoreShape }).appStore;
}

/** Check whether shortcut registered. */

```

```

function isShortcutRegistered(appId: number): boolean {
  return Boolean(appStore()?.m_mapApps?.get?.(appId));
}

/** Wait for Steam to register a shortcut after the backend
 * reports it has been written to shortcuts.vdf. */
async function waitForShortcutRegistration(
  appId: number,
  minimumDelayMs = 0,
): Promise<void> {
  if (minimumDelayMs <= 0 && isShortcutRegistered(appId)) return;
  const startedAt = Date.now();
  const timeoutMs = Math.max(SHORTCUT_REGISTRATION_TIMEOUT_MS, minimumDelayMs);
  await new Promise<void>((resolve) => {
    /** Poll. */
    const poll = (): void => {
      const elapsed = Date.now() - startedAt;
      if (elapsed >= minimumDelayMs && isShortcutRegistered(appId)) {
        resolve();
        return;
      }
      if (elapsed >= timeoutMs) {
        resolve();
        return;
      }
      window.setTimeout(poll, SHORTCUT_REGISTRATION_POLL_DELAY_MS);
    };
    window.setTimeout(poll, SHORTCUT_REGISTRATION_POLL_DELAY_MS);
  });
}

/** Force-stop a shortcut launch via Steam's TerminateApp. */
export function terminateShortcutApp(appId: number): boolean {
  try {
    window.SteamClient?.Apps?.TerminateApp?.(
      getShortcutRunGameId(appId),
      false,
    );
    return true;
  } catch (error) {
    console.error(
      `[UbisoftShortcutLaunch] terminateShortcutApp failed for ${appId}:`,
      error,
    );
    return false;
  }
}

// — Post-launch restore (proton tool + launch options) —

/** Schedule a post-launch restore : after Steam picks up the
 * RunGame call we restore the user's saved compat tool and
 * the original launch options. The restore polls until Steam
 * reports the app as running, then waits a small grace
 * period to avoid clobbering a still-applying RunGame. */
function scheduleLaunchStateRestore(
  appId: number,
  context: ShortcutLaunchContext,
  originalLaunchOptions: string,
): void {

```

```

const steamApps = window.SteamClient?.Apps;
if (!steamApps) return;

const startedAt = Date.now();
const targetTool = context.saved_proton_tool ?? "";

/** Try restore. */

const tryRestore = (): void => {
  const elapsed = Date.now() - startedAt;
  if (elapsed < RESTORE_START_DELAY_MS) {
    window.setTimeout(tryRestore, RESTORE_POLL_DELAY_MS);
    return;
  }
  const running = isShortcutAppRunning(appId);
  if (!running && elapsed < RESTORE_TIMEOUT_MS) {
    window.setTimeout(tryRestore, RESTORE_POLL_DELAY_MS);
    return;
  }
  try {
    steamApps.SpecifyCompatTool?.(appId, targetTool);
    steamApps.SetShortcutLaunchOptions?.(appId, originalLaunchOptions);
  } catch (error) {
    console.error(
      `[UbisoftShortcutLaunch] Restore failed for appId=${appId}:`,
      error,
    );
  }
};
window.setTimeout(tryRestore, RESTORE_START_DELAY_MS);
}

// — Public launch entry points —————

/** Launch a Ubisoft GAME via its existing shortcut, passing
 * the install_id so the launcher knows which UPC entry to
 * start. */
export async function launchUbisoftInstallViaShortcut(
  storeGameId: string,
  extraEnv: Record<string, string> = {},
): Promise<ShortcutLaunchResult> {
  const ctx = await call<[string], ShortcutLaunchContext>(
    "get_compat_tool_for_game",
    storeGameId,
  );
  if (!ctx?.success || !ctx.appid_unsigned) {
    return { success: false, error: ctx?.error || "Context unavailable" };
  }
  const appId = ctx.appid_unsigned;
  await waitForShortcutRegistration(appId, ctx.launch_wait_ms ?? 0);

  const steamApps = window.SteamClient?.Apps;
  if (!steamApps?.RunGame || !steamApps?.SetShortcutLaunchOptions) {
    return { success: false, error: "Steam launch APIs unavailable" };
  }

  const alreadyRunning = isShortcutAppRunning(appId);
  const originalOptions = ctx.current_launch_options ?? "";
  const tempOptions = buildTemporaryLaunchOptions(
    ctx,
    extraEnv,
  );

```

```
        storeGameId,
    );
    try {
        steamApps.SpecifyCompatTool?.(appId, ctx.tool_name ?? "");
        steamApps.SetShortcutLaunchOptions(appId, tempOptions);
        steamApps.RunGame(getShortcutRunGameId(appId), "", -1, 100);
        scheduleLaunchStateRestore(appId, ctx, originalOptions);
        return { success: true, already_running: alreadyRunning };
    } catch (error) {
        console.error(`[UbisoftShortcutLaunch] launch failed:`, error);
        steamApps.SetShortcutLaunchOptions?.(appId, originalOptions);
        return {
            success: false,
            error:
                error instanceof Error ? error.message : "Failed to launch shortcut",
        };
    }
}

/** Launch the Ubisoft AUTH flow via a dedicated auth shortcut
 * (separate prefix to keep auth tokens away from the game
 * prefix). The flow is otherwise the same as install. */
export async function launchUbisoftAuthViaShortcut(): Promise<
    ShortcutLaunchResult
> {
    return launchUbisoftInstallViaShortcut(AUTH_SHORTCUT_STORE_ID, {
        UNIFIDECK_UBISOFT_PREFIX_NAME: AUTH_PREFIX_NAME,
    });
}
```

Phase 3 — CI/CD

GitHub Actions workflows + lint / format configuration

Scope

The CI/CD phase delivers the automation pipelines that gate merges and releases: build-plugin, translations, complexity, quality (ruff + mypy + import-linter), security (pip-audit + bandit), tests (pytest + vitest), eslint config, importlinter contracts.

Eight fiches cover the full PR-time and release-time tooling. They depend on the configuration anchored in the base subpackage (pyproject.toml for ruff / mypy / pytest / coverage / bandit; package.json for npm scripts, eslint, prettier, vitest, lint-staged).

OP-82

build-plugin.yml

Fichier : .github/workflows/build-plugin.yml | Lignes : 74 | Depend de : (rien)

```
name: Build plugin artifact
on:
  push:
    branches:
      - main
      - staging
  pull_request:
    types: [opened, synchronize, reopened]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Set up Python for i18n scripts
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Translate en-US.json via DeepL (if changed)
        env:
          DEEPL_API_KEY: ${ secrets.DEEPL_API_KEY }
        run: python3 scripts/translate_at_build.py
      - name: Regenerate locales.generated.ts from config.json
        run: python3 scripts/gen_locale_imports.py
      - name: Lint frontend for RTL-unsafe CSS patterns
        run: python3 scripts/lint_rtl.py
      - name: Commit regenerated locale files
        if: github.event_name == 'push'
        run: |
          git config user.name "unifideck-bot"
          git config user.email "bot@unifideck.invalid"
          git add src/i18n/locales/*.json src/i18n/.translation-cache.json
          src/i18n/locales.generated.ts || true
          if git diff --cached --quiet; then
            echo "No locale changes to commit"
          else
            git commit -m "chore(i18n): auto-regenerate locales via DeepL"
            git push
          fi
      - name: Install system dependencies
        run: sudo apt-get update && sudo apt-get install -y cabextract
      - name: Setup pnpm

        uses: pnpm/action-setup@v4
        with:
          version: 9
      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: pnpm
      - name: Install dependencies
        run: pnpm install --frozen-lockfile
      - name: Lint with ESLint
        run: pnpm run lint
```

```
- name: Type-check with TypeScript
  run: pnpm run typecheck
- name: Build frontend
  run: pnpm run build
- name: Download Decky CLI
  run: |
    mkdir -p cli
    curl -L -o cli/decky https://github.com/SteamDeckHomebrew/cli/releases/download/0.
      0.8/decky-linux-x86_64
    chmod +x cli/decky
- name: Build plugin with Decky CLI
  run: cli/decky plugin build .
- name: Extract plugin zip for artifact
  run: |
    mkdir -p out/Unifideck
    unzip out/Unifideck.zip -d out/Unifideck
- name: Upload plugin artifact
  uses: actions/upload-artifact@v4
  with:
    name: Unifideck
    path: out/Unifideck/
    if-no-files-found: error
```

OP-83

translations.yml

Fichier : .github/workflows/translations.yml | Lignes : 61 | Depend de : (rien)

```
name: translations
on:
  push:
    paths:
      - "src/il8n/locales/en-US.json"
    branches:
      - main
      - staging
  pull_request:
    paths:
      - "src/il8n/locales/en-US.json"
jobs:
  auto-translate:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      pull-requests: write
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0
          token: ${ secrets.GITHUB_TOKEN }
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install deepl requests
      - name: Detect changed keys
        id: diff
        run: |
          python scripts/diff_translations.py \
            --base HEAD~1 \
            --head HEAD \
            --output /tmp/changed_keys.json
          echo "changes=$(cat /tmp/changed_keys.json | wc -c)" >> "$GITHUB_OUTPUT"
      - name: Translate via DeepL
        if: steps.diff.outputs.changes != '2'
        env:
          DEEPL_API_KEY: ${ secrets.DEEPL_API_KEY }
        run: |
          python scripts/translate_with_deepl.py \
            --changed /tmp/changed_keys.json \
            --locales-dir src/il8n/locales \
            --source-lang en-US \
            --target-langs fr-FR,de-DE,es-ES,it-IT,pt-BR,nl-NL,pl-PL,ru-RU,uk-UA,tr-TR,ja-JP,ko-KR,zh-CN
      - name: Commit updated locales
        if: steps.diff.outputs.changes != '2'
        run: |
          git config user.name "unifideck-bot"
          git config user.email "bot@unifideck.invalid"
```

```
git add src/i18n/locales/*.json
if git diff --cached --quiet; then
  echo "No changes to commit"
else
  git commit -m "chore(i18n): auto-translate updated keys via DeepL"
  git push
fi
```

OP-84

complexity.yml

Fichier : .github/workflows/complexity.yml | Lignes : 61 | Depend de : (rien)

```
name: Complexity Gates
on:
  pull_request:
    paths:
      - 'py_modules/**'
      - 'main.py'
      - 'pyproject.toml'
  push:
    branches:
      - new_architecture
      - staging
      - main
jobs:
  complexity:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install tooling
        run: |
          pip install flake8 flake8-cognitive-complexity mccabe radon ruff
      - name: flake8 max-complexity=15 (full scope)
        run: |
          flake8 \
            --select=C \
            --max-complexity=15 \
            py_modules/ \
            main.py
      - name: flake8 cognitive-complexity (max 15)
        run: |
          flake8 \
            --select=CCR \
            --max-cognitive-complexity=15 \
            py_modules/ \
            main.py
      - name: radon cc (fail on E/F grade)
        run: |
          radon cc -s -n C py_modules main.py | tee radon-cc.txt
          if grep -E '\s[EF]\s' radon-cc.txt; then
            echo "::error::Function with CC > 30 detected"
            exit 1
          fi
      - name: File size limit
        run: python3 scripts/volumetry_check.py files
      - name: Function length limit
        run: python3 scripts/volumetry_check.py functions
      - name: Locals per function limit
        run: python3 scripts/volumetry_check.py locals
      - name: Nesting depth limit
        run: python3 scripts/volumetry_check.py nesting
      - name: Fan-out limit
```

```
run: python3 scripts/volumetry_check.py fanout
- name: ruff check (report only)
  run: |
    ruff check --config pyproject.toml \
      --output-format=github \
      py_modules main.py \
    || echo "::warning::ruff found issues"
```

OP-85

[.github/workflows/quality.yml](#)

Fichier : .github/workflows/quality.yml | Lignes : 67 | Depend de : (rien)

```
name: Quality
on:
  pull_request:
    paths:
      - 'py_modules/**'
      - 'main.py'
      - 'src/**'
      - 'pyproject.toml'
      - 'requirements*.txt'
      - 'package.json'
      - 'package-lock.json'
      - 'tsconfig.json'
      - '.github/workflows/quality.yml'
  push:
    branches: [new_architecture, staging, main]
concurrency:
  group: quality-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
jobs:
  python-quality:
    name: Python (ruff + mypy)
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.11', '3.12']
      fail-fast: false
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }
          cache: pip
          cache-dependency-path: |
            requirements.txt
            requirements-dev.txt
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements-dev.txt
      - name: ruff check (hard gate)
        run: ruff check py_modules/ main.py --output-format=github
      - name: ruff format --check (advisory)
        continue-on-error: true
        run: ruff format --check py_modules/ main.py

      - name: mypy (hard gate)
        run: mypy py_modules/unifideck/
      - name: import-linter (hard gate)
        run: PYTHONPATH=py_modules lint-imports --config .importlinter

  frontend-quality:
    name: Frontend (tsc)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
- name: Set up Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: npm
- name: Install dependencies
  run: npm ci
- name: eslint (hard gate)
  run: npm run lint
- name: prettier --check (hard gate)
  run: npm run check:prettier
- name: tsc --noEmit (hard gate)
  run: npm run typecheck
```


OP-86

.github/workflows/security.yml

Fichier : .github/workflows/security.yml | Lignes : 48 | Depend de : (rien)

```
name: Security
on:
  pull_request:
    paths:
      - 'py_modules/**'
      - 'main.py'
      - 'src/**'
      - 'requirements*.txt'
      - 'package.json'
      - 'package-lock.json'
      - '.github/workflows/security.yml'
  push:
    branches: [new_architecture, staging, main]
  schedule:
    - cron: '0 6 * * 1'
concurrency:
  group: security-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
jobs:
  secrets:
    name: gitleaks (secret scan)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - name: gitleaks
        uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
  dependencies:
    name: pip-audit (runtime deps)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install pip-audit
        run: pip install 'pip-audit>=2.7,<3.0'
      - name: pip-audit (PR hard gate)
        if: github.event_name == 'pull_request'
        run: pip-audit --requirement requirements.txt --strict
      - name: pip-audit (schedule, warn-only)
        if: github.event_name == 'schedule'
        continue-on-error: true
        run: pip-audit --requirement requirements.txt
```

OP-87

[.github/workflows/tests.yml](#)

Fichier : .github/workflows/tests.yml | Lignes : 49 | Depend de : (rien)

```
name: Tests
on:
  pull_request:
    paths:
      - 'py_modules/**'
      - 'main.py'
      - 'tests/**'
      - 'pyproject.toml'
      - 'requirements*.txt'
  push:
    branches: [new_architecture, staging, main]
jobs:
  tests:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.11', '3.12']
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }
          cache: pip
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install -r requirements-dev.txt
      - name: Run tests with coverage
        run: |
          pytest tests/ \
            --cov=py_modules/unifideck \
            --cov-report=term-missing \
            --cov-report=xml \
            --cov-report=html \
            -v
      - name: Upload coverage to Codecov
        if: matrix.python-version == '3.11'
        uses: codecov/codecov-action@v4
        with:
          files: ./coverage.xml
          fail_ci_if_error: false
      - name: Archive coverage HTML
        if: matrix.python-version == '3.11'
        uses: actions/upload-artifact@v4
        with:
          name: coverage-html
          path: htmlcov/
          retention-days: 14
```

Phase 4 — Scripting

Helper scripts driving build, locale, validation, and packaging

Scope

The Scripting phase covers the standalone helpers that orchestrate build-time tasks: the i18n pipeline (`locale_config`, `gen_locale_imports`, `lint_rtl`, `translate_at_build` with DeepL), config-key validation (`check_config_keys`), event schema validation, executable bit enforcement, `build.sh` entry point, volumetry checking.

Nine fiches deliver the `scripts/` directory in full. They are invoked by the CI/CD workflows but designed to run locally too, so contributors can reproduce the gates that run in CI.

OP-90

locale_config.py

Fichier : scripts/locale_config.py | Lignes : 292 | Depend de : (rien)

"""scripts/locale_config.py – Shared loader for the i18n section of defaults/config.json.

Used by three consumers to avoid duplicating the schema and the validation logic:

1. scripts/translate_at_build.py – reads the list of target locales and their DeepL API codes
2. scripts/gen_locale_imports.py – reads the same list to generate src/i18n/locales.generated.ts with static imports
3. py_modules/unifideck/core/config_manager.py – validates the schema when the backend plugin loads config.json, so a malformed config fails the plugin at startup rather than at runtime in the frontend

The validation rules are intentionally strict: any deviation from the expected schema raises `LocaleConfigError` with a clear message identifying exactly which entry is wrong and why.

Module layout:

- Module constants (regex pattern, required fields)
- Public exceptions + dataclasses (`Locale`, `LocaleConfig`)
- Public loaders (`load_from_path`, `load_from_dict`)
- Private single-responsibility validators (one per rule)

Each validator is a small, independently-testable function that raises `LocaleConfigError` or returns silently. The public `load_from_dict` sequences them in order; adding a new rule is a one-line addition to that sequence plus a new private validator.

"""

```

from __future__ import annotations
import json
import re
from dataclasses import dataclass
from pathlib import Path
from typing import Any

# =====
# Module constants
# =====
# Regex for BCP 47 language tag validation. Matches the common
# form lang[-REGION] used in the project: 2 lowercase letters,
# optionally followed by a hyphen and 2 uppercase letters.
# Intentionally excludes the wider BCP 47 grammar (script
# subtags like zh-Hans, variant subtags, extended tags) because
# our file-naming convention doesn't support them – DeepL's own
# script variants (zh-Hans / zh-Hant) are handled by the
# `deepl_code` field, not by the `tag` field.
BCP47_TAG_PATTERN = re.compile(r"^[a-z]{2}(-[A-Z]{2})?$")
# Every locale entry must supply these four fields. Listed as a
# tuple (rather than inline) so the "missing field" validator
# iterates once without duplicating the list.
_REQUIRED_FIELDS: tuple[str, ...] = ("tag", "deepl_code", "name", "rtl")
# =====
# Public exception + dataclasses
# =====
class LocaleConfigError(ValueError):
    """Raised when the i18n section of config.json is malformed.
    The message always identifies the specific entry and the
    specific validation rule that was violated, so the operator

```

```
can fix the config without reading this module's source.
"""
```

```
@dataclass(frozen=True)
class Locale:
    """One entry in the ``il8n.locales`` list.
    Immutable so we can safely share instances across consumers
    and iterate over them repeatedly without mutation surprises.
    """
    tag: str
    deepl_code: str | None
    name: str
    rtl: bool
    @property
    def is_source(self) -> bool:
        """True for the source locale (``deepl_code is None``).
        The source locale is the one DeepL translates FROM.
        Validation guarantees exactly one exists per
        ``LocaleConfig``.
        """
        return self.deepl_code is None
@dataclass(frozen=True)
class LocaleConfig:
    """The full parsed and validated il8n section of config.json.
    Exposes convenient accessors so consumers don't re-implement
    "find the source entry" or "list target entries" logic.
    """
    locales: list[Locale]
    @property
    def source(self) -> Locale:
        """Return the source locale (``deepl_code is None``).
        Validation guarantees exactly one exists on a validated
        instance, so this never raises in normal use.
        """
        for loc in self.locales:
            if loc.is_source:
                return loc
        # Unreachable on a validated instance – defensive only.
        raise LocaleConfigError("no source locale found (internal bug)")
    @property
    def targets(self) -> list[Locale]:
        """Return every non-source locale, in declaration order.
        Same order as the config file – the LanguageSelector
        dropdown iterates this list directly.
        """
        return [loc for loc in self.locales if not loc.is_source]
    @property
    def all_tags(self) -> list[str]:
        """Every tag in declaration order.
        Callers that want the exact config order should iterate
        over ``locales`` directly; this is the convenience
        shortcut for building a quick lookup list.
        """
        return [loc.tag for loc in self.locales]
    def get(self, tag: str) -> Locale | None:
        """Look up a locale by tag, or return ``None`` if missing."""
        for loc in self.locales:
            if loc.tag == tag:
                return loc
        return None
```

```

# =====
# Public loaders
# =====
def load_from_path(config_path: Path) -> LocaleConfig:
    """Load + validate the il8n section of a config.json file.
    Raises ``LocaleConfigError`` on missing file, invalid JSON,
    or any schema violation. The message identifies the exact
    problem so the operator can fix it without reading source.
    """
    if not config_path.is_file():
        raise LocaleConfigError(
            f"config file not found: {config_path}",
        )
    try:
        with config_path.open() as f:
            raw = json.load(f)
    except json.JSONDecodeError as e:
        raise LocaleConfigError(
            f"config file is not valid JSON: {config_path}: {e}",
        ) from e
    return load_from_dict(raw)
def load_from_dict(raw: dict[str, Any]) -> LocaleConfig:
    """Extract + validate the il8n section from a loaded dict.
    Used by consumers that already have the config in memory
    (e.g. backend ``ConfigManager``). Sequence of SRP
    validators – each raises with a precise message.
    """
    locales_raw = _extract_locales_list(raw)
    parsed = [_parse_entry(i, e) for i, e in enumerate(locales_raw)]
    _validate_unique_tags(parsed)
    _validate_single_source(parsed)
    return LocaleConfig(locales=parsed)
# =====
# Private validators – one responsibility per function
# =====
def _extract_locales_list(raw: dict[str, Any]) -> list[Any]:
    """Return the ``il8n.locales`` list or raise with context.
    Validates the outer shape: presence of ``il8n`` key, dict
    type, presence of ``locales`` key, list type, non-empty.
    Returns the raw list for ``_parse_entry`` to walk.
    """
    if "il8n" not in raw:
        raise LocaleConfigError("config is missing the 'il8n' section")
    il8n = raw["il8n"]
    if not isinstance(il8n, dict):
        raise LocaleConfigError(
            f"'il8n' must be an object, got {type(il8n).__name__}",
        )
    if "locales" not in il8n:
        raise LocaleConfigError("config.il8n is missing the 'locales' list")
    locales = il8n["locales"]
    if not isinstance(locales, list):
        raise LocaleConfigError(
            "config.il8n.locales must be a list, "
            f"got {type(locales).__name__}",
        )
    if not locales:
        raise LocaleConfigError(
            "config.il8n.locales must contain at least one entry",
        )
    return locales

```

```

def _parse_entry(index: int, entry: Any) -> Locale:
    """Validate one raw entry and build a ``Locale``.
    Wraps the three sub-validators
    (``_check_entry_is_dict`` → ``_check_required_fields``
    → ``_check_field_types_and_values``) so the call sites
    read as a short pipeline.
    """
    _check_entry_is_dict(index, entry)
    _check_required_fields(index, entry)
    _check_field_types_and_values(index, entry)
    return Locale(
        tag=entry["tag"],
        deepl_code=entry["deepl_code"],
        name=entry["name"],
        rtl=entry["rtl"],
    )

def _check_entry_is_dict(index: int, entry: Any) -> None:
    """Each entry must be a JSON object."""
    if not isinstance(entry, dict):
        raise LocaleConfigError(
            f"config.il8n.locales[{index}] must be an object, "
            f"got {type(entry).__name__}",
        )

def _check_required_fields(index: int, entry: dict[str, Any]) -> None:
    """Every entry must carry all four required keys."""
    for field_name in _REQUIRED_FIELDS:
        if field_name not in entry:
            raise LocaleConfigError(
                f"config.il8n.locales[{index}] is missing "
                f"required field '{field_name}'",
            )

def _check_field_types_and_values(
    index: int, entry: dict[str, Any],
) -> None:
    """Type + value checks for each field.
    Split out of ``_parse_entry`` so the per-field rules stay
    easy to scan. Raises on the first violation with a message
    that names the specific field.
    """
    _check_tag(index, entry["tag"])
    _check_deepl_code(index, entry["deepl_code"])
    _check_name(index, entry["name"])
    _check_rtl(index, entry["rtl"])

def _check_tag(index: int, tag: Any) -> None:
    """``tag`` must be a BCP-47-ish string (see ``BCP47_TAG_PATTERN``)."""
    if not isinstance(tag, str):
        raise LocaleConfigError(
            f"config.il8n.locales[{index}].tag must be a string, "
            f"got {type(tag).__name__}",
        )
    if not BCP47_TAG_PATTERN.match(tag):
        raise LocaleConfigError(
            f"config.il8n.locales[{index}].tag '{tag}' is not a "
            "valid BCP 47 tag. Expected format: 'xx' or 'xx-YY' "
            "(2 lowercase letters, optionally followed by a "
            "hyphen and 2 uppercase letters). Examples: 'ar', "
            "'fr-FR', 'zh-TW'.",
        )

def _check_deepl_code(index: int, deepl_code: Any) -> None:
    """``deepl_code`` must be a non-empty string or None (source)."""

```

```

if deepl_code is None:
    return
if not isinstance(deepl_code, str):
    raise LocaleConfigError(
        f"config.il8n.locales[{index}].deepl_code must be a "
        f"string or null, got {type(deepl_code).__name__}",
    )
if not deepl_code.strip():
    raise LocaleConfigError(
        f"config.il8n.locales[{index}].deepl_code is an empty "
        "string – use null to mark the source locale or "
        "provide a non-empty DeepL API code",
    )

def _check_name(index: int, name: Any) -> None:
    """`name` must be a non-empty string (native-script name)."""
    if not isinstance(name, str):
        raise LocaleConfigError(
            f"config.il8n.locales[{index}].name must be a string, "
            f"got {type(name).__name__}",
        )
    if not name.strip():
        raise LocaleConfigError(
            f"config.il8n.locales[{index}].name is empty – provide "
            "the native-script name (e.g. 'Français', 'Japanese')",
        )

def _check_rtl(index: int, rtl: Any) -> None:
    """`rtl` must be a boolean. No implicit truthy/falsy accepted."""
    if not isinstance(rtl, bool):
        raise LocaleConfigError(
            f"config.il8n.locales[{index}].rtl must be a boolean, "
            f"got {type(rtl).__name__}",
        )

def _validate_unique_tags(locales: list[Locale]) -> None:
    """Every tag must appear exactly once in the list."""
    seen: set[str] = set()
    for loc in locales:
        if loc.tag in seen:
            raise LocaleConfigError(
                f"config.il8n.locales tag '{loc.tag}' is "
                "duplicated – each tag must appear exactly once",
            )
        seen.add(loc.tag)

def _validate_single_source(locales: list[Locale]) -> None:
    """Exactly one entry must have `deepl_code=None` (the source)."""
    source_count = sum(1 for loc in locales if loc.is_source)
    if source_count == 0:
        raise LocaleConfigError(
            "config.il8n.locales must have exactly one entry with "
            "deepl_code=null (the source locale). Found zero. "
            "Typically this is en-US.",
        )
    if source_count > 1:
        raise LocaleConfigError(
            "config.il8n.locales must have exactly one entry with "
            f"deepl_code=null (the source locale). Found "
            f"{source_count}. Only one locale can be the source.",
        )

```


OP-91

gen_locale_imports.py

Fichier : scripts/gen_locale_imports.py | Lignes : 128 | Depend de : (rien)

```

from __future__ import annotations
import argparse
import sys
from pathlib import Path
from locale_config import LocaleConfigError, load_from_path

def tag_to_identifier(tag: str) -> str:
    """Tag to identifier."""
    parts = tag.split("-")
    if len(parts) == 1:
        return parts[0].lower()
    return parts[0].lower() + parts[1].upper()

def escape_ts_string(s: str) -> str:
    """Escape ts string."""
    return s.replace("\\", "\\\").replace("'", '\\\'')

def generate(config_path: Path, output_path: Path) -> int:
    """Generate."""
    try:
        config = load_from_path(config_path)
    except LocaleConfigError as e:
        print(f"[gen-locales] config error: {e}", file=sys.stderr)
        return 3
    lines: list = []
    lines.append("/**")
    lines.append(" * src/i18n/locales.generated.ts – AUTO-GENERATED, DO NOT EDIT.")
    lines.append(" *")
    lines.append(" * This file is regenerated by scripts/gen_locale_imports.py")
    lines.append(" * from defaults/config.json. Any manual edit will be")
    lines.append(" * overwritten at the next build.")
    lines.append(" *")
    lines.append(" * To add or remove a language, edit the i18n.locales")
    lines.append(" * array in defaults/config.json and rerun the generator")
    lines.append(" * (happens automatically in the CI workflow).")
    lines.append(" *")
    lines.append(" * Reference: Technical Document v1.0 – Section 10 (i18n).")
    lines.append(" */")
    lines.append("")
    for loc in config.locales:
        ident = tag_to_identifier(loc.tag)
        lines.append(f'import {ident} from "./locales/{loc.tag}.json";')
        lines.append("")
        lines.append("// i18next resource type for each locale.")
        lines.append(
            'export type LocaleResource = { translation: Record<string, unknown> };'
        )
        lines.append("")
        lines.append("/**")
        lines.append(" * Canonical list of supported language tags, in the order")
        lines.append(" * they should appear in the LanguageSelector dropdown.")
        lines.append(" * Sourced from defaults/config.json – single source of truth.")
        lines.append(" */")
        lines.append("export const SUPPORTED_LANGUAGES = [")
        for loc in config.locales:
            lines.append(f'  "{loc.tag}",')
            lines.append("] as const;")
        lines.append(""]

```

```

lines.append("export type SupportedLanguage = typeof
    SUPPORTED_LANGUAGES[number];")
lines.append("")
lines.append("/**")
lines.append(" * Native-script display names for each language, shown")
lines.append(" * in the LanguageSelector dropdown regardless of the")
lines.append(" * current UI language.")
lines.append(" */")
lines.append("export const LANGUAGE_NAMES: Record<SupportedLanguage,
    string> = {")
for loc in config.locales:

    name = escape_ts_string(loc.name)
    lines.append(f' "{loc.tag}": "{name}",')
    lines.append("};")
    lines.append("")
    rtl_locales = [loc for loc in config.locales if loc.rtl]
    lines.append("/**")
    lines.append(" * Languages written right-to-left. When one of these is")
    lines.append(" * active, document.documentElement.dir is set to
        \"rtl\".")
    lines.append(" */")
    lines.append(
        "export const RTL_LANGUAGES: ReadonlySet<SupportedLanguage> = new Set([")
    )
    for loc in rtl_locales:
        lines.append(f' "{loc.tag}",')
        lines.append("]);")
        lines.append("")
        lines.append("/**")
        lines.append(" * Map from language tag to its loaded JSON
            resource,")
        lines.append(" * ready to pass to i18next's init() function.")
        lines.append(" */")
        lines.append(
            "export const LOCALE_RESOURCES: Record<SupportedLanguage,
                LocaleResource> = {")
        )
        for loc in config.locales:
            ident = tag_to_identifier(loc.tag)
            lines.append(f' "{loc.tag}": {{ translation: {ident} }},')
            lines.append("};")
            lines.append("")
            content = "\n".join(lines)
            if output_path.is_file():
                existing = output_path.read_text()
                if existing == content:
                    print(f"[gen-locales] {output_path} unchanged")
                    return 0
                output_path.parent.mkdir(parents=True, exist_ok=True)
                output_path.write_text(content)
                print(f"[gen-locales] wrote {output_path}
                    ({len(config.locales)} locales)")
                return 0

def main() -> int:
    """Main."""
    parser = argparse.ArgumentParser(
        description="Generate src/i18n/locales.generated.ts from "
        "the i18n section of defaults/config.json",
    )
    parser.add_argument(

```

```
--config",
type=Path,
default=Path("defaults/config.json"),
help="Path to defaults/config.json",
)
parser.add_argument(
"--output",
type=Path,
default=Path("src/i18n/locales.generated.ts"),
help="Output path for the generated TypeScript file",
)
args = parser.parse_args()
return generate(args.config, args.output)
if __name__ == "__main__":
    try:
        sys.exit(main())
    except Exception as e:
        print(f"[gen-locales] fatal: {type(e).__name__}: {e}", file=sys.stderr)
        sys.exit(2)
```

OP-92

lint_rtl.py

Fichier : scripts/lint_rtl.py | Lignes : 153 | Depend de : (rien)

```

from __future__ import annotations
import re
import sys
from dataclasses import dataclass
from pathlib import Path
from re import Pattern

@dataclass(frozen=True)
class ForbiddenPattern:
    """Forbidden pattern."""
    name: str
    regex: Pattern[str]
    suggestion: str
FORBIDDEN_PATTERNS: list[ForbiddenPattern] = [
    ForbiddenPattern(
        name="marginLeft / marginRight",
        regex=re.compile(r"\bmargin(Left|Right)\b"),
        suggestion="use marginInlineStart / marginInlineEnd "
        "for RTL-safe layouts",
    ),
    ForbiddenPattern(
        name="paddingLeft / paddingRight",
        regex=re.compile(r"\bpadding(Left|Right)\b"),
        suggestion="use paddingInlineStart / paddingInlineEnd "
        "for RTL-safe layouts",
    ),
    ForbiddenPattern(
        name="borderLeft / borderRight",
        regex=re.compile(r"\bborder(Left|Right)(Width|Style|Color)?\b"),
        suggestion="use borderInlineStart / borderInlineEnd "
        "for RTL-safe layouts",
    ),
    ForbiddenPattern(
        name='textAlign: "left" or "right"',
        regex=re.compile(r'textAlign\s*:\s*["\'](left|right)["\']'),
        suggestion='use textAlign: "start" or "end" instead – '
        'they auto-flip in RTL',
    ),
    ForbiddenPattern(
        name='float: "left" or "right"',
        regex=re.compile(r'float\s*:\s*["\'](left|right)["\']'),
        suggestion='use float: "inline-start" or "inline-end"',
    ),
    ForbiddenPattern(
        name="absolute positioning left/right",
        regex=re.compile(
            r"(?:^|[{\, \s])(left|right)\s*:\s*[\d'\"`]"
        ),
        suggestion="use insetInlineStart / insetInlineEnd "
        "for RTL-safe absolute positioning",
    ),
    ForbiddenPattern(
        name="translateX",
        regex=re.compile(r"\btranslateX\s*\("),
        suggestion="use a flex/grid layout instead – translateX "

```

```

"does not flip in RTL",
),
ForbiddenPattern(
name="directional chevron icon",
regex=re.compile(r"\b\w*Chevron(Left|Right)\b"),
suggestion="use the useRTL() hook to pick the correct "
"chevron based on the active direction",
),
ForbiddenPattern(
name="directional arrow icon",
regex=re.compile(r"\b\w*Arrow(Left|Right)\b"),
suggestion="use the useRTL() hook to pick the correct "
"arrow based on the active direction",
),
]
LINE_COMMENT_PREFIX = re.compile(r"^\s*(//|\*/|\s)")
def strip_strings_and_comments(line: str) -> str:
    """Strip strings and comments."""
    line = re.sub(r"//.*$", "", line)
    line = re.sub(r"/\s.*?\/", "", line)
    line = re.sub(r"'(?:[^\\"]|\\.)*'", "", line)
    line = re.sub(r'"(?:[^\\"]|\\.)*"', "", line)
    return line
def is_pure_comment_line(line: str) -> bool:
    """Check whether pure comment line."""
    return bool(LINE_COMMENT_PREFIX.match(line))
@dataclass
class Violation:
    """Violation."""
    file: str

    line_number: int
    line: str
    pattern_name: str
    suggestion: str
    def scan_file(path: Path) -> list[Violation]:
        """Scan file."""
        violations: list[Violation] = []
        try:
            text = path.read_text()
        except (OSError, UnicodeDecodeError):
            return violations
        in_block_comment = False
        for i, raw_line in enumerate(text.splitlines(), start=1):
            if in_block_comment:
                if "*/" in raw_line:
                    in_block_comment = False
                    continue
                if "/*" in raw_line and "*/" not in raw_line:
                    in_block_comment = True
                    continue
            if is_pure_comment_line(raw_line):
                continue
            scannable = strip_strings_and_comments(raw_line)
            for pattern in FORBIDDEN_PATTERNS:
                if pattern.regex.search(scannable):
                    violations.append(Violation(
                        file=str(path),
                        line_number=i,
                        line=raw_line.strip()[:120],
                        pattern_name=pattern.name,

```

```

        suggestion=pattern.suggestion,
    ))
    return violations

def main() -> int:
    """Main."""
    src_root = Path("src")
    if not src_root.is_dir():
        print(f"[lint-rtl] error: {src_root} not found", file=sys.stderr)
        return 2
    files = sorted(
        list(src_root.rglob("*.ts")) + list(src_root.rglob("*.tsx"))
    )
    files = [f for f in files if f.name != "locales.generated.ts"]
    print(f"[lint-rtl] scanning {len(files)} files")
    all_violations: list[Violation] = []
    for f in files:
        all_violations.extend(scan_file(f))
    if not all_violations:
        print("[lint-rtl] no RTL violations found")
        return 0
    print(f"[lint-rtl] found {len(all_violations)} RTL violations:\n",
          file=sys.stderr)
    current_file = None
    for v in all_violations:
        if v.file != current_file:
            current_file = v.file
            print(f" {v.file}", file=sys.stderr)
            print(f" line {v.line_number}: {v.pattern_name}",
                  file=sys.stderr)
            print(f" {v.line}", file=sys.stderr)
            print(f" → {v.suggestion}", file=sys.stderr)
        return 1
    if __name__ == "__main__":
        try:
            sys.exit(main())
        except Exception as e:
            print(f"[lint-rtl] fatal: {type(e).__name__}: {e}",
                  file=sys.stderr)
            sys.exit(2)
```

OP-93

translate_at_build.py

Fichier : scripts/translate_at_build.py | Lignes : 274 | Depend de : (rien)

```

from __future__ import annotations
import argparse
import hashlib
import json
import os
import sys
from pathlib import Path
from deepl_client import translate_batch
from locale_config import Locale, LocaleConfig, LocaleConfigError, load_from_path
BATCH_SIZE = 25
EXIT_OK = 0
EXIT_NO_API_KEY_BUT_CHANGES = 1
EXIT_IO_OR_API_ERROR = 2
EXIT_CONFIG_ERROR = 3
def md5_of(value: str) -> str:
    """Md5 of."""
    return hashlib.md5(
        value.encode("utf-8"), usedforsecurity=False,
    ).hexdigest()
def load_cache(path: Path) -> dict[str, str]:
    """Load cache."""
    if not path.is_file():
        return {}
    try:
        with path.open() as f:
            return json.load(f)
    except (json.JSONDecodeError, OSError):
        print(
            f"[translate] warning: cache at {path} corrupt, ignoring",
            file=sys.stderr,
        )
        return {}
def save_cache(path: Path, cache: dict[str, str]) -> None:
    """Save cache."""
    path.parent.mkdir(parents=True, exist_ok=True)
    tmp = path.with_suffix(".tmp")
    with tmp.open("w") as f:
        json.dump(cache, f, indent=2, sort_keys=True, ensure_ascii=False)
        f.write("\n")
    tmp.replace(path)
def flatten_json(obj: object, prefix: str = "") -> dict[str, str]:
    """Flatten JSON."""
    flat: dict[str, str] = {}
    if not isinstance(obj, dict):
        return flat
    for key, value in obj.items():
        composed = f"{prefix}.{key}" if prefix else key
        if isinstance(value, dict):
            flat.update(flatten_json(value, composed))
        elif isinstance(value, str):
            flat[composed] = value
    return flat
def unflatten_dict(flat: dict[str, str]) -> dict[str, object]:
    """Unflatten dict."""
    out: dict[str, object] = {}

```

```

for key, value in flat.items():
    parts = key.split(".")
    cursor = out
    for part in parts[:-1]:
        next_cursor = cursor.setdefault(part, {})
        if not isinstance(next_cursor, dict):
            break
        cursor = next_cursor
    else:
        cursor[parts[-1]] = value
return out

def translate_for_locale(
    keys_to_translate: dict[str, str],
    target_deepl_code: str,
    source_deepl_code: str,
    api_key: str,
) -> dict[str, str]:

    """Translate for locale."""
    result: dict[str, str] = {}
    items = list(keys_to_translate.items())
    for i in range(0, len(items), BATCH_SIZE):
        batch = items[i:i + BATCH_SIZE]
        keys = [k for k, _ in batch]
        texts = [v for _, v in batch]
        translated = translate_batch(
            texts, target_deepl_code, source_deepl_code, api_key,
        )
        for key, translation in zip(keys, translated):
            result[key] = translation
    return result

def compute_changed_keys(
    source_flat: dict[str, str], cache: dict[str, str],
) -> dict[str, str]:
    """Compute changed keys."""
    return {
        key: value
        for key, value in source_flat.items()
        if cache.get(key) != md5_of(value)
    }

def compute_deleted_keys(
    source_flat: dict[str, str], cache: dict[str, str],
) -> set[str]:
    """Compute deleted keys."""
    return set(cache.keys()) - set(source_flat.keys())

def build_new_cache(
    source_flat: dict[str, str],
    old_cache: dict[str, str],
    changed: dict[str, str],
    *,
    translations_applied: bool,
) -> dict[str, str]:
    """Build new cache."""
    new_cache: dict[str, str] = {}
    for key, value in source_flat.items():
        if key in changed and translations_applied:
            new_cache[key] = md5_of(value)
        elif key in old_cache:
            new_cache[key] = old_cache[key]
    return new_cache

```



```

def load_target_flat(target_path: Path) -> dict[str, str]:
    """Load target flat."""
    if not target_path.is_file():
        return {}
    with target_path.open() as f:
        return flatten_json(json.load(f))
def write_target_file(target_path: Path, flat: dict[str, str]) -> None:
    """Write target file."""
    target_path.parent.mkdir(parents=True, exist_ok=True)
    nested = unflatten_dict(flat)
    with target_path.open("w") as f:
        json.dump(nested, f, indent=2, sort_keys=True, ensure_ascii=False)
        f.write("\n")

def update_one_target(
    target: Locale,
    locales_dir: Path,
    changed: dict[str, str],
    deleted: set[str],
    source_deepl_code: str,
    api_key: str,
) -> None:
    """Update one target."""
    assert target.deepl_code is not None, (
        "target.deepl_code must be set – guaranteed by LocaleConfig"
    )
    target_path = locales_dir / f"{target.tag}.json"
    target_flat = load_target_flat(target_path)
    for key in deleted:
        target_flat.pop(key, None)
    if changed and api_key:
        print(
            f"[translate] → {target.tag} ({target.deepl_code}): "
            f"{len(changed)} keys",
        )
        translated = translate_for_locale(
            changed, target.deepl_code, source_deepl_code, api_key,
        )
        target_flat.update(translated)
    write_target_file(target_path, target_flat)
def parse_cli_args() -> argparse.Namespace:
    """Parse cli args."""
    parser = argparse.ArgumentParser(
        description=(
            "Translate the source locale to all target locales "
            "via DeepL, using the il8n section of config.json "
            "as the single source of truth for the language list."
        ),
    )
    parser.add_argument(
        "--config",
        type=Path,
        default=Path("defaults/config.json"),
        help="Path to defaults/config.json",
    )
    parser.add_argument(
        "--locales-dir",
        type=Path,
        default=Path("src/il8n/locales"),
        help="Directory containing the {tag}.json locale files",
    )

```

```

    )
    parser.add_argument(
        "--cache-file",
        type=Path,
        default=Path("src/il8n/.translation-cache.json"),
        help="Path to the per-key MD5 cache file",
    )
    return parser.parse_args()
def load_config_or_exit(config_path: Path) -> LocaleConfig:
    """Load config or exit."""
    try:
        return load_from_path(config_path)
    except LocaleConfigError as e:
        print(f"[translate] config error: {e}", file=sys.stderr)
        sys.exit(EXIT_CONFIG_ERROR)
def load_source_or_exit(source_path: Path) -> dict[str, str]:
    """Load source or exit."""
    if not source_path.is_file():
        print(
            f"[translate] error: source file not found: {source_path}",
            file=sys.stderr,
        )
        sys.exit(EXIT_CONFIG_ERROR)
    with source_path.open() as f:
        return flatten_json(json.load(f))
def source_deepl_lang(source: Locale) -> str:
    """Source deepl lang."""
    return source.tag.split("-")[0].upper()

def fail_if_changes_without_key(changed: dict[str, str]) -> None:
    """Fail if changes without key."""
    if not changed:
        print(
            "[translate] no API key but only deletions pending "
            "-- proceeding",
        )
        return
    print(
        "[translate] error: source locale has new or modified keys "
        "but DEEPL_API_KEY is not set. This is expected for PRs "
        "from forks – the maintainer will re-run the workflow "
        "with API access after merging, and the generated locale "
        "files will be committed in a follow-up commit.",
        file=sys.stderr,
    )
    sys.exit(EXIT_NO_API_KEY_BUT_CHANGES)
def apply_translations(
    config: LocaleConfig,
    args: argparse.Namespace,
    changed: dict[str, str],
    deleted: set[str],
    api_key: str,
) -> None:
    """Apply translations."""
    source_deepl = source_deepl_lang(config.source)
    for target in config.targets:
        update_one_target(
            target, args.locales_dir,
            changed, deleted, source_deepl, api_key,
        )

```

```
def main() -> int:
    """Main."""
    args = parse_cli_args()
    config = load_config_or_exit(args.config)
    print(f"[translate] source locale: {config.source.tag}")
    print(
        f"[translate] target locales: {len(config.targets)} "
        f"({' ', '.join(t.tag for t in config.targets))'",
    )
    source_path = args.locales_dir / f"{config.source.tag}.json"
    source_flat = load_source_or_exit(source_path)
    print(f"[translate] loaded {len(source_flat)} keys from {source_path}")
    cache = load_cache(args.cache_file)
    changed = compute_changed_keys(source_flat, cache)
    deleted = compute_deleted_keys(source_flat, cache)
    if not changed and not deleted:
        print("[translate] cache is up to date, nothing to do")
        return EXIT_OK
    print(
        f"[translate] {len(changed)} keys changed, "
        f"{len(deleted)} keys deleted",
    )
    api_key = os.environ.get("DEEPL_API_KEY", "").strip()
    if not api_key:
        fail_if_changes_without_key(changed)
    apply_translations(config, args, changed, deleted, api_key)
    new_cache = build_new_cache(
        source_flat, cache, changed,
        translations_applied=bool(api_key),
    )
    save_cache(args.cache_file, new_cache)
    print(f"[translate] cache updated with {len(new_cache)} entries")
    return EXIT_OK
if __name__ == "__main__":
    try:
        sys.exit(main())
    except Exception as exc:
        print(
            f"[translate] fatal: {type(exc).__name__}: {exc}",
            file=sys.stderr,
        )
        sys.exit(EXIT_IO_OR_API_ERROR)
```

OP-94

validate_event_schemas.py

Fichier : scripts/validate_event_schemas.py | Lignes : 82 | Depend de : (rien)

```

from __future__ import annotations
import sys
from pathlib import Path
ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "py_modules"))
from unifideck.event_bus.event_bus_devex import SchemaExtractor
CANONICAL_SCHEMA: dict[str, set[str]] = {
    "AUTH_STARTED": {"store", "trigger"},
    "AUTH_COMPLETE": {"store", "user_id", "duration_ms"},
    "AUTH_FAILED": {"store", "error"},
    "SYNC_STARTED": {"stores"},
    "SYNC_PROGRESS": {"store", "progress", "message"},
    "SYNC_COMPLETE": {"games", "stores_synced", "duration_ms"},
    "SYNC_FAILED": {"store", "error"},
    "GAME_INSTALLED": {"store", "game_id", "install_path"},
    "GAME_UNINSTALLED": {"store", "game_id"},
    "GAME_LAUNCHED": {"store", "game_id", "session_id"},
    "GAME_STOPPED": {"store", "game_id", "session_id", "duration_s"},
    "DOWNLOAD_STARTED": {"store", "game_id", "size_bytes"},
    "DOWNLOAD_PROGRESS": {"store", "game_id", "progress", "speed_bps"},
    "DOWNLOAD_COMPLETE": {"store", "game_id", "duration_ms"},
    "DOWNLOAD_FAILED": {"store", "game_id", "error"},
    "ARTWORK_REQUEST": {"store", "game_id", "kind"},
    "ARTWORK_READY": {"store", "game_id", "path"},
    "SHORTCUT_CREATED": {"store", "game_id", "app_id"},
    "STORE_AUTH_COMPLETE": {"store"},
    "STORE_LOGOUT": {"store"},
}
def walk_sources(root: Path) -> dict[str, set[str]]:
    """Walk sources."""
    merged: dict[str, set[str]] = {}
    for path in root.rglob("*.py"):
        if "__pycache__" in path.parts:
            continue
        try:
            source = path.read_text(encoding="utf-8")
        except OSError:
            continue
        extracted = SchemaExtractor.extract_from_source(source)
        for event, kwargs in extracted.items():
            merged.setdefault(event, set()).update(kwargs)
    return merged
def compare(
    actual: dict[str, set[str]],
    canonical: dict[str, set[str]],
) -> int:
    """Compare."""
    errors = 0
    for event, allowed in canonical.items():
        seen = actual.get(event)
        if seen is None:
            continue
        unexpected = seen - allowed
        if unexpected:
            print(

```

```
        f" x {event}: unexpected kwargs "
        f"{sorted(unexpected)} (allowed: {sorted(allowed)})"
    )
    errors += 1
for event in sorted(actual.keys() - canonical.keys()):
    if event.endswith("_batch"):
        continue
    print(f" △ {event}: emitted but not in CANONICAL_SCHEMA")
    errors += 1
return errors

def main() -> int:
    """Main."""
    target = ROOT / "py_modules" / "unifideck"
    if not target.is_dir():
        print(f"x source dir not found: {target}")
        return 2
    print(f"→ walking {target}")
    actual = walk_sources(target)
    print(f"→ extracted {len(actual)} distinct events from source")
    errors = compare(actual, CANONICAL_SCHEMA)
    if errors == 0:
        print("\n✓ event schemas valid")
        return 0
    print(f"\nx {errors} schema error(s) found")
    return 1
if __name__ == "__main__":
    sys.exit(main())
```

OP-95

ensure_executable_bits.py

Fichier : scripts/ensure_executable_bits.py | Lignes : 37 | Depend de : (rien)

```
from __future__ import annotations
import logging
import sys
from pathlib import Path
def main(argv: list[str]) -> int:
    """Main."""
    logging.basicConfig(
        level=logging.INFO,
        format="[ensure_executable_bits] %(message)s",
    )
    if len(argv) > 2:
        print(
            "Usage: ensure_executable_bits.py [plugin_root]",
            file=sys.stderr,
        )
        return 2
    plugin_root = Path(argv[1]).resolve if len(argv) == 2 else Path.cwd
    expected_marker = (
        plugin_root / "py_modules" / "unifideck" / "launcher"
    )
    if not expected_marker.is_dir:
        print(
            f"error: {plugin_root} does not look like a Unifideck "
            f"plugin root (missing {expected_marker})",
            file=sys.stderr,
        )
        return 1
    sys.path.insert(0, str(plugin_root / "py_modules"))
    from unifideck.launcher.packaging import ensure_executable_files
    fixed = ensure_executable_files(plugin_root)
    if fixed == 0:
        logging.info("all launcher files already executable")
    else:
        logging.info("fixed executable bits on %d file(s)", fixed)
        return 0
    if __name__ == "__main__":
        sys.exit(main(sys.argv))
```

OP-97

check_config_keys.py

Fichier : scripts/check_config_keys.py | Lignes : 128 | Depend de : (rien)

```

from __future__ import annotations
import ast
import pathlib
import re
import sys
from typing import Iterable
REPO_ROOT = pathlib.Path(__file__).resolve().parent.parent
PY_ROOT = REPO_ROOT / "py_modules" / "unifideck"
_GET_PATTERNS = (
    "config.get",
)
)
class _ConfigKeyVisitor(ast.NodeVisitor):
    """Config key visitor."""
    def __init__(self, path: pathlib.Path) -> None:
        """Initialize the instance."""
        self.path = path
        self.keys: list[tuple[str, int]] = []
    def visit_Call(self, node: ast.Call) -> None:
        """Visit call."""
        key = self._extract_key(node)
        if key is not None:
            self.keys.append((key, node.lineno))
        self.generic_visit(node)
    @staticmethod
    def _extract_key(node: ast.Call) -> str | None:
        """Extract key."""
        func = node.func
        if isinstance(func, ast.Attribute) and func.attr == "get":
            owner = func.value
            owner_name = _ConfigKeyVisitor._last_attr_or_name(owner)
            if owner_name and "config" in owner_name.lower():
                if node.args and isinstance(node.args[0], ast.Constant):
                    val = node.args[0].value
                    if isinstance(val, str):
                        return val
        if isinstance(func, ast.Name) and func.id == "_cfg":
            if (
                len(node.args) >= 2
                and isinstance(node.args[1], ast.Constant)
                and isinstance(node.args[1].value, str)
            ):
                return node.args[1].value
            return None
    @staticmethod
    def _last_attr_or_name(node: ast.AST) -> str | None:
        """Last attr or name."""
        if isinstance(node, ast.Name):
            return node.id
        if isinstance(node, ast.Attribute):
            return node.attr
        return None

def collect_call_site_keys() -> dict[str, list[tuple[pathlib.Path, int]]]:
    """Collect call site keys."""

```

```

sites: dict[str, list[tuple[pathlib.Path, int]]] = {}
for p in PY_ROOT.rglob("*.py"):
    if p.name == "key_presence.py":
        continue
    try:
        tree = ast.parse(p.read_text(encoding="utf-8"))
    except SyntaxError:
        continue
    visitor = _ConfigKeyVisitor(p)
    visitor.visit(tree)
    for key, line in visitor.keys:
        sites.setdefault(key, []).append((p, line))
return sites

def load_registered_keys() -> set[str]:
    """Load registered keys."""
    kp_path = PY_ROOT / "config" / "key_presence.py"
    tree = ast.parse(kp_path.read_text(encoding="utf-8"))
    for node in ast.walk(tree):
        if isinstance(node, ast.Assign) and len(node.targets) == 1:
            target = node.targets[0]
            value = node.value
        elif isinstance(node, ast.AnnAssign):
            target = node.target
            value = node.value
        else:
            continue
        if (
            isinstance(target, ast.Name)
            and target.id == "RUNTIME_REQUIRED_KEYS"
            and isinstance(value, ast.Tuple)
        ):
            return {
                elt.value for elt in value.elts
                if isinstance(elt, ast.Constant)
                and isinstance(elt.value, str)
            }
    raise RuntimeError(
        "RUNTIME_REQUIRED_KEYS tuple not found in key_presence.py",
    )

def main(argv: Iterable[str] | None = None) -> int:
    """Main."""
    sites = collect_call_site_keys()
    registered = load_registered_keys()
    unregistered = sorted(k for k in sites if k not in registered)
    if not unregistered:
        print(
            f"[check_config_keys] OK – {len(sites)} distinct keys, "
            f"all present in RUNTIME_REQUIRED_KEYS.",
        )
        return 0
    print(
        "[check_config_keys] FAIL – the following keys are read by "
        "code but NOT listed in key_presence.RUNTIME_REQUIRED_KEYS:",
        file=sys.stderr,
    )
    for key in unregistered:
        locations = sites[key]
        first_file, first_line = locations[0]
        rel = first_file.relative_to(REPO_ROOT)
        extra = (
            f" (+{len(locations) - 1} more site(s))"

```



```
        if len(locations) > 1 else ""
    )
    print(f" {key} → {rel}:{first_line}{extra}", file=sys.stderr)
print(
    "\n[check_config_keys] Add each key to "
    "`py_modules/unifideck/config/key_presence.py` under "
    "`RUNTIME_REQUIRED_KEYS`, then ensure the key is also "
    "declared in `defaults/config.json` and "
    "`py_modules/unifideck/config/schema.json`.",
    file=sys.stderr,
)
return 1
if __name__ == "__main__":
    sys.exit(main())
```

OP-98

volumetry_check.py

Fichier : scripts/volumetry_check.py | Lignes : 570 | Depend de : (rien)

```
from __future__ import annotations
import ast
import os
import sys
from typing import Dict, Tuple
FILE_CAP_DEFAULT = 550
FILE_CAP_MAIN_PY = 800
FUNC_FAIL = 80
FUNC_WARN = 50
LOCALS_CAP = 15
NESTING_CAP = 4
FANOUT_CAP = 10
FANOUT_EXCLUDED = frozenset({
    "len", "str", "int", "bool", "float", "list", "dict", "set",
    "tuple", "isinstance", "issubclass", "hasattr", "getattr",
    "setattr", "print", "repr", "hash", "id", "type", "iter",
    "next", "range", "enumerate", "min", "max", "sum", "any",
    "all", "sorted", "reversed", "zip", "map", "filter",
    "debug", "info", "warning", "error", "exception", "critical",
    "log",
    "format", "join", "strip", "rstrip", "lstrip", "split",
    "splitlines", "startswith", "endswith", "lower", "upper",
    "replace", "encode", "decode",
    "get", "setdefault", "update", "items", "keys", "values",
    "copy", "pop", "append", "extend", "insert", "remove",
    "clear", "sort", "reverse",
    "read", "write", "close", "open", "exists",
})
FILES_ALLOWLIST: Dict[str, int] = {
}
FUNCTIONS_ALLOWLIST: Dict[str, int] = {
    "py_modules/unifideck/stores/gog/store.py::__init__": 87,
}
LOCALS_ALLOWLIST: Dict[str, int] = {
}
NESTING_ALLOWLIST: Dict[str, int] = {
}
FANOUT_ALLOWLIST: Dict[str, int] = {
    "py_modules/unifideck/stores/gog/store.py::__init__": 14,
}
def _iter_py_files() -> list[str]:
    files = ["main.py"] if os.path.exists("main.py") else []
    for dp, _, fns in os.walk("py_modules"):
        if "__pycache__" in dp:
            continue
        for fn in fns:
            if fn.endswith(".py"):
                files.append(os.path.join(dp, fn))
    return files

def check_files() -> int:
    violations = 0
    seen_listed = set()
    for path in _iter_py_files():
        with open(path, encoding="utf-8") as f:
```

```

        loc = sum(1 for _ in f)
    cap = FILE_CAP_MAIN_PY if path == "main.py" else FILE_CAP_DEFAULT
    if path in FILES_ALLOWLIST:
        seen_listed.add(path)
        baseline = FILES_ALLOWLIST[path]
        if loc <= cap:
            print(
                f":error file={path}::Grandfathered file is "
                f"now {loc} LOC (<= {cap} cap). Remove from "
                f"FILES_ALLOWLIST in tools/volumetry_check.py.",
            )
            violations += 1
        elif loc > baseline:
            print(
                f":error file={path}::File grew to {loc} LOC "
                f"(baseline was {baseline}). Grandfathered "
                f"entries can only shrink – refactor or split.",
            )
            violations += 1
    elif loc > cap:
        print(
            f":error file={path}::File exceeds cap "
            f"({loc} > {cap} LOC). Split or refactor.",
        )
        violations += 1
    for listed in FILES_ALLOWLIST:
        if listed not in seen_listed:
            print(
                f":warning::Allowlisted file {listed} not found "
                f"on disk. Remove from FILES_ALLOWLIST.",
            )
    return 1 if violations else 0
def _iter_functions():
    for path in _iter_py_files():
        try:
            with open(path, encoding="utf-8") as f:
                tree = ast.parse(f.read())
        except (SyntaxError, UnicodeDecodeError):
            continue
        for n in ast.walk(tree):
            if isinstance(n, (ast.FunctionDef, ast.AsyncFunctionDef)):
                yield path, n
def _count_function_lines(n) -> int:
    return getattr(n, "end_lineno", n.lineno) - n.lineno
def _count_locals(fn) -> int:
    args = {a.arg for a in fn.args.args}
    args.update(a.arg for a in fn.args.kwonlyargs)
    if fn.args.vararg:
        args.add(fn.args.vararg.arg)
    if fn.args.kwarg:
        args.add(fn.args.kwarg.arg)
    locals_ = set[str] = set()
    for n in ast.walk(fn):
        if isinstance(n, ast.Assign):
            for tgt in n.targets:
                if isinstance(tgt, ast.Name):
                    locals_.add(tgt.id)
        elif isinstance(n, (ast.AugAssign, ast.AnnAssign)):
            if isinstance(n.target, ast.Name):
                locals_.add(n.target.id)

```

```

        elif isinstance(n, ast.For) and isinstance(n.target, ast.Name):
            locals_.add(n.target.id)
        elif isinstance(n, ast.comprehension):
            if isinstance(n.target, ast.Name):
                locals_.add(n.target.id)
        elif isinstance(n, ast.withitem):
            if n.optional_vars and isinstance(
                n.optional_vars, ast.Name,
            ):
                locals_.add(n.optional_vars.id)
        elif isinstance(n, ast.ExceptHandler) and n.name:
            locals_.add(n.name)
    return len(locals_ - args)
def _nesting_depth(fn) -> int:
    NESTING = (
        ast.If, ast.For, ast.AsyncFor, ast.While,
        ast.With, ast.AsyncWith, ast.Try,
    )
    def rec(node, depth: int = 0) -> int:
        m = depth
        for child in ast.iter_child_nodes(node):
            if isinstance(child, NESTING):
                m = max(m, rec(child, depth + 1))
            else:
                m = max(m, rec(child, depth))
        return m
    return rec(fn, 0)
def _fan_out(fn) -> int:
    seen: set[str] = set()
    for n in ast.walk(fn):
        if not isinstance(n, ast.Call):
            continue
        target: str | None = None
        if isinstance(n.func, ast.Name):
            target = n.func.id
        elif isinstance(n.func, ast.Attribute):
            target = n.func.attr
        if target and target not in FANOUT_EXCLUDED:
            seen.add(target)
    return len(seen)

def _gate_metric(
    metric_name: str,
    cap: int,
    allowlist: Dict[str, int],
    measure,
    warn_threshold: int | None = None,
) -> int:
    violations = 0
    seen_listed: set[str] = set()
    for path, fn in _iter_functions():
        value = measure(fn)
        key = f"{path}::{fn.name}"
        line = fn.lineno
        if key in allowlist:
            seen_listed.add(key)
            baseline = allowlist[key]
            if value <= cap:
                print(
                    f"::error file={path},line={line}::"
                    f"Grandfathered function {fn.name}() now has "

```

```

        f"{metric_name}={value} (<= {cap} cap). "
        f"Remove from the allowlist.",
    )
    violations += 1
elif value > baseline:
    print(
        f"::error file={path},line={line}::"
        f"{fn.name}() {metric_name} grew to {value} "
        f"(baseline {baseline}). Refactor or split.",
    )
    violations += 1
elif value > cap:
    print(
        f"::error file={path},line={line}::"
        f"{fn.name}() {metric_name}={value} "
        f"> {cap} cap). Split or refactor.",
    )
    violations += 1
elif warn_threshold is not None and value > warn_threshold:
    print(
        f"::warning file={path},line={line}::"
        f"{fn.name}() {metric_name}={value} "
        f"(approaching {cap} cap).",
    )
for listed in allowlist:
    if listed not in seen_listed:
        print(
            f"::warning::Allowlisted entry {listed} not found. "
            f"Remove from the allowlist.",
        )
    return 1 if violations else 0
def check_functions() -> int:
    return _gate_metric(
        "lines", FUNC_FAIL, FUNCTIONS_ALLOWLIST,
        _count_function_lines, FUNC_WARN,
    )
def check_locals() -> int:
    return _gate_metric(
        "locals", LOCALS_CAP, LOCALS_ALLOWLIST, _count_locals,
    )
def check_nesting() -> int:
    return _gate_metric(
        "nesting depth", NESTING_CAP, NESTING_ALLOWLIST,
        _nesting_depth,
    )
def check_fanout() -> int:
    return _gate_metric(
        "fan-out", FANOUT_CAP, FANOUT_ALLOWLIST, _fan_out,
    )

CHECKS = {
    "files": check_files,
    "functions": check_functions,
    "locals": check_locals,
    "nesting": check_nesting,
    "fanout": check_fanout,
}
def main(argv: list[str]) -> int:
    if len(argv) < 2 or argv[1] not in CHECKS:
        print(
            "usage: python3 tools/volumetry_check.py "

```

```

        "{" + "|".join(CHECKS) + "}",
        file=sys.stderr,
    )
    return 2
    return CHECKS[argv[1]]()
if __name__ == "__main__":
    sys.exit(main(sys.argv))

```

Migration – Files to delete

Legacy files replaced by the new architecture

This table lists files from the current repository whose business logic is absorbed by the new code. Once the migration is complete and validated (tests green, E2E smoke passed), these files can be deleted without loss of functionality. Files are sorted by decreasing volume.

```

#
Legacy file (to delete)
Lines
Replaced by
Lines
1
py_modules/unifideck/stores/ubisoft.py
4 639
py_modules/unifideck/stores/ubisoft
/ (10 modules)
9 893
2
py_modules/unifideck/stores/gog.py
2 590
py_modules/unifideck/stores/gog/ (9
modules)
5 305
3
py_modules/unifideck/shortcuts/shortcuts_mana
ger.py
2 344
py_modules/unifideck/services/short
cut_service.py
250
4
py_modules/unifideck/download/manager.py
1 764
py_modules/unifideck/services/downl
oad_service.py
302
5
py_modules/unifideck/cloud/cloud_save.py
1 042
py_modules/unifideck/services/cloud
_save_service.py
345
6
py_modules/unifideck/stores/epic.py
969
py_modules/unifideck/stores/epic/
(7 modules)
1 784
7
py_modules/unifideck/stores/microsoft.py
957
py_modules/unifideck/stores/microso

```

```

ft/ (6 modules)
1 558
8
py_modules/unifideck/stores/microsoft_chromiu
m.py
913
py_modules/unifideck/auth/edge_brow
ser/ (7 modules)
1 840
9
py_modules/unifideck/stores/amazon.py
928
py_modules/unifideck/stores/amazon/
(6 modules)
1 600
10
py_modules/unifideck/playtime/session_tracker
.py
501
py_modules/unifideck/services/playt
ime_service.py
226
11
py_modules/unifideck/accounts/account_manager
.py
314
py_modules/unifideck/services/accou
nt_service.py
157
12
py_modules/unifideck/stores/base.py
169
py_modules/unifideck/core/store_bas
e.py
263
13
py_modules/unifideck/stores/manager.py
80
py_modules/unifideck/core/store_reg
istry.py
290
Sous-total (refactoring core)
17 210

```

Note: main.py (7,794 lines at the repository root) does not appear in this table because it is replaced in place by the thin router (434 lines) at the same path. The migration overwrites it; there is nothing to delete. The 13 files listed above total 17,210 lines of legacy code to remove after the migration has been validated.

Legacy unifideck/ directories to delete

In addition to the 13 monolithic files above, 8 whole subdirectories of py_modules/unifideck/ can be deleted. Three are

packages that have been fully replaced (compat → compatibility, discovery → core/manifest, shortcuts → services). The

other five now contain only a compatibility __init__.py shim (try: from .legacy import X; except ImportError: pass) whose

sole purpose is to re-export the legacy classes – once those classes are removed, the shims are dead.

#

Directory to delete

Lines

Replaced by

```

1
py_modules/unifideck/shortcuts/
2 587
services/shortcut_service.py + steam/shortcuts.py
2
py_modules/unifideck/playtime/
1 993
services/playtime_service.py (7 files → 1 service)

3
py_modules/unifideck/download/
1 911
services/download_service.py
4
py_modules/unifideck/cloud/
1 043
services/cloud_save_service.py +
launcher/cloud_failure.py
5
py_modules/unifideck/compat/
773
compatibility/ (renamed + rewritten)
6
py_modules/unifideck/registry/
321
core/store_registry.py + core/io/games_map.py
7
py_modules/unifideck/accounts/
315
services/account_service.py
8
py_modules/unifideck/discovery/
200
core/manifest.py + integrated into store installers
Total (8 dossiers)
9 143
Audit complémentaire – bin/ legacy
Le répertoire bin/ du dépôt actuel contient 22 scripts Python et 8 binaires natifs. L'audit
croisé avec le nouveau code révèle
trois catégories : (A) supplantés par le nouveau code, (B) portés dans le subpackage
launcher/proton/, (C) features
orphelins récupérées dans cette session, (D) binaires natifs à conserver sans fiche.
A. Fichiers legacy à effacer
#
Fichier legacy
Lignes
Remplacé par
1
bin/xcloud_cdp_inject.py
740
launcher/xcloud_cdp.py + cdp/xcloud_browser_shims.py
2
bin/ubisoft_setup.py
647
stores/ubisoft/installer.py + 6 helpers
3
bin/gog_setup.py
643
stores/gog/install/installer.py + 15 helpers
4
bin/cloud_save_sync.py

```



```
582
services/cloud_save_service.py +
launcher/cloud_failure.py
5
bin/winetricks_gog.py
430
launcher/proton/game_fixes.py +
stores/gog/install/installer.py
6
bin/ubisoft_inject_session.py
360
stores/ubisoft/session_payload.py
7
bin/ubisoft_capture_session.py
318
stores/ubisoft/session.py
8
bin/epic_prerequisites.py
288
stores/epic/install.py + launcher/proton/epic.py
9
bin/winetricks_installer.py
271
launcher/proton/game_fixes.py
10
bin/proton_installer.py
212
launcher/proton/umu_runtime.py +
launcher/proton/core.py
11
bin/umu_lookup.py
125
launcher/proton/umu_runtime.py
12
bin/prefetch_winetricks.py
89
launcher/proton/game_fixes.py (intégré)
13
bin/fix_epic_launcher_prefix.py
159
launcher/proton/epic_prefix_fix.py
14
bin/game_fixes.py
179
launcher/proton/game_fixes.py
15
bin/epic_registry_setup.py
145
launcher/proton/epic_registry.py
16
bin/gog_linux_dosbox.py
107
launcher/proton/gog_linux_dosbox.py (module -m)
17
py_modules/unifideck/Dockerfile
8
Artefact template Decky (backend = pur Python, aucun
build)
18
py_modules/unifideck/entrypoint.sh
4
```

```

Companion Dockerfile – contenu : exit 0
19
py_modules/unifideck/dlc.py
47
Relocalisé dans stores/dlc.py (import actif)
20
assets/css/tab-hider.css
35
Hack CSS obsolète – router patching Decky le remplace

Total A (à effacer)
5 389
B. Binaires à conserver dans bin/ (sans fiche)
#
Fichier bin/
Taille
Rôle
1
bin/legendary
10 MB
CLI Epic Games Store (upstream project)
2
bin/nile
10 MB
CLI Amazon Games (upstream project)
3
bin/EpicGamesLauncher.exe
150 KB
Stub wrapper Windows pour Legendary
4
bin/stubs/GalaxyCommunication.exe
binaire
Stub overlay GOG Galaxy (copié par galaxy_stub.py)
5
bin/unifideck-launcher
122 KB
Entry-point shell du plugin
6
bin/unifideck-runner
1.8 KB
Wrapper shell minimal
7
bin/vcruntime_fix.reg
1 KB
Fichier de registre Windows (data file)
8
bin/umu/umu
upstream
Runtime umu-run (upstream project)
C. Dossiers py_modules/ tiers à supprimer
Le répertoire py_modules/ legacy contient des dépendances Python tierces vendored
(installées par pip install --target).
Plusieurs d'entre elles sont devenues inutiles ou ont été remplacées. certifi, vdf et
websockets restent nécessaires et ne
sont pas impactés par la migration. aiohttp et cryptography (absents du legacy) doivent
être ajoutés comme dépendances
vendored dans le nouveau bundle.
#
Dossier à supprimer
Taille
Motif

```

```

1
py_modules/pip/ +
pip-25.3.dist-info/
5,2 MB
Outil de packaging pip. Erreur d'inclusion legacy – n'a pas
sa place dans un plugin runtime.
2
py_modules/requests/
179 KB
Client HTTP synchrone. Remplacé par aiohttp (async) dans
l'intégralité de la nouvelle architecture.
3
py_modules/urllib3/
409 KB
Dépendance transitive de requests. Tombe avec la suppression
de requests.
4
py_modules/idna/
332 KB
Dépendance transitive de requests (IDNA domain names). Tombe
avec la suppression de requests.
5
py_modules/charset_normalizer/
134 KB
Dépendance transitive de requests. Tombe avec la suppression
de requests.
6
py_modules/steamgrid/
53 KB
Wrapper tiers python-steamgriddb. Remplacé volontairement
par un appel HTTP direct dans steam/steamgriddb.py (supprime
une dep, simplifie le build, réduit la surface d'attaque).
7
py_modules/bin/
2 KB
Scripts CLI pip (pip, pip3, pip3.13). Artefacts de
l'installation – pas des dépendances runtime.
Total à supprimer
~6,3 MB

```

Note : les packages certifi, vdf et websockets ne figurent pas dans ce tableau car ils restent nécessaires au nouveau code et ne sont pas impactés par la migration. Deux nouvelles dépendances doivent être ajoutées en vendored au bundle final : aiohttp (client HTTP async, utilisé dans 10 modules) et cryptography (chiffrement tokens dans le package security).

Synthèse complète : la migration implique la suppression de 17 210 lignes dans 13 fichiers monolithiques py_modules/ 9 143 lignes dans 8 dossiers unifideck/ legacy entiers 5 389 lignes dans bin/ et py_modules/ (A 20 fichiers) et ~6 3 MB de packages tiers obsolètes (C 7 dossiers). Les 8 binaires natifs de la catégorie B restent en place sans fiche. Les 5 nouveaux modules Python ajoutés dans launcher/proton/ (language_setup galaxy_stub auth_args_stripper gog_linux_dosbox prefix_layout) sont documentés par les fiches OP-30j à OP-30n.

Migration — Files to delete

Legacy files replaced by the new architecture

This table lists files from the current repository whose business logic is absorbed by the new code. Once the migration is complete and validated (tests green, E2E smoke passed), these files can be deleted without loss of functionality. Files are sorted by decreasing volume.

#	Legacy file (to delete)	Lines	Replaced by	Lines
1	py_modules/unifideck/stores/ubisoft.py	4 639	py_modules/unifideck/stores/ubisoft / (10 modules)	9 893
2	py_modules/unifideck/stores/gog.py	2 590	py_modules/unifideck/stores/gog/ (9 modules)	5 305
3	py_modules/unifideck/shortcuts/shortcuts_manager.py	2 344	py_modules/unifideck/services/shortcut_service.py	250
4	py_modules/unifideck/download/manager.py	1 764	py_modules/unifideck/services/download_service.py	302
5	py_modules/unifideck/cloud/cloud_save.py	1 042	py_modules/unifideck/services/cloud_save_service.py	345
6	py_modules/unifideck/stores/epic.py	969	py_modules/unifideck/stores/epic/ (7 modules)	1 784
7	py_modules/unifideck/stores/microsoft.py	957	py_modules/unifideck/stores/microsoft/ (6 modules)	1 558
8	py_modules/unifideck/stores/microsoft_chromium.py	913	py_modules/unifideck/auth/edge_browser/ (7 modules)	1 840
9	py_modules/unifideck/stores/amazon.py	928	py_modules/unifideck/stores/amazon/ (6 modules)	1 600
10	py_modules/unifideck/playtime/session_tracker.py	501	py_modules/unifideck/services/playtime_service.py	226
11	py_modules/unifideck/accounts/account_manager.py	314	py_modules/unifideck/services/account_service.py	157
12	py_modules/unifideck/stores/base.py	169	py_modules/unifideck/core/store_base.py	263
13	py_modules/unifideck/stores/manager.py	80	py_modules/unifideck/core/store_registry.py	290
Sous-total (refactoring core)		17 210		

Note: main.py (7,794 lines at the repository root) does not appear in this table because it is replaced in place by the thin router (434 lines) at the same path. The migration overwrites it; there is nothing to delete. The 13 files listed above total 17,210 lines of legacy code to remove after the migration has been validated.

Legacy unifideck/ directories to delete

In addition to the 13 monolithic files above, 8 whole subdirectories of **py_modules/unifideck/** can be deleted. Three are packages that have been fully replaced (compat → compatibility, discovery → core/manifest, shortcuts → services). The other five now contain only a compatibility **__init__.py** shim (*try: from .legacy import X; except ImportError: pass*) whose sole purpose is to re-export the legacy classes — once those classes are removed, the shims are dead.

#	Directory to delete	Lines	Replaced by
1	py_modules/unifideck/shortcuts/	2 587	services/shortcut_service.py + steam/shortcuts.py
2	py_modules/unifideck/playtime/	1 993	services/playtime_service.py (7 files → 1 service)

3	py_modules/unifideck/download/	1 911	services/download_service.py
4	py_modules/unifideck/cloud/	1 043	services/cloud_save_service.py + launcher/cloud_failure.py
5	py_modules/unifideck/compat/	773	compatibility/ (renamed + rewritten)
6	py_modules/unifideck/registry/	321	core/store_registry.py + core/io/games_map.py
7	py_modules/unifideck/accounts/	315	services/account_service.py
8	py_modules/unifideck/discovery/	200	core/manifest.py + integrated into store installers
Total (8 dossiers)		9 143	

Audit complémentaire — bin/ legacy

Le répertoire **bin/** du dépôt actuel contient 22 scripts Python et 8 binaires natifs. L'audit croisé avec le nouveau code révèle trois catégories : (A) supplantés par le nouveau code, (B) portés dans le subpackage **launcher/proton/**, (C) features orphelines récupérées dans cette session, (D) binaires natifs à conserver sans fiche.

A. Fichiers legacy à effacer

#	Fichier legacy	Lignes	Remplacé par
1	bin/xcloud_cdp_inject.py	740	launcher/xcloud_cdp.py + cdp/xcloud_browser_shims.py
2	bin/ubisoft_setup.py	647	stores/ubisoft/installer.py + 6 helpers
3	bin/gog_setup.py	643	stores/gog/install/installer.py + 15 helpers
4	bin/cloud_save_sync.py	582	services/cloud_save_service.py + launcher/cloud_failure.py
5	bin/winetricks_gog.py	430	launcher/proton/game_fixes.py + stores/gog/install/installer.py
6	bin/ubisoft_inject_session.py	360	stores/ubisoft/session_payload.py
7	bin/ubisoft_capture_session.py	318	stores/ubisoft/session.py
8	bin/epic_prerequisites.py	288	stores/epic/install.py + launcher/proton/epic.py
9	bin/winetricks_installer.py	271	launcher/proton/game_fixes.py
10	bin/proton_installer.py	212	launcher/proton/umu_runtime.py + launcher/proton/core.py
11	bin/umu_lookup.py	125	launcher/proton/umu_runtime.py
12	bin/prefetch_winetricks.py	89	launcher/proton/game_fixes.py (intégré)
13	bin/fix_epic_launcher_prefix.py	159	launcher/proton/epic_prefix_fix.py
14	bin/game_fixes.py	179	launcher/proton/game_fixes.py
15	bin/epic_registry_setup.py	145	launcher/proton/epic_registry.py
16	bin/gog_linux_dosbox.py	107	launcher/proton/gog_linux_dosbox.py (module -m)
17	py_modules/unifideck/Dockerfile	8	Artefact template Decky (backend = pur Python, aucun build)
18	py_modules/unifideck/entrypoint.sh	4	Companion Dockerfile – contenu : exit 0
19	py_modules/unifideck/dlc.py	47	Relocalisé dans stores/dlc.py (import actif)
20	assets/css/tab-hider.css	35	Hack CSS obsolète – router patching Decky le remplace

	Total A (à effacer)	5 389	
--	---------------------	-------	--

B. Binaires à conserver dans bin/ (sans fiche)

#	Fichier bin/	Taille	Rôle
1	bin/legendary	10 MB	CLI Epic Games Store (upstream project)
2	bin/nile	10 MB	CLI Amazon Games (upstream project)
3	bin/EpicGamesLauncher.exe	150 KB	Stub wrapper Windows pour Legendary
4	bin/stubs/GalaxyCommunication.exe	binaire	Stub overlay GOG Galaxy (copié par galaxy_stub.py)
5	bin/unifideck-launcher	122 KB	Entry-point shell du plugin
6	bin/unifideck-runner	1.8 KB	Wrapper shell minimal
7	bin/vcruntime_fix.reg	1 KB	Fichier de registre Windows (data file)
8	bin/umu/umu	upstream	Runtime umu-run (upstream project)

C. Dossiers py_modules/ tiers à supprimer

Le répertoire **py_modules/** legacy contient des dépendances Python tierces vendored (installées par pip install --target). Plusieurs d'entre elles sont devenues inutiles ou ont été remplacées. **certifi**, **vdf** et **websockets** restent nécessaires et ne sont pas impactés par la migration. **aiohttp** et **cryptography** (absents du legacy) doivent être ajoutés comme dépendances vendored dans le nouveau bundle.

#	Dossier à supprimer	Taille	Motif
1	py_modules/pip/ + pip-25.3.dist-info/	5,2 MB	Outil de packaging pip. Erreur d'inclusion legacy – n'a pas sa place dans un plugin runtime.
2	py_modules/requests/	179 KB	Client HTTP synchrone. Remplacé par aiohttp (async) dans l'intégralité de la nouvelle architecture.
3	py_modules/urllib3/	409 KB	Dépendance transitive de requests. Tombe avec la suppression de requests.
4	py_modules/idna/	332 KB	Dépendance transitive de requests (IDNA domain names). Tombe avec la suppression de requests.
5	py_modules/charset_normalizer/	134 KB	Dépendance transitive de requests. Tombe avec la suppression de requests.
6	py_modules/steamgrid/	53 KB	Wrapper tiers python-steamgriddb. Remplacé volontairement par un appel HTTP direct dans steam/steamgriddb.py (supprime une dep, simplifie le build, réduit la surface d'attaque).
7	py_modules/bin/	2 KB	Scripts CLI pip (pip, pip3, pip3.13). Artefacts de l'installation – pas des dépendances runtime.
	Total à supprimer	~6,3 MB	

Note : les packages **certifi**, **vdf** et **websockets** ne figurent pas dans ce tableau car ils restent nécessaires au nouveau code et ne sont pas impactés par la migration. Deux nouvelles dépendances doivent être ajoutées en vendored au bundle final : **aiohttp** (client HTTP async, utilisé dans 10 modules) et **cryptography** (chiffrement tokens dans le package security).

Synthèse complète : la migration implique la suppression de **17 210** lignes dans 13 fichiers monolithiques py_modules/ **9 143** lignes dans 8 dossiers unifideck/ legacy entiers **5 389** lignes dans bin/ et py_modules/ (A 20 fichiers) et **~6 3 MB** de packages tiers obsolètes (C 7 dossiers). Les 8 binaires natifs de la catégorie B restent en place sans fiche. Les 5 nouveaux modules Python ajoutés dans launcher/proton/ (language_setup galaxy_stub auth_args_stripper gog_linux_dosbox prefix_layout) sont documentés par les fiches OP-30j à OP-30n.

Migration — Frontend files to delete

Legacy src/ files replaced by the Phase 3 refactor

This table mirrors the backend migration table at the start of this section, but for the staging branch's src/ tree. Each row is a file whose business logic is fully absorbed by one or more Phase 3 files. Once the refactor lands and the vitest suite is green, these files can be deleted without loss of functionality. Sorted by decreasing volume.

#	Legacy file	Lines	Replaced by
1	src/components/PlayButtonOverride.tsx	2 479	src/components/play/* (4 files, OP-F07*)
2	src/components/GameInfoPanel.tsx	1 232	src/components/info/* (4 files, OP-F08*)
3	src/spoofing/SteamStorePatcher.ts	731	src/lib/steam-bridge/SteamBridge.ts (OP-F02a)
4	src/components/StorageSettings.tsx	623	settings/StorageSettings + StoragePathPicker
5	src/tabs/filters.ts	436	Replaced by views/AppDetailsPatch + sync filtering
6	src/tabs/TabContainer.ts	385	Replaced by views/QuickAccessPanel
7	src/components/InstallInfoDisplay.tsx	356	Absorbed by play/DownloadingButtons (OP-F07c)
8	src/hooks/gameActionInterceptor.ts	305	hooks/usePlaySection + hooks/useGameActions
9	src/tabs/LibraryPatch.ts	278	Replaced by views/AppDetailsPatch (OP-F15b)
10	src/spoofing/CollectionManager.ts	275	Steam collections read via SteamBridge.getCollections()
11	src/views/UnifiedLibraryView.tsx	220	views/QuickAccessPanel + components/shared/GameGrid
12	src/components/CloudSaveConflictModal.tsx	208	components/modals/CloudSaveConflictModal (OP-F10d)
13	src/components/GameGrid.tsx	197	components/shared/GameGrid (OP-F12b)
14	src/components/DownloadsTab.tsx	155	components/downloads/DownloadsTab (OP-F11a)
15	src/tabs/protondb.ts	133	ProtonDB tier rendered by info/GameInfoScores (OP-F08d)
16	src/components/UninstallConfirmModal.tsx	120	components/modals/UninstallConfirmModal (OP-F10c)
17	src/components/LanguageSelector.tsx	99	components/settings/LanguageSelector (OP-F09e)
18	src/components/GOGLanguageSelectModal.tsx	94	Decision moved to backend store config + per-install RPC
19	src/components/ChromiumInstallModal.tsx	80	Edge install routed via dispatch_unifideck_action
20	src/components/AccountSwitchModal.tsx	76	components/modals/AccountSwitchModal (OP-F10a)
21	src/components/ForceSyncModal.tsx	72	Inline confirmation in settings/LibrarySync (OP-F09d)
22	src/components/SteamRestartModal.tsx	50	components/modals/SteamRestartModal (OP-F10b)
23	src/components/settings/StoreAuthButton.tsx	46	Inlined in settings/StoreConnections (OP-F09c)

Migration — Frontend files to delete (continued)

#	Legacy file	Lines	Replaced by
24	src/components/StoreIcon.tsx	26	components/shared/StoreIcon (OP-F12a)
25	src/tabs/index.ts	23	Removed — barrel for now-deleted tabs/ package
Total (25 files)		8 699	

Note: src/index.tsx (2 409 LOC) does not appear in this table because it is replaced in place by the thin entry (69 LOC) at the same path — see OP-F18. The migration overwrites it ; there is nothing to delete.

src/utls/microsoftShortcutLaunch.ts is also deleted but is intentionally absent here: it is fused into the unified authShortcutLaunch.ts (OP-F20c) and listed in the conservation table below for traceability.

Files conserved (imported or refactored in place)

Path	Disposition
src/i18n/index.ts	Imported as-is (OP-F19)
src/i18n/translations.ts	Imported as-is (OP-F19a)
src/i18n/locales/en-US.json	Imported as-is (OP-F19b) — other locales CI-generated
src/utls/format.ts	Imported as-is (OP-F20a)
src/utls/controllerConfig.ts	Refactored to route through SteamBridge (OP-F20b)
src/utls/authShortcutLaunch.ts	Unified to cover Microsoft (OP-F20c) — replaces microsoftShortcutLaunch.ts
src/utls/ubisoftShortcutLaunch.ts	Imported with shared types extracted to lib/steam-bridge (OP-F20d)
src/types/store.ts	Refactored under types/* sub-package (OP-F01c)
src/types/steam.ts	Refactored under types/* sub-package (OP-F01d)
src/types/downloads.ts	Refactored under types/* sub-package (OP-F01e)
src/types/playtime.ts	Refactored under types/* sub-package (OP-F01f)
src/types/syncProgress.ts	Refactored under types/* sub-package (OP-F01g)
src/components/settings/LibrarySync.tsx	Reworked to consume SyncContext (OP-F09d)
src/components/settings/StoreConnections.tsx	Reworked to consume StoreContext + AuthContext (OP-F09c)
src/hooks/useSteamLibrary.ts	Reworked to call through SteamBridge (OP-F05i)
src/index.tsx	Refactored in place: 2409 LOC → 69 LOC (OP-F18)
src/utls/microsoftShortcutLaunch.ts	DELETED — fused into authShortcutLaunch.ts (OP-F20c)